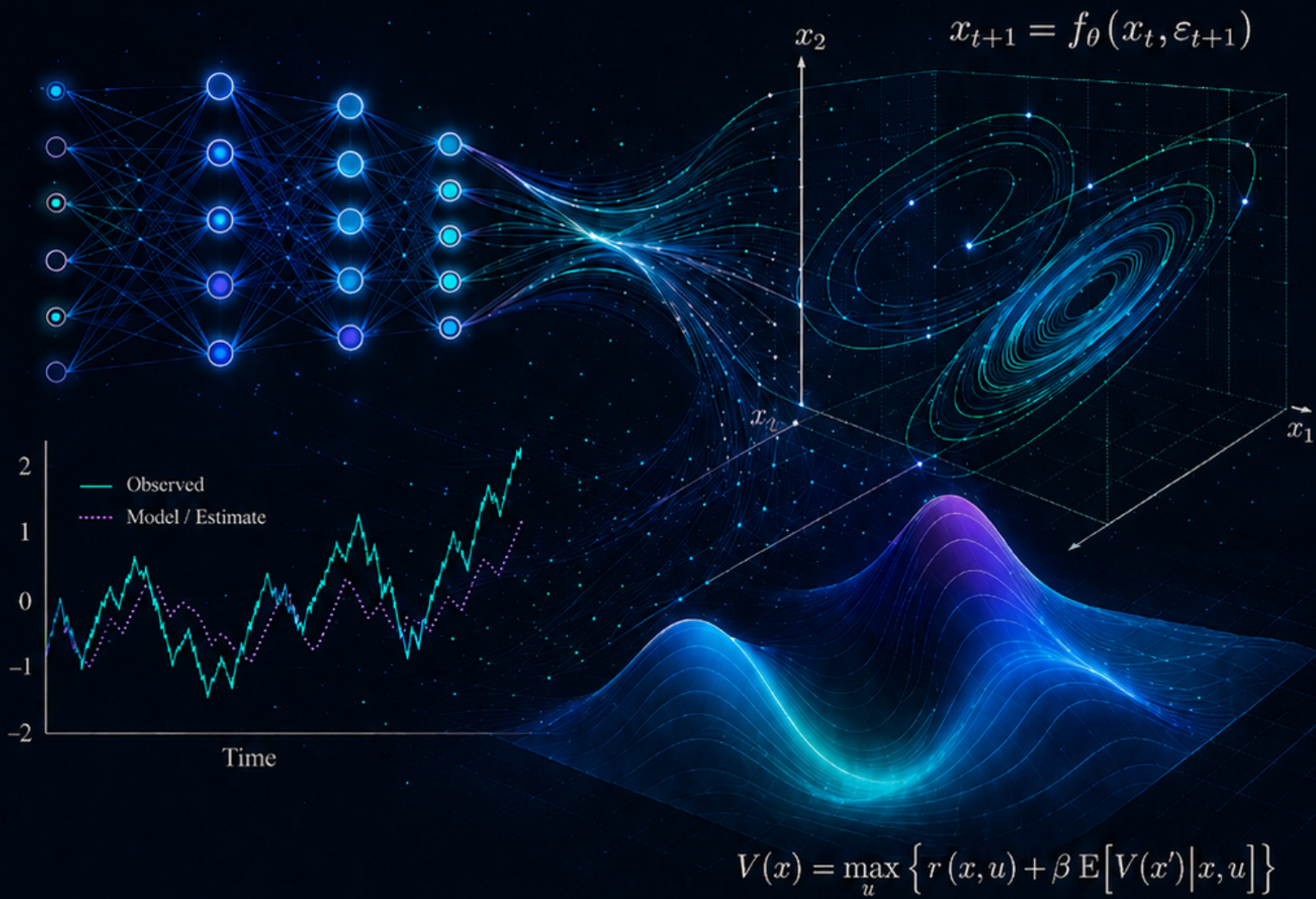




Deep Learning *for* Solving and Estimating Dynamic Models in Economics and Finance

Simon Scheidegger

HEC, University of Lausanne



Graduate Lecture Script



Computational Economics & Finance



Course materials: github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Deep Learning for Solving and Estimating Dynamic Models in Economics and Finance

August 2026

Simon Scheidegger
HEC, University of Lausanne

The most recent version of this manuscript, together with the companion code, is maintained at:

https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

This version: August 2026.

© 2026 Simon Scheidegger. Code under MIT, text/figures under CC0 1.0 Universal:
<https://creativecommons.org/publicdomain/zero/1.0/>

Abstract

This script offers an implementation-oriented introduction to deep learning methods for solving and estimating high-dimensional dynamic stochastic models in economics and finance. Its starting point is the curse of dimensionality: heterogeneous-agent economies, overlapping-generations models with aggregate risk, continuous-time models with occasionally binding constraints, climate-economy models, and macro-finance environments with many assets and frictions all generate state and parameter spaces that strain classical tensor-product grid methods.

The exposition is organized around four complementary methodologies. Deep Equilibrium Nets embed discrete-time equilibrium conditions directly into neural-network loss functions. Physics-Informed Neural Networks approximate continuous-time Hamilton–Jacobi–Bellman, Kolmogorov forward, and related partial differential equations. Deep surrogate models provide fast, differentiable approximations to expensive structural models, while Gaussian processes add a probabilistic layer that quantifies approximation uncertainty; together they support estimation, sensitivity analysis, and constrained policy design. Gaussian-process-based dynamic programming, combined with active learning and dimension reduction, extends value-function iteration to settings with very large continuous state spaces.

Applications range from representative-agent and international real business cycle models to overlapping-generations and heterogeneous-agent economies, continuous-time macro-finance, structural estimation by simulated method of moments, and climate economics under uncertainty. The emphasis throughout is practical: economic restrictions are translated into trainable residuals, algorithms are paired with diagnostics, and companion notebooks in TensorFlow and PyTorch invite students to experiment, modify, and learn by doing.

These notes are not intended as an exhaustive textbook. They are a deliberately subjective and inevitably incomplete snapshot of a rapidly evolving field. The selection of architectures, references, and case studies reflects one researcher’s judgment about useful entry points at the time of writing. Their purpose is to equip PhD students and researchers with enough conceptual and computational machinery to engage with this frontier hands-on.

“The machine, the frozen form of a living intelligence, is the power that expands the potential of your life by raising the productivity of your time.”

— Ayn Rand, *Atlas Shrugged* (1957), Galt’s Speech (Part Three, Chapter VII)

Contents

Preface	7
How to Read This Script	11
Notation and Symbols	13
Execution Map	16
1 Introduction to Machine Learning and Deep Learning	17
1.1 Motivation and Applications	17
1.2 Types of Machine Learning	19
1.2.1 Supervised Learning	19
1.2.2 Unsupervised Learning	20
1.2.3 Reinforcement Learning	21
1.3 Course Focus: Supervised vs. Unsupervised Learning	21
1.4 The Supervised Learning Pipeline	22
1.4.1 Beyond MSE: Robust and Asymmetric Losses	24
1.5 From Perceptrons to Deep Networks	24
1.6 Training: Loss Functions, Gradient Descent, and Backpropagation	27
1.6.1 The Adam Optimizer	28
1.6.2 The Optimizer Family Tree: Momentum, RMSprop, Adam, AdamW	28
1.6.3 Learning Rate Schedules	29
1.6.4 Backpropagation: The Chain Rule at Scale	30
1.7 Weight Initialization	30
1.8 Activation Functions in Depth	31
1.9 Vanishing and Exploding Gradients	32
1.9.1 Batch Normalization	33
1.9.2 Normalization Variants Beyond BatchNorm	34
1.10 Generalization: Overfitting, Regularization, and Double Descent	35
1.10.1 Train / Validation / Test Split	35
1.10.2 A Pointer to the Theory: Neural Tangent Kernel (NTK) and Benign Overfitting	37
1.11 Sequence Models: RNNs, LSTMs, and Attention	38
1.11.1 Recurrent Neural Networks	38
1.11.2 Long Short-Term Memory (LSTM) and GRUs	39
1.11.3 Limits of Recurrent Models	41
1.11.4 The Transformer Architecture	41
1.11.5 Advanced Aside: In-Context Learning of an AR(1) Process	45
Further Reading	48
Exercises	48

2	Deep Equilibrium Nets	50
2.1	The Curse of Dimensionality in Economics	50
2.2	From Supervised Learning to Self-Supervised Equilibrium Solving	52
2.3	The DEQN Training Algorithm	54
2.4	The Brock–Mirman Benchmark	56
2.5	Encoding Equilibrium Conditions: Hard vs. Soft Constraints	59
2.6	Quadrature Rules for Conditional Expectations	61
2.6.1	Why We End Up Integrating, and What Numerical Integration Is	61
2.6.2	Tensor-Product Gauss–Hermite	63
2.6.3	Monomial (Stroud-3) Cubature with Linear Scaling	63
2.6.4	Inverse-CDF Transform and Quasi–Monte Carlo	64
2.7	Automatic Differentiation for DEQNs	65
2.7.1	Three ways to take a derivative	65
2.7.2	Computational graph; forward and reverse modes	68
2.7.3	The autodiff Euler residual	69
2.7.4	Pitfalls of autodiff	71
2.8	Data Parallelization with MPI	72
2.9	Choice of Loss Kernel: Beyond Mean-Squared Loss	72
	Further Reading	75
	Exercises	76
3	The International Real Business Cycle Model	78
3.1	Why IRBC for Macro-Finance Research?	78
3.2	Model Setup	79
3.3	The Planner’s Problem and Equilibrium Conditions	81
3.4	DEQN Formulation	85
3.5	Persistent-Simulation Training	87
3.6	Results and Scalability	88
	Further Reading	91
	Exercises	91
4	Neural Architecture Search and Loss Normalization	94
4.1	The Hyperparameter Space	95
4.2	Grid Search	95
4.3	Random Search	95
4.4	Bayesian Optimization	96
4.4.1	A Brief Introduction to Gaussian Processes	96
4.4.2	Expected Improvement	97
4.5	Hyperband and Successive Halving	98
4.6	Method Comparison	100
4.7	Implementing the Search in Practice	100
4.8	Multi-Component Losses: The Scale Problem	101
4.8.1	Inverse-Loss Weighting	102
4.8.2	SoftAdapt	102
4.8.3	ReLoBRaLo: Adaptive Loss Balancing	102
4.8.4	GradNorm	103
4.8.5	Summary of Balancing Methods	104
	Further Reading	105
	Exercises	106

5	Overlapping Generations Models with DEQNs	107
5.1	Why Overlapping Generations?	107
5.2	The 6-Agent Analytic OLG Model	109
5.3	Mapping the Analytic OLG to a DEQN	111
5.4	Inequality Constraints and KKT Complementarity	114
5.5	The 56-Agent Benchmark	115
	Further Reading	119
	Exercises	119
6	Heterogeneous Agents and Young’s Method	122
6.1	From Representative to Heterogeneous Agents	123
6.2	The Krusell–Smith (1998) Economy	124
6.3	Young’s (2010) Non-Stochastic Simulation	125
6.4	DEQN with a Continuum of Agents	132
6.5	Results and Discussion	134
6.6	Alternative Deep-Learning Approaches to Krusell–Smith	135
6.6.1	All-in-One Deep Learning (Maliar, Maliar & Winant, 2021)	135
6.6.2	DeepHAM: Generalized Moments via DeepSets (Han, Yang & E, 2024)	137
6.6.3	Beyond Walrasian Markets: DeepSAM and Search Frictions	139
6.6.4	Which Method, When?	139
6.7	Extension: Deep Learning in the Sequence Space	140
	Further Reading	148
	Exercises	148
7	Physics-Informed Neural Networks	152
7.1	From DEQNs to PINNs: Discrete vs. Continuous Time	152
7.2	The PINN Loss and Automatic Differentiation for PDEs	152
7.2.1	A Concrete Example: Solving a 1D ODE with a PINN	154
7.3	Boundary Conditions: Soft vs. Hard Enforcement	155
7.3.1	2D Poisson Benchmark	158
7.4	Common PINN Failure Modes and Remedies	159
7.5	The Deep Galerkin Method (DGM) Architecture	160
7.6	Application: HJB Equation and the Cake-Eating Problem	161
7.7	Continuous-Time Heterogeneous Agent Models	165
7.8	Application: Black–Scholes PDE	165
7.9	From PINNs to Operator Learning: One Network, Many Problems	166
	Further Reading	167
	Exercises	167
8	Continuous-Time Heterogeneous Agent Models	169
8.1	Why Continuous Time?	170
8.2	Stochastic Calculus Refresher	170
8.2.1	Brownian Motion	171
8.2.2	Itô Processes and SDEs	171
8.2.3	Itô’s Lemma	172
8.3	The Kolmogorov Forward Equation	172
8.3.1	Derivation from First Principles	172
8.3.2	Examples	173
8.3.3	From Physics to Economics: A Continuum of Agents	173
8.4	The Hamilton–Jacobi–Bellman Equation	174
8.5	Competitive Equilibrium: Huggett and Aiyagari	176
8.5.1	The Coupled HJB–KFE System	176

8.5.2	Huggett (1993): Endowment Economy	176
8.5.3	Aiyagari (1994): Production Economy	177
8.6	A PINN Solver for the Stationary Huggett–Aiyagari System	178
8.7	The Master Equation	180
8.8	EMINNs: Solving the Master Equation with Deep Learning	181
8.8.1	Three Approximation Approaches for the Distribution	181
8.8.2	The EMINN Algorithm	182
8.8.3	Shape Constraints and Training Stability	183
8.8.4	Results and Method Comparison	183
	Further Reading	184
	Exercises	185
9	Gaussian Processes	186
9.1	Gaussian Process Regression	186
9.2	Kernel Functions and Hyperparameter Learning	188
9.2.1	Kernel Composition	188
9.3	Bayesian Active Learning for Sample-Efficient Training	189
9.4	When to Use GPs vs. DNNs	190
9.4.1	GP Regression in Practice	192
9.5	Scaling GPs to High Dimensions: Active Subspaces	192
9.5.1	Nonlinear Generalization: Deep Active Subspaces	196
9.6	Dynamic Programming with Gaussian Processes	199
9.6.1	Why GPs for DP: a Supervised-Learning View of the Bellman Operator	199
9.6.2	The Dynamic Programming Problem	200
9.6.3	The Stochastic Optimal Growth Model	201
9.6.4	GP-Based Value Function Iteration (ASGP)	201
9.6.5	Leave-One-Out Error: a Held-out Diagnostic for Free	202
9.6.6	Active Learning Inside the VFI Loop	202
9.6.7	Results: From One to 500 Dimensions	203
9.6.8	Uncertainty Quantification and Ergodic Sets	204
9.6.9	Comparison: GPs vs. DEQNs for Solving Dynamic Models	205
9.7	Deep Kernel Learning	205
9.8	GPs Among Their Bayesian Cousins	207
	Further Reading	208
	Exercises	208
10	Deep Surrogate Models and Structural Estimation	210
10.1	Motivation: The Computational Bottleneck	211
10.2	Pseudo-States: Parameters as “State” Variables	213
10.2.1	Worked Example: Black–Scholes Surrogate	214
10.3	Brock–Mirman with Parameters as Pseudo-States	214
10.4	Simulated Method of Moments	215
10.5	The SMM Workflow in the Exercise	217
10.6	Practical Considerations	218
10.7	GP Surrogate over the Moment Map	219
10.7.1	The Two-Layer Surrogate Architecture	220
10.7.2	Leave-One-Out Validation of the Moment Surrogate	221
10.7.3	Active Learning of the Moment Surrogate	222
10.7.4	The 2D SMM Criterion Surface and Partial Identification	222
10.8	Beyond SMM: Indirect Inference and Simulation-Based Inference	223
	Further Reading	224
	Exercises	225

11 Climate Economics and Deep Uncertainty Quantification	226
11.1 The Macroeconomics of Climate Change	226
11.2 The DICE Model	228
11.2.1 The IAM Landscape	228
11.2.2 Production and the Ramsey–Cass–Koopmans backbone	229
11.2.3 Industrial emissions, abatement, and the backstop technology	229
11.2.4 Land-use emissions and net output	230
11.2.5 Carbon cycle	231
11.2.6 Two-layer energy balance and radiative forcing	231
11.2.7 Damage function: closing the climate–economy loop	232
11.2.8 CDICE: recalibration of the climate module	233
11.2.9 Calibration and initial conditions, in one place	235
11.2.10 The full IAM, summarized	235
11.3 Why DICE Breaks the Stationary DEQN	235
11.4 What Changes in the DEQN Setup	236
11.4.1 Time and trends as states	236
11.5 The Non-Stationary DEQN Algorithm	236
11.6 The Planner’s Lagrangian and FOCs	237
11.7 From FOCs to a Single Loss	241
11.8 From CDICE to Stochastic IAMs	244
11.9 Bayesian Learning About Climate Sensitivity	246
11.10 Epstein–Zin Preferences	247
11.11 Deep Uncertainty Quantification via Surrogates	248
11.12 Constrained Pareto-Improving Carbon Tax in OLG-IAMs	251
11.12.1 The OLG-IAM Model	251
11.12.2 The 3-Step ML Pipeline	253
11.12.3 Results: Why Transfers Matter	255
11.13 Discussion and Outlook	260
Further Reading	261
Exercises	261
12 Synthesis and Outlook	265
12.1 The Unifying Paradigm	265
12.2 Decision Guide: When to Use Which Method	266
12.3 Running Benchmarks Through Every Lens	266
12.4 Bridges Between Methods	267
12.5 Open Challenges and Future Directions	268
12.6 When <i>Not</i> to Use Deep Learning	270
12.7 Practical Tips and Common Pitfalls	271
12.8 Concluding Remarks	272
Exercises	273
A Glossary	275
B Matrix Calculus and Automatic Differentiation	279
B.1 Matrix calculus identities	279
B.2 Reverse-mode AD in one paragraph	279
B.3 Higher-order AD	279

C	Itô Calculus, Brownian Motion, and Ergodicity	280
C.1	Brownian motion	280
C.2	Itô's lemma	280
C.3	Ergodicity in one paragraph	280
D	Fixed Points, Contraction Mappings, and the Bellman Operator	281
D.1	Banach's theorem	281
D.2	The Bellman operator is a contraction	281
D.3	Why this matters for DEQNs	281
E	Reproducibility Information	282
F	Solutions and Guidance for Exercises	284
F.1	Chapter 1: Introduction to Machine Learning and Deep Learning	284
F.2	Chapter 2: Deep Equilibrium Nets	286
F.3	Chapter 3: The International Real Business Cycle Model	289
F.4	Chapter 4: Neural Architecture Search and Loss Normalization	292
F.5	Chapter 5: Overlapping Generations Models with DEQNs	294
F.6	Chapter 6: Heterogeneous Agents and Young's Method	297
F.7	Chapter 7: Physics-Informed Neural Networks	299
F.8	Chapter 8: Heterogeneous Agent Models in Continuous Time	301
F.9	Chapter 9: Gaussian Processes	303
F.10	Chapter 10: Deep Surrogate Models and Structural Estimation	305
F.11	Chapter 11: Climate Economics and Deep Uncertainty Quantification	307
F.12	Chapter 12: Synthesis and Outlook	310

Preface

Quantitative economics and finance increasingly rely on models that are richer, more realistic, and more computationally demanding than anything attempted a generation ago. In macroeconomics, heterogeneous-agent economies, overlapping-generations models with aggregate risk, continuous-time models with occasionally binding constraints, and integrated assessment models coupling climate and economic dynamics all share the challenge of high-dimensional state spaces where traditional grid-based numerical methods are computationally infeasible. Macro-finance and asset pricing raise the same challenge from a different direction: production economies with many risky assets and leverage or collateral constraints, sovereign-default and international macro-finance models with occasionally binding frictions, dynamic portfolio choice with transaction costs and multiple illiquid assets, and the large cross-section of test assets that modern empirical asset pricing must confront all push state and parameter spaces well past the reach of tensor-product grids. Deep learning provides a promising new set of tools for addressing these challenges.

This script grew out of courses taught at multiple universities and central bank seminars over the past several years. It offers an implementation-oriented introduction to deep learning methods for solving and estimating dynamic stochastic economic models, drawing heavily on the author’s own research but also on the rapidly growing body of work by many colleagues in this field. Although the companion course is delivered as 18 lectures, the manuscript is written to be self-contained and suitable for independent study. Throughout the text, the exposition follows chapters and sections rather than the calendar of the live course. The list of references is necessarily incomplete; the literature is expanding faster than any single manuscript can track, and the selection of topics reflects one researcher’s judgment about which methods give economists and finance researchers some of the most useful entry points today. For comprehensive surveys of deep learning for dynamic economic models, the reader is referred to [Fernández-Villaverde et al. \(2024\)](#); for machine learning in empirical asset pricing and finance, to [Gu et al. \(2020\)](#), [Nagel \(2021\)](#), and [Kelly and Xiu \(2023\)](#); and to the references therein. The present script aims to complement such surveys by providing concrete algorithms, working code, and step-by-step examples that readers can adapt to their own research problems.

What these notes cover. The exposition is organized around four complementary methodologies:

1. **Deep Equilibrium Nets (DEQNs)**, which embed the equilibrium conditions of a dynamic stochastic model directly into the loss function of a neural network, replacing the traditional numerical solution step with gradient-based optimization ([Azinovic et al., 2022](#)). We apply this framework to representative-agent models, international business cycles, overlapping generations, and heterogeneous-agent economies, including the histogram-based treatment of distributions building on [Young \(2010\)](#) and the later sequence-space extension of [Azinovic-Yang and Žemlička \(2025a\)](#).
2. **Physics-Informed Neural Networks (PINNs)**, which approximate Hamilton–Jacobi–Bellman equations and Kolmogorov forward equations in continuous time without computing conditional expectations ([Sirignano and Spiliopoulos, 2018](#); [Raissi et al., 2019](#)). We develop

this machinery for consumption-savings problems and option pricing, and compare it with finite-difference HJB–KFE methods for continuous-time heterogeneous-agent models in the tradition of Achdou et al. (2022) and with the Economic Model Informed Neural Networks (EMINNs) of Gu et al. (2024).

3. **Deep surrogate models and Gaussian processes**, which construct fast, differentiable approximations of expensive structural models, enabling rapid estimation, uncertainty quantification, and global sensitivity analysis (Scheidegger and Bilonis, 2019; Chen et al., 2026). Once trained, the same surrogates can also be used to *design constrained optimal policies*, including in dynamic stochastic heterogeneous-agent models: a long search over policy parameters that would require thousands of full re-solves of the structural model collapses into a small optimization on the surrogate. The climate-economics chapter uses this construction to derive Pareto-improving carbon-tax rules in an OLG–IAM with deep uncertainty (Kübler et al., 2026).
4. **GP-based dynamic programming**, which embeds Gaussian process regression into a value function iteration algorithm (Engel et al., 2005; Deisenroth et al., 2009; Renner and Scheidegger, 2018) and, when combined with active subspaces (Constantine, 2015), scales to economies with up to 500 continuous state variables (Scheidegger and Bilonis, 2019).

The unifying thesis is that economic structure drives the learning problem rather than being separated from it. In DEQNs, PINNs, and EMINNs the structure is embedded directly into a neural-network loss as residual terms (equilibrium conditions, PDEs, Bellman equations), making the training objective itself unsupervised; in the surrogate and GP-based estimation chapters the structure instead appears upstream, in the model that generates the (input, output) training pairs the surrogate then learns in a standard supervised regression. Either way, the four methodologies above are different surfaces for the same idea.

How these notes are organized. At a high level, Chapters 1–6 build the discrete-time toolkit (foundations, DEQNs, IRBC, architecture search and loss normalization, OLG, heterogeneous agents). Chapters 7–8 turn to continuous-time methods (PINNs, HJB–KFE for heterogeneous agents). Chapter 9 develops the Gaussian-process toolkit, including kernel design, Bayesian active learning, active subspaces, and a GP-based value-function-iteration scheme that solves high-dimensional dynamic programs by surrogating the Bellman operator itself. Chapter 10 develops the deep-surrogate pseudo-state pattern that treats structural parameters as additional state variables and puts it to work for structural estimation via SMM, with a second GP layer on top of the moment map for high-throughput bootstrap and Bayesian post-processing. Chapter 11 applies the toolbox to climate economics under deep uncertainty, and Chapter 12 closes by comparing methods and highlighting open problems. The next chapter, *How to Read This Script*, expands this overview into a full reading guide: cover-to-cover and selective paths, notation and cross-references, the role of the companion notebooks, and the visual conventions used throughout.

Audience and prerequisites. These notes are aimed at PhD students in economics, finance, and computational social science, as well as researchers at central banks and policy institutions who wish to apply these methods to their own models. We assume familiarity with basic econometrics, Python programming, and undergraduate-level calculus and probability. For readers who need a refresher on these foundations, we recommend Goodfellow et al. (2016) for deep learning, Judd (1998) for numerical methods in economics, the open-source QuantEcon lectures (<https://quantecon.org>) for Python programming and quantitative economics, and the Econ-ARK toolkit (<https://econ-ark.org>) for heterogeneous-agent modeling.

Relation to the literature. These notes complement several existing references. The comprehensive survey by Fernández-Villaverde et al. (2024) provides a broad overview of deep learning for economics, while Gu et al. (2020), Nagel (2021), and Kelly and Xiu (2023) survey the rapidly growing machine-learning literature in empirical asset pricing and finance; Judd (1998) remains the standard reference for classical numerical methods; the textbooks by Stokey et al. (1989) and Ljungqvist and Sargent (2018) cover the economic theory that underpins the models we solve; the open first-year Ph.D. textbook by Azzimonti et al. (2026) provides a general introduction to state-of-the-art macroeconomics; and the open dynamic-programming volumes by Sargent and Stachurski (2026) provide a modern computational treatment of recursive methods and operator-based analysis. The present script is distinguished by its focus on *implementation*: every method is accompanied by working code, and the exposition is driven by concrete economic applications rather than abstract theory.

Software and reproducibility. All code examples are available as executable Jupyter notebooks in TensorFlow and PyTorch, hosted on the companion GitHub repository.¹ The notebooks are designed to run on standard hardware; GPU acceleration is beneficial but not required for the classroom-scale examples. Each chapter indicates which notebooks accompany it.

A snapshot of a fast-moving field. Deep learning for quantitative economics and finance is evolving at an extraordinary pace, with new architectures, training algorithms, and applications appearing month after month. Both this script and the accompanying lecture suite are therefore best read as a snapshot of a highly dynamically evolving field rather than as a definitive treatment, and they are necessarily incomplete: the algorithms, models, and references collected here are those that, at the time of writing, strike the author as the most useful entry points for an economist or finance researcher beginning to work with deep learning, but the frontier is moving quickly enough that some choices will inevitably look dated within a year or two, and many worthy contributions have had to be left out. The companion GitHub repository will track revisions as the field matures, and readers are encouraged to consult it for the most recent versions of the code and references.

Feedback and errata. This script is a living document. Corrections, suggestions, and pull requests are welcome on the companion GitHub repository’s issue tracker.² Direct correspondence may be addressed to simon.scheidegger@unil.ch.

Acknowledgments. I am grateful to the many seminar and course participants whose questions and comments have shaped this material. I also thank Marlon Azinovic, Johannes Brumm, Christopher Carroll, Hui Chen, Antoine Didisheim, Richard Evans, Jesús Fernández-Villaverde, Lukas Frank, Aleksandra Friedl, Luca Gaegauf, Kenneth Gillingham, Pavel Ievlev, Mitsuru Igami, Felix Kübler, Gianni Lombardo, Maria Pia Lombardo, Marko Mlikota, Galo Nuño, Jonathan Payne, Philipp Renner, Andreas Schaab, Frank Schorfheide, Anna Smirnova, Oliver Surbek, Fabio Trojani, Yucheng Yang, and Jan Žemlička for the research and discussions that underpin this work. Part of this manuscript was written while visiting the Bank for International Settlements (BIS); I thank the BIS for its hospitality. This script also benefited substantially from Anthropic’s Claude as a writing and coding assistant, in particular for polishing earlier drafts of the manuscript and for modernizing the companion code examples that had aged across several previous iterations of the course. I would also like to extend special thanks to Matthew McKay, Thomas J. Sargent, and John Stachurski for their support with the QuantEcon e-book edition (<https://quantecon.org/books/>). Most importantly, I want to thank my wife,

¹https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

²https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models/issues

Michaela, and my daughter, Kira, for making this work possible through their unwavering support. Responsibility for any remaining errors, of course, rests with the author.

Simon Scheidegger

HEC, University of Lausanne, 2026

How to Read This Script

The four methodologies sketched in the Preface, namely DEQNs, PINNs, deep surrogates with Gaussian processes, and GP-based dynamic programming, share a common philosophy but differ in their prerequisites, dependencies, and natural reading order. This short orientation translates that conceptual map into a practical reading plan: how the chapters, companion notebooks, notation tables, and visual conventions fit together, and how the manuscript can be used either as a sequential textbook for a reader encountering these tools for the first time or as a reference one can drop into at the level of a single method.

Although the live course groups the material into eight classroom sessions, the manuscript is organized by conceptual dependency rather than by calendar time. Each main chapter follows the same rhythm: the economic problem first, then the computational obstacle that classical methods leave open, then the neural-network or GP construction that closes it, then the algorithmic implementation, and finally exercises and notebooks that turn the method into running code. The notebooks are not an optional appendix; they are part of the exposition. They make explicit which residuals are minimized, which variables enter as states or pseudo-states, how expectations are approximated, and how accuracy is checked in practice, and the chapters are written on the assumption that the reader will at least skim the corresponding code.

Cover to cover. A reader new to both deep learning and computational economics should read the script in order. Chapter 1 supplies the machine-learning vocabulary used everywhere else: losses, gradients, backpropagation, architectures, optimization, and generalization. Chapters 2–6 then develop the discrete-time equilibrium toolkit, moving from representative-agent DEQNs to international business cycles, loss balancing, overlapping generations, and heterogeneous-agent distributions. Chapters 7–8 shift from conditional expectations to continuous-time PDE residuals, first through PINNs and then through HJB–KFE systems. Chapters 9–10 introduce surrogate models, Gaussian processes, active subspaces, and structural estimation. Chapter 11 uses these pieces in a climate-economics application, and Chapter 12 closes by comparing methods and highlighting open problems.

Selective reading paths. Readers who already know part of the material can take a shorter route, but the dependencies still matter:

- **Discrete-time equilibrium methods:** read Chapters 1–6. This path covers the DEQN template, expectation approximation, high-dimensional DSGE examples, loss normalization, OLG models, and histogram-based heterogeneous-agent distributions.
- **Continuous-time methods:** read Chapter 1 for neural-network basics, Chapter 7 for PINNs and PDE residuals, and Chapter 8 for the continuous-time heterogeneous-agent HJB–KFE system. Chapter 6 is useful background if the distribution μ_t is unfamiliar.
- **Surrogates and estimation:** read Chapter 1 for optimization and approximation, Chapter 9 for the Gaussian-process toolkit (kernels, Bayesian active learning, active subspaces, and GP-based dynamic programming), and Chapter 10 for the deep-surrogate

pseudo-state pattern and the SMM pipeline that uses it. Chapter 2 is the minimum DEQN background for the Brock–Mirman examples used there.

- **Climate economics:** read Chapter 11 together with Chapter 2 for DEQN solution logic and Chapter 9 for surrogate-based uncertainty quantification. The climate chapter is written as an application chapter, not as a replacement for those methodological chapters.

These reading paths are minimum self-sufficient sets: chapters not listed are not strictly required for the path, but several later sections cross-reference earlier chapters more deeply, and the chapter introductions flag those dependencies explicitly when they arise.

Code and reproducibility. Each chapter is accompanied by executable Jupyter notebooks that implement the methods described in the text. The notebooks are listed at the end of each chapter and are available on the companion GitHub repository.³ The classroom-scale examples are intentionally smaller than research-scale calibrations: their role is to make the residuals, training loops, diagnostics, and validation logic transparent. Production-scale settings, when relevant, are noted in comments within the notebooks. A useful workflow is to read the chapter once for the economic and mathematical structure, run or inspect the notebook to see the implementation, and then return to the chapter’s exercises to check whether the approximation and validation choices are understood.

Notation and cross-references. The notation tables following this orientation are meant to reduce ambiguity across chapters. Some symbols are necessarily reused because the script covers several literatures: for example, ρ can denote neural-network parameters, an optimizer coefficient, or a continuous-time discount rate depending on context. When a symbol changes role, the notation table records the chapter-specific convention. Cross-references should be used actively: many later chapters refer back to the same core objects, such as residual losses, ergodic sampling, market-clearing errors, HJB residuals, or GP posterior variances. The goal is not to memorize every symbol on first reading, but to use the notation tables as a stable reference while moving between chapters and notebooks.

Visual conventions. The script uses three colored callout boxes plus an algorithm box. Each color is consistent across the entire manuscript and signals a particular kind of content (the first instance of each box appears in Chapter 1):

- **Blue boxes (Definitions and Algorithms).** Used for formal definitions, key constructions, and step-by-step pseudocode. When a section introduces a new method, the algorithmic core is typically displayed in a blue box.
- **Green boxes (Remarks and Worked Examples).** Used for practical guidance, worked numerical examples, and short discussions that contextualize a result. Green boxes can be skipped on a first read without losing the main thread.
- **Crimson boxes (Key Insights).** Used sparingly to flag the highest-level take-aways of a section, ideas the reader should remember even after the implementation details fade.
- **Numbered algorithms.** Inside blue boxes labeled “Algorithm: ...,” the pseudocode follows the conventions of the `algorithmic` package: INPUT / OUTPUT statements, indented FOR / IF blocks, and arrow-style assignments.

A few additional typographic conventions: file paths and code identifiers are set in `monospace`; emphasized terms appear in **bold crimson**; cross-references to other chapters use the form (Ch. N) or (§ $N.M$); and figure / table captions sit below their objects.

³https://github.com/sisctei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Notation and Symbols

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^d$	Input or state vector, depending on context
$p(\mathbf{x})$	Policy/control associated with state $\mathbf{x}$
$G(\mathbf{x}, p(\mathbf{x}))$	Equilibrium or residual operator (e.g., Euler equation)
$\mathcal{N}_\rho$	Neural network with parameters ρ
ℓ	Loss function (supervised or residual-based)
η	Learning rate
β_1, β_2	Adam momentum coefficients
$\mathcal{D}$	Dataset or collocation set
θ	Generic model or structural parameter vector; when structural parameters and neural-network weights must be distinguished, ρ denotes network weights
A	Number of OLG cohorts (Ch. 5)
$G_t(k_i, \varepsilon_j)$	Histogram mass at grid point k_i , shock ε_j (Ch. 6)
μ_t	Cross-sectional wealth distribution (Ch. 6–8)
$V(a, z)$	Value function in continuous-time HJB (Ch. 7–8)
$g(a, z)$	Stationary density from the KFE / Fokker–Planck equation (Ch. 7–8)
SCC_t	Social cost of carbon (Ch. 11)

Where necessary, chapter-specific notation (e.g., HJB/PDE operators, kernel functions) is introduced locally to avoid ambiguity. In a few places, the script intentionally reuses symbols such as η when that is standard in the underlying literature; in those cases, the local chapter definition takes precedence.

Symbols with conflicting uses across chapters. Several symbols below are reused with different meanings depending on the chapter, because each chapter inherits the convention of its primary source. This table collects the conflicts in one place; chapters that introduce a new local meaning also add a one-line warning at first use.

Symbol	Meanings (by chapter)
γ	IES = 1/CRRA in Ch. 3 (IRBC); CRRA in Ch. 7 (cake-eating) and Ch. 8 (continuous-time HA); reused as σ_u in Ch. 11 (OLG-IAM) to free σ_t for emissions intensity; RL discount factor $\gamma \in [0, 1]$ and BatchNorm scale parameter in Ch. 1; Hyperband / Successive-Halving reduction factor in Ch. 4.
η	Learning rate (Ch. 1–2); TFP shock in OLG (Ch. 5); idiosyncratic productivity in Krusell–Smith (Ch. 6); OU mean-reversion (Ch. 8); small numerical shift in I-spline basis; normalized temperature costate (Ch. 11); normalized spatial coordinate in PINN bilinear BC construction (Ch. 7); functional-derivative test perturbation in KFE adjoint argument (Ch. 8).
α	Capital share in Cobb–Douglas production (Ch. 2, 5, 6, 8, 11); ReLoBRaLo smoothing parameter (Ch. 4); boundary MPC head in I-spline (Ch. 7).
ζ vs. α	Capital share is denoted ζ in Ch. 3 (Azinovic et al. convention) and α everywhere else.
T	ReLoBRaLo softmax temperature (Ch. 4); time horizon (Ch. 7, 11); atmospheric temperature T_t^{AT} in DICE (Ch. 11); data sample size in SMM (Ch. 10).
δ	Capital depreciation rate (most chapters); Dirac measure in master equation (Ch. 6); Huber-loss threshold in robust regression (Ch. 1).
ψ	IES in Ch. 11 Epstein–Zin preferences (paired with γ_u for risk aversion); Cobb–Douglas capital exponent in the GP-VFI growth model used to demonstrate active-subspace scaling (Ch. 9, §9.6).
μ	Cross-sectional wealth distribution μ_t (Ch. 6–8); SGD momentum coefficient (Ch. 1); Lagrange / KKT multiplier on investment irreversibility (Ch. 3); emissions abatement rate $\mu_t \in [0, 1]$ (Ch. 11).
ρ	Network parameters $\mathcal{N}_\rho$ (most chapters); RMSprop decay coefficient and recurrence spectral radius (Ch. 1); discount rate in continuous-time HJB (Ch. 7, 8); ReLoBRaLo baseline-mix coefficient (Ch. 4). The variant ϱ is reserved for shock persistence in Ch. 2. TFP persistence is denoted ρ_z in Ch. 3 (Azinovic et al. convention) and ϱ elsewhere.
σ	Logistic activation $\sigma(z) = 1/(1 + e^{-z})$ in Ch. 1 (single-output and final-layer use); the same symbol is used as a generic non-linearity in the RNN recurrence and as the LSTM gate non-linearity later in the same chapter, so the meaning is always logistic but the typographic role (specific vs. generic) shifts. Ch. 11 (climate) reserves σ_t for emissions intensity, σ_u for household CRRA.
τ	Bounded time index τ_t used as a network input in the deterministic CDICE-DEQN derivation (Ch. 11, §11.11); separately, the per-period carbon tax rate (also written τ_t) later in the same chapter, in the OLG-IAM and Pareto-improving-tax discussion. The notation is reset locally at each first use; readers should rely on the surrounding sentence rather than on the symbol alone.

Default reading. When in doubt, default to the most common usage: ρ is the neural-network parameter vector, η is the learning rate, α is the Cobb–Douglas capital share, and σ is the logistic activation. Chapter-specific reuses always override locally, and the chapter introductions flag a divergent meaning at first use. The conflict table above is a forward-reference for non-linear readers; a cover-to-cover reader will see each meaning introduced once and need not consult the table on a first pass.

Abbreviations and acronyms. The following acronyms appear throughout the script. They are introduced in full at first use within each chapter; this list serves as a quick reference.

ABC	Approximate Bayesian Computation	KFE	Kolmogorov forward (Fokker–Planck) Eq.
ACE	Analytic Climate Economy (Traeger)	KKT	Karush–Kuhn–Tucker conditions
AD	Automatic Differentiation	KS	Krusell–Smith (1998) economy
AdamW	Adam with decoupled weight decay	LSTM	Long Short-Term Memory net
AS	Active Subspace	MAGICC	Reduced-complexity climate emulator
BAL	Bayesian Active Learning	MC	Monte Carlo
BC	Boundary Condition	MFG	Mean Field Game
BSDE	Backward Stochastic Differential Eq.	ML	Machine Learning
CDICE	Calibrated DICE (Folini 2024)	MLE	Maximum Likelihood Estimator
CRN	Common Random Numbers	MLP	Multi-Layer Perceptron
CRRA	Constant Relative Risk Aversion	MMW	Maliar–Maliar–Winant (2021)
DEQN	Deep Equilibrium Net	MPC	Marginal Propensity to Consume
DGM	Deep Galerkin Method	NAS	Neural Architecture Search
DICE	Dyn. Integ. Climate-Econ. model	NTK	Neural Tangent Kernel
DKL	Deep Kernel Learning	OLG	Overlapping Generations
DL	Deep Learning	PDE	Partial Differential Equation
DNN	Deep Neural Network	PINN	Physics-Informed Neural Net
ECS	Equilibrium Climate Sensitivity	QMC	Quasi-Monte Carlo
EGM	Endogenous Grid Method	ReLoBRaLo	Relative Loss Balancing
ELU	Exponential Linear Unit	RL	Reinforcement Learning
EMINN	Economic Model Informed NN	RNN	Recurrent Neural Network
FaIR	Reduced-complexity climate emulator	SBI	Simulation-Based Inference
FB	Fischer–Burmeister loss	SCC	Social Cost of Carbon
FD	Finite Differences	SDE	Stochastic Differential Equation
FNO	Fourier Neural Operator	SGD	Stochastic Gradient Descent
FOC	First-Order Condition	SMM	Simulated Method of Moments
GE	General Equilibrium	TF / TF2	TensorFlow / TensorFlow 2
GMM	Generalized Method of Moments	UQ	Uncertainty Quantification
GP	Gaussian Process	VFI	Value Function Iteration
HA	Heterogeneous Agent	ZLB	Zero Lower Bound
HJB	Hamilton–Jacobi–Bellman Eq.	XLA	Accelerated Linear Algebra (TF/JAX)
IRBC	Internat. Real Business Cycle	DeepONet	Deep Operator Network
JAX	JAX autodiff library (Google)	DeepHAM	Deep Heterogeneous-Agent Model

Execution Map

The following table maps each manuscript chapter to its companion slide deck(s) and Jupyter notebooks. All paths are relative to the repository root; the short names are intentionally compact so the table remains readable.

Execution map: manuscript chapters, slides, and notebooks

Ch.	Topic	Lecture folder & deck	Notebooks (role)
1	Intro to ML & DL	L02: 01_Intro_to_DL	L02 01–09 (c×9)
2	Deep Equilibrium Nets	L03: 02_DEQNs; L07: 05b_AutoDiff	L03 01–02 (c), 03–04 (e/s), 05 (c); L07 01–04 (c)
3	IRBC Model	L04: 03_IRBC	L04 01–02 (c)
4	NAS & Loss Norm.	L05: 04_NAS, 05_Loss	L05 02–04 (c), 05 (e)
5	OLG Models	L08: 08_OLG	L08 07–10 (c), 11 (e)
6	HA, Young, Seq. Space	L09: 09_HA_Young; L10: 10_SeqSpace	L09 10–12 (c); L10 05, 05b, 06 (c), KrusellSmith_Tutorial_CPU (x)
7	PINNs	L11: 06_PINNs	L11 01–05 (c)
8	CT Het. Agents	L12: 07_CT_Theory; L13: 08_CT_Num	L13 06–08 (c), 09 (e)
9	Surrogates, GPs, DKL	L14: 07_Surrogates_GPs	L14 01, 02, 04–08 (c), 07, 09, 10 (x)
10	Structural Estimation	L15: 08_Struct_Est	L15 03, 03b (c)
11	Climate & Deep UQ	L16: 08_Climate; L17: 09_UQ	L16 01–03 (c); L17 09_DICE_2P_UQ_Analysis (c) plus 4 .py pipeline drivers
12	Synthesis & Outlook	L18: 10_Wrap_Up	—

Path conventions. The repository organizes lectures by stable block id (`lectures/lecture_N_BY_Y_*/`); the leading *LN* in the table is the student-facing lecture number and the parenthetical *BY_Y* is the canonical block id (which is stable across renumberings). Slide and notebook names in the table are abbreviated; full paths follow `lectures/lecture_N_BY_Y_*/{slides,code}/`. Notebook role letters: **c** = core, **e** = exercise, **s** = solution (paired with an exercise notebook), **x** = extension/self-study. See the README for complete file names and direct links.

Workshop material. The live course includes a hands-on workshop on agentic programming (using AI agents as coding partners), delivered as L06. Because this field is evolving quickly, it is presented through slides, two Python helper scripts, and exercises rather than as a fixed manuscript chapter. The manuscript remains organized chapter by chapter, with the workshop material collected in the L06 slides and exercise prompts.

Reproducibility. Random-seed conventions, the `RUN_MODE` budget split, hardware and software pins, and GPU-determinism flags used by every notebook in the table above are documented in Appendix E. Worked solutions and guidance for the end-of-chapter exercises are collected in Appendix F.

Chapter 1

Introduction to Machine Learning and Deep Learning

The Preface argued that quantitative economics needs new tools because the models of interest, heterogeneous-agent economies, OLG models with aggregate risk, integrated assessment models, all share state spaces too large for traditional grid-based methods. This chapter takes that argument as given and supplies the technical machinery the rest of the manuscript will build on. Readers who want a fuller pitch for *why* deep learning is the right response should re-read the Preface; readers who already accept the premise can dive directly into the machinery below.

We begin with a brief refresher on the motivation, mostly to fix vocabulary and citations, then survey the three broad paradigms of machine learning (supervised, unsupervised, and reinforcement learning), and then develop the core technical machinery: neural network architectures, loss functions, gradient-based optimization, backpropagation, weight initialization, activation functions, and the modern theory of generalization including the double descent phenomenon. Readers already comfortable with these topics may skim this chapter and proceed to Chapter 2, where the economic applications begin.

The foundational references for the material in this chapter include McCulloch and Pitts (1943), Hebb (1949) and Rosenblatt (1958) for the historical origins of artificial neurons and the first learning rules, Cybenko (1989) and Hornik et al. (1989) for universal approximation, Rumelhart et al. (1986) for backpropagation, Robbins and Monro (1951) for the stochastic approximation roots of SGD, Geman et al. (1992) for the bias/variance dilemma that underpins modern generalization theory, Kingma and Ba (2015) for the Adam optimizer, Ioffe and Szegedy (2015) and Srivastava et al. (2014) for batch normalization and dropout, and Goodfellow et al. (2016) for a comprehensive textbook treatment, complemented by the historical survey of Schmidhuber (2015).

1.1 Motivation and Applications

The past decade has witnessed a remarkable convergence of three developments that have transformed machine learning from a niche academic pursuit into a practical tool of extraordinary power: the availability of large-scale datasets, the advent of massively parallel hardware (GPUs and TPUs), and algorithmic innovations in training deep neural networks (Figure 1.1). While much of the public attention has focused on applications such as image recognition, natural language processing, and game playing, the implications for economics and finance are equally profound.

Deep learning has already demonstrated its potential across a broad range of economic applications. In macroeconomics, neural networks serve as global approximators of policy and value functions in high-dimensional equilibria that classical grid-based methods cannot reach

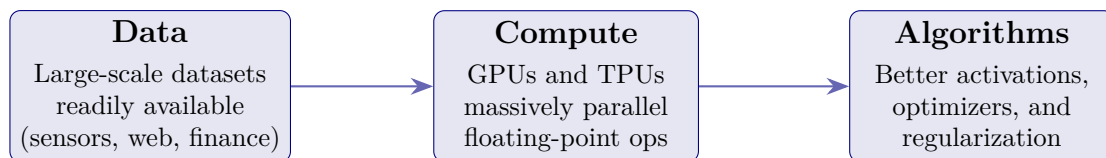


Figure 1.1: The three enablers of the deep-learning revolution: large-scale data, massively parallel compute, and algorithmic improvements. None of the three alone is sufficient; their co-availability since the early 2010s is what has turned neural networks from a niche academic curiosity into a workhorse scientific and industrial tool.

(Maliar et al., 2021; Azinovic et al., 2022); heterogeneous-agent extensions encode cross-sectional distributions via histograms or permutation-invariant moment networks (Young, 2010; Han et al., 2024; Yang et al., 2025). Search-and-matching models with aggregate shocks (Payne et al., 2025) and continuous-time macro-finance settings requiring HJB approximation or deep-BSDE solvers (Gopalakrishna, 2024; Duarte et al., 2024; Han et al., 2018) add further coverage, as do optimal monetary policy rules under persistent supply shocks (Nuño et al., 2024). In climate economics, surrogate-based workflows solve integrated assessment models and derive Pareto-improving carbon-tax rules in OLG-IAMs with deep uncertainty (Kübler et al., 2026; Folini et al., 2025; Fernández-Villaverde et al., 2025). In finance, surrogate models accelerate option pricing, sovereign-default computation, and portfolio optimization (Hutchinson et al., 1994; Scheidegger and Treccani, 2018; Arellano, 2008; Gaegauf et al., 2023; Chen et al., 2025). For structural estimation, neural-network surrogates make inference tractable and enable global uncertainty quantification for IAMs (Kase et al., 2022; Friedl et al., 2023; Chen et al., 2026). Finally, these methods connect to an earlier generation of neural-network function approximation in economic computation: adaptive learning (Chen and White, 1998), derivative pricing (Hutchinson et al., 1994), and parameterized expectations (Duffy and McNelis, 2001), with Valaitis and Villa (2024) showing how the parameterized-expectations approach extends to contemporary deep architectures; structural discrete-choice estimation rounds out the historical picture (Norets, 2012).

Two themes cut across these application areas and motivate the rest of the script. First, every application area listed there involves a state space whose dimension grows with the number of agents, assets, shocks, or climate states; tensor-product grids become infeasible long before the modeling questions become uninteresting. Second, neural networks are universal approximators with cost that scales with parameter count rather than with N^d , so they are the natural replacement function class once the problem becomes high-dimensional. Subsequent chapters take this through-line and develop it: Chapter 2 introduces the DEQN methodology that all macro / heterogeneous-agent / search applications above share; Chapter 3 scales it to a 100+-country benchmark; Chapters 7–8 develop the continuous-time analogue; Chapter 9 develops the Gaussian-process methodology, and Chapters 10–11 put the deep-surrogate and GP machinery to work on structural estimation and integrated assessment models.

In this course, we focus on the recent advances enabled by *deep* neural networks, modern hardware, and algorithmic innovations in training.

For central banks, in particular, these tools address pressing practical needs. Many of the models listed above, such as heterogeneous-agent New Keynesian (HANK) models, search and matching models with aggregate shocks, overlapping-generations (OLG) economies with aggregate risk, and integrated assessment models coupling climate and economic dynamics, involve state spaces of dimension $d \gg 5$, where traditional grid-based numerical methods are computationally infeasible: a tensor-product grid with N points per dimension requires N^d nodes, the canonical *curse of dimensionality* of Bellman (1961). Deep learning provides a mesh-free function approximator. For Barron-class functions, two-layer networks achieve dimension-independent rates in the number of hidden units (often stated as squared L^2 error of

order $\mathcal{O}(1/n_1)$ under a bounded Barron norm), whereas tensor-product grids for generic smooth functions suffer rates such as $\mathcal{O}(M^{-2/d})$ in the total number $M = N^d$ of grid nodes. The point is not that every economic object satisfies a Barron bound, but that neural approximators avoid the mechanical tensor-product explosion. Chapter 2 returns to this comparison with concrete numbers for a six-shock DSGE.

Specifically, this course focuses on using deep neural networks as computational tools for: (i) solving high-dimensional dynamic stochastic general equilibrium (DSGE) models, (ii) approximating value and policy functions in continuous-time settings via physics-informed neural networks, (iii) constructing fast and accurate surrogate models for parameter estimation and uncertainty quantification, and (iv) leveraging Gaussian processes for sample-efficient Bayesian active learning (Azinovic et al., 2022; Friedl et al., 2023; Chen et al., 2026). Supporting techniques include neural architecture search and adaptive multi-objective loss balancing (one option, ReLoBRaLo (Bischof and Kraus, 2025), is developed in the notebooks; alternatives include SoftAdapt and GradNorm). For a comprehensive textbook treatment of the foundations, we refer the reader to Goodfellow et al. (2016) and Chollet (2017); for concise surveys we recommend LeCun et al. (2015).

1.2 Types of Machine Learning

Before diving into the technical details, it is useful to recall the three broad paradigms of machine learning, each defined by the nature of the available data and the feedback signal provided to the algorithm.

1.2.1 Supervised Learning

Given a set of *labeled* input–output pairs $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, the goal is to learn a mapping $h : \mathcal{X} \rightarrow \mathcal{Y}$ that generalizes to unseen inputs. The two main tasks are *regression* and *classification*.

Regression ($y \in \mathbb{R}$): predict a continuous target from input features. A simple linear model takes the form

$$h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x,$$

where the parameters $\boldsymbol{\theta} = (\theta_0, \theta_1)$ are learned from data. Figure 1.2 illustrates regression on a house-price dataset: each dot is a training observation, and the line is the fitted model.

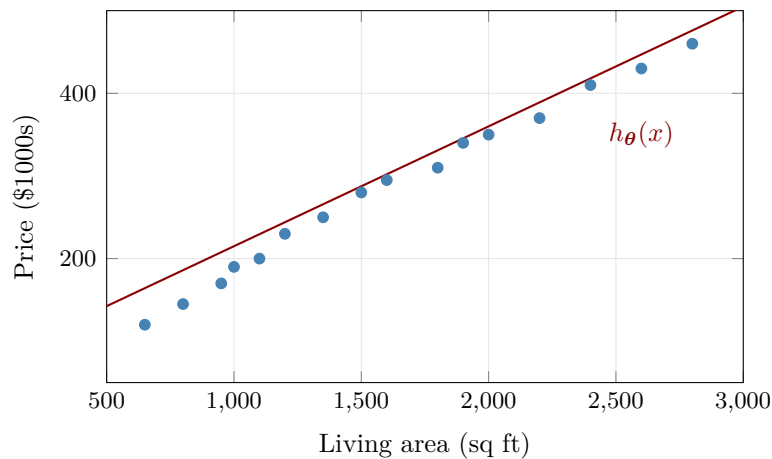


Figure 1.2: Supervised learning: regression. The model $h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x$ (red line) is fitted to observed house prices (blue dots).

Classification. ($y \in \{0, 1, \dots, K\}$): assign an input $\mathbf{x}$ to one of K discrete categories. A linear classifier predicts class 1 whenever $\mathbf{w}^\top \mathbf{x} + b > 0$ and class 0 otherwise. Figure 1.3 shows a

credit-scoring example: applicants are classified as low-risk or high-risk based on income and savings, and the dashed line is the learned decision boundary.

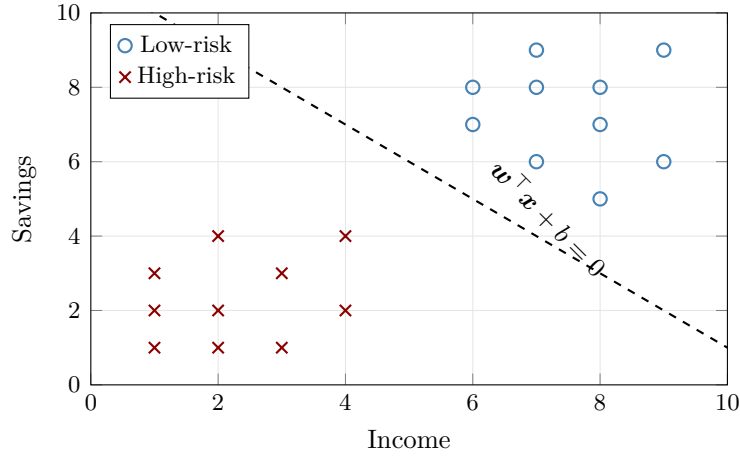


Figure 1.3: Supervised learning: classification. A linear decision boundary separates low-risk (blue circles) from high-risk (red crosses) applicants in the income–savings feature space.

1.2.2 Unsupervised Learning

Given only *unlabeled* data $\{\mathbf{x}^{(i)}\}_{i=1}^m$, the goal is to discover hidden structure without any target signal. Two common tasks are:

- **Clustering:** partitioning data into groups of similar observations. *Example:* segmenting firms into peer groups based on financial characteristics such as size, leverage, and profitability.
- **Dimensionality reduction:** compressing features into fewer dimensions while preserving important variation. *Example:* principal component analysis of yield curves, where three factors (level, slope, curvature) capture most of the cross-sectional variation.

Figure 1.4 illustrates a clustering task: unlabeled data points in two dimensions are partitioned into three clusters, each indicated by a different color and centroid marker.

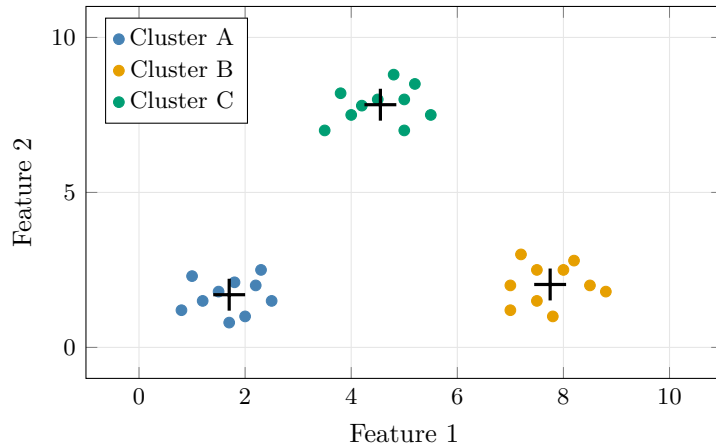


Figure 1.4: Unsupervised learning: clustering. Unlabeled data points are grouped into three clusters; the + markers indicate cluster centroids. No target labels are used; the algorithm discovers the grouping from the data alone.

1.2.3 Reinforcement Learning

In reinforcement learning, an *agent* interacts with an *environment* over a sequence of time steps. At each step t , the agent observes a state s_t , selects an action $a_t = \pi(s_t)$ according to its policy π , and receives a reward r_t from the environment. The goal is to learn a policy that maximizes the expected cumulative discounted return:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad \gamma \in [0, 1),$$

where $\mathbb{E}_{\pi}[\cdot]$ is taken over the trajectory distribution induced jointly by the policy π and the (possibly stochastic) environment dynamics, starting from a given initial-state distribution. Figure 1.5 illustrates this agent–environment interaction loop.

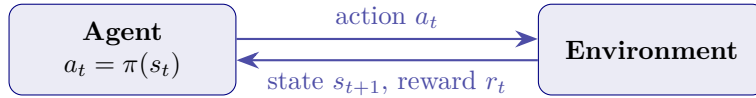


Figure 1.5: Reinforcement learning: the agent–environment loop. The agent observes a state, takes an action, and receives a reward signal. Over time, it learns a policy π that maximizes cumulative discounted reward.

Example: an algorithmic trader learning an execution strategy by optimizing realized profit over sequences of order placements; or a central bank learning an interest-rate rule by maximizing a welfare criterion over simulated macroeconomic trajectories.

1.3 Course Focus: Supervised vs. Unsupervised Learning

This course begins with the *supervised learning* paradigm, which provides the essential building blocks: choosing a parameterized model, defining a loss function, and minimizing it via gradient descent.

The core methods of this course, DEQNs and PINNs, are not supervised in the classical labeled-data sense. More precisely, they are *self-supervised residual methods*: the economic equilibrium conditions or governing equations generate the training signal. To see why, recall the key distinction: in supervised learning, the loss function measures the discrepancy between the network’s prediction $\hat{y}^{(i)}$ and a known target label $y^{(i)}$, for example, a mean squared error $\frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$. This requires a dataset of correct input–output pairs.

In DEQNs and PINNs, *no such labels exist*. Consider the key differences:

- **DEQNs:** A neural network approximates the unknown policy function of a dynamic economic model. The loss function is the *Euler equation residual*, which measures how much the network’s output violates the model’s optimality conditions at sampled state points. The “correct” policy values are never provided; instead, the network discovers the equilibrium by driving these residuals to zero.
- **PINNs:** A neural network approximates the unknown solution to a partial differential equation (PDE). The loss function is the *PDE residual*, which evaluates how well the network satisfies the differential equation at randomly sampled collocation points. Again, the true solution is not available as training data; the network learns it by enforcing the PDE constraint.

In both cases, the training data consists only of *input locations* (sampled states or collocation points) with no associated output labels. The loss is defined entirely by the structure of the economic model or the governing equation, not by example solutions. They are therefore

unsupervised in the narrow sense of using no labels, but the more informative term is equation-based self-supervision.

Despite this fundamental difference, the optimization machinery is shared: these approaches define a loss $J(\boldsymbol{\theta})$ over trainable parameters and minimize it via (stochastic) gradient descent. This is why we introduce the supervised learning pipeline first in the next section: it establishes the model–loss–optimizer framework that DEQNs and PINNs then adapt by replacing the data-driven loss with a physics-based one.

1.4 The Supervised Learning Pipeline

Every supervised learning algorithm follows the same three-step recipe, regardless of whether the model is a linear regression, a random forest, or a deep neural network (Figure 1.6). Understanding this pipeline is essential because the DEQN and PINN methods in later chapters modify step 2 (replacing data-driven losses with physics-based residuals) while keeping steps 1 and 3 intact.

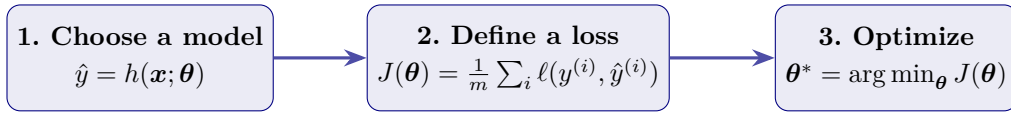


Figure 1.6: The three-step supervised-learning recipe that underpins every model in this course. Choose a parametric hypothesis $h(\mathbf{x}; \boldsymbol{\theta})$, measure its misfit on a labeled dataset via a loss $J(\boldsymbol{\theta})$, and minimize J over the parameter vector $\boldsymbol{\theta}$. DEQNs and PINNs modify step 2 (replacing the data-driven loss with an equilibrium or PDE residual) while keeping steps 1 and 3 identical.

Given a training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ of input–output pairs, we seek a hypothesis $h : \mathcal{X} \rightarrow \mathcal{Y}$ parameterized by $\boldsymbol{\theta}$ that minimizes the empirical risk. For regression problems, the default choice is the mean squared error (MSE):

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2. \quad (1.1)$$

This loss is not chosen arbitrarily. If the data are generated by

$$y^{(i)} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2),$$

then the log-likelihood of the sample is, up to constants, proportional to $-\sum_i (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2$. Minimizing MSE is therefore equivalent to maximum likelihood under homoskedastic Gaussian observation noise. This is one reason why squared error is the natural benchmark loss for regression; see Bishop (2006); Goodfellow et al. (2016); Deisenroth et al. (2020).

For classification tasks, the model must output class probabilities. In the binary case ($K = 2$), the simplest approach passes a single scalar score z through the *sigmoid* function,

$$p = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1),$$

and assigns class 1 whenever $p > 0.5$, equivalently whenever $z > 0$ (Figure 1.7). For $K > 2$ classes the natural generalization maps a raw score vector $\mathbf{z} \in \mathbb{R}^K$ onto the probability simplex via *softmax*: $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$. In both cases the misfit between predicted probabilities and true labels is measured by the cross-entropy loss:

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}, \quad (1.2)$$

where $\hat{y}_k^{(i)}$ is the predicted probability that observation i belongs to class k . If the target is encoded as a one-hot vector, exactly one component of $(y_1^{(i)}, \dots, y_K^{(i)})$ equals one and all others are zero, so for an observation whose true class is c the loss contribution reduces to $-\log \hat{y}_c^{(i)}$. The model is rewarded for assigning high probability to the correct class and penalized heavily when that probability is near zero.

The origin of cross-entropy is again likelihood theory. In binary classification, with label $y \in \{0, 1\}$ and predicted success probability p , the Bernoulli log-likelihood is

$$\log L = y \log p + (1 - y) \log(1 - p).$$

Negating and averaging this expression gives the binary cross-entropy. The K -class formula above is the corresponding negative log-likelihood for a categorical distribution with probabilities generated by sigmoid ($K = 2$) or softmax ($K > 2$). Cross-entropy is therefore the statistically natural loss whenever the model output is meant to represent class probabilities; see again [Bishop \(2006\)](#); [Deisenroth et al. \(2020\)](#).

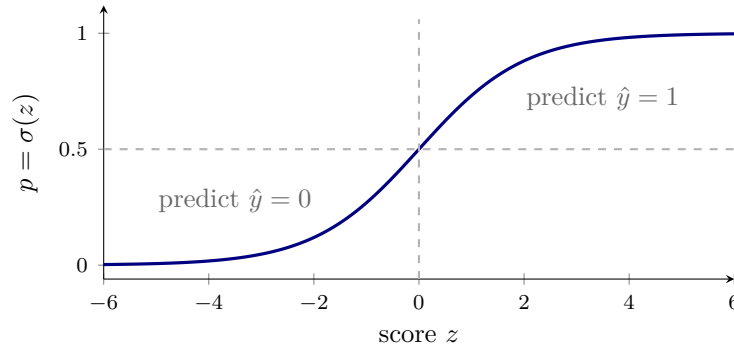


Figure 1.7: Binary classification with a sigmoid output. A scalar score z (the model’s raw output) is mapped to a probability $p = \sigma(z) \in (0, 1)$. The prediction rule assigns class 1 whenever $p > 0.5$, equivalently whenever $z > 0$. The dashed lines mark the decision threshold; no neural-network architecture is assumed—any model that produces a real-valued score can be combined with this mapping and the binary cross-entropy loss.

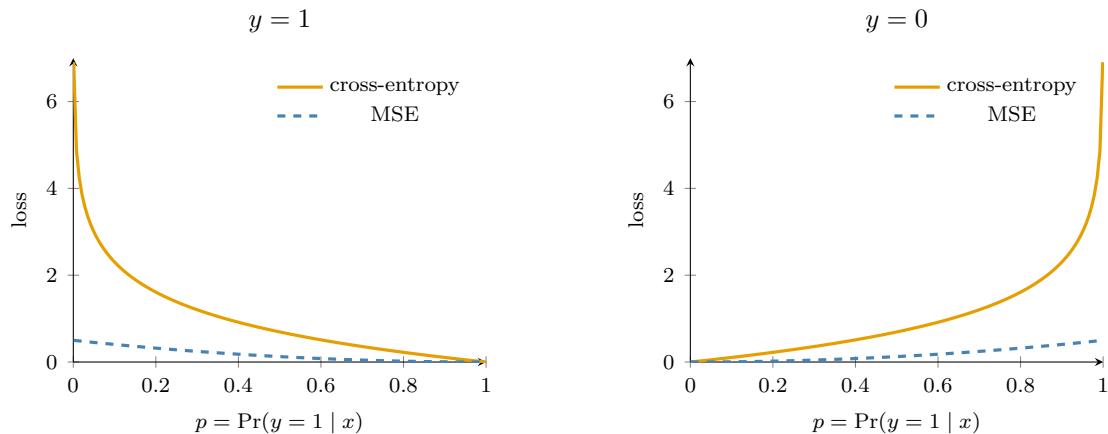


Figure 1.8: Binary cross-entropy and mean squared error as functions of the predicted class probability p . Cross-entropy rises much more sharply near confident mistakes, which is why it is usually better aligned with probabilistic classification.

Figure 1.8 makes the practical difference visible. If the true label is $y = 1$ but the model predicts a very small p , then the cross-entropy loss explodes because $-\log p \rightarrow \infty$ as $p \downarrow 0$. The

same holds symmetrically when $y = 0$ and $p \uparrow 1$. Mean squared error does penalize mistakes, but it does so much more mildly near the boundaries. For probabilistic classification, that weaker penalty is usually undesirable because it does not strongly discourage overconfident wrong predictions.

The optimization is performed via gradient descent or one of its stochastic variants, which we discuss in Section 1.6.

1.4.1 Beyond MSE: Robust and Asymmetric Losses

MSE is optimal under Gaussian noise, but real-world economic and financial data often contain outliers and heavy tails that inflate the squared penalty disproportionately. Two classical alternatives are useful in this course and beyond.

Huber loss. Introduced by [Huber \(1964\)](#) in the context of robust location estimation, the Huber loss behaves like MSE near the origin and like L_1 in the tails, capping the influence of any single observation:

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta) & \text{otherwise,} \end{cases} \quad r \equiv y - \hat{y}. \quad (1.3)$$

The threshold δ controls the transition and is typically chosen to be a few times the noise scale. Huber loss retains the smoothness needed for gradient-based optimization while reducing the weight of extreme residuals, which makes it the default choice for regression problems with suspected outliers.

Quantile (pinball) loss. [Koenker and Bassett \(1978\)](#) proposed the *check function*

$$\ell_\tau(r) = \max(\tau r, (\tau - 1)r), \quad \tau \in (0, 1), \quad (1.4)$$

whose minimizer is the conditional τ -quantile of y given $\mathbf{x}$ rather than the conditional mean. Setting $\tau = 0.5$ recovers the median (absolute-error) regression; setting $\tau = 0.05$ or $\tau = 0.95$ targets the lower or upper tail. In financial risk management this is precisely the statistic of interest: $\tau = 0.05$ yields a neural-network estimator of the lower-tail 5%-quantile of returns, which corresponds to Value-at-Risk (VaR) at the conventional 5% level, and averaging the pinball loss over many quantiles traces out the full conditional distribution of returns, an approach known as quantile regression or distributional regression.

Why this matters in economics and finance

Tail risk is often more important than average performance. The Huber and quantile losses let the network focus explicitly on robustness to outliers and on worst-case outcomes, respectively. A single quantile loss gives a Value-at-Risk estimator at the chosen probability level; Expected Shortfall requires additional structure, such as averaging lower-tail quantiles, fitting several quantiles jointly, or using a dedicated joint VaR–ES scoring rule. Notebook 07_Genz_Approximation_and_Loss_Functions compares MSE, Huber, and quantile losses on a common regression task.

1.5 From Perceptrons to Deep Networks

The building block of every neural network is the artificial neuron, first proposed by [McCulloch and Pitts \(1943\)](#). A companion question, how the synaptic weights w_i themselves should adapt with experience, was first addressed by [Hebb \(1949\)](#), whose rule “neurons that fire together, wire together” ($\Delta w_{ij} = \eta x_i y_j$) is the conceptual ancestor of all gradient-based learning rules

discussed below. [Rosenblatt \(1958\)](#) then introduced the Perceptron, the first trainable binary classifier built on these ideas. A single neuron computes a weighted linear combination of its inputs, adds a bias term, and passes the result through a nonlinear activation function:

$$\hat{y} = g(w_0 + \mathbf{x}^\top \mathbf{w}), \quad (1.5)$$

where $\mathbf{w} = (w_1, \dots, w_d)^\top$ are the synaptic weights, w_0 is the bias, and $g(\cdot)$ is the activation function (Figure 1.9).

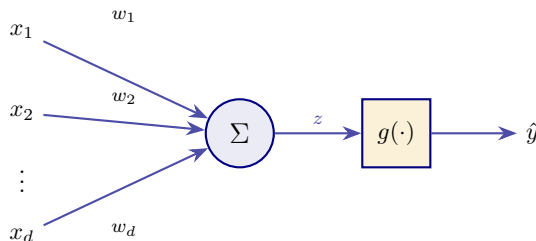


Figure 1.9: An artificial neuron in the McCulloch–Pitts lineage. Inputs x_i are multiplied by synaptic weights w_i , summed into a pre-activation z , and passed through a nonlinear activation $g(\cdot)$ to yield the output $\hat{y}$. The original [McCulloch and Pitts \(1943\)](#) unit used a binary threshold for g ; the modern artificial neuron generalizes this to arbitrary smooth activations, and all deep networks are compositions of neurons of this form.

Common choices for g include the sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$, the hyperbolic tangent $\tanh(z)$, and the rectified linear unit $\text{ReLU}(z) = \max(0, z)$ ([Nair and Hinton, 2010](#); [Glorot et al., 2011](#)). Without a nonlinear activation, any composition of linear layers collapses to a single affine map, a mathematical fact of fundamental importance: $\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2\mathbf{W}_1\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$.

From a single neuron to a layer. The single-neuron equation $\hat{y} = g(w_0 + \mathbf{x}^\top \mathbf{w})$ produces a scalar output. In practice we want vector-valued outputs (and, more importantly, vector-valued *intermediate features*). A *layer* of n parallel neurons, each with its own weights $\mathbf{w}^{(j)}$ and bias $w_0^{(j)}$, is a vector-valued generalization

$$\mathbf{a} = g(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{W} \in \mathbb{R}^{n \times d}, \mathbf{b} \in \mathbb{R}^n, \mathbf{a} \in \mathbb{R}^n,$$

where the nonlinearity g is applied componentwise. Each row of $\mathbf{W}$ is the weight vector of one neuron; stacking n of them gives the matrix $\mathbf{W}$ at once, so the layer evaluates n neurons in a single matrix–vector product.

From one layer to a deep composition. A single hidden layer is already a universal approximator ([Cybenko 1989](#); [Hornik et al. 1989](#); the universal-approximation theorem stated in the next subsection), but its hidden-layer width can grow exponentially in the input dimension to attain a target accuracy. Stacking layers on top of one another reuses earlier features as inputs to later neurons; the resulting compositional representation is dramatically more efficient for many functions of interest ([Telgarsky, 2016](#); [Barron, 1993](#)). A *deep feedforward network* with L layers is therefore a nested composition of layer maps:

$$f(\mathbf{x}; \boldsymbol{\theta}) = g_L(\mathbf{W}^{(L)} g_{L-1}(\dots g_1(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \dots + \mathbf{b}^{(L-1)}) + \mathbf{b}^{(L)}). \quad (1.6)$$

The architecture that implements (1.6) is sketched in Figure 1.10.

A useful geometric intuition, popularized by [Chollet \(2017\)](#), is that each layer of the network performs a nonlinear coordinate transformation, successively “untangling” the manifold on which the data lies. In the input space, the data may be entangled in complex ways (e.g., two

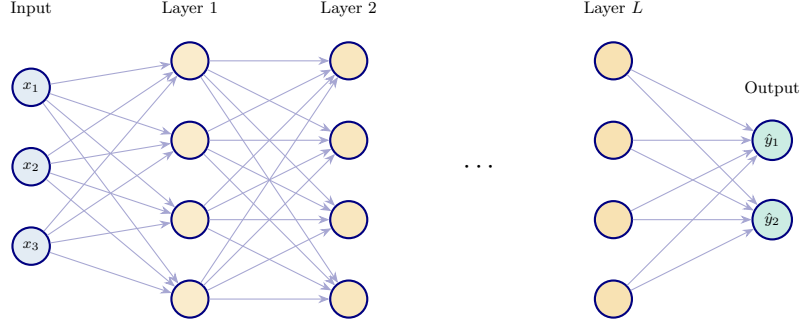


Figure 1.10: An L -layer deep feedforward network. Each layer applies an affine map followed by a pointwise nonlinearity; the composition realizes Eq. (1.6). Depth (rather than width) is what gives neural networks their efficient representational power for compositionally structured functions.

classes forming concentric spirals); each hidden layer warps the space so that the data become progressively more linearly separable. By the final hidden layer, a simple linear readout suffices. This perspective, formalized also by Goodfellow et al. (2016), explains why depth is so powerful: each layer adds an additional coordinate transformation, and the composition of many simple transformations can represent very complex mappings with far fewer parameters than a single, wide layer would require.

Other architectures. This course focuses almost entirely on feedforward networks of the form (1.6), because DEQNs and PINNs operate on unstructured state vectors $\mathbf{x} \in \mathbb{R}^d$ for which feedforward maps are the natural choice. For structured inputs other architecture families exist and are used widely elsewhere: convolutional networks (e.g., LeCun et al., 2015) for image data, graph neural networks for relational data, and Transformers (Section 1.11.4) for sequences. We mention them so that readers who encounter these models in the empirical-finance or applied-ML literatures know where they fit; none are required for the methods developed in later chapters.

The *universal approximation theorem* (Cybenko, 1989; Hornik et al., 1989) guarantees that even a single hidden layer with sufficiently many neurons can approximate any continuous function on a compact set to arbitrary precision. However, in practice, deep (multi-layer) networks achieve the same accuracy with exponentially fewer parameters than wide (single-layer) ones, which motivates the use of depth; Telgarsky (2016) makes this precise by exhibiting compositional functions that a depth- L network can represent in $\text{poly}(d, L)$ parameters but for which any depth- $(L-1)$ network requires width exponential in L . Barron (1993) provides classical dimension-independent approximation rates for Barron-class targets, often stated as squared L^2 error of order $\mathcal{O}(1/n_1)$ in the hidden width, whereas tensor-product methods for generic smooth functions scale poorly in the total number of grid nodes. This qualified comparison is the formal version of the “deep learning can beat grids” argument that motivates DEQNs in Chapter 2.

Universal Approximation Theorem

Let $g : \mathbb{R} \rightarrow \mathbb{R}$ be a bounded, non-constant, continuous activation function. For any continuous function $f : \mathcal{K} \rightarrow \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a single-hidden-layer network $F(\mathbf{x}) = \sum_{j=1}^{n_1} v_j g(\mathbf{w}_j^\top \mathbf{x} + b_j)$ such that $\sup_{\mathbf{x} \in \mathcal{K}} |F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$.

1.6 Training: Loss Functions, Gradient Descent, and Backpropagation

Given a loss function $J(\boldsymbol{\theta})$, training proceeds by iteratively updating the parameters in the direction of steepest descent:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \quad (1.7)$$

where $\eta > 0$ is the learning rate. In this introductory chapter $\boldsymbol{\theta}$ denotes generic trainable model parameters; in later chapters, when structural parameters and neural-network weights appear together, the script uses $\boldsymbol{\theta}$ for the structural parameters and $\boldsymbol{\rho}$ for the network weights. Computing the gradient $\nabla_{\boldsymbol{\theta}} J$ for a deep network is achieved through *backpropagation* (Rumelhart et al., 1986), an efficient application of the chain rule that propagates error signals from the output layer back to the input layer. Appendix B collects the matrix-calculus identities and the one-paragraph reverse-mode AD summary used throughout the script.

To see why the chain rule is central, consider a network with a single hidden layer. Let $\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$, $\mathbf{a} = g(\mathbf{z})$, and $\hat{y} = \mathbf{w}^{(2)\top}\mathbf{a} + b^{(2)}$ with loss $J = \frac{1}{2}(\hat{y} - y)^2$. Define the “delta” at the hidden layer:

$$\boldsymbol{\delta}^{(1)} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{a}} \cdot \frac{\partial \mathbf{a}}{\partial \mathbf{z}} = (\hat{y} - y) \mathbf{w}^{(2)} \odot g'(\mathbf{z}). \quad (1.8)$$

Then the weight gradient follows immediately:

$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top. \quad (1.9)$$

The key insight of Rumelhart et al. (1986) is that this chain rule application can be organized as a single backward pass through the network, reusing intermediate quantities (the $\boldsymbol{\delta}$ vectors) from the forward pass. The resulting algorithm has computational cost proportional to the forward pass; there is no need for finite differences or other expensive gradient approximations.

In practice, evaluating the full gradient over all m training examples is expensive. *Stochastic gradient descent* (SGD), whose roots go back to the stochastic-approximation scheme of Robbins and Monro (1951), replaces the full sum with a random mini-batch $\mathcal{B} \subset \{1, \dots, m\}$, yielding an unbiased estimate of the empirical-risk gradient at much lower cost per iteration:

$$\widehat{\nabla_{\boldsymbol{\theta}} J_{\mathcal{B}}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}). \quad (1.10)$$

With mini-batch sizes of 32–256, SGD achieves both computational efficiency and an implicit regularization effect from gradient noise. Two strands of theoretical work explain why this matters in deep nets specifically: the loss landscape of a deep network is dominated by saddle points rather than isolated bad local minima (Dauphin et al., 2014), and SGD’s gradient noise tends to bias training toward *flat* rather than *sharp* regions of the loss surface, which often generalize better (Keskar et al., 2017). Even on linearly separable data, gradient descent on the logistic loss converges to the maximum-margin solution, an instance of the broader principle that the optimizer itself imposes an *implicit bias* that contributes to generalization (Soudry et al., 2018). For a comprehensive modern review of stochastic optimization for large-scale learning (including convergence rates, adaptive methods, and variance reduction), see Bottou et al. (2018).

1.6.1 The Adam Optimizer

Modern optimizers such as Adam (Kingma and Ba, 2015) adapt the learning rate for each parameter based on running averages of the first and second moments of the gradient:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla J_t, \quad (1.11)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla J_t)^2, \quad (1.12)$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t), \quad (1.13)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon). \quad (1.14)$$

The bias-corrected first moment $\hat{\mathbf{m}}_t$ provides momentum (smoothing out gradient noise), while the second moment $\hat{\mathbf{v}}_t$ provides per-parameter adaptive learning rates (parameters with large gradients receive smaller effective steps). The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ work well across a wide range of problems, including all the economic applications in this course.

1.6.2 The Optimizer Family Tree: Momentum, RMSprop, Adam, AdamW

Adam did not appear out of thin air; it inherits from a family of refinements to plain SGD whose interactions are worth being explicit about for readers who will tune optimizers in practice. Table 1.1 traces the lineage; each row is a one-line modification of the row above it.

Optimizer	Update rule (one parameter)	Reference
SGD	$\theta \leftarrow \theta - \eta g_t$	Robbins and Monro (1951)
SGD + momentum	$v_t = \mu v_{t-1} + g_t$; $\theta \leftarrow \theta - \eta v_t$	Sutskever et al. (2013)
RMSprop	$s_t = \rho s_{t-1} + (1 - \rho) g_t^2$; $\theta \leftarrow \theta - \eta g_t / \sqrt{s_t + \varepsilon}$	Tieleman and Hinton (2012)
Adam	momentum on g_t and on g_t^2 with bias correction (Eqs. above)	Kingma and Ba (2015)
AdamW	Adam plus <i>decoupled</i> weight decay $\theta \leftarrow (1 - \eta\lambda)\theta - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon)$	Loshchilov and Hutter (2019)

Table 1.1: Lineage from plain SGD to AdamW. Each row introduces exactly one new ingredient: momentum buffers gradient noise; RMSprop adds a per-parameter learning-rate scaling by the running second moment; Adam combines the two with bias correction; AdamW separates weight decay from the gradient step so that the implicit L_2 regularizer does not interact with the adaptive denominator. PINN training in continuous-time chapters (Chapters 7–8) often uses Adam or AdamW; DEQNs in Chapters 2–6 use plain Adam with the default $(\beta_1, \beta_2) = (0.9, 0.999)$ as in Azinovic et al. (2022).

The Adam-vs-AdamW distinction is sharper than the one-line table entry suggests, so it is worth writing out both rules side by side. With $\hat{\mathbf{m}}_t$, $\hat{\mathbf{v}}_t$ the bias-corrected first and second moment of the gradient and λ the weight-decay rate, Adam-with- L_2 (i.e. Adam applied to the loss $J + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$) updates

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t + \lambda \boldsymbol{\theta}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon},$$

so the implicit regularizer is itself rescaled by the adaptive denominator $\sqrt{\hat{\mathbf{v}}_t} + \varepsilon$. AdamW separates the two:

$$\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda) \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon},$$

so the weight-decay term shrinks every parameter by the same proportional factor regardless of gradient magnitude. This is why AdamW recovers the textbook intuition “weight decay shrinks weights uniformly” that Adam-with- L_2 loses.

Figure 1.11 gives a schematic comparison of the qualitative convergence patterns behind this optimizer family tree.

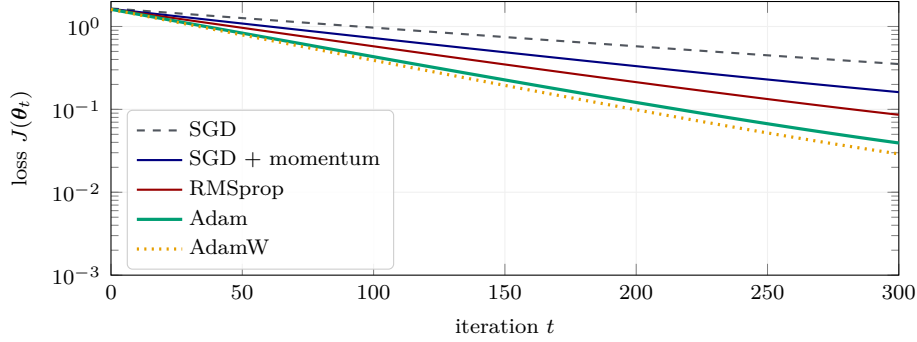


Figure 1.11: Schematic loss-trajectory comparison on a moderately ill-conditioned objective. Momentum and adaptive rescaling often improve early convergence relative to plain SGD, and Adam/AdamW are therefore useful defaults in the notebooks. The ranking is illustrative rather than universal: on some objectives, carefully tuned SGD or RMSprop can match or beat Adam-family methods.

1.6.3 Learning Rate Schedules

The choice of learning rate η is arguably the single most important hyperparameter in deep learning. Too large, and the optimizer diverges; too small, and convergence is impractically slow. A popular schedule is *cosine annealing* (Loshchilov and Hutter, 2017), which smoothly decays the learning rate according to:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T)), \quad (1.15)$$

where T is the total number of training iterations. Figure 1.12 compares the three learning-rate strategies used most often in practice.

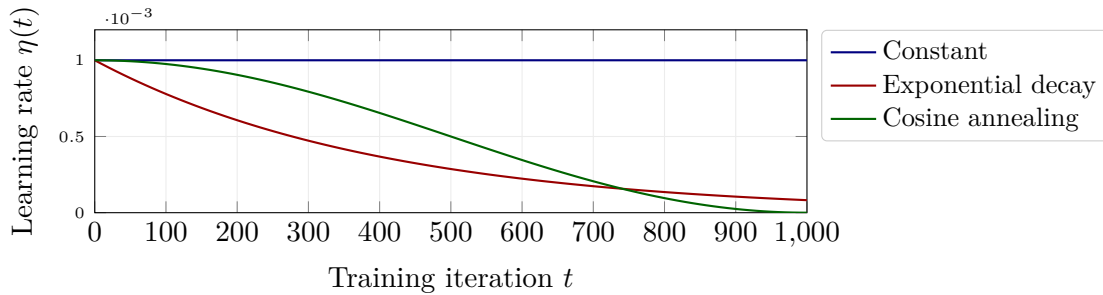


Figure 1.12: Three common learning-rate schedules. A constant rate is simple but often converges slowly in the fine-tuning phase; exponential decay shrinks monotonically; cosine annealing (Loshchilov and Hutter, 2017) provides a smooth warm-to-cold transition that empirically performs well across a wide range of problems. DEQNs and PINNs typically use exponential decay or cosine annealing to polish the solution after the initial coarse-grained phase.

In practice, decaying schedules such as exponential decay or cosine annealing tend to refine solutions in the later stages of training, once the optimizer has found a good basin of attraction.

Loss function landscape

The loss surface of a deep network is high-dimensional and non-convex, containing saddle points, plateaus, and sharp minima. Stochastic optimization methods navigate this landscape effectively because the noise from mini-batch sampling helps escape shallow local minima and saddle points. In economic applications, the loss function has direct economic interpretation, whether an Euler equation residual, a PDE residual, or a moment matching criterion, which provides a natural metric for assessing convergence quality.

1.6.4 Backpropagation: The Chain Rule at Scale

For a network with L layers, denote the pre-activation at layer l as $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ and the activation as $\mathbf{a}^{(l)} = g(\mathbf{z}^{(l)})$. If the final layer is linear, set $g'(z) = 1$ at $l = L$ and interpret $\mathbf{a}^{(L)}$ as the prediction $\hat{y}$. The backpropagation algorithm computes $\partial J / \partial \mathbf{W}^{(l)}$ for all layers simultaneously by propagating a “delta” vector backward:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} J \odot g'(\mathbf{z}^{(L)}), \quad (1.16)$$

$$\boldsymbol{\delta}^{(l)} = ((\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}) \odot g'(\mathbf{z}^{(l)}), \quad l = L - 1, \dots, 1, \quad (1.17)$$

where $\odot$ denotes element-wise multiplication. The parameter gradients are then $\partial J / \partial \mathbf{W}^{(l)} = \boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^\top$ and $\partial J / \partial \mathbf{b}^{(l)} = \boldsymbol{\delta}^{(l)}$. The computational cost is linear in the number of layers and the total number of parameters, a remarkable efficiency that enables training networks with millions of parameters. Figure 1.13 shows the forward and backward passes side by side.

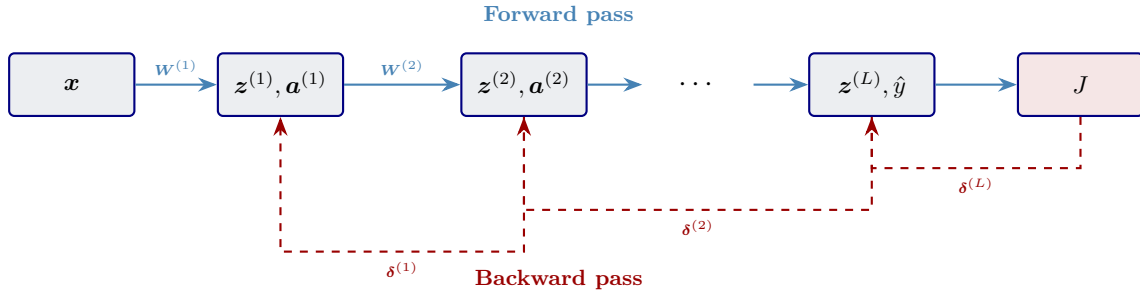


Figure 1.13: Backpropagation as forward and backward passes through the network. The forward pass (blue) produces and caches every layer’s pre-activations $\mathbf{z}^{(l)}$ and activations $\mathbf{a}^{(l)}$; the backward pass (red, dashed) propagates the “delta” vectors $\boldsymbol{\delta}^{(l)}$ from the loss back to the inputs, reusing the cached quantities to compute all parameter gradients in a single sweep.

In modern deep learning frameworks such as TensorFlow and PyTorch, backpropagation is implemented automatically through computational graph tracing (“autograd”). The user only needs to define the forward computation; the framework handles the differentiation. This same automatic differentiation capability is what makes PINNs (Chapter 7) possible: the PDE residual requires derivatives of the network output with respect to its inputs, which autograd provides exactly and efficiently.

1.7 Weight Initialization

The initialization of network weights has a profound effect on training dynamics. If weights are initialized too large, activations explode through the layers; if too small, they vanish to zero. In both cases, the gradient signal degrades and training stalls. The key principle is to choose initial weights so that the variance of activations remains approximately constant across layers.

Xavier/Glorot initialization. Glorot and Bengio (2010) derived the following rule for networks with symmetric activations (such as tanh). For a layer with n_{in} input neurons and n_{out} output neurons, initialize weights as:

$$W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right] \quad \text{or equivalently} \quad W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right). \quad (1.18)$$

This ensures $\text{Var}[a^{(l)}] \approx \text{Var}[a^{(l-1)}]$ under the assumption that activations are in the linear regime of tanh.

He initialization. For ReLU activations, He et al. (2015) showed that the weight variance should be doubled relative to the forward fan-in rule:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right). \quad (1.19)$$

The justification is a *second-moment-preserving* calculation, not a variance one. For a centered symmetric pre-activation z ,

$$\mathbb{E}[\text{ReLU}(z)^2] = \int_0^\infty z^2 p(z) dz = \frac{1}{2} \mathbb{E}[z^2],$$

because $p(z)$ is symmetric about zero, so the negative half of the integrand is killed and the positive half is preserved. Doubling the input weight variance therefore preserves the second moment $\mathbb{E}[(z^{(l)})^2]$ across layers under ReLU. Strictly speaking $\mathbb{E}[\text{ReLU}(z)] > 0$, so the *variance* (centered second moment) is slightly smaller than the second moment, and the factor of 2 is an approximation rather than an identity; in practice the approximation is excellent and He initialization is the default for ReLU-family networks throughout this course.

Practical guidance

The applications in this course use different activations depending on the task: **(i)** the introductory DEQN examples (Brock–Mirman, Chapter 2, §2.4) use *ReLU* for its simplicity and fast training; **(ii)** the multi-country IRBC model (Chapter 3) and the deep surrogate (Chapter 10) use *Swish* ($z\sigma(z)$) for its smooth gradients; **(iii)** all PINN examples (Chapter 7) use tanh, whose C^∞ smoothness is essential when the loss involves second-order derivatives. For ReLU-family networks, He initialization (`kaiming_normal_` in PyTorch, `he_normal` in Keras) is the natural default. For tanh and sigmoid networks, Xavier/Glorot initialization is usually the cleaner starting point; Swish and related smooth non-monotone activations often work well with either He-style or Xavier-style scaling, so the notebooks state the chosen initializer explicitly when it matters.

1.8 Activation Functions in Depth

Beyond the three classical choices (sigmoid, tanh, ReLU), several modern activation functions address specific shortcomings. Table 1.2 summarizes the options used in this course.

Leaky ReLU and ELU address the dying-neuron issue by providing a small but nonzero gradient for negative inputs. The Swish activation $\text{swish}(z) = z\sigma(z)$ (Ramachandran et al., 2017), which is used extensively in the DEQN and IRBC implementations of this course, combines the benefits of ReLU (non-saturating for large z) with smoothness at the origin. Its derivative $\text{swish}'(z) = \sigma(z) + z\sigma(z)(1 - \sigma(z))$ is smooth everywhere and bounded between approximately -0.1 and 1.1 , which can improve optimization stability.

Activation	Formula	Range	Key property
Sigmoid	$\sigma(z) = (1 + e^{-z})^{-1}$	$(0, 1)$	Smooth, saturates
Tanh	$\tanh(z)$	$(-1, 1)$	Zero-centered, saturates
ReLU	$\max(0, z)$	$[0, \infty)$	Non-saturating for $z > 0$
Leaky ReLU	$\max(\alpha z, z), \alpha=0.1$	$(-\infty, \infty)$	No dead neurons
ELU	z if $z > 0$; $\alpha(e^z - 1)$ else	$[-\alpha, \infty)$	Negative saturation; C^1 if $\alpha = 1$
Swish	$z \cdot \sigma(z)$	$\approx [-0.28, \infty)$	Smooth, non-monotone
GELU	$z \cdot \Phi(z)$	$\approx [-0.17, \infty)$	Smooth, default in BERT / GPT
Mish	$z \cdot \tanh(\ln(1 + e^z))$	$\approx [-0.31, \infty)$	Smooth, used in YOLOv4
Softplus	$\ln(1 + e^z)$	$(0, \infty)$	Smooth ReLU approximation

Table 1.2: Activation functions used throughout the course. Origin papers: ReLU (Nair and Hinton, 2010), Leaky ReLU (Maas et al., 2013), ELU (Clevert et al., 2016), Swish (Ramachandran et al., 2017), GELU (Hendrycks and Gimpel, 2016), Mish (Misra, 2019). *Range* is the set of output values for $z \in \mathbb{R}$. *Smoothness* matters when derivatives of the network output are needed: sigmoid, tanh, Swish, GELU, Mish, and softplus are C^∞ ; ReLU is only C^0 ; Leaky ReLU is piecewise linear; ELU is piecewise C^∞ and is C^1 at the origin only for $\alpha = 1$. Smooth activations are required for PINN applications that involve second-order derivatives (Chapter 7).

For PDE applications (Chapter 7), the choice of activation function is particularly important because the PINN loss involves derivatives of the network output. Since $\text{ReLU}''(z) = 0$ almost everywhere, a ReLU network cannot represent second-order PDE residuals faithfully. Smooth activations such as tanh (C^∞) or Swish are therefore required for PINN applications involving second-order PDEs. Figure 1.14 plots seven representative activations from Table 1.2.

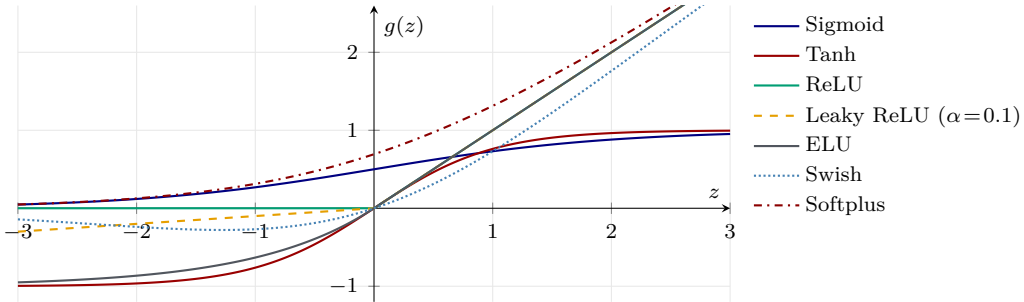


Figure 1.14: Seven representative activation functions from Table 1.2. Sigmoid and tanh saturate at large $|z|$ (vanishing gradients); ReLU is non-saturating but kinked at the origin; Leaky ReLU and ELU repair the dead-neuron problem with a small negative response; Swish and Softplus are everywhere C^∞ , which the PINN chapter (Chapter 7) requires.

1.9 Vanishing and Exploding Gradients

A central obstacle to training deep networks is that the gradient signal reaching early layers can become either astronomically small or astronomically large as it is back-propagated through many layers. The backward recursion derived above for the “delta” vector is

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}) \odot g'(\mathbf{z}^{(l)}), \quad (1.20)$$

so the magnitude of $\delta^{(l)}$ is governed, roughly, by the product of derivatives $\prod_{k=l}^L g'(\mathbf{z}^{(k)})$ and the norms $\|\mathbf{W}^{(k)}\|$. Two symmetric failure modes follow:

- **Vanishing gradients.** For the sigmoid activation, $|g'(z)| \leq 1/4$; for tanh, $|g'(z)| \leq 1$; and both derivatives approach zero when $|z|$ is large. In the worst sigmoid case each layer

shrinks the gradient by a factor close to $1/4$, so after $L = 10$ layers the signal at the first layer can be attenuated by roughly $(1/4)^{10} \approx 10^{-6}$. Early layers stop learning.

- **Exploding gradients.** If $|g'(z)| > 1$ or if $\|\mathbf{W}^{(k)}\|$ is large, the product grows instead of shrinking. Updates become huge, parameters diverge, and the loss “blows up”.

Three ingredients, each already introduced separately, combine to tame these problems:

1. **Non-saturating activations.** ReLU has $g'(z) = 1$ for $z > 0$, eliminating the $(1/4)^L$ decay; Swish and tanh avoid vanishing when activations remain in a moderate range.
2. **Variance-preserving initialization.** Xavier/Glorot (Glorot and Bengio, 2010) and He (He et al., 2015) pick $\text{Var}[W] \propto 1/n_{\text{in}}$ precisely so that $\text{Var}[\mathbf{z}^{(l)}]$ is constant across layers, keeping activations in the useful range of g (Section 1.7).
3. **Batch normalization** (Ioffe and Szegedy, 2015). Re-centering and re-scaling the pre-activations of each mini-batch prevents them from drifting toward the saturated tails of g during training and allows much larger learning rates. Its affine parameters (γ, β) are learned.

A practical complement is *gradient clipping*: if $\|\nabla_{\theta} J\|$ exceeds a threshold, rescale it to the threshold. This eliminates the most damaging exploding-gradient events at negligible cost and is standard in RNN training; it is occasionally useful in DEQNs and PINNs when the residual magnitudes are highly unbalanced across collocation points.

Course-wide implication

Everywhere in this course where we train a network of non-trivial depth (Chapters 2–7), the combination of *He/Xavier initialization*, a *smooth non-saturating activation* (ReLU, Swish, tanh), and Adam’s *per-parameter adaptive step* keeps the gradient flow well conditioned. Batch normalization is used when depth exceeds roughly ten layers or when the input distribution shifts substantially during training.

1.9.1 Batch Normalization

Among the three mitigations listed above, batch normalization (Ioffe and Szegedy, 2015) (BN) deserves a closer look because it is simple to state, surprisingly effective in practice, and has become a default building block in supervised deep networks. At its core BN is the standardization trick familiar from regression, re-centering each variable to mean zero and rescaling it to unit variance, applied separately to every layer’s pre-activations and recomputed on every mini-batch of training data.

Let $z_1, \dots, z_B$ denote the pre-activations at one neuron over the B examples in a mini-batch $\mathcal{B}$. Batch normalization replaces them by

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^B z_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_{\mathcal{B}})^2, \quad \hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{z}_i + \beta, \quad (1.21)$$

where ε is a small constant for numerical stability and (γ, β) are *learnable* scalar parameters specific to that neuron. The transformed activation y_i is what the next layer sees. In the standard recipe BN is inserted between the linear map $\mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ and the elementwise nonlinearity $g(\cdot)$.

Why standardization, layer by layer. Without BN, the input distribution to a hidden layer ℓ depends on every weight in layers $1, \dots, \ell - 1$. As earlier weights update during gradient descent, the distribution faced by layer ℓ *drifts* from one optimization step to the next: each layer therefore chases a moving target, a phenomenon Ioffe and Szegedy (2015) called *internal covariate shift*. BN pins the input distribution of every layer to mean zero and unit variance at every step (Figure 1.15). Gradients become better conditioned, and substantially larger learning rates become safe.

The role of the affine parameters. At first glance the learnable shift and scale (γ, β) seem to undo the normalization that BN just imposed. This is exactly the point. If a layer happens to prefer non-standard inputs, for example a tanh layer that needs slightly negative pre-activations to operate in its linear regime, the network is free to recover them via (γ, β) . BN therefore never reduces the network’s representational capacity; it merely shifts to a parameterization in which the optimization trajectory is easier to follow.

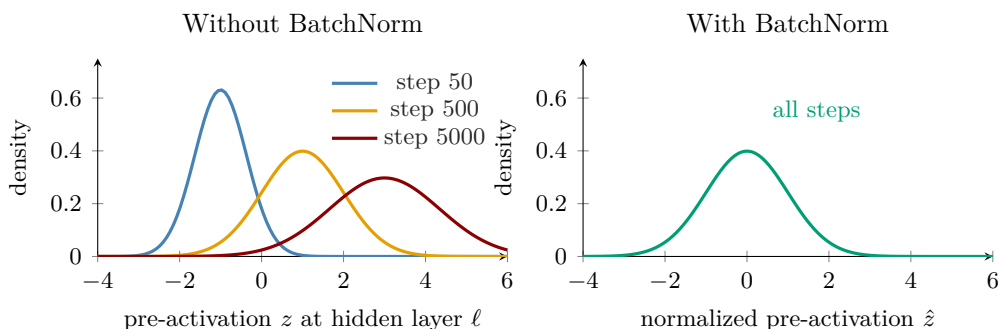


Figure 1.15: Distribution of pre-activations at one hidden neuron, sampled at three points during training. *Left:* without BatchNorm, the distribution drifts in mean and in variance as earlier layers update, each layer chases a moving target. *Right:* with BatchNorm, the affine pre-normalization transformation pins the inputs to $\mathcal{N}(0, 1)$ at every training step, before the learned scale γ and shift β are applied. The downstream layer always operates on inputs of the same scale, and the gradient signal flowing back is well conditioned.

Why higher learning rates work. A precise Lipschitz bound depends on the operator norms of the surrounding weight matrices, the activation derivatives, and the learned affine scale γ . The useful intuition is that BN reduces sensitivity to shifts and rescalings of intermediate activations, making the local optimization problem better conditioned and allowing step sizes that would otherwise cause divergence. Santurkar et al. (2018) argue that loss-landscape smoothing, rather than the original “internal covariate shift” interpretation, better explains why BN helps optimization; the two views are complementary, but the smoothing perspective is the more directly testable one.

At inference time. During training, BN uses the current mini-batch to compute μ_B, σ_B^2 . At inference the mini-batch may be a single example, in which case those statistics would be ill-defined. Implementations therefore maintain a running average of μ and σ^2 across training mini-batches and use these fixed estimates at test time, so the network’s output is deterministic at deployment.

1.9.2 Normalization Variants Beyond BatchNorm

Batch normalization is the original normalization trick, but for several common deep-learning settings, small mini-batches, recurrent / sequence models, generative models, transformers, it is

not the right one. All variants share the form $\hat{z} = (z - \mu)/\sigma$ followed by a learned affine $\gamma\hat{z} + \beta$; they differ only in *which axes* the statistics (μ, σ) are pooled over:

- **LayerNorm** (Ba et al., 2016): pool across all features of a single example. Decouples training from batch size and is the de-facto choice for RNNs and transformers; it can also be useful in small-batch residual methods, though PINNs more commonly rely on careful input/output scaling and smooth activations.
- **GroupNorm** (Wu and He, 2018): pool over a group of channels of a single example. Interpolates between LayerNorm ($G=1$) and InstanceNorm ($G = \text{channels}$); the default in object detection and small-batch CNNs.
- **WeightNorm** (Salimans and Kingma, 2016): reparameterize $\mathbf{w} = (g/\|\mathbf{v}\|)\mathbf{v}$ so the activation statistics never enter the gradient. Avoids any batch-size dependence at the cost of slightly less representation power.

For the methods in this script, the practical guidance is: use BatchNorm for large supervised datasets (Chapter 1), LayerNorm for recurrent or attention-based architectures and as an option when the effective batch size is small, and skip normalization entirely for small DEQN or PINN MLPs when input/output scaling plus Adam already condition the problem well.

1.10 Generalization: Overfitting, Regularization, and Double Descent

1.10.1 Train / Validation / Test Split

Before discussing overfitting formally, it is essential to fix the experimental protocol that every supervised-learning study should follow. The available data are partitioned into three disjoint subsets:

- **Training set** (typically $\sim 70\%$): used to fit the model parameters θ by minimizing the loss.
- **Validation set** (typically $\sim 20\%$): used for *model selection*, choosing hyperparameters, comparing architectures, deciding when to stop training. The model’s performance on this set guides these decisions but its parameters are never trained on it.
- **Test set** (typically $\sim 10\%$): touched *once*, at the very end, to report an unbiased estimate of generalization performance.

The key discipline is that no decision about the model (not hyperparameter tuning, not architecture, not early-stopping patience) may be informed by the test set. Using the test set multiple times turns it into an implicit validation set and invalidates its role as a measure of out-of-sample error. For small datasets, k -fold cross-validation replaces the fixed train/validation split: the training data are partitioned into k equal folds; for each fold, the model is trained on the other $k - 1$ folds and evaluated on the held-out fold; the k resulting validation scores are averaged. Common choices are $k = 5$ or $k = 10$; the test set is always held separately. In DEQNs and PINNs (Chapters 2 and 7), training and validation points are drawn from the same state distribution, and “generalization” is measured against an *independently simulated test trajectory* rather than a held-out labeled set.

A model that memorizes the training data but fails on unseen examples is said to *overfit*. To understand overfitting precisely, consider the following thought experiment. The decomposition below is the classical bias/variance analysis of Geman et al. (1992), which provided the canonical framework for thinking about generalization in neural networks long before modern

overparameterized regimes were studied. Suppose we draw many independent training sets $\mathcal{D}$, each of size n , from the same data-generating process $y = f(\mathbf{x}) + \varepsilon$, where ε is zero-mean noise with variance σ^2 . On each training set we fit our model, obtaining a predictor $\hat{f}_{\mathcal{D}}$. Conditioning on a fixed test input $\mathbf{x}_0$ and averaging over both the training set and the new test noise ε_0 , the squared prediction error decomposes into exactly three terms:

$$\mathbb{E}_{\mathcal{D}, \varepsilon_0}[(y_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0))^2] = \underbrace{(f(\mathbf{x}_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}. \quad (1.22)$$

Each term captures a distinct source of error:

- **Bias²** measures the systematic error: how far the *average* prediction $\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}_0)]$ is from the true function $f(\mathbf{x}_0)$. A model that is too simple (e.g., a linear function fit to a nonlinear target) will have high bias regardless of how much data it sees.
- **Variance** measures the sensitivity of the prediction to the particular training set drawn. A highly flexible model (e.g., a large neural network) may fit each training set well, but the predictions can differ wildly across draws, which is overfitting.
- **Irreducible noise σ^2** is the error that no model can eliminate, because it stems from randomness in the data-generating process itself.

In classical statistics, there is a fundamental trade-off: reducing bias requires more flexible models, which increases variance. However, modern deep networks challenge this picture. Zhang et al. (2017) showed empirically that standard architectures can perfectly fit randomly labeled data, implying a VC-style capacity far beyond what classical bounds predict, yet still generalize well on real data. Belkin et al. (2019) subsequently demonstrated that with sufficient overparameterization, models exhibit a *double descent* phenomenon: test error first increases as the model becomes more complex (classical regime), but then decreases again as the number of parameters greatly exceeds the number of data points (interpolation regime). Nakkiran et al. (2020) showed that this phenomenon extends to deep networks and occurs not only as a function of model size, but also as a function of training time (“epoch-wise double descent”) and dataset size.

The key techniques for preventing overfitting in neural networks are:

1. **Early stopping** (Prechelt, 1998): monitor validation loss and stop training when it begins to rise.
2. **Weight decay (L_2 regularization)** (Krogh and Hertz, 1991): add $\frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$ to the loss.
3. **Dropout** (Srivastava et al., 2014): randomly drop a fraction p of activations at every training step. Two implementation conventions exist. The original convention drops units during training and multiplies the outgoing weights or activations by the keep probability $1-p$ at test time, so that the expected activation matches the training-time expectation. The now-standard *inverted-dropout* convention divides the retained activations by $1-p$ during training, so no rescaling is needed at test time. Either way, the mechanism is equivalent to training, on each mini-batch, a different sub-network drawn from an exponentially large ensemble that shares weights; the final network approximates the ensemble average. Typical values are $p = 0.5$ for hidden layers and $p = 0.1$ – 0.2 for inputs. Dropout is less commonly used in DEQN and PINN applications, where the loss is already noisy (stochastic collocation) and regularization is often supplied implicitly by the state-space sampling scheme.
4. **Data augmentation**: synthetically enlarge the training set via transformations.

5. **Batch normalization** (Ioffe and Szegedy, 2015): normalize activations within each mini-batch to stabilize training; its mini-batch statistics also act as a mild regularizer.

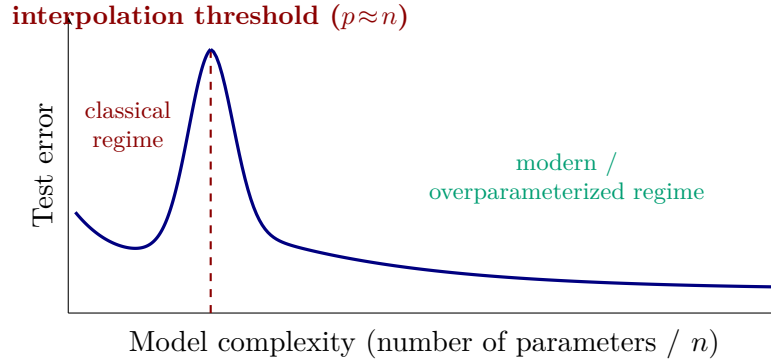


Figure 1.16: Schematic of the double-descent phenomenon. In the classical regime ($p < n$) test error follows the standard bias–variance U-curve; around the interpolation threshold $p \approx n$ test error can peak sharply because the fitted function is highly sensitive to noise; in the modern overparameterized regime ($p \gg n$) test error decreases again (Belkin et al., 2019; Nakkiran et al., 2020). In some linearized, kernel, max-margin, or least-norm settings, gradient methods exhibit an implicit bias toward particular low-complexity interpolants; in nonlinear finite-width networks this bias depends on architecture, data, optimizer, initialization, and training protocol. Axes are unitless; the qualitative shape, not the scale, is the point. The curve is illustrative, not a measurement.

Figure 1.16 illustrates why classical bias–variance intuition breaks down for modern deep networks. In the classical regime ($p < n$), increasing model capacity beyond a point leads to overfitting. At the interpolation threshold ($p \approx n$), the model has just enough parameters to perfectly fit the training data, and the resulting solution can be extremely sensitive to noise. In the modern regime ($p \gg n$), test error often decreases again because optimization and architecture bias select comparatively regular interpolating solutions rather than arbitrary ones (Belkin et al., 2019).

This phenomenon has been documented across many architectures and datasets by Nakkiran et al. (2020), who showed that it persists even when controlling for effective model complexity. The implications for computational economics are substantial but should not be overstated. In DEQN and PINN applications, the practitioner controls both the network size (number of parameters p) and the amount of training data (number of collocation points n), and those collocation points are often resampled rather than fixed once and for all. Overparameterized networks can therefore be useful and sometimes necessary, but their credibility must be checked by independent residual diagnostics, simulated trajectories, and benchmark comparisons rather than by parameter counting alone.

The double descent phenomenon can be explored interactively in the companion notebook `03_Double_Descent.ipynb`, which reproduces the spirit of the double-descent curve in a small kernel-regression / random-Fourier-features setting (Belkin et al. 2019); see also Nakkiran et al. (2020) for the deep-network / CIFAR-10 version of the experiment, which the notebook does *not* attempt to replicate at scale.

1.10.2 A Pointer to the Theory: Neural Tangent Kernel (NTK) and Benign Overfitting

Why *does* an over-parameterized network with $p \gg n$ generalize instead of merely memorizing? Two complementary lines of theory have emerged.

Neural Tangent Kernel (NTK). In the limit of infinite width and a particular initialization scaling, gradient-descent training of a deep network is described by kernel gradient descent with a fixed, deterministic kernel, the *Neural Tangent Kernel* of [Jacot et al. \(2018\)](#). In that lazy-training limit the network’s function evolves almost linearly around its initialization, which explains why first-order optimization can be well behaved for very wide networks even though the finite-parameter objective is non-convex.

Benign overfitting. In linear and kernel settings, gradient methods often select a minimum-norm interpolating solution, which can behave much like a ridge-regularized least-squares estimator and inherit good generalization properties. [Bartlett et al. \(2020\)](#) make this precise for linear regression, showing that interpolation can be benign provided the spectrum of the input covariance has heavy enough tails. Subsequent work has extended both stories beyond the simplest settings; for the practitioner the takeaway is that the NTK regime helps explain *why training succeeds*, while benign-overfitting theory explains why interpolation need not imply poor test error under additional assumptions.

These two threads are not the final word: finite-width deviations from the NTK matter for feature learning, and benign overfitting requires conditions on the data covariance. They nevertheless help explain why the rest of this script can use networks substantially wider than a classical degrees-of-freedom calculation would recommend, provided the numerical residuals are validated out of sample.

1.11 Sequence Models: RNNs, LSTMs, and Attention

Optional section

This section and the in-context AR(1) aside (§1.11.5) survey sequence architectures that the rest of the script does not use: Chapters 2–11 operate on unstructured state vectors and rely entirely on feedforward MLPs. Readers focused on DEQN, PINN, and the structural-estimation chapters can skip directly to the Chapter Summary (page 48) without loss of continuity. The material below is included for completeness and as a reference for readers who later encounter Transformers in empirical-finance or applied-ML work.

The chapters that follow rely almost entirely on feedforward networks, but many economic and financial datasets are intrinsically sequential. Before closing this introduction, it is therefore useful to briefly survey the main architectures for sequence data. We represent a sequence as *tokens* $\mathbf{x}_1, \dots, \mathbf{x}_n$, where each token is a vector. The word “token” is borrowed from natural-language processing, where a token is typically a word or sub-word piece; in the economic and financial context we use throughout this course a token is simply one element of the sequence, for example a scalar return, a vector of macroeconomic variables at one quarter, or a price-volume pair at one tick. The algorithms below are agnostic to this choice; they only need the sequence elements to be real-valued vectors of a fixed dimension.

1.11.1 Recurrent Neural Networks

The traditional approach to sequence data is the *Recurrent Neural Network* (RNN) ([Elman, 1990](#)). Unlike an MLP, which maps an input to an output in a single forward pass, an RNN maintains an internal *hidden state* $\mathbf{h}_t$ that acts as a memory of past information. At each time step t , the network updates this state using the current input $\mathbf{x}_t$ and the previous state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}), \quad (1.23)$$

where σ is an activation function. Concretely: for a scalar time series $\mathbf{x}_t$ is a scalar (e.g. log-return at date t), $\mathbf{h}_t \in \mathbb{R}^d$ is a d -dimensional hidden vector summarizing everything the network has

seen so far, and $\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}$ are learnable parameters. The same update is applied at every time step, so this recursive structure lets the network process sequences of arbitrary length with a fixed parameter budget. Figure 1.17 shows the resulting unrolled computation graph.

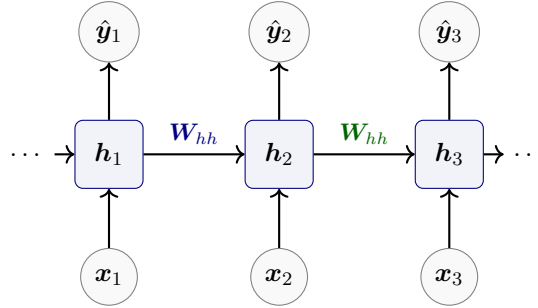


Figure 1.17: An unrolled Recurrent Neural Network. The same parameters $\mathbf{W}_{hh}, \mathbf{W}_{xh}$ are reused at every time step, allowing the hidden state $\mathbf{h}_t$ to accumulate historical information.

Training: Backpropagation Through Time (BPTT). Because the unrolled RNN *is* a feedforward graph of depth T with *shared* weights, one can apply ordinary backpropagation and then *sum* the weight-gradients across time. Concretely, let $\mathcal{L}_T = \sum_{t=1}^T \ell(\hat{\mathbf{y}}_t, \mathbf{y}_t)$ denote the total loss. For the recurrence above, the forward single-step Jacobian is

$$\mathbf{J}_t \equiv \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\sigma'(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}))\mathbf{W}_{hh}.$$

With column gradients, the backward pass multiplies by $\mathbf{J}_t^\top$. Differentiating with respect to an early hidden state $\mathbf{h}_k$ therefore yields the schematic product

$$\nabla_{\mathbf{h}_k} \mathcal{L}_T \approx \mathbf{J}_{k+1}^\top \mathbf{J}_{k+2}^\top \cdots \mathbf{J}_T^\top \nabla_{\mathbf{h}_T} \mathcal{L}_T, \quad (1.24)$$

so the gradient picks up $T - k$ products of matrices containing the same recurrent weight matrix $\mathbf{W}_{hh}$. The relevant singular values or spectral radii of these factors determine the asymptotics. If they are mostly below one, the gradient *vanishes* exponentially in $T - k$ and the network cannot learn dependencies that span many steps; if they are mostly above one, it *explodes*, producing NaNs during training. This is the *vanishing/exploding gradient problem*, originally analyzed by Hochreiter (1991) and developed formally in Bengio et al. (1994); Hochreiter et al. (2001), and revisited with a modern optimization-theoretic view by Pascanu et al. (2013).

Three practical remedies partially alleviate the pathology without changing the architecture. *Gradient clipping* (Pascanu et al., 2013) rescales the parameter-gradient whenever its norm exceeds a threshold; it eliminates the worst exploding events at negligible cost and is now a standard training default for any recurrent model. *Orthogonal or identity-like initialization* of $\mathbf{W}_{hh}$ places its singular values near 1 so that the Jacobian product preserves norms at the start of training. *Truncated BPTT* unrolls only K steps back at each update, capping both the memory footprint and the effective gradient horizon at K . These help but do not cure the problem: the structural fix is to change the recurrence itself, which is the move made by *gated cells*.

1.11.2 Long Short-Term Memory (LSTM) and GRUs

Before writing equations, it helps to keep a plain-language picture in mind. A vanilla RNN asks one hidden state $\mathbf{h}_t$ to do three jobs at once: retain useful old information, absorb new information, and expose the relevant part to the next layer. This is a fragile design. The LSTM separates these tasks. It keeps a dedicated *memory lane* $\mathbf{C}_t$ flowing through time and at each

date makes three soft decisions: what to keep, what to write, and what to reveal. That is the intuition behind the gate equations below.

The LSTM cell (Hochreiter and Schmidhuber, 1997) replaces the single-state recurrence $\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$ by a pair of states: a *cell state* $\mathbf{C}_t$ that flows along the top of the cell with *additive* updates, and a *hidden state* $\mathbf{h}_t$ that is read off it. Three learned sigmoid gates, each depending on the concatenation $[\mathbf{h}_{t-1}, \mathbf{x}_t]$, control what information flows through:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate}) \quad (1.25)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate}) \quad (1.26)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (\text{candidate cell}) \quad (1.27)$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (\text{cell state update}) \quad (1.28)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate}) \quad (1.29)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (\text{hidden state}). \quad (1.30)$$

Each of $\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t \in (0, 1)^d$ acts as a soft switch applied element-wise. The crucial structural change is in equation (1.28): the cell state is *additively* corrected rather than multiplicatively overwritten. Along the direct memory path, differentiating $\mathbf{C}_t$ with respect to $\mathbf{C}_{t-1}$ contributes $\text{diag}(\mathbf{f}_t)$ in place of a full recurrent matrix product. The full derivative also contains indirect terms because the gates depend on $\mathbf{h}_{t-1}$ and hence on earlier cell states, but the direct path is the constant-error-carousel intuition: when the cell judges information worth keeping, it can open the forget gate ($\mathbf{f}_t \approx \mathbf{1}$) and allow gradients to flow through *as if* the sequence were shorter. Figure 1.18 sketches the resulting cell.

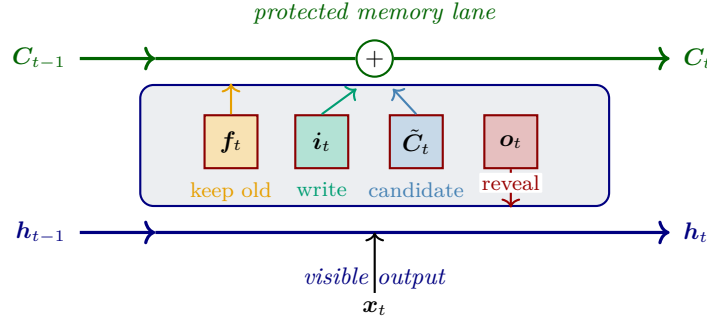


Figure 1.18: The LSTM cell. The green top lane is the *protected memory lane*: old memory can be kept, new information can be written additively, and the resulting state can later be revealed through the hidden output (*visible output*, blue bottom lane). This is the intuition behind the forget, input, candidate, and output components shown inside the cell.

The *Gated Recurrent Unit* (GRU) of Cho et al. (2014) is a lighter sibling that merges the forget and input gates into a single *update* gate $\mathbf{z}_t$ and drops the separate cell state in favor of the hidden state itself:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z), \quad (1.31)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r), \quad (1.32)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \quad (1.33)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (1.34)$$

A GRU uses roughly 25% fewer parameters than an LSTM of the same hidden size and performs comparably on many sequence tasks (Chung et al., 2014), at the cost of a slightly less expressive memory channel.

LSTMs remain particularly effective for economic time-series tasks where capturing specialized temporal patterns is more important than massive scalability. For example, [Holt et al. \(2024\)](#) use LSTMs to detect *Edgeworth cycles* in retail gasoline-price data. These cycles are asymmetric, high-frequency price movements that are difficult to identify with traditional spectral analysis or simple rule-based methods. In this setting the LSTM architecture excels at identifying the characteristic sawtooth patterns (sudden price jumps followed by slow decays) across thousands of retail stations, providing a robust tool for antitrust analysis and competition policy.

1.11.3 Limits of Recurrent Models

Even with gated cells, recurrent architectures retain two fundamental limitations that no amount of tuning can remove.

1. **Inherently sequential.** Step t cannot begin until step $t - 1$ has finished, so the full computation takes T serial wavefronts regardless of how many cores or tensor units are available. On a modern GPU that can evaluate thousands of MLP rows in parallel, this forced serialization makes RNN training slow and wall-clock expensive. Training a contemporary large language model on a trillion-token corpus with a plain LSTM would take years of wall-clock time rather than weeks.
2. **Path-length-dependent decay.** Information from position 1 must travel through all intermediate hidden states to influence position T , each hop applying a matrix and a nonlinearity. Gating mitigates but does not eliminate this decay, and empirically LSTM performance saturates on sequences well below the theoretical limit implied by the cell’s capacity.

Both problems have the same structural cause: the recurrence forces a path of length T between the two extreme positions of the sequence. The remedy is to drop recurrence entirely and let every position read every other position *directly*, in parallel. This is the Transformer.

1.11.4 The Transformer Architecture

The *Transformer* architecture ([Vaswani et al., 2017](#)) replaced recurrence entirely with *self-attention*. This allows the model to weigh the importance of all tokens in a sequence simultaneously, enabling massive parallelization and better capturing global dependencies.¹

For a first pass, four steps are enough. Add position information so the model knows order; project each token into a *query*, *key*, and *value*; compare each query to all keys; then take a weighted average of the values. Multi-heads, LayerNorm, residual connections, and pointwise MLPs are refinements of this core idea, not a different idea.

The Self-Attention Mechanism

Intuition: search, then retrieval. Before writing the mechanism formally it is useful to see it as a *soft library lookup*. Picture a small library with n shelves, one per position in the sequence. Each shelf i carries three vectors: a *query* q_i (“what am I looking for?”), a *key* k_i (“what is printed on my label?”), and a *value* v_i (“what do I actually contain?”). All three are linear projections of the same input token x_i , and the projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are *learned* during training; the librarian (queries and keys) and the books (values) are co-designed for whatever task the training objective encodes.

To produce the updated representation $\mathbf{o}_i$ at shelf i , the following four steps happen.

¹The idea that a network can compute its own weights from its inputs has a long history: the *Fast Weight Programmers* of [Schmidhuber \(1992\)](#) use one network to write the weights of another from context, which is widely viewed as a conceptual precursor of attention. [Schlag et al. \(2021\)](#) make the formal equivalence explicit, showing that linear-attention Transformers *are* Fast Weight Programmers.

1. **Score.** Shelf i 's query q_i is compared against every shelf's key k_j ; the dot product $q_i^\top k_j$ is a *similarity score*, high when shelf j 's label matches what shelf i is looking for.
2. **Normalize.** The n scores are divided by $\sqrt{d_k}$ (to keep them in a numerically sane range) and pushed through a softmax, producing probability weights $\alpha_{ij} \in [0, 1]$ with $\sum_j \alpha_{ij} = 1$.
3. **Retrieve.** The weights are applied to the values: $\mathbf{o}_i = \sum_j \alpha_{ij} v_j$ is a weighted average of the n shelf contents.
4. **Repeat.** Steps 1–3 happen in parallel for every shelf i , producing the whole output sequence $\mathbf{o}_1, \dots, \mathbf{o}_n$ in a single layer.

Shelves whose label matched the query contribute most to the output; shelves whose label was off-topic are almost ignored.

Why this is useful: a worked example. Consider the sentence “*The cat sat on the mat. It purred.*” For the model to process “it” properly, it must first decide what “it” refers to, the cat or the mat. Self-attention performs exactly this disambiguation: the query vector at the “it” position probes the key vectors at every earlier position, and the softmax converts the raw similarity scores into probability weights that concentrate most of the mass on the correct antecedent. Figure 1.19 below illustrates the resulting pattern: the bulk of the weight lands on “cat”, and the updated representation at the “it” position is formed as a weighted average of the values, driven mostly by “cat”. The same mechanism, run in parallel for every position, produces all of $\mathbf{o}_1, \dots, \mathbf{o}_n$ in a single layer.

A small concrete example. Suppose we have only three shelves and two-dimensional q 's and k 's. Take the query at shelf 3 to be $q_3 = (1, 0)$ and the three keys $k_1 = (0.9, 0.1)$, $k_2 = (0.1, 0.8)$, $k_3 = (1, 0)$. The raw similarity scores are $q_3^\top k_1 = 0.9$, $q_3^\top k_2 = 0.1$, $q_3^\top k_3 = 1.0$. Ignoring the $1/\sqrt{d_k}$ scaling to keep the arithmetic clean, the softmax weights come out to roughly (0.405, 0.182, 0.448). Shelf 3's output $\mathbf{o}_3$ is therefore a blend of the three values in those proportions: most mass on shelf 3 itself and on shelf 1 (the closest match in label space); shelf 2, whose label k_2 points in an orthogonal direction, contributes about 18%. The softmax is “soft” precisely in this sense, a smoothed nearest-neighbor retrieval rather than a hard argmax over the most similar shelf.

The “self” in *self*-attention says that every shelf plays both roles simultaneously: every shelf's query probes every shelf's key, and every shelf receives an updated representation as output. A single attention layer therefore pairs all n positions with all n positions in parallel. Contrast this with an RNN or LSTM: to let position t peek at information from position 1, the signal must be shuttled through every intermediate position via the hidden state, with each hop applying its own weight matrix and nonlinearity, so long-range information is either blurred or lost entirely. Attention short-circuits that chain: position t reads position 1 directly, with no intermediate hops and no path-length-dependent attenuation. This is the architectural source of the Transformer's advantage on long sequences, and the reason why attention-based models quickly displaced recurrent ones for language, code, and (increasingly) time-series forecasting.

Given a sequence of input vectors, stack the tokens as rows of $\mathbf{X} \in \mathbb{R}^{n \times d}$. The attention mechanism computes three projections for each token: a *Query* (Q), a *Key* (K), and a *Value* (V), using learned weight matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$:

$$Q = \mathbf{X}\mathbf{W}_Q, \quad K = \mathbf{X}\mathbf{W}_K, \quad V = \mathbf{X}\mathbf{W}_V. \quad (1.35)$$

The output of the attention layer is a weighted sum of the values, where the weights are determined by the compatibility (dot product) of the queries with the keys:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (1.36)$$

The scaling factor $\sqrt{d_k}$ (the dimensionality of the keys) prevents the dot products from growing too large in magnitude, which would otherwise make the softmax unstable.

An econometric lens. The attention layer is a *data-dependent, learnable kernel smoother*. Compare it to the Nadaraya–Watson estimator $\hat{f}(x) = \sum_i w_i(x) y_i$ with kernel-based weights $w_i(x) \propto k(x, x_i)$: attention has exactly this form, but the similarity $k(\cdot, \cdot)$ is the parametric bilinear form $(q, k) \mapsto q^\top k / \sqrt{d_k}$ and both q and k are themselves *learned* projections of the input. From this vantage point the Transformer’s “magic” is less mysterious: it is a nonparametric smoother whose kernel the optimizer tunes to whatever task the training objective encodes. Self-attention further recovers the classical recurrence-free property that every pair of positions interacts in a single parallel layer, with no signal decay along the sequence.

Figure 1.19 renders the attention pattern of the worked “cat/it” example on a compressed five-token version of the sentence. The output $\mathbf{o}_{it}$ is the new representation at the “it” position, formed as a weighted average of the values, with most weight coming from “cat”.

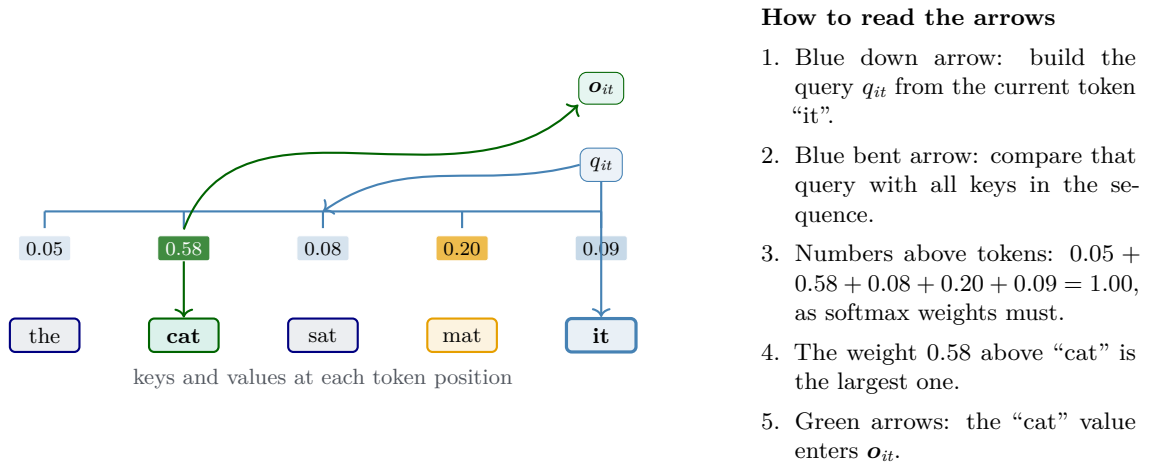


Figure 1.19: A worked self-attention pattern. Both q_{it} and $\mathbf{o}_{it}$ sit above the “it” position: q_{it} is the query projection of “it” (blue), and $\mathbf{o}_{it}$ is the updated representation at that same position (green). The blue arrows show how the query is built from “it” and then compared with every key. The softmax weights above each token sum to one; here the largest weight lands on “cat”, so the green aggregation arrow starts above “cat” and curves up to $\mathbf{o}_{it}$, indicating that the update at “it” is driven mainly by the value at “cat”.

Multi-Head Attention

A single attention layer implements *one* similarity pattern between positions. Linguistic and economic sequences, however, contain many relations that matter simultaneously: subject–verb agreement, coreference of pronouns, topic alignment, or, in a macro panel, sector co-movements, shock transmission lags, and autocorrelation structure. A single head is forced to compress all of them into one kernel, and typically does a poor job.

The fix is *multi-head attention*. Run H attention layers in parallel, each with its *own* projection matrices $(\mathbf{W}_Q^{(h)}, \mathbf{W}_K^{(h)}, \mathbf{W}_V^{(h)})$ mapping the input to a lower-dimensional subspace of size $d_k = d/H$, compute H attention outputs independently, then concatenate and linearly project back:

$$\text{head}_h = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(h)}, \mathbf{X}\mathbf{W}_K^{(h)}, \mathbf{X}\mathbf{W}_V^{(h)}), \quad (1.37)$$

$$\text{MHA}(\mathbf{X}) = [\text{head}_1; \text{head}_2; \dots; \text{head}_H] \mathbf{W}_O. \quad (1.38)$$

The total parameter count is essentially unchanged, because each head works on a $1/H$ -dimensional slice, but the inductive bias is richer: different heads are free to specialize in

different relations. Interpretability studies of trained Transformers routinely find heads that focus on the previous token, on the closing bracket matching an open one, on the subject of the current clause, or, in time-series models, on the most recent analogue of the current calendar month. Multi-head attention is therefore *structurally analogous to a mixture of learned kernels* in a nonparametric regression; the weights $\mathbf{W}_O$ are the mixing coefficients, and the softmax-scored pairs are the kernels themselves.

Positional Encoding

Self-attention treats its input as a *set*: permuting the positions of the tokens permutes the rows of $\mathbf{X}$ and permutes the rows of the output, but changes nothing else. This is problematic, because order matters in every sequence application (“dog bites man” vs. “man bites dog”; “inflation before the shock” vs. “after”). A model without a notion of position cannot distinguish them.

The fix that Vaswani et al. (2017) propose is to *add* a deterministic vector $\text{PE}(t) \in \mathbb{R}^d$ to the input embedding at each position t , chosen so that the dot products $\text{PE}(t)^\top \text{PE}(t+k)$ encode the relative distance k . The original sinusoidal scheme is

$$\text{PE}(t, 2k) = \sin(t/10000^{2k/d}), \quad \text{PE}(t, 2k+1) = \cos(t/10000^{2k/d}), \quad k = 0, \dots, d/2 - 1. \quad (1.39)$$

Three properties make this choice useful. First, each coordinate $2k$ is a sine wave of wavelength $2\pi \cdot 10000^{2k/d}$, so the encoding spans wavelengths from roughly 2π up to $2\pi \cdot 10000$; the model sees both fine local positions and coarse global ones at once. Second, for each sine/cosine pair, a fixed offset can be represented by a 2×2 rotation: $\text{PE}(t+r)$ is a linear function of $\text{PE}(t)$ with coefficients depending only on the relative offset r . This gives attention a convenient way to represent relative positions. Third, the encoding extrapolates: a model trained on sequences of length 512 can be fed sequences of length 1024 and the position code is still well-defined. Modern alternatives, rotary position embeddings (RoPE) and ALiBi, refine these properties but keep the same philosophy.

The Transformer Block

Single attention layers, even multi-headed, are not yet expressive enough: they are essentially linear in the values, with a nonlinear mixing pattern. A full *Transformer block* wraps one MHA layer and one pointwise MLP with residual connections and LayerNorm:

$$\mathbf{x}^+ = \mathbf{x} + \text{MHA}(\text{LN}(\mathbf{x})), \quad (1.40)$$

$$\mathbf{x}^{\text{out}} = \mathbf{x}^+ + \text{MLP}(\text{LN}(\mathbf{x}^+)). \quad (1.41)$$

The LayerNorm steps (Ba et al., 2016) standardize across feature coordinates; together with the residual additions they stabilize training of very deep stacks. Equations (1.40)–(1.41) describe the modern *pre-norm* variant (LN before each sub-block), which is easier to train than the original *post-norm* variant of Vaswani et al. (2017). Figure 1.20 shows the architecture schematically.

Encoder, decoder, causal masking. The original Transformer of Vaswani et al. (2017) pairs an *encoder* stack (processes the source sequence) with a *decoder* stack (generates the target sequence), linked by a cross-attention layer. Most modern large language models are *decoder-only*: they use the same block as above, but the attention softmax is applied to a masked score matrix that sets all entries above the diagonal to $-\infty$. This *causal mask* forbids a position from attending to future positions, turning the Transformer into a left-to-right autoregressive predictor suitable for language modeling.

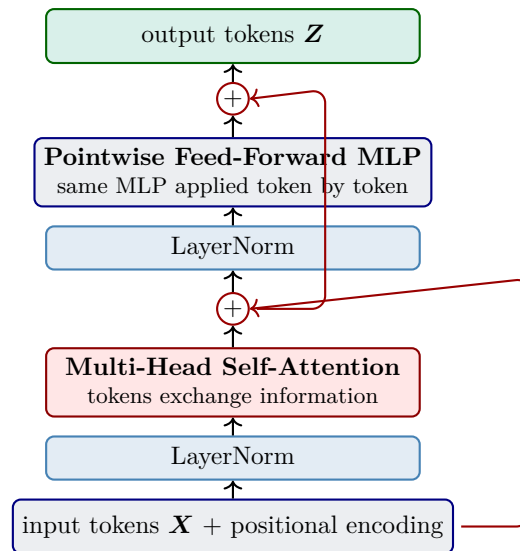


Figure 1.20: One *Transformer block* in pre-norm form. Self-attention first mixes information across token positions, then the pointwise MLP transforms each token separately. The red skip paths are the residual connections that let deep stacks train stably. A full Transformer stacks L such blocks; GPT-3, for instance, uses $L = 96$.

Scaling and parallelism. For day 1 the key engineering fact is simpler than the modern LLM discussion: attention is parallel, recurrence is not. Because every sub-block is pointwise in time and the only cross-position operation (attention) is fully parallelizable, a Transformer with L blocks, H heads, and hidden dimension d can be trained on accelerators at roughly the theoretical peak throughput. This is why modern foundation models, GPT- n , BERT, ViT, Claude, are Transformers rather than RNNs. Empirical studies of *scaling laws* (Kaplan et al., 2020; Hoffmann et al., 2022) document that test loss decreases as a power law in parameters, data, and compute, giving rise to the compute-optimal prescriptions used by the largest labs. For an economist, the relevant takeaway is that the marginal cost of additional capability is governed by a smooth, quantifiable, and *very large* compute bill.

At a glance: RNN vs LSTM vs Transformer

Table 1.3 summarizes the three architectures along the dimensions most relevant to a practitioner's choice.

A practical rule of thumb follows immediately. If the task is a specialized time-series problem with moderate history length and limited data, an LSTM remains a strong baseline. If context is long and accelerator-friendly parallelism matters, one should usually start with a Transformer.

1.11.5 Advanced Aside: In-Context Learning of an AR(1) Process

The material up to this point is the core Chapter 1 message; readers comfortable with the RNN $\rightarrow$ LSTM $\rightarrow$ Transformer summary may skip directly to the Chapter Summary on page 48. This subsection is an optional but illuminating detour: it shows why economists often find attention intuitive once they see it through a regression lens, and it is the analytical companion to notebook 09_Transformer_InContext_AR1.

A remarkable emergent property of large Transformers is *in-context learning*: the ability to solve a task *at inference time*, given only examples in the prompt, with no weight updates. A model pretrained on a universe of sequences can be shown a fresh series it has never seen and produce sensible next-step forecasts. For an economist this is striking: the Transformer behaves as though it *runs a regression inside its forward pass*, with the prompt playing the role of the

	RNN	LSTM / GRU	Transformer
Hidden state	single $\mathbf{h}_t$	$\mathbf{h}_t$ and $\mathbf{C}_t$	none per step; the residual stream across layers is an implicit state
Path length $1 \rightarrow T$	$\mathcal{O}(T)$	$\mathcal{O}(T)$	$\mathcal{O}(1)$
Parallelism over t	none	none	full (all positions at once)
Compute per layer	$\mathcal{O}(T d^2)$	$\mathcal{O}(T d^2)$	$\mathcal{O}(T^2 d + T d^2)$
Memory per layer	$\mathcal{O}(T d)$	$\mathcal{O}(T d)$	$\mathcal{O}(T^2 + T d)$
Training stability	gradient decay/blow-up	much better, gated	stable with LN + residuals
Sweet spot	short sequences	mid-length, niche patterns	long context, massive parallelism

Table 1.3: Comparison of the three sequence architectures. Transformers trade a quadratic-in- T attention cost for full parallelism and unit-length paths between any pair of positions, which is an excellent trade on modern accelerators and for the long sequences typical in language and high-frequency finance.

training sample and the final token playing the role of the test point. [Von Oswald et al. \(2023\)](#) make this formal by showing that self-attention layers can implement gradient-based optimization internally, so a stack of such layers iteratively reduces an implicit loss.

Consider the simplest concrete setting, an autoregressive process of order 1:

$$x_t = \varrho x_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2). \quad (1.42)$$

If we provide a Transformer with a sequence $(x_1, \dots, x_t)$, it can predict x_{t+1} by implicitly estimating ϱ from the history. There are two closely related but distinct perspectives. First, under suitable linear-attention parameterizations, self-attention layers can implement gradient-descent-like updates on least-squares objectives ([Von Oswald et al., 2023](#)); specialized to the AR(1) target above, this reads as descent on $\min_{\varrho} \sum_{i=2}^t (x_i - \varrho x_{i-1})^2$. Second, with ordinary softmax attention, *one* natural Q/K/V assignment behaves like a kernel smoother:

- **Query:** the current state $Q_t = x_t$ (“where am I now?”),
- **Key:** the lagged state $K_i = x_{i-1}$ (“past states that preceded each outcome”),
- **Value:** the realized successor $V_i = x_i$ (“what came next”).

This is one particular parameterization that works because the AR(1) target is linear in the lagged state; with this choice the softmax-attention output becomes

$$\hat{x}_{t+1} = \sum_{i=2}^t \alpha_i x_i \approx \hat{\varrho} x_t, \quad (1.43)$$

where the softmax weights $\alpha_i \propto \exp(x_t x_{i-1})$ concentrate mass on those past states x_{i-1} that *look like* the current state x_t .

Why this approximates ϱx_t , in three short steps. Equation (1.43) is exactly a Nadaraya–Watson kernel regression of x_i on x_{i-1} , evaluated at $x_{i-1} = x_t$, with kernel $K(x_*, x_{i-1}) = \exp(x_* x_{i-1})$. The intuition is then standard:

1. *Population fact.* The AR(1) data-generating process implies $\mathbb{E}[x_i \mid x_{i-1} = x_*] = \varrho x_*$ exactly. So the population conditional mean we are trying to estimate at $x_* = x_t$ is just ϱx_t .

2. *Kernel smoother.* The softmax attention output $\sum_i \alpha_i x_i$ with $\alpha_i \propto K(x_t, x_{i-1})$ is a kernel-weighted average of the values x_i at past time steps i , with the weights peaked where the lagged state x_{i-1} is closest to x_t . Provided enough past observations land near x_t (so the kernel concentrates around x_t), this is the empirical Nadaraya–Watson estimator of $\mathbb{E}[x \mid x_{i-1} = x_t]$.
3. *Conclusion.* Combining the two, $\sum_i \alpha_i x_i \approx \mathbb{E}[x_i \mid x_{i-1} = x_t] = \varrho x_t$. The shock variance σ^2 controls how much the realized x_i ’s scatter around the conditional mean and therefore the variance of the kernel estimate, but it does not enter the Nadaraya–Watson *location* at first order.

Note that the unscaled inner product $x_t x_{i-1}$ used in the kernel is dimension-1, so the standard $1/\sqrt{d_k}$ scaling of multi-head attention plays no role here: even without it, the softmax concentrates around the past x_{i-1} ’s closest to x_t as soon as the prompt is long enough.

For the econometrician’s mental library the closest classical objects are the Nadaraya–Watson and local-linear estimators; the novelty is that the kernel is not hand-chosen but jointly learned with the data representation, over a large corpus of related tasks. This is a form of *meta-learning*: the network learned “how to regress” during pretraining, and at inference it runs that regression on a brand-new series. Self-attention can therefore implement optimization-like or regression-like computations internally, not merely local pattern matching.

Code examples. The following Jupyter notebooks implement and extend the material in this chapter:

- `01_BasicML_intro`: linear regression, classification, and loss functions.
- `02_GradientDescent_and_SGD`: implementing and visualizing optimizers.
- `03_Double_Descent`: the modern generalization regime.
- `04_Gentle_DNN`: building a simple DNN from scratch.
- `05_Tensorboard`: monitoring training progress.
- `06_PyTorch_intro`: introduction to PyTorch fundamentals.
- `07_Genz_Approximation_and_Loss_Functions`: high-dimensional integration using Genz test functions and robust losses.
- `08_MLP_LSTM_Transformer_Edgeworth_Cycles`: a three-way comparison on the same Edgeworth-cycle task. An MLP that sees only x_t collapses near the cycle mean (no memory); an LSTM with a 32-unit hidden state tracks the sawtooth almost exactly via its gated memory; a tiny encoder-only Transformer ($d_{\text{model}} = 16$, two layers, four heads, $\sim 4.7\text{k}$ parameters) attends to the full window in parallel and beats the MLP by an order of magnitude, while remaining slightly behind the LSTM on this small, highly periodic, low-data signal – a deliberate illustration that architectural inductive bias matters as much as flexibility on small problems. Quantitatively, the test-set MAE ranking that the notebook prints in its summary cell is, in order, $\text{LSTM} < \text{Transformer} \ll \text{MLP}$, with the Transformer typically within a small multiple of the LSTM and the MLP an order of magnitude worse; readers should consult the notebook’s printed table for the seed-specific numbers.
- `09_Transformer_InContext_AR1`: advanced / optional notebook. A tiny 2-layer Transformer learns *how to regress* across many AR(1) draws; at inference it recovers $\hat{\varrho}$ in-context without weight updates, reproducing the analytical prediction above.

Chapter Summary

- Deep networks compose simple nonlinear coordinate transformations: sufficiently wide shallow networks already attain universal approximation (Cybenko, 1989; Hornik et al., 1989), but depth gives provably more efficient representations for compositional functions (Telgarsky, 2016; Barron, 1993).
- Training rests on three pillars: He/Xavier initialization, smooth non-saturating activations, and adaptive optimizers (SGD with momentum $\rightarrow$ RMSprop $\rightarrow$ Adam $\rightarrow$ AdamW); backpropagation makes the gradient cost essentially the same as the forward pass (Rumelhart et al., 1986; Baydin et al., 2018).
- Generalization in modern over-parameterized regimes is described by the double-descent curve and explained theoretically via the Neural Tangent Kernel and benign-overfitting results, not by classical bias–variance (Jacot et al., 2018; Belkin et al., 2019; Nakkiran et al., 2020; Bartlett et al., 2020).
- Sequence architectures (RNN $\rightarrow$ LSTM $\rightarrow$ Transformer) are mentioned for completeness; the rest of the script uses feed-forward MLPs because DEQNs and PINNs operate on unstructured state vectors.

Further Reading

- Goodfellow et al. (2016), the standard graduate textbook covering everything in this chapter at greater depth.
- Schmidhuber (2015), a historical survey tracing the deep-learning lineage; useful for context on LSTMs, Highway Networks, and Fast Weight Programmers.
- Bishop (2006), pattern recognition from a Bayesian/statistical viewpoint; the natural complement for econometric readers.
- Kingma and Ba (2015); Loshchilov and Hutter (2019), the canonical references on Adam and AdamW; read together they explain modern optimizer tuning.
- Bottou et al. (2018), a comprehensive survey of stochastic optimization for large-scale learning, including convergence rates.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

Workload labels. Throughout the script, every exercise carries one of three workload tags inside its title. *[Core]* marks short analytical or pencil-and-paper questions suitable for a weekly problem set. *[Computational]* marks notebook-based exercises that involve running or modifying companion code; allow yourself a long evening or a weekend with verification gates and starter code in hand. *[Advanced/project]* marks longer, research-style assignments that may require a multi-day investment, a proper compute budget, or a small term-project plan. The labels are advisory rather than prescriptive: students with prior exposure can promote a *[Computational]* exercise to a quick warm-up, while those new to the material can treat several *[Advanced/project]* entries as inspiration for term work.

1. **[Core] Backprop on a 2-layer net.** Take a single hidden layer with ReLU activation: $\hat{y} = w_2 \sigma(w_1 x + b_1)$ and squared loss $\ell = (\hat{y} - y)^2$. Derive $\partial \ell / \partial w_1$ and $\partial \ell / \partial w_2$ by hand and compare with what `torch.autograd` returns on a worked numerical example ($x = 2$, $y = 1$, all weights 0.5).

2. **[Core] MSE vs. MLE.** Show that the maximum-likelihood estimator of the slope of $y_i = \beta x_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, coincides with the OLS estimator and with the minimizer of $\sum_i (y_i - \beta x_i)^2$. Then repeat with ε_i Laplace-distributed and discuss why the squared loss is no longer optimal.
3. **[Core] Activation choice for a PINN.** Argue carefully why ReLU networks cannot be used to solve a second-order PDE in strong form. Construct an explicit example where the second derivative of the network output is identically zero except on a measure-zero set.
4. **[Core] Adam vs. AdamW.** In a regression with L_2 regularization, write down the per-parameter update rule for Adam (with L_2 added to the loss) and for AdamW (decoupled). Show that the two updates do *not* coincide and explain which one preserves the intuition that “weight decay shrinks weights uniformly.”
5. **[Core] RNN forward pass by hand.** Take a vanilla recurrent unit with hidden dimension $H = 2$, scalar input, and update $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$, scalar readout $\hat{y}_t = W_y h_t$. Use $W_h = 0.5 I_2$, $W_x = (1, 0)^\top$, $b = (0, 0)^\top$, $W_y = (1, 1)$, $h_0 = (0, 0)^\top$, and the input sequence $x_{1:3} = (1, 0, 1)$. (i) Compute h_t and $\hat{y}_t$ for $t = 1, 2, 3$. (ii) Derive a closed-form expression for $\partial \hat{y}_3 / \partial x_1$. (iii) Show that with $\|W_h\|_2 < 1$ this gradient decays exponentially in the sequence length, and explain how this connects to the vanishing-gradient problem (§1.11).
6. **[Core] Attention by hand.** Consider a single attention head on a 3-token scalar sequence $x_{1:3} = (0.0, 1.0, 0.5)$ with identity projections $W_Q = W_K = W_V = 1$ (so $q_i = k_i = v_i = x_i$) and scaling $\sqrt{d_k} = 1$. Compute the attention weights $a_{ij} = \text{softmax}_j(q_i k_j)$ and the output $o_i = \sum_j a_{ij} v_j$ for $i = 1, 2, 3$. Verify that each o_i is a convex combination of $\{v_1, v_2, v_3\}$ and identify which token attends most to which.
7. **[Computational] Notebook.** In 05_Tensorboard, plot the train and validation loss for three optimizer choices (SGD, Adam, AdamW) on a small classification task. Comment on which converges fastest, which generalizes best, and where the curves diverge.

Chapter 2

Deep Equilibrium Nets

This chapter introduces Deep Equilibrium Nets (DEQNs), the central computational framework of this script. Rather than generating labeled training data, a DEQN embeds the equilibrium conditions of a dynamic stochastic model (Euler equations, budget constraints, complementarity conditions) directly into the loss function of a neural network. The network then learns policy functions by minimizing the violation of these conditions via gradient descent. We develop the methodology on the classical Brock–Mirman growth model, for which an analytical solution exists and serves as a benchmark. The primary reference is [Azinovic et al. \(2022\)](#).

Notation transition. Chapter 1 used θ for the parameter vector of a generic supervised model, following the textbook convention. From this chapter onward we switch to ρ (so that the network is denoted $\mathcal{N}_\rho(\mathbf{x})$), matching the notation of [Azinovic et al. \(2022\)](#) and most of the subsequent DEQN literature. The two symbols denote the same object; we use ρ for parameters wherever a network appears, and reserve θ for structural economic parameters that the network does not see (Chs. 10–11).

2.1 The Curse of Dimensionality in Economics

A central challenge in computational economics is that many models of interest, such as overlapping-generations (OLG) economies, heterogeneous-agent models, and international business cycle models, feature state spaces of high dimension. In an OLG model with N cohorts and idiosyncratic risk, the state vector may include individual capital holdings, productivity levels, and wealth shares for each cohort, leading to state spaces of dimension $d \gg 10$. Traditional grid-based solution methods, such as value function iteration on a Cartesian grid with n nodes per dimension, require n^d grid points; this is the *curse of dimensionality*, the term coined by [Bellman \(1961\)](#) to describe the exponential growth of computational cost with the number of state variables. A concrete back-of-the-envelope makes the bottleneck plain: at the modest resolution of $n = 100$ nodes per dimension, a $d = 5$ state space already requires $100^5 = 10^{10}$ grid points, which is infeasible under any reasonable compute budget; even more modest resolutions of $n = 20$ – 30 become prohibitive once d exceeds ten.

Table 2.1 illustrates the curse concretely. Even with a modest $n = 10$ grid points per dimension, the total number of grid points grows astronomically.

The volume paradox. A complementary, and more geometric, perspective on the curse of dimensionality starts from a single ratio. The volume of the unit d -ball relative to the smallest cube that contains it, $[-1, 1]^d$, is

$$\frac{V_{\text{ball}}(d)}{V_{\text{cube}}(d)} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)}. \quad (2.1)$$

d	Grid points (10^d)	Memory (64-bit)	Feasibility
1	10	80 B	trivial
2	100	800 B	trivial
5	10^5	800 KB	feasible
10	10^{10}	80 GB	borderline
20	10^{20}	8×10^{11} GB	impossible
50	10^{50}	—	absurd
100	10^{100}	—	a googol

Table 2.1: Size of an $n = 10$ Cartesian grid and the 64-bit memory required to store one floating-point value per grid point, as a function of state-space dimension d . Grid-based methods are comfortable only at low dimension; by $d = 10$ even storing one scalar per grid point is already borderline before policies, interpolation objects, residuals, and simulation output are added.

This ratio collapses with d (Figure 2.1): it is $\pi/4 \approx 0.785$ at $d = 2$ and $\pi/6 \approx 0.524$ at $d = 3$, has fallen to $\approx 2.5 \times 10^{-3}$ by $d = 10$, and is below 10^{-70} at $d = 100$. The geometry behind the numbers is sharp. The inscribed ball touches the center of each of the $2d$ faces, at distance 1 from the center, but its surface never gets within $\sqrt{d} - 1$ of any of the 2^d corners, which sit at distance $\sqrt{d}$. As d grows the corners march off to infinity while the ball stays put, so essentially all of the cube’s volume drains into its corners, far from the center. A box-shaped object in high dimensions is, to a first approximation, all corners and no middle.

Why should an economist care about a fact about cubes and balls? Because solving a dynamic stochastic model means producing a *global* solution¹, and the region over which that solution actually has to be accurate is the model’s *ergodic set*, the states the economy visits under its own law of motion. That set is typically a thin, curved, low-volume sliver of any hypercube $[\mathbf{x}_{\min}, \mathbf{x}_{\max}]^d$ that one would draw around it, often well under one percent of the bounding box already at moderate d (Judd et al., 2011). Yet most of the high-dimensional approximation machinery on offer, tensor-product grids, adaptive sparse grids, tensor-product quadrature, is built on the hypercube; by the volume paradox an exponentially growing fraction of those nodes lands in corners the model never visits, so most of the computational effort is spent representing the function where it does not matter. Figure 2.2 visualizes this mismatch. This is exactly the waste the DEQN approach is designed to avoid: instead of tiling a box, it trains the network on states sampled from the model’s own simulated trajectories, putting the approximation effort where the economics lives.

Distance itself is the second casualty of high dimensions, and it is worth flagging now because it returns later with teeth. Draw a point uniformly from the unit d -ball: its radius has cumulative distribution r^d and density dr^{d-1} on $[0, 1]$, which for large d piles essentially all of its mass against the shell $r \rightarrow 1$. Random points sit on the pulp, not in the core, and they do so with overwhelming probability. Relatedly, pairwise Euclidean distances among random points concentrate so tightly that the nearest and the farthest neighbor of a query point become almost equidistant, so “distance” loses most of its power to discriminate (Aggarwal et al., 2001) (Figure 2.3). This is not a curio: any method that judges similarity through $\|\mathbf{x} - \mathbf{x}'\|$, from k -nearest-neighbors to kernel ridge regression to the Gaussian-process surrogates of Chapter 9, whose RBF and Matérn kernels are functions of $\|\mathbf{x} - \mathbf{x}'\|$ alone, loses resolution as d grows. It is one of the reasons Chapter 9 reaches for dimension reduction, active subspaces and deep kernels, before fitting a GP in a high-dimensional input space.

¹In the sense of Brumm and Scheidegger (2017): an approximation of the equilibrium policy (or value) functions over the entire economically relevant region of the state space, in particular over the model’s ergodic set, as opposed to a *local* (perturbation) solution that is accurate only in a neighborhood of the deterministic steady state.

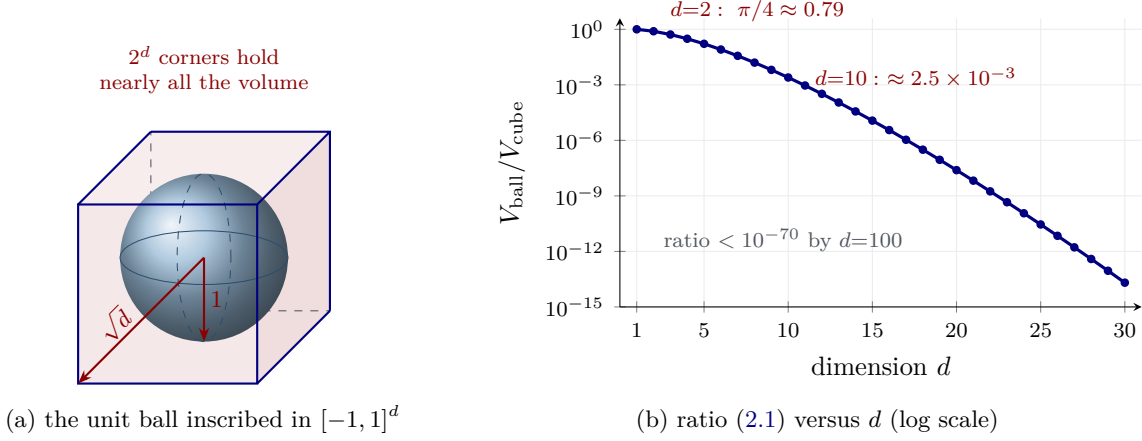


Figure 2.1: The volume paradox behind the curse of dimensionality. (a) The largest ball that fits inside the cube $[-1, 1]^d$ touches the center of every face, at distance 1 from the center, but its surface stays $\sqrt{d} - 1$ away from each of the 2^d corners, which lie at distance $\sqrt{d}$. As d grows the corners recede while the ball does not, so almost all of the cube’s volume ends up in the corners (tinted), far from the center. (b) The ball-to-cube volume ratio of equation (2.1) on a logarithmic scale: $\pi/4$ at $d = 2$, about 2.5×10^{-3} at $d = 10$, and below 10^{-70} at $d = 100$. A grid or quadrature rule built on the bounding hypercube therefore spends an exponentially growing share of its nodes in corners that the model’s ergodic set never reaches.

2.2 From Supervised Learning to Self-Supervised Equilibrium Solving

A word on terminology before we start, recalling the taxonomy of Section 1.3. A learning method is *supervised* when its loss compares the network’s output to an externally given target label; it is *unsupervised* when there are no target labels at all (clustering, density estimation, dimension reduction); and it is *self-supervised* when, although no human-provided labels exist, the method manufactures its own supervisory target out of the available structure. Deep Equilibrium Nets are unsupervised in the narrow “no labels” sense, but the sharper and more useful description is *equation-based self-supervision*: at every state where we evaluate the model, the equilibrium conditions tell us what the correct relationship between today’s policy and tomorrow’s must be, namely a residual of zero, and that residual plays exactly the role a label plays in supervised learning. The DEQN objective introduced below is therefore a genuine regression loss, only against a target the model itself dictates rather than one handed to us as data.

With that in mind, the key conceptual shift introduced by Azinovic et al. (2022) is to replace the labeled training data of standard supervised learning with the structural equations of the economic model. In a generic dynamic stochastic economic model, the state of the economy at time t is summarized by a vector $\mathbf{x}_t \in \mathbb{R}^d$, and agents choose policy (or decision) variables $p_t = p(\mathbf{x}_t) \in \mathbb{R}^m$. The equilibrium is characterized by a system of functional equations

$$G(\mathbf{x}_t, p(\mathbf{x}_t), \mathbb{E}_t[H(\mathbf{x}_{t+1}, p(\mathbf{x}_{t+1}))]) = 0, \quad \forall \mathbf{x}_t, \quad (2.2)$$

where G encodes optimality conditions (e.g., Euler equations) and $\mathbb{E}_t[\cdot]$ is the conditional expectation over next-period shocks via the transition law $\mathbf{x}_{t+1} = T(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$. The fundamental challenge is that this is a *functional equation*: we seek functions $p : \mathbb{R}^d \rightarrow \mathbb{R}^m$ rather than finite-dimensional parameter vectors. The DEQN approach parameterizes p as a neural network and solves this functional equation via stochastic optimization (Figure 2.4).

In the standard (supervised) paradigm, one minimizes the discrepancy between the network’s predictions and known target values. In the DEQN framework, the loss function is instead the

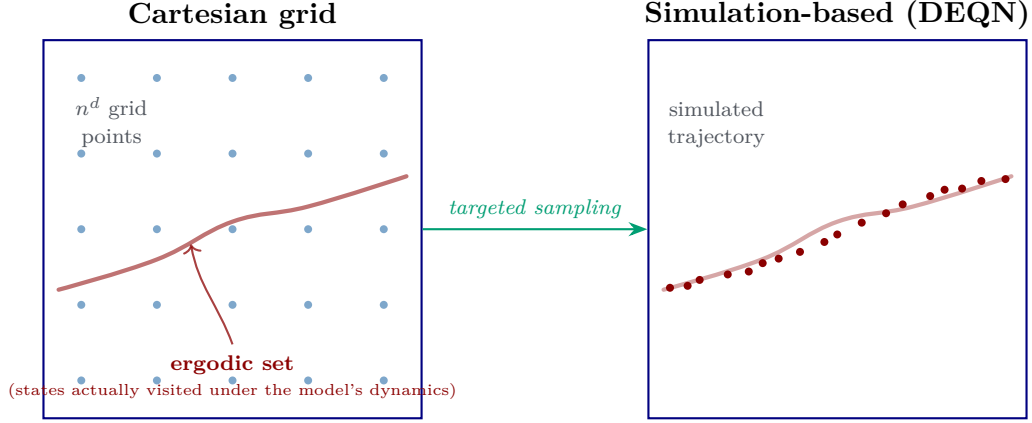


Figure 2.2: Grid-based vs. simulation-based state sampling. A Cartesian grid (left) allocates effort uniformly over the hypercube and places most of its n^d points far from the model’s *ergodic set*, the small subset of the state space that the economy actually visits under its own dynamics (red band). The DEQN algorithm samples states along simulated trajectories (right), concentrating training points exactly on that set. As d grows, the fraction of the cube reached by the ergodic set shrinks rapidly, so the grid’s relative waste grows exponentially.

squared norm of the equilibrium residual:

$$\ell_\rho = \frac{1}{N} \sum_{i=1}^N \|G(\mathbf{x}_i, \mathcal{N}_\rho(\mathbf{x}_i))\|^2, \quad (2.3)$$

where $\mathcal{N}_\rho$ denotes a deep neural network with parameters ρ that maps states to policies. If the residual loss is zero on a sufficiently rich set of states and the conditional expectations inside G are evaluated accurately (rather than with a single shock realization), the network satisfies the equilibrium conditions on those states; global accuracy must always be assessed on independent validation trajectories and, where available, against benchmark solutions. The continuous-time analogue is the residual-loss formulation of [Raissi et al. \(2019\)](#), which inspired the PINN literature developed in Chapter 7. The training points $\mathbf{x}_i$ are drawn not from an exogenous dataset, but from the model’s own simulated dynamics: the network learns on the ergodic distribution of the economy, concentrating computational effort on economically relevant regions of the state space. The idea of using simulated paths as the support for projection goes back to the parameterized-expectations algorithm of [Marcet \(1988\)](#) and [Marcet and Marshall \(1994\)](#), which the DEQN literature reframed as ergodic-set sampling to focus computation on economically relevant states; the numerical stability of stochastic-simulation projection was further developed by [Judd et al. \(2011\)](#).

Connection to the modern overparameterized regime. Note that DEQNs operate squarely in the “modern regime” of Section 1.10: the network often has $p \gg n$ parameters relative to the mini-batch size n . Unlike supervised learning with a fixed dataset, however, the training distribution is *renewable*: each episode can generate fresh state samples from the model’s own simulation. This changes the fixed-dataset double-descent analogy, but it does not remove the need for validation: p can still exceed the effective sample budget seen during training, and a flexible network can fit residuals on a narrow simulated region. The practical lesson is to combine overparameterized networks and SGD with independent validation trajectories, held-out Euler residuals, and, where available, closed-form benchmarks.

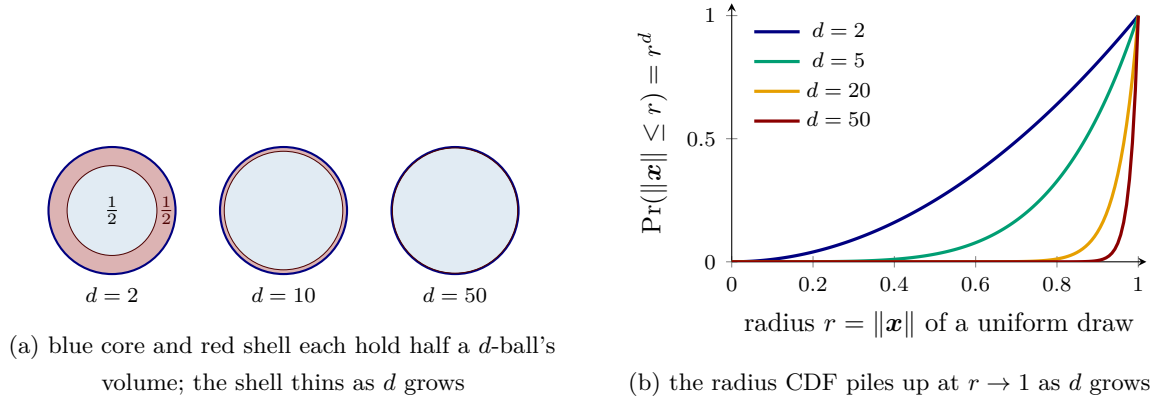


Figure 2.3: Why a “random” point in high dimensions lives on the shell, not in the core. (a) For each d , the blue inner disk and the red outer shell each hold half of the d -ball’s volume; the shell’s fractional thickness $1 - (1/2)^{1/d}$ shrinks from ≈ 0.29 at $d = 2$ to ≈ 0.07 at $d = 10$ to ≈ 0.014 at $d = 50$, so half of the mass crowds into an ever thinner rim near the surface. (b) Equivalently, the radius of a uniform draw has cumulative distribution r^d , which for large d stays near zero until r is almost 1 and then jumps: the radius is essentially deterministic at 1. The companion fact is that pairwise Euclidean distances among random points concentrate, so a query point’s nearest and farthest neighbors become nearly indistinguishable (Aggarwal et al., 2001); this is why distance-based methods, including the Gaussian-process kernels of Chapter 9, lose resolution in high dimensions.

2.3 The DEQN Training Algorithm

Algorithm: Deep Equilibrium Net Training

Input: Initial state $\mathbf{x}_0$, network $\mathcal{N}_\rho$, learning rate η , episodes E , simulation horizon T_{sim} , training steps T_{train} , expectation rule $\mathcal{Q}$ (path-average or quadrature)

[NEW] Burn-in: draw the first episode’s states from a broad prior (uniform box around the deterministic steady state) and maintain a small replay buffer of early states.

[NEW] Independent validation trajectory $\mathbf{x}_{0:T_{\text{val}}}^{\text{val}}$ for out-of-sample residual diagnostics, simulated once with frozen ρ at each checkpoint.

for episode $e = 1, \dots, E$ **do**

Simulate path: $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_{T_{\text{sim}}}$ using $\mathcal{N}_\rho$ and transition law; guard infeasible states (e.g. clip $C \leq 0$).

for gradient step $t = 1, \dots, T_{\text{train}}$ **do**

 Draw mini-batch $\mathcal{B} \subset \{\mathbf{x}_0, \dots, \mathbf{x}_{T_{\text{sim}}}\} \cup \text{replay}$

 Compute loss: $\ell_\rho = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x}_i \in \mathcal{B}} \|G(\mathbf{x}_i, \mathcal{N}_\rho(\mathbf{x}_i); \mathcal{Q})\|^2$ [NEW: $\mathcal{Q}$ is the chosen path-average or Gauss–Hermite / monomial / QMC rule]

 Update: $\rho \leftarrow \rho - \eta \cdot \nabla_\rho \ell_\rho$ [NEW: wrap the per-step kernel in `@tf.function` / `torch.compile` / `@jax.jit` for 5–50× speed-up]

end for

end for

Output: Trained network $\mathcal{N}_{\rho^*}$ approximating the policy function; report Euler residuals on $\mathbf{x}^{\text{val}}$.

Several features of this algorithm deserve emphasis:

- **Endogenous training distribution.** As the policy network improves, the simulated paths become more accurate, and the training points concentrate on the ergodic set of the true equilibrium. This is fundamentally different from supervised learning, where the training distribution is fixed. Early in training, however, the simulated distribution is generated by a poor and rapidly changing policy: it can be unstable, infeasible, or far too



Figure 2.4: Supervised learning (left) versus DEQN training (right). Both paradigms train a parameter vector by minimizing a residual loss with SGD; the difference is the *source* of the training signal, labeled data $(\mathbf{x}_i, y_i)$ in the supervised case, structural equilibrium equations $G(\mathbf{x}, p(\mathbf{x})) = 0$ in the DEQN case. No labeled solution data are required for DEQNs. For Brock–Mirman the right-hand side specializes to $G(K, z) = 1 - \beta (C/C') \cdot \alpha z' K'^{\alpha-1}$ with $C = \mathcal{N}_\rho(K, z)$ and $K' = zK^\alpha - C$; the network learns the policy by driving the squared mean of this residual to zero on simulated trajectories.

narrow. Practical implementations therefore (i) burn in with broad sampling from the prior or a uniform box around the deterministic steady state, (ii) clip or guard against infeasible states (e.g. negative consumption), and (iii) often maintain a small replay buffer so that early states are revisited as the policy improves. Out-of-sample validation on an independent trajectory remains essential.

- **No labeled data.** The algorithm requires no “ground truth” solutions, only the structural equations of the model. The loss function has direct economic interpretation: it measures the violation of equilibrium conditions.
- **Stochastic optimization.** Mini-batch SGD naturally handles the stochasticity of the model: different mini-batches sample different states, providing implicit exploration of the state space.
- **Scalability.** The DEQN avoids explicit tensor-product state grids entirely: the per-iteration cost scales with network size, batch size, the number of equations, and the cost of integrating the conditional expectation; the input/output layer widths and the simulation step also depend on the state and shock dimensions, but only linearly. This favorable empirical scaling does not eliminate all high-dimensional costs (sample complexity, expectation accuracy, optimization difficulty), but it does mitigate the practical grid-based curse of dimensionality.
- **JIT-compile every gradient step.** In any production or classroom-scale implementation, the per-batch gradient step (lines 5–6 inside the inner loop) should be wrapped in `@tf.function` (TensorFlow), `torch.compile` (PyTorch), or `@jax.jit` (JAX). The speed-up over an un-traced Python loop is typically 5–50× for the per-step kernel, and the trace cost is amortized after the first call. All companion notebooks in this course follow this convention; un-traced Python loops are simply too slow for classroom use, let alone for research-scale runs.

Episode length T_{sim} . The simulated-trajectory length T_{sim} controls the effective size of the training set within an episode. Azinovic et al. (2022) use $T_{\text{sim}} = 10,000$ time steps (split into many short mini-batches) in their 113-dimensional 56-agent OLG benchmark. Shorter episodes (e.g., $T_{\text{sim}} = 1000$) speed up early training because the distribution is re-drawn more frequently, at the cost of higher variance; longer episodes produce smoother gradient estimates but run the risk of stale data if the policy has drifted during training. A pragmatic rule is to start short,

lengthen T_{sim} as the loss plateaus, and check out-of-sample accuracy on an *independent* simulated trajectory.

Connection to continuous-time methods. The DEQN residual loss is the discrete-time analogue of the Deep Galerkin Method (DGM) of [Sirignano and Spiliopoulos \(2018\)](#), which minimizes a PDE residual on neural-network-parameterized PDE solutions in continuous time, and of the deep BSDE solver of [Han et al. \(2018\)](#) (Han–Jentzen–E), which formulates the same problem as a backward stochastic differential equation. Within the discrete-time deep-learning toolkit, [Maliar et al. \(2021\)](#) unify lifetime-reward, Bellman-equation, and Euler-equation training into a single framework, paired with an “all-in-one” integration operator that estimates all conditional expectations from a single Monte Carlo realization; DEQN as developed here is the Euler-equation branch of that taxonomy. Chapters 7 and 8 develop the continuous-time analogues explicitly; the present chapter should be read as introducing the discrete-time machinery on which those continuous-time methods are built.

2.4 The Brock–Mirman Benchmark

Having laid out the generic training loop, we now put it to work on a concrete model. To validate the DEQN methodology, [Azinovic et al. \(2022\)](#) begin with the stochastic growth model of [Brock and Mirman \(1972\)](#), which admits a closed-form solution and therefore serves as an ideal test case.

The social planner solves:

$$\max_{\{C_t, K_{t+1}\}_{t=0}^{\infty}} \mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t \ln(C_t) \right] \quad \text{s.t.} \quad K_{t+1} + C_t = z_t K_t^\alpha, \quad \ln z_{t+1} = \varrho \ln z_t + \sigma_z \epsilon_{t+1}, \quad (2.4)$$

where $\beta \in (0, 1)$ is the discount factor, $\alpha \in (0, 1)$ the capital share, $\varrho \in [0, 1)$ the shock persistence, $\sigma_z > 0$ the shock volatility, and $\epsilon_{t+1} \sim \mathcal{N}(0, 1)$ i.i.d. innovations. Depreciation is full ($\delta = 1$).

Derivation of the Euler equation. To derive the optimality conditions, form the Lagrangian by attaching a multiplier $\beta^t \lambda_t$ to the resource constraint at each date t :

$$\mathcal{L} = \mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t \left[\ln(C_t) + \lambda_t (z_t K_t^\alpha - C_t - K_{t+1}) \right] \right]. \quad (2.5)$$

Since the planner chooses C_t and K_{t+1} at each date, we take partial derivatives with respect to each:

FOC w.r.t. C_t :

$$\frac{\partial \mathcal{L}}{\partial C_t} = \beta^t \left(\frac{1}{C_t} - \lambda_t \right) = 0 \quad \implies \quad \lambda_t = \frac{1}{C_t}. \quad (2.6)$$

FOC w.r.t. K_{t+1} : The variable K_{t+1} appears in the date- t constraint (with coefficient -1) and in the date- $(t+1)$ constraint via output $z_{t+1} K_{t+1}^\alpha$. Differentiating:

$$\frac{\partial \mathcal{L}}{\partial K_{t+1}} = \beta^t (-\lambda_t) + \beta^{t+1} \mathbb{E}_t [\lambda_{t+1} \alpha z_{t+1} K_{t+1}^{\alpha-1}] = 0. \quad (2.7)$$

Dividing both sides by β^t yields:

$$\lambda_t = \beta \mathbb{E}_t [\lambda_{t+1} \alpha z_{t+1} K_{t+1}^{\alpha-1}]. \quad (2.8)$$

Finally, substituting $\lambda_t = 1/C_t$ from (2.6) into this expression gives the Euler equation:

$$\frac{1}{C_t} = \beta \mathbb{E}_t \left[\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} \right]. \quad (2.9)$$

The economic intuition is transparent: the left-hand side is the marginal utility cost of saving one additional unit today; the right-hand side is the expected discounted marginal utility gain from the extra output $\alpha z_{t+1} K_{t+1}^{\alpha-1}$ that unit produces tomorrow. At the optimum, the planner is indifferent between consuming and saving at the margin.

Analytical solution. When depreciation is full ($\delta = 1$), this model admits a closed-form solution. One can verify by direct substitution into the Euler equation (2.9) that the optimal consumption policy is:

$$C_t^* = (1 - \beta\alpha) z_t K_t^\alpha. \quad (2.10)$$

The derivation proceeds by guessing that the value function takes the form $V(K, z) = V_0 + B \ln K + D \ln z$ (with the constant V_0 written as V_0 rather than A to avoid clashing with the TFP / cohort-count uses of A elsewhere in the script) and using the envelope theorem. Substituting this guess into the Bellman equation $V(K, z) = \max_C \{\ln C + \beta \mathbb{E}[V(K', z')]\}$ with $K' = zK^\alpha - C$ yields a system of equations for V_0 , B , and D . Matching coefficients on $\ln K$ pins down $B = \alpha/(1 - \alpha\beta)$; matching on $\ln z$ pins down $D = 1/[(1 - \alpha\beta)(1 - \beta\varrho)]$ (so D , unlike B , depends on the shock persistence); and the constant V_0 then absorbs the remaining terms. The first-order condition $1/C_t = \beta B/K_{t+1}$ together with $C_t + K_{t+1} = z_t K_t^\alpha$ delivers the linear savings rule $K_{t+1} = \beta\alpha z_t K_t^\alpha$, from which (2.10) follows immediately via the resource constraint. Notably, this closed-form solution holds regardless of the shock persistence ϱ for the savings rule itself, because z_{t+1} cancels in the ratio z_{t+1}/C_{t+1} under the linear consumption rule: $C_{t+1} \propto z_{t+1}$, so $z_{t+1}/C_{t+1} = 1/[(1 - \beta\alpha)K_{t+1}^\alpha]$ no longer depends on the next-period shock.

Persistent shocks. When $\varrho > 0$, productivity shocks are autocorrelated and the state of the economy becomes the pair (K_t, z_t) , since the current shock level z_t now carries information about future productivity. Under log utility with Cobb–Douglas production *and full depreciation* ($\delta = 1$), the analytical solution (2.10) continues to hold regardless of the persistence ϱ . The reason fits in one line: under the linear consumption rule $C_t = (1 - \beta\alpha) z_t K_t^\alpha$ we have $z_{t+1}/C_{t+1} = 1/[(1 - \beta\alpha)K_{t+1}^\alpha]$, so z_{t+1} cancels and the conditional expectation in the Euler equation (2.9) no longer depends on the next-period shock; the constant D in the value function still depends on ϱ , but D enters V and cancels in the FOC for the savings rule. However, for more general preferences (e.g., CRRA with $\gamma \neq 1$), partial depreciation ($\delta < 1$), or non-Cobb–Douglas production, the closed-form solution breaks down and numerical methods, and the DEQN approach in particular, become essential: the policy function $C(K_t, z_t)$ must be approximated rather than derived analytically. (Concretely: the deterministic companion notebook uses $\delta = 1$ so that the closed form applies and can be used as a benchmark; the stochastic companion notebook switches to $\delta = 0.1$, where it does not, and the policy is genuinely learned. Only the loss-kernel notebook of §2.9 returns to $\delta = 1$, precisely so that the closed-form savings rate $s^* = \alpha\beta$ is available as ground truth.)

DEQN formulation. We parametrize the consumption policy as $C_t = \mathcal{N}_\rho(K_t, z_t)$ and define the residual

$$G(K_t, z_t) = 1 - \beta \frac{C_t}{C_{t+1}} \cdot \alpha z_{t+1} K_{t+1}^{\alpha-1}, \quad (2.11)$$

where z_{t+1} denotes a single realization of the next-period shock and $K_{t+1} = z_t K_t^\alpha - \mathcal{N}_\rho(K_t, z_t)$. In the code listing below the network actually emits a *savings share* $s_t \in (0, 1)$ through a sigmoid head rather than C_t directly, which guarantees $C_t > 0$ and $K_{t+1} > 0$ at every training step; Section 2.5 explains the hard-vs-soft constraint split this exemplifies. Note that the Euler equation (2.9) involves the conditional expectation $\mathbb{E}_t[\cdot]$, whereas the residual G above is written for a single realization of z_{t+1} . Two approaches are common for handling this expectation:

1. **Path averaging:** draw a single z_{t+1} per state, compute G for that draw, and average G^2 over many states along the simulated path. This minimizes $\mathbb{E}_{(K_t, z_t, z_{t+1})}[G^2]$ – the squared *pathwise* residual. It is an unbiased Monte Carlo objective for that stronger loss, and Jensen’s inequality gives $(\mathbb{E}_t[G])^2 \leq \mathbb{E}_t[G^2]$. Thus $G = 0$ almost surely implies the conditional Euler equation, but the converse need not hold: a policy can have $\mathbb{E}_t[G] = 0$ while G varies with the shock. In the log/full-depreciation Brock–Mirman benchmark the closed-form policy happens to drive the pathwise residual itself to zero, so this stronger target is harmless; in richer models it is a modeling choice. This is the approach used in the Brock–Mirman code listing below. Note that the pathwise-residual-zero coincidence is special to $\delta = 1$: for $\delta < 1$ (the calibration of the stochastic notebook) a converged policy has $\mathbb{E}_\varepsilon[G \mid x] = 0$ but G itself is nonzero realization-by-realization, so path averaging then minimizes a strictly stronger objective than the conditional Euler residual.
2. **Quadrature:** for each state (K_t, z_t) , approximate $\mathbb{E}_t[\cdot]$ explicitly via deterministic nodes and weights (Section 2.6), form the residual from the estimated expectation, and then square. This targets $(\mathbb{E}_t[G])^2$ directly and is the approach used for the IRBC model of Chapter 3, where accurate expectations are critical for convergence.

Both approaches recover the analytical solution (2.10) to high accuracy, providing a rigorous validation of the methodology. Because convergence curves depend on the exact training run, random seed, and solver configuration, this manuscript does *not* include a hand-drawn convergence plot. In practice, one should report diagnostics from the *actual notebook run*: residual trajectories, held-out Euler errors, and the gap between the learned policy and the analytical benchmark. Figure 2.5 sketches the qualitative shape one should expect to see.

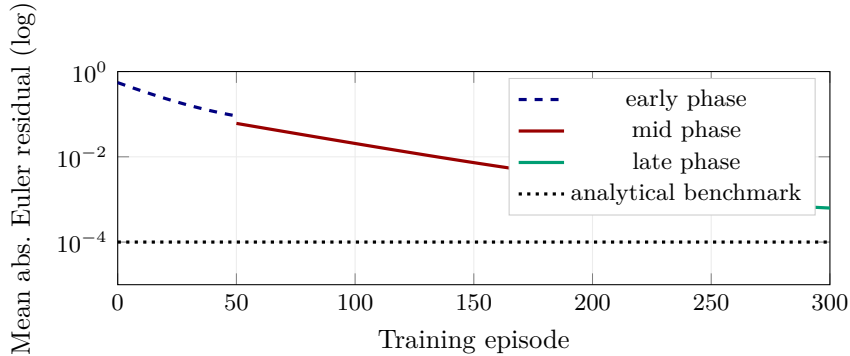


Figure 2.5: Schematic, not measured: the qualitative convergence behavior typical of a successful Brock–Mirman DEQN run. An early high-residual phase reflects an untrained network feeling out the state space; a mid phase descends roughly exponentially as the policy locks onto the equilibrium structure; a late phase plateaus near the irreducible quadrature/training-noise floor. The dotted line marks the analytical benchmark below which the residual cannot be driven without higher-precision quadrature. For the actual numbers on a specific seed, consult the companion notebook.

Relative Euler equation error. Following Judd (1998) and Azinovic et al. (2022) (Eq. 43), the standard accuracy diagnostic for a DEQN is the *relative* Euler error, which measures the percentage consumption error implied by a violation of the Euler equation. For Brock–Mirman with log utility, the relative error at state $\mathbf{x}_j = (K_j, z_j)$ is

$$e_{\mathbf{x}_j}^{\text{REE}}(\rho) = \frac{(\beta \mathbb{E}[\alpha z'(K_j')^{\alpha-1} / C'_\rho(\mathbf{x}_j') \mid \mathbf{x}_j])^{-1}}{C_\rho(\mathbf{x}_j)} - 1, \quad (2.12)$$

where $C_\rho = \mathcal{N}_\rho$, K'_j is next-period capital under the network policy, the expectation conditions on $\mathbf{x}_j$, and the $^{-1}$ inverts the marginal-utility relation $u'(c) = 1/c$. The value $e^{\text{REE}} = 10^{-4}$ means the agent's optimal consumption is mispriced by 0.01%, independent of units or utility scale. This is the metric reported in Table 3 of Azinovic et al. (2022), where the trained DEQN achieves mean relative Euler errors of order 10^{-4} on the 113-dimensional 56-agent OLG benchmark.

Why does this work so well?

The success of the DEQN approach rests on three pillars. First, neural networks are universal function approximators that can represent the smooth policy functions arising in most economic models. Second, the training distribution is endogenous: the network learns on the model's own ergodic distribution, concentrating computational effort precisely where it matters. Third, stochastic gradient descent operates directly on the economic equilibrium conditions, so the loss function has a clear economic interpretation: a pointwise relative Euler error of 10^{-4} means the consumption level implied by the Euler equation differs from the network's consumption by about 0.01%.

```

1 def deqn_loss(model, K, z, beta, alpha, z_next):
2     Y = z * K**alpha # output today
3     s = model(K, z) # savings share in (0,1)
4     via sigmoid
5     C = (1.0 - s) * Y # consumption today (>0)
6     K_next = s * Y # capital tomorrow (>0)
7     Y_next = z_next * K_next**alpha
8     s_next = model(K_next, z_next)
9     C_next = (1.0 - s_next) * Y_next # consumption tomorrow
10    # z_next: single draw; expectation via path averaging
11    G = 1.0 - beta * (C / C_next) * alpha * z_next * K_next**(alpha-1)
12    return tf.reduce_mean(G**2)

```

Listing 2.1: Representative DEQN loss for Brock–Mirman with path averaging. The network outputs a savings share $s \in (0, 1)$ via a sigmoid, which jointly enforces $C > 0$ and $K' > 0$; softplus on C alone would not, since $C > zK^\alpha$ would yield $K' < 0$ and an undefined $K'^{\alpha-1}$.

A Gauss–Hermite variant (developed formally in §2.6.2) replaces the single shock draw $\mathbf{z}_{\text{next}}$ by a Q -node weighted sum $\mathbb{E}[\alpha z' K'^{\alpha-1}/C' \mid z] \approx \sum_{q=1}^Q w_q \alpha z'_q K'^{\alpha-1}/C'(K', z'_q)$ with $z'_q = z^\rho \exp(\sigma_z \varepsilon_q)$ at the rescaled Hermite nodes ε_q ; in practice $Q = 5$ already drives the quadrature error below the training error. The autodiff companion notebook 03_Brock_Mirman_Uncertainty_AutoDiff_DEQN.ipynb implements this variant explicitly.

2.5 Encoding Equilibrium Conditions: Hard vs. Soft Constraints

The Brock–Mirman DEQN loss above already used one design choice without dwelling on it: the network emits a savings share through a sigmoid rather than consumption directly. This is an instance of a general principle. Azinovic et al. (2022) §4.2.2 do not treat every equilibrium equation symmetrically in the loss (2.3); they split the conditions into two groups. We make the split explicit on the Brock–Mirman model of the previous section and then indicate how it generalizes.

Hard constraints, encoded in the architecture (never in the loss). Some equations can be satisfied exactly for every state $\mathbf{x} = (K, z)$:

- The *resource / state-transition* equation $K_{t+1} = z_t K_t^\alpha - C_t$ defines next-period capital from the network's consumption policy. It is *not* minimized; it is evaluated as a closed-form function of $\mathbf{x}_t$ and C_t .

- The economic requirements $C_t > 0$ and $K_{t+1} > 0$ are jointly imposed by parameterizing the network's output as a *savings share* $s_t \in (0, 1)$ via a *sigmoid* activation, $\text{sigmoid}(z) = 1/(1 + e^{-z})$, and recovering both quantities in closed form from the resource constraint, $C_t = (1 - s_t) z_t K_t^\alpha$ and $K_{t+1} = s_t z_t K_t^\alpha$. A softplus head on C_t alone would guarantee $C_t > 0$ but not $K_{t+1} > 0$ (the network could output $C_t > z_t K_t^\alpha$). This sigmoid-savings parameterization removes an entire class of infeasible candidate policies before training begins; see the code listing in Section 2.4 above.

Soft constraint, minimized in the loss. The only equilibrium condition that cannot be enforced analytically is the *Euler equation* (2.9). The squared relative Euler error (2.12) is averaged over the mini-batch and driven toward zero by stochastic gradient descent.

This split is pedagogically important for three reasons: (i) only the genuinely non-closed-form conditions enter the loss, which speeds up training; (ii) it eliminates a family of bad local minima in which the network produces, e.g., slightly negative consumption or infeasible states; and (iii) it explains why the loss typically converges to a small but nonzero value even at the optimum, since the Euler residual is intrinsic and cannot be removed by re-parameterization. Figure 2.6 summarizes this construction for Brock–Mirman.

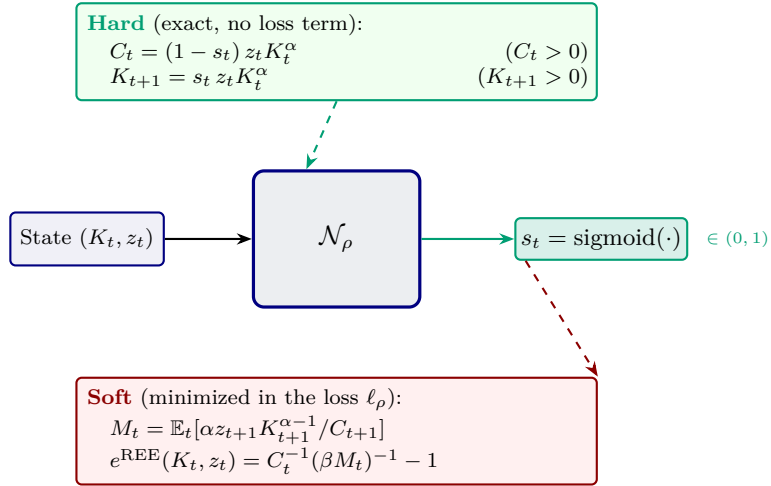


Figure 2.6: Hard vs. soft constraints in the DEQN architecture for Brock–Mirman. The network $\mathcal{N}_\rho$ reads the state (K_t, z_t) and emits a *savings share* $s_t \in (0, 1)$ via a *sigmoid* head. Both consumption $C_t = (1 - s_t) z_t K_t^\alpha$ and next-period capital $K_{t+1} = s_t z_t K_t^\alpha$ are then defined in closed form (green, top), guaranteeing $C_t > 0$ and $K_{t+1} > 0$ simultaneously; a softplus head on C_t alone could not, since $C_t > z_t K_t^\alpha$ would push $K_{t+1} < 0$ and make $K_{t+1}^{\alpha-1}$ undefined. Only the relative Euler-equation residual (red, bottom) is squared, averaged over the mini-batch, and minimized by SGD. This figure matches the code listing in Section 2.4 above. In richer models (OLG, Chapter 5) the same split extends to firm first-order conditions, household budget constraints, and KKT multipliers with softplus heads and Fischer–Burmeister complementarity residuals.

How the split generalizes. In models with more equilibrium objects (the OLG benchmark of Azinovic et al. (2022), the IRBC model of Chapter 3, the HA economies of Chapter 6), the same logic extends: firm first-order conditions for prices (w_t, r_t) are closed-form in the aggregate state, the household budget constraint algebraically determines consumption given savings, and each inequality constraint contributes a softplus-headed KKT multiplier paired with a Fischer–Burmeister complementarity residual in the soft part of the loss (Chapter 3). The Brock–Mirman case above is the minimal instance of the pattern.

Market-clearing layers. One can push this design philosophy further by encoding the *market-clearing condition itself* into the network as a dedicated output layer rather than minimizing a market-clearing residual in the loss: the network outputs unnormalized cohort savings (or shares) and a final layer rescales them so that aggregate clearing holds by construction. Azinovic-Yang and Žemlička (2024) use such a “market-clearing layer” in an OLG economy with rare disasters; this is the discrete-time counterpart of the PINN-style practice of baking conservation laws directly into the network output (Chapter 7). Sigmoid heads, softplus heads, Fischer–Burmeister residuals, and market-clearing rescaling layers thus form a small toolbox of architecture-level encodings that each move part of the equilibrium structure from the soft loss into the hard part of the network.

2.6 Quadrature Rules for Conditional Expectations

2.6.1 Why We End Up Integrating, and What Numerical Integration Is

Why an integral always shows up. Sooner or later, every dynamic stochastic model in this script presents the same algorithmic bottleneck: a forward-looking agent has to form an expectation over future shocks before any decision can be made. The Euler equation (2.9) hinges on $\mathbb{E}_t[\alpha z_{t+1} K_{t+1}^{\alpha-1} / C_{t+1}]$; the Bellman operator behind a value-function iteration evaluates $\mathbb{E}[V(\mathbf{s}') \mid \mathbf{s}, a]$ at every state and action; the relative Euler error (2.12) embeds the same expectation in the diagnostic; the integrated-assessment models of Chapter 11 require expected discounted utility under climate-shock distributions; and the structural-estimation moments of Chapter 10 are themselves expectations over simulated paths. Outside a handful of conjugate or fully-linear-quadratic-Gaussian special cases, none of these expectations admit closed-form expressions, so the integral $\mathbb{E}[g(\boldsymbol{\varepsilon})] = \int g(\boldsymbol{\varepsilon}) \mu(\boldsymbol{\varepsilon}) d\boldsymbol{\varepsilon}$ has to be approximated numerically by a finite, cheap-to-evaluate, ideally differentiable, weighted sum $\sum_q w_q g(\boldsymbol{\varepsilon}_q)$. Choosing the nodes $\boldsymbol{\varepsilon}_q$ and weights w_q well is the entire content of *quadrature theory*. The path-averaging recipe used in the Brock–Mirman code above is one such choice (a Monte Carlo sample average with one node per state along a simulated trajectory); the rest of this section makes the deterministic and quasi-random alternatives explicit and explains when each is preferable.

Picture 1: a definite integral as area, and the Riemann/midpoint sum. Strip away the economics. A definite integral $\int_a^b f(x) dx$ is the (signed) area between the graph of f and the x -axis on $[a, b]$. The simplest deterministic numerical rule, the *midpoint rule*, replaces this area by a stack of N rectangles of equal width $\Delta x = (b - a)/N$, each one as tall as f at the midpoint of its base (Figure 2.7, left). Adding the rectangle areas yields

$$I_N = \Delta x \sum_{i=1}^N f(x_i^{\text{mid}}) \xrightarrow{N \rightarrow \infty} \int_a^b f(x) dx, \quad \text{with error } |I_N - I| = \mathcal{O}(N^{-2}) \quad (2.13)$$

for smooth f . Trapezoid and Simpson rules refine the same idea by replacing each rectangle with a trapezoid or a parabolic arc and reach $\mathcal{O}(N^{-2})$ and $\mathcal{O}(N^{-4})$ respectively; Gauss–Hermite, the workhorse of §2.6.2, is the optimal rule when the integrand is multiplied by the Gaussian weight e^{-x^2} and reaches degree- $(2Q - 1)$ exactness using only Q nodes. The same tile-the-area idea generalizes to higher dimensions by laying out a Cartesian grid. With N nodes per coordinate the cost is N^d ; equivalently, with $M = N^d$ total nodes the midpoint rate becomes $\mathcal{O}(M^{-2/d})$. Notice that it is the *exponent* of the rate, not just the constant, that degrades from -2 in 1D to $-2/d$ in d -D Cartesian, the curse of dimensionality made quantitative. This is the explicit form of the curse of dimensionality flagged in Section 2.1, and it motivates both the Stroud monomial rules of §2.6.3 and the randomized methods of §2.6.4.

Picture 2: throwing darts to estimate π . An entirely different idea is to abandon the grid and approximate the integral by a *sample average*: draw N random points $\mathbf{u}_1, \dots, \mathbf{u}_N$ uniformly from the integration domain Ω and report $I_N = \text{vol}(\Omega) \cdot \frac{1}{N} \sum_{i=1}^N f(\mathbf{u}_i)$. This is plain Monte Carlo (MC), and its error decays as $\mathcal{O}(N^{-1/2})$ with a rate that is *independent of the dimension* d , although the variance constant can still worsen with dimension. This is why MC dominates deterministic grids when d is large. The cleanest illustration uses the unit square $[0, 1]^2$ and the indicator function of a quarter-disc:

$$\frac{\pi}{4} = \int_0^1 \int_0^1 \mathbf{1}[x^2 + y^2 \leq 1] dx dy \approx \frac{N_{\text{in}}}{N}, \quad \hat{\pi}_N = 4 \cdot \frac{N_{\text{in}}}{N}, \quad (2.14)$$

where N_{in} counts how many of the N uniform “darts” land inside the quarter-circle $x^2 + y^2 \leq 1$ (Figure 2.7, right). With $N = 100$ a typical run gives $\hat{\pi}_{100} \approx 3.04$ (about 3% off); with $N = 10^6$ a typical run gives $\hat{\pi}_{10^6} \approx 3.1417$ (about 10^{-4} off). The error shrinks as $\mathcal{O}(1/\sqrt{N})$, requiring a hundredfold increase in N to gain one extra decimal of accuracy, which is glacially slow compared to $\mathcal{O}(N^{-4})$ for Simpson’s rule on a smooth 1D integrand. But the MC *rate* has no dependence on the dimension of the domain: replacing the quarter-disc by a d -dimensional unit ball would leave the rate untouched, while the deterministic grid would suffer the N^d cost explosion of the curse of dimensionality. This is what makes MC and its quasi-random refinement (QMC, §2.6.4) the natural tools for the conditional expectations encountered in DEQNs at $d \gtrsim 10$.

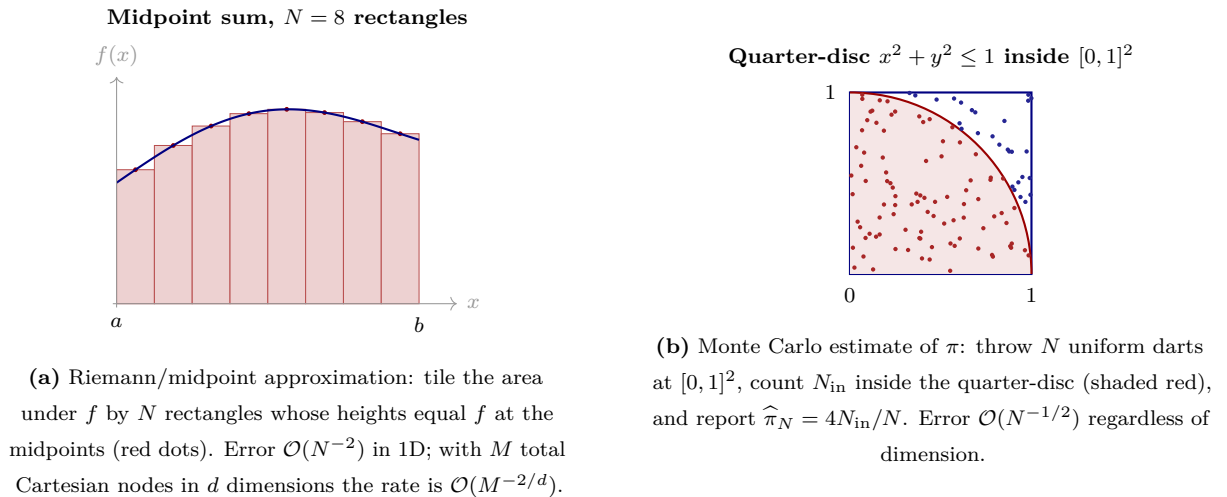


Figure 2.7: Two paradigms for numerical integration that underlie every rule in this section. Deterministic tiling (left) is exact and highly accurate at low dimension but suffers exponential cost growth in d . Random sampling (right) is dimension-independent in its error rate but slow to converge in any single dimension. The Gauss–Hermite, monomial, and QMC rules of §§2.6.2–2.6.4 are sophisticated descendants of the two ideas, designed to combine deterministic accuracy with manageable cost in d .

Where the rest of this section is going. With this picture in hand, the design of every quadrature rule in the literature can be read as an answer to two questions: (i) where do we place the nodes ε_q , and (ii) what weights w_q do we attach to them? Tensor-product Gauss–Hermite (§2.6.2) places nodes deterministically on a Cartesian grid of Hermite roots; the Stroud-3 monomial rule of §2.6.3 places only $2d$ nodes on the principal axes and accepts a controlled bias on fourth-order moments in exchange for linear-in- d scaling; the QMC construction of §2.6.4 places nodes from a low-discrepancy sequence in the unit cube and pulls them back through the inverse CDF. The cost–accuracy trade-offs differ dramatically with d , as Table 2.2 (page 64) makes concrete. Chapter 3 returns to these numbers when scaling DEQNs to multi-country economies.

2.6.2 Tensor-Product Gauss–Hermite

The classical Gauss–Hermite rule approximates integrals against the weight function e^{-x^2} :

$$\int_{-\infty}^{\infty} f(x) e^{-x^2} dx \approx \sum_{q=1}^Q \tilde{w}_q f(x_q), \quad (2.15)$$

where the Q nodes $\{x_q\}$ are the roots of the Q -th Hermite polynomial $H_Q(x)$ and the weights are $\tilde{w}_q = 2^{Q-1} Q! \sqrt{\pi} / (Q^2 [H_{Q-1}(x_q)]^2)$. For expectations under the standard normal distribution, we apply the change of variables $\varepsilon = \sqrt{2} x$ to obtain:

$$\mathbb{E}[h(\varepsilon)] = \int_{-\infty}^{\infty} h(\varepsilon) \frac{e^{-\varepsilon^2/2}}{\sqrt{2\pi}} d\varepsilon \approx \sum_{q=1}^Q w_q h(\varepsilon_q), \quad w_q = \frac{\tilde{w}_q}{\sqrt{\pi}}, \quad \varepsilon_q = \sqrt{2} x_q. \quad (2.16)$$

The 1D rule (2.16) is exact for univariate polynomials in ε of degree at most $2Q - 1$, an attractive accuracy guarantee for smooth integrands.

Worked example: Brock–Mirman Euler expectation. In the Brock–Mirman model, the conditional expectation in the Euler equation (2.9) is one-dimensional:

$$\mathbb{E}_t \left[\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} \right], \quad z_{t+1} = z_t^\theta \exp(\sigma_z \varepsilon_{t+1}), \quad \varepsilon_{t+1} \sim \mathcal{N}(0, 1). \quad (2.17)$$

Replacing path averaging by Gauss–Hermite with $Q = 5$ nodes turns the residual (2.9) into a deterministic function of (K_t, z_t) and the network parameters, eliminating the shock noise inside the loss while costing only five extra forward passes per state. This is the simplest concrete instance of the general construction below and a useful sanity check: at $Q = 5$ the quadrature error is negligible relative to the training error in this benchmark, so a well-trained network should match the analytical Brock–Mirman policy to numerical tolerance.

For shock dimension $d > 1$, the multivariate expectation is computed via a tensor product of one-dimensional rules. Each multi-dimensional quadrature node is a d -tuple $(\varepsilon_1^{q_1}, \dots, \varepsilon_d^{q_d})$ with weight $\prod_{j=1}^d w_{q_j}$. The total number of quadrature nodes grows as Q^d , which becomes prohibitive once d is large (see Judd, 1998, Section 7.7, for a textbook treatment of product rules and their cost). For $d \geq 10$, the monomial rule of §2.6.3 or *sparse-grid* (Smolyak) alternatives can substantially reduce the computational cost: Gerstner and Griebel (1998) introduce the construction in numerical analysis, Heiss and Winschel (2008) apply it to econometric likelihoods, and Brumm and Scheidegger (2017) develop adaptive sparse grids tailored to dynamic-programming and equilibrium computations. The polynomial-versus-exponential gap depends on smoothness assumptions and the effective dimension of the integrand: sparse grids attain polynomial rates for sufficiently regular functions; QMC achieves close-to- $\mathcal{O}(1/M)$ rates under randomized constructions and finite-variation / low-effective-dimension conditions (Sobol', 1967; Niederreiter, 1992; Owen, 1995; Caffisch, 1998; Novak and Woźniakowski, 2008), but neither method is universally polynomial.

2.6.3 Monomial (Stroud-3) Cubature with Linear Scaling

The exponential cost Q^d of the previous subsection is the price paid for being exact in the highest-degree univariate polynomial along *each* coordinate. In equilibrium-residual applications, however, the integrand $h(\varepsilon')$ obtained by composing the policy network with model primitives is typically only mildly nonlinear in the shock vector, and reproducing every cross-coordinate monomial of total degree up to $2Q - 1$ is overkill. Once one is willing to give up that high degree, dramatically cheaper rules become available.

The simplest such rule is the *Stroud degree-3 monomial cubature* (Stroud, 1971, formula E_n² 3-1), a cornerstone of the toolkit reviewed in Judd (1998, §7.5) and Maliar and Maliar (2014, §5.3) for large-scale dynamic models. For a d -dimensional standard normal expectation it uses only $2d$ nodes, all on the principal axes:

$$\mathbb{E}[h(\varepsilon')] \approx \frac{1}{2d} \sum_{k=1}^d \left[h(+\sqrt{d} \mathbf{e}_k) + h(-\sqrt{d} \mathbf{e}_k) \right], \quad (2.18)$$

where $\mathbf{e}_k$ is the k -th standard basis vector in $\mathbb{R}^d$. All weights are equal to $1/(2d)$, and the nodes lie at the common radius $\sqrt{d}$ from the origin (so they expand outward as the dimension grows, consistent with the fact that the bulk of a high-dimensional Gaussian sits in a thin spherical shell, recalling the dimension-of-volumes discussion in Section 2.1). At $d = 1$, the rule collapses to the two-point cubature $\{+1, -1\}$ with weights $\{\frac{1}{2}, \frac{1}{2}\}$, exact for polynomials in ε up to degree three.

Why it works (and where it stops). Rule (2.18) integrates every monomial in ε' of total degree at most three exactly: the first moments vanish by $\pm$ -symmetry, the variances $\mathbb{E}[\varepsilon_i'^2]$ all evaluate to 1 (each axis contributes $\frac{1}{2d} \cdot 2 \cdot d = 1$), the cross-moments $\mathbb{E}[\varepsilon_i' \varepsilon_j']$ ($i \neq j$) vanish because each node has only one nonzero coordinate, and the third moments vanish again by symmetry. Higher moments are not exact: $\mathbb{E}[\varepsilon_i'^4]$ is reproduced as d rather than the truth 3, so the rule introduces a controlled bias proportional to the integrand's fourth-order shock content. In the IRBC class of Chapter 3 this bias is empirically negligible at the Euler-equation tolerance one cares about ($\sim 10^{-3}$); see Pichler (2011) for a careful demonstration on a multi-country RBC and Maliar and Maliar (2014, Table 5) for a broader cost-accuracy benchmark. When higher accuracy is required, Stroud's degree-5 monomial rule with $2d^2 + 1$ nodes still scales polynomially, not exponentially (Judd, 1998, §7.5).

Cost comparison. At $Q = 3$ nodes per dimension, the tensor-product rule of §2.6.2 costs 3^d evaluations per state, while (2.18) costs $2d$. Table 2.2 contrasts the two for the dimensionalities encountered later in the script. The monomial rule is the default integration scheme behind the production DEQN code for IRBC-type models in Chapter 3 once the shock dimension exceeds about five, dropped in by replacing a single quadrature kernel.

shock dim. d	where it appears	tensor-product 3^d	monomial $2d$	speed-up
1	Brock–Mirman (Ch. 2)	3	2	$\sim 1.5\times$
3	2-country IRBC (Ch. 3)	27	6	$\sim 5\times$
6	5-country IRBC	729	12	$\sim 60\times$
11	10-country IRBC	177,147	22	$\sim 8,000\times$
21	20-country IRBC	$\sim 10^{10}$	42	$\sim 2 \times 10^8\times$
51	50-country IRBC	$\sim 10^{24}$	102	$\sim 10^{22}\times$

Table 2.2: Quadrature cost per residual evaluation, tensor-product Gauss–Hermite ($Q = 3$) versus the Stroud-3 monomial rule of equation (2.18), as a function of the shock dimension d . The tensor-product cost grows exponentially in d ; the monomial cost grows linearly. Brock–Mirman ($d = 1$) is the simplest case where the two rules agree to within a factor of 1.5; the gap explodes once the shock vector is multi-dimensional.

2.6.4 Inverse-CDF Transform and Quasi-Monte Carlo

For very high-dimensional expectations ($d \geq 10$), even sparse-grid rules can become expensive, and, when the integrand contains material fourth-order shock content, the bias of the monomial rule (2.18) starts to dominate. An effective alternative is Quasi-Monte Carlo (QMC) integration

combined with the inverse-CDF transform (see Judd, 1998, for a textbook treatment). This method maps the unbounded domain of normal shocks $\mathbb{R}^d$ to the unit hypercube $[0, 1]^d$ using the inverse normal CDF:

$$\mathbb{E}[f(\varepsilon)] \approx \frac{1}{M} \sum_{m=1}^M f(\Phi^{-1}(\mathbf{u}_m)), \quad (2.19)$$

where $\varepsilon \in \mathbb{R}^d$ is the vector of i.i.d. standard normal shocks, Φ^{-1} is the element-wise inverse standard normal CDF, and $\mathbf{u}_m$ are points from a low-discrepancy sequence (e.g., Sobol) on $[0, 1]^d$. For sufficiently smooth integrands, randomized QMC often achieves error rates close to $\mathcal{O}(1/M)$ up to logarithmic factors, compared with the $\mathcal{O}(1/\sqrt{M})$ rate of standard Monte Carlo. This makes QMC highly effective in many high-dimensional applications, including the multi-country IRBC of Chapter 3. The companion notebook `07_Genz_Approximation_and_Loss_Functions.ipynb` explores QMC integration using the Genz test functions, a standard suite of integrands for benchmarking high-dimensional quadrature methods.

2.7 Automatic Differentiation for DEQNs

The quadrature rules of the previous section answer the question “how do we evaluate the expectation in the Euler equation?” This section answers the companion question that every DEQN application faces: “how do we evaluate the *derivatives* in the Euler equation, the marginal utility $u'(C_t)$, the marginal product $\partial Y/\partial K$, and the derivative of the value function $V'(K)$?” For the textbook Brock–Mirman model of §2.4 we derived all of these by hand. But every variation of the model, whether CRRA utility in place of log, CES production in place of Cobb–Douglas, a capital adjustment cost, or a borrowing constraint, would force us to redo pages of algebra before writing the loss. Automatic differentiation (AD) removes that tax: the user writes only the *period payoff* Π of the model once, and the first-order conditions, the envelope theorem, and all higher-order sensitivities are computed exactly from the code that evaluates Π . The result is not a numerical approximation of the Euler equation: it is the textbook Euler equation, produced by a different mechanism.

2.7.1 Three ways to take a derivative

There are three generic ways to compute a derivative of a programmable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at a point x_0 : symbolic, finite differences, and autodiff. Each has a distinct profile.

Symbolic differentiation uses a computer algebra system (Mathematica, SymPy) to manipulate the algebraic expression for f into an algebraic expression for f' . Its output is exact and human-readable, and for clean analytic formulas it is ideal. Its two weaknesses are (i) *expression swell*: the chain rule applied to a deeply composed expression can produce a result with many more terms than the original, sometimes enough to defeat further symbolic manipulation; and (ii) it cannot handle arbitrary code with loops and conditional branches.

Finite differences (FD) approximates the derivative by the definition itself. For the common central-difference variant,

$$\hat{f}'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} = f'(x_0) + \frac{1}{6} f'''(x_0) h^2 + \mathcal{O}(h^4). \quad (2.20)$$

The geometric content of this formula is the mundane fact that the slope of a smooth curve at a point can be approximated by the slope of a nearby *secant line*. Figure 2.8 makes this concrete: the true tangent at x_0 (red) is approximated by the secant connecting $(x_0 - h, f(x_0 - h))$ to $(x_0 + h, f(x_0 + h))$ (blue dashed). As h shrinks the secant rotates toward the tangent, and its slope converges to $f'(x_0)$; taking h too small, however, forces the numerator $f(x_0 + h) - f(x_0 - h)$

to become the difference of two nearly equal floating-point numbers, at which point catastrophic cancellation dominates.

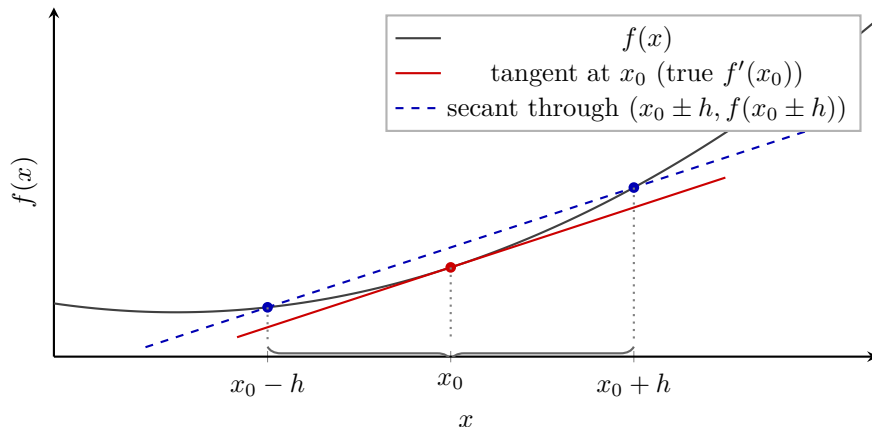


Figure 2.8: Geometric meaning of a central finite difference. The derivative $f'(x_0)$ is the slope of the tangent to f at x_0 (red). Finite differences replace this slope by the slope of the secant through the two nearby points $(x_0 - h, f(x_0 - h))$ and $(x_0 + h, f(x_0 + h))$ (blue, dashed). For small h , the secant approaches the tangent at rate $\mathcal{O}(h^2)$; for h too small, the subtraction in the numerator becomes unreliable in finite-precision arithmetic and the approximation degrades. Figure 2.9 plots the resulting error against h .

Two sources of error compete as h changes. The *truncation error* $\frac{1}{6}f'''(x_0)h^2$ shrinks as h decreases; the *rounding error* associated with computing the near-zero subtraction $f(x_0 + h) - f(x_0 - h)$ in floating-point arithmetic grows as ϵ/h , where $\epsilon \approx 2.2 \cdot 10^{-16}$ is machine precision in double precision. The total error is minimized at the step size that balances the two,

$$h^* \sim \epsilon^{1/3}, \quad \text{minimum error} \sim \epsilon^{2/3} \approx 10^{-11}. \quad (2.21)$$

The practical consequence is that central FD loses roughly *five digits of precision* relative to the input precision. For second derivatives the loss is worse: Hessians obtained by FD are usually unusable for precision-critical work. FD has one undeniable advantage: it treats f as a black box, requiring only the ability to evaluate f . For debugging or for a one-off comparative-statics check it is fine. For *training* a DEQN, where the gradient is used millions of times, it is badly suboptimal. Figure 2.9 makes the trade-off visible.

Automatic differentiation exploits the fact that every piece of Python (or any other imperative language) that evaluates a numerical function is, at run-time, a composition of *elementary operations* ($+$, $-$, $\times$, $/$, $\exp$, $\log$, $\sin$, $\cos$, ...) whose derivatives are known in closed form. The chain rule composes these local derivatives into the derivative of the whole program, exactly. The cost is a small multiple of one function evaluation, typically 2 – $4\times$ in reverse mode, and is *independent of the number of inputs*. There is no step-size h to tune, and the result is exact to machine precision. For ML-scale problems with 10^6 parameters and a scalar loss, AD is the *only* feasible method. The classic computer-science references are [Baydin et al. \(2018\)](#) and [Margossian \(2019\)](#); both motivate AD from machine-learning workloads and lay out the forward/reverse-mode taxonomy used below.

Where do local derivatives come from? A tiny built-in table. It is worth pausing on a question that beginners reasonably ask: “How does the computer know that the derivative of $\sin(x)$ is $\cos(x)$?” The answer is deflating but honest: it does not compute this from first principles. Every autodiff framework ships with a short, hand-written table of derivatives for the

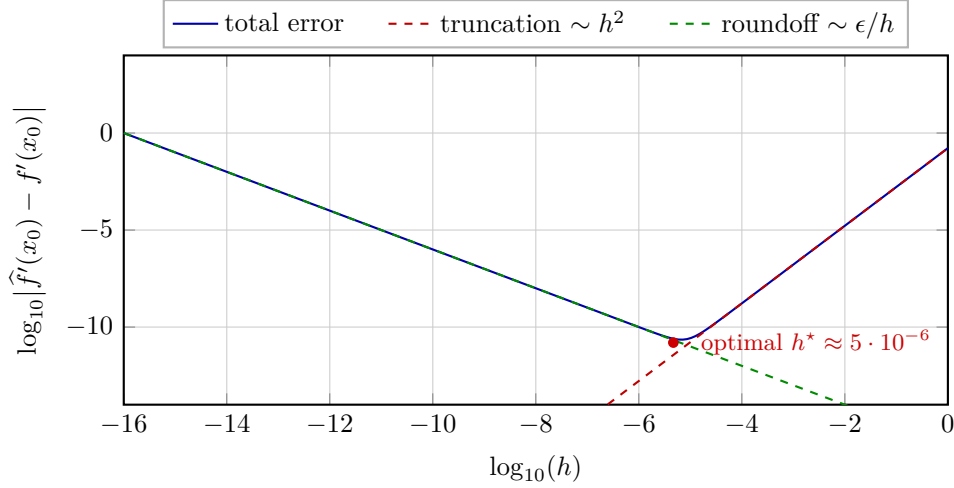


Figure 2.9: The classic U-curve of central finite differences, here for $f(x) = e^x$ at $x_0 = 1$. Truncation error (red, dashed) falls as h^2 ; roundoff error (green, dashed) grows as ϵ/h . Their sum (blue) has a minimum at $h^* \approx 5 \cdot 10^{-6}$, where the achievable precision is around 10^{-11} , roughly five digits worse than the input precision. Autodiff, by contrast, attains machine precision at no step-size tuning.

elementary operations it supports, a few dozen entries such as those in Table 2.3. When your program executes $y = \sin(x)$, the framework looks up the rule “gradient of sin is cos” in that table and records $\cos(x)$ on the edge of the computational graph. The chain rule then composes those table entries. *Autodiff is not a theorem prover*; it is a library of primitive rules plus an automatic application of the chain rule. This explains both its power (every program expressible in terms of these primitives is differentiable) and its limits (genuinely new functions require extending the table).

Primitive	Local derivative (built in)	Where to find it
$y = x + z$	$\partial y / \partial x = 1, \partial y / \partial z = 1$	<code>tf.math.add</code>
$y = x \cdot z$	$\partial y / \partial x = z, \partial y / \partial z = x$	<code>tf.math.multiply</code>
$y = x^\alpha$	$\partial y / \partial x = \alpha x^{\alpha-1}$	<code>tf.math.pow</code>
$y = \exp(x)$	$\partial y / \partial x = \exp(x)$	<code>tf.math.exp</code>
$y = \log(x)$	$\partial y / \partial x = 1/x$	<code>tf.math.log</code>
$y = \sin(x)$	$\partial y / \partial x = \cos(x)$	<code>tf.math.sin</code>
$y = \cos(x)$	$\partial y / \partial x = -\sin(x)$	<code>tf.math.cos</code>
$y = \text{ReLU}(x)$	$\partial y / \partial x = \mathbf{1}\{x > 0\}$	<code>tf.nn.relu</code>

Table 2.3: A representative slice of the primitive-derivative table that every autodiff framework ships with. TensorFlow, PyTorch, and JAX each include a few hundred such entries: one per elementary operation they support. User code composes these primitives; the chain rule composes their derivatives. When an economist writes the Brock–Mirman period payoff $\Pi(K_t, K_{t+1}) = \log(K_t^\alpha + (1 - \delta)K_t - K_{t+1})$, only entries from this table are invoked; no calculus is performed at runtime.

This explanation also makes the *limits* of AD concrete. Operations absent from the table (a black-box solver call, a non-differentiable step like sorting or `argmax`) require either a custom rule (via `@tf.custom_gradient`) or a redesign of the code. The pitfalls in §2.7.4 are essentially cases where either the table’s entry is degenerate at a point (kinks, $\partial|x|/\partial x$ at $x = 0$) or the composed gradient is numerically unstable even though each local rule is correct.

2.7.2 Computational graph; forward and reverse modes

Every numerical function, once evaluated, corresponds to a directed acyclic *computational graph* whose nodes are elementary operations and whose edges carry intermediate values. Take the toy example $y = f(x) = x^2 + \sin(x)$ evaluated at $x_0 = 2$:

$$x \rightarrow v_1 = x^2 \rightarrow y = v_1 + v_2, \quad x \rightarrow v_2 = \sin(x) \rightarrow y = v_1 + v_2.$$

The values along the graph are $v_1 = 4$, $v_2 = \sin(2) \approx 0.909$, $y \approx 4.909$. The *edges* carry local derivatives: $\partial v_1 / \partial x = 2x = 4$, $\partial v_2 / \partial x = \cos(x) \approx -0.416$, $\partial y / \partial v_1 = \partial y / \partial v_2 = 1$. AD combines the edge derivatives by the chain rule in one of two traversal orders. Figure 2.10 shows the graph together with both traversals.

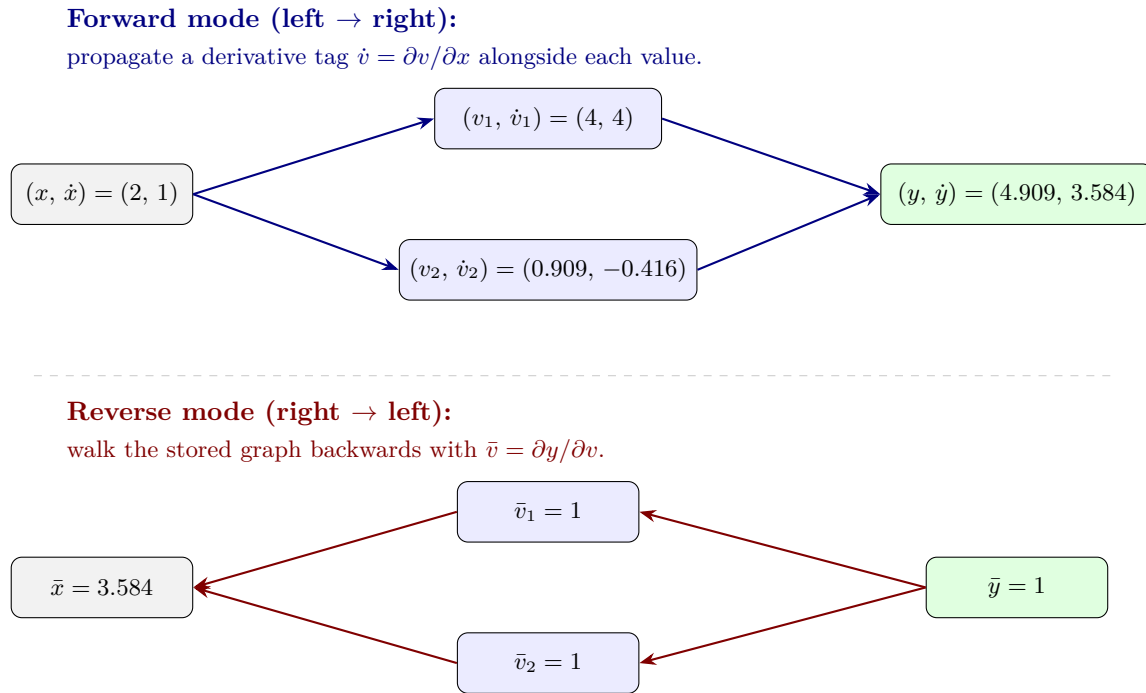


Figure 2.10: The two modes of autodiff on $y = x^2 + \sin(x)$ at $x = 2$. *Top*: forward mode carries a derivative tag $\dot{v} = \partial v / \partial x$ alongside each value and reads $\dot{y} = dy/dx$ at the output. *Bottom*: reverse mode evaluates f forward, stores the graph, then walks backwards with $\bar{v} = \partial y / \partial v$ and reads $\bar{x} = dy/dx$ at the input. Both deliver 3.584, equal to $f'(2) = 2 \cdot 2 + \cos(2)$ at machine precision. Forward mode scales linearly with the number of *inputs*; reverse mode scales linearly with the number of *outputs*, which is why it wins for DEQN training (scalar loss, many parameters).

Forward mode carries a derivative tag $\dot{v}$ next to every value v and updates both in a single forward pass through the graph. One seeds $\dot{x} = 1$ at the input; by the chain rule, $\dot{v}_1 = 4$, $\dot{v}_2 = -0.416$, and $\dot{y} = \dot{v}_1 + \dot{v}_2 = 3.584$, which is exactly $f'(2) = 2x + \cos x$ evaluated at $x = 2$. The cost is one extra float per variable and one extra pass through the graph. For a function with n inputs and m outputs, the cost of the full Jacobian is $\mathcal{O}(n)$ forward passes; forward mode is therefore attractive when *the number of inputs is small*.

Reverse mode first evaluates f forward, storing the computational graph and all intermediate values. It then walks the graph backwards, carrying a sensitivity $\bar{v} = \partial y / \partial v$ along each edge.

Seeded with $\bar{y} = 1$ at the output, the backward pass accumulates

$$\bar{v}_i = \sum_{j: v_i \rightarrow v_j} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i},$$

yielding at the input node $\bar{x} = \bar{v}_1 \cdot 4 + \bar{v}_2 \cdot (-0.416) = 1 \cdot 4 + 1 \cdot (-0.416) = 3.584$. The computational cost is $2\text{--}4\times$ a single forward pass, and, crucially, *one reverse pass produces the full gradient with respect to all inputs simultaneously*. Reverse mode is what `tf.GradientTape`, `torch.autograd`, and `jax.grad` use by default. The price paid is memory: the entire forward graph must be stored for the backward pass. For extremely long simulations this can be a binding constraint; we return to the point in the pitfalls below.

Which mode for DEQNs. The DEQN loss is a *scalar* mean-squared Euler residual, computed from a network with $n \sim 10^3$ to 10^5 parameters. Reverse mode produces the full gradient with respect to all network parameters in $2\text{--}4\times$ one forward pass, a speedup of $n/4$ over forward mode. This is the same reason reverse mode is the workhorse of deep-learning training: it exactly fits the “few outputs, many inputs” regime.

2.7.3 The autodiff Euler residual

We now apply autodiff to the very object we spent §2.4 deriving by hand: the Euler equation of the Brock–Mirman planner. Define the *period payoff* as a function of the current state and the current choice:

$$\Pi(K_{\text{in}}, K_{\text{out}}, z_{\text{in}}) = u(z_{\text{in}} K_{\text{in}}^\alpha + (1 - \delta) K_{\text{in}} - K_{\text{out}}), \quad (2.22)$$

where the three argument *slots* are the state K_{in} , the choice K_{out} , and the exogenous shock z_{in} (the same slot names the code listing below uses); applying Π at a date t means plugging K_t into slot 1, K_{t+1} into slot 2, and z_t into slot 3. This is the only place the model enters. Change u , Y , or δ and nothing else in what follows needs to move. The Bellman equation of §2.4 then reads

$$V(K_t, z_t) = \max_{K_{t+1}} \Pi(K_t, K_{t+1}, z_t) + \beta \mathbb{E}[V(K_{t+1}, z_{t+1}) | z_t]. \quad (2.23)$$

Notation: what do $\partial_1 \Pi$ and $\partial_2 \Pi$ mean? Throughout this section the subscript names the *slot* of Π being differentiated, not a time index. With the three-slot definition $\Pi(K_{\text{in}}, K_{\text{out}}, z_{\text{in}})$ of (2.22), we write

$$\partial_1 \Pi \equiv \frac{\partial \Pi}{\partial K_{\text{in}}}, \quad \partial_2 \Pi \equiv \frac{\partial \Pi}{\partial K_{\text{out}}}.$$

The first slot K_{in} is the *state*, the second slot K_{out} is the *choice*, the third slot z_{in} is an exogenous parameter and is not differentiated. An expression like $\partial_2 \Pi(K_t, K_{t+1}, z_t)$ therefore denotes the derivative of Π in its second slot, evaluated with K_t plugged into slot 1, K_{t+1} into slot 2, and z_t into slot 3, so it equals $\partial \Pi / \partial K_{t+1}$. The expression $\partial_1 \Pi(K_{t+1}, K_{t+2}, z_{t+1})$ denotes the derivative in the *first* slot, evaluated at the time- $(t+1)$ state pair, so it also equals $\partial \Pi / \partial K_{t+1}$ but for a different physical reason (because K_{t+1} now sits in the state slot of the period- $t+1$ problem). This overloading is the whole point: the same partial expression handles the FOC in one period and the envelope term in the next.

Two facts, both derived in §2.4, are now worth restating in terms of Π :

FOC w.r.t. the choice K_{t+1} :

$$\partial_2 \Pi(K_t, K_{t+1}, z_t) + \beta \mathbb{E}_{z_{t+1}|z_t}[V'(K_{t+1}, z_{t+1})] = 0.$$

Envelope at the optimum, evaluated at the state K_t :

$$V'(K_t, z_t) = \partial_1 \Pi(K_t, g(K_t, z_t), z_t),$$

where g is the optimal policy.

Substituting the envelope (evaluated one period ahead, so that K_{t+1} sits in the *state* slot) into the FOC, the familiar Euler equation becomes

$$0 = \partial_2 \Pi(K_t, K_{t+1}, z_t) + \beta \mathbb{E}_{z_{t+1}|z_t} [\partial_1 \Pi(K_{t+1}, K_{t+2}, z_{t+1})]. \quad (2.24)$$

Here $K_{t+2} = g(K_{t+1}, z_{t+1})$. Both terms in (2.24) are derivatives with respect to the same physical variable K_{t+1} , but one treats K_{t+1} as the *choice* of period t (slot 2) and the other treats K_{t+1} as the *state* of period $t+1$ (slot 1). Every term is therefore a *partial derivative of the same function* Π . Neither u' nor the marginal product of capital, nor the envelope formula $V'(K) = u'(C)(\alpha K^{\alpha-1} + 1 - \delta)$ need to be written out by hand. `tf.GradientTape` records the forward evaluation of Π and produces both partials on demand; the expectation is approximated by any of the quadrature rules of §2.6. The code is shorter than the pen-and-paper counterpart and, more importantly, entirely *model-agnostic*: swapping log for CRRA utility means editing the body of `Pi` and nothing else.

```

1 def Pi(K_in, K_out, z_in):
2     Y = z_in * K_in ** alpha
3     C = Y + (1.0 - delta) * K_in - K_out
4     return tf.math.log(C)
5
6 def euler_residual(K_t, z_t, K_tp1, K_tp2, z_tp1):
7     # FOC term:      d Pi / d K_{t+1} at (K_t, K_{t+1}, z_t)
8     with tf.GradientTape() as t1:
9         t1.watch(K_tp1)
10        pi_t = Pi(K_t, K_tp1, z_t)
11        d2 = t1.gradient(pi_t, K_tp1)
12        # Envelope term: d Pi / d K_t at (K_{t+1}, K_{t+2}, z_{t+1})
13        with tf.GradientTape() as t2:
14            t2.watch(K_tp1)
15            pi_tp1 = Pi(K_tp1, K_tp2, z_tp1)
16            d1 = t2.gradient(pi_tp1, K_tp1)
17        return d2 + beta * d1

```

Listing 2.2: Autodiff Euler residual for Brock–Mirman. The function `Pi` is the only model-specific code; the rest is generic. A full implementation lives in the autodiff chapter’s code folder, notebook `02_Brock_Mirman_AutoDiff_DEQN.ipynb`.

Cross-checking the autodiff loss. An important pedagogical point is that the autodiff residual (2.24) and the hand-derived residual of §2.4 are the same mathematical object. The companion autodiff notebooks implement both expressions on the same network and report the maximum absolute difference, which in our (seeded) runs is of order 10^{-6} – 10^{-7} in the deterministic case and 10^{-6} – 10^{-5} in the stochastic case with Gauss–Hermite quadrature (float32 arithmetic; float64 tightens this by roughly seven orders of magnitude). In every case the residual is consistent with finite-precision arithmetic, graph ordering, and quadrature accumulation rather than with any difference in the underlying mathematics. Under full depreciation ($\delta = 1$) the trained policy from the autodiff loss matches the analytical closed-form $K_{t+1} = \alpha\beta K_t^\alpha$ (respectively $\alpha\beta z_t K_t^\alpha$) to mean relative error $\sim 10^{-4}$ in the deterministic case and $\sim 10^{-3}$ on a coarse classroom grid in the stochastic case (the residual rises with the quadrature footprint), confirming that minimizing the autodiff residual recovers the true policy when one is available.

2.7.4 Pitfalls of autodiff

AD is exact and cheap but it is not magical. Five pitfalls are worth naming; each has a concrete workaround.

Non-differentiable kinks. Operations such as $\max(0, x)$ (ReLU), $|x|$, $\min$, $\max$, `argmax`, `sort`, and indicators are non-differentiable at isolated points. Frameworks return a *subgradient* at the kink – in TensorFlow and PyTorch, $\partial \text{ReLU} / \partial x|_{x=0} = 0$ by convention. If the loss repeatedly lands on such a kink, SGD can receive an uninformative gradient and stall. The cure is to either *smooth* the kink (softplus for ReLU, $\sqrt{x^2 + \varepsilon^2}$ for $|x|$, log-sum-exp for max) or, in complementarity problems, to replace the non-smooth indicator by a Fischer–Burmeister residual (Chapter 3, §3.3).

Boundary singularities. The Brock–Mirman loss contains $\log C_t$ and hence $1/C_t$. If the network’s output happens to drive C_t close to zero during training, AD dutifully returns a very large gradient; SGD then takes a very large step and the network explodes or produces NaN. Two cures of different quality are in wide use:

- *Reparameterize so the domain is respected by architecture.* In the Brock–Mirman notebooks, the network outputs the *savings share* $s \in (0, 1)$ through a sigmoid; this guarantees $C_t > 0$ and $K_{t+1} > 0$ simultaneously, at every training iteration, with no penalty term. This is the hard/soft constraint split of Figure 2.6.
- *Use numerically stable primitives.* Prefer `tf.math.log1p(x)` to `log(1+x)`, `tf.math.softplus` to a hand-coded `log(1 + ex)`, and `tf.math.xlogy(a, b)` to $a \log b$ when the $a = 0$ convention matters. AD does not see the cancellations these functions implement; the human must.

Reverse-mode memory. Reverse mode stores the entire forward graph so it can walk it backwards. Unrolling a 10,000-step simulation and back-propagating through all of it has memory cost $\mathcal{O}(T \times \text{dim state})$ and can exhaust GPU memory on non-trivial models. Standard mitigations are (i) *gradient checkpointing* (`tf.recompute_grad`), which recomputes parts of the graph on the backward pass in exchange for memory, (ii) *truncated back-propagation through time* for long recurrences, and (iii) path-averaging mini-batches of trajectory segments rather than full trajectories (notebook 02 of Day 2 already does this).

Loss of structural insight. AD returns a number, not an algebraic expression. Useful structural facts – the cancellation $u'(C_t) - \beta u'(C_{t+1})R$ has the sign of the Euler wedge, the Euler residual is homogeneous of a known degree in productivity, and so on – are invisible to AD and come from the hand derivation. In practice one should retain the ability to derive the FOC on a toy version of the model, precisely as we did in §2.4: AD scales that derivation; it does not replace the understanding it provides.

stop_gradient and the envelope theorem. A subtle point worth emphasizing. The envelope identity $V'(K) = \partial_1 \Pi(K, g(K))$ holds at the optimum because the FOC kills the term $(-u'(C) + \beta V'(g(K))) \cdot g'(K)$ that otherwise contributes to the total derivative of the composed continuation value. During training we are not yet at the optimum. Two natural codings of the next-period term therefore give different numbers: one differentiates *through* the policy g and computes a total derivative of a rollout, while the other treats $K' = g(K)$ as a fixed choice and computes the partial derivative used in the Euler FOC. The autodiff residual (2.24) uses the latter, which is what falls out of `t2.gradient(pi_tp1, K_tp1)` in Listing 2.2. This is the correct residual for checking the Euler equation; differentiating through the policy is a different object. The two codings agree as the FOC residual vanishes, but off the optimum they can disagree.

Companion notebooks. Three notebooks (in the autodiff chapter’s code folder) put the above into practice, with a fourth provided as self-study:

- **01_AutoDiff_Analytical_Examples:** warm-up ($y = x^2 + \sin x$), the FD U-curve regenerated numerically, CRRA utility’s first and second derivative, Cobb–Douglas production’s 2-D gradient field, the capital adjustment cost $\Gamma(K, K')$ with AD vs hand partials side by side, and the Hessian of Cobb–Douglas via a nested `GradientTape`.
- **02_Brock_Mirman_AutoDiff_DEQN:** the loss of (2.24) implemented on the deterministic model of §2.4; cross-check against the hand residual at float32 tolerance; cross-check of the trained policy against $K_{t+1} = \alpha\beta K_t^\alpha$.
- **03_Brock_Mirman_Uncertainty_AutoDiff_DEQN:** the same, with AR(1) productivity and Gauss–Hermite quadrature; cross-check against the stochastic hand residual and the closed-form $K_{t+1} = \alpha\beta z_t K_t^\alpha$ in a full-depreciation side-experiment.
- **04_IRBC_AutoDiff_DEQN (self-study):** the same $\partial_2\Pi + \beta\mathbb{E}[\partial_1\Pi]$ template scaled up to the multi-country IRBC model of Chapter 3 (per-country planner Lagrangian, Fischer–Burmeister irreversibility, tensor-product Gauss–Hermite), with a machine-precision cross-check against the Chapter 3 hand-derived residual.

2.8 Data Parallelization with MPI

For very large-scale applications (e.g., $N \geq 50$ countries in the IRBC model of Chapter 3), training can be accelerated by distributing the gradient computation across multiple GPUs or compute nodes. The standard approach uses synchronous data parallelism via MPI `Allreduce`; in the DEQN paper the corresponding implementation is built on *Horovod* (Sergeev and Del Balso, 2018), which wraps `Allreduce` into a drop-in replacement for the training optimizer.

Data-Parallel DEQN Training

```

Input:  $P$  workers, each with local copy of  $\mathcal{N}_\rho$ 
for each training iteration do
    Each worker  $p$  draws local mini-batch  $\mathcal{B}_p$  from simulation
    Each worker computes local gradient:  $\mathbf{g}_p = \nabla_\rho \ell(\mathcal{B}_p)$ 
    MPI_Allreduce:  $\bar{\mathbf{g}} = \frac{1}{P} \sum_{p=1}^P \mathbf{g}_p$ 
    Each worker updates:  $\rho \leftarrow \rho - \eta \bar{\mathbf{g}}$ 
end for

```

Since each worker processes an independent mini-batch and the `Allreduce` operation averages the gradients, the effective batch size scales linearly with the number of workers. In practice, this yields near-linear speedup for moderate numbers of workers ($P \leq 32$), with communication overhead becoming significant only for very large clusters. The key advantage for economics applications is that each worker can simulate its own trajectory of the economy, naturally exploring different regions of the state space and improving the diversity of the training distribution.

2.9 Choice of Loss Kernel: Beyond Mean-Squared Loss

Every DEQN derivation in this script ends with the same step: take the equilibrium residual $r(\mathbf{x})$, square it, average over a mini-batch, and minimize. The squared average, mean-squared error (MSE), is the canonical default, but it is only one of many reasonable reductions of a residual vector to a scalar. Different reductions emphasize different parts of the residual distribution and have visibly different convergence properties, even on the same model with the same network and data. This section makes the trade-off concrete on the stochastic Brock–Mirman model with

full depreciation, where the optimal savings rate is known in closed form ($s^* = \alpha\beta$) so that the deviation between learned and optimal policy can be measured exactly.

Six kernels. We compare:

- **MSE:** $\ell = \frac{1}{B} \sum r^2$. Quadratic everywhere; large residuals dominate the gradient.
- **MAE:** $\ell = \frac{1}{B} \sum |r|$. Robust to outliers, but the gradient has *constant magnitude*, so the optimizer takes the same step size regardless of how close to zero a residual already is. This is fine for learning a coarse fit, but it means MAE training plateaus at a finite “noise floor” that is several orders of magnitude above what MSE attains at the same training budget.
- **Huber:** quadratic for $|r| \leq \delta$, linear for $|r| > \delta$. Smooth hybrid that combines MSE’s fine convergence with MAE’s tail robustness, but the transition at $|r| = \delta$ is a kink in the gradient, which can introduce small training-loop oscillations as residuals migrate across the threshold.
- **Quantile pinball** at level τ : $\ell = \frac{1}{B} \sum \max(\tau r, (\tau - 1)r)$. Targets the signed τ -quantile of the residual distribution; high τ emphasizes positive residual tails, low τ emphasizes negative tails, and all observations contribute with asymmetric weights.
- **CVaR-style** at confidence α : $\ell = \text{mean of the top } (1-\alpha) \text{ fraction of } |r|$. Explicitly trains the worst-case states.
- **LogCosh:** $\ell = \frac{1}{B} \sum \log \cosh(r)$. This kernel is C^∞ everywhere and combines MSE-like behavior near $r = 0$ ($\log \cosh r \approx \frac{1}{2}r^2$) with MAE-like saturation in the tails ($\log \cosh r \approx |r| - \log 2$, gradient saturates at ± 1). Crucially, the transition between the two regimes is smooth: there is no kink at any threshold.

Convergence behavior. Figure 2.11 shows the mean, p_{90} , and p_{99} of the absolute relative Euler error on a fixed evaluation grid as a function of the training episode, for each of the six kernels. Three patterns are immediate.

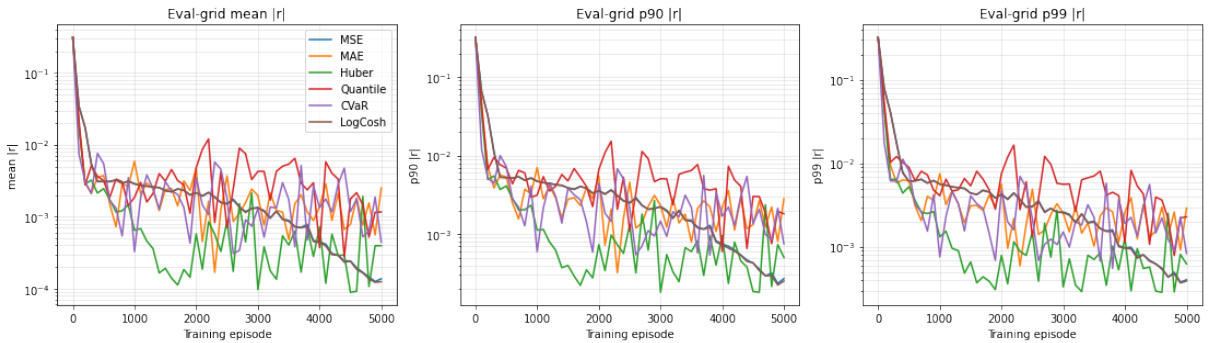


Figure 2.11: Convergence of the relative Euler-error distribution under six different loss kernels on the stochastic Brock–Mirman model with full depreciation. Same network (2×32 swish, sigmoid head), same Adam optimizer, same CRN training stream; only the loss kernel changes. Left: mean $|r|$. Middle: $p_{90} |r|$. Right: $p_{99} |r|$. All on a log scale. Closed-form benchmark: $s^* = \alpha\beta$.

First, MAE clearly stalls at a noise floor an order of magnitude above the other kernels, exactly the constant-gradient pathology described above; the curve is monotone but flattens out and refuses to drop further. *Second*, MSE and log-cosh achieve essentially the same final mean residual ($\sim 1.4 \times 10^{-4}$), and both pull the p_{99} down with comparable efficacy. *Third*, the tail-aware kernels (Huber, Quantile, CVaR) typically produce the narrowest spread between mean and max residual

but pay a small constant cost in the bulk; the practical pay-off shows up in problems where rare-but-consequential states matter.

Why log-cosh tends to converge most smoothly. Of the six kernels, log-cosh is the one whose convergence curve is essentially monotone with no spikes. This is not an accident: $\log \cosh(r)$ is a convenient smooth interpolant between $\frac{1}{2}r^2$ near zero and $|r|$ in the tails, with no kink at any threshold. Near the optimum its gradient is $\tanh(r) \approx r$ (linear, like MSE), so it inherits MSE’s fine-grained convergence. Far from the optimum its gradient saturates at ± 1 (like MAE), so an unusually large residual cannot dominate the mini-batch gradient. In effect, log-cosh is what one would design if asked for “MSE near zero and MAE in the tails, smoothly”. For practical DEQN settings where the residual distribution is unknown in advance, log-cosh is therefore a useful default to compare against MSE rather than a mere robustness afterthought.

Why this is an *economic* comparison. The relative Euler-equation residual already has a direct economic interpretation: a 1% relative Euler error translates approximately into a 1% per-period consumption error along the simulated path (Judd, 1998, §4.2). The mean / p_{90} / p_{99} panels of Figure 2.11 therefore measure, in interpretable units, the average and tail consumption mistakes that each loss kernel leaves behind. The point of the experiment is that *the training-loss ranking is not the same as the ranking on this economic metric*: a kernel that achieves a slightly higher *mean* residual but a much narrower *tail* produces smaller worst-case consumption errors, which is what matters when rare states are economically consequential (occasionally binding constraints, fat-tailed shocks, tipping risks). The training loss is the instrument; the relative Euler error along simulated paths is the criterion. Choosing the kernel with that hierarchy in mind is part of the modeling decision. The companion notebook pushes this one step further by collapsing the per-period error distribution into a single consumption-equivalent welfare loss against s^* ; readers who want a single-number summary of the trade-off should consult that table directly.

The full experiment, including a path-residual histogram, a policy-error heatmap on the $(z, \log K)$ plane, and a CE-welfare-loss summary table, is in the companion notebook `05_StochasticBM_LossComparison.ipynb`.

Extensions of the basic DEQN template. The Brock–Mirman model establishes the core DEQN recipe, but it is only the starting point. Later chapters modify the same template in several directions: Chapter 5 adds lifecycle structure and complementarity constraints, Chapter 6 replaces a finite vector of states by a histogram representation of the cross-sectional distribution, Section 6.7 then shows that one can also feed *shock histories* to the network rather than the current endogenous aggregate state, and Chapter 11 extends the method to nonstationary climate-economy models with pseudo-states. The equilibrium logic is the same in all cases: choose a network parameterization, simulate the model forward, evaluate equilibrium residuals, and update the network by gradient descent.

Code examples. The following Jupyter notebooks implement and extend the material in this chapter. Notebooks 01 and 02 illustrate the *sampling progression* that is pedagogically central to the method: 01 uses uniform random states to isolate the loss-and-architecture mechanics, while 02 replaces the exogenous grid by simulated trajectories and learns on the model’s ergodic set (§2.3); both derive the FOC and apply the envelope theorem on paper before writing the loss. Notebooks 03 and 04 introduce additional techniques (endogenous labor, a KKT-constrained labor-time ceiling encoded via a *Fischer–Burmeister* complementarity function, and a six-period OLG extension) that anticipate material developed formally in Chapters 3 and 5. Three further notebooks that re-solve the same two Brock–Mirman models with an *autodiff* loss (illustrating the template of §2.7) are placed in the autodiff-chapter code folder as the autodiff primer:

01_AutoDiff_Analytical_Examples.ipynb, 02_Brock_Mirman_AutoDiff_DEQN.ipynb, 03_Brock_Mirman_Uncertainty_AutoDiff_DEQN.ipynb.

- 01_Brock_Mirman_1972_DEQN.ipynb: deterministic Brock–Mirman; exogenous uniform sampling; hand-derived FOC + envelope.
- 02_Brock_Mirman_Uncertainty_DEQN.ipynb: stochastic Brock–Mirman ($\varrho > 0$, $\sigma_z > 0$); simulation-based sampling on the ergodic set; hand-derived FOC + envelope.
- 03_DEQN_Exercises_Blanks.ipynb: four guided exercises (endogenous labor, KKT + Fischer–Burmeister, simple OLG extension); blank version.
- 04_DEQN_Exercises_Solutions.ipynb: the same four exercises with complete solutions.
- 05_StochasticBM_LossComparison.ipynb: the stochastic Brock–Mirman is re-solved six times with identical network, optimizer, and CRN training data, and only the loss kernel changes (MSE, MAE, Huber, quantile pinball, CVaR, log-cosh). The notebook switches to full depreciation $\delta = 1$ so that the closed-form optimal savings rate $s^* = \alpha\beta$ is available, then evaluates each trained policy on the *economic* metric: the relative Euler-equation error along a simulated path, plus the consumption-equivalent welfare loss against s^* . The exercise makes concrete that the choice of training loss and the convergence of the metric we ultimately care about are not the same thing, and that tail-aware kernels (Huber, CVaR, quantile pinball) trade a small loss in the bulk for a much cleaner *p99*.

Chapter Summary

- DEQNs solve dynamic-equilibrium models by treating the equilibrium conditions $G(\mathbf{x}_t, p(\mathbf{x}_t), \mathbb{E}_t[H(\cdot)]) = 0$ as a residual loss; SGD on this loss replaces the traditional fixed-point iteration (Azinovic et al., 2022).
- The hard/soft split is the key design pattern: state transitions and budget constraints are encoded *exactly* in the network architecture (e.g. a sigmoid savings-share head); only the Euler residual is minimized in the loss. This eliminates infeasible policies and accelerates training.
- The Brock–Mirman benchmark with closed-form solution validates the methodology and is the reference example reused throughout the script.
- Pathwise and conditional-expectation residuals target different objectives: path averaging is cheap and can work well in benchmarks such as Brock–Mirman, while explicit quadrature directly targets the conditional Euler equation and is preferred when expectation accuracy is central.

Further Reading

- Azinovic et al. (2022), the foundational DEQN paper; required reading.
- Maliar et al. (2021), “all-in-one” deep learning, an alternative formulation discussed in Chapter 6.
- Fernández-Villaverde et al. (2024), broad survey of deep learning in macro, situating DEQNs against PINNs and other approaches.
- Judd (1998), the classical numerical-methods reference whose toolkit DEQNs supplement (rather than replace); especially Chapters 7 and 7.5 on Gauss–Hermite quadrature and monomial rules underlying §2.6.

- [Stroud \(1971\)](#), the canonical reference for the monomial cubature formulas of §2.6.3.
- [Pichler \(2011\)](#) and [Maliar and Maliar \(2014\)](#), monomial integration in large-scale dynamic economic models.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Core] Closed-form Brock–Mirman.** Verify that for log utility, $\delta = 1$, and AR(1) productivity $\ln z_{t+1} = \varrho \ln z_t + \sigma_z \varepsilon_{t+1}$, the optimal savings share is the constant $s^* = \alpha\beta$. Use this to check that a converged DEQN’s average sigmoid output should equal $\alpha\beta$.
2. **[Core] Hard vs. soft constraints.** Consider a softplus head on C_t alone (so $C_t > 0$ but K_{t+1} unconstrained). Construct an explicit input (K_t, z_t) for which a randomly initialized network would predict $K_{t+1} < 0$. Explain why the sigmoid-savings parameterization eliminates this failure mode.
3. **[Core] Path averaging vs. conditional expectation.** Let $G_\rho(x_t, \varepsilon_{t+1})$ denote the one-shock Euler residual under network parameters ρ . Show that, under ergodicity, the path average of G_ρ^2 converges almost surely to $\mathbb{E}_{\mu, \varepsilon}[G_\rho(x, \varepsilon)^2]$ as $T_{\text{sim}} \rightarrow \infty$. Compare this target with the conditional residual objective $\mathbb{E}_\mu[(\mathbb{E}[G_\rho(x, \varepsilon) \mid x])^2]$. Use Jensen’s inequality to explain why the pathwise squared objective is generally stronger, and discuss the finite-sample variance trade-off.
4. **[Core] Brock–Mirman with Gauss–Hermite.** In the Brock–Mirman Euler equation (2.9), replace the single-shock pathwise residual by a $Q = 5$ Gauss–Hermite expectation (§2.6.2). Write out the resulting deterministic loss in closed form, and check on a few representative (K_t, z_t) that the value matches a Monte Carlo estimate with 10^4 shock draws to four significant digits.
5. **[Core] Monomial rule by hand.** Take the Stroud-3 rule of equation (2.18) for shock dimension $d = 4$ (so 8 nodes). Verify by direct computation that the rule reproduces $\mathbb{E}[\varepsilon'_i] = 0$, $\mathbb{E}[\varepsilon_i'^2] = 1$, $\mathbb{E}[\varepsilon'_i \varepsilon'_j] = 0$ for $i \neq j$, and $\mathbb{E}[\varepsilon_i'^3] = 0$ exactly, but gives $\mathbb{E}[\varepsilon_i'^4] = d = 4$ instead of the true value 3. Show that the relative bias on the fourth moment grows linearly in d , and discuss when this matters for an Euler-equation residual.
6. **[Core] Loss-kernel selection.** Three application scenarios are described below; match each to the most appropriate loss kernel from the menu {MSE, MAE, Huber(δ), pinball loss at τ , CVaR at α , log-cosh} and justify the choice in two or three sentences. (a) “Quadrature noise occasionally produces a few large Euler residuals; we want a smooth loss that behaves quadratically near zero so gradients vanish at the optimum, but only linearly in the tails so that a single noisy point cannot dominate the gradient.” (b) “A regulator will inspect the worst 1% of residuals; we want the optimizer to drive the conditional mean above the 99th percentile down to tolerance, not the average.” (c) “We care that the *median* Euler residual is small. Tails are an artefact of badly conditioned states near the borrowing constraint and should not pull the gradient.”
7. **[Computational] Multivariate shock scaling.** Extend notebook `lecture_03_02_Brock_Mirman_Uncertainty_DEQN.ipynb` to d independent productivity shocks, $d \in \{1, 2, 4, 8\}$ (e.g., add country-level TFP terms to the production technology, all i.i.d. standard normal). For each d , train the network twice: once with tensor-product Gauss–Hermite at $Q = 3$ (3^d nodes per residual), once with the Stroud-3 rule of (2.18) ($2d$ nodes). Plot training time per epoch and final relative Euler error against d on the same axes. Confirm that the Stroud-3 cost

grows linearly while Gauss–Hermite is exponential, and identify the d at which Gauss–Hermite becomes impractical on a single GPU.

8. **[Computational] Implementation.** Modify notebook `lecture_03_02_Brock_Mirman_Uncertainty_DEQN.ipynb` to use a $\tanh$ activation instead of Swish. Does training still converge? How does the time-to-converge change?

Chapter 3

The International Real Business Cycle Model

Having established the DEQN framework on the one-dimensional Brock–Mirman model in Chapter 2, we now scale it to the multi-country international real business cycle (IRBC) model of Backus et al. (1992). This model features N countries with heterogeneous productivity, complete markets, irreversible investment, and convex capital adjustment costs. It is the standard testbed for high-dimensional solution methods in macroeconomics, and applying DEQNs to it illustrates how the framework handles high-dimensional state spaces, multiple equilibrium conditions, and complementarity constraints.

3.1 Why IRBC for Macro-Finance Research?

Beyond its computational-testbed role, the IRBC model is the workhorse framework for *open-economy asset pricing* and *international risk sharing*. Several first-order questions in macro-finance can be posed sharply within it:

- **International risk sharing.** Under complete markets and homogeneous preferences, the planner’s allocation implies perfectly correlated consumption growth across countries, whereas the data show consumption correlations that are lower than output correlations (the *consumption-correlation puzzle* of Backus et al. 1992): the benchmark BKK model predicts a cross-country consumption correlation close to 1, while the empirical correlation between the US and a typical industrial country is in the range 0.3–0.5 and is systematically below the corresponding output correlation. IRBC extensions with incomplete markets, frictions, or heterogeneous preferences are designed precisely to close this gap.
- **Asset-market structure and welfare.** Heathcote and Perri (2002) use an IRBC-style setup to quantify the welfare cost of moving from complete markets to one-bond economies (“financial autarky”), obtaining a welfare cost of the same order of magnitude as the business cycle itself. More generally, variants of this model are the standard laboratory for comparing complete vs. incomplete market structures.
- **Capital flows and current-account dynamics.** With intertemporal savings and heterogeneous productivity, the IRBC delivers persistent current-account imbalances as an equilibrium outcome rather than as a reduced-form residual. This is the starting point for the modern open-economy DSGE literature.
- **Home bias.** The frictionless benchmark of near-unit consumption correlation is the anchor against which observed portfolio home bias must be explained; Heathcote and Perri (2013) show that accounting for nontraded goods and labor-income hedging substantially narrows the gap between theory and observed portfolios.

- **Macro-financial transmission.** Adjustment costs, borrowing constraints, and Pareto weights become levers for studying how financial frictions propagate across borders. This is the class of extensions that motivates the DEQN treatment: once frictions are added, the policy functions acquire kinks and nonlinearities that are hard to handle with traditional grid-based methods.

The IRBC model is therefore an interesting substantive object, not merely a scaling test. Its combination of a clean complete-markets benchmark and rich, realistic frictions makes it a natural next step after the one-country Brock–Mirman benchmark of Chapter 2.

A calibration caveat for the puzzles above. The shock decomposition of §3.2 below, $z^{j'} = \rho_z z^j + \sigma_e(\varepsilon^j + \varepsilon^{\text{agg}})$, hard-wires a cross-country innovation correlation of exactly 1/2 for any number of countries N . The consumption-correlation and Backus–Smith puzzles cited in the bullets above should therefore be read as statements about this specific calibration: richer correlation structures, country-specific factor loadings, or fewer aggregate factors would change the quantitative bite of the puzzles in this model. This is calibration, not theory.

3.2 Model Setup

Symbol	Role	Range / sign	Calibration
γ_j	IES of country j (<i>not</i> CRRA)	> 0	$[0.25, 1.0]$ linearly spaced
τ^j	Pareto weight on country j	> 0	$(A_{\text{tfp}} - \delta)^{1/\gamma_j}$
λ_t	Aggregate resource-constraint multiplier	> 0	$\lambda_{\text{ss}} = 1$
μ_t^j	Irreversibility KKT multiplier on $I^j \geq 0$	≥ 0	0 in slack regime
A_{tfp}	TFP normalization constant	> 0	≈ 0.0559
ζ	Capital share in Cobb–Douglas	$\in (0, 1)$	0.36
Γ^j	Quadratic adjustment-cost level	≥ 0	$\kappa = 0.50$
ρ_z	TFP persistence	$\in [0, 1)$	0.95
σ_e	Innovation s.d. per component	> 0	0.01
ε^j	Idiosyncratic innovation	$\mathcal{N}(0, 1)$	i.i.d. across j, t
ε^{agg}	Aggregate innovation	$\mathcal{N}(0, 1)$	common factor
κ	Adjustment-cost intensity	≥ 0	0.50

Table 3.1: Symbol cheat-sheet for the IRBC model. Note the IES-vs-CRRA convention: here γ_j is the intertemporal elasticity, and the implied risk aversion is $1/\gamma_j$; later chapters on continuous-time HA models and climate use γ for CRRA and ψ for IES.

The international real business cycle (IRBC) model, introduced by Backus et al. (1992), extends the single-country growth model to N heterogeneous countries, each endowed with country-specific capital k^j and total factor productivity z^j . The model features complete markets, irreversible investment, and convex capital adjustment costs, and serves as the workhorse test case for high-dimensional solution methods (see, e.g., Brumm and Scheidegger, 2017). Here, we apply the DEQN methodology of Azinovic et al. (2022) to this setting.

Preferences. Each country j has CRRA utility

$$u^j(c) = \begin{cases} \frac{c^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j}, & \gamma_j \neq 1, \\ \ln c, & \gamma_j = 1, \end{cases} \quad (3.1)$$

where the intertemporal elasticity of substitution (IES) γ_j is heterogeneous across countries; risk aversion under this CRRA specification equals $1/\gamma_j$. **Notation warning:** this chapter uses γ

for the IES, while later chapters on continuous-time HA models and climate use γ for CRRA risk aversion and ψ for the IES. The convention is stated explicitly at the start of each chapter. A social planner maximizes

$$\max \sum_{t=0}^{\infty} \beta^t \mathbb{E} \left[\sum_{j=1}^N \tau^j u^j(c_t^j) \right] \quad (3.2)$$

with Pareto weights $\tau^j > 0$.

Production. Country j produces $Y^j = A_{\text{tfp}} \exp(z^j)(k^j)^\zeta$, where the total factor productivity constant A_{tfp} is calibrated to normalize the steady-state capital stock to unity. In steady state (where $z^j = 0$, $k^j = 1$, and $k^{j'} = 1$), the Euler equation implies:

$$A_{\text{tfp}} = \frac{1/\beta - 1 + \delta}{\zeta}. \quad (3.3)$$

This normalization ensures that the deterministic steady state lies at $(k^*, z^*) = (1, 0)$ for all countries, which simplifies the network's learning task and provides a natural center for the training distribution.

TFP process. Log productivity follows an AR(1) with common and idiosyncratic shocks:

$$z^{j'} = \rho_z z^j + \sigma_e(\varepsilon^j + \varepsilon^{\text{agg}}), \quad \varepsilon^j, \varepsilon^{\text{agg}} \sim \mathcal{N}(0, 1) \text{ i.i.d.}, \quad |\rho_z| < 1 \quad (3.4)$$

The persistence restriction $|\rho_z| < 1$ guarantees stationarity of the TFP process, which in turn underlies the existence of an ergodic distribution on which DEQN training samples (Section 2.3). Here σ_e is the per-component standard deviation, so the marginal innovation variance for country j is $2\sigma_e^2$ and the cross-country innovation covariance is σ_e^2 . These two facts imply a fixed cross-country innovation correlation of $1/2$ regardless of N , a direct consequence of the equal-weighted aggregate-shock decomposition $\varepsilon^j + \varepsilon^{\text{agg}}$. Asset-pricing implications (in particular the international consumption-correlation puzzle and the cyclicity of trade balances) inherit this hard-wired common-factor structure: results below should be interpreted with that calibration choice in mind. If a desired total innovation scale $\bar{\sigma}$ is targeted instead, set $\sigma_e = \bar{\sigma}/\sqrt{2}$.

Adjustment costs and irreversibility. Changing the capital stock incurs a quadratic adjustment cost:

$$\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j'}}{k^j} - 1 \right)^2, \quad (3.5)$$

with marginal derivatives that appear in the Euler equations:

$$\frac{\partial \Gamma^j}{\partial k^{j'}} = \kappa \left(\frac{k^{j'}}{k^j} - 1 \right), \quad \frac{\partial \Gamma^j}{\partial k^j} = \frac{\kappa}{2} \left(1 - \left(\frac{k^{j'}}{k^j} \right)^2 \right). \quad (3.6)$$

Note that $\partial \Gamma^j / \partial k^j$ is *negative* whenever $k^{j'} > k^j$, i.e. in expanding states. Consequently the term $-\partial \Gamma^j / \partial k^j$ that appears in the marginal product of capital below (3.15) *raises* MPK in expansion phases; a reader who plugs in $|\partial \Gamma / \partial k|$ here will introduce a sign error. Investment is irreversible: $I^j = k^{j'} - (1 - \delta)k^j \geq 0$.

Pareto-weight calibration. With heterogeneous IES γ_j , a symmetric deterministic steady state is most easily obtained by choosing the Pareto weights as

$$\tau^j = (A_{\text{tfp}} - \delta)^{1/\gamma_j}, \quad j = 1, \dots, N. \quad (3.7)$$

The derivation is a two-step inversion of the planner's first-order condition. The consumption-sharing condition (3.13) (derived in the next section from the FOC for c_t^j) reads $\tau^j (c_t^j)^{-1/\gamma_j} = \lambda_t$, so $c_t^j = (\lambda_t / \tau^j)^{-\gamma_j}$. In the deterministic steady state with the normalizations $\lambda_{ss} = 1$ and $k_{ss}^j = 1$ we want every country to consume the same amount $c_{ss}^j = A_{\text{tfp}} - \delta$ implied by the resource constraint $c_{ss}^j = Y_{ss}^j - I_{ss}^j$. Setting $(1/\tau^j)^{-\gamma_j} = A_{\text{tfp}} - \delta$ and solving for τ^j gives Eq. (3.7). The symmetric steady state thus serves as a natural anchor for training: the network's initial predictions need only match this point to avoid infeasible economies during the early simulated trajectories.

Reference calibration. Throughout the companion notebooks `lecture_04_01_IRBC_DEQN_smooth.ipynb` and `lecture_04_02_IRBC_DEQN_irreversible.ipynb`, we use the quarterly calibration summarized in Table 3.2. The implied total factor productivity and deterministic steady-state quantities can then be computed analytically.

Symbol	Name	Value	Description
β	Discount factor	0.99	Quarterly
ζ	Capital share	0.36	Cobb–Douglas
δ	Depreciation	0.01	Low quarterly rate
ρ_z	TFP persistence	0.95	Highly persistent
σ_e	Shock std. dev.	0.01	Small innovations
κ	Adjustment-cost intensity	0.50	Moderate frictions
$\gamma_{\min}$	Min IES	0.25	Risk aversion = 4
$\gamma_{\max}$	Max IES	1.00	Log utility
k^*	Steady-state capital	1.00	Normalization

Table 3.2: Reference IRBC calibration used in the companion notebook. Countries' IES values γ_j are linearly spaced in $[\gamma_{\min}, \gamma_{\max}]$. Pareto weights are computed from (3.7).

Worked steady state. Equation (3.3) is most compactly written as $A_{\text{tfp}} = (1/\beta - 1 + \delta)/\zeta$; multiplying numerator and denominator by β gives the algebraically equivalent form $A_{\text{tfp}} = (1 - \beta(1 - \delta))/(\zeta\beta)$ used below. Substituting the reference values:

$$A_{\text{tfp}} = \frac{1 - \beta(1 - \delta)}{\zeta\beta} = \frac{1 - 0.99 \cdot 0.99}{0.36 \cdot 0.99} \approx 0.0559, \quad (3.8)$$

$$Y_j^* = A_{\text{tfp}} (k^*)^\zeta \approx 0.0559, \quad I_j^* = \delta k^* = 0.01, \quad c_j^* = Y_j^* - I_j^* \approx 0.0459. \quad (3.9)$$

The aggregate resource constraint (3.18) is then satisfied country by country, $Y_j^* - I_j^* - c_j^* = 0$, as a trivial check. These numbers provide a baseline against which the trained network's predictions on an out-of-sample simulation can be compared.

3.3 The Planner's Problem and Equilibrium Conditions

The planner's problem. The social planner maximizes the weighted sum of utilities across all N countries, subject to the aggregate resource constraint (3.18), the irreversibility constraints, the production technology, and the TFP process (3.4):

$$\max_{\{c_t^j, k_{t+1}^j\}_{j,t}} \sum_{t=0}^{\infty} \beta^t \mathbb{E} \left[\sum_{j=1}^N \tau^j u^j(c_t^j) \right] \quad (3.10)$$

with Pareto weights $\tau^j > 0$ and discount factor $\beta \in (0, 1)$.

The Lagrangian. Following the same approach as in Section 2.4 for the Brock–Mirman model, we form the Lagrangian by attaching discounted multipliers to each constraint. Let $\beta^t \lambda_t$ be the multiplier on the aggregate resource constraint at date t , and $\beta^t \mu_t^j$ the multiplier on the irreversibility constraint for country j at date t . The Lagrangian is:

$$\mathcal{L} = \mathbb{E} \left[\sum_{t=0}^{\infty} \beta^t \left(\sum_{j=1}^N \tau^j \frac{(c_t^j)^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j} + \lambda_t \sum_{j=1}^N (Y_t^j + (1 - \delta)k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j) + \sum_{j=1}^N \mu_t^j (k_{t+1}^j - (1 - \delta)k_t^j) \right) \right]. \quad (3.11)$$

The planner chooses c_t^j and k_{t+1}^j for each country j and each date t . The complementary slackness conditions require $\mu_t^j \geq 0$, $I_t^j \geq 0$, and $\mu_t^j \cdot I_t^j = 0$. Two notation reminders before we differentiate. First, the irreversibility multiplier is μ_t^j , not the resource-constraint multiplier λ_t ; the two play different roles (λ_t shadow-prices the aggregate goods market; μ_t^j shadow-prices country j 's individual investment floor) and they enter the FOCs through entirely different channels. Second, $\mu_t^j \geq 0$ is the standard KKT sign: the multiplier on a $\geq$ -constraint is non-negative at the optimum, and the Fischer–Burmeister residual constructed below packages this sign restriction together with the slackness condition into a single smooth squared term that is compatible with SGD.

FOC w.r.t. c_t^j : Differentiating the Lagrangian with respect to c_t^j :

$$\frac{\partial \mathcal{L}}{\partial c_t^j} = \beta^t [\tau^j (c_t^j)^{-1/\gamma_j} - \lambda_t] = 0 \quad \implies \quad \tau^j (c_t^j)^{-1/\gamma_j} = \lambda_t. \quad (3.12)$$

This is the *consumption-sharing condition*: the planner equates the Pareto-weighted marginal utility of consumption across all countries to a common shadow price λ_t . Solving (3.12) for c_t^j :

$$c_t^j = \left(\frac{\lambda_t}{\tau^j} \right)^{-\gamma_j}. \quad (3.13)$$

This shows that all N consumption levels are determined by the single variable λ_t : a higher shadow price (resources are scarcer) lowers consumption in every country. Countries with a higher IES γ_j respond more elastically to changes in λ_t .

FOC w.r.t. k_{t+1}^j : The variable k_{t+1}^j appears in three places in the Lagrangian: (i) the date- t resource constraint with coefficient $-\lambda_t(1 + \partial \Gamma_t^j / \partial k_{t+1}^j)$, (ii) the date- t irreversibility constraint with coefficient $+\mu_t^j$, and (iii) the date- $(t+1)$ terms via output Y_{t+1}^j , depreciated capital $(1 - \delta)k_{t+1}^j$, adjustment costs Γ_{t+1}^j , and the irreversibility constraint. Differentiating and collecting terms:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial k_{t+1}^j} = & \beta^t \left[-\lambda_t \left(1 + \frac{\partial \Gamma_t^j}{\partial k_{t+1}^j} \right) + \mu_t^j \right] \\ & + \beta^{t+1} \mathbb{E}_t \left[\lambda_{t+1} \left(\frac{\partial Y_{t+1}^j}{\partial k_{t+1}^j} + (1 - \delta) - \frac{\partial \Gamma_{t+1}^j}{\partial k_{t+1}^j} \right) - \mu_{t+1}^j (1 - \delta) \right] = 0. \end{aligned} \quad (3.14)$$

Now define the *marginal product of capital* (inclusive of depreciation and adjustment cost effects):

$$\text{MPK}^j \equiv 1 - \delta + \zeta A_{\text{tfp}} \exp(z^j) (k^j)^{\zeta-1} - \frac{\partial \Gamma^j}{\partial k^j}, \quad (3.15)$$

and note from (3.6) that $\partial\Gamma_t^j/\partial k_{t+1}^j = \kappa(k_{t+1}^j/k_t^j - 1)$. Dividing (3.14) by β^t and substituting the MPK definition:

$$\lambda_t \left(1 + \frac{\partial\Gamma_t^j}{\partial k_{t+1}^j} \right) - \mu_t^j = \beta \mathbb{E}_t[\lambda_{t+1} \text{MPK}_{t+1}^j - (1 - \delta) \mu_{t+1}^j]. \quad (3.16)$$

This is the *Euler equation* for country j . The left-hand side is the cost of investing one more unit in country j 's capital: the shadow price λ_t of the resources used (scaled by the marginal adjustment cost) minus the value μ_t^j of relaxing the irreversibility constraint. The right-hand side is the expected discounted benefit: next period's shadow price times the marginal product of capital, minus the option-value loss from tightening next period's irreversibility constraint.

Relative error form. For numerical purposes, regroup (3.16) so that the cost-of-investment term $\lambda_t(1 + \partial\Gamma_t^j/\partial k_{t+1}^j)$ stands alone on the left, $\lambda_t(1 + \partial\Gamma_t^j/\partial k_{t+1}^j) = \beta \mathbb{E}_t[\lambda_{t+1} \text{MPK}_{t+1}^j - (1 - \delta) \mu_{t+1}^j] + \mu_t^j$, and divide through by it. This gives a scale-free formulation:

$$\frac{\beta \mathbb{E}_t[\lambda' \cdot \text{MPK}^{j'} - (1 - \delta) \mu^{j'}] + \mu^j}{\lambda(1 + \partial\Gamma^j/\partial k^{j'})} - 1 = 0, \quad j = 1, \dots, N. \quad (3.17)$$

This ensures that all N Euler equations are dimensionless and residuals can be interpreted directly as percentage deviations from optimality.

Aggregate resource constraint. All output is allocated to consumption, investment, and adjustment costs:

$$\sum_{j=1}^N [Y^j + (1 - \delta)k^j - k^{j'} - \Gamma^j - c^j] = 0. \quad (3.18)$$

Summary of equilibrium conditions. The complete system consists of three blocks:

1. **Consumption sharing** (3.13): determines all N consumption levels from λ_t .
2. **Euler equations** (3.17): N intertemporal optimality conditions, one per country.
3. **Aggregate resource constraint** (3.18): closes the model by equating world supply and demand.

In addition, the N irreversibility constraints are enforced via complementary slackness ($\mu^j \geq 0$, $I^j \geq 0$, $\mu^j I^j = 0$).

Fischer–Burmeister complementarity. The irreversibility constraint is enforced via a smoothed Fischer–Burmeister residual:

$$\text{FB}_\varepsilon(\mu^j, I^j) = \mu^j + I^j - \sqrt{(\mu^j)^2 + (I^j)^2 + \varepsilon^2} = 0. \quad (3.19)$$

The exact Fischer–Burmeister map is the limiting case $\text{FB}_0(\mu, I) = \mu + I - \sqrt{\mu^2 + I^2}$. Its zero set coincides with the positive axes in the (μ, I) -plane, ensuring $\mu^j \geq 0$, $I^j \geq 0$, and $\mu^j \cdot I^j = 0$ (Figure 3.1). The smoothed version with $\varepsilon > 0$ rounds the corner at the origin and is differentiable there, improving numerical conditioning at the cost of a slight relaxation of exact complementarity. The companion notebooks use $\varepsilon = 10^{-4}$ as the default; tighter values (10^{-6} – 10^{-5}) are sometimes preferred when complementarity must hold to higher accuracy, at the cost of stiffer gradients near the origin.

The complementarity conditions $\mu^j \geq 0$, $I^j \geq 0$, $\mu^j \cdot I^j = 0$ have a natural economic interpretation: when investment is strictly positive ($I^j > 0$), the irreversibility constraint is slack

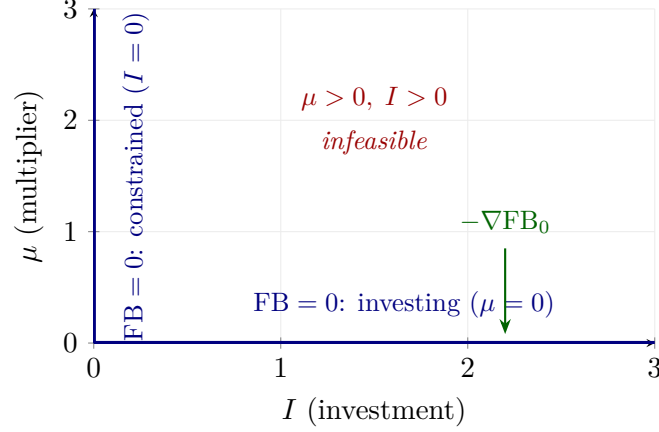


Figure 3.1: The Fischer–Burmeister complementarity function, drawn in the investment–multiplier plane: investment I^j on the horizontal axis, the irreversibility multiplier μ^j on the vertical axis. The exact map $\text{FB}_0(\mu, I) = \mu + I - \sqrt{\mu^2 + I^2}$ packs the three Karush–Kuhn–Tucker conditions $\mu \geq 0$, $I \geq 0$, $\mu I = 0$ into a single smooth equation: $\text{FB}_0 = 0$ holds *exactly* on the two heavy blue half-axes and nowhere else. The horizontal half-axis ($\mu = 0$, $I > 0$) is the *investing* regime, where the country invests a strictly positive amount, the irreversibility constraint is slack, and its shadow price μ is therefore zero. The vertical half-axis ($I = 0$, $\mu > 0$) is the *constrained* regime, where the constraint binds, investment is pinned at zero, and $\mu > 0$ measures how much the planner would pay to relax it; the origin is the knife-edge where both hold with equality. The open interior of the first quadrant ($\mu > 0$ and $I > 0$ together) is *infeasible* because it violates complementarity, and there $\text{FB}_0 > 0$ strictly (since $\mu + I > \sqrt{\mu^2 + I^2}$ whenever both are positive). This is exactly what makes the function useful as a loss term: when the network’s predicted (μ^j, I^j) lands in that forbidden region, the squared residual FB_ε^2 is positive and its negative gradient $-\nabla \text{FB}_0$ (green arrow) pushes the iterate back toward the nearest feasible half-axis, so the network learns which regime applies at each state without any explicit regime switch. The exact map has a single kink, at the origin; the smoothed version $\text{FB}_\varepsilon(\mu, I) = \mu + I - \sqrt{\mu^2 + I^2 + \varepsilon^2}$ actually used in the code rounds that corner, restoring differentiability everywhere at the price of an $\mathcal{O}(\varepsilon)$ relaxation of exact complementarity.

and the multiplier is zero ($\mu^j = 0$); conversely, when the constraint binds ($I^j = 0$), the multiplier is positive, reflecting the shadow value of the binding constraint. The FB function smoothly encodes both regimes, allowing the neural network to learn which regime applies for each state without explicit regime switching.

Why Fischer–Burmeister works so well in DEQNs

The squared FB residual converts a discrete regime-switching problem (constraint slack vs binding) into a smooth gradient field that SGD can navigate. Three properties matter: (i) the zero set of FB_0 *exactly* coincides with the KKT complementarity axes, so a converged network satisfies the constraint structure to whatever tolerance the loss is driven; (ii) the residual is smooth everywhere away from the origin, so backpropagation through it is well behaved; and (iii) the ε^2 smoothing rounds the single remaining kink at the origin, restoring differentiability there at the price of an $\mathcal{O}(\varepsilon)$ relaxation of exact complementarity. In the IRBC context, the network learns which states fall on the “investing” axis and which on the “constrained” axis without ever being told which regime applies, a major saving over methods that require manual regime indicators.

3.4 DEQN Formulation

From Brock–Mirman to IRBC. It is useful to see the IRBC as the natural extension of the one-country benchmark of Chapter 2. Table 3.3 summarizes what changes.

	Brock–Mirman (Ch. 2)	IRBC (this chapter)
Countries	1	N
States	(K, z)	$(k^1, \dots, k^N, z^1, \dots, z^N)$
Policies	C	$(k^{1'}, \dots, k^{N'}, \lambda, \mu^1, \dots, \mu^N)$
Loss terms	1 Euler	N Euler + 1 ARC + N Fischer–Burmeister
Constraints	none	irreversibility, convex adjustment costs
Shocks per period	1	$N + 1$ (one idiosyncratic per country + one aggregate)
Output activation	softplus or sigmoid	softplus
Analytical solution	yes (log utility, $\delta = 1$)	no

Table 3.3: The DEQN template is the same in both cases; only the input/output dimensions, the number of loss terms, and the presence of complementarity constraints change.

The full system of equations comprises N Euler equations, N Fischer–Burmeister conditions, and 1 aggregate resource constraint, totaling $2N + 1$ equations. Table 3.4 summarizes how the problem dimensions scale with N .

N	States	Policies	Equations	Shock dim.	GH nodes ($Q = 3$)	Stroud-3 nodes
2	4	5	5	3	27	6
5	10	11	11	6	729	12
10	20	21	21	11	1.8×10^5	22
50	100	101	101	51	$\sim 2.2 \times 10^{24}$	102
100	200	201	201	101	$\sim 1.5 \times 10^{48}$	202

Table 3.4: Scaling of the IRBC state, policy, equation, and quadrature dimensions with the number of countries N . The state, policy, and equation counts grow linearly. Tensor-product Gauss–Hermite quadrature grows as Q^{N+1} , while the Stroud-3 monomial rule uses only $2(N + 1)$ nodes; this is why the notebook uses Gauss–Hermite only for the two-country classroom case and switches to monomial or QMC rules in larger IRBC applications.

The neural network maps the full state vector $\mathbf{s} = (k^1, \dots, k^N, z^1, \dots, z^N) \in \mathbb{R}^{2N}$ to all $2N + 1$ policy variables $(k^{1'}, \dots, k^{N'}, \lambda, \mu^1, \dots, \mu^N)$ simultaneously through the small Swish–softplus network in Figure 3.3.

The hidden layers use the Swish activation $\text{swish}(x) = x \cdot \sigma(x)$, while the output layer employs the softplus function $\ln(1 + e^x)$ to keep the multipliers and capital choice positive. Two approximation caveats deserve emphasis. First, $\text{softplus}(x) > 0$ for all x , so the multipliers μ^j are strictly positive rather than exactly zero when the constraint is slack; complementarity is enforced only approximately. Second, irreversibility requires $I^j = k^{j'} - (1 - \delta)k^j \geq 0$; a softplus on $k^{j'}$ alone does *not* enforce this, since the network can output a positive $k^{j'}$ that nonetheless implies negative investment. A cleaner alternative is to output investment directly via $I^j = \text{softplus}(r^j)$ and set $k^{j'} = (1 - \delta)k^j + I^j$, which hard-enforces the constraint by construction.

The total DEQN loss aggregates the equilibrium conditions. In the smooth benchmark (companion notebook `lecture_04_01_IRBC_DEQN_smooth.ipynb`) only the Euler and aggregate-resource-constraint residuals appear:

$$\ell_\rho^{\text{smooth}} = \frac{1}{N_s} \sum_{i=1}^{N_s} \left[\sum_{j=1}^N (\text{Euler}^j(\mathbf{s}_i))^2 + (\text{ARC}(\mathbf{s}_i))^2 \right]. \quad (3.20)$$

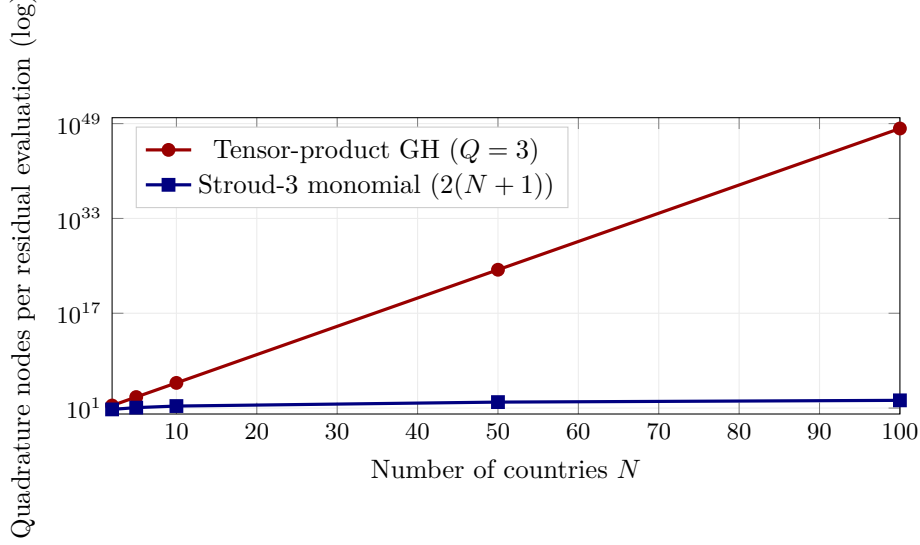


Figure 3.2: Quadrature-cost crossover for the IRBC model as a function of the number of countries N . Tensor-product Gauss–Hermite (red) grows exponentially in N and becomes infeasible by $N = 10$; the Stroud-3 monomial rule (blue) grows linearly and stays well under 10^3 nodes even at $N = 100$. This is the operational reason every IRBC application beyond the classroom $N = 2$ case uses monomial or QMC integration.

The irreversibility extension (companion notebook `lecture_04_02_IRBC_DEQN_irreversible.ipynb`) augments (3.20) with the Fischer–Burmeister complementarity block:

$$\ell_{\rho}^{\text{irrev}} = \ell_{\rho}^{\text{smooth}} + \frac{1}{N_s} \sum_{i=1}^{N_s} \sum_{j=1}^N (\text{FB}^j(\mathbf{s}_i))^2, \quad (3.21)$$

where N_s is the number of training states. When the individual loss components differ in magnitude across countries (which is typical when countries differ in size or calibration), an adaptive loss-balancing scheme from Chapter 4 (e.g., ReLoBRaLo, SoftAdapt, GradNorm) can be applied to reweight the components during training.

Representative implementation. The architecture is a 2-hidden-layer Swish network with a softplus output head. In the smooth benchmark the head has dimension $N + 1$ (the N capital choices and the resource-constraint multiplier λ); in the irreversible extension the head expands to $2N + 1$, adding the irreversibility multipliers $\mu^j \geq 0$ (softplus enforces non-negativity by construction). Only the irreversible loss carries a non-textbook line, the Fischer–Burmeister smoothing of the complementarity $0 \leq \mu^j \perp I^j \geq 0$:

```
1 def fischer_burmeister(mu, I, eps=1e-4):
2     return mu + I - tf.sqrt(mu**2 + I**2 + eps**2)
```

Listing 3.1: Fischer–Burmeister smoothing of $\mu \perp I$ (irreversible companion notebook only).

This residual is then squared elementwise and averaged across the mini-batch and across the N countries, in line with the squared-residual treatment of the Euler and ARC blocks; that elementwise square is what makes the gradient field push iterates toward the complementarity axes (see Figure 3.1). Inside the per-batch cost function of the irreversible notebook, this residual is squared and averaged alongside the Euler-equation residual (whose conditional expectation is handled by the Stroud-3 monomial rule of §2.6.3 – $2(N + 1)$ nodes for the N idiosyncratic and one aggregate shock) and the aggregate-resource-constraint residual. The smooth companion implements the same `compute_cost` pipeline with the μ^j outputs and the FB block removed.

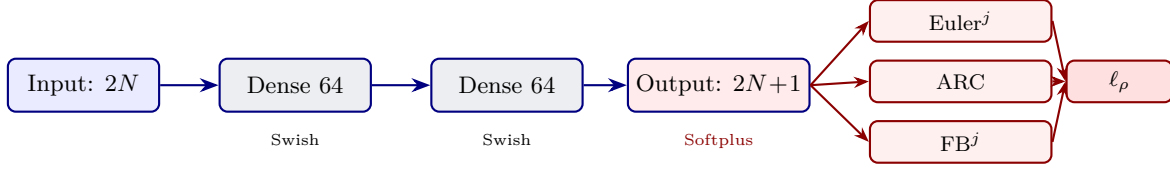


Figure 3.3: Reference network architecture used for the N -country IRBC model. The diagram shows the irreversible companion notebook (`lecture_04_02_IRBC_DEQN_irreversible.ipynb`): two hidden layers of 64 Swish units mapping the $2N$ -dimensional state to a $2N + 1$ -dimensional output (N capital choices, the resource-constraint multiplier λ , and the N irreversibility multipliers μ^j); softplus on the λ and μ^j heads enforces non-negativity, and capital choices use the bounded growth head described below. The smooth-benchmark companion (`lecture_04_01_IRBC_DEQN_smooth.ipynb`) drops the μ^j block, leaving an $N + 1$ -dimensional output head and no Fischer–Burmeister residual; in both notebooks the capital head is parameterized as the bounded log-growth $k_{t+1}^j = k_t^j \exp\{\bar{g} \tanh r_j(\mathbf{s})\}$ (smooth) or the additive form $k_{t+1}^j = (1 - \delta)k_t^j + \text{softplus}(r_j)$ (irreversible), both of which keep $k_{t+1}^j > 0$ by construction.

3.5 Persistent-Simulation Training

The companion notebooks train the IRBC DEQN with a single training pipeline: a continuing ensemble of stochastic trajectories that evolves alongside the policy network. There is no Phase 1 / Phase 2 switch and no reset to the steady state between training segments.

Persistent-Simulation Training

Maintain a vector of M stochastic trajectory heads $\mathbf{X}_t^{(1)}, \dots, \mathbf{X}_t^{(M)}$. Each *training segment* simulates these heads forward for T stochastic periods under the *current* policy network, flattens the simulated states into a training cloud of size $M \cdot T$, performs a fixed number of SGD passes on that cloud, and then continues from the segment’s terminal states $\mathbf{X}_{t+T}^{(m)}$. The trajectory ensemble therefore co-evolves with the policy and is never reset to the steady state.

What makes the single-pipeline approach feasible is that both companion notebooks parameterize the policy so that capital cannot leave the feasible set, even at random initialization. In the smooth notebook the network outputs a bounded log-growth term, $k_{t+1}^j = k_t^j \exp\{\bar{g} \tanh r_j(\mathbf{s})\}$, which keeps k_{t+1}^j strictly positive and per-period capital growth bounded by $\exp\{\pm \bar{g}\}$. In the irreversible notebook the policy network outputs an investment fraction shaped by a sigmoid head and the law of motion $k_{t+1}^j = (1 - \delta)k_t^j + I^j$ is hard-coded with $I^j \geq 0$. Either choice removes the reason historical implementations needed a uniform-sampling burn-in: the simulation cannot diverge.

A `SAMPLING_MODE` switch (`simulation` vs `exogenous`) is exposed for ablation studies and debugging, exogenous sampling on a wide box can be useful to confirm that a finding is not an artefact of the ergodic set, but the default `simulation` mode runs for the entire training horizon without a phase change.

A typical schedule on the two-country benchmark uses $M = 10$ trajectories of length $T = 256$ per segment, a batch size of 256, and one or a small number of optimizer passes per segment, with Adam at learning rate $\eta \sim 10^{-3}$ and a cosine decay; convergence is read off the diagnostics of the next section rather than off a phase-transition criterion. As a budgeting reference, the companion notebooks typically run on the order of 200–500 training segments before mean Euler errors drop below 10^{-3} on a held-out trajectory.

Why persistent-simulation training works

Three properties make the single-pipeline approach robust. First, the bounded capital-growth heads of the previous section keep the simulation feasible at every weight setting, so there is no need for a separate uniform-sampling warm-up phase to prevent the trajectories from diverging. Second, because the training cloud co-evolves with the policy, the network is always trained on states drawn from the current policy’s ergodic distribution, which is the same distribution out-of-sample evaluation will face; there is no train/test distributional shift. Third, the lack of a phase-transition criterion makes the protocol model-agnostic: scaling from $N = 2$ to $N = 10$ requires only changing the network’s input dimension and the number of equations in the loss, not redesigning the training schedule. The trade-off is that early-training states reflect a poor and rapidly changing policy, so a small replay buffer or generous mini-batch size is helpful to keep the gradient signal stable.

3.6 Results and Scalability

The DEQN approach has been successfully applied to IRBC models with up to $N = 100$ countries (200 state variables, 201 policy outputs), producing equilibrium errors below 10^{-3} in all Euler equations, a level comparable to the best existing solution methods at a fraction of the computational cost, while substantially mitigating curse-of-dimensionality effects in practice.

Convergence diagnostics. The quality of the DEQN solution is assessed using several complementary diagnostics:

1. **Euler equation errors:** For each country j , compute $\max_{\mathbf{s} \in \mathcal{S}_{\text{test}}} |\text{Euler}^j(\mathbf{s})|$. Errors below 10^{-3} indicate that the optimality condition is violated by less than 0.1% of consumption, an acceptable tolerance for most applications.
2. **Resource constraint residual:** Verify that $|\text{ARC}(\mathbf{s})| < 10^{-4}$ on the test set.
3. **Complementarity check (irreversible companion only):** Confirm that $\text{FB}^j \approx 0$ and that the multiplier μ^j is positive only when investment is at its lower bound.
4. **Economic diagnostics:** Verify that the ergodic distribution of capital, output, and consumption has sensible properties (e.g., positive trade balances for productive countries, capital flowing to high-productivity states).
5. **Policy-drift / time-invariance check:** Evaluate the policy on a fixed anchor cloud $\mathbf{X}_{\text{anchor}}$ after each monitoring interval and report `policy_drift_rms` and `policy_drift_max`. The architecture has no calendar-time input, so any fixed weight vector is a stationary recursive policy by construction; the empirical question is whether SGD has stopped moving the policy function. The run is treated as time-invariant once both drift statistics fall below the prescribed tolerances `TIME_INVARIANCE_TOL_RMS` and `TIME_INVARIANCE_TOL_MAX`.
6. **Zero-shock stochastic steady state (SSS):** Iterate the learned policy from `ZERO_SHOCK_N_STARTS` dispersed feasible starts with all shocks set to zero. A well-trained policy converges to a common point with $I^j \approx \delta k^j$ and (in the irreversible case) $\mu^j \approx 0$; the SSS is a fixed point of the learned stochastic policy that is not imposed during training.

Validation Protocol

To keep the manuscript self-contained, we summarize here the validation diagnostics used for the IRBC model:

1. **Held-out residual table.** Evaluate mean and max absolute residuals on an out-of-sample test set for each equation block (Euler and ARC always; FB only in the irreversible companion). In the two-country benchmark, typical values are mean $\sim 10^{-4}$ and max $\sim 10^{-3}$ for Euler/ARC, with smaller FB residuals.
2. **Euler-side comparison.** Compare left and right sides of the Euler equation directly on the test set (scatter around the 45-degree line). Target thresholds are mean relative error below 10^{-3} and max relative error below 10^{-2} .
3. **Constraint diagnostics (irreversible companion only).** Verify $I^j \geq 0$ everywhere and that (μ^j, I^j) lies close to the complementarity axes ($\mu^j \approx 0$ when $I^j > 0$).
4. **Economic sanity checks.** Confirm market-wide accounting identities (e.g., trade balances summing to zero), sensible consumption-sharing behavior, and stable ergodic state distributions around economically plausible regions.
5. **Policy-drift / time-invariance check.** Track `policy_drift_rms` and `policy_drift_max` on a fixed anchor cloud across training segments; flag the run as time-invariant once both drop below the prescribed tolerances. This check distinguishes “the policy has stabilized” from “the residuals are small”; both are needed for a trustworthy recursive solution.
6. **Zero-shock stochastic steady state.** Simulate the learned policy with all shocks set to zero from several dispersed feasible initial states. Convergence to a single point with $I^j \approx \delta k^j$ (and $\mu^j \approx 0$ in the irreversible case) is a coordinate-free sanity check that complements the held-out residual table.

This protocol makes solution quality auditable and comparable across model sizes and network configurations.

Policy function properties. The learned policy functions exhibit the expected economic properties. Consumption sharing follows the Pareto-weight and IES structure in (3.13): holding the common shadow price λ_t fixed, a higher Pareto weight raises country j ’s consumption, and with heterogeneous IES the consumption ratio varies with λ_t . Productivity affects consumption only indirectly through the equilibrium shadow price and the resource constraint, not through a mechanical bilateral ratio z^j/z^k . This is the textbook complete-markets prediction: the cross-country consumption ratio depends on the Pareto weights τ^j/τ^k and the IES gap, not on the productivity differential. The empirical failure of this prediction is the consumption-correlation puzzle introduced in §3.1; a closely related but distinct failure is the Backus–Smith puzzle, which concerns the correlation between relative consumption growth and the real exchange rate, predicted to be near one under complete markets but empirically near zero or even negative. Any model that aims to reproduce either puzzle has to break some of the assumptions used here (e.g. by restricting the asset menu, [Heathcote and Perri \(2002\)](#), or adding non-traded goods, [Heathcote and Perri \(2013\)](#)). Investment responds procyclically to productivity shocks: a high realization of z^j raises the marginal product of capital in country j , triggering increased investment. When the irreversibility constraint binds ($I^j = 0$), capital cannot be disinvested and the multiplier μ^j becomes positive; the network learns this regime-switching behavior smoothly through the Fischer–Burmeister loss. Trade balances adjust to channel resources toward productive countries: positive trade balances (net exports of goods) correspond to countries whose current productivity exceeds the average, and the implied capital flows are consistent with standard international macroeconomic theory.

The key advantage of the DEQN approach is its scaling behavior: while traditional Cartesian grid-based methods ([Judd, 1998](#)) exhibit exponential growth in computation time as N increases, and even adaptive sparse grid methods ([Brumm and Scheidegger, 2017](#)), which significantly mitigate the curse of dimensionality, become computationally demanding for $N > 10$, DEQN

runtimes in our implementations are reported close to linear in N over a broad range of model sizes (see [Azinovic et al. \(2022\)](#), Table 2 and surrounding discussion, for timings across $N \in \{2, \dots, 100\}$). This favorable empirical scaling arises because the network’s parameter count grows roughly linearly (more input/output neurons), while each SGD step avoids state-space grids. The companion notebooks (`lecture_04_01_IRBC_DEQN_smooth.ipynb` and `lecture_04_02_IRBC_DEQN_irreversible.ipynb`) only run the $N = 2$ case, so the linear-scaling claim cannot be reproduced from the in-class material; readers who wish to verify it directly should consult the published timings or replicate the larger- N runs from the Azinovic et al. codebase.

Comparison with adaptive sparse grids. The approach of [Brumm and Scheidegger \(2017\)](#) handles kinks in the policy function (e.g., those induced by the irreversibility constraint) by *refining* the grid locally around the kink using hierarchical surplus indicators. This keeps the method accurate but the grid remains anchored to a hypercube, so computation still scales poorly once the number of active kinks or the dimensionality grows. DEQNs do not represent kinks by grid refinement; instead, they fit a smooth approximator (Swish/softplus network) to the Fischer–Burmeister-regularized problem, which produces a globally smooth policy that tracks the true piecewise structure without needing localized grid points. The two methods are therefore complementary: adaptive sparse grids give deterministic error bounds on a hypercube; DEQNs give simulation-based error bounds on the ergodic set with no grid at all. From a theoretical perspective, [Montanelli and Du \(2019\)](#) establish error bounds showing that deep ReLU networks can approximate functions on sparse grids without the exponential growth in parameters that afflicts classical polynomial methods, providing formal underpinning for why deep learning can mitigate (though not eliminate) the high-dimensional approximation cost. Exact runtimes depend on architectural choices, quadrature design, and hardware; the robust finding is that the DEQN formulation avoids explicit tensor-product state grids and remains computationally viable in dimensions where standard methods become prohibitively expensive.

Beyond the IRBC setting, closely related neural-equilibrium methods have been applied to other policy-relevant problems. [Nuño et al. \(2024\)](#) use DEQNs to compute optimal *monetary policy rules* under persistent supply shocks, replacing the linearization step around steady state with a globally trained policy network. [Bretscher et al. \(2022\)](#) apply DEQN to multi-country international real business cycles with comparative advantage. Most recently, [Azinovic-Yang and Žemlička \(2025a\)](#) replace the endogenous cross-sectional state with a *truncated history of exogenous aggregate shocks* (the sequence-space representation), so that the network’s input dimension scales with the truncation horizon rather than with the number of agents, which is the heterogeneous-agent extension developed in Chapter 6.

Chapter Summary

- The IRBC model is the standard testbed for high-dimensional global solution methods: N countries each with capital and TFP push the state dimension to $2N$ in the planner formulation used here, and the irreversibility constraint introduces non-smooth kinks via Karush–Kuhn–Tucker complementarity.
- Fischer–Burmeister smoothing converts the non-differentiable irreversibility complementarity $0 \leq \mu^j \perp I^j \geq 0$ into a smooth squared residual Φ_ε^2 that is compatible with SGD.
- Persistent-simulation training, a single continuing ensemble of stochastic trajectories, never reset to steady state, is the recipe used in the companion notebooks; bounded policy parameterizations ($k_{t+1}^j = k_t^j \exp\{\bar{g} \tanh r_j(\mathbf{s})\}$ in the smooth notebook, $k_{t+1}^j = (1 - \delta)k_t^j + I^j$ with $I^j \geq 0$ in the irreversible notebook) keep the simulation feasible without a separate burn-in phase.
- Time-invariance via policy drift on a fixed anchor cloud, and a zero-shock stochastic steady-state check from dispersed feasible starts, are the two convergence diagnostics specific to recursive DEQN training; both are run inside the companion notebooks.
- Gauss–Hermite tensor-product quadrature (§2.6.2, introduced in the previous chapter) handles expectations in low-dimensional shock spaces; once $N \gtrsim 5$ the linear-scaling Stroud-3 monomial rule (§2.6.3) is the workhorse for IRBC, and is also what the companion notebooks use for $N = 2$, with QMC (§2.6.4) and sparse grids reserved for high-accuracy or very high-dimensional cases.

Further Reading

- Brumm and Scheidegger (2017), adaptive sparse grids for IRBC, the classical-method benchmark this chapter contrasts with.
- Pichler (2011), an IRBC-specific application of the monomial rule of §2.6.3, useful as a sanity check for the multi-country setting.
- Niederreiter (1992), the standard reference for quasi-Monte Carlo and low-discrepancy sequences for the high-dimensional integrals encountered at large N .
- Nuño et al. (2024), a recent DEQN application to optimal monetary policy.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. [Core] **Fischer–Burmeister.** (a) Show that $\Phi(a, b) = a + b - \sqrt{a^2 + b^2} = 0 \iff a \geq 0, b \geq 0, ab = 0$. (b) Plot the level set $\Phi(a, b) = 0$ and a smoothed variant $\Phi_\varepsilon(a, b) = a + b - \sqrt{a^2 + b^2 + \varepsilon^2}$ for $\varepsilon = 10^{-3}$ in the (a, b) plane. (c) Compute the gradient $\nabla\Phi(a, b) = (1 - a/\sqrt{a^2 + b^2}, 1 - b/\sqrt{a^2 + b^2})$ and evaluate it at $(a, b) = (1, 1)$. Note that the raw gradient $\nabla\Phi$ at $(1, 1)$ equals $(1 - 1/\sqrt{2}, 1 - 1/\sqrt{2}) > 0$ and therefore points *away* from the complementarity axes (northeast). Now show that gradient descent on the *squared* residual Φ^2 instead points toward the zero set: in the open positive quadrant $\Phi(a, b) > 0$, so $-\nabla\Phi^2 = -2\Phi \nabla\Phi$ points southwest, back to the L-shape. (d) Explain in two sentences why replacing Φ by $-\Phi$ has no effect on the squared loss: $\Phi^2 = (-\Phi)^2$, so the gradient field of Φ^2 is unchanged. In particular, the sign convention $a + b - \sqrt{a^2 + b^2}$ versus $\sqrt{a^2 + b^2} - a - b$ is

irrelevant once the residual is squared, and the operative quantity for SGD is $-\nabla(\Phi^2)$ rather than $-\nabla\Phi$.

2. **[Core] State-space scaling.** For an IRBC with N symmetric countries, write down the state, policy, and equation dimensions. At what N does a tensor-product Gauss–Hermite rule with $Q = 3$ nodes per dimension exceed 10^4 evaluations per Euler equation? At what N does the Stroud-3 monomial rule of §2.6.3 stay below 100 nodes?
3. **[Core] Persistent-simulation feasibility.** The companion notebooks parameterize next-period capital as $k_{t+1}^j = k_t^j \exp\{\bar{g} \tanh r_j(\mathbf{s}_t)\}$ in the smooth model and $k_{t+1}^j = (1 - \delta)k_t^j + I^j$ with $I^j \geq 0$ in the irreversible model. (a) Show that under either parameterization the simulated capital stock cannot leave the feasible set $\{k > 0\}$ even when r_j is generated by a randomly initialized network. (b) Contrast with the naive head $k_{t+1}^j = \text{softplus}(r_j)$: construct a $\mathbf{s}_t$ at which a random initialization can drive the simulation arbitrarily far from any economically sensible region in one step. (c) Explain in two sentences why feasibility-by-construction is what allows the notebooks to dispense with a separate uniform-sampling burn-in phase.
4. **[Core] Adjustment-cost partials and Tobin’s Q.** Starting from the quadratic adjustment cost (3.5), $\Gamma^j = (\kappa/2) k^j (k^{j'}/k^j - 1)^2$, derive the partial derivatives $\partial\Gamma^j/\partial k^{j'}$ and $\partial\Gamma^j/\partial k^j$ shown in (3.6). Show that both partials vanish at the steady state $k^{j'} = k^j$, so the deterministic steady-state allocation is identical to the one of the frictionless model. Define the *net investment rate* $g^j = k^{j'}/k^j - 1$ and re-express the marginal adjustment cost $\partial\Gamma^j/\partial k^{j'}$ as κg^j ; this is the Tobin’s-Q-style wedge that the planner must balance against the marginal product of capital in the Euler equation. Discuss qualitatively how raising κ changes the speed of convergence to the steady state under a positive productivity shock.
5. **[Core] Complete-markets risk sharing.** Use the consumption-sharing condition (3.13) to derive the cross-country consumption ratio c_t^i/c_t^j as a function of the Pareto weights (τ^i, τ^j) , the IES parameters (γ_i, γ_j) , and the shadow price λ_t . (i) Show that with homogeneous preferences ($\gamma_i = \gamma_j = \gamma$), the ratio c_t^i/c_t^j is *constant in time*, reproducing the perfect-risk-sharing result of Backus et al. (1992). (ii) Show that with heterogeneous IES, the ratio fluctuates with λ_t , but consumption growth rates are still scalar multiples of the same aggregate shadow-price growth when all $\gamma_j > 0$. What does this imply for cross-country consumption-growth correlations in this planner allocation? (iii) Explain in two sentences why heterogeneous IES alone does not resolve the empirical consumption-correlation puzzle.
6. **[Core] Notebook: steady-state comparative statics.** Open `lecture_05_05_IRBC_Exercise.ipynb` (it lives in the Lecture-05 code folder, since it doubles as the entry point to the NAS / loss-normalization material of Chapter 4). Working from the closed-form deterministic steady state, in which the Euler condition pins $\text{MPK} = 1/\beta$ and hence $k_{\text{ss}} = [(1/\beta - 1 + \delta)/(\zeta A_{\text{tfp}})]^{1/(\zeta-1)}$, compute k_{ss} and c_{ss} for (a) higher depreciation $\delta = 0.05$, (b) a more impatient household $\beta = 0.95$, and (c) a lower capital share $\zeta = 0.30$, each relative to the baseline. Predict the sign of each change before computing, and explain why k_{ss} falls in every case. (The notebook deliberately uses a standalone calibration $A_{\text{tfp}} = 1$; the IRBC training notebook `lecture_04_01_IRBC_DEQN_smooth.ipynb` instead pins A_{tfp} so that $k_{\text{ss}} = 1$, a rescaling that does not change any of the qualitative conclusions.)
7. **[Computational] Notebook: loss-component weighting.** In the same notebook, take a snapshot of the IRBC training loss with five components (two Euler residuals, the aggregate resource constraint, and two Fischer–Burmeister complementarity residuals) whose magnitudes span several orders ($\ell_{\text{ARC}} \sim 5$, $\ell_{\text{FB}} \sim 10^{-4}$). Replace the equal weights $w_i = 1$ with inverse-loss weights $w_i = (1/\ell_i)/\sum_j(1/\ell_j)$, verify that every weighted contribution $w_i\ell_i$ is then equal, and identify which component receives the most weight (and why). Finally, on the supplied pair

of (synthetic) equal-weight vs. inverse-weight training curves, plot both on a log axis and quantify the convergence speed-up. In which regime do you expect inverse-loss weighting to help most, and when can it hurt; think of a component that is small because it is genuinely satisfied by construction, e.g., a hard-coded resource constraint?

Chapter 4

Neural Architecture Search and Loss Normalization

The DEQN models of Chapters 2–3 involve several hyperparameters (network depth, width, activation functions, learning rate) and multi-component loss functions whose relative scales can differ by orders of magnitude. To fix ideas, even a modest sweep over depth $\in \{1, \dots, 10\}$ and width $\in \{16, 32, 64, 128, 256, 512\}$ on the companion NAS regression task already spans $10 \times 6 = 60$ configurations, and the production sweep below (six axes) reaches $\sim 3,000$; on that task (illustrative numbers), the best mean absolute error ($\approx 3 \times 10^{-3}$) is attained at a 5×256 network, while 10×512 overfits by almost an order of magnitude ($\approx 2 \times 10^{-2}$). Hand-tuning at this scale is infeasible. This chapter addresses both challenges. We first survey hyperparameter-search methods (random search, Bayesian optimization, Hyperband, and BOHB, which combines TPE with Hyperband, (Bergstra and Bengio, 2012; Snoek et al., 2012; Jamieson and Talwalkar, 2016; Li et al., 2018; Falkner et al., 2018; Garnett, 2023), and then develop a classroom-friendly version of the ReLoBRaLo algorithm (Bischof and Kraus, 2025) for adaptive multi-objective loss balancing. In the notebooks, this is implemented as a deterministic convex blend of step-wise and baseline loss comparisons, which keeps the code compact while preserving the balancing intuition.

A terminology note before we begin: in this chapter we use “NAS” loosely to cover both *hyperparameter optimization* (HPO; choosing widths, depths, activations, learning rates from a fixed parameterization) and “true” NAS in the sense of Elsken et al. (2019), where the network’s wiring graph itself is searched (e.g. regularized evolution (Real et al., 2019)). The two literatures share methodology (Random / Bayesian / Hyperband-style search) but differ in scope. All four methods discussed here are HPO; for graph-level NAS, the textbook reference is Hutter et al. (2019). Elsken et al. (2019) provide the canonical survey; the local copy in `readings/` is recommended as the first deep-dive reference.

Hands-on notebooks for this chapter. Two NAS walkthroughs are provided alongside the ReLoBRaLo notebook, plus the IRBC exercise notebook that doubles as the entry point to this chapter. All four live in the NAS chapter’s code folder:

- `02_NAS_Random_Search_10D.ipynb`: a library-free Random Search loop (model in TF/K-eras) on a 10-dimensional analytical regression task, used to illustrate the projection argument of Bergstra and Bengio (2012) in its cleanest form.
- `03_NAS_RandomSearch_Hyperband.ipynb`: from-scratch Random Search and Successive Halving (Hyperband’s inner loop) on a two-dimensional Genz Gaussian, written in ~ 25 lines of plain Python so the algorithms in this chapter are visible without library abstraction; after the first run, the cached records in `nas_results/` short-circuit re-runs for instant re-inspection.

- `04_Loss_Normalization.ipynb`: the classroom ReLoBRaLo implementation, matched to the notation below.
- `05_IRBC_Exercise.ipynb`: the IRBC exercise notebook (closed-form steady-state comparative statics and inverse-loss weighting on a multi-component IRBC residual); it is the notebook referenced by Chapter 3 Exercises 6–7, and it reuses the loss-balancing ideas of this chapter on a deliberately small, library-free example.

4.1 The Hyperparameter Space

The performance of a neural network depends sensitively on its architecture (number of layers, neurons per layer, activation functions) and training configuration (learning rate, batch size, optimizer, regularization). Choosing these hyperparameters by hand is tedious and often suboptimal.

To appreciate the scale of the search problem, consider as a stylized example a typical DEQN setup where we must select: the number of hidden layers ($L \in \{2, 3, 4, 5\}$), neurons per layer ($n \in \{32, 64, 128, 256\}$), activation function (ReLU, Swish, Tanh), learning rate ($\eta \in \{10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$), batch size ($B \in \{64, 128, 256, 512\}$), and weight decay ($\lambda \in \{0, 10^{-5}, 10^{-4}, 10^{-3}\}$). The total number of configurations is $4 \times 4 \times 3 \times 4 \times 4 \times 4 = 3,072$. If each configuration requires 30 minutes to train, exhaustive evaluation would take 64 days on a single GPU. With additional choices (optimizer type, learning rate schedule, dropout rate), the space easily exceeds 10^4 configurations. (The slide deck uses a slightly larger illustrative space, $5 \times 8 \times 3 \times 20 \times 4 = 9,600$ configurations, for the same point.)

4.2 Grid Search

The simplest approach is to define a grid of values for each hyperparameter and evaluate all combinations. For d hyperparameters, each taking k values, the cost is $\mathcal{O}(k^d)$, the same exponential scaling that plagues grid-based PDE solvers. Grid search is deterministic and easy to implement, but it has a fundamental flaw: it allocates the same density of evaluations to all hyperparameters, including those to which performance is insensitive. If only 2 out of 6 hyperparameters matter (which is typical in practice), the remaining 4 dimensions contribute only wasted computation.

4.3 Random Search

Bergstra and Bengio (2012) demonstrated that random sampling of hyperparameter configurations often outperforms grid search, particularly when only a few hyperparameters are important. The key insight is a *projection argument*: when a random configuration is projected onto any single hyperparameter axis, the marginal distribution covers the entire range densely, regardless of how many other hyperparameters exist. In contrast, a grid with the same total number of evaluations provides only $k = N^{1/d}$ distinct values per axis, which can be very coarse in high dimensions.

For example, with a budget of $N = 60$ evaluations in $d = 6$ dimensions, a grid provides only $60^{1/6} \approx 2$ values per hyperparameter, while random search provides 60 distinct values per hyperparameter (in the marginal sense). This makes random search much more likely to find good values for the hyperparameters that matter most. Figure 4.1 shows the same projection argument in two dimensions.

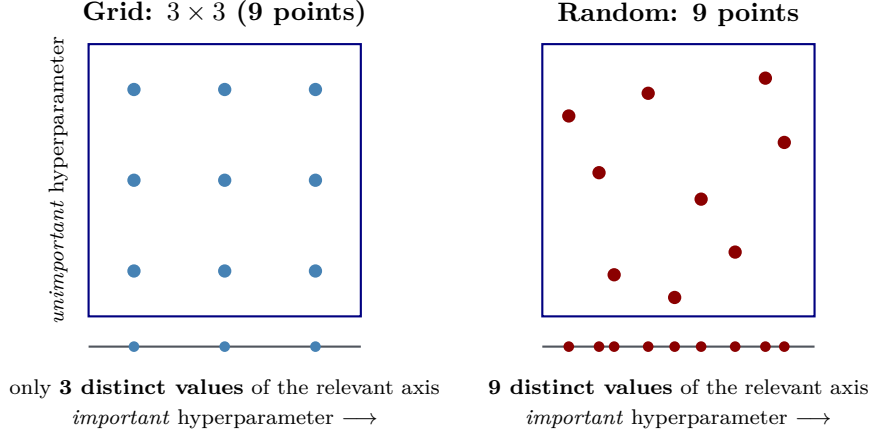


Figure 4.1: Why random search beats grid search when only one hyperparameter matters. Both designs spend the same budget of nine evaluations on a two-dimensional space in which only the horizontal axis affects performance; the vertical axis is a “nuisance” hyperparameter. Project each design onto that important axis (the strip below each panel): the 3×3 grid stacks its nine points into only three columns, so it probes just three distinct values of the parameter that matters, whereas the random design lands on nine distinct values. Equivalently, with a budget of N points in d dimensions a grid resolves only $N^{1/d}$ values per axis no matter which axes matter, while a random design resolves N values per axis in the marginal sense. Since in practice only two or three of many hyperparameters typically drive performance (Bergstra and Bengio, 2012), the random design extracts far more information about the dimensions that count for the same compute.

4.4 Bayesian Optimization

Bayesian optimization (BO) treats the validation loss $\ell(\mathbf{h})$ as an expensive black-box function of the hyperparameter vector $\mathbf{h}$, and uses a probabilistic surrogate, fitted to the configurations evaluated so far, to decide where to evaluate next (Snoek et al., 2012). The standard surrogate is a Gaussian process (GP). The next subsection gives a self-contained primer in the notation that Chapter 9 will reuse, so the reader can follow the BO logic without flipping ahead; the full kernel zoo, marginal-likelihood hyperparameter learning, and Bayesian active learning are developed there. Throughout this section the GP input is the hyperparameter vector itself, $\mathbf{x} \equiv \mathbf{h}$, and the kernel length-scale symbol ℓ that appears in (4.2) below is the standard GP notation of Chapter 9; it is distinct from the validation-loss symbol $\ell(\mathbf{h})$ and the two are always clear from context.

4.4.1 A Brief Introduction to Gaussian Processes

A Gaussian process is a probability distribution over real-valued functions, equivalently, a random function f whose values at any finite collection of inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$ are jointly Gaussian (Rasmussen and Williams, 2005). It is fully specified by a mean function $\mu(\mathbf{x})$ and a covariance (kernel) function $k(\mathbf{x}, \mathbf{x}')$ that says how correlated the function’s values at two inputs are:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')). \quad (4.1)$$

This means that, for any finite test set $\mathbf{x}_1, \dots, \mathbf{x}_n$, the vector $\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_n))^\top$ is distributed as $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, K)$ with kernel matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. For concreteness, we use the squared-exponential (RBF) kernel throughout this section:

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \quad (4.2)$$

where ℓ is the kernel length scale (the distance in input space over which two function values remain correlated) and σ_f^2 is the signal variance (the typical squared amplitude of the function). Both are kernel hyperparameters that one would normally fit by marginal-likelihood maximization; here we treat them as fixed at sensible values, since BO calls the GP once per outer iteration and modest mis-tuning only changes the candidate ranking marginally.

Posterior conditioning, in two formulas. Given n training configurations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with possibly noisy outputs $y_i = f(\mathbf{x}_i) + \varepsilon_i$ and $\varepsilon_i \sim \mathcal{N}(0, \sigma_y^2)$, write $\mathbf{y} = (y_1, \dots, y_n)^\top$. Three matrices and one vector enter the posterior: the kernel matrix $K \in \mathbb{R}^{n \times n}$ with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$; its noise-augmented version $K_y = K + \sigma_y^2 I$; the prior-mean vector $\boldsymbol{\mu}_X$ whose i -th entry is $\mu(\mathbf{x}_i)$; and the cross-covariance $\mathbf{k}_* \in \mathbb{R}^n$ whose i -th entry is $k(\mathbf{x}_*, \mathbf{x}_i)$, where $\mathbf{x}_*$ is any query point. Conditioning the joint Gaussian $(\mathbf{y}, f(\mathbf{x}_*))$ on the observations yields a Gaussian posterior over the latent value $f(\mathbf{x}_*)$, with closed-form mean and variance:

$$\bar{f}_* = \mu(\mathbf{x}_*) + \mathbf{k}_*^\top K_y^{-1}(\mathbf{y} - \boldsymbol{\mu}_X), \quad (4.3)$$

$$\sigma_{f,*}^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top K_y^{-1} \mathbf{k}_*. \quad (4.4)$$

These two equations carry all of the GP intuition we will use below. In the noiseless limit $\sigma_y^2 \rightarrow 0$, the posterior mean $\bar{f}_*$ passes exactly through every observation, so the GP is an interpolator, and the posterior variance $\sigma_{f,*}^2$ collapses to zero at observation points and grows in the gaps between them. The pair $\bar{f}_* \pm 2\sigma_{f,*}$ therefore traces an honest error bar: tight where the data are dense and loose where they are sparse. Chapter 9 derives (4.3)–(4.4) step by step, works a hand-traceable 1D example, and explains the marginal-likelihood Occam’s razor that selects $(\ell, \sigma_f, \sigma_y)$ from data; for the BO logic that follows, (4.3)–(4.4) are sufficient.

4.4.2 Expected Improvement

We now plug the GP posterior into a decision rule for choosing the next configuration to evaluate. Specialize the GP input to the hyperparameter vector, $\mathbf{x} = \mathbf{h}$, and treat each observation $y_i = \ell_i = \ell(\mathbf{h}_i)$ as the validation loss at the i -th evaluated configuration. At any untried $\mathbf{h}$ the GP returns a posterior mean $\bar{f}(\mathbf{h})$ and posterior standard deviation $\sigma_f(\mathbf{h})$, where we drop the $*$ subscript when $\mathbf{h}$ is understood as the query point. We score each candidate $\mathbf{h}$ by how much, in expectation under the GP, its loss would beat the best loss seen so far, $\ell^* = \min_i \ell_i$:

$$\text{EI}(\mathbf{h}) = \mathbb{E}[\max(\ell^* - \ell(\mathbf{h}), 0)]. \quad (4.5)$$

Reading this as “the GP believes the loss at $\mathbf{h}$ is roughly $\bar{f}(\mathbf{h})$ plus a Gaussian fluctuation of size $\sigma_f(\mathbf{h})$, and we credit only the improvement, not the deterioration,” gives the intuition. Under the GP posterior the expectation evaluates in closed form:

$$\text{EI}(\mathbf{h}) = (\ell^* - \bar{f}(\mathbf{h})) \Phi(Z) + \sigma_f(\mathbf{h}) \phi(Z), \quad Z = \frac{\ell^* - \bar{f}(\mathbf{h})}{\sigma_f(\mathbf{h})}, \quad (4.6)$$

where Φ and ϕ are the standard normal CDF and PDF, respectively (Garnett, 2023, §5). The first term rewards *exploitation* (the predicted mean $\bar{f}$ lies below the current best ℓ^*); the second rewards *exploration* (σ_f is large where no data constrain the loss). EI peaks where the two conspire, which is exactly the place a sensible researcher would probe by hand. Figure 4.2 illustrates one iteration of this rule in one dimension, and the algorithmic box below collects it into a four-step recipe. Bayesian optimization is particularly effective when the number of hyperparameters is moderate ($d \leq 20$) and each evaluation is expensive, which describes many economic applications well.

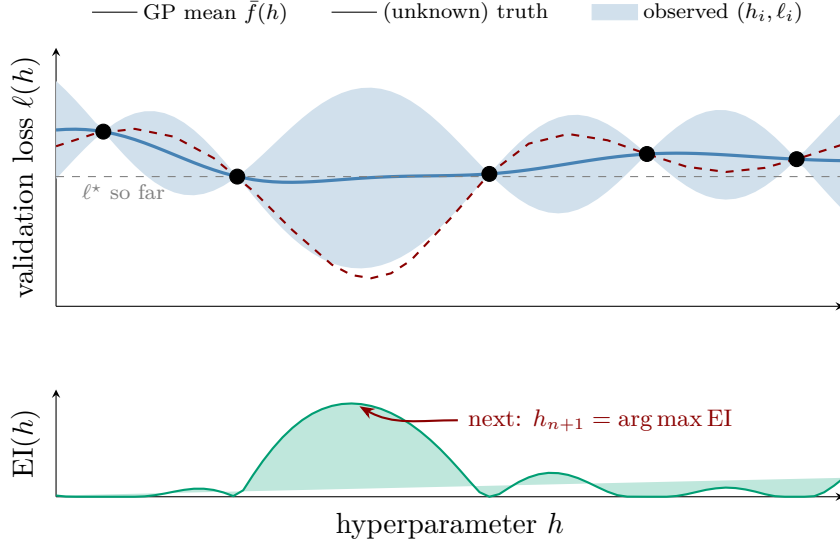


Figure 4.2: Bayesian optimization in one dimension; the same setup is reproduced in the companion notebook. *Top*: after five evaluations (black dots), the GP posterior mean $\bar{f}(h)$ (solid blue) interpolates the observations and the $\bar{f}(h) \pm 2\sigma_f(h)$ credible band (shaded) pinches to zero at each observation and widens in the gaps; this is the picture predicted by (4.3)–(4.4). The dashed red curve is the (in practice unknown) true loss; the horizontal gray line marks ℓ^* , the best observation so far (near $h \approx 2.3$). *Bottom*: Expected Improvement $EI(h)$ is essentially zero at the existing data, where there is nothing to learn, rises in the unexplored gaps, and peaks at $h \approx 3.75$, which combines a predicted mean already below ℓ^* with substantial residual uncertainty. The maximizer (red arrow) is selected as the next configuration to evaluate; EI thus balances exploitation against exploration automatically, and here it steers the search at the neighborhood of the hidden true minimum.

Bayesian Optimization with Expected Improvement

Input: initial design $\mathcal{D}_{n_0} = \{(\mathbf{h}_i, \ell_i)\}_{i=1}^{n_0}$, total budget N
for $n = n_0, n_0 + 1, \dots, N - 1$ **do**
 Fit the GP posterior $(\bar{f}, \sigma_f)$ to $\mathcal{D}_n$ using (4.3)–(4.4)
 Evaluate $EI(\mathbf{h})$ on a fine candidate grid (or via a local optimizer) over the search space
 Select $\mathbf{h}_{n+1} = \arg \max_{\mathbf{h}} EI(\mathbf{h})$
 Evaluate the validation loss $\ell_{n+1} = \ell(\mathbf{h}_{n+1})$ and append to $\mathcal{D}_{n+1} = \mathcal{D}_n \cup \{(\mathbf{h}_{n+1}, \ell_{n+1})\}$
end for
Output: best observed configuration $\arg \min_i \ell_i$

4.5 Hyperband and Successive Halving

Li et al. (2018) proposed an entirely different approach based on *adaptive resource allocation*, building on the Successive Halving Algorithm popularized by Jamieson and Talwalkar (2016). The key observation is that poor configurations can often be identified early in training, without running them to completion.

The Successive Halving Algorithm (SHA) turns this observation into a concrete schedule. Start with n_0 configurations, give each a small initial budget of r_0 epochs, train, then keep only the top $1/\eta$ fraction of survivors and multiply the per-candidate budget by η for the next round. Two facts about this schedule do most of the explanatory work below. First, the per-round compute is approximately constant: round s runs n_s survivors for r_s epochs each, and the rule

“ $\eta \times$ fewer candidates at $\eta \times$ the budget” keeps the product $n_s r_s$ fixed. Second, the cumulative cost across the cascade therefore scales with the *number of rounds* rather than with the worst-case “train every candidate to the final budget” benchmark. Both points are made concrete in the worked example that follows the pseudocode.

Successive Halving Algorithm

Input: initial candidates n_0 , initial per-candidate budget r_0 (epochs), reduction factor $\eta = 3$
 Draw a set S_0 of n_0 configurations from the search space
for round $s = 0, 1, \dots, \lceil \log_\eta n_0 \rceil - 1$ **do**
 Train each of the n_s survivors in S_s for r_s epochs and record their validation losses
 Keep the top $1/\eta$ fraction of S_s to form S_{s+1}
 Set $n_{s+1} = \lceil n_s/\eta \rceil$ and $r_{s+1} = \eta r_s$
end for
Output: best surviving configuration

To see the per-round invariant on a concrete schedule, take $n_0 = 81$, $r_0 = r$, and $\eta = 3$. Round 0 trains all 81 candidates for r epochs and keeps the top 27; round 1 trains those 27 for $3r$ epochs and keeps the top 9; round 2 trains the 9 for $9r$ and keeps 3; round 3 trains those 3 for $27r$ and selects the winner. At each round the round-level compute is $n_s r_s = 81r$ (the $1/3$ shrink in survivors is exactly offset by the $3 \times$ growth in budget), so the four rounds together cost $4 \cdot 81r = 324r$, equivalent to four full $R = 81r$ -epoch training runs rather than the 81 that naive parallel evaluation would require. Figure 4.3 visualizes this resource-allocation cascade.

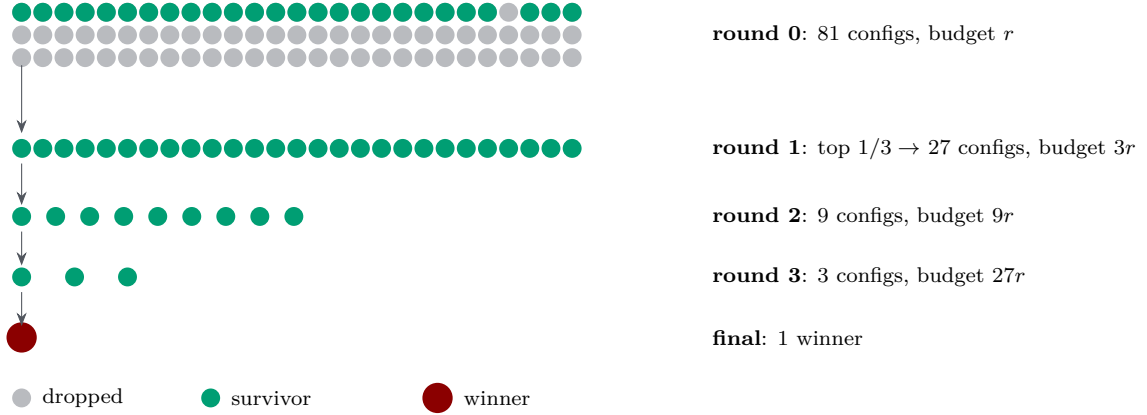


Figure 4.3: Successive Halving with 81 initial candidates and reduction factor $\eta = 3$. Each round trains the surviving configurations for η times the previous budget, then discards the bottom $(1 - 1/\eta)$ fraction. Total compute per bracket is only $\mathcal{O}(B)$ rather than $\mathcal{O}(nB)$ for training every candidate to completion. Hyperband runs several such brackets in parallel with different (n, r) trade-offs to hedge against unknown early-vs-late performance correlations (Li et al., 2018).

Hyperband extends SHA by running not one but several SHA cascades, each starting from a different trade-off between the number of candidates and the per-candidate initial budget; the intuition is hedging. A bracket that starts with many cheap candidates (large s) is well suited to problems where the eventual winner reveals itself early, while a bracket that starts with fewer well-trained candidates (small s) is better when ranks shuffle late in training, and Hyperband simply runs both kinds. The cascade shown in Figure 4.3, $(81, 1) \rightarrow (27, 3) \rightarrow (9, 9) \rightarrow (3, 27) \rightarrow (1, 81)$, is the most exploratory of the brackets ($s = s_{\max} = 4$); Table 4.1 unrolls the full canonical schedule for maximum resource $R = 81$ and reduction factor $\eta = 3$.

The companion notebook `03_NAS_RandomSearch_Hyperband.ipynb` implements the SHA inner loop only; the full Hyperband schedule is a straightforward outer loop that iterates this inner loop across the five (n_s, r_s) starting points in Table 4.1.

Bracket s	(n_s, r_s)	Unrolled schedule (candidates @ epochs)	Total cost B_s
4	(81, 1)	81@1 $\rightarrow$ 27@3 $\rightarrow$ 9@9 $\rightarrow$ 3@27 $\rightarrow$ 1@81	$5R$
3	(27, 3)	27@3 $\rightarrow$ 9@9 $\rightarrow$ 3@27 $\rightarrow$ 1@81	$4R$
2	(9, 9)	9@9 $\rightarrow$ 3@27 $\rightarrow$ 1@81	$3R$
1	(6, 27)	6@27 $\rightarrow$ 2@81	$4R$
0	(5, 81)	5@81	$5R$

Table 4.1: Hyperband bracket schedule for $R = 81$, $\eta = 3$, following Table 1 of Li et al. (2018). Each bracket s is a standalone SHA cascade with n_s initial candidates and per-candidate initial budget r_s epochs; the survivors are then halved according to the rule of Algorithm 4.3. Large s probes many cheap candidates and halves them aggressively (exploration); small s trains a handful of candidates to the full R -epoch budget from the start (exploitation). Per-bracket totals B_s are reported in units of R and are not monotone in s because n_s is integer-rounded. Hyperband runs all five brackets and returns the best surviving configuration across them.

4.6 Method Comparison

Table 4.2 contrasts the four hyperparameter-search strategies covered above on three dimensions that matter in practice: the cost of N objective evaluations, the degree to which the evaluations can be parallelized, and the sample efficiency (how much of the budget actually improves the best-so-far value).

Method	Cost	Parallelizable	Sample efficiency	Best for
Grid search	$\mathcal{O}(k^d)$ evals	fully	low	$d \leq 3$
Random search	N evals	fully	moderate	general use
Bayesian opt.	N evals + GP fit	limited	high	expensive evals
Hyperband	N resource units	within brackets	moderate	cheap evals

Table 4.2: Comparison of hyperparameter-search methods. Grid search scales exponentially in the number of hyperparameters d ; random search and Hyperband scale linearly in the chosen evaluation/resource budget and parallelize well; Bayesian optimization has the highest per-evaluation information gain but adds surrogate-fitting overhead and is partly sequential.

For the DEQN and PINN applications in this course, random search or Bayesian optimization are typically the most practical choices. Hyperband is attractive when training is relatively cheap and many configurations need to be screened quickly.

4.7 Implementing the Search in Practice

To keep the algorithms transparent, the companion notebook `03_NAS_RandomSearch_Hyperband.ipynb` implements both Random Search (§4.3) and the Successive Halving Algorithm (§4.5) directly in plain Python, with no hyperparameter-search library involved. The search space is encoded as an ordinary dict (number of hidden layers $\in \{1, \dots, 5\}$, units per layer $\in \{32, 64, \dots, 256\}$, activation function $\in \{\text{relu}, \text{tanh}, \text{swish}\}$, and learning rate log-uniform in $[10^{-4}, 10^{-2}]$), and a single `sample_config(rng)` function draws candidates from it. Random Search is then a 30-iteration loop that builds, trains, and scores each candidate; Successive Halving is the same loop wrapped in a halving schedule ($n_0 = 27$ candidates at $r_0 = 8$ epochs $\rightarrow 9$ at 24 $\rightarrow 3$ at 72 $\rightarrow$ winner, with $\eta = 3$). Both implementations fit on a single slide and reproduce the qualitative finding of Li et al. (2018) that Successive Halving reaches comparable accuracy to Random Search at substantially lower compute: in the notebook run, the same MAE is recovered with $\sim 2.3\times$ less compute (648 SHA config-epochs vs. 1500 for 30 Random Search trials at 50 epochs each) at

a comparable number of architectures (27 vs. 30). The precise multipliers are notebook-specific; the magnitudes reported in Li et al. vary by benchmark.

Production tooling (footnote). Real projects rarely hand-roll the search loop. Several established libraries wrap (and parallelize) the same algorithms behind uniform APIs: `KerasTuner`¹ (Random, Bayesian, Hyperband; tight Keras integration), `Optuna`² (TPE, CMA-ES, Hyperband, NSGA-II; framework-agnostic), `Ray Tune`³ (all of the above plus ASHA and population-based training, distributed by design), `Hyperopt`⁴ (the original TPE reference), `Ax / BoTorch`⁵ (PyTorch-native multi-objective Bayesian optimization), `NNI`⁶ (Microsoft; full graph-NAS support), and `AutoKeras`⁷ (full AutoML pipeline). We deliberately teach the algorithms rather than the wrappers because library APIs change every few years; the underlying search procedures (Random, SHA / Hyperband, GP+EI, TPE) do not. The notebook additionally compares the best NAS-found architecture to a hand-tuned baseline, which makes the pedagogical value of automated search concrete.

4.8 Multi-Component Losses: The Scale Problem

In many applications, including DEQNs and PINNs, the loss function is a weighted sum of several components:

$$\ell = \sum_{k=1}^K w_k \ell_k, \quad (4.7)$$

where $\ell_k \geq 0$ is the k -th *loss component* (one individual residual, e.g. a per-country Euler equation, a market-clearing identity, a complementarity-condition residual in an OLG model, or a boundary or initial-condition penalty in a PINN), $w_k \geq 0$ is its scalar weight, and K is the total number of components. In the DEQN setting of this script, each ℓ_k is typically the mean-squared residual of a particular equilibrium equation evaluated on the training mini-batch, so $\ell_k = 0$ at a solution and ℓ is jointly minimized over the network parameters.

From a multi-objective-optimization standpoint, the vector $(\ell_1, \dots, \ell_K)$ is the object of interest: the equilibrium is defined by all K residuals being zero, and any nonzero weight vector $\mathbf{w}$ picks a particular scalarization of the same underlying Pareto problem. When the components are on comparable scales, uniform weights work; when they are not, the scalarized problem is dominated by a single component and the optimizer effectively ignores the others. Adaptive weighting methods (discussed below) can be seen as online strategies for traversing the Pareto frontier rather than committing to a single scalarization up front. If the individual components ℓ_k differ in magnitude by several orders of magnitude, training can become unstable or converge slowly. Consider a concrete example from the IRBC model with $N = 10$ countries: at initialization, the Euler equation residual for country 1 might be $\ell_1 \approx 50$, while for country 10 it might be $\ell_{10} \approx 0.05$, a ratio of 10^3 . With uniform weights, the gradient is dominated by $\nabla \ell_1$, and the optimizer essentially ignores ℓ_{10} until ℓ_1 is nearly converged. This sequential convergence pattern can be 5–10 $\times$ slower than balanced convergence.

The fundamental difficulty is that the gradient of the total loss $\nabla \ell = \sum_k w_k \nabla \ell_k$ is dominated by the components with the largest $|w_k \nabla \ell_k|$. Even if all components are equally important for the economic solution, the optimizer cannot “see” the small components through the noise of the large ones.

¹https://keras.io/keras_tuner/

²<https://optuna.org/>

³<https://docs.ray.io/en/latest/tune/>

⁴<http://hyperopt.github.io/hyperopt/>

⁵<https://ax.dev/>

⁶<https://nni.readthedocs.io/>

⁷<https://autokeras.com/>

4.8.1 Inverse-Loss Weighting

A simple first approach is to set $w_k = 1/\bar{\ell}_k$, where $\bar{\ell}_k$ is an exponential moving average of ℓ_k . This normalizes each component to have approximately unit magnitude. While straightforward, this method can be unstable when loss components change rapidly.

4.8.2 SoftAdapt

Heydari et al. (2019) proposed a more principled approach based on the *rates of change* of the loss components. Define $\Delta_k^{(t)} = \ell_k^{(t)} - \ell_k^{(t-1)}$.⁸ The SoftAdapt weights are:

$$w_k^{(t)} = \frac{\exp(\Delta_k^{(t)}/\tau)}{\sum_{j=1}^K \exp(\Delta_j^{(t)}/\tau)}, \quad (4.8)$$

where $\tau > 0$ is a temperature parameter. Components that are decreasing slowly (or increasing) receive higher weight, directing the optimizer’s attention to the lagging components. In practice, SoftAdapt uses smoothed rates (averaged over a window of recent iterations) for stability. SoftAdapt is discussed here for context; the companion notebook `04_Loss_Normalization.ipynb` implements equal-, inverse-loss-, and ReLoBRaLo-weighting, and Exercise 6 asks you to add a GradNorm balancer to the same testbed.

4.8.3 ReLoBRaLo: Adaptive Loss Balancing

The *Relative Loss Balancing with Random Lookback* (ReLoBRaLo) algorithm of Bischof and Kraus (2025) motivates the deterministic classroom implementation used here. In the notebooks, we use a convex blend of the same ingredients, which is easier to follow while preserving the balancing logic.

High-level intuition. ReLoBRaLo combines two complementary signals into a single weight per loss component. The *step-wise* signal asks “which component lagged the most since the last iteration?” and rewards it with more weight; this is fast and reactive but noisy. The *baseline* signal asks “which component lagged the most since the start of training?” and is slow but globally aware. The two are then averaged with a one-step smoother to dampen oscillations. Concretely the algorithm stacks three pieces (Components 1–3 below); only the temperature T usually needs tuning, while the smoothing parameters α, ρ work at their textbook defaults.

Component 1: Relative balancing. At iteration t , compute relative losses with respect to the previous iteration:

$$\begin{aligned} r_{k,\text{step}}^{(t)} &= \frac{\ell_k^{(t)}}{T \ell_k^{(t-1)} + \varepsilon_{\text{num}}}, \\ \hat{w}_{k,\text{step}}^{(t)} &= K \cdot \frac{\exp(r_{k,\text{step}}^{(t)})}{\sum_{j=1}^K \exp(r_{j,\text{step}}^{(t)})}. \end{aligned} \quad (4.9)$$

This upweights components whose relative loss increased (lagging behind) and downweights those that improved. The small ε_{num} prevents division by zero; in code the softmax is evaluated after subtracting the largest ratio for numerical stability.

⁸The raw $\Delta_k^{(t)}$ is dimensional, so a component at scale 10^3 produces fluctuations that swamp one at scale 10^{-3} ; in practice one rescales before the softmax, e.g. $\tilde{\Delta}_k^{(t)} = \Delta_k^{(t)} / (\ell_k^{(t)} + \varepsilon)$, so the rule reacts to *relative* progress, the same idea that underlies ReLoBRaLo’s loss *ratios* below.

Component 2: Baseline comparison. To maintain a global perspective, compare the current losses to their initial values at $t = 0$:

$$\begin{aligned} r_{k,\text{base}}^{(t)} &= \frac{\ell_k^{(t)}}{T \ell_k^{(0)} + \varepsilon_{\text{num}}}, \\ \hat{w}_{k,\text{base}}^{(t)} &= K \cdot \frac{\exp(r_{k,\text{base}}^{(t)})}{\sum_{j=1}^K \exp(r_{j,\text{base}}^{(t)})}. \end{aligned} \quad (4.10)$$

This baseline comparison provides robustness to non-monotone loss trajectories and prevents the algorithm from losing sight of overall training progress.

Component 3: Smoothed combination. The final weight blends historical weights, baseline weights, and step-wise weights:

$$w_k^{(t)} = \alpha [\rho w_k^{(t-1)} + (1 - \rho) \hat{w}_{k,\text{base}}^{(t)}] + (1 - \alpha) \hat{w}_{k,\text{step}}^{(t)}, \quad (4.11)$$

where $\alpha \in [0, 1]$ is a smoothing parameter controlling how much to trust historical weights versus the current step-wise signal, and $\rho \in [0, 1]$ is a baseline-mix coefficient controlling the relative importance of the previous weights versus the initial-loss comparison. Equivalently, $w_k^{(t)}$ is a convex combination of $\{w_k^{(t-1)}, \hat{w}_{k,\text{base}}^{(t)}, \hat{w}_{k,\text{step}}^{(t)}\}$ with mixture weights $(\alpha\rho, \alpha(1 - \rho), 1 - \alpha)$, which sum to 1. (In the original ReLoBRaLo formulation, ρ governs a stochastic Bernoulli lookback mechanism; here and in the notebooks we use a deterministic convex blend, which is simpler and easier to reproduce.) Typical values are collected in Table 4.3.

Hyperparameter	Role	Typical value
T (temperature)	Controls weight concentration (softmax sharpness)	0.5–2.0
α (smoothing)	History vs. new information	0.99–0.999
ρ (mixing coefficient)	Initial-loss baseline vs. historical weight	0.99–0.999

Table 4.3: ReLoBRaLo hyperparameters used in the companion notebook. Default ranges follow [Bischof and Kraus \(2025\)](#). T is the only one that usually needs tuning; α and ρ at their defaults give slow, stable adaptation that works across a wide range of multi-component problems.

4.8.4 GradNorm

An alternative approach proposed by [Chen et al. \(2018\)](#) directly normalizes the gradient magnitudes rather than the loss values. GradNorm adjusts the weights so that $\|w_k \nabla \ell_k\|$ is approximately equal across all components, using the ratio of each component’s training rate to the average training rate as a signal. While more computationally expensive than ReLoBRaLo (it requires computing per-component gradient norms), GradNorm can be effective when gradient magnitudes are a better proxy for training difficulty than loss magnitudes.

Figure 4.4 illustrates the typical behavior: without adaptive reweighting, the optimizer focuses almost exclusively on ℓ_1 (the largest component), allowing ℓ_2 to stagnate; with adaptive loss balancing (e.g., ReLoBRaLo, GradNorm), all components converge at comparable rates. As a concrete reference, an unweighted run of the two-country IRBC training loop in the companion notebook typically prints something like the trace below (numbers indicative, seed-dependent):

```

1 epoch    0:  e11_1=49.700  e11_2=0.510  e11_arc=4.820
2 epoch   200:  e11_1=0.0123 e11_2=0.494  e11_arc=0.041
3 epoch   500:  e11_1=8.2e-4  e11_2=0.470  e11_arc=3.5e-3

```

Listing 4.1: Indicative residual log from an unweighted two-country IRBC run; the largest component falls quickly, the smaller component stalls.

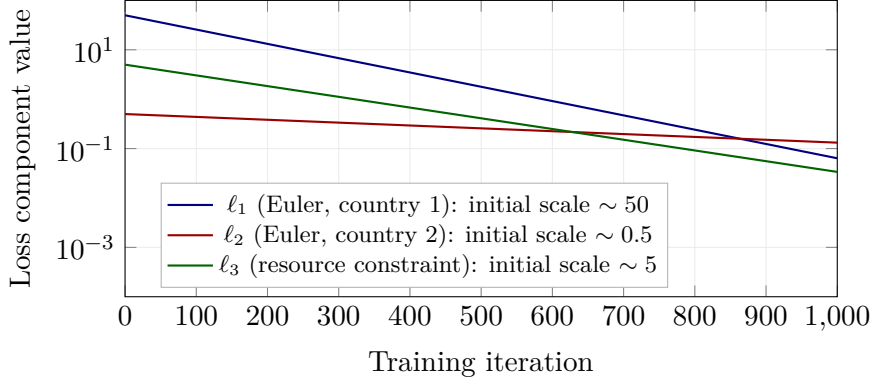


Figure 4.4: *Stylized* sketch of the multi-component loss-scale problem, drawn to mimic what one typically sees early in a two-country IRBC training run; this is *not* measured data. The three curves are hand-picked exponentials $a_k e^{-t/\tau_k}$ (with $a_1=50, \tau_1=150$; $a_2=0.5, \tau_2=750$; $a_3=5, \tau_3=200$), chosen only to make the mechanism visible: at initialization the residuals differ by about two orders of magnitude, and under *uniform* weighting the optimizer drives the largest component ℓ_1 (blue) down fastest because it dominates the summed gradient, while the smaller-scale but equally important country-2 Euler residual ℓ_2 (red) decays roughly five times more slowly and is left all but flat next to the others. Adaptive loss balancing such as ReLoBRaLo re-weights the components so that all three decrease at comparable rates. For the *actual* recorded trajectories on this problem, see the companion notebook `04_Loss_Normalization.ipynb`.

The pathology is immediate: ℓ_1 drops four orders of magnitude while ℓ_2 barely moves. Replacing the equal weights with ReLoBRaLo (§4.8.3) typically produces a trace in which all three components decay together; see the companion notebook for the actual ReLoBRaLo trace on the same seed. Reported convergence-speed improvements vary across schemes and benchmarks; multi-physics PINN benchmarks have shown substantial gains with ReLoBRaLo (Bischof and Kraus, 2025), while gains on DEQN-style Euler-equation losses tend to be smaller and problem-specific (the multi-component scale gap there is usually one to two orders of magnitude rather than the four to six common in PINN systems). A complementary line uses Neural-Tangent-Kernel diagnostics to choose the weights (Wang et al., 2022), and an older multi-task baseline weights losses by their predictive uncertainty (Kendall et al., 2018).

4.8.5 Summary of Balancing Methods

Table 4.4 compares the four balancing strategies on the two dimensions that reliably matter in practice: runtime overhead per step and the number of hyperparameters the user must set.

Method	Overhead	Hyperparameters
Uniform weights	none	0
Inverse-loss	negligible	1 (smoothing)
Uncertainty weighting (Kendall et al., 2018)	negligible	K (one log-variance per loss)
SoftAdapt (Heydari et al., 2019)	negligible	2 (τ , window)
ReLoBRaLo (Bischof and Kraus, 2025)	negligible	3 (T , α , ρ)
GradNorm (Chen et al., 2018)	moderate	1 (α)
NTK-based (Wang et al., 2022)	moderate-high	0 (data-driven)

Table 4.4: Summary of adaptive loss-balancing methods. Overhead is a per-step wall-clock cost (additional softmaxes for SoftAdapt/ReLoBRaLo; per-component gradient norms for GradNorm). Quantitative speedups depend strongly on the problem; see the companion notebook `04_Loss_Normalization.ipynb` for problem-specific measurements.

Quantitative speedup claims depend on the specific problem (PDE vs. Euler residual, number of components, imbalance ratio), the baseline (uniform vs. manually tuned), and the success criterion. The companion notebook `04_Loss_Normalization.ipynb` runs the four methods on a shared multi-scale regression task so that the reader can generate problem-specific numbers rather than rely on headline speedup factors from unrelated benchmarks.

Practical guidance

When implementing ReLoBRaLo:

- Set $T \in [0.5, 2.0]$; higher values yield more uniform weighting. In the limit, $T \rightarrow 0$ approximates a winner-take-all scheme that concentrates all weight on the single most-lagging component, while $T \rightarrow \infty$ recovers uniform weighting regardless of loss dynamics.
- Start with $\alpha = \rho = 0.999$ and reduce if weights change too slowly.
- ReLoBRaLo adds negligible computational overhead (one softmax per iteration) but can dramatically improve convergence for multi-component losses; GradNorm and SoftAdapt make analogous trade-offs.
- For PINN applications (Chapter 7), adaptive loss balancing in general (ReLoBRaLo, GradNorm, NTK-based schemes) is particularly effective at balancing PDE residual terms against boundary condition penalties.
- In DSGE applications, multi-component losses arise naturally: a model with N countries has N Euler equations, an aggregate resource constraint, and N complementarity conditions, often differing by several orders of magnitude. Without loss balancing, the optimizer focuses on the largest component and ignores smaller but economically important residuals (see Chapter 3).

Chapter Summary

- Architecture and hyperparameter choice are first-order: random search dominates grid search whenever only a few hyperparameters matter, and Bayesian optimization dominates random search whenever each evaluation is expensive.
- Hyperband-style multi-fidelity scheduling is the modern compromise: cheap evaluations of many candidates, with budget concentrated on the survivors.
- Multi-component DEQN/PINN losses suffer from scale imbalance; ReLoBRaLo and related adaptive schemes restore balance by reweighting components on the fly.
- All three families (NAS, Bayesian optimization, adaptive loss balancing) are implementation details, but they make the difference between a DEQN that converges on a coffee break and one that diverges overnight.

Further Reading

- [Bergstra and Bengio \(2012\)](#), the original case for random over grid search.
- [Snoek et al. \(2012\)](#), foundational reference for Bayesian optimization in ML.
- [Li et al. \(2018\)](#), Hyperband and successive halving.
- [Bischof and Kraus \(2025\)](#), ReLoBRaLo loss-balancing scheme used throughout the PINN chapter.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Core] Random vs. grid.** Reproduce the random-vs-grid figure of [Bergstra and Bengio \(2012\)](#). With 9 evaluations and only one “important” axis out of two, suppose the near-optimal region occupies a fraction p of that axis and its location relative to the grid is unknown. What is the hit probability for a 3×3 grid, and what is the hit probability for random search?
2. **[Computational] Bayesian optimization toy problem.** Implement a Gaussian-process-based BO loop on the 1D function $f(x) = -\sin(3x) - x^2 + 0.7x$ over $x \in [-1, 2]$. How many BO evaluations are typically needed before it matches the best value found by a grid with step size 0.01?
3. **[Core] Hyperband budget allocation.** For a Hyperband run with maximum resource $R = 81$ and $\eta = 3$, list the brackets (n_i, r_i) used and the total resource consumed. Compare with a naive “train each candidate to R ” strategy at fixed candidate count $n_0 = 27$.
4. **[Core] Loss balancing.** In a multi-component PINN loss with three terms of magnitude $10^0, 10^{-2}, 10^{-4}$, write down what fixed weights λ_i would be needed to equalize their gradient contributions. Why does this become impractical when the gradients are correlated?
5. **[Core] Pareto frontier geometry.** Consider the toy two-component loss $\mathcal{L}(\theta; \lambda) = \lambda(\theta - a)^2 + (1 - \lambda)(\theta - b)^2$ with $\theta \in \mathbb{R}$, $a < b$, and $\lambda \in [0, 1]$. (i) Solve for the minimizer $\theta^*(\lambda)$ in closed form. (ii) Compute the per-component residuals $\ell_1^*(\lambda) = (\theta^* - a)^2$ and $\ell_2^*(\lambda) = (\theta^* - b)^2$. (iii) Eliminate λ to express the Pareto frontier in the (ℓ_1, ℓ_2) plane and show it is the curve $\sqrt{\ell_1} + \sqrt{\ell_2} = b - a$ for $\ell_1, \ell_2 \geq 0$, hence convex. (iv) Sketch the frontier and identify which point on it corresponds to the equal-weight choice $\lambda = 1/2$. (v) In higher-dimensional parameter spaces, explain why nonzero gradient inner products $\langle \nabla \ell_1, \nabla \ell_2 \rangle$ make fixed scalar weights fragile. Contrast ReLoBRaLo’s relative-loss progress rule with GradNorm’s direct gradient-norm balancing.
6. **[Computational] ReLoBRaLo vs. GradNorm.** In notebook 04_Loss_Normalization, swap the ReLoBRaLo balancer for a GradNorm balancer ([Chen et al. 2018](#); see the chapter for the per-component gradient-norm equalization rule). Run both on the same multi-scale regression target with components of magnitude $10^0, 10^{-2}, 10^{-4}$. Report (i) wall-clock training time per epoch, (ii) final residual on each component, (iii) the gradient-balance ratio $\|\nabla \ell_k\| / \sum_j \|\nabla \ell_j\|$ at convergence. Comment on the cost–benefit trade-off: under what circumstances is the extra per-step cost of GradNorm worth it?
7. **[Core] HPO vs. full NAS decision.** You have access to either (a) a single consumer GPU (RTX 3060, ~ 12 GB) or (b) one A100 (80 GB), for one week of compute. Your search problem is either (i) a fixed-topology MLP with unknown depth $\in \{1, \dots, 5\}$, width $\in \{32, \dots, 512\}$, activation $\in \{\text{ReLU}, \text{Swish}, \text{tanh}\}$, learning rate (log-uniform), or (ii) a flexible network that can be MLP / RNN / shallow Transformer with unknown layer connectivity (graph-level NAS). For each of the four (hardware $\times$ problem) cells, recommend in three to five sentences whether to use Random Search with Successive Halving, Bayesian Optimization, or full graph-level NAS. Justify by referencing the per-method overhead and search-space size.

Chapter 5

Overlapping Generations Models with DEQNs

In Chapters 2–3 all agents were infinitely lived. We now extend the DEQN framework to *overlapping generations* (OLG) models (Diamond, 1965), where A finitely-lived cohorts coexist in every period. OLG models introduce lifecycle savings, intergenerational transfers, age-dependent heterogeneity, and inequality constraints on portfolio choices, phenomena that are central to fiscal policy analysis, pension reform, and demographic modeling. We proceed in two stages. We first solve a deliberately small 6-agent OLG that admits a *closed-form solution* (Krueger and Kubler, 2004), which gives a clean ground truth against which to validate the neural-network solver. We then scale up to the 56-agent research benchmark of Azinovic et al. (2022), where the no-short-sale-of-capital constraint $k^t \geq 0$ binds on a non-trivial slice of the ergodic set; that constraint introduces a kink, the main new computational challenge of the benchmark, and we handle it by combining softplus output activations (for non-negativity) with squared product residuals for the orthogonality conditions in the loss. The model also carries a collateral constraint $k^t + \kappa b^t \geq 0$ that the current notebook parameterization of $\hat{q}^h$ keeps slack on the learned ergodic set; we develop both constraints below so that the architecture is in place when a future calibration makes the collateral side bind.

5.1 Why Overlapping Generations?

In the Brock–Mirman and IRBC models of Chapters 2–3, all agents are infinitely lived. Picture instead a photograph of the economy taken at a single instant: it contains a twenty-something just entering the workforce with no savings, a forty-something at peak earnings putting money aside, and a retiree drawing down a lifetime of accumulated wealth, all making decisions in the same period and all linked through the prices that their collective saving determines. The infinitely-lived-agent assumption collapses this picture and rules out several economically important phenomena:

- **Lifecycle savings.** Agents accumulate wealth when young, draw it down in old age.
- **Intergenerational transfers.** Pensions, social security, and bequests cannot be studied without age structure.
- **Age-dependent heterogeneity.** Labor endowments, risk preferences, and portfolio composition vary systematically over the lifecycle.

An OLG economy consists of A cohorts that coexist in each period: a new cohort of age 1 is born, the oldest cohort of age A dies, and everyone else ages by one period. Crucially, the number of agent types is *finite*, so the cross-sectional distribution has only A entries and the

state space remains finite-dimensional, in contrast to the continuum-of-agents models treated in Chapter 6. The mechanism that ties the three phenomena above together is consumption smoothing over a hump-shaped earnings path (Figure 5.1): because labor income rises and then falls over the lifecycle while agents prefer a steady consumption stream, they accumulate assets in their high-earning years and run them down afterwards, and the equilibrium interest rate is whatever clears the resulting demand for savings against the economy’s capital stock.

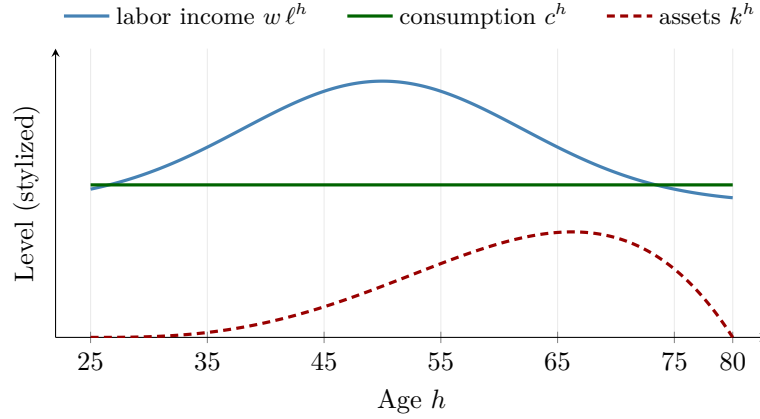


Figure 5.1: Stylized lifecycle profiles in an OLG economy (schematic, *not* a solution of the model). Labor income (blue) is hump-shaped, peaking in mid-career, while agents prefer a roughly flat consumption path (green); so they accumulate assets out of income during their high-earning years and run them down near the end of life. The asset profile (red, dashed) is therefore a hump that starts near zero for the newborn cohort, peaks toward the end of working life, and returns to zero for the oldest cohort, which consumes everything. The 6-agent analytic model of §5.2 is a stripped-down version of this picture (only the youngest cohort earns labor income); the 56-agent benchmark of §5.5 reproduces the full hump.

We develop the OLG framework in two stages. Section 5.2 works through the 6-agent model with a closed-form solution, maps it to a DEQN (§5.3), and validates the trained network against the analytical savings rates; §5.4 then explains how binding borrowing and collateral constraints are encoded, and §5.5 solves the 56-agent research benchmark with exactly the same training loop.

5.2 The 6-Agent Analytic OLG Model

End-to-end vignette: a 6-agent OLG in five steps

Before stepping through the formal model, it is worth seeing the full DEQN pipeline in one breath. The same five steps apply to every model in this script.

1. **State and policy.** Stack the 6 cohort capital holdings, the aggregate K_t , prices (r_t, w_t) , the TFP shock η_t , and the depreciation shock δ_t into a state vector $\mathbf{x}_t$. A single MLP $\mathcal{N}_\theta(\mathbf{x}_t) \rightarrow \mathbb{R}^5$ outputs the savings of cohorts 1–5 (cohort 6 saves nothing).
2. **Loss.** Stack 5 Euler-equation residuals into a single mean-square loss; a sigmoid savings-fraction head makes each $k'^h \in [0, \text{inc}^h]$, so $k'^h \geq 0$ and $c^h \geq 0$ both hold exactly, and capital-market clearing is satisfied by construction (aggregate K_{t+1} is read off as the sum of cohort savings). Small additive penalties on negative c and K remain in the loss as numerical backstops but are inactive with this head.
3. **Training distribution.** At each segment, construct a state cloud: in the persistent notebooks the cloud is generated by rolling parallel trajectories forward under the current policy; in the exogenous companions it is drawn from broad feasible boxes. Mini-batches are sampled from the current cloud.
4. **Optimization.** Adam with the standard moment coefficients (0.9, 0.999) (avoiding the β symbol of the household discount factor) and learning rate $\sim 10^{-4}$ in the short presets, decaying to 10^{-5} in the production preset of the analytic-OLG notebook. In the 56-agent benchmark of §5.5, the production schedule is 10^{-5} for the first 60,000 episodes followed by 10^{-6} for the remaining episodes.
5. **Diagnostics.** At convergence, check (i) the average savings rate by cohort against the closed-form β_h in Eq. (5.6) below, (ii) Euler residuals across the simulated ergodic set, (iii) (in the 56-agent benchmark only) the bond-market-clearing residual, and (iv) policy-drift / time-invariance on a fixed anchor cloud: evaluate the policy on `X_anchor` after each monitoring interval and flag the run as *time-invariant* once `policy_drift_rms` and `policy_drift_max` fall below `TIME_INVARIANCE_TOL_RMS` and `TIME_INVARIANCE_TOL_MAX`. In the 6-agent analytic model, capital-market clearing is satisfied by construction, since aggregate next-period capital is taken as the sum of cohort savings.

The notebook `lecture_08_08_OLG_Analytic_DEQN_persistent.ipynb` ships with a `RUN_MODE` switch: "smoke" (~30 s, sanity check), "teaching" (~5 min, savings close to the closed form and mean Euler residuals $\sim 10^{-3}$ on the simulated cloud), and "production" (longer run, Table 3-level Euler accuracy on the full ergodic set); a feedback-free exogenous-sampling companion is available as `lecture_08_07_OLG_Analytic_DEQN_exogenous.ipynb`. A fifth notebook, `lecture_08_11_OLG_Exercise.ipynb`, is a self-contained warm-up (closed-form savings rates, a single-cohort lifecycle simulation, and a discount-factor comparison) on the same analytic model. The 56-agent benchmark of §5.5 (notebook `lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb`, with the exogenous-sampling companion `lecture_08_09_OLG_Benchmark_DEQN_exogenous.ipynb`) runs in a few hours on a GPU with the *same* code template.

Krueger and Kubler (2004) proposed a deliberately simple OLG model with a *closed-form solution*, making it an ideal validation benchmark for the DEQN approach. We develop it here as the first of the two OLG instances of this chapter; §5.3 maps it to a DEQN and validates the trained network against the closed form derived below.

We instantiate the OLG environment with $A = 6$ overlapping cohorts, indexed by age $h \in \{1, \dots, 6\}$. Time is discrete and infinite. The model equations below are written for general A and specialized to $A = 6$ in the calibration that follows.

Household problem. An agent of age h at time t maximizes expected lifetime utility:

$$\max_{\{c_{t+j}^{h+j}, k_{t+j+1}^{h+j+1}\}_{j=0}^{A-h}} \mathbb{E}_t \left[\sum_{j=0}^{A-h} \beta^j u(c_{t+j}^{h+j}) \right], \quad (5.1)$$

subject to the period budget constraint

$$c_t^h + k_{t+1}^{h+1} = r_t k_t^h + w_t \ell^h \equiv \text{inc}_t^h, \quad (5.2)$$

where k_t^h denotes capital holdings, r_t is the gross return on capital, w_t is the wage, ℓ^h is an age-dependent labor endowment, and inc_t^h is total income.

Boundary conditions.

- Newborns have no initial wealth: $k_t^1 = 0$.
- The oldest cohort consumes everything: $k_{t+1}^{A+1} = 0$.
- Borrowing is not permitted: $k_{t+1}^{h+1} \geq 0$ for all h .

Euler equations. The first-order conditions yield $A - 1$ Euler equations (for ages $h = 1, \dots, A - 1$):

$$u'(c_t^h) = \beta \mathbb{E}_t \left[r_{t+1} u'(c_{t+1}^{h+1}) \right]. \quad (5.3)$$

Firm problem and market clearing. A representative firm operates a Cobb–Douglas technology with value added $F_t = \eta_t K_t^\alpha L_t^{1-\alpha}$, where η_t is a TFP shock and $L_t = \sum_{h=1}^A \ell^h$; the gross resource available to households is $Y_t = F_t + (1 - \delta_t) K_t = r_t K_t + w_t L_t$ (it is Y_t , not F_t , that the notebook passes as an engineered feature). Competitive factor markets imply:

$$r_t = \alpha \eta_t K_t^{\alpha-1} L_t^{1-\alpha} + (1 - \delta_t), \quad w_t = (1 - \alpha) \eta_t K_t^\alpha L_t^{-\alpha}, \quad (5.4)$$

where δ_t is the depreciation rate (potentially stochastic). Market clearing requires that aggregate capital at $t + 1$ is the sum of holdings across cohorts:

$$\sum_{h=2}^A k_{t+1}^h = K_{t+1}, \quad (5.5)$$

with $k_{t+1}^1 = 0$ as a newborn boundary condition (cohort 1 enters life with no assets), and where k_{t+1}^h for $h = 2, \dots, A$ is the savings of cohort $h - 1$ at date t (which becomes the date- $(t + 1)$ holdings of the cohort once it has aged by one period).

Calibration. The model has $A = 6$ agents with log utility ($\gamma = 1$), Cobb–Douglas production ($\alpha = 0.3$), and discount factor $\beta = 0.7$. Only agent 1 works ($\ell = (1, 0, 0, 0, 0, 0)$); this stripped-down labor profile is what gives the closed form below, not a realistic lifecycle assumption, and the 56-agent benchmark of §5.5 restores a hump-shaped endowment. Four exogenous shock states combine TFP $\eta \in \{0.95, 1.05\}$ and depreciation $\delta \in \{0.5, 0.9\}$, with i.i.d. transitions ($\pi_{ss'} = 0.25$).

Analytical solution. With log utility and i.i.d. shocks, the optimal savings rate has a closed form. Define the age-dependent savings rate:

$$\beta_h = \beta \cdot \frac{1 - \beta^{A-h}}{1 - \beta^{A-h+1}}, \quad h = 1, \dots, A-1. \quad (5.6)$$

The optimal policy is then $k'^h = \beta_h \cdot \text{inc}^h$: each agent saves a *fixed fraction* of total income, regardless of the shock. Two features of the calibration drive this clean form. First, under log utility the income and substitution effects of a return shock exactly cancel, so the savings *rate* is invariant to (r_t, w_t) . Second, because the shocks are i.i.d. there is nothing about the future to forecast, so the rate does not depend on the current shock either; only the horizon matters. The fraction β_h therefore declines with age: cohort h has only $A - h$ remaining periods over which to spread its future income, so the marginal incentive to carry resources forward weakens as h grows. For $A = 6$, $\beta = 0.7$, Table 5.1 reports the resulting savings rates. Young agents save more

Age h	1	2	3	4	5
β_h	0.660	0.639	0.605	0.543	0.412

Table 5.1: Closed-form age-specific savings rates in the 6-agent analytic OLG with log utility and $\beta = 0.7$.

(more periods ahead); old agents save less; Figure 5.2 plots the same numbers across h . This vector is the validation target: at convergence, the trained network’s average sigmoid output should reproduce β_h cohort by cohort.

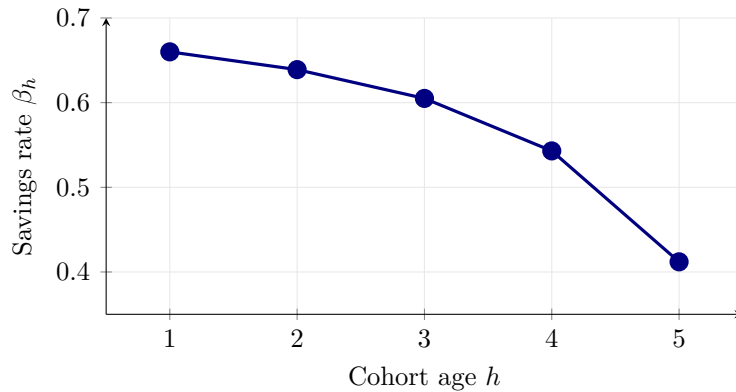


Figure 5.2: Closed-form savings rates β_h from Table 5.1 for the 6-agent analytic OLG ($\beta = 0.7$, log utility). The monotone decline with age reflects the shrinking forward horizon: cohort h has only $A - h$ remaining periods over which to consume future income, so the marginal incentive to save weakens as h grows. This is the validation target the trained DEQN’s average sigmoid output should match cohort by cohort.

5.3 Mapping the Analytic OLG to a DEQN

The mapping follows the same “states $\rightarrow$ network $\rightarrow$ loss” structure as Brock–Mirman (Chapter 2). We now write each block explicitly for the 6-agent analytic model just set up; this is exactly what slides II.7–II.9 of `lectures/lecture_08_olg_models_deqns/slides/lecture_08_olg_models_deqns.tex` render in pictures. The 56-agent benchmark of §5.5 extends the same template with two extra policy blocks (multipliers, bond price) and an additional market-clearing residual; we write that version out there.

State $\mathbf{x}_t$ entering the network. What does the network actually need to know? The *informational* state of the analytic model is just the pair

$$(z_t, \mathbf{k}_t) \in \{1, \dots, 4\} \times \mathbb{R}^A \quad \text{where} \quad \mathbf{k}_t = (k_t^1, \dots, k_t^A), \quad (5.7)$$

the current shock index plus the cross-sectional capital distribution. This is the minimal vector that pins down the equilibrium, and it is what slide II.8 displays in the FREE signature. Everything else, the aggregate capital $K_t = \sum_h k_t^h$, the prices (r_t, w_t) , output Y_t , each cohort's income, the row of next-period transition probabilities, is a deterministic function of $(z_t, \mathbf{k}_t)$. The network could in principle re-derive all of it from the raw pair, but there is no reason to make it: we hand the network those quantities pre-computed, which is a pure change of input coordinates that leaves the equilibrium map untouched and frees the network's capacity for the one genuinely hard thing it has to learn, the savings policy. Concretely the notebook feeds an *extended state* of dimension $16 + 4A$,

$$\mathbf{x}_t = \left(\underbrace{z_t, \mathbf{1}\{z_t\}, \eta_t, \delta_t, K_t, L_t, r_t, w_t, Y_t}_{12 \text{ aggregate}}, \underbrace{k_t^{1:A}, \text{fw}_t^{1:A}, \text{linc}_t^{1:A}, \text{inc}_t^{1:A}}_{4A \text{ per-agent}}, \underbrace{\pi(z_t, \cdot)}_{4 \text{ transition probs}} \right) \in \mathbb{R}^{16+4A}, \quad (5.8)$$

with $\mathbf{1}\{z_t\}$ the 4-state one-hot of the current shock, $K_t = \sum_h k_t^h$, $L_t = \sum_h \ell^h$, (r_t, w_t) from (5.4), Y_t the gross resource $\eta_t K_t^\alpha L_t^{1-\alpha} + (1 - \delta_t)K_t$, and the per-agent blocks $\text{fw}_t^h = r_t k_t^h$ (capital income), $\text{linc}_t^h = w_t \ell^h$ (labor income), $\text{inc}_t^h = \text{fw}_t^h + \text{linc}_t^h$ (total income). Since the map $(z_t, \mathbf{k}_t) \mapsto \mathbf{x}_t$ is deterministic, (5.7) and (5.8) carry exactly the same information. For $A = 6$ this is $16 + 4 \cdot 6 = 40$ inputs (the notebook constant `FEATURE_DIM`).

Policies approximated by the network. A single multilayer perceptron with a *sigmoid savings-fraction* output head approximates the equilibrium policy as a function of the state. (Throughout this OLG chapter we use θ for the network parameters rather than the ρ of Chapters 2–3; both refer to the same object, and the switch follows the convention of the public OLG reference implementation.)

$$\boxed{\mathcal{N}_\theta : \mathbb{R}^{16+4A} \longrightarrow \mathbb{R}^{A-1}, \quad \mathcal{N}_\theta(\mathbf{x}_t) = (\hat{\beta}^1(\mathbf{x}_t), \dots, \hat{\beta}^{A-1}(\mathbf{x}_t)), \quad \hat{a}^h(\mathbf{x}_t) := \hat{\beta}^h(\mathbf{x}_t) \text{inc}_t^h} \quad (5.9)$$

where the network output $\hat{\beta}^h(\mathbf{x}_t) \in (0, 1)$ is cohort h 's *savings rate* and $\hat{a}^h(\mathbf{x}_t)$ its savings level (slide II.9, output column). This parameterization mirrors the closed-form solution's structure (each cohort saves a fixed fraction of income, Eq. (5.6)). Cohort A saves nothing by terminal boundary, so the network has $A - 1$ outputs rather than A . Three by-construction guarantees follow:

- Non-negativity of savings $\hat{a}^h \geq 0$ holds at every iteration, so the borrowing constraint (5.12) is satisfied without an explicit Lagrange multiplier (in this calibration $\lambda^h \equiv 0$ on the ergodic set; see §5.4 for the multiplier-based variant used in the 56-agent benchmark).
- Non-negativity of consumption $\hat{c}^h = (1 - \hat{\beta}^h) \text{inc}_t^h \geq 0$ also holds by construction, so the soft penalty on $\max(-\hat{c}^h, 0)$ in the loss (next paragraph) is a dead backstop.
- Capital-market clearing $K_{t+1} = \sum_{h=2}^A k_{t+1}^h$ also holds by construction, since aggregate next-period capital is read off as the sum of the network's savings outputs together with the newborn boundary $k_{t+1}^1 = 0$.

Equilibrium residual. Each cohort $h \in \{1, \dots, A - 1\}$ contributes one *relative* Euler-equation residual, built from three quantities. First, the implied current consumption, read off from the budget (5.2) as $\hat{c}^h(\mathbf{x}_t) := \text{inc}_t^h - \hat{a}^h(\mathbf{x}_t)$. Second, the next-state map Φ , which combines the current policy with a fresh shock z_{t+1} to produce next period's extended state $\hat{\mathbf{x}}_{t,+} = \Phi(\mathbf{x}_t, z_{t+1}; \theta)$

(the construction of Φ is spelled out in the next paragraph). Third, the implied next-period consumption $\hat{c}^{h+1}(\hat{\mathbf{x}}_{t,+})$ of the cohort that has just aged from h to $h + 1$. The relative Euler-equation residual is then

$$e_{\text{REE}}^h(\mathbf{x}_t) := \frac{(u')^{-1}\left(\beta \mathbb{E}\left[r(\hat{\mathbf{x}}_{t,+}) u'(\hat{c}^{h+1}(\hat{\mathbf{x}}_{t,+}))\right]\right)}{\hat{c}^h(\mathbf{x}_t)} - 1, \quad h = 1, \dots, A - 1, \quad (5.10)$$

with $u(c) = \ln c$ in the analytic model so $(u')^{-1}(y) = 1/y$. Equation (5.10) is the unit-free residual of the standard Euler equation (5.3): a value of 10^{-3} means cohort h 's implied consumption is mispriced by 0.1% relative to the conditional certainty equivalent. This is the residual displayed in slide II.7.

Sampling the conditional expectation. The expectation in (5.10) is over the next-period shock z_{t+1} . Because the analytic-model shock has only four states with i.i.d. transition $\pi_{ss'} = 1/4$, the expectation is computed *exactly* (no Monte Carlo) by summing over the four next-period shocks: $\mathbb{E}\left[r(\hat{\mathbf{x}}_{t,+}) u'(\hat{c}^{h+1})\right] = \frac{1}{4} \sum_{s'=1}^4 r(\Phi(\mathbf{x}_t, s'; \theta)) u'(\hat{c}^{h+1}(\Phi(\mathbf{x}_t, s'; \theta)))$. For each candidate s' the next-state map Φ ages the cross-section by one period, sets the newborn to $k^1 = 0$, evaluates the firm prices (5.4) at K_{t+1} , and produces the next-period extended state $\hat{\mathbf{x}}_{t,+}$ on which the network is evaluated again to obtain $\hat{c}^h(\hat{\mathbf{x}}_{t,+})$ and hence $\hat{c}^{h+1}(\hat{\mathbf{x}}_{t,+})$. When the shock has more states or is continuous, the same construction is replaced by a sample of $z_{t+1} \sim \pi(\cdot|z_t)$ inside the mini-batch (see §5.5).

The DEQN loss for the analytic OLG. Given a mini-batch $D_{\text{train}} \subset \{\mathbf{x}_j\}_{j=1}^N$ sampled from the ergodic set of the current policy, the loss is the mean-squared relative Euler residual averaged across cohorts and states:

$$\mathcal{L}_{D_{\text{train}}}(\theta) = \frac{1}{|D_{\text{train}}|} \frac{1}{A-1} \sum_{\mathbf{x}_j \in D_{\text{train}}} \sum_{h=1}^{A-1} (e_{\text{REE}}^h(\mathbf{x}_j))^2 \quad (5.11)$$

(*matching slide II.7*). Two small barrier-style additive penalties on rescaled negative-consumption and negative-aggregate-capital hinges are summed in alongside (5.11) to keep training numerically robust away from convergence; in the notebook they carry the weight `PENALTY_WEIGHT = 10` and act on terms such as $\max(-\hat{c}^h, 0)/(1 + |\hat{c}^h|)$ rather than the raw squared hinge. With the sigmoid savings-fraction head described above these hinges are in fact identically zero (savings stay in $[0, \text{inc}^h]$, so $\hat{c}^h \geq 0$ and $K_{t+1} \geq 0$ always), so the penalties are pure backstops and do not bias the solution.¹

DEQN architecture and training. The network takes a 40-dimensional input (the extended state (5.8), $16 + 4 \times 6$) and outputs 5 savings *rates* $\hat{\beta}^h$ via a $40 \rightarrow 100 \rightarrow 50 \rightarrow 5$ architecture with ReLU hidden layers and a sigmoid savings-fraction output ($\approx 9,400$ parameters). Training uses the episode-based procedure from Chapter 2: the current network generates a capital path (episode), equilibrium residuals are computed and used for SGD updates, and a new episode is simulated periodically. The companion notebook exposes a `RUN_MODE` switch with three calibrated budgets: "**smoke**" (~ 25 training segments, ~ 30 s on CPU; a code-path sanity check, well short of convergence), "**teaching**" (~ 500 segments, ~ 5 min on CPU; savings rates match the closed form

¹The 56-agent benchmark of §5.5 adds two genuine extras to (5.11): KKT product residuals (because the borrowing and collateral constraints actually bind) and an explicit bond-market-clearing residual (because the network outputs each agent's bond holding independently). An orthogonal extension is to encode capital-market clearing *exactly* via a dedicated output layer that rescales unnormalized cohort savings so that $\sum_{h=2}^A k_{t+1}^h = K_{t+1}$ holds by construction; Azinovic-Yang and Žemlička (2024) adopt this design in an OLG economy with rare disasters.

to a few parts in 10^4 and mean relative Euler errors are already $\sim 10^{-3}$ on the simulated cloud, though larger off-trajectory), and "production" ($\sim 10,000$ segments with longer trajectories, several hours on CPU; mean Euler errors $\sim 10^{-3}$ or below, matching Table 3 of [Azinovic et al. \(2022\)](#)). Adam is used throughout (learning rate $\sim 3 \times 10^{-4}$ in the short presets, 10^{-5} in the production preset); the analogous decay to 10^{-6} used by the 56-agent benchmark (§5.5) is not needed at the analytic model's scale.

Validation principle

The 6-agent analytic OLG provides a known ground truth against which the DEQN can be validated exactly. This validation step gives confidence that the same framework will produce reliable results for models without closed-form solutions.

5.4 Inequality Constraints and KKT Complementarity

The 6-agent calibration above is deliberately frictionless: the no-short-sale-of-capital constraint never binds on its ergodic set, so we could solve it with a plain sigmoid-savings head and no multipliers. Realistic OLG economies are not so kind. The 56-agent benchmark of the next section carries a no-short-sale-of-capital constraint that binds on a non-trivial slice of states and a collateral constraint that the current notebook parameterization keeps slack on the learned ergodic set; binding inequality constraints in general bring in Karush–Kuhn–Tucker (KKT) complementarity, with its characteristic non-smooth orthogonality condition. This section sets out how the DEQN framework encodes that complementarity; the next section puts it to work.

The no-short-sale-of-capital constraint $k^h \geq 0$ introduces a complementarity condition via the Karush–Kuhn–Tucker (KKT) system:

$$k^h \geq 0, \quad \lambda^h \geq 0, \quad k^h \cdot \lambda^h = 0, \quad (5.12)$$

where λ^h is the KKT multiplier on the constraint. In a generic non-linear program, the orthogonality condition $k^h \cdot \lambda^h = 0$ is non-smooth at the origin and cannot be differentiated through naively.

The DEQN setup of [Azinovic et al. \(2022\)](#) sidesteps the kink by splitting enforcement across the architecture and the loss:

- **Hard side (architecture).** The savings k^h and the multiplier λ^h are both produced by the network through a softplus activation, so the inequalities $k^h \geq 0$ and $\lambda^h \geq 0$ hold by construction at every iteration.
- **Soft side (loss).** With non-negativity already guaranteed, the orthogonality $k^h \cdot \lambda^h = 0$ is enforced by adding the squared product residual $(k^h \lambda^h)^2$ to the loss.

This product form is what the public reference implementation accompanying [Azinovic et al. \(2022\)](#) uses, and what we adopt in the *56-agent benchmark* of §5.5 (Notebook `lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb`). As noted above, in the 6-agent analytic calibration of §5.2 the no-short-sale-of-capital constraint is non-binding everywhere on the ergodic set, so $\lambda^h \equiv 0$ and the multipliers (and the KKT residual) drop out of both the network output and the loss; that is why the mapping there was the simpler $\mathcal{N}_\theta : \mathbb{R}^{16+4A} \rightarrow \mathbb{R}^{A-1}$ of §5.3 above, with no multiplier outputs. The smoother *Fischer–Burmeister* (FB) reformulation, $\Phi(a, b) = a + b - \sqrt{a^2 + b^2}$, is an alternative used in the IRBC notebook of Chapter 3 for the investment-irreversibility constraint.

When to choose product form vs. Fischer–Burmeister. The product form $(k^h \lambda^h)^2$ is simpler, gradient-cheaper, and sufficient whenever the constraint is rarely active on the ergodic

set, since the optimizer just needs to verify slackness in expectation. The Fischer–Burmeister residual $\Phi(a, b)^2$ keeps gradient information *on both sides* of the active set: when the constraint is frequently binding (e.g. the IRBC irreversibility constraint on a non-trivial fraction of states), product-form gradients vanish whenever the constraint is locally inactive, which can stall training; FB does not have this pathology. As a rule of thumb: product form for occasionally-binding KKT, FB for frequently-binding KKT. In the OLG benchmark of §5.5 the no-short-sale-of-capital constraint binds on a thin slice of the ergodic set, so the product form was sufficient; the IRBC application of the previous chapter binds more often and benefits from FB.

The two OLG models we solve, side by side. We have now built and solved the first of the two OLG instances that anchor the rest of the manuscript: the 6-agent analytic model used to validate the DEQN against a closed form (Sections 5.2–5.3). The second is the 56-agent benchmark of Azinovic et al. (2022), developed in the next section. Table 5.2 summarizes the structural and computational gap between them before we turn to it.

	6-agent analytic (§5.2)	56-agent benchmark (§5.5)
Cohorts A	6 (childhood-style)	56 (ages 25–80, one period = one year)
Utility	Log ($\gamma = 1$)	CRRA ($\gamma = 2$)
Shocks	i.i.d. TFP & depreciation, 4 states	Persistent Markov on (η, δ)
Labor profile	Only youngest cohort works	Hump-shaped lifecycle endowment ℓ^h
Assets	Capital only	Capital + bonds
Constraints	None binding in calibration	No-short-sale of capital $k'^h \geq 0$ binds; collateral $k'^h + \kappa b^h \geq 0$ kept slack by the $\hat{q}^h$ parameterization
Adjustment cost	None	Quadratic $\frac{\zeta}{2}(k'^h - rk^h)^2$
Network input dim	40 (extended; minimal 7)	240 (extended; minimal 113)
Output dim	5 (savings rates of cohorts 1–5)	221 ($4(A - 1) + 1$: policies, multipliers, price)
Loss terms	5 Euler + market clearing by construction	221: $4(A - 1)$ Euler/KKT + 1 bond clearing
Network	Input(40) $\rightarrow$ 100 $\rightarrow$ 50 $\rightarrow$ 5	Input(240) $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 221 (teaching) / Input(240) $\rightarrow$ 1000 $\rightarrow$ 1000 $\rightarrow$ 221 (production)
Validation target	Closed-form β_h of Krueger and Kubler (2004)	Mean Euler residual on simulated trajectory
Notebook	lecture_08_08_OLG_Analytic_DEQN_persistent.ipynb	lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb

Table 5.2: The two OLG models solved in this chapter, side by side. The economic richness of the 56-agent benchmark adds two assets, an effectively binding no-short-sale-of-capital constraint (the collateral constraint is kept slack by the $\hat{q}^h$ parameterization), persistent shocks, lifecycle labor, and adjustment costs, raising the network input dimension from 40 to 240 and the output dimension from 5 to 221. The DEQN training loop is structurally identical in both cases. Each variant additionally ships with a feedback-free exogenous-sampling companion notebook (lecture_08_07_OLG_Analytic_DEQN_exogenous.ipynb, lecture_08_09_OLG_Benchmark_DEQN_exogenous.ipynb) that exercises the same model under a non-co-evolving training cloud.

5.5 The 56-Agent Benchmark

Table 5.2 above previewed the gap; we now develop the second model in full. The benchmark of Azinovic et al. (2022) scales the OLG framework to $A = 56$ agents (ages 25–80) with several

realistic features:

- **CRRA utility** with $\gamma = 2$ (replacing log utility).
- **Two assets:** capital k and one-period bonds b , with bond price p determined in equilibrium.
- **Hump-shaped labor endowment** e^h peaking in the early 50s.
- **No-short-sale of capital:** $k'^h \geq 0$ (the constraint historically labeled the “borrowing constraint” in this literature; we keep the more precise name to free “borrowing” for the bond side).
- **Collateral constraint:** $k'^h + \kappa b'^h \geq 0$, where $\kappa = 1/(1 - \delta_{\max})$.
- **Capital adjustment costs:** $\Psi^h = \frac{\zeta}{2}(k'^h - r \cdot k^h)^2$.
- **Persistent shocks:** a 4-state Markov chain for $\text{TFP} \times \text{depreciation}$ (contrast with i.i.d. in the analytic model).

Lifecycle labor endowments. The labor endowment profile e^h follows [Brumm et al. \(2017\)](#). In the implementation used here, e^h is a quadratic in age that rises from 0.60 at age 25, peaks at ≈ 1.36 around age 53, then decays linearly between ages ~ 62 and ~ 70 to a flat post-retirement floor of ≈ 0.64 . Table 5.3 lists the values produced by the notebook formula at a few representative ages. This hump-shaped profile ensures realistic savings heterogeneity: young

Age	25	30	40	48	53	65	80
e^h	0.60	0.85	1.20	1.34	1.36	1.04	0.64

Table 5.3: Representative points on the lifecycle labor-endowment profile in the 56-agent benchmark.

agents with low labor income and no initial wealth are borrowing-constrained, mid-career agents with high earnings accumulate both capital and bonds, and older agents gradually decumulate toward the end of life.

Persistent aggregate shocks. The 4-state Markov chain combines TFP η and depreciation δ into the pairs $(\eta, \delta) \in \{(0.978, 0.08), (1.022, 0.08), (0.978, 0.11), (1.022, 0.11)\}$. The transition matrix is persistent (diagonal entries ~ 0.63 – 0.88), in contrast to the i.i.d. shocks in the analytic model. This persistence creates richer dynamics in capital accumulation: a sequence of bad TFP draws can push young agents deep into their borrowing constraint, producing endogenous amplification that a single-period shock would not generate.

Budget constraint. Each agent of age h faces:

$$c^h + k'^h + p \cdot b'^h + \Psi^h = r \cdot k^h + b^h + w \cdot e^h. \quad (5.13)$$

The collateral constraint $k'^h + \kappa b'^h \geq 0$ acts as a margin requirement: it limits bond borrowing ($b'^h < 0$) relative to capital holdings. Since $\kappa = 1/(1 - \delta_{\max})$, the constraint tightens when depreciation is high, precisely when agents are most likely to seek insurance through borrowing.

State x_t entering the network. The informational state of the benchmark is the triple $(z_t, \mathbf{k}_t, \mathbf{b}_t) \in \{1, \dots, 4\} \times \mathbb{R}^A \times \mathbb{R}^A$, where $\mathbf{k}_t = (k_t^1, \dots, k_t^A)$ and $\mathbf{b}_t = (b_t^1, \dots, b_t^A)$ are the cross-sectional capital and bond distributions, so the minimal state has dimension $1 + 2A = 113$. As in the analytic case, the notebook feeds the network an extended state of the same $16 + 4A$ form – twelve aggregate scalars (shock index and its one-hot, $\eta_t, \delta_t, K_t, L_t, r_t, w_t$, and the gross resource

$Y_t = \eta_t K_t^\alpha L_t^{1-\alpha} + (1 - \delta_t) K_t$), four per-agent blocks (k_t^h , financial income $r_t k_t^h + b_t^h$, labor income $w_t e^h$, and cash $r_t k_t^h + b_t^h + w_t e^h$ – the bond holdings b_t^h are recoverable from financial income and are *not* passed as a separate block, and the bond price $\hat{p}_t$ is an output, not an input), and the row of next-period transition probabilities $\pi(z_t, \cdot)$ (used by the conditional-expectation block of the loss); concretely $240 = 12 + 4 \times 56 + 4$ (the notebook constant `FEATURE_DIM`). This is the analogue of slide III.8.

Policies approximated by the network. A single network $\mathcal{N}_\theta$ with softplus output produces a $4(A - 1) + 1$ -dimensional vector that is sliced into five economic blocks (slide III.9):

$$\boxed{\mathcal{N}_\theta : \mathbb{R}^{240} \longrightarrow \mathbb{R}^{4(A-1)+1}, \quad \mathcal{N}_\theta(\mathbf{x}_t) = (\hat{k}^{1:A-1}, \hat{\lambda}_b^{1:A-1}, \hat{q}^{1:A-1}, \hat{\mu}^{1:A-1}, \hat{p})(\mathbf{x}_t)} \quad (5.14)$$

where $\hat{k}^h$ are capital savings, $\hat{\lambda}_b^h$ the no-short-sale-of-capital multipliers, $\hat{q}^h \equiv \hat{k}^h + \kappa \hat{b}^h$ the collateral requirement (from which bond holdings are recovered as $\hat{b}^h = (\hat{q}^h - \hat{k}^h)/\kappa$), $\hat{\mu}^h$ the collateral-constraint multipliers, and $\hat{p}$ the equilibrium bond price. Each raw output is mapped to an admissible value: softplus for the multipliers, and a bounded-exponential map around a baseline for the positive levels. Concretely, writing z_h^k , z_h^q , and z^p for the raw network outputs, the heads are

$$\hat{k}^h = k_{\text{baseline}}^h \exp(\tanh z_h^k), \quad \hat{q}^h = q_{\text{baseline}}^h \exp(\tanh z_h^q), \quad \hat{p} = p_{\text{baseline}} \exp(\tanh z^p), \quad (5.15)$$

so the four non-negativity inequalities $\hat{k}^h \geq 0$, $\hat{\lambda}_b^h \geq 0$, $\hat{q}^h \geq 0$, $\hat{\mu}^h \geq 0$ hold *by construction*, leaving the orthogonality conditions of the KKT systems to be enforced softly in the loss (next paragraph).² The production network uses 1000×1000 hidden units (~ 1.5 M parameters); the teaching version uses 128×128 .

Equilibrium residuals. Each cohort $h \in \{1, \dots, A - 1\}$ contributes *four* residuals, one per equilibrium condition (slide III.6). To keep the displayed form compact, introduce *numerator/-denominator shorthands* for the two Euler conditions:

$$\begin{aligned} \mathcal{N}_k^h(\mathbf{x}_t) &:= \beta \mathbb{E}[r_{t+1} \mathcal{D}_k^{h+1}(\hat{\mathbf{x}}_{t,+}) u'(\hat{c}_{t+1}^{h+1})] + \hat{\lambda}_b^h + \hat{\mu}^h, & \mathcal{D}_k^h(\mathbf{x}_t) &:= 1 + \zeta(\hat{k}^h - r_t k_t^h), \\ \mathcal{N}_b^h(\mathbf{x}_t) &:= \beta \mathbb{E}[u'(\hat{c}_{t+1}^{h+1})] + \kappa \hat{\mu}^h, & \mathcal{D}_b^h(\mathbf{x}_t) &:= \hat{p}. \end{aligned}$$

Here $\mathcal{D}_k^h$ is the marginal-adjustment-cost wedge from $\Psi^h = \frac{\zeta}{2}(k^h - r_t k^h)^2$: the capital Euler equation in envelope form reads $u'(c_t^h) \mathcal{D}_k^h = \beta \mathbb{E}[r_{t+1} \mathcal{D}_k^{h+1} u'(c_{t+1}^{h+1})] + \lambda_b^h + \mu^h$, so the same wedge appears next period on the marginal return to capital (this is the factor `adj_factor_next` in the notebook). With $\zeta = 0$ it collapses to the textbook Euler equation. The bond Euler reduces to the textbook stochastic-discount-factor form $\hat{p} = \beta \mathbb{E}[u'(c')]/u'(c)$ *only when the collateral constraint is slack* ($\hat{\mu}^h = 0$); whenever $\hat{\mu}^h > 0$, the bond price carries an additional shadow-value term $\kappa \hat{\mu}^h / u'(\hat{c}^h)$ that captures the value of relaxing the collateral constraint. The four per-cohort

²In the current notebook implementation $\hat{q}^h$ is parameterized *relative* to $\hat{k}^h$, so it cannot fall to zero while $\hat{k}^h > 0$; the collateral-complementarity residual is then satisfied by $\hat{\mu}^h \rightarrow 0$, and the collateral constraint is effectively non-binding on the learned ergodic set, consistent with the chapter-opening note. Allowing it to bind exactly requires a free positive slack output (a softplus head on $\hat{q}^h$); the architecture above already accommodates this swap.

residuals are then

$$\begin{aligned}
e_{\text{REE},k}^h(\mathbf{x}_t) &:= \frac{(u')^{-1}(\mathcal{N}_k^h(\mathbf{x}_t) / \mathcal{D}_k^h(\mathbf{x}_t))}{\hat{c}^h(\mathbf{x}_t)} - 1, & (\text{Euler}, k) \\
e_{\text{REE},b}^h(\mathbf{x}_t) &:= \frac{(u')^{-1}(\mathcal{N}_b^h(\mathbf{x}_t) / \mathcal{D}_b^h(\mathbf{x}_t))}{\hat{c}^h(\mathbf{x}_t)} - 1, & (\text{Euler}, b) \\
e_{\text{KKT},b}^h(\mathbf{x}_t) &:= \hat{\lambda}_b^h \cdot \hat{k}'^h, & (\text{borrowing complementarity}) \\
e_{\text{KKT},c}^h(\mathbf{x}_t) &:= \hat{\mu}^h \cdot (\hat{k}'^h + \kappa \hat{b}'^h) = \hat{\mu}^h \cdot \hat{q}^h. & (\text{collateral complementarity})
\end{aligned} \tag{5.16}$$

On top of these per-agent residuals the bond market must clear: bonds are in zero net supply, so the residual is the cross-sectional sum of bond holdings against the target $\bar{B} = 0$,

$$e_{\text{MC},b}(\mathbf{x}_t) := \sum_{h=1}^A \hat{b}'^h(\mathbf{x}_t) - \bar{B} = \frac{1}{\kappa} \sum_{h=1}^{A-1} (\hat{q}^h - \hat{k}'^h), \quad \bar{B} = 0. \tag{5.17}$$

Capital-market clearing $K_{t+1} = \sum_{h=2}^A k_{t+1}^h$ is once again satisfied by construction and does *not* appear as a residual. The conditional expectation in the two Euler equations is computed exactly as in (5.10): by summing over the four next-period shocks weighted by the persistent-Markov transition probabilities $\pi(z_t, \cdot)$.

The DEQN loss for the 56-agent benchmark. Stack the four per-cohort residuals into one squared-cohort term $R^h(\mathbf{x})^2 := (e_{\text{REE},k}^h)^2 + (e_{\text{REE},b}^h)^2 + (e_{\text{KKT},b}^h)^2 + (e_{\text{KKT},c}^h)^2$, then add the bond-market-clearing residual. The mini-batch loss is

$$\mathcal{L}_{D_{\text{train}}}(\theta) = \frac{1}{|D_{\text{train}}|} \frac{1}{4(A-1)+1} \sum_{\mathbf{x}_j \in D_{\text{train}}} \left[\sum_{h=1}^{A-1} R^h(\mathbf{x}_j)^2 + (e_{\text{MC},b}(\mathbf{x}_j))^2 \right] \tag{5.18}$$

(*matching slide III.6*). With $A = 56$ this is $4 \times 55 + 1 = 221$ squared residuals per training state. Each residual enters with weight one: no adaptive loss balancing (cf. Chapter 4) is applied because the relative-Euler convention (5.10) already homogenizes the per-cohort Euler scales, and the product-form KKT residuals are unit-free under the softplus head; ReLoBRaLo or GradNorm would be the natural next step if a future calibration broke this homogeneity. Comparison with (5.11): the analytic case is the special instance of (5.18) in which the no-short-sale-of-capital constraint never binds (so $\lambda_b^h \equiv 0$), there are no bonds (so all b - and collateral-related blocks drop out), and $4(A-1)+1$ collapses to $A-1$. The two losses are the same template instantiated at different complexity. Table 5.4 unpacks the residual blocks.

Component	Symbol	Count
Euler (capital)	$e_{\text{REE},k}^h$	55
Euler (bonds)	$e_{\text{REE},b}^h$	55
KKT (borrowing)	$e_{\text{KKT},b}^h = \hat{\lambda}_b^h \hat{k}'^h$	55
KKT (collateral)	$e_{\text{KKT},c}^h = \hat{\mu}^h \hat{q}^h$	55
Market clearing (bonds)	$e_{\text{MC},b} = \sum_h \hat{b}'^h$	1
Total residuals		221

Table 5.4: Residual blocks entering the 56-agent benchmark loss for one training state.

Training and results. Production training uses 60,000 episodes at $\text{lr} = 10^{-5}$ followed by 140,000 episodes at $\text{lr} = 10^{-6}$, with runtime of several hours on GPU. The teaching version (~ 200 segments, 128-128 hidden units) runs in a few minutes on CPU and is meant to show the

mechanics and qualitative lifecycle patterns, not final accuracy. The loss trajectory typically exhibits oscillations, caused by re-simulation of the capital path at each episode, but the overall trend is steadily downward.

Lifecycle diagnostics. The trained model produces economically plausible lifecycle patterns. Capital savings k^h follow a hump shape that mirrors the labor income profile: young agents save little (borrowing constraint binds), mid-career agents accumulate rapidly, and older agents decumulate. Bond holdings b^h are initially negative (young agents borrow against future income) and increase with age as agents shift from illiquid capital to liquid bonds. Bond prices vary across shock states, with higher prices in high-TFP states reflecting stronger demand for savings. In the teaching run the Euler residuals are still large enough to treat the output as diagnostic; in production runs the mean Euler equation errors are of order 10^{-4} – 10^{-3} for both capital and bond equations (matching Table 3 of Azinovic et al. (2022)), corresponding to a $\sim 0.01\%$ – 0.1% deviation in consumption. Market clearing residuals are comparably small. Convergence is also confirmed by the policy-drift check on the fixed anchor cloud: the run is treated as time-invariant once `policy_drift_rms` and `policy_drift_max` fall below their prescribed tolerances.

Scalability of the DEQN framework

The *same algorithm* that solved the 6-agent model scales to 56 agents with two assets and inequality constraints. Only the network size and training duration change; the DEQN framework itself is unchanged. The 56-agent benchmark has a 113-dimensional minimal state and a 240-dimensional engineered network input, both infeasible for traditional grid-based methods.

Chapter Summary

OLG models discretize the cross-section into a finite number of cohorts A , so the minimal state vector contains the aggregate shock together with the cohort asset holdings and has dimension $\mathcal{O}(A)$. Market clearing $\sum_h k_{t+1}^h = K_{t+1}$ closes the model and is the natural place to encode aggregation exactly via a market-clearing output layer (Chapter 2, footnote on Azinovic–Yang & Žemlička 2024). No-short-sale-of-capital and collateral constraints introduce KKT complementarity, which we enforce by combining softplus output activations (for non-negativity) with squared product residuals $(a \cdot b)^2$ in the loss. Across both instances, the 6-agent analytic OLG and the 56-agent IER benchmark, the *same* training loop covers the spectrum from textbook closed-form validation to research-scale scalability.

Further Reading

- Azinovic et al. (2022), the IER paper that established the 56-agent benchmark.
- Auerbach and Kotlikoff (1987), the classical OLG reference.
- Azinovic-Yang and Žemlička (2024), market-clearing output layer in OLG with rare disasters.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. [Core] **OLG market clearing for $A = 3$.** Write the budget constraint and Euler equation for each of three cohorts (young, middle, old) and the market-clearing condition that closes the model. Verify that the count of equations matches the count of unknown policies.

2. **[Core] KKT residuals: product form vs. Fischer–Burmeister.** For a borrowing-constrained OLG agent, write the KKT conditions and rewrite them as *both* (i) a product-form residual $(k'^h \cdot \lambda^h)^2$ and (ii) a Fischer–Burmeister residual $\Phi(\lambda^h, k'^h)^2$ with $\Phi(a, b) = a + b - \sqrt{a^2 + b^2}$. In each case, show that the loss vanishes *exactly* at any KKT point, not just approximately. Using the rule of thumb in §5.4, argue which formulation you would choose for the *6-agent analytic OLG* (§5.2, where the borrowing constraint is non-binding on the ergodic set) versus the *investment-irreversibility constraint* of Chapter 3 (which binds on a non-trivial fraction of states).
3. **[Computational] Hump-shaped lifecycle.** Argue why a hump-shaped labor-income profile (rising through working years, falling after retirement) implies a hump in the savings policy k^h . Sketch the expected shape and check it against the trained-network output in notebook `lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb`.
4. **[Computational] Hard aggregation layer.** The analytic notebook currently predicts cohort savings and then defines aggregate next-period capital as their sum, so capital-market clearing is already exact. Implement an alternative architecture in notebook `lecture_08_08_OLG_Analytic_DEQN_persistent.ipynb` in which the network outputs a scalar $\widehat{K}_{t+1} > 0$ and unnormalized cohort scores $(z^2, \dots, z^A)$, then sets $s^h = \text{softmax}(z^h)$ and $k_{t+1}^h = \widehat{K}_{t+1} s^h$. Verify that $\sum_{h=2}^A k_{t+1}^h = \widehat{K}_{t+1}$ is at machine precision (below 10^{-12}) at every training step. Compare the Euler residual and runtime against the current “sum-of-savings” implementation, and explain why the hard layer is useful mainly when aggregate K_{t+1} is a separate policy head.
5. **[Core] Bond pricing in equilibrium.** Assume the borrowing/collateral constraint is slack for cohort h throughout this exercise. In the 56-agent OLG (§5.5) cohort h holds capital k^h and one-period riskless bonds b^h ; capital pays the stochastic gross return R_{t+1} , bonds pay one unit of consumption next period and trade at price p_t . Write the agent’s Euler equations for capital and for bonds separately. Eliminate the marginal utilities to derive the equilibrium bond-pricing equation
$$p_t = \frac{\beta \mathbb{E}_t[u'(c_{t+1}^{h+1})]}{u'(c_t^h)},$$
and show that the same expression equals $\mathbb{E}_t[M_{t,t+1}]$ for the stochastic discount factor $M_{t,t+1} := \beta u'(c_{t+1}^{h+1})/u'(c_t^h)$. Show that absence of arbitrage between bonds and capital implies $1/p_t = \mathbb{E}_t[R_{t+1}] + \text{Cov}_t(M_{t,t+1}, R_{t+1})/p_t$, equivalently $\mathbb{E}_t[R_{t+1}] - 1/p_t = -\text{Cov}_t(M_{t,t+1}, R_{t+1})/p_t$, the standard risk-premium decomposition. Then explain qualitatively how the bond-pricing equation changes when the collateral multiplier $\mu_t^h > 0$ is positive (i.e., the constraint binds), and identify which cohorts are most likely to be affected. Briefly explain why the 6-agent analytic OLG of §5.2 does not need an explicit bond-market residual in its DEQN loss.
6. **[Advanced/project] Collateral sensitivity.** In the 56-agent benchmark notebook `lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb`, the collateral constraint is $k'^h + \kappa b'^h \geq 0$ with reference $\kappa = 1/(1 - \delta_{\max}) \approx 1.1236$. Sweep $\kappa \in \{0.8, 1.0, 1.1236, 1.3, 1.5\}$, retraining the network for each. Report (i) the fraction of ergodic states at which the collateral constraint binds, measured by $\hat{q}^h < 10^{-4}$ together with a positive multiplier $\hat{\mu}^h > 10^{-4}$, (ii) the cross-cohort dispersion of bond holdings b^h at the deterministic steady state, (iii) the equilibrium bond price p_{ss} . Plot all three against κ and explain qualitatively how tightening the collateral requirement (larger κ , hence less negative admissible bond positions for a fixed k'^h) changes portfolio choice.
7. **[Advanced/project] KKT-binding frequency under aggregate volatility.** Still in the 56-agent benchmark, vary the standard deviation of the aggregate productivity shock $\sigma_z \in \{0.005, 0.01, 0.02, 0.04\}$ (the reference calibration uses $\sigma_z \approx 0.01$). For each, retrain and

report the fraction of ergodic-set draws on which (a) the borrowing constraint $k'^h \geq 0$ binds and (b) the collateral constraint $k'^h + \kappa b'^h \geq 0$ binds, broken out by cohort age. Use the same small-slack/positive-multiplier convention as in Exercise 6. Show that the binding fraction grows roughly linearly in σ_z for the youngest cohorts but stays near zero for older cohorts, and connect this to the chapter's recommendation (§5.4) that product-form KKT residuals suffice when the constraint binds rarely.

Chapter 6

Heterogeneous Agents and Young’s Method

The OLG models of Chapter 5 featured a finite number of agent types, so the cross-sectional state was simply a vector $(k^1, \dots, k^A)$. Many important macroeconomic applications instead require a *continuum* of agents subject to idiosyncratic shocks. In the [Krusell and Smith \(1998\)](#) economy, an aggregate productivity shock additionally moves the cross section in a stochastic way, and the entire wealth distribution μ_t then becomes part of the aggregate state. The [Aiyagari \(1994\)](#) model is the special case without aggregate risk: μ_t is fixed in stationary equilibrium and evolves deterministically along transitions, so it is a parameter of the equilibrium rather than a stochastic state variable. Incomplete markets prevent full insurance in both, making explicit distributional tracking essential, but the “master-equation” challenge of treating μ_t as a high-dimensional aggregate state arises only once aggregate risk is added on top of Aiyagari. Why represent μ_t as a histogram on a discrete grid rather than as a *Monte Carlo panel* (a finite simulated sample of N agents whose empirical cross-section stands in for μ_t)? Two reasons motivate the choice up front: a histogram is *deterministic*, so re-running the same equilibrium gives identical aggregates and the loss is a smooth function of the network weights, and it is *noise-free*, so Euler-equation residuals reflect approximation error rather than $\mathcal{O}(1/\sqrt{N})$ Monte Carlo sampling noise (Figure 6.5 makes this contrast quantitative). This chapter develops Young’s non-stochastic simulation method ([Young, 2010](#)) for representing μ_t as a histogram and shows how to embed that method within the DEQN framework of [Azinovic et al. \(2022\)](#) to solve heterogeneous-agent economies with neural network policy functions.

How this chapter maps onto the slides and notebooks. The heterogeneous-agent material of this chapter, together with the companion deck in `lecture_09_heterogeneous_agents_youngs_method/slides/`, can be read independently of the sequence-space material in §6.7; readers already comfortable with Krusell–Smith may skip straight there on a first pass. Two notebooks in `lecture_09_heterogeneous_agents_youngs_method/code/` accompany Sections 6.3–6.4: `lecture_09_10_Youngs_Method_Examples.ipynb` isolates Young’s redistribution operator on toy examples, and `lecture_09_11_Continuum_of_Agents_DEQN.ipynb` embeds the same operator inside the Appendix A.5 endowment-economy DEQN. Section 6.6 on alternative deep-learning approaches is paired with `lecture_09_12_KrusellSmith_DeepLearning.ipynb`, a classroom-scale all-in-one DL solver in the spirit of [Maliar et al. \(2021\)](#).

The Bewley–Huggett–Aiyagari lineage. The continuum-agent framework that this chapter operationalizes has three foundational sources. [Bewley \(1986\)](#) introduced stationary monetary equilibrium with a continuum of agents subject to iid endowment shocks; the explicit self-insurance-through-borrowing-constraints mechanism that defines the modern incomplete-markets workhorse is due to [İmrohoroglu \(1989\)](#), [Huggett \(1993\)](#), and [Aiyagari \(1994\)](#). [Huggett \(1993\)](#)

cast the idea as a tractable endowment economy with a single risk-free asset, focusing on the equilibrium interest rate. Aiyagari (1994) added neoclassical production, closing the model in general equilibrium with capital accumulation; this is the canonical incomplete-markets economy. Krusell and Smith (1998) layered aggregate productivity shocks on top, producing the modern Krusell–Smith economy that this chapter targets, and its continuous-time reformulation is developed by Achdou et al. (2022) and revisited in Chapter 8.

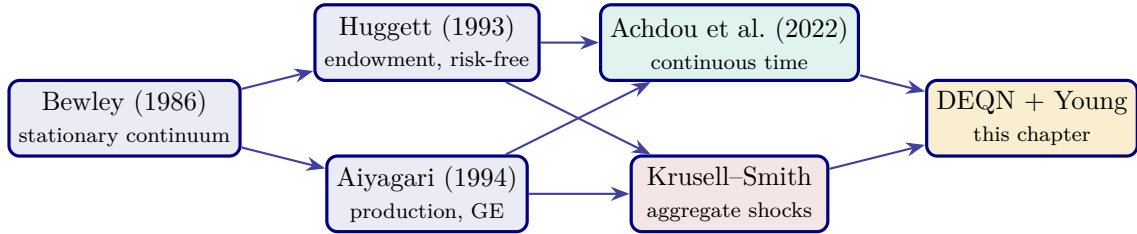


Figure 6.1: Genealogy of the heterogeneous-agent models treated in this script. This chapter targets the Krusell–Smith branch (right) by combining a DEQN policy with Young’s histogram update; the continuous-time branch (Achdou–Han–Lasry–Lions–Moll) is taken up in Chapter 8.

6.1 From Representative to Heterogeneous Agents

In the representative-agent models of Chapters 2–3, aggregate capital K_t is a sufficient statistic for the economy’s state. In reality, agents differ in wealth, income, and employment status, and incomplete markets prevent full insurance against idiosyncratic risk. This heterogeneity matters for policy analysis:

- Fiscal stimulus: agents near the borrowing constraint have high marginal propensities to consume; wealthy agents save most of a windfall.
- Monetary policy affects borrowers and savers differently.
- The shape of the wealth distribution feeds back into aggregate demand and equilibrium prices.

The mathematical challenge depends on whether the economy carries aggregate risk. Without it (the Aiyagari case), the cross-section μ_t is fixed in stationary equilibrium and evolves deterministically along transitions, so it parameterizes the equilibrium rather than serving as a stochastic state variable. With aggregate risk (the Krusell–Smith case that this chapter targets), an aggregate shock moves the cross-section in a stochastic way, and the entire distribution μ_t then becomes part of the aggregate state. Since μ_t is in either case a measure (an infinite-dimensional object), it cannot be placed on a grid without encountering the curse of dimensionality.

Two approaches. There are two main strategies for handling heterogeneous agents in discrete time. *Finite-agent methods* keep a high-dimensional but finite vector $(k_1, \dots, k_N)$ of individual states, which is exact in μ_t but pays a permutation-symmetry cost that scales poorly with N (the OLG models of Chapter 5, and the finite-agent Monte-Carlo *implementation* of the continuum-agent Krusell–Smith model by Maliar et al. (2010), fall in this category). *Continuum-agent methods* replace the agent labels with the distributional state $\mu_t(\cdot)$, which is infinite-dimensional but eliminates sampling noise and the permutation problem, at the price of needing to approximate μ_t (Krusell and Smith (1998) and Young (2010) are the canonical references). This chapter pursues the second approach: tracking the distribution via a histogram, with Young’s histogram method (Young, 2010) providing the distribution update.

6.2 The Krusell–Smith (1998) Economy

The canonical heterogeneous-agent model with aggregate uncertainty is due to Krusell and Smith (1998). Its key features are:

- A *continuum* of ex ante identical, infinitely-lived agents.
- *Incomplete markets*: no insurance against idiosyncratic labor income risk.
- *Aggregate uncertainty*: productivity shocks that affect all agents.
- A single asset (capital/bonds) with a borrowing constraint.

Each agent must forecast future prices to make optimal savings decisions. But prices depend on the entire wealth distribution, which is infinite-dimensional. The key insight of Krusell and Smith (1998) is that the *mean* of the wealth distribution is a nearly sufficient statistic for forecasting: a log-linear forecasting rule

$$\hat{H}(m_1, a) = \exp(A(a) + B(a) \log m_1), \quad (6.1)$$

where $\hat{H}(m_1, a)$ is the approximate next-period aggregate capital (the price-forecasting function), $A(a)$ and $B(a)$ are the OLS intercept and slope coefficients (each a function of the aggregate shock state a), and $m_1 = \int k d\mu$ is mean capital; this rule achieves $R^2 > 0.9999$ in practice.

Approximate aggregation: what it means and what it does *not* mean

The empirical observation that a single moment $m_1 = \int k d\mu$ is sufficient for price forecasting is known as *approximate aggregation*. Conceptually, it says that the equilibrium price mapping r_{t+1}, w_{t+1} depends on the cross-sectional distribution *almost only through its mean*, so the high-dimensional state μ_t collapses to a one-dimensional summary for forecasting purposes.

Importantly, Krusell and Smith (1998) did not restrict themselves to the one-moment specification. Their original paper explicitly reports results for two- and three-moment forecasting rules (adding the variance, and further adding the skewness of the wealth distribution) and finds that additional moments provide only marginal improvement: R^2 barely changes. The one-moment benchmark is therefore an *empirical* finding, not a modeling assumption.

Two cautions are essential. First, this is a property of the *Krusell–Smith model class* (one risky asset, modest wealth dispersion, moderate income risk) and is not a general theorem; in models with multiple assets, large MPC heterogeneity, or occasionally binding constraints that bite on a non-trivial mass of agents, the forecasting rule typically requires higher moments or a more flexible function approximator (Chapters 9, 11). Second, $R^2 > 0.9999$ on simulated mean dynamics does *not* imply that the distribution itself is well summarized by its mean: tails, percentiles, and the borrowing-constrained mass continue to matter for welfare and policy analysis. Approximate aggregation is therefore best read as a *computational* result, and Sections 6.3–6.4 accordingly track the full histogram and use the mean only as one of the network’s input features rather than as a state-space substitute.

Equilibrium. A recursive competitive equilibrium consists of:

1. Individual optimization: each agent chooses savings k' to maximize utility given the forecasting rule for aggregate prices (equivalently, for next-period aggregate capital K').
2. Market clearing: aggregate demand equals supply.

3. Consistency between individual policies and the law of motion for the distribution: $\mu_{t+1} = T(\mu_t, a_t)$ given the optimal savings policies just derived.
4. Rational expectations: agents' forecasts are self-confirming.

The question is: how do we simulate the distribution forward to compute the realized mean and check the forecasting rule?

The traditional Krusell–Smith algorithm in detail. Before turning to the distribution-update step, it is useful to write the canonical KS algorithm explicitly. It is a nested fixed-point iteration with an *outer* loop over forecasting-rule coefficients and an *inner* loop that solves the individual household's Bellman equation given those coefficients.

Krusell–Smith (1998), Traditional Algorithm

Require: Initial forecast coefficients $(A^{(0)}(a), B^{(0)}(a))$ from Eq. (6.1); capital grid $\mathcal{K}$; tolerances $\varepsilon_{\text{inner}}, \varepsilon_{\text{outer}}$; simulation length T_{sim} .

Key notation: a : aggregate TFP shock; ε : idiosyncratic employment shock; $m_1 = \int k d\mu$: mean capital; β : household discount factor; $u(c)$: period utility; $c = w(\hat{H})y(\varepsilon) + (1 + r(\hat{H}))k - k'$: consumption (wages w and return r come from the forecasting rule $\hat{H}$); k' : end-of-period savings (the policy choice); ε', a' : next-period shocks; R^2 : OLS coefficient of determination.

for outer iteration $\ell = 0, 1, 2, \dots$ until $\|A^{(\ell+1)} - A^{(\ell)}\| + \|B^{(\ell+1)} - B^{(\ell)}\| < \varepsilon_{\text{outer}}$ **do**

(a) **Solve individual Bellman equation** given $\bar{H}(m_1, a) = \exp(A^{(\ell)}(a) + B^{(\ell)}(a) \log m_1)$.

Iterate $V^{(n+1)}(k, \varepsilon, m_1, a) = u(c) + \beta \mathbb{E}[V^{(n)}(k', \varepsilon', \hat{H}(m_1, a), a')]$ until $\|V^{(n+1)} - V^{(n)}\| < \varepsilon_{\text{inner}}$.

Return policy $k'(k, \varepsilon, m_1, a)$.

(b) **Simulate the cross-section forward** for T_{sim} periods with (say) 10,000 agents, starting from an initial distribution and drawing idiosyncratic shocks independently.

(c) **Update forecasting rule** by OLS on the simulated path: regress $\log m_1^{t+1}$ on $\log m_1^t$ within each aggregate-shock state to obtain $(A^{(\ell+1)}(a), B^{(\ell+1)}(a))$.

(d) **Convergence check:** retain the rule if $R^2 \geq 0.9999$ and the coefficient update $\|A^{(\ell+1)} - A^{(\ell)}\| + \|B^{(\ell+1)} - B^{(\ell)}\|$ is below $\varepsilon_{\text{outer}}$.

end for

The remarkable empirical finding of Krusell and Smith (1998) is that a log-linear rule in m_1 alone achieves $R^2 > 0.9999$ and very small forecast errors: the “approximate aggregation” result. Textbook implementations (e.g., the `econ-ark/KrusellSmith` REMARK) typically converge in ~ 20 outer iterations with standard parameters $\beta = 0.99$, $\alpha = 0.36$, log utility, aggregate shocks on {good, bad} with persistence ≈ 0.875 , and unemployment rates 4% (good) vs. 10% (bad).

Bottleneck. The inner Bellman solve on a two-dimensional (k, m_1) grid is the expensive step. It scales exponentially if one tries to add additional moments (e.g., tracking variance or skewness), which is why the KS algorithm as stated caps out at one moment. Extensions that need more moments must use more powerful function approximators. This is where the Young-histogram DEQN of §6.4 (and the alternatives in §6.6) come in.

6.3 Young's (2010) Non-Stochastic Simulation

Young (2010) proposed a deterministic method for propagating the wealth distribution that avoids Monte Carlo sampling entirely.

Core idea. Represent the wealth distribution at time t as a *histogram* over discrete bins, using two ingredients:

- a fixed capital grid $\{k_1, k_2, \dots, k_{N_k}\}$, where N_k is the number of bins and $i \in \{1, \dots, N_k\}$ indexes individual grid points k_i ;
- a finite set of idiosyncratic-shock states $\{\varepsilon_1, \dots, \varepsilon_{N_\varepsilon}\}$, indexed by $j \in \{1, \dots, N_\varepsilon\}$.

The histogram value $G_t(k_i, \varepsilon_j) \in [0, 1]$ is the probability mass of agents sitting at capital bin k_i in shock state ε_j at time t ; by construction, $\sum_{i,j} G_t(k_i, \varepsilon_j) = 1$.

Given the household policy function $k' = g(k, \varepsilon, m_1, a)$, where:

- k is the individual's current capital and ε their current idiosyncratic shock;
- $m_1 = \int k d\mu$ is aggregate mean capital (the summary statistic from Krusell–Smith);
- a is the aggregate productivity shock (the same a that drives prices in the KS setup above);

the key operation is *mass redistribution*. Evaluating g at a bin (k_i, ε_j) produces the savings target $k' = g(k_i, \varepsilon_j, m_1, a)$, which generically lies *off* the grid. Let $n = n(i, j) \in \{1, \dots, N_k - 1\}$ denote the bracketing index defined by $k_n \leq k' < k_{n+1}$, and let

$$m \equiv G_t(k_i, \varepsilon_j)$$

be the probability mass currently sitting at bin (k_i, ε_j) . This mass m is then split between the two bracketing grid points k_n and k_{n+1} using linear-interpolation weights:

$$\omega = 1 - \frac{k' - k_n}{k_{n+1} - k_n}, \quad \text{mass } \omega \cdot m \text{ to } k_n, \quad (1 - \omega) \cdot m \text{ to } k_{n+1}. \quad (6.2)$$

Figure 6.2 illustrates this operation. A mass m sitting at the off-grid savings target k' is split between the two bracketing grid points k_n and k_{n+1} , with the weight ω proportional to the proximity of k' to k_n (so $\omega \rightarrow 1$ when $k' \rightarrow k_n$ and $\omega \rightarrow 0$ when $k' \rightarrow k_{n+1}$).

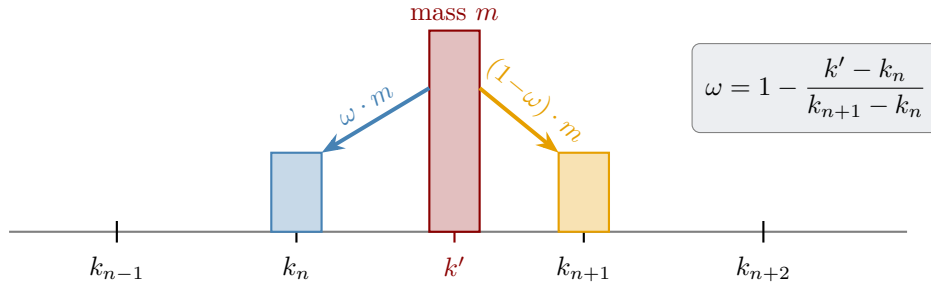


Figure 6.2: Linear interpolation in Young's method. Mass m at off-grid point k' is redistributed to the two bracketing grid points k_n and k_{n+1} using weights ω and $1 - \omega$. Closer proximity to a grid point yields a larger share of the mass.

Why exactly this weight? The lottery weight ω in Eq. (6.2) is not an arbitrary choice: it is the *unique* value for which the two-point split has conditional mean equal to the off-grid policy choice k' . Solving the equation $\omega \cdot k_n + (1 - \omega) \cdot k_{n+1} = k'$ for ω recovers Eq. (6.2) in one line:

$$\omega \cdot k_n + (1 - \omega) \cdot k_{n+1} = k_n + (1 - \omega)(k_{n+1} - k_n) = k_n + (k' - k_n) = k'. \quad (6.3)$$

By linearity of expectation, this conditional mean equality at every grid bin extends to the full distribution: the unconditional mean of G_{t+1} equals the unconditional mean of the policy-implied next-period capital. Higher moments (variance, percentiles) are only approximated; the leading error is of order $(\Delta k)^2$ on smooth densities, so a finer grid improves higher-moment fidelity at no cost to mean preservation.

Lottery interpretation: what makes this “non-stochastic”

Equation (6.2) can equivalently be read as a *fair lottery*: every agent at (k, ε) whose policy choice lands at off-grid k' is reassigned to k_n with probability ω and to k_{n+1} with probability $1 - \omega$. In a Monte Carlo panel of size N , each agent draws this lottery once and the empirical histogram converges to its expectation at rate $\mathcal{O}(N^{-1/2})$. Young’s method instead computes that expectation in closed form, which is equivalent to “running” an infinite number of agents through the lottery and integrating out the result. This is why the procedure is called non-stochastic: the lottery is still there, but its outcome is computed analytically rather than sampled. The same logic, applied to the discrete shock transitions $\pi_{\varepsilon'|\varepsilon}$, sends each piece of mass into all reachable next-period ε' in proportion to π rather than drawing one realization.

Worked example. We illustrate the full histogram update with a small grid of $N_k = 4$ capital levels $\{1.0, 2.0, 3.0, 4.0\}$ and two idiosyncratic states $\varepsilon \in \{\text{low}, \text{high}\}$.

Step 1, Setup. The initial histogram G_0 (masses summing to 1):

	$k = 1.0$	$k = 2.0$	$k = 3.0$	$k = 4.0$	Row sum
$\varepsilon = \text{low}$	0.10	0.20	0.10	0.05	0.45
$\varepsilon = \text{high}$	0.05	0.15	0.20	0.15	0.55

The mean capital is $\bar{k}_0 = \sum_{i,j} G_0(k_i, \varepsilon_j) \cdot k_i = 0.10(1) + 0.20(2) + 0.10(3) + 0.05(4) + 0.05(1) + 0.15(2) + 0.20(3) + 0.15(4) = 2.55$.

Step 2, Policy evaluation. Let $y(\varepsilon)$ denote the dollar productivity associated with shock state ε , with $y_{\text{low}} = 1$ and $y_{\text{high}} = 3$ (here ε is the *state index*, y is its numerical value). Using a simple linear savings rule $k' = 0.4k + 0.5y(\varepsilon)$:

	$k = 1.0$	$k = 2.0$	$k = 3.0$	$k = 4.0$
$\varepsilon = \text{low: } k'$	0.9	1.3	1.7	2.1
$\varepsilon = \text{high: } k'$	1.9	2.3	2.7	3.1

Step 3, Interpolation weights. Since the grid is equispaced with $\Delta k = 1.0$, each off-grid k' is bracketed by $[k_n, k_{n+1}]$ with weight $\omega = 1 - (k' - k_n)/\Delta k$ (it is the equispacing, not the unit value, that makes this expression linear in the bracket index):

	$k = 1.0$	$k = 2.0$	$k = 3.0$	$k = 4.0$
$\varepsilon = \text{low: bracket, } \omega$	[1, 1], clip	[1, 2], 0.7	[1, 2], 0.3	[2, 3], 0.9
$\varepsilon = \text{high: bracket, } \omega$	[1, 2], 0.1	[2, 3], 0.7	[2, 3], 0.3	[3, 4], 0.9

When $k' < k_1 = 1.0$ (here $k' = 0.9$ for the low-state, lowest-capital agents), all mass is assigned to k_1 (clipped to the boundary).

Step 4, Mass redistribution. For simplicity, assume the shock transition is the identity ($\varepsilon' = \varepsilon$), so mass stays in its current ε -state. Building G_1 by accumulating the redistributed mass from each source bin (mass is conserved bin-by-bin: e.g. the $\varepsilon = \text{low}$, $k = 2$ source of mass 0.20 contributes $0.7 \cdot 0.20 = 0.14$ to $k = 1$ and $0.3 \cdot 0.20 = 0.06$ to $k = 2$):

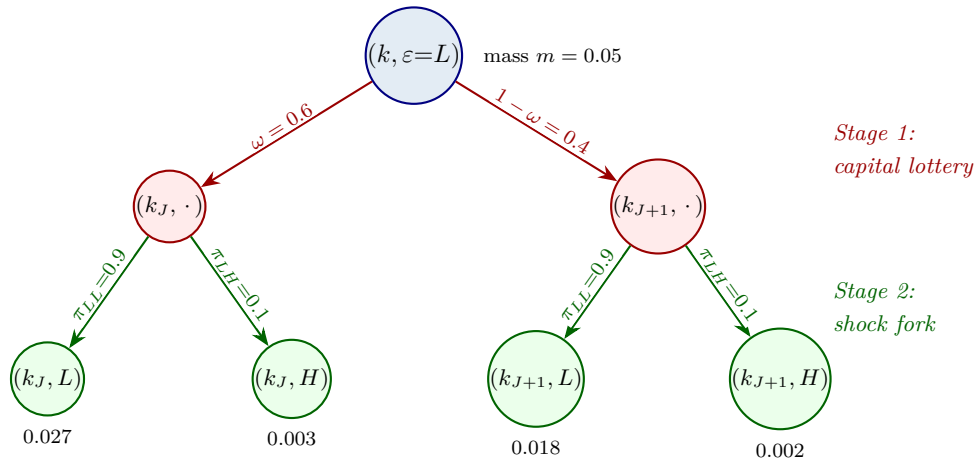
	$k = 1.0$	$k = 2.0$	$k = 3.0$	$k = 4.0$	Row sum
$\varepsilon = \text{low}$	0.270	0.175	0.005	0.000	0.450
$\varepsilon = \text{high}$	0.005	0.210	0.320	0.015	0.550
total	0.275	0.385	0.325	0.015	1.000

Step 5, Mean verification. The mean of G_1 is $\bar{k}_1 = 0.275(1) + 0.385(2) + 0.325(3) + 0.015(4) = 2.08$. The unclipped, policy-implied mean is $\sum_{i,j} G_0(k_i, \varepsilon_j) k'(k_i, \varepsilon_j) = 2.07$, so boundary clipping at $k' = 0.9$ raises the mean by only 0.01. Mean preservation is exact for source bins whose policy choice k' lies strictly inside the grid; clipping at the boundary slightly biases the mean, here upward, because mass that would have landed at $k' = 0.9$ is forced to $k = 1$. In general, boundary clipping biases the mean in the *direction of the violated boundary*: clipping at $k_{\min}$ biases the mean upward (mass is pulled in from below the grid), clipping at $k_{\max}$ biases it downward. With a wider grid ($k_{\min} < 0.9$) the mean would be preserved exactly.

Practical take-away

This small example shows why the grid must extend beyond the range of the policy function. In production code, a safeguard checks that boundary bins contain negligible mass (typically $< 10^{-6}$).

The full picture: one cell splits into four. The worked example above used identity shock transitions to keep the arithmetic visible. The general case combines the capital lottery with a *shock fork*: each piece of mass split between k_J and k_{J+1} is then split again across the reachable next-period ε' values according to $\pi_{\varepsilon'|\varepsilon,a}$. With two shocks $\varepsilon \in \{L, H\}$ this produces *four* destination bins for every source bin, with weights given by the product $\{\omega, 1 - \omega\} \times \{\pi_{\varepsilon L}, \pi_{\varepsilon H}\}$. Figure 6.3 reproduces this two-stage cascade (essentially Fig. 1 of Young (2010)), annotated with a concrete numerical case.



Verification: $0.027 + 0.003 + 0.018 + 0.002 = 0.05 = m$ ✓. The conditional mean of next-period capital is $\omega k_J + (1 - \omega) k_{J+1} = k'$, by Eq. (6.3).

Figure 6.3: Young's cascade for one source bin (essentially Fig. 1 of Young (2010)). Mass m at $(k, \varepsilon=L)$ flows in two stages: the capital lottery (ω vs. $1 - \omega$) sends it to the bracketing grid points k_J and k_{J+1} , and the shock fork (π_{LL} vs. π_{LH}) splits each piece across the reachable next-period idiosyncratic states. Each of the four leaves receives the product of its stage-1 and stage-2 weights times the source mass; the four leaf masses sum back to m . Repeating the cascade for every active source bin and accumulating the leaves yields the new histogram G_{t+1} .

A reader implementing the method should recognize three properties from the figure: (i) mass is conserved bin-by-bin because $\omega + (1 - \omega) = 1$ and $\sum_{\varepsilon'} \pi_{\varepsilon \varepsilon'} = 1$; (ii) the capital lottery's expected next-period k equals the policy choice k' by Eq. (6.3); (iii) the entire update is *linear* in G_t : $G_{t+1} = T(\rho) G_t$, where the transition operator T depends on the current policy. Linearity is what enables differentiation through the histogram step; separately, T is *sparse* (each column has at most $2|\mathcal{E}|$ non-zeros from the bracket lottery times the shock transitions), which is what makes the matrix-vector product cheap to evaluate in practice. This linearity is what makes the histogram

update differentiable in the policy values almost everywhere, conditional on the interpolation brackets, when we embed it inside a neural-network training loop in §6.4. Index changes at bin boundaries and clipping at domain edges are nondifferentiable; standard implementations either ignore these measure-zero events, smooth the assignment, or stop gradients through the index-selection step, and in practice none of these choices materially affects training in the calibrations covered in this chapter.

The histogram update algorithm. At each time step, the histogram is updated deterministically:

Algorithm 1 Young’s histogram update at time t

Require: Current histogram G_t , policy $g(\cdot)$, transition matrix $\pi_{\varepsilon'|\varepsilon,a}$, grid $\{k_j\}$

```

Initialize  $G_{t+1}(k_j, \varepsilon') = 0$  for all  $(j, \varepsilon')$ 
for each  $(k_i, \varepsilon_j)$  with  $G_t(k_i, \varepsilon_j) > 0$  do
    Compute optimal savings:  $k' = g(k_i, \varepsilon_j, m_1, a_t)$ 
    Find bracketing grid points:  $k' \in [k_J, k_{J+1}]$ 
    Compute weight:  $\omega = 1 - (k' - k_J)/(k_{J+1} - k_J)$ 
    for each next-period shock  $\varepsilon'$  do
         $G_{t+1}(k_J, \varepsilon') += \omega \cdot \pi_{\varepsilon'|\varepsilon_j, a_t} \cdot G_t(k_i, \varepsilon_j)$ 
         $G_{t+1}(k_{J+1}, \varepsilon') += (1 - \omega) \cdot \pi_{\varepsilon'|\varepsilon_j, a_t} \cdot G_t(k_i, \varepsilon_j)$ 
    end for
end for
return Updated histogram  $G_{t+1}$ 

```

Implementation cheatsheet. The pseudocode above translates into a few lines of NumPy; the inner double loop can also be vectorised with `np.add.at` for production use.

```

import numpy as np

def young_step(G, kp, pi_eps, k_grid):
    """One Young (2010) histogram update on a uniform k-grid.
    G[i,j]      mass at (k_grid[i], eps_j),      sums to 1
    kp[i,j]     policy choice k'(k_i, eps_j),    possibly off-grid
    pi_eps[j,jp] = Pr(eps_{t+1}=jp | eps_t=j)
    """
    G_next = np.zeros_like(G)
    dk      = k_grid[1] - k_grid[0]                # uniform spacing
    n_k     = len(k_grid)
    for j in range(G.shape[1]):                    # current shock
        for i in range(G.shape[0]):                # current capital bin
            x = kp[i, j]
            # Boundary handling: clip BOTH the bracket index J AND the
            # lottery weight omega. Otherwise an off-grid x produces
            # omega < 0 or omega > 1 and hence negative or super-unit mass.
            if x <= k_grid[0]:
                J, omega = 0, 1.0                    # all mass to the floor
            elif x >= k_grid[-1]:
                J, omega = n_k - 2, 0.0              # all mass to the cap
            else:
                J = int((x - k_grid[0]) // dk)
                J = min(max(J, 0), n_k - 2)
                omega = (k_grid[J + 1] - x) / dk      # in [0, 1]
            for jp in range(G.shape[1]):            # next-period shock
                w = pi_eps[j, jp] * G[i, j]          # shock fork x source
                mass
                G_next[J, jp] += omega * w
                G_next[J + 1, jp] += (1 - omega) * w

```

```
return G_next
```

The four scatter-add lines correspond exactly to the four leaves of Figure 6.3: each leaf receives ω or $(1-\omega)$ from the capital lottery, multiplied by $\text{pi_eps}[j, \text{jp}]$ from the shock fork, multiplied by the source mass. The Krusell–Smith JAX tutorial in `lectures/lecture_10_sequence_space_deqns/code/lecture_10_KrusellSmith_Tutorial_CPU.ipynb` implements this same operation as `distribution_step`, vectorised over the grid via `jax.vmap` and accumulated with `.at[].add()`.

Closed-form bracketing on GPUs, and how to keep it on log-spaced grids

The expensive sub-step of Young’s update (and of any piecewise-linear interpolation) is the *bracketing index*

$$J(k') = \max\{n : k_n \leq k'\}, \quad k' \in [k_J, k_{J+1}).$$

The textbook implementation uses a binary search (e.g. `numpy.searchsorted` or `jnp.searchsorted`): $\mathcal{O}(\log N_k)$ per query, with data-dependent branches. On a CPU this is essentially free; on a GPU under XLA/CUDA it is one of the worst patterns one can write, because (i) SIMT threads of the same warp take different branches (warp divergence), (ii) the resulting gathers are irregular, and (iii) the opaque search op breaks operator fusion with the surrounding arithmetic, forcing extra kernel launches.

For an *equidistant* grid the search collapses to a single fused multiply–add and a cast,

$$J(k') = \text{clip}\left(\left\lfloor \frac{k' - k_0}{\Delta k} \right\rfloor, 0, N_k - 2\right),$$

which is exactly what the line `J = int((x - k_grid[0]) // dk)` in the cheatsheet does. This is branch-free, uniform across threads, and fuses with the lottery weight $\omega = (k_{J+1} - k')/\Delta k$ into a single GPU kernel.

Log-spaced grids without losing $\mathcal{O}(1)$ lookup. Uniform k -grids waste resolution where the consumption policy is flat (the right tail) and starve resolution where it is steepest (near the borrowing constraint $k \rightarrow 0$). A simple fix preserves the closed-form bracketing: place equidistant knots in a *transformed* coordinate $x = \phi(k)$ that maps the desired refinement region uniformly. For a log-spaced grid with shift $c > 0$,

$$x_n = x_0 + n \Delta x, \quad k_n = e^{x_n} - c, \quad n = 0, \dots, N_k,$$

the bracketing index becomes

$$J(k') = \text{clip}\left(\left\lfloor \frac{\log(c+k') - x_0}{\Delta x} \right\rfloor, 0, N_k - 2\right),$$

again $\mathcal{O}(1)$ per query and branch-free. Crucially, the *index* is computed in x -space but the lottery *weights* are taken in the original k -space (using k_J, k_{J+1} from the table), so the interpolated policy remains piecewise linear in k , consistent with the on-grid consumption values and with the mean-preserving lottery of Eq. (6.3). More generally, any bijective ϕ for which ϕ^{-1} is cheap admits the same trick. See `interpolate()` and `distribution_step()` in the Krusell–Smith JAX tutorial of Azinovic-Yang and Žemlička (2025a) for a production implementation.

Figure 6.4 visualizes the five stages of a single forward step.

Comparison with Monte Carlo. Young’s method produces *zero sampling noise* (deterministic), preserves the mean *exactly*, requires only $\sim 100\text{--}5,000$ grid points (versus $> 50,000$ agents for

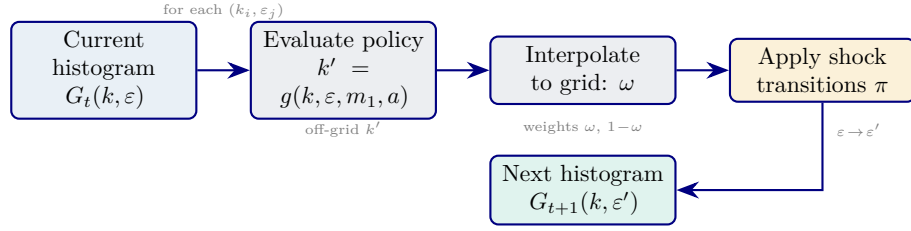


Figure 6.4: Flow diagram for one forward step of Young’s histogram update (Algorithm 1). Starting from G_t , the policy function is evaluated at every active bin, the resulting off-grid savings are interpolated back onto the grid, and idiosyncratic shock transitions redistribute mass across ε -states to produce G_{t+1} .

Monte Carlo), and is fully reproducible. The trade-off is of a different kind than Monte Carlo’s: the mean-preserving lottery introduces a deterministic *discretization bias* into higher moments (it tends to flatten cross-sectional kurtosis at coarse grids and is sensitive to the choice of bracket scheme), whereas a Monte Carlo panel is unbiased in all moments but pays $\mathcal{O}(N^{-1/2})$ sampling noise on each. Both methods also require a grid (Young) or an empirical range (Monte Carlo) that is wide enough to contain all mass. The following table summarizes the comparison:

	Young’s method	Panel simulation
Sampling noise	None	$\mathcal{O}(1/\sqrt{N})$
Mean preservation	Exact	Approximate
Typical size	100–5,000 grid points	>50,000 agents
Reproducibility	Deterministic	Seed-dependent
Higher moments	Approximated	Approximated

Figure 6.5 contrasts the two approaches visually: the histogram method yields a smooth, noise-free distribution, while a Monte Carlo panel of comparable size exhibits visible sampling noise.

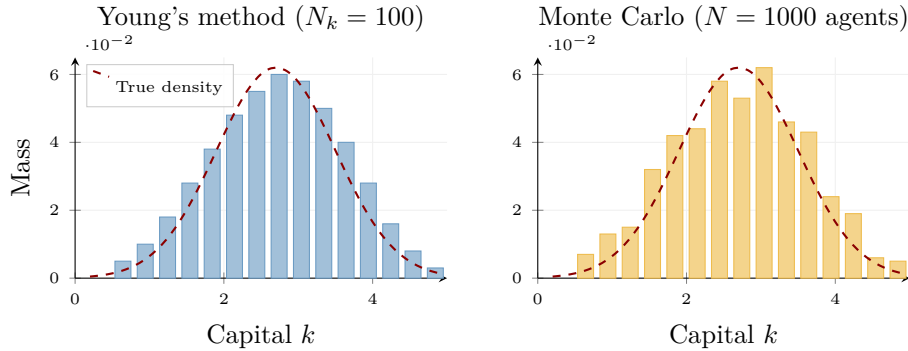


Figure 6.5: Young’s histogram (left) versus Monte Carlo panel simulation (right). Both approximate the same underlying wealth density (dashed). The histogram method is deterministic and smooth; the Monte Carlo panel exhibits $\mathcal{O}(1/\sqrt{N})$ sampling noise that contaminates downstream OLS regressions in the Krusell–Smith algorithm. *The bars in this figure are a TikZ schematic illustrating the two regimes; for the actual histograms produced by the algorithm see notebook `lecture_09_10_Youngs_Method_Examples` in the `Lecture-09 code/` folder.*

The absence of sampling noise matters for the Krusell–Smith algorithm: Monte Carlo noise in the realized mean contaminates the OLS regression that updates the forecasting rule, potentially destabilizing convergence.

Grid design. Two separate grids are used in practice. The *value function grid* (typically ~ 150 irregularly spaced points) is finer near the borrowing constraint where the policy function has

high curvature and coarser for large k where behavior is smooth. The *simulation grid* for Young’s histogram (typically $\sim 1,000$ – $5,000$ uniformly spaced points) uses uniform spacing to produce smooth histograms without artifacts. The upper bound $k_{\max}$ must be chosen large enough that no mass reaches the boundary; if $k' > k_{\max}$ for any agent, all mass is assigned to the last grid point, which violates mean preservation. A practical safeguard is to run a preliminary simulation and verify that the boundary bins contain negligible mass.

The full Krusell–Smith algorithm. Combining value function iteration (VFI) with Young’s simulation yields:

1. Initialize forecasting coefficients $A(a), B(a)$.
2. Solve the household problem via VFI given the forecasting rule $\hat{H}$.
3. Forward-simulate the distribution for T periods via Young’s method, recording realized means m_1^t .
4. Re-estimate $A(a), B(a)$ by OLS on simulated (m_1^t, m_1^{t+1}, a_t) .
5. Check convergence; if not converged, return to step 2.

Young’s non-stochastic simulation makes step 3 essentially noise-free and fast relative to a large Monte Carlo panel. The full outer-loop iteration count and wall-clock time, however, remain implementation- and tolerance-dependent because the VFI solve and forecasting-rule fixed point are still present; the speedup applies to the simulation step, not as a generic wall-clock guarantee for the traditional KS workflow.

Convergence and accuracy. A remarkable empirical finding is that the log-linear forecasting rule (6.1) achieves $R^2 > 0.9999$ in the standard Krusell–Smith economy: the first moment of the wealth distribution is a nearly sufficient statistic for predicting next-period prices. Adding higher moments (variance, skewness) to the forecasting rule barely improves the fit. This “approximate aggregation” result does not hold universally; it relies on the specific features of the Krusell–Smith calibration (small aggregate shocks, moderate borrowing constraint), but it is a useful benchmark against which richer models can be compared.

6.4 DEQN with a Continuum of Agents

The traditional Krusell–Smith algorithm provides the benchmark logic for this chapter, but the classroom DEQN notebook used in the course is the simpler Bewley endowment economy of Appendix A.5 in Azinovic et al. (2022). This distinction is pedagogically useful. Krusell–Smith explains *why* distribution tracking matters and *why* Young’s method is valuable inside an outer forecasting-rule loop. Appendix A.5 then shows *how* the same histogram machinery enters a neural-equilibrium implementation once one replaces the forecasting rule by a price network. By combining Young’s histogram method with neural network policies, the DEQN approach overcomes both main limitations of the traditional KS workflow: the network can condition on the *full histogram*, and there is no need for a separate forecasting rule because the price network directly takes the distribution as input.

What §6.4 inherits from Appendix A.5. Five features of the Appendix-A.5 teaching model anchor the rest of this section and the companion notebook `11_Continuum_of_Agents_DEQN.ipynb`; they are the distinguishing departures from the canonical Krusell–Smith calibration of §§6.2–6.3:

- **Endowment economy, not production.** Aggregate output is exogenous, $Y_t = w(a_t)$, instead of $Y_t = z_t K_t^\alpha L_t^{1-\alpha}$; there is no capital and no firm problem.

- **Bonds in unit net supply.** Households trade a single one-period bond at endogenous price p_t , and the market-clearing condition is $\int b_{t+1} d\mu_t = \bar{B} = 1$.
- **Epstein–Zin recursive utility.** Risk aversion ($\sigma = 8$) and inverse IES ($\rho = 2$) are separated, with discount factor $\beta = 0.95$. The KS benchmark in §6.2 instead used log utility with $\beta = 0.99$.
- **Six-state aggregate shock.** $a_t \in \{0, \dots, 5\}$ encodes a 2-state uncertainty regime crossed with a 3-state income level, replacing the 2-state TFP shock of canonical KS.
- **Two idiosyncratic productivity types.** Labour endowment $\eta_t \in \{0.8, 1.2\}$ on a transition matrix Π_η , in place of the employed/unemployed two-state Markov chain of canonical KS.

Histogram encoding. The aggregate state is encoded as a vector containing three aggregate-shock index entries plus N_b histogram values for each idiosyncratic shock type. In the Appendix A.5 notebook these three entries are the combined six-state aggregate index, the income-level index, and the uncertainty-regime index:

$$x_t^{agg} = (\underbrace{z_t^{\text{idx}}, \text{inc}_t^{\text{idx}}, \text{unc}_t^{\text{idx}}}_{3 \text{ shock-index entries}}, \underbrace{h_t^{\eta=0.8}(b_1), \dots, h_t^{\eta=0.8}(b_{N_b})}_{N_b \text{ values}}, \underbrace{h_t^{\eta=1.2}(b_1), \dots, h_t^{\eta=1.2}(b_{N_b})}_{N_b \text{ values}}). \quad (6.4)$$

For $N_b = 100$ grid points and 2 idiosyncratic states, the aggregate state has dimension $3 + 200 = 203$. The full input to the policy network adds the individual state (b_t, η_t) , giving total dimension 205. This is the **production** setting; the checked-in **smoke** and **teaching** runs use $N_b = 50$, so the aggregate state has dimension 103 and the policy input 105. Figure 6.6 shows how the histogram vector and individual state are assembled and fed into the two networks.

How the notebooks fit together. The companion notebook sequence mirrors this decomposition. `10_Youngs_Method_Examples.ipynb` isolates Young’s redistribution operator on toy examples, first in one dimension and then with idiosyncratic shocks. `11_Continuum_of_Agents_DEQN.ipynb` then reuses the same logic inside the full aggregate state vector (6.4). In other words, the second notebook is not a new distributional device; it is the first notebook’s histogram update embedded in a larger equilibrium-learning loop.

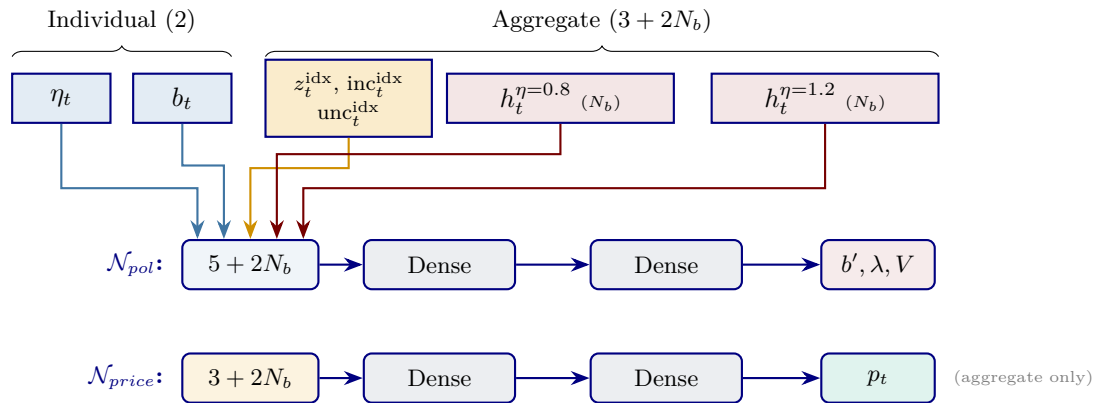


Figure 6.6: Histogram encoding and neural network architecture. The individual state (η_t, b_t) (blue boxes / blue arrows) and the aggregate state $(z_t^{\text{idx}}, \text{inc}_t^{\text{idx}}, \text{unc}_t^{\text{idx}}, h_t^{\eta=0.8}, h_t^{\eta=1.2})$ (orange box for the three shock-index entries, red boxes for the two histograms) are concatenated as input to the policy network $\mathcal{N}_{pol}$; each input arrow is colored to match its source box and enters the policy-input layer at a distinct horizontal offset, so the five arrows are uniquely identifiable at a glance. The price network $\mathcal{N}_{price}$ receives only the aggregate state. Both networks use softplus output activations.

Neural network architecture. Two networks are trained jointly:

- **Policy network** $\mathcal{N}_{pol}$: takes individual + aggregate state ($\mathbb{R}^{5+2N_b}$) $\rightarrow$ outputs savings b' , borrowing multiplier λ , and value V (3 outputs with softplus activation).
- **Price network** $\mathcal{N}_{price}$: takes only aggregate state ($\mathbb{R}^{3+2N_b}$) $\rightarrow$ outputs the bond price p (1 output with softplus).

Both networks use two hidden layers with 500 units (`production`) or 128 (the checked-in `smoke/teaching` runs).

Equilibrium conditions as loss. The loss function comprises five terms, all of which should be zero in equilibrium:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left[\text{EE}_i^2 + \text{BE}_i^2 + (n_Z \text{MC}_i)^2 + \text{KKT}_i^2 + \text{CB}_i^2 \right], \quad (6.5)$$

where EE is the Euler equation residual, BE is the Bellman equation consistency condition, MC is the bond market clearing condition, KKT is the borrowing complementarity, and CB penalizes negative consumption. The market-clearing residual is rescaled by n_Z (the number of aggregate-shock states) before squaring, which puts the single market-clearing residual on the same scale as the per-state Euler, Bellman, and complementarity residuals (themselves evaluated in relative terms, divided through by consumption).

Market clearing via histogram. A key advantage of the histogram representation is that market clearing can be computed *exactly* (no Monte Carlo). The equilibrium condition is $\sum_{\eta,j} h_t(\eta, b_j) b'(\eta, b_j, x_t^{agg}) = \bar{B}$; the residual that enters the loss is its deviation from zero,

$$\text{MC} = \sum_{\eta} \sum_{j=1}^{N_b} h_t(\eta, b_j) \cdot b'(\eta, b_j, x_t^{agg}) - \bar{B} = 0 \text{ at equilibrium}, \quad (6.6)$$

where $h_t(\eta, b_j)$ is the histogram mass, $b'(\cdot)$ is the policy network output, and $\bar{B}$ is aggregate net bond supply. We write the *signed* residual here because it enters the loss (6.5) squared, which is also how the notebook computes it. The Appendix A.5 notebook normalizes $\bar{B} = 1$, which is the source of the “−1” term sometimes seen in code and slides.

Young’s method inside the training loop. During each training episode, the histogram is propagated forward using Young’s method (Algorithm 1) with the current neural network providing the policy function. This creates a sequence of aggregate states $(x_0^{agg}, x_1^{agg}, \dots, x_T^{agg})$ on which the equilibrium residuals (6.5) are evaluated. Young’s redistribution operator is differentiable almost everywhere (linear interpolation conditional on fixed brackets), so the *one-step* histogram update that enters the market-clearing residual at each sampled state carries gradients back to the policy network. The longer simulated path of aggregate states is treated as data: it is regenerated each episode from the current network and held fixed inside the gradient tape, rather than backpropagated through end to end. Even so the distribution co-evolves with the network: early in training, when the policy network is inaccurate, the simulated path will be “wrong,” but the market-clearing residuals evaluated along it provide corrective feedback through the loss, and as the network improves the distribution converges toward the ergodic steady state.

6.5 Results and Discussion

In the Appendix A.5 teaching model, the DEQN with histogram encoding achieves competitive accuracy: Euler equation errors are of order 10^{-3} , and market-clearing residuals are comparably

small. These figures come from the checked-in `teaching/smoke` configuration of the companion notebook (a small network trained for a modest number of episodes); the `production` configuration tightens them further. The broader lesson for the Krusell–Smith benchmark is conceptual rather than model-specific. Compared to the traditional KS algorithm, the DEQN approach has two advantages: (i) the neural network can condition on the *full* distribution rather than just its mean, providing a richer approximation that can capture situations where higher moments of the distribution matter for prices, and (ii) there is no need for a separate outer loop to update forecasting coefficients, since equilibrium conditions are enforced directly through the loss function.

Connection to continuous time

The discrete-time histogram approach presented here has a natural continuous-time counterpart. In Chapter 8, we will see that the Kolmogorov forward equation (KFE; Fokker–Planck) describes the evolution of the wealth *density* in continuous-time heterogeneous-agent models such as Achdou et al. (2022). While the mathematical formulation differs (histogram update vs. PDE), the economic question is the same: how does the wealth distribution evolve, and how does it feed back into equilibrium prices?

6.6 Alternative Deep-Learning Approaches to Krusell–Smith

Before turning to the deep-learning alternatives, it is useful to set the histogram-DEQN method in the broader landscape of solution techniques for heterogeneous-agent equilibria with aggregate shocks. Table 6.1 compares classical and modern approaches along four dimensions that drive method choice in practice.

Whereas Table 6.1 is panoramic (classical and DL methods on common axes), Table 6.2 drills into the DL trio along the axes that matter when choosing among them. Histogram-DEQN is not the only deep-learning approach to heterogeneous-agent equilibria, and it is pedagogically useful to see how the space of deep-learning strategies decomposes. Three broad families have emerged in the literature. Two informative axes organize them: how the cross-sectional distribution is represented as input to the network, and what objective is optimized. Histogram-DEQN and All-in-One DL minimize residuals of the structural equilibrium equations; DeepHAM instead maximizes cumulative utility along simulated paths and uses Bellman residuals as a validation diagnostic.

6.6.1 All-in-One Deep Learning (Maliar, Maliar & Winant, 2021)

Maliar et al. (2021) propose what they call *all-in-one deep learning*. The key observation is that every dynamic economic model can be cast, at its core, as a collection of *expectation conditions* (optimality, feasibility, market clearing) that vanish at the true solution. They rewrite these conditions as non-linear regression equations with zero dependent variable, parameterize the policy and value functions by deep neural networks, and minimize the expected-squared residual by stochastic gradient descent on simulated paths.

For the Krusell–Smith benchmark with aggregate shocks and a continuum of agents, Maliar et al. (2021) formulate the following components of the loss. An alternative deep-learning route to the same problem is the symmetry-exploiting parameterization of Kahou et al. (2021), which uses a permutation-invariant aggregation of agent-level features; the modern perturbation route is the refined Reiter implementation of Bayer and Luetticke (2020). Both are useful complements to the Young/DEQN combination developed below, with different scaling profiles and code-complexity tradeoffs.

Method	Distribution rep.	Aggregate state	Solution principle	Best when
Classical (Krusell and Smith, 1998)	KS Panel of $N \sim 10^4$ agents	First moment(s)	Bounded-rationality fixed point of forecasting rule	Standard incomplete-markets, low-dim aggregate state
Reiter (back-loaded) (Reiter, 2009)	Histogram on a fixed grid	First-order perturbation around stationary	Linearize after solving the steady state	Small aggregate shocks, smooth policies
Continuous-time Achdou et al. (Achdou et al., 2022)	Density solving a KFE PDE	In the limit, the entire density	Coupled HJB+KFE finite differences	PDE-friendly model, smooth densities
Histogram-DEQN (Azinovic et al., 2022)	Young histogram on grid	Histogram entries (or moments thereof)	SGD on equilibrium residuals	Strong nonlinearities, occasionally binding constraints
All-in-one (Maliar et al., 2021)	DL Panel of $N \gtrsim 10^3$ agents	Agent-level states, policy and aggregate together	SGD on stacked Euler+market-clearing residuals	Many states per agent, GPU available
DeepHAM (Han et al., 2024)	Permutation-invariant set encoder (DeepSets)	Learned $M \ll N$ generalized moments	Cumulative utility along simulated paths (policy-gradient with structural individual dynamics)	Want a low-dim aggregate state without committing to a moment <i>a priori</i>

Table 6.1: Heterogeneous-agent solution methods at a glance. The first three rows are classical or finite-difference; the last three are the modern deep-learning families compared in detail in Table 6.2 and the rest of this section.

- **Euler residual.** The household’s consumption-saving first-order condition:

$$\text{EE}(s) = u'(c_t(s)) - \beta \mathbb{E}_t[u'(c_{t+1}(s')) R_{t+1}], \quad (6.7)$$

where $s = (k_t, \varepsilon_t; \mathbf{S}_t)$ is the individual-plus-aggregate state and $\mathbf{S}_t$ is an N -agent panel of capital holdings. The policy network outputs $c_t = \pi_\rho(s)$; the Euler residual is evaluated at simulated states and the expectation by Monte Carlo over next-period idiosyncratic and aggregate shocks. With the borrowing constraint $k' \geq 0$, the plain Euler residual is insufficient: at a binding constraint the equation can be slack, so the loss must combine EE with a complementarity condition via Fischer–Burmeister or KKT (see below).

- **Bellman residual / value-function error** for off-policy learning of a value function, used in the “lifetime reward” formulation.
- **Market-clearing residual:** $\sum_i k_{t+1}^i = \bar{K}_{t+1}$ summed across the N agents in the panel.
- **Borrowing constraint** non-negativity enforced architecturally (softplus on savings); the complementarity optimality condition still enters the loss via a Fischer–Burmeister term regardless.

A central computational contribution is the *all-in-one integration operator*: a single Monte Carlo realization of the next-period state is used to estimate the combined expectation across all residual terms simultaneously, rather than evaluating separate quadrature nodes for each conditional expectation. This reduces the cost of computing the conditional expectations from $\mathcal{O}(Q^m)$ (with Q nodes per shock and m shocks) to $\mathcal{O}(1)$ per state: a single draw of the next-period shocks plays the role of the full quadrature grid in the training loss.

Scale reported by the authors. Maliar et al. (2021) demonstrate the approach across several increasingly demanding setups: a Krusell–Smith variant with $N \geq 1,000$ explicit agents and 2,001 state variables (two per agent plus one aggregate) in the baseline parameterization, scaled up to $N = 10,000$ agents in a moments-reduced variant where the cross section is summarized by 10–20 aggregate moments; and on a one-agent consumption-savings problem with kinked policies they report approximation errors of at most a fraction of one percentage point. A Python/TensorFlow replication is available at <https://github.com/marcmaliar/deep-learning-euler-method-krusell-smith> (Maliar, 2022); it is the basis for the companion notebook `lectures/lecture_09_heterogeneous_agents_youngs_method/code/lecture_09_12_KrusellSmith_DeepLearning.ipynb` (described below).

Why this approach? All-in-one DL is the closest cousin of the DEQN framework of Chapter 2. The only conceptual difference is that the aggregate state is represented by an *explicit panel* of N agents (a large vector $\mathbf{S}_t \in \mathbb{R}^N$) rather than by a histogram on a fixed grid. Stochastic mini-batches draw both individual state and aggregate state, and the law of large numbers delivers permutation-invariance in the limit $N \rightarrow \infty$ without requiring a permutation-invariant architecture. The appeal is conceptual simplicity; the cost is that the input dimension grows with N and the learner must rediscover permutation symmetry from the data.

6.6.2 DeepHAM: Generalized Moments via DeepSets (Han, Yang & E, 2024)

Han et al. (2024) ask a more ambitious question: can the network *learn* which summary statistics of the cross-sectional distribution are relevant for individual decisions, rather than having the researcher commit to tracking a histogram or the first moment? They propose replacing the distribution μ_t with a small set of *generalized moments* obtained by averaging a flexible neural feature over the cross-section,

$$m_t^\ell = \frac{1}{N} \sum_{i=1}^N \phi_\theta^\ell(s_t^i) \xrightarrow{N \rightarrow \infty} \int \phi_\theta^\ell(s) d\mu_t(s), \quad \ell = 1, \dots, M, \quad (6.8)$$

with $\mathbf{m}_t = (m_t^1, \dots, m_t^M)$ and $\phi_\theta^\ell: \mathbb{R}^d \rightarrow \mathbb{R}$ a neural feature encoder trained jointly with the policy and value networks. Equation (6.8) is the canonical DeepSets architecture of permutation-invariant set functions (Zaheer et al., 2017): averaging (or, in the continuum limit, integration against μ_t) makes $\mathbf{m}_t$ invariant to permutations of the agents, and the ϕ_θ^ℓ encoders are flexible enough (by universal approximation on the permutation-invariant functions) to represent any fixed-arity moment.

The individual’s value and policy functions then take the form

$$V_t = V_\rho(s_t^i; \mathbf{m}_t, a_t), \quad k_{t+1}^i = \pi_\rho(s_t^i; \mathbf{m}_t, a_t), \quad (6.9)$$

and all networks $(V_\rho, \pi_\rho, \phi_\theta)$ are trained jointly. The primitive training objective is to *maximize* cumulative utility along simulated paths,

$$J(\theta, \rho) = \mathbb{E} \left[\sum_{t=0}^{T-1} \beta^t u(c_\rho(s_t^i; \mathbf{m}_t, a_t)) + \beta^T V_\psi(s_T^i; \mathbf{m}_T, a_T) \right], \quad (6.10)$$

where the expectation is taken over simulated idiosyncratic and aggregate histories generated under the current policy. Squared Bellman residuals are reported as a validation diagnostic, not as the optimization target. Because the individual law of motion, the budget constraint, and the transition structure are known economic objects, they are written directly into the computational graph: gradients of J flow through these structural dynamics, in contrast to model-free reinforcement learning where transitions are observed only as samples (Yang et al., 2025). Because the generalized moments are themselves parameters of the optimization (not

hyperparameters), the method automatically discovers the minimal set of distributional statistics required for equilibrium pricing.

A practical consequence of the policy-gradient formulation is that DeepHAM is well suited to problems where first-order conditions are difficult to write down or inconvenient to use, including constrained-efficiency problems with aggregate shocks, optimal-policy design, and behavioral macro questions; the headline application in Han et al. (2024) is precisely a constrained-efficiency problem solved by simulating the economy under candidate allocation rules and updating the rules to improve social welfare.

Reported accuracy (validation diagnostics). The numbers that follow are taken from Han et al. (2024); they have not been independently replicated by the companion notebooks of this chapter, and should be read as reported results rather than as figures we can vouch for from our own runs. On the baseline Krusell–Smith model, the authors report the following Bellman-error reductions, computed *ex post* as validation measures of the converged solution rather than as the training objective:

- DeepHAM with *only* the first moment in the state vector reduces the Bellman residual by **61.6%** compared to the classical KS algorithm.
- DeepHAM with *one* learned generalized moment (on top of, or replacing, the first moment) reduces the residual by **68.9%**.
- The method extends to HA models with multiple shocks, multiple endogenous states, large shocks (risky steady state), and nonlinearities (e.g., ZLB) where both KS and the local perturbation method of Reiter (2009) break down.

Conceptually, DeepHAM bridges two traditions: the approximate-aggregation insight of Krusell and Smith (1998) (a small number of moments can suffice) and the deep-function-approximator philosophy of Maliar et al. (2021) and Azinovic et al. (2022) (the NN need not be restricted to pre-specified basis functions). The official reference implementation, including replication code for the Krusell–Smith benchmarks above, is available at <https://github.com/frankhan91/DeepHAM>.

Interpretability of the learned moments. Although the encoders ϕ_θ^ℓ are flexible neural networks, the moments they produce are interpretable in the economic sense relevant to heterogeneous-agent macro: they summarize how the cross-section affects welfare and policy. In the Krusell–Smith application of Han et al. (2024), the learned basis function is concave in assets, so a marginal asset held by a poor household contributes more to the moment than a marginal asset held by a rich household. This links the learned distributional representation directly to marginal propensities to save, redistribution, and welfare effects, which is the natural object of interest in HA models. Generalized moments are therefore not just a flexible compression of μ_t ; they double as a device for reading economic content out of the trained equilibrium.

From learned moments to learned state spaces. Once equilibrium computation is written as policy-gradient optimization over simulated paths, one can ask a sharper question: do agents need the full distribution as a state variable at all? In Walrasian heterogeneous-agent models, agents care about μ_t only indirectly, through equilibrium prices. Yang et al. (2025) exploit this in their *structural reinforcement learning* (SRL) framework: agents’ policy functions take low-dimensional prices, or short price histories, as the aggregate state, while μ_t remains part of the simulated environment used to clear markets. This sidesteps the master equation entirely for a substantial class of HA economies and produces a natural conceptual arc: KS chooses moments *a priori* (Section 6.3); DeepHAM *learns* moments from the equilibrium objective (this section); SRL replaces moments with prices when the model permits, and lets ML help *define* a tractable equilibrium concept rather than only solving a fixed one.

6.6.3 Beyond Walrasian Markets: DeepSAM and Search Frictions

A natural follow-up question is whether the DeepHAM machinery extends to economies in which the cross-sectional distribution affects decisions through channels other than aggregate prices. In standard heterogeneous-agent models the distribution is felt only through equilibrium prices: μ_t enters individual problems by setting $r_t = r(K_t)$ and $w_t = w(K_t)$. In labor markets, search-and-matching, and other non-Walrasian settings, the distribution enters more directly: through the matching technology, the type composition on each side of the market, outside options, and bargained transfers. This makes the equilibrium mapping intrinsically non-Walrasian and forecloses simple price-only summaries of the cross-section.

Payne et al. (2025) address this case with *DeepSAM*, a deep-learning solver for search-and-matching models with two-sided heterogeneity and aggregate shocks. The architecture inherits the DeepHAM idea of a permutation-invariant set encoder, applied to each side of the market separately, and feeds the resulting type-composition summaries into networks that approximate value functions, matching surplus, and policy. The training objective is again policy-improvement on simulated paths, with the matching technology and bargaining rule embedded structurally. In Walrasian HAM the cross-sectional asset/wealth/income distribution enters individual problems only *indirectly*, through equilibrium prices r_t, w_t ; in SAM the type composition on each side of the matching market enters *directly*, through the matching technology, outside options, and bargained transfers. That is why DeepHAM and DeepSAM, despite sharing a permutation-invariant set-encoder architecture, treat the distribution differently.

6.6.4 Which Method, When?

The three deep-learning approaches (Histogram DEQN, §6.4; All-in-One DL, §6.6.1; and DeepHAM, §6.6.2) are complements rather than substitutes. Table 6.2 summarizes the practical trade-offs:

- For *teaching* purposes, the Histogram DEQN is the cleanest: the network input is an interpretable distribution vector, and the training loop directly mirrors the DEQN template introduced in Chapter 2.
- For *research* problems where the number of agents is the natural state dimension (e.g., overlapping generations with many cohorts, or finite-agent social-planner problems), All-in-One DL is often more convenient because it requires no grid design.
- For *policy analysis in richer HA environments* (risky steady states, multiple endogenous states, ZLB), DeepHAM’s learned-moment representation pays off both in accuracy and in interpretability, because the learned moments can be plotted and analyzed as functions of the distribution.

Table 6.3 distills the same trade-offs into a quick decision aid: when the model fits the row’s “When it shines” column, the matching method is the first one to try.

Notebook: a classroom-scale all-in-one KS solver. The accompanying Jupyter notebook `lecture_09_12_KrusellSmith_DeepLearning.ipynb` (introduced in §6.5 and described in detail in the README) implements a classroom-scale version of the all-in-one DL approach of Maliar et al. (2021) on the Krusell–Smith benchmark with the parameters of Krusell and Smith (1998). It uses a single policy network $\pi_\rho(k, \varepsilon, K, a)$ parameterized by TensorFlow/Keras and an explicit panel of N agents as the distributional input; the loss is the squared Euler residual, augmented with a Fischer–Burmeister complementarity term at the borrowing constraint, and aggregate consistency is imposed by construction (next-period capital is the cross-sectional mean of the panel’s savings choices) rather than through a separate market-clearing penalty. This is a deliberate simplification of the full all-in-one formulation of §6.6.1, which also carries a value

network and an explicit market-clearing residual. The notebook is annotated cell-by-cell with the correspondence to the equations in this chapter, and is designed to converge in under ten minutes on a standard CPU. For the production-scale counterpart (up to $N = 10^4$ agents, GPU acceleration, richer shock structure), we refer the reader to the replication repository of [Maliar \(2022\)](#).

Benchmarks and replication pointers. For readers who want to benchmark any of the deep-learning approaches against the traditional KS algorithm, the canonical reference implementation is `econ-ark/KrusellSmith` ([The Econ-ARK Team, 2020](#)), which implements the forecasting-rule-update algorithm of [Krusell and Smith \(1998\)](#) and reports $R^2 > 0.9999$ within about 20 outer iterations under standard parameters ($\beta = 0.99$, $\alpha = 0.36$, log utility, two aggregate states). Any deep-learning method must at least match that accuracy on the baseline model; the advantages are supposed to appear when the model is extended in directions KS cannot handle cleanly (more moments, many endogenous states, risky steady state).

6.7 Extension: Deep Learning in the Sequence Space

The histogram-based DEQN above is transparent because it feeds a direct approximation of the endogenous cross-sectional distribution into the neural network. The price of that transparency is dimensionality: in richer heterogeneous-agent economies, the aggregate state can contain hundreds of histogram entries. [Azinovic-Yang and Žemlička \(2025a\)](#) propose a different representation of the aggregate state. Instead of feeding the *current endogenous state* to the network, they feed a *truncated history of exogenous aggregate shocks*. The equilibrium logic does not change: one still enforces Euler equations, market clearing, and occasionally binding constraints inside the loss. What changes is the object that summarizes the aggregate state for the network. Figure 6.7 contrasts the two views.

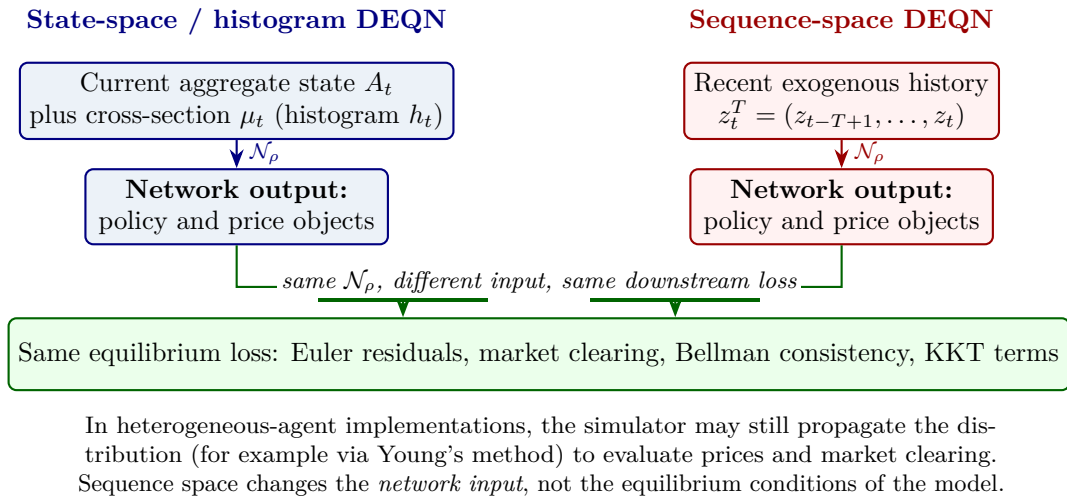


Figure 6.7: Two ways to encode the aggregate state in deep equilibrium learning. Each pipeline reads top-to-bottom: the upper (colored) box is the *input* the user gives to the same neural network $\mathcal{N}_\rho$, the middle (colored) box is the network's *output* (policy and price objects), and the green box is the equilibrium loss that consumes those outputs. Histogram DEQNs (left, blue) feed an endogenous state representation (A_t, μ_t) ; sequence-space DEQNs (right, red) feed a truncated exogenous shock history z_t^T . Crucially, the network and the residual-based training loss are identical across the two pipelines, only the input changes.

The sequence-space representation. Let $z_t^T := (z_{t-T+1}, \dots, z_t) \in \mathbb{R}^T$ denote the last T

realizations of the exogenous aggregate shock. The key claim is that, in an ergodic economy, this history is an *approximate sufficient statistic* for the endogenous aggregate state. In the Brock–Mirman warm-up notebook, the network maps the shock history to a bounded savings rate, from which next-period capital follows by the resource constraint,

$$s_t = \sigma(\mathcal{N}_\rho(z_t^T)) \in (0, 1), \quad K_{t+1} = s_t z_t K_t^\alpha,$$

where σ is the logistic squashing that keeps K_{t+1} feasible. In the richer heterogeneous-agent version, the network instead maps the same history to higher-level equilibrium objects such as policy-function coefficients or pricing objects. This connects the method to the MIT-shock and sequence-space Jacobian literature of Boppart et al. (2018) and Auclert et al. (2021), but replaces local linear approximations with a global residual-based neural approximation.

State-space vs sequence-space: the equilibrium operator

The two formulations can be written symmetrically. Let y_t denote the equilibrium objects of interest (policies, prices) at date t , x_t the endogenous aggregate state, and ε_t the exogenous shock.

State-space recursion. The decision rule is a function f of the current state, the state evolves through a known transition H , and equilibrium is the functional equation

$$y_t = f(x_t), \quad x_{t+1} = H(x_t, y_t, \varepsilon_{t+1}), \quad G(f, x) = 0 \quad \forall x.$$

Sequence-space formulation. Let $\mathcal{E}_t = (\varepsilon_t, \varepsilon_{t-1}, \dots)$ denote the full shock history. The decision rule is now a function Ψ of the history (with initial condition x_0), and the state x_t is recovered by iterating the same law of motion under Ψ :

$$y_t = \Psi(\mathcal{E}_t \mid x_0), \quad x_t = \mathcal{H}(\mathcal{E}_t, x_0 \mid \Psi), \quad G(\Psi, \mathcal{E}, x_0) = 0 \quad \forall \mathcal{E}, \forall x_0.$$

Both formulations describe the *same* equilibrium. What differs is the *domain of approximation*: f lives on the (potentially infinite-dimensional) endogenous state space, while Ψ lives on the (also infinite-dimensional but exogenously driven) space of shock histories. In an ergodic economy the partial derivative $\partial \Psi / \partial \varepsilon_{t-\tau}$ vanishes as $\tau \rightarrow \infty$, so Ψ admits a finite-history truncation $\hat{\Psi}(z_t^T)$ with controllable error. This truncation step, developed in the next paragraph, is what makes the sequence-space formulation computable.

Intuition first. The easiest way to think about the method is as a *memory compression* device. A positive aggregate shock today raises output and therefore raises tomorrow's capital. That extra capital still matters the period after, but only through the production elasticity α , so its influence is smaller. One more period later it is smaller again. In other words, the current aggregate state stores a decaying memory of past shocks. The sequence-space idea is to feed that shock history directly to the network rather than feeding the current endogenous state itself.

Brock–Mirman: what changes relative to Chapter 2? The Brock–Mirman warm-up is useful because the change can be written down exactly. In Chapter 2, the state-space DEQN uses the current state as input,

$$x_t^{\text{state}} = (K_t, z_t), \quad C_t = \mathcal{N}_\rho(K_t, z_t), \quad K_{t+1} = z_t K_t^\alpha - C_t.$$

In the sequence-space version, the *economic model* is unchanged, but the network sees a different input:

$$x_t^{\text{seq}} = z_t^T = (z_{t-T+1}, \dots, z_t), \quad s_t = \sigma(\mathcal{N}_\rho(z_t^T)), \quad K_{t+1} = s_t z_t K_t^\alpha, \quad C_t = (1 - s_t) z_t K_t^\alpha.$$

The Euler residual is the same object as before,

$$G_t = 1 - \beta \frac{C_t}{C_{t+1}} \alpha z_{t+1} K_{t+1}^{\alpha-1},$$

so the economics are unchanged. What changes is the computational representation:

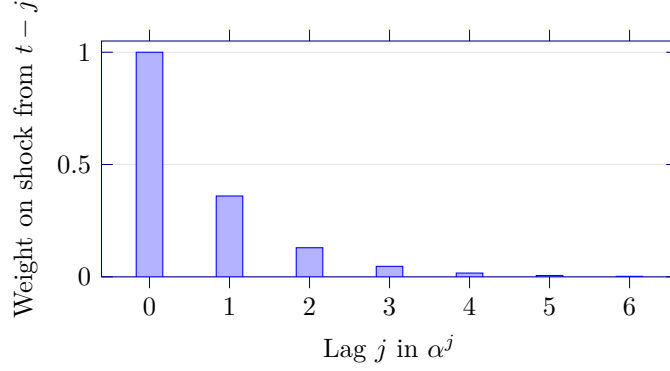


Figure 6.8: Intuition for sequence space in Brock–Mirman. $\log K_t$ depends on past shocks with weights that decay like α^j in the lag j (here $\alpha = 0.36$, the standard capital share). Already at $j = 3$ the weight has fallen to ~ 0.05 , so a finite history of recent shocks summarizes the relevant aggregate information; very old shocks matter little.

- the **network input** changes from the current state (K_t, z_t) to the recent history z_t^T ;
- the **network output** changes from current consumption C_t to a bounded savings rate $s_t \in (0, 1)$ in the warm-up notebook, so that $K_{t+1} = s_t z_t K_t^\alpha$ is feasible by construction;
- the **current capital stock** is no longer fed directly into the network, but is generated recursively from the initial condition and previously predicted capital choices;
- the **training samples** are overlapping shock histories rather than pointwise states (K_t, z_t) .

This distinction is important conceptually. For Brock–Mirman, sequence space is *not* a dimensionality reduction, since (K_t, z_t) is only two-dimensional whereas a history of length $T = 25$ is larger. The Brock–Mirman notebook is therefore a pedagogical demonstration of the idea that histories can stand in for endogenous states. The dimensionality gain appears only in richer heterogeneous-agent models, where the relevant alternative is a large histogram or other high-dimensional distributional summary.

Intermediate bridge: sequence-space IRBC. Between the one-shock Brock–Mirman warm-up and the infinite-dimensional Krusell–Smith state, the companion notebook `lectures/lecture_10_sequence_space_deqns/code/lecture_10_05b_SequenceSpace_IRBC.ipynb` re-trains the two-country IRBC model of Chapter 3 under sequence-space inputs: the policy network reads the last $T = 80$ shock vectors (a 240-dimensional history with $\rho_z^T \approx 1.7 \times 10^{-2}$ truncation error) instead of the four-dimensional current state. The $2N + 1$ equilibrium residuals (Euler, ARC, Fischer–Burmeister), the Gauss–Hermite quadrature, and the cloud-method sampler are literally unchanged from nb 01; only the input domain changes. Because the current capital stock is no longer an input, we parametrize the output head around the steady state, $k'_j = k_{ss} \exp(\tanh z_j^k)$ and $\lambda = \lambda_{ss} \exp(\tanh z^\lambda)$, which keeps gradients lively at the target policy and prevents the cold-start divergence that plagued a naive softplus head. This notebook is a *pedagogical bridge* rather than a computational win, at a four-dimensional state the history is much larger, not smaller, but it shows that the same template handles a multi-equation system with multiple independent shock channels before we hand the method over to Krusell–Smith, where the dimensionality gain is real.

Training logic. The computational pattern is also close to the rest of this chapter. One samples an exogenous shock path, constructs overlapping history windows z_t^T , evaluates the network on those windows, and then uses the resulting decisions to simulate the endogenous economy forward. In the Brock–Mirman warm-up this produces the capital sequence directly;

in the Krusell–Smith tutorial it produces policy-function objects, while Young’s method still propagates the cross-sectional distribution inside the simulator. Residuals are then evaluated on the simulated path and backpropagated through the full pipeline. Figure 6.9 summarizes this workflow.

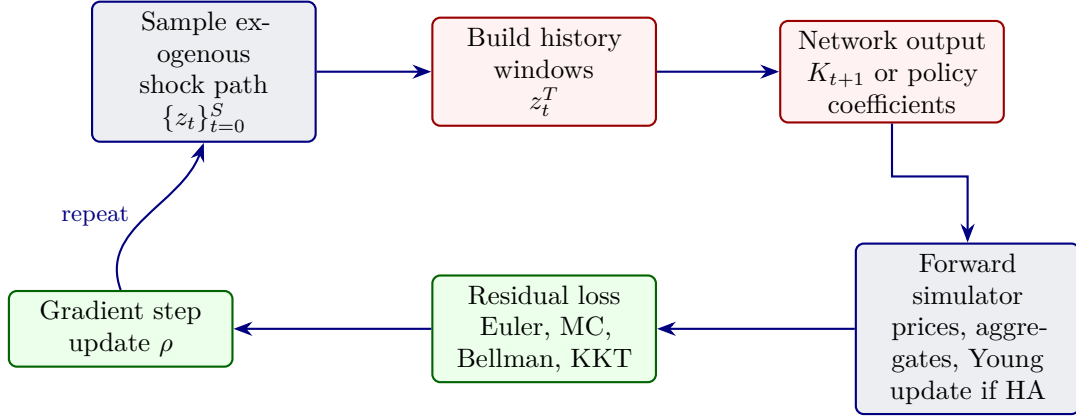


Figure 6.9: Training flow for sequence-space DEQNs. The exogenous shock history is the network input, but the forward simulator still produces endogenous objects such as prices, aggregate capital, or cross-sectional distributions needed for residual evaluation.

Worked example: what the network input looks like

In the companion notebook `KrusellSmith_Tutorial_CPU.ipynb`, the helper function `encode_Z_history` represents a discrete shock history as a one-hot block concatenated with the corresponding realized levels. Suppose $N_Z = 2$ (so $Z_t \in \{Z_L, Z_H\}$ with $Z_L = 0.93$, $Z_H = 1.07$) and the truncated history of length $H = 3$ is (Z_L, Z_H, Z_L) . `encode_Z_history` then returns

$$\underbrace{\begin{bmatrix} \underbrace{1, 0}_{Z_L}, & \underbrace{0, 1}_{Z_H}, & \underbrace{1, 0}_{Z_L} \end{bmatrix}}_{\text{one-hot block, length } H \cdot N_Z} \parallel \underbrace{\begin{bmatrix} 0.93, & 1.07, & 0.93 \end{bmatrix}}_{\text{level block, length } H},$$

a single vector of length $H \cdot (N_Z + 1) = 9$ that is fed to the MLP. In the Krusell–Smith tutorial, $H = 50$ and $N_Z = 2$, giving an input of length 150. The corresponding histogram-based input, by contrast, would have hundreds of bins from the wealth distribution alone.

Why truncated histories can work. The Brock–Mirman warm-up makes the logic especially transparent. With full depreciation ($\delta = 1$) and log utility, recursive substitution shows that the capital stock depends on the last T shocks up to an error of order $\alpha^T \log(K_{t-T})$. Since $\alpha < 1$ (typically $\alpha \approx 0.36$), this error vanishes exponentially: for $T = 25$, the truncation error is of order 10^{-11} . More generally, in ergodic economies with persistent aggregate shocks, the approximation error decays at roughly $\max\{|\varrho|, |\alpha|\}^T$, where ϱ denotes the persistence of the aggregate productivity shock (as in Ch. 2). In richer heterogeneous-agent models this is no longer an exact algebraic statement, so the history length T becomes an empirical accuracy choice rather than a theorem.

Why this is useful in heterogeneous-agent models. Two advantages are worth separating. First, *as a network input*, a history of $T \approx 25$ shocks can be much smaller than a histogram with hundreds of bins. Second, exogenous shock histories are sampled from a fixed distribution. This removes one source of instability in residual-based training: the set of network inputs is anchored by model primitives even though the endogenous simulator still evolves with the current policy

network. In the Krusell–Smith tutorial, this means that the network is conditioned on shock histories, while Young’s method remains responsible for propagating the distribution used in market-clearing calculations.

Why sequence-space training is more stable: the feedback loop

The second advantage above deserves to be unpacked, because it is empirically the single biggest source of stability gains reported in [Azinovic-Yang and Žemlička \(2025a\)](#). In a state-space deep-learning HA solver the network reads the endogenous distribution μ_t as part of its input, and the training set of μ ’s is generated by simulating the economy under the *current* policy network. This creates a self-amplifying loop:

$$\begin{aligned} \rho^{(k)} &\longrightarrow \pi_{\rho^{(k)}} \longrightarrow \{\mu_t\}_{\rho^{(k)}} \longrightarrow \text{input distribution shifts} \\ &\longrightarrow \text{out-of-distribution evaluations} \longrightarrow \text{large residual gradient} \longrightarrow \rho^{(k+1)} \text{ overshoots,} \end{aligned}$$

and the next outer iteration starts from inputs the network has never seen before, often producing even larger shifts. In sequence space the network input is the truncated shock history z_t^T , drawn from the *exogenous* law of motion of the aggregate shock. That distribution is fixed by model primitives and *does not move* with the policy update. The feedback loop is broken at its first link: training inputs are stationary even when the policy is far from optimal. Empirically this often turns a calibration that fails to converge in the state-space formulation (across random seeds and learning rates) into one that converges robustly in sequence space.

Shape-preserving operator learning. A second contribution of [Azinovic-Yang and Žemlička \(2025a\)](#) is to let the network output *policy-function objects* rather than a single scalar choice. In particular, they construct architectures that guarantee monotonicity and concavity of the predicted consumption rule by representing it with an I-spline basis and non-negative coefficients. In the Krusell–Smith tutorial, the network maps the shock history to these coefficients; the resulting policy can then be evaluated at all idiosyncratic states on the wealth grid. This operator-learning view pairs naturally with the endogenous grid method (EGM) of [Carroll \(2006\)](#) and avoids ad hoc penalties for monotonicity or concavity.

Explicit I-spline MPC parameterization. Having seen *why* a shape-preserving output head matters (above), we now write the construction down concretely; this is the most technical paragraph of the section and a reader who already accepts the monotonicity/concavity guarantees can skip to “Fischer–Burmeister KKT loss” below. Let $\{k_n\}_{n=0}^N$ be a fixed log-spaced wealth grid and let $B \in \mathbb{R}^{J \times (N+1)}$ be a precomputed I-spline basis evaluated on it,

$$B_{j,n} = I_j(\log(\eta + k_n)), \quad j = 1, \dots, J, \quad n = 0, \dots, N,$$

where $\eta > 0$ is a small numerical shift (the `BASIS_SHIFT` constant in the notebook) and each I_j is an integrated B-spline that is monotonically increasing from 0 to 1. For each idiosyncratic state ε , the network outputs two objects: a boundary marginal propensity to consume $\alpha(\varepsilon) \in (0, 1)$ (sigmoid head) and non-negative weights $\tilde{w}_j(\varepsilon) \geq 0$ with $\sum_j \tilde{w}_j(\varepsilon) < 1$ (a “phantom-zero” softmax head). The grid MPC is

$$\text{MPC}_{\varepsilon,n} = \alpha(\varepsilon) \left(1 - \sum_{j=1}^J \tilde{w}_j(\varepsilon) B_{j,n} \right), \quad (6.11)$$

which is decreasing in n by construction (positive weights times an increasing basis, subtracted off a constant), bounded in $[0, \alpha(\varepsilon)] \subset [0, 1]$, and continuous in the network parameters. Consumption is then recovered on the grid by cumulation of the MPC schedule along cash-on-hand $m = w\varepsilon + Rk$,

$$c(\varepsilon, k_0) = \text{MPC}_{\varepsilon,0} m(\varepsilon, k_0), \quad c(\varepsilon, k_n) = c(\varepsilon, k_{n-1}) + \text{MPC}_{\varepsilon,n} R(k_n - k_{n-1}), \quad (6.12)$$

and off-grid evaluation uses piecewise-linear interpolation. Equations (6.11)–(6.12) guarantee, by construction and without any auxiliary penalty, that the consumption rule is non-negative,

monotonically increasing in k , concave in k , and feasible ($c \leq m$). In code, B is the matrix `ispline_basis`, α and $\tilde{w}$ come from the two heads of `actor_c_grid`, and the cumulation is the closing block of that same function.

Fischer–Burmeister KKT loss. Households face a borrowing constraint $k_{t+1} \geq 0$. The Karush–Kuhn–Tucker conditions of the household problem split into two regimes: at an interior optimum the Euler equation holds with equality, while at a binding constraint the Euler equation can be slack but next-period capital is zero. Define the (relative) Euler residual and the (relative) savings slack (in this section g is reused as the Euler residual, matching the tutorial code’s variable name; it is *not* the household policy function $g(k, \varepsilon, \dots)$ of §6.3)

$$g = \frac{c_{\text{Euler}} - c}{c}, \quad s = \frac{k'}{c},$$

where $c_{\text{Euler}} = (u')^{-1}(\beta \mathbb{E}_t[R' u'(c')])$ is the consumption level implied by the Euler equation given the network’s continuation policy. The KKT pair is then

$$g = 0, s \geq 0 \quad (\text{interior}) \quad \text{or} \quad g \geq 0, s = 0 \quad (\text{constrained}),$$

which is a complementarity condition. The Fischer–Burmeister envelope, in the same sign convention used in Ch. 3 and Ch. 5,

$$\text{FB}(g, s) = g + s - \sqrt{g^2 + s^2 + \epsilon_{\text{fb}}} \tag{6.13}$$

is smooth and satisfies $\text{FB}(g, s) = 0$ if and only if $\min(g, s) = 0$ with both non-negative; the small constant ϵ_{fb} (set to 10^{-12} in the notebooks) is a numerical stabilizer for the square root. The upstream JAX tutorial code uses the negative-sign variant $\sqrt{g^2 + s^2 + \epsilon} - g - s$, which has the same zero set when squared. The training loss is the buffer-and-grid average of FB^2 ,

$$\mathcal{L}(\rho) = \mathbb{E}_{(z^H, \mu) \sim \mathcal{B}} \left[\frac{1}{N_{\varepsilon}(N+1)} \sum_{\varepsilon, n} \text{FB}(g_{\varepsilon, n}(\rho), s_{\varepsilon, n}(\rho))^2 \right],$$

so that one differentiable scalar simultaneously enforces the Euler equation in the interior region and the complementarity condition at the borrowing constraint, without case splits or shadow-price augmentation. This reuses the smooth complementarity construction of Fischer (1992) that is standard in nonlinear programming, applied here to a heterogeneous-agent equilibrium loss.

Putting the pieces together: the HA training loop. The Krusell–Smith tutorial assembles the encoder, the I-spline policy head, Young’s distribution step, and the Fischer–Burmeister loss into a single replay-buffer training loop. Algorithm 2 states it explicitly.

Algorithm: Sequence-Space DEQN with Young's method (KS tutorial)

Input: network $\mathcal{N}_\rho$ (I-spline MPC heads), I-spline basis B , transition matrices Π_ε, Π_Z , prices $R(K, L, Z), w(K, L, Z)$

Hyperparameters: history length $H=50$, roll-out $T=100$, buffer cap $C=128$, $N_{\text{agg}}=8$, FB steps per epoch $S=10$, mini-batch $B_{\text{fb}}=16$, learning rate $\alpha=5 \times 10^{-5}$, gradient clip $\|\cdot\|_2 \leq 1$

Initialize replay buffer $\mathcal{B}$ with copies of $(\mathbf{0}_H, \mu_0)$, where μ_0 is centered at the deterministic-RA reference K_{ss}

for epoch $e = 1, 2, \dots$ **do**

 Draw N_{agg} replay states $\{(z_i^H, \mu_i)\}$ from $\mathcal{B}$

 Draw fresh shock paths $\{Z_{i,1:T}\}$ with Π_Z from each terminal $z_i^H[-1]$

for $i = 1, \dots, N_{\text{agg}}$ **do**

Forward roll (no grad): for $t = 1, \dots, T$ compute K_t, L_t from $\mu_{i,t-1}$, prices R_t, w_t , MPC and c_t from $\mathcal{N}_\rho$ on the running history, advance $\mu_{i,t}$ via the Young step

 Append $(z_{i,T}^H, \mu_{i,T})$ to $\mathcal{B}$; evict oldest if $|\mathcal{B}| > C$

end for

for $s = 1, \dots, S$ **do**

 Sample mini-batch $\mathcal{M} \subset \mathcal{B}$ of size B_{fb}

 Compute Fischer–Burmeister loss $\mathcal{L}(\rho)$ on $\mathcal{M}$ using (6.13)

 Update $\rho \leftarrow \text{Adam}(\rho, \nabla_\rho \mathcal{L}(\rho))$ with global-norm clipping

end for

end for

Output: trained $\mathcal{N}_{\rho^*}$ that maps a shock history to grid-MPC coefficients

Three implementation choices in Algorithm 2 are worth flagging. First, the forward roll is wrapped in `stop_gradient` so that gradients only flow through the FB residual evaluated on the buffer, not through long simulation chains; this is what makes training tractable for $T = 100$ horizons. Second, because Young's step gives *exact* aggregate sums, the only stochasticity in the gradient comes from buffer mini-batching, which acts as standard SGD noise rather than a Monte-Carlo aggregation noise floor. Third, the buffer simultaneously decouples training-state drawing from the current policy and lets the network see distributions μ generated by earlier policies, which improves coverage of the ergodic set during early training.

Two training algorithms: residual minimization vs. time iteration. Algorithm 2 is one of two families of training schemes used in the sequence-space DL literature. In *direct residual minimization* (the version above and in our notebook), the network is trained by gradient descent on the squared equilibrium residual itself. In *time iteration with EGM*, the network is trained on a sequence of supervised regression problems: at each outer iteration, one (i) uses the current network to construct next-period policies, (ii) backs out implied current-period policies via the endogenous-grid method of Carroll (2006), and (iii) updates the network by minimizing the squared error against those EGM targets. Time iteration is more involved and requires a per-batch root-finding step, but it is more flexible: it tolerates non-trivial market clearing and, crucially, handles *non-convex* choices (e.g., a discrete retirement decision) and non-monotone Laffer curves where the Euler equation has multiple roots that direct residual minimization cannot disambiguate. In practice, residual minimization is the simpler entry point on smooth, convex problems; switch to time iteration when convergence stalls, when the model contains discrete choices, or when the continuation value has convex regions that admit multiple optimal savings.

Practical heuristics for sequence-space DEQNs

Four operational rules of thumb, distilled from Azinovic-Yang and Žemlička (2025a):

- **Choosing the truncation length T .** Three sensible heuristics, in increasing order of conservatism: (i) for OLG models, set T to roughly two life-cycles, so that all shocks experienced by any household alive today are inside the window; (ii) start short and iteratively increase T , monitoring the equilibrium residual; (iii) “overkill”, set T such that $\varrho_{\text{shock}}^T \leq \text{tol}$ with $\text{tol} \in [10^{-8}, 10^{-6}]$, since long histories are cheap to feed.
- **Innovations or realizations?** For an AR(1) shock $z_t = \varrho z_{t-1} + \varepsilon_t$, feeding the history of *realizations* z_t^T to the network typically allows shorter truncation than feeding the history of *innovations* ε_t^T , because the levels already integrate the persistence. Feeding both is the most accurate option in practice.
- **What to approximate.** Smooth, bounded equilibrium objects (savings rates, MPCs) are easier to learn and yield better Euler accuracy than raw policy levels (consumption, savings). This is the deeper reason the I-spline MPC parameterization in (6.11)–(6.12) pays off so much.
- **When the method breaks.** The truncation argument relies on *ergodicity* of the underlying dynamics, $\partial\Psi/\partial\varepsilon_{t-\tau} \rightarrow 0$ as $\tau \rightarrow \infty$. Models with stable limit cycles or deterministic chaos violate this assumption; in those exotic settings the sequence-space approach is not guaranteed to converge to the equilibrium policy.

Applications and notebooks. The paper applies this framework to three demanding models: (i) an OLG economy with portfolio choice and aggregate risk, (ii) an economy with a continuum of heterogeneous firms and households featuring idiosyncratic and aggregate shocks, and (iii) a lifecycle model with a discrete early-retirement choice that introduces local convexities. Mean Euler equation errors are below 0.2% in all cases. The two TensorFlow 2 companion notebooks are intentionally complementary: `05_SequenceSpace_BrockMirman.ipynb` is a Brock–Mirman warm-up that isolates the history-to-policy logic in the simplest possible environment, while `06_SequenceSpace_KrusellSmith.ipynb` is a compact teaching implementation that combines sequence-space inputs, I-spline policies, and Young’s method in a heterogeneous-agent setting.

A third notebook, `KrusellSmith_Tutorial_CPU.ipynb`, is a JAX/optax port of the upstream pedagogical tutorial released by the paper’s authors. It exposes the same shape-preserving I-spline MPC parameterization, the same Young step inside the simulator, and the same Fischer–Burmeister KKT loss as the TensorFlow notebook, but in the original JAX form. It is adapted from the upstream tutorial `01_KrusellSmith_Tutorial_CPU.ipynb` in the companion code repository (Azinovic-Yang and Žemlička, 2025b), available at <https://github.com/azinoma/DeepLearningInTheSequenceSpace>; the local adaptation adds an explicit shape-guarantee diagnostic and additional inline commentary, leaving the algorithm unchanged.

Math-to-code glossary for the KS sequence-space tutorial

Symbol	Meaning	Notebook name(s)
ρ	Policy-network parameters	psi
z_t^T (or z^H)	Truncated shock history of length H	z_history, z_hist
encoded z^H	One-hot $\parallel$ value vector, length $H(N_Z+1)$	output of <code>encode_Z_history</code>
$\mathcal{N}_\rho$	MLP mapping history to MPC heads	actor_c_grid
$B_{j,n}$	Precomputed I-spline basis matrix	ispline_basis
$\alpha(\varepsilon)$	Boundary MPC head (sigmoid output)	alpha
$\tilde{w}_j(\varepsilon)$	Non-negative I-spline weights (phantom-zero softmax)	w_tilde
$\text{MPC}_{\varepsilon,n}, c$	Decreasing MPC and cumulated grid consumption	mpc, c_grid
$\mu(\varepsilon, k)$	Cross-sectional distribution on the (ε, k) grid	mu
K_t, L_t	Exact aggregates from μ	distribution_aggregates
Young step	Non-stochastic distribution update $\mu \mapsto \mu'$	distribution_step
g, s	Relative Euler residual and savings slack	g, s
$\text{FB}(g, s)$	Smooth complementarity envelope	fb in <code>fb_loss_one_state</code>
$\mathcal{B}$	Replay buffer of (z^H, μ) pairs	buffer_z, buffer_mu

Chapter Summary

Continuum-agent equilibria require explicit distribution tracking; Young’s (2010) histogram is a deterministic mass-redistribution scheme on a fixed grid that converges to the ergodic distribution exactly as the grid is refined. Embedding Young’s update inside a DEQN training loop yields fully differentiable heterogeneous-agent solutions, with gradients flowing through the histogram and the policy network simultaneously. The sequence-space DEQN of [Azinovic-Yang and Žemlička \(2025a\)](#) is the natural alternative when the entire path of aggregate variables, rather than the cross-section, is the state of interest. Bewley–Huggett–Aiyagari–Krusell–Smith is the foundational lineage; [Achdou et al. \(2022\)](#) supply the continuous-time counterpart taken up in Chapter 8.

Further Reading

- [Young \(2010\)](#), the original non-stochastic histogram paper.
- [Krusell and Smith \(1998\)](#), the canonical heterogeneous-agent benchmark with aggregate shocks.
- [Azinovic-Yang and Žemlička \(2025a\)](#), sequence-space DEQNs, the natural extension.
- [Achdou et al. \(2022\)](#), the continuous-time treatment that Chapter 8 builds on.
- [Maliar et al. \(2021\)](#); [Han et al. \(2024\)](#), alternative deep-learning approaches to KS, contrasted in §6.6.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Core] Mean-preserving lottery.** Show that the unique two-point split that places probability ω at k_n and $1 - \omega$ at k_{n+1} such that $\omega k_n + (1 - \omega)k_{n+1} = k'$ is given by $\omega = (k_{n+1} - k') / (k_{n+1} - k_n)$. Verify mass conservation.

2. **[Computational] Closed-form bracketing on log-spaced grids.** Implement the $\mathcal{O}(1)$ bracketing index for a log-spaced grid $k_n = e^{x_0 + n\Delta x} - c$ in five lines of NumPy. Verify against `numpy.searchsorted` on a random batch of queries; measure the relative speed-up.
3. **[Core] Approximate aggregation, scope.** Construct an example economy in which the KS log-linear forecasting rule fails (e.g. multiple assets with switching liquidity). Why does adding higher moments not always rescue the rule?
4. **[Computational] Sequence-space vs. histogram DEQN.** Use Notebook 11 as the histogram-state baseline and compare it with the sequence-space implementation in Notebook 06 or the JAX tutorial. In the sequence-space notebook, vary the history length T (or H in the code) and compare the residual after training. Comment on which formulation generalizes better to a much longer test horizon and why.
5. **[Core] DeepSets permutation invariance.** Consider the DeepHAM aggregator from (6.8), $\mathbf{m}_t = (\sum_i g_\theta^1(s_t^i), \dots, \sum_i g_\theta^M(s_t^i))$. (i) Show that for any permutation π of the agents, $\mathbf{m}_t(\pi \cdot s) = \mathbf{m}_t(s)$, so the encoder is permutation-invariant by construction. (ii) Show that the policy $\pi_\rho(s_t^i; \mathbf{m}_t, a_t)$ is consequently equivariant in the agent index i : if we permute the agents, each agent's policy moves with its own index but the dependence on the population through $\mathbf{m}_t$ is unchanged. (iii) State the converse (Zaheer et al. (2017)): any continuous permutation-invariant function on $\mathbb{R}^d$ -valued sets of fixed cardinality can be written in the form $\rho(\sum_i g(s^i))$. Use this to argue why the DeepHAM architecture can in principle replace tracking the entire histogram with a finite vector $\mathbf{m}_t$ of learned moments, provided the dimension M is large enough.
6. **[Computational] Histogram noise vs. Monte Carlo sampling.** In the Krusell–Smith tutorial notebook `KrusellSmith_Tutorial_CPU.ipynb`, fix one aggregate shock path and one initial distribution. Run Young's histogram with $N_{\text{grid}} = 100$ wealth grid points once, and run R repeated Monte Carlo panels under the same aggregate path for $N \in \{100, 1000, 10\,000\}$ agents. At each $t = 1, \dots, 200$, record aggregate capital $K_t^{(r)}$ for every Monte Carlo replication r and the corresponding Young aggregate K_t^Y . Plot the cross-replication standard deviation $\text{sd}_r(K_t^{(r)})$ over time, or its time average, rather than the time-series standard deviation of K_t . Verify (i) Young's method has zero sampling variance conditional on the aggregate path, (ii) MC sampling variance scales as $1/\sqrt{N}$, (iii) at $N \approx N_{\text{grid}}^2 = 10^4$, MC's stochastic noise becomes comparable to Young's discretization error on a smooth statistic like K_t . Comment on why the discretization-error-vs-MC-error trade-off depends on which functional of μ_t is targeted (mean, variance, tail mass).
7. **[Computational] Two-moment forecasting rule.** In the same notebook, replace the Krusell–Smith log-linear forecasting rule $\log K_{t+1} = a_0 + a_1 \log K_t + a_2 \mathbb{1}[z_t = z_g]$ with a two-moment extension that also conditions on cross-sectional dispersion: $\log K_{t+1} = a_0 + a_1 \log K_t + a_2 \mathbb{1}[z_t = z_g] + a_3 \log V_t$, where $V_t = \text{Var}_{\mu_t}(k)$. Re-fit the rule and the network jointly. Report (i) the forecasting R^2 for $\log K_{t+1}$ before and after, (ii) the maximum Euler residual after training, (iii) by how much the second moment “buys” you in absolute residual terms. Connect the result to the chapter's discussion of *approximate aggregation*: in the standard Krusell–Smith calibration the marginal information in the second moment is small, but it becomes material once you introduce features (e.g., binding borrowing constraints in a non-trivial fraction of the population, multiple assets) that break aggregation.

	Histogram DEQN (Azinovic et al., 2022)	All-in-one DL (Maliar et al., 2021)	DeepHAM (Han et al., 2024)
State distribution representation	Explicit histogram vector on a fixed grid	Explicit panel of N agents' states	Learned generalized moments via a permutation-invariant encoder
Dimension of aggregate state seen by the network	$\mathcal{O}(N_b)$ histogram entries	$\mathcal{O}(N)$ agent states	$M \ll N_b$ learned scalars
Interpretability of distributional state	High (histogram readable)	Low (permutation-dependent)	High in the economic sense (the learned basis is interpretable, e.g. concave in assets, linking the moment to MPCs and redistribution)
Permutation invariance	Automatic (histogram is invariant)	Requires careful architecture / data augmentation	Baked into the encoder by DeepSets structure
Training objective	Euler + Bellman + market clearing + FB residuals (squared)	Euler + Bellman + market clearing residuals (squared)	Cumulative utility along simulated paths plus value-function error; Bellman residual is a validation diagnostic only
Reported accuracy (baseline KS)	Euler errors $\sim 10^{-3}$ – 10^{-4}	Approximation errors $< 1\%$ with $> 10^3$ agents	Bellman residual (used as a validation diagnostic) reduced by 68.9% vs. KS with one learned generalized moment

Table 6.2: Three deep-learning approaches to heterogeneous-agent equilibria, all applied to the Krusell–Smith benchmark. They differ on two axes: how the cross-sectional distribution is encoded as input to the network, and what objective is optimized. Histogram-DEQN and All-in-One DL minimize squared structural residuals (Euler, Bellman, market clearing); DeepHAM instead maximizes cumulative utility along simulated paths, with the individual law of motion embedded in the computational graph and Bellman residuals tracked as a validation diagnostic.

Method	Pros	When it shines
Histogram DEQN (Azinovic et al., 2022)	Interpretable state; exact market clearing; reuses the DEQN template	Teaching; N_b moderate; smooth policies
All-in-One DL (Maliar et al., 2021)	No grid design; large- N panels; single optimizer for all residuals	Large- N research problems; OLG-many-cohort extensions
DeepHAM (Han et al., 2024)	Learned moments; cumulative-utility objective; risky steady state; ZLB	Rich HA macro-finance; constrained-efficiency / optimal-policy design

Table 6.3: Practical chooser for the three DL approaches to Krusell–Smith. When several rows look applicable, the recommended ordering is: start with Histogram DEQN if a clean teaching narrative is the goal; switch to All-in-One DL if grid design is awkward; switch to DeepHAM if the cumulative-utility objective or learned moments are first-order to the question. Adapted from the L09 deck’s *Head-to-Head* slide.

Chapter 7

Physics-Informed Neural Networks

Chapters 2–6 operated in discrete time, where the equilibrium conditions take the form of expectational (Euler) equations. We now shift to *continuous time*, where the optimality conditions are partial differential equations – Hamilton–Jacobi–Bellman (HJB) equations for optimal control and Kolmogorov forward equations for wealth distributions. *Physics-Informed Neural Networks* (PINNs) approximate the solution of such PDEs by minimizing the PDE residual at collocation points, using automatic differentiation to compute the required derivatives (Sirignano and Spiliopoulos, 2018; Raissi et al., 2019). The PINN loss is structurally analogous to the DEQN loss of Chapter 2, but instead of time-stepping a fixed-point iteration we solve a single PDE globally over the state space. This chapter develops the PINN methodology from simple ODEs through boundary-condition strategies to the HJB equation for consumption–savings problems, with applications to macro-finance.

7.1 From DEQNs to PINNs: Discrete vs. Continuous Time

Many frontier models in macroeconomics and finance are written in continuous time. Examples include Hamilton–Jacobi–Bellman (HJB) equations for optimal control, Kolmogorov forward equations for wealth distributions, and Black–Scholes PDEs for asset pricing. The DEQN methodology from Chapters 2–3 handles discrete-time equilibrium conditions. Its continuous-time analogue relies on derivatives of the network with respect to its inputs, which automatic differentiation provides directly.

The resulting framework, known as *Physics-Informed Neural Networks* (PINNs), was introduced by Raissi et al. (2019). In economics and finance, continuous-time models governed by HJB and related PDEs arise naturally in asset pricing under climate uncertainty (Barnett et al., 2020a), portfolio choice (Duarte et al., 2024), and heterogeneous-agent economies with aggregate shocks (Gopalakrishna, 2024). Duarte et al. (2024) apply machine-learning methods to continuous-time finance problems, while Gopalakrishna (2024) develops the ALIENs framework for solving continuous-time economies with deep learning. PINNs provide a natural and scalable computational framework for such problems. Figure 7.1 summarizes the discrete-time DEQN template and its continuous-time PINN analogue.

7.2 The PINN Loss and Automatic Differentiation for PDEs

Prerequisites in one paragraph. A few terms from PDE numerics recur throughout the chapter and deserve to be fixed in advance. *Collocation points* are interior evaluation points at which the PDE residual is enforced; they do not need to lie on a Cartesian grid, and in high dimensions they are typically drawn at random or from a low-discrepancy sequence. The *strong form* minimizes the residual point-wise and requires the network’s activation to be at least C^k

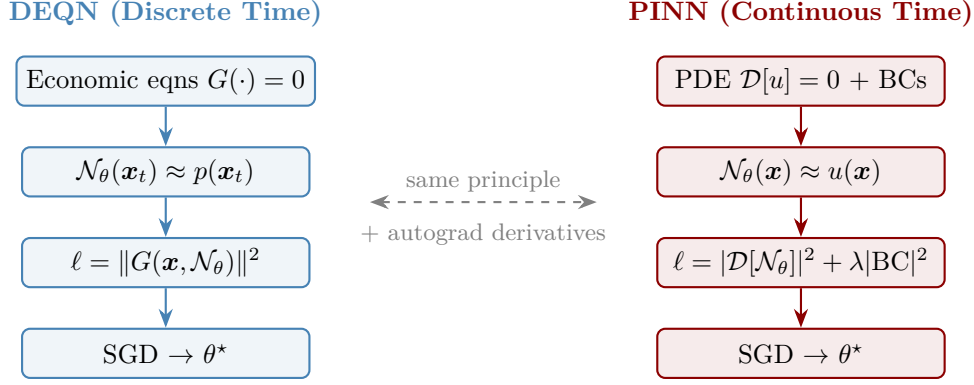


Figure 7.1: Discrete-time DEQNs and continuous-time PINNs use the same residual-minimization principle with different mathematical residuals. DEQNs minimize algebraic equilibrium conditions such as Euler-equation errors evaluated along simulated states, while PINNs minimize PDE and boundary residuals evaluated at collocation points using derivatives of the network with respect to its inputs.

if the differential operator is of order k . The *weak form* integrates the residual against test functions, which tolerates rougher solutions but is rarely needed for the PDEs in this chapter. We assume throughout that each problem is *well-posed* (existence, uniqueness, and continuous dependence on data); proving well-posedness for a particular HJB or KFE is part of the PDE literature, not of this course.

Given a PDE $\mathcal{D}[u](\mathbf{x}) = 0$ on a domain Ω with boundary conditions $\mathcal{B}[u](\mathbf{x}) = 0$ on $\partial\Omega$, the PINN loss is:

$$\ell_{\theta}^{\text{PINN}} = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{D}[\mathcal{N}_{\theta}](\mathbf{x}_i^r)|^2}_{\text{PDE residual}} + \underbrace{\frac{\lambda}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}[\mathcal{N}_{\theta}](\mathbf{x}_j^b)|^2}_{\text{boundary conditions}}. \quad (7.1)$$

The derivatives appearing in $\mathcal{D}[\mathcal{N}_{\theta}]$ are computed algorithmically via automatic differentiation¹ (e.g., `torch.autograd.grad` in PyTorch), up to floating-point precision and without any finite-difference approximation. This removes finite-difference truncation error in derivative terms and is a major practical advantage in high dimensions because no state-space grid is required. Remaining error sources are function-approximation error, optimization error, and collocation sampling error.

Activation function requirements. The choice of activation function is particularly important for classical *strong-form* PINNs that compute high-order derivatives via automatic differentiation. If the PDE operator $\mathcal{D}$ involves k -th order derivatives, the network’s activation function must be at least C^k for the strong residual to be well-defined. $\text{ReLU}(z) = \max(0, z)$ is only C^0 and its second derivative is zero almost everywhere (and undefined at the kink), so a ReLU network is not suitable for the strong form of a second-order PDE. For second-order PDEs (HJB, Black–Scholes, Poisson), one should use C^∞ activations such as tanh, Swish, or softplus. This requirement stands in contrast to the supervised setting, where ReLU is the default. The trade-off is real: smooth saturating activations propagate gradients more slowly than ReLU on the same architecture (the upper plateau of tanh has near-zero slope), so PINNs typically need wider networks or more iterations to reach the same training loss as a comparable supervised

¹Automatic differentiation is the algorithmic backbone of every deep-learning framework discussed in this script; Baydin et al. (2018) survey forward- and reverse-mode AD, the dual-number and tape-based implementations, and the practical trade-offs that determine why reverse mode is the standard for neural-network training (one backward pass costs roughly the same as one forward pass, irrespective of the parameter count).

ReLU network. In practice this is the price of having well-defined second derivatives. Note that the limitation is specific to strong-form auto-differentiation PINNs: weak/variational formulations and methods that handle nonsmooth solutions explicitly can still use ReLU activations, since high-order derivatives never need to be differentiated point-wise. More broadly, PINN optimization can suffer from gradient pathologies across loss terms, which is why adaptive loss balancing is often beneficial (Wang et al., 2021; Bischof and Kraus, 2025).

Framework choice: PyTorch vs. TensorFlow

The PINN notebooks use PyTorch, whereas the DEQN code (Chapters 2–3) uses TensorFlow. This is a practical implementation choice rather than a mathematical one: PINNs repeatedly differentiate the network with respect to its *inputs* (e.g., V_{SS} in the Black–Scholes PDE), and PyTorch’s eager-mode `autograd` makes those higher-order derivatives easy to compose. TensorFlow remains a natural fit for the DEQN training loop already used earlier in the course. The underlying mathematics is identical in both frameworks: a PINN written here in PyTorch can be ported to TensorFlow line-for-line by replacing `torch.autograd.grad` with `tf.GradientTape` (one tape per derivative order, since TF tapes are by default not persistent), and `torch.compile` with `tf.function`.

7.2.1 A Concrete Example: Solving a 1D ODE with a PINN

To build intuition, consider the boundary value problem

$$y''(x) = -1, \quad x \in (0, 1), \quad y(0) = y(1) = 0, \quad (7.2)$$

whose analytical solution is $y^*(x) = \frac{1}{2}x(1-x)$. Approximate the unknown function by a raw neural network $\mathcal{N}_\theta(x)$, with no boundary information built in. The strong-form residual is

$$R(x; \theta) = \partial_{xx}\mathcal{N}_\theta(x) + 1, \quad (7.3)$$

where $\partial_{xx}\mathcal{N}_\theta$ is obtained by two applications of `torch.autograd.grad`. The boundary conditions are then added to the loss as a penalty term – the *soft* enforcement strategy:

$$\ell_\theta = \frac{1}{N_r} \sum_{i=1}^{N_r} R(x_i; \theta)^2 + \lambda(\mathcal{N}_\theta(0)^2 + \mathcal{N}_\theta(1)^2), \quad (7.4)$$

with collocation points x_i drawn uniformly from $(0, 1)$ and a weight $\lambda > 0$ on the boundary term. Building the boundary conditions directly into the network output instead – the *hard* enforcement alternative, which removes the λ term entirely – is taken up in §7.3.

Collocation strategies

Uniform random sampling is the simplest collocation strategy but not necessarily the most efficient. Alternatives include low-discrepancy Sobol sequences (Sobol’, 1967); Latin Hypercube Sampling (LHS; McKay et al., 1979), which provides better space coverage; other quasi-Monte Carlo points (Niederreiter, 1992; Owen, 1995), which fill the domain more evenly; and residual-based adaptive refinement (RAR; Lu et al., 2021b), which concentrates points in regions where the current PDE residual is largest. For most applications in this course, uniform or LHS sampling is sufficient, but adaptive refinement can be beneficial for PDEs with sharp gradients, boundary layers, and near payoff discontinuities (e.g., the kink at $S = K$ in Black–Scholes). *Mini-batch sizes.* Per-step collocation batches are typically 32–256 points in low-dimensional problems and 512–4096 when the input dimension or interaction-order grows; the right number is the largest that fits comfortably in GPU memory together with the AD graph for the highest-order derivative needed. All companion notebooks make this an explicit hyperparameter near the top of the training cell.

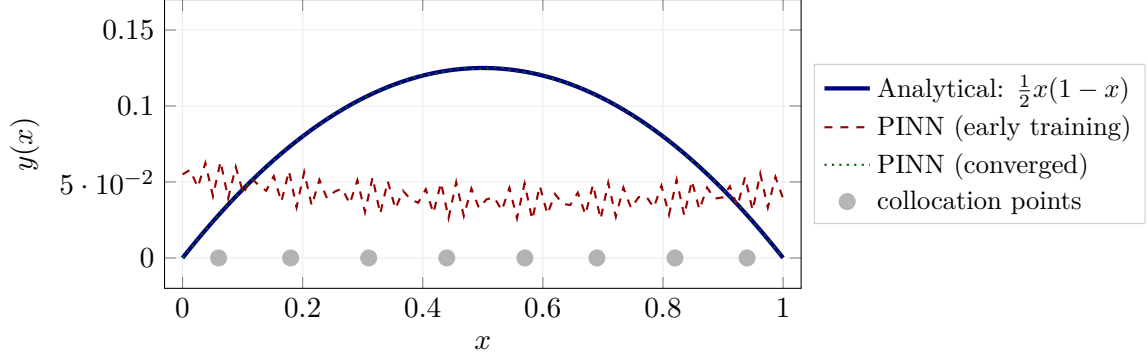


Figure 7.2: PINN solution of the 1D ODE $y'' = -1$ on $(0, 1)$ with $y(0) = y(1) = 0$ and the soft-penalty loss (7.4). The analytical solution $\frac{1}{2}x(1-x)$ (solid blue) is recovered to plotting accuracy by the converged network (dotted green); the dashed red curve illustrates a typical early-training iterate, which still misses the endpoints because the boundary penalty is enforced only approximately. Tick marks on the x -axis are the uniformly drawn collocation points. The curves above are TikZ illustrations rather than direct exports. Notebook `lecture_11_01_ODE_PINN_ZeroBCs` runs exactly this experiment; notebook `lecture_11_02_ODE_PINN_SoftVsHardBCs` then contrasts the soft penalty against the hard trial-solution construction of §7.3 on a non-zero-BC variant.

Figure 7.2 shows this calculation graphically. This simple example illustrates the key ingredients of every PINN: (i) a neural network that approximates the unknown function, (ii) automatic differentiation to compute derivatives, (iii) a loss function built from the PDE residual (plus, here, a boundary penalty), and (iv) collocation points sampled from the domain interior. The same machinery extends directly to PDEs in two or more dimensions, as we demonstrate in the following applications.

7.3 Boundary Conditions: Soft vs. Hard Enforcement

The treatment of boundary conditions is a critical design choice in PINNs.

Soft enforcement. Boundary conditions are penalized as an additional loss term, weighted by λ . The difficulty is that λ must be tuned: too small and the BCs are violated; too large and the optimizer ignores the PDE interior. Figure 7.3 sketches the two failure modes and the compromise regime in between.

Hard enforcement. A *trial solution* is constructed that satisfies the boundary conditions *by construction*:

$$\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x), \quad (7.5)$$

where $A(x)$ is an anchor function satisfying the BCs exactly, and $B(x)$ is a mask function that vanishes at the boundary. For Dirichlet BCs $\hat{y}(0) = a$, $\hat{y}(1) = b$, one may choose $A(x) = a + (b-a)x$ and $B(x) = x(1-x)$. Figure 7.4 visualizes this anchor-plus-mask decomposition for non-zero endpoint data.

Hard enforcement eliminates the boundary loss term entirely, reducing the number of hyperparameters and improving accuracy near the boundaries. The trade-off is real, however: because $\hat{y} = A + B \cdot \mathcal{N}_\theta$ multiplies the network output by a mask B that vanishes on $\partial\Omega$, the input-gradient $\partial\hat{y}/\partial x$ inherits a factor of B near the boundary and is thus damped by construction. When the true solution exhibits a steep boundary layer (e.g., a Hamilton–Jacobi–Bellman equation at the borrowing constraint, see §8.4), the network must compensate by making $\mathcal{N}_\theta$ itself locally large,

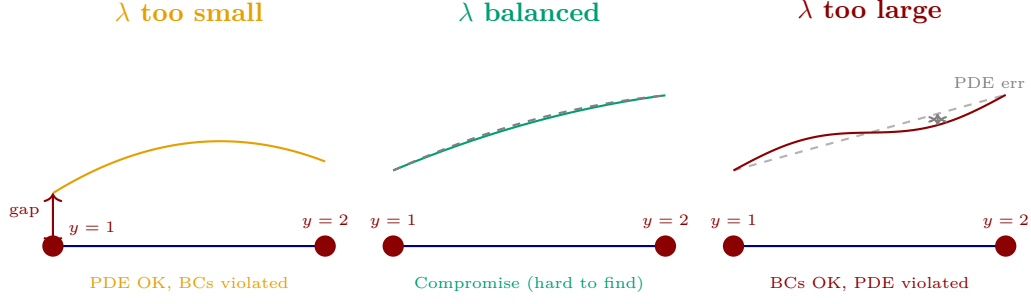


Figure 7.3: Failure modes of soft boundary-condition enforcement on a Dirichlet problem with $y(0) = 1$, $y(1) = 2$. *Left*: the BC penalty weight λ is too small, so the optimizer minimizes the interior PDE residual but lets the candidate solution miss both endpoints (visible as the “gap” at $x = 0$). *Right*: λ is too large, so the network nails the boundary values but distorts the interior, producing a wiggly profile with PDE residual error against the affine reference (dashed gray). *Centre*: a balanced λ approximately satisfies both objectives, but the right-shaped value depends on the network, the PDE, and the geometry, and is not known a priori. This trade-off motivates the hard-enforcement construction below, which removes the boundary loss term entirely.

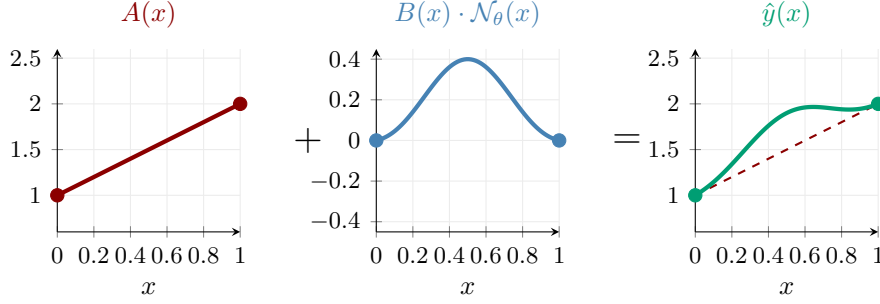


Figure 7.4: Hard boundary-condition decomposition for the trial solution $\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)$ with Dirichlet data $\hat{y}(0) = 1$, $\hat{y}(1) = 2$. *Left*: anchor $A(x) = 1 + x$ matches the boundary values exactly. *Centre*: mask times network, $B(x) \cdot \mathcal{N}_\theta(x)$ with mask $B(x) = x(1 - x)$, which vanishes at $x \in \{0, 1\}$ regardless of $\mathcal{N}_\theta$. *Right*: their sum $\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)$ (solid green) is a candidate solution that satisfies both BCs by construction; the dashed red line is the affine anchor $A(x) = 1 + x$ for visual reference. Training loss reduces to the interior PDE residual alone.

which can slow convergence. In short: hard BCs trade boundary-loss tunability for vanishing input gradients at $\partial\Omega$, and the trade is favorable for smooth Dirichlet problems but less obviously so for problems with sharp boundary features.

A worked 1D instance. As a concrete instantiation, return to the simple ODE $y''(x) + y(x) = 0$ on $[0, \pi/2]$ with $y(0) = 0$, $y(\pi/2) = 1$, whose analytical solution is $\sin x$. A trial solution that builds in both endpoints is

$$\hat{y}(x) = \underbrace{\frac{2x}{\pi}}_{A(x)} + x \underbrace{\left(\frac{\pi}{2} - x\right)}_{B(x)} \cdot \mathcal{N}_\theta(x), \quad (7.6)$$

which has the anchor-plus-mask form (7.5): the anchor A matches the boundary data ($A(0) = 0$, $A(\pi/2) = 1$) and the mask B vanishes at both endpoints, so $\hat{y}(0) = 0$ and $\hat{y}(\pi/2) = 1$ hold

exactly for any network output, and the loss reduces to the interior PDE residual alone,

$$\ell_\theta = \frac{1}{N_r} \sum_{i=1}^{N_r} (\hat{y}''(x_i) + \hat{y}(x_i))^2,$$

with $\hat{y}''$ obtained by two applications of `torch.autograd.grad` and collocation points x_i drawn uniformly from $[0, \pi/2]$.

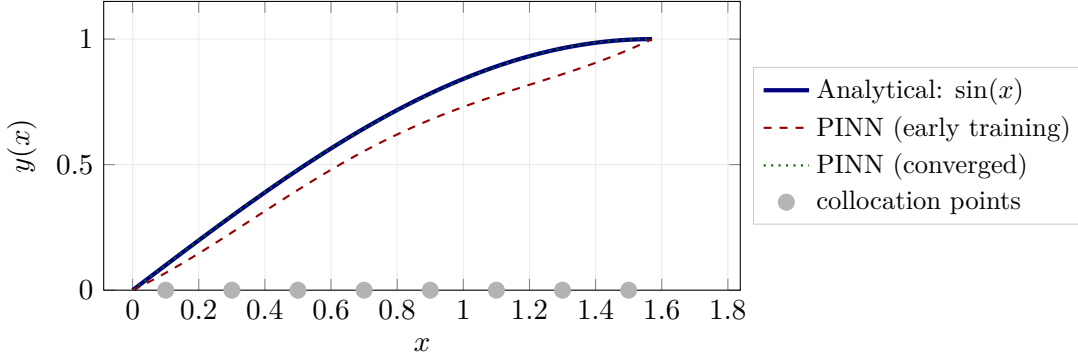


Figure 7.5: PINN solution of the 1D ODE $y'' + y = 0$ on $[0, \pi/2]$ with the hard-BC trial solution (7.6). The analytical solution $\sin(x)$ (solid blue) is recovered to plotting accuracy by the converged network (dotted green); the dashed red curve illustrates a typical early-training iterate. Tick marks on the x -axis are the uniformly drawn collocation points. The curves above are TikZ illustrations rather than direct exports. Notebook `lecture_11_02_ODE_PINN_SoftVsHardBCs` contrasts this hard-trial-solution construction with the soft-penalty alternative on a non-zero-BC variant.

Figure 7.5 confirms that the converged trial solution recovers $\sin x$ to plotting accuracy, with the boundary values exact by construction; contrast the early-training iterate of Figure 7.2, which still misses the endpoints under the soft penalty.

Transfinite interpolation for 2D domains. The 1D hard-enforcement idea ($\hat{y} = A + B \cdot \mathcal{N}_\theta$) extends naturally to rectangular 2D domains, but the anchor function A must now interpolate prescribed boundary data along entire *edges*, not just at two endpoints. The classical technique for this is transfinite interpolation (also known as Gordon–Coons blending): it constructs a function over the rectangle that exactly matches given boundary curves, in much the same way that bilinear interpolation matches four corner values, except that here we match four continuous edge functions rather than four discrete values.

Consider a rectangular domain $[a, b] \times [c, d]$ with Dirichlet boundary data $f_L(y)$ (left edge, $x = a$), $f_R(y)$ (right edge, $x = b$), $f_B(x)$ (bottom edge, $y = c$), and $f_T(x)$ (top edge, $y = d$). Introduce the normalized coordinates $\xi = (x - a)/(b - a) \in [0, 1]$ and $\eta = (y - c)/(d - c) \in [0, 1]$. The idea is to blend the four edge functions using linear weights, then correct for the corners that get counted twice.

Step 1, Interpolate in x : The function $(1 - \xi) f_L(y) + \xi f_R(y)$ matches the left and right edges exactly for every y , but says nothing about the top and bottom edges.

Step 2, Interpolate in y : Similarly, $(1 - \eta) f_B(x) + \eta f_T(x)$ matches the bottom and top edges exactly for every x .

Step 3, Add and correct. Summing these two would double-count the corner values (e.g., $f_L(c)$ and $f_B(a)$ both contribute at (a, c)). We subtract a bilinear interpolant through the four

corner values to compensate. The result is the blending (anchor) function:

$$\begin{aligned}
A(x, y) = & \underbrace{(1 - \xi) f_L(y) + \xi f_R(y)}_{\text{left/right interpolation}} + \underbrace{(1 - \eta) f_B(x) + \eta f_T(x)}_{\text{bottom/top interpolation}} \\
& - \underbrace{[(1 - \xi)(1 - \eta) f_{BL} + \xi(1 - \eta) f_{BR} + (1 - \xi)\eta f_{TL} + \xi\eta f_{TR}]}_{\text{bilinear corner interpolant}},
\end{aligned} \tag{7.7}$$

where $f_{BL} = f_B(a) = f_L(c)$, etc., are the four corner values. These values must be *consistent* across the two edges that meet at each corner (e.g., the value of f_L at $y = c$ must equal the value of f_B at $x = a$); otherwise the corner-correction bilinear cancels imperfectly and A does not match any of the four edges exactly at the offending corner. Under consistent corner data, A matches all four edge functions exactly by construction.

The mask function must vanish on all four edges so that the network can modify the interior without disturbing the boundaries:

$$B(x, y) = \xi(1 - \xi)\eta(1 - \eta). \tag{7.8}$$

Note that $B = 0$ whenever $\xi \in \{0, 1\}$ or $\eta \in \{0, 1\}$, i.e., on every edge of the rectangle. The hard-enforced trial solution is then:

$$\hat{u}(x, y) = A(x, y) + B(x, y) \cdot \mathcal{N}_\theta(x, y). \tag{7.9}$$

On any boundary edge, $B = 0$ and $\hat{u}$ reduces to A , which matches the prescribed data. In the interior, $B > 0$ and the network $\mathcal{N}_\theta$ is free to learn whatever shape is needed to satisfy the PDE. This construction satisfies all four Dirichlet conditions exactly for *any* network output $\mathcal{N}_\theta$.

When to use hard vs. soft BCs

Hard enforcement is preferred whenever a valid trial solution can be constructed analytically, which is the case for most standard boundary value problems in economics (e.g., Dirichlet conditions on finite domains). **Soft enforcement** is necessary when the boundary conditions are complicated (e.g., free-boundary problems, state-dependent constraints) or when the domain geometry makes it difficult to construct the mask function $B(x)$. In practice, hard enforcement typically improves accuracy by 1–2 orders of magnitude near the boundaries.

7.3.1 2D Poisson Benchmark

As a pedagogical 2D benchmark, the accompanying notebook `lecture_11_03_PDE_PINN_Poisson2D` solves a Poisson equation on the unit square,

$$\nabla^2 u(x, y) = f(x, y), \quad (x, y) \in [0, 1]^2, \tag{7.10}$$

with manufactured solution $u^*(x, y) = x^2 + y + \sin(\pi x) \sin(\pi y)$, yielding $f(x, y) = 2 - 2\pi^2 \sin(\pi x) \sin(\pi y)$ and non-homogeneous Dirichlet data $u|_{y=0} = x^2$, $u|_{y=1} = x^2 + 1$, $u|_{x=0} = y$, $u|_{x=1} = 1 + y$. (Some PDE references prefer the standard-elliptic convention $-\nabla^2 u = f$; switching to that form flips the sign of f but leaves the boundary data and the network architecture unchanged.) Because $\sin(\pi x) \sin(\pi y)$ vanishes on $\partial\Omega$, the polynomial part $A(x, y) = x^2 + y$ already matches all four edges (and their corner values) exactly, so it serves as the transfinite-interpolation anchor (the corner-consistency requirement of §7.3 is automatic here). The mask $B(x, y) = x(1 - x)y(1 - y)$ vanishes on every edge, and the hard-enforced trial solution $\hat{u}(x, y) = A(x, y) + B(x, y)\mathcal{N}_\theta(x, y)$ satisfies all Dirichlet conditions by construction; the network is trained on the interior PDE residual alone. In the reference notebook configuration, the verification gate is a mode-dependent

relative- L_2 error against the manufactured solution on a 100×100 test grid (disabled in the **smoke** CI run, with progressively tighter bounds in **teaching** and **production**), computed for a modest network and a few thousand collocation points; the exact numbers depend on seed, hardware, optimizer settings, and the collocation sample. This benchmark is the clean 2D extension of the 1D ODE example and a useful bridge before turning to economic applications, and it exercises the full hard-BC transfinite-interpolation machinery on non-zero edge data rather than collapsing to the trivial $u|_{\partial\Omega} = 0$ case.

7.4 Common PINN Failure Modes and Remedies

In practice, PINN training often fails for optimization reasons rather than approximation capacity. Table 7.1 lists the most common pathologies and practical remedies.

Symptom	Typical cause	Practical remedy
Large boundary violations	Soft BC penalties underweighted or unstable	Prefer hard BC trial solutions when possible; otherwise use adaptive loss balancing (e.g., ReLo-BRaLo).
Good BC fit, poor interior PDE fit	Boundary penalties overweighted	Decrease BC weights or rebalance losses dynamically.
Residual spikes in narrow regions	Collocation undercoverage (boundary layers / kinks)	Use residual-based adaptive resampling, Sobol/LHS points, and local point refinement.
Slow or stalled optimization	Gradient imbalance across loss terms; or initialization in a basin disconnected from the true solution	Normalize loss components and monitor per-term gradients; use adaptive balancing schemes (Wang et al., 2021; Bischof and Kraus, 2025); restart with different seeds or warm-start from a coarser solver if the loss plateaus far from any plausible solution.
Optimizer stuck on a symmetric / structureless ansatz	Initialization too symmetric to represent an asymmetric solution; gradient updates preserve the symmetry	Break the symmetry explicitly: asymmetric weight initialization, an auxiliary symmetry-breaking input feature, or a short pre-training pass that nudges the ansatz toward the correct shape.
Oscillatory solution near payoff kink	Spectral bias and non-smooth target geometry	Increase sampling density near kink and split domain if needed; use problem-specific transforms.

Table 7.1: Common PINN failure modes and practical remedies. The main implementation lesson is to monitor loss components separately: a small total loss can hide boundary violations, interior residual spikes, or gradient imbalance across terms.

The penultimate row above is folklore in computational quantum chemistry, where solvers routinely ship dedicated symmetry-breaking modules because a symmetric initial guess is preserved exactly under gradient training; the analogous trap shows up in PINNs whenever the true solution lacks a symmetry that the architecture happens to enforce.

For the course applications, a robust default stack is: smooth activation (tanh or Swish), hard BCs when analytically available, adaptive loss balancing, and adaptive collocation near high-residual regions.

7.5 The Deep Galerkin Method (DGM) Architecture

The plain MLP used in the examples so far – and, below, for the cake-eating HJB – is sufficient for low-dimensional PDEs with smooth solutions, but its expressive bottleneck shows up in two regimes: (i) genuinely high-dimensional state spaces (where curse-of-dimensionality effects compound across layers), and (ii) PDEs with sharp internal features such as boundary layers, wave fronts, or kinks at policy switches. The Deep Galerkin Method addresses both by adding LSTM-style gates and skip connections from the input to every layer, so that the network can carry input information forward unchanged through depth and route it past intermediate transformations. In the 1D problems studied so far, an MLP and a DGM block train to comparable accuracy and the MLP is preferred for transparency; the architectural complexity pays off as dimension grows or sharp features appear. Table 7.2 gives a qualitative comparison across the problem classes treated in this chapter; the chapter’s exercises invite a concrete benchmark of the two architectures on the same PDE.

Problem class	MLP	DGM	Where DGM helps
1D ODE (§7.3)	comparable	comparable	not in 1D smooth problems; MLP preferred for transparency
2D Poisson (§7.3)	comparable	comparable	marginally, on stiff sources
Cake-eating HJB (§7.6)	preferred	comparable	MLP wins when hard BCs already kill the boundary loss; DGM wins if kinks at policy-switching points dominate
Black–Scholes (§7.8)	adequate	cleaner near kink	sharp payoff kink at $S = K$; DGM gates absorb local geometry better
High-dim HJB ($d \gtrsim 4$)	curse of dim. visible	preferred	input-skip connections retain raw coordinates at every depth, mitigating expressivity loss across layers

Table 7.2: Qualitative MLP vs. DGM comparison across the problem classes of this chapter. “Comparable” means the two architectures train to similar final residual at similar wall time; “preferred” indicates which one a practitioner should reach for first. A quantitative benchmark on a fixed pair (residual, wall time, parameters) is the natural project extension and is not run here.

For high-dimensional PDEs, [Sirignano and Spiliopoulos \(2018\)](#) introduced the Deep Galerkin Method (DGM), an architecture with LSTM-style gating ([Hochreiter and Schmidhuber, 1997](#)) and skip connections from the input layer to every hidden layer reminiscent of Highway Networks ([Srivastava et al., 2015](#)) (the immediate precursor of ResNets, in which a learned gate controls how much of the previous representation is carried forward unchanged versus transformed).² A related deep BSDE-based formulation was introduced by [E et al. \(2017\)](#) (E–Han–Jentzen) and developed in the companion paper of [Han et al. \(2018\)](#) (Han–Jentzen–E). An accessible exposition of DGM together with several PDE applications is given by [Al-Arabi et al. \(2018\)](#), which also introduces the gate naming convention adopted below. The original DGM architecture of [Sirignano and Spiliopoulos \(2018\)](#) uses four gates at each layer l :

$$Z^{(l)} = \sigma(\mathbf{W}_z^{(l)} \mathbf{a}^{(l-1)} + \mathbf{U}_z^{(l)} \mathbf{x} + \mathbf{b}_z^{(l)}), \quad (\text{update gate}) \quad (7.11)$$

$$G^{(l)} = \sigma(\mathbf{W}_g^{(l)} \mathbf{a}^{(l-1)} + \mathbf{U}_g^{(l)} \mathbf{x} + \mathbf{b}_g^{(l)}), \quad (\text{forget gate}) \quad (7.12)$$

$$R^{(l)} = \sigma(\mathbf{W}_r^{(l)} \mathbf{a}^{(l-1)} + \mathbf{U}_r^{(l)} \mathbf{x} + \mathbf{b}_r^{(l)}), \quad (\text{relevance gate}) \quad (7.13)$$

$$H^{(l)} = \tanh(\mathbf{W}_h^{(l)} (R^{(l)} \odot \mathbf{a}^{(l-1)}) + \mathbf{U}_h^{(l)} \mathbf{x} + \mathbf{b}_h^{(l)}), \quad (\text{candidate state}) \quad (7.14)$$

²Notation reuse: the gate $G^{(l)}$ in the DGM block below is unrelated to the cross-sectional density g that appears in heterogeneous-agent models (Chapters 6, 8); the symbols share a letter only.

and the state update is:

$$\mathbf{a}^{(l)} = (1 - G^{(l)}) \odot H^{(l)} + Z^{(l)} \odot \mathbf{a}^{(l-1)}. \quad (7.15)$$

Note that this formulation uses two separate gates: $Z^{(l)}$ controls how much of the previous state $\mathbf{a}^{(l-1)}$ to retain, while $(1 - G^{(l)})$ scales the new candidate $H^{(l)}$. Because Z and G are independent in the original Sirignano–Spiliopoulos formulation, the two coefficients need not sum to one. A simpler GRU-style variant (Cho et al., 2014) collapses the two gates into a single convex combination $\mathbf{a}^{(l)} = (1 - G) \odot \mathbf{a}^{(l-1)} + G \odot H$, where the coefficients sum to one; in practice, both variants perform comparably on the PDE benchmarks in this course. The gate $R^{(l)}$ controls which parts of the previous state are relevant for computing the candidate $H^{(l)}$. The gate naming convention (Z, G, R, H) used here follows Al-Aradi et al. (2018); in GRU terminology, Z corresponds to the update gate and R to the reset gate. The input $\mathbf{x}$ enters every layer through the $\mathbf{U}$ matrices, providing skip connections that ensure input information is available at every depth. This gating mechanism, directly analogous to LSTM recurrent networks, helps the DGM architecture learn functions that depend sensitively on the input coordinates, a common feature of PDE solutions near boundary layers. Figure 7.6 shows the resulting feed-forward architecture.

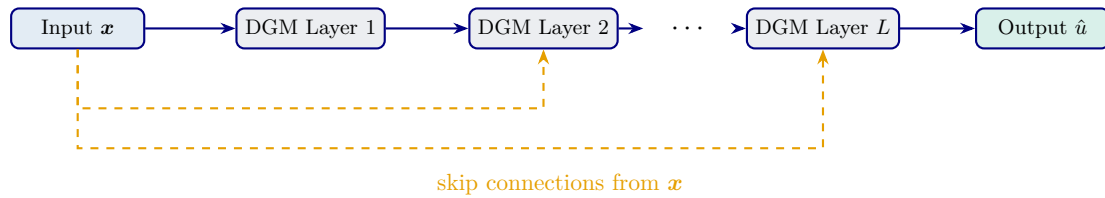


Figure 7.6: The DGM (Deep Galerkin Method) architecture of Sirignano and Spiliopoulos (2018). The input $\mathbf{x}$ feeds the first layer through a standard forward path (solid arrows) and is, in addition, routed to every subsequent DGM block via skip connections (dashed arrows), so each layer can see $\mathbf{x}$ directly as well as the running hidden state. Each DGM block combines $\mathbf{x}$ with the hidden state through update, forget, and relevance gates (see body text), in the spirit of LSTM/GRU recurrences applied across depth rather than time.

Stationary vs. evolutionary PDEs. The PINN framework applies uniformly to both *stationary* PDEs, where the unknown depends only on state variables ($u = u(\mathbf{x})$), and *evolutionary* PDEs, where time enters as an additional input ($u = u(\mathbf{x}, t)$). The network architecture is identical in both cases: only the input dimension changes. The HJB equation below is stationary: the value function $V(a)$ depends on a single state variable. The Black–Scholes PDE of Section 7.8 is evolutionary: $V(S, t)$ depends on both the asset price and time to maturity.

7.6 Application: HJB Equation and the Cake-Eating Problem

Notation: γ as CRRA

In this section, γ denotes the coefficient of relative risk aversion (CRRA), with utility $c^{1-\gamma}/(1-\gamma)$. This convention differs from Chapter 3, where γ_j denotes the intertemporal elasticity of substitution (IES = 1/CRRA). Both conventions are standard in their respective literatures.

Consider a household with wealth $a(t) > 0$ that chooses a consumption stream $c(t) \geq 0$ to maximize discounted lifetime utility:

$$\max_{\{c(t)\}_{t \geq 0}} \int_0^\infty e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} dt \quad \text{s.t.} \quad \dot{a}(t) = r a(t) - c(t), \quad (7.16)$$

where $\rho > 0$ is the subjective discount rate, $\gamma > 0$ ($\gamma \neq 1$) is the coefficient of relative risk aversion (CRRA), and r is the interest rate on wealth. The budget constraint says that wealth grows at rate r minus consumption; we call this the “cake-eating” problem in a loose sense because the household consumes out of a single finite stock of wealth. Strictly speaking, the textbook cake-eating problem has $r = 0$ (the cake does not grow); the $r > 0$ variant solved here is the consumption–savings problem with deterministic returns, and Exercise 3 traces the $r = 0$ limit explicitly. The dynamic-programming perspective is classical in economics; see [Stokey et al. \(1989\)](#) for the standard textbook treatment. Conceptually, this is the same recursive optimization that produced the discrete-time Bellman equations of Chapters 2–3; the only difference is that the household now optimizes over a continuous time grid, so the resulting optimality condition is a partial differential equation (the HJB) rather than an algebraic Euler equation. The derivation below makes this $\Delta t \rightarrow 0$ link precise.

Deriving the HJB equation. Define the *value function* $V(a)$ as the maximum attainable lifetime utility starting from wealth a :

$$V(a) = \max_{\{c(t)\}} \int_0^\infty e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} dt. \quad (7.17)$$

Since the problem is stationary (no explicit time dependence in the return or the law of motion), V depends only on the current state a , not on calendar time.

To derive the HJB equation, apply the *principle of optimality*: over a short interval $[0, \Delta t]$, the household chooses c optimally and then continues optimally from the resulting state $a + \dot{a} \Delta t$. This gives:

$$V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} \Delta t + e^{-\rho \Delta t} V(a + (ra - c) \Delta t) \right\} + \mathcal{O}(\Delta t^2). \quad (7.18)$$

Now expand both the discount factor and the value function to first order in Δt :

$$e^{-\rho \Delta t} \approx 1 - \rho \Delta t, \quad (7.19)$$

$$V(a + (ra - c) \Delta t) \approx V(a) + V'(a) (ra - c) \Delta t. \quad (7.20)$$

Where the upwind scheme comes from

The expansion (7.20) silently assumes that V is differentiable at a . When V has a kink, a common feature of HJB solutions, e.g. at borrowing constraints or policy-switching points, the expansion still holds, but $V'(a)$ must be replaced by the appropriate *one-sided* derivative chosen by the sign of the drift $(ra - c)$. This is exactly the origin of the upwind scheme used in finite-difference HJB solvers: the side of the derivative is selected to follow information flow. PINNs that minimize the strong-form residual implicitly demand two-sided differentiability and therefore tend to over-smooth genuine kinks, one of the mechanisms behind the oscillatory failure mode in Table 7.1.

Substituting (7.19) and (7.20) into (7.18):

$$V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} \Delta t + (1 - \rho \Delta t) [V(a) + V'(a)(ra - c) \Delta t] \right\} + \mathcal{O}(\Delta t^2). \quad (7.21)$$

Expanding the product and dropping terms of order $(\Delta t)^2$:

$$V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} \Delta t + V(a) + V'(a)(ra - c) \Delta t - \rho V(a) \Delta t \right\} + \mathcal{O}(\Delta t^2). \quad (7.22)$$

Cancel $V(a)$ from both sides, divide by Δt , and let $\Delta t \rightarrow 0$:

$$\boxed{\rho V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + V'(a)(ra - c) \right\}}. \quad (7.23)$$

This is the *Hamilton–Jacobi–Bellman (HJB) equation*. The left-hand side is the “cost of holding wealth a ” (the required return ρV). The right-hand side is the “benefit”: instantaneous utility from consuming c , plus the capital gain $V'(a) \dot{a}$ from the change in wealth.

First-order condition and optimal consumption. The maximization in (7.23) is over c with the objective being concave in c (since $\gamma > 0$). Differentiating the term inside the braces with respect to c and setting to zero:

$$\frac{\partial}{\partial c} \left[\frac{c^{1-\gamma}}{1-\gamma} + V'(a)(ra - c) \right] = c^{-\gamma} - V'(a) = 0 \quad \implies \quad c^*(a) = (V'(a))^{-1/\gamma}. \quad (7.24)$$

The economic content is intuitive: the marginal utility of consumption $c^{-\gamma}$ equals the marginal value of wealth $V'(a)$. A higher $V'(a)$ (wealth is more valuable) implies lower optimal consumption.

Analytical solution. The HJB (7.23) with the FOC (7.24) substituted in becomes a nonlinear ODE in $V(a)$. To solve it, conjecture $V(a) = \Lambda a^{1-\gamma}/(1-\gamma)$ for some constant $\Lambda > 0$. Then $V'(a) = \Lambda a^{-\gamma}$, and from the FOC (7.24):

$$c^* = (\Lambda a^{-\gamma})^{-1/\gamma} = \Lambda^{-1/\gamma} a. \quad (7.25)$$

Substituting into the HJB:

$$\rho \Lambda \frac{a^{1-\gamma}}{1-\gamma} = \frac{(\Lambda^{-1/\gamma} a)^{1-\gamma}}{1-\gamma} + \Lambda a^{-\gamma}(ra - \Lambda^{-1/\gamma} a). \quad (7.26)$$

Dividing through by $a^{1-\gamma}/(1-\gamma)$ and solving for Λ gives $\Lambda = \kappa^{-\gamma}$ with $\kappa = (\rho - (1-\gamma)r)/\gamma$, so $V^*(a) = \kappa^{-\gamma} a^{1-\gamma}/(1-\gamma)$ and the optimal consumption rule is $c^*(a) = \kappa a$. This solution is well-defined provided $\kappa > 0$, i.e., $\rho > (1-\gamma)r$, a standard transversality condition ensuring that the agent discounts future utility sufficiently relative to wealth growth. This closed form is the natural validation target for the PINN in notebook `lecture_11_04_Cake_Eating_HJB_PINN`: the trained network should recover $V(a)$ to mean relative error well below 10^{-3} , and any larger discrepancy signals an under-trained network or a malformed loss.

PINN formulation and PDE residual. To solve the HJB equation (7.23) with a PINN (rather than using the closed-form above), we proceed as follows. A neural network $\hat{V}(a) = \mathcal{N}_\theta(a)$ approximates the value function. Its derivative $\hat{V}'(a)$ is computed by automatic differentiation up to floating-point precision, so no finite differences are needed. From this derivative, we reconstruct the optimal consumption via the FOC (7.24):

$$\hat{c}(a) = (\hat{V}'(a))^{-1/\gamma}. \quad (7.27)$$

Substituting $\hat{V}$, $\hat{V}'$, and $\hat{c}$ into the HJB (7.23) and moving everything to one side defines the *PDE residual*. In practice both the FOC inversion and the residual evaluation use a positivity-guarded derivative $\tilde{V}_a := \text{softplus}(\hat{V}'(a)) + \varepsilon$ rather than the raw $\hat{V}'(a)$, so that consumption is well-defined even where the network’s raw derivative is negative during early training (see also the listing below):

$$\hat{c}(a) = \tilde{V}_a^{-1/\gamma}, \quad \mathcal{R}(a) = \rho \hat{V}(a) - \left[\frac{\hat{c}(a)^{1-\gamma}}{1-\gamma} + \tilde{V}_a(ra - \hat{c}(a)) \right]. \quad (7.28)$$

Once training has converged with $\hat{V}'(a) \gg 0$ on the support, $\text{softplus}(\hat{V}'(a)) \approx \hat{V}'(a)$ to high precision (the exponential tail decays rapidly), so the safeguarded residual is asymptotically equivalent to the original HJB residual; using $\tilde{V}_a$ throughout keeps the FOC inversion and the HJB residual mutually consistent during training, when $\hat{V}'(a)$ may transiently be small or negative. If the network has perfectly learned the true value function, $\mathcal{R}(a) = 0$ for all a . The PINN loss is the mean squared residual over a set of collocation points $\{a_i\}_{i=1}^M$ sampled in the interior of the domain:

$$\ell_{\text{HJB}} = \frac{1}{M} \sum_{i=1}^M \mathcal{R}(a_i)^2. \quad (7.29)$$

The DGM (Deep Galerkin Method) architecture of [Sirignano and Spiliopoulos \(2018\)](#), which provides LSTM-style gating and skip connections from the input to every hidden layer, may be used in place of the plain MLP below to improve expressivity for this type of PDE problem; the listing keeps a plain MLP for clarity. In low dimension the trial-solution MLP of notebook `lecture_11_04_Cake_Eating_HJB_PINN` typically outperforms a soft-BC DGM, because the boundary loss no longer competes with the interior residual; DGM becomes worthwhile primarily in higher dimensions or for PDEs with sharp internal features.

Residual loss alone is not enough

A small HJB residual is necessary but not sufficient for an economically meaningful solution. The same residual minimum can correspond to wildly different value-function levels when the boundary anchor is weak, and to spurious non-monotonic policies when $\hat{V}'(a) \leq 0$ is left unpenalized. Practical safeguards used in notebook `lecture_11_04_Cake_Eating_HJB_PINN`: (i) hard-BC trial solution that fixes both endpoints exactly, removing the boundary–interior trade-off; (ii) a softplus-transformed derivative $\tilde{V}_a = \text{softplus}(\hat{V}_a) + \varepsilon$ inside the FOC inversion, eliminating the negative-derivative pathology in training; (iii) checking the implied consumption policy, the raw derivative $\hat{V}_a$, and the HJB residual against the closed-form benchmark. When the residual, boundary conditions, and policy diagnostics agree, the solution is also economically sensible; when only the HJB residual is small, it usually is not.

Optimizer pipeline and double precision

The PINN notebooks of this chapter use a two-stage pipeline: *Adam* on resampled collocation points to find a basin of attraction, then a *deterministic L-BFGS* polish on a fixed grid in float64. Two practical points are not cosmetic. First, switching collocation points each L-BFGS evaluation breaks the strong Wolfe line search; the polish requires a deterministic objective. Second, second derivatives of a tanh MLP lose substantial precision in float32, which is exactly the scale L-BFGS probes when comparing successive line-search points. FP64 is therefore a stability device, not a guarantee of machine-precision residuals: in a longer `teaching/production` run, the one-dimensional cake HJB reaches final HJB loss about 3×10^{-4} (the checked-in `RUN_MODE="smoke"` notebook stops well short of that), while the heterogeneous-agent examples still require residual, policy, density, and aggregate diagnostics. Recent quantitative discussions of PINN precision and floating-point limits are reviewed in the further-reading list at the end of the chapter.

```

1 def pde_residual(model, a, gamma, rho, r):
2     a.requires_grad_(True)
3     V = model(a)
4     V_a = torch.autograd.grad(V.sum(), a, create_graph=True)[0]
5     safe_Va = F.softplus(V_a) + 1e-6      # positivity guard; +1e-6 avoids
    division by zero in safe_Va.pow(-1/gamma)

```

```

6   c = safe_Va.pow(-1.0 / gamma)          # FOC
7   u_c = c.pow(1 - gamma) / (1 - gamma)
8   R = rho * V - (u_c + safe_Va * (r * a - c))
9   return R

```

Listing 7.1: PINN residual for the HJB equation (PyTorch).

7.7 Continuous-Time Heterogeneous Agent Models

Many frontier models in macroeconomics feature a continuum of agents who face idiosyncratic risk and interact through equilibrium prices. Chapter 6 studied the discrete-time version with Young’s histogram method inside a DEQN. In continuous time the same economic question becomes a *coupled PDE system*: a Hamilton–Jacobi–Bellman equation for individual optimization and a Kolmogorov forward (Fokker–Planck) equation for the stationary cross-sectional density, closed by a market-clearing condition that pins down prices. This is the canonical example of a PINN applied to a *system* of equilibrium PDEs, in contrast to the single HJB of the cake-eating problem (§7.6) and the single Black–Scholes PDE (§7.8): a PINN parameterizes the value function and the density by two networks $\hat{V}_\theta(a, z)$, $\hat{g}_\psi(a, z)$ and minimizes a four-term loss – the HJB residual, the KFE residual, the no-flux boundary residual at the borrowing constraint, and a mass-normalization residual – with the loss-balancing and boundary-layer issues familiar from the rest of this chapter.

The full development – the stochastic-calculus background, the formal HJB and KFE derivations, the Huggett and Aiyagari equilibria, the PINN solver for the stationary coupled system, the master equation that handles aggregate shocks, and the EMINN method – is the subject of Chapter 8: see §8.5 and §8.6 for the stationary problem and §8.7–§8.8 for the aggregate-shock case. Following Achdou et al. (2022) for the model setup (whose own numerical solution uses finite differences), that treatment replaces the traditional fixed-point iteration over r with joint training of all components.

7.8 Application: Black–Scholes PDE

The Black–Scholes PDE has a closed-form solution, so a PINN run on it is not motivated by the absence of an analytical answer. The pedagogical purpose is exactly the opposite: it is a known-answer benchmark. We verify that the same PINN recipe (smooth activations, hard or soft BCs, autodiff for V_S and V_{SS} , Adam-then-L-BFGS) reproduces a textbook formula on a clean domain before applying it to PDEs without closed forms (American options, jump diffusions, multi-asset pricing, HJBs with multiple state variables). If the network cannot recover Black–Scholes to plotting accuracy, no further trust should be placed in its output on harder problems.

The Black–Scholes PDE for a European call option with strike K , maturity T , risk-free rate r , and volatility σ is the canonical option-pricing benchmark of Black and Scholes (1973):

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0, \quad (7.30)$$

with terminal condition $V(S, T) = \max(S - K, 0)$ and boundary conditions $V(0, t) = 0$, $V(S_{\max}, t) = S_{\max} - Ke^{-r(T-t)}$.

The PINN approach approximates $V(S, t) \approx \mathcal{N}_\theta(S, t)$ and computes the partial derivatives V_t , V_S , and V_{SS} via automatic differentiation. The total PINN loss for the Black–Scholes problem

has four terms:

$$\begin{aligned}
\ell = & \underbrace{\frac{1}{N_r} \sum_i \left| V_t(S_i^r, t_i^r) + \frac{1}{2} \sigma^2 (S_i^r)^2 V_{SS}(S_i^r, t_i^r) + r S_i^r V_S(S_i^r, t_i^r) - r V(S_i^r, t_i^r) \right|^2}_{\text{PDE residual}} \\
& + \underbrace{\frac{\lambda_{\text{TC}}}{N_T} \sum_j |V(S_j, T) - \max(S_j - K, 0)|^2}_{\text{terminal condition}} + \underbrace{\frac{\lambda_0}{N_0} \sum_k |V(0, t_k)|^2}_{\text{lower BC}} \\
& + \underbrace{\frac{\lambda_\infty}{N_\infty} \sum_l \left| V(S_{\max}, t_l) - (S_{\max} - K e^{-r(T-t_l)}) \right|^2}_{\text{upper BC}}.
\end{aligned} \tag{7.31}$$

The loss has four terms (PDE residual, terminal condition, lower BC, upper BC) and three relative penalty weights (λ_{TC} , λ_0 , λ_∞); the PDE residual coefficient is normalized to one. These weights must be tuned or adaptively balanced via ReLoBRaLo from Chapter 4. As a practical diagnostic of what goes wrong if the weights are mis-set: underweighting λ_{TC} produces a smooth surface that mis-prices the payoff at maturity (the network ignores the kink at $S = K$); overweighting λ_{TC} pins the network at $t = T$ but the interior PDE residual then fails to balance the V_t term, producing visible drift in the early-time slice. In notebook `lecture_11_05_Black_Scholes_PINN`, the benchmark parameters are $K = 50$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, and $S_{\max} = 100$, for which the analytical at-the-money call value is roughly 4.8. A longer `teaching/production` run reports a max absolute error around 0.13 and a mean absolute error around 0.04 against the analytical Black–Scholes formula (the checked-in `smoke` notebook prints larger errors); at the ATM value scale this is roughly a 2.7% peak and 0.8% average relative error. Accuracy is therefore a diagnostic to check after training, not a guaranteed property of the architecture. The choice of $\tanh$ is natural here: it is C^∞ (so V_{SS} is well-defined everywhere), and its bounded range $(-1, 1)$ provides a stable starting point for learning the option price surface.

7.9 From PINNs to Operator Learning: One Network, Many Problems

This section is script-only outlook material; it has no companion slide and no notebook. It can be skipped on a first read without loss of continuity, and is intended for readers preparing to scale a PINN-based pipeline across many parameter configurations.

A PINN learns *one solution* to one PDE. Each new boundary condition, parameter set, or coefficient field forces a fresh training run. In economic and financial applications this is often exactly the bottleneck: we want option prices for many strike–maturity pairs, value functions for many discount factors, or HJB solutions across an entire parameter sweep. *Operator learning* flips the question: instead of learning a function $u : \mathbb{R}^d \rightarrow \mathbb{R}$ that solves the PDE for a single instance, one learns the *solution operator*

$$\mathcal{G} : (\text{input field, BCs, parameters}) \mapsto \text{solution function } u,$$

i.e. a map between two function spaces.

Two mature architectures dominate the literature.

DeepONet (Lu et al., 2021a). Inspired by the universal-approximation theorem for nonlinear operators, DeepONet uses two sub-networks: a *branch net* encodes the input field at sensor locations into a latent vector, and a *trunk net* encodes the query point at which the output is requested; their inner product is the predicted operator output. The architecture is generic and trains entirely on input–output pairs of an offline solver.

Fourier Neural Operator (FNO) (Li et al., 2021). FNO parameterizes a kernel integral operator by truncating it in Fourier space and applying a learned linear transformation to the low-frequency modes per layer. This captures global interactions cheaply ($\mathcal{O}(n \log n)$ via FFT) and exhibits resolution invariance: a network trained on one grid can be evaluated on a finer grid at test time.

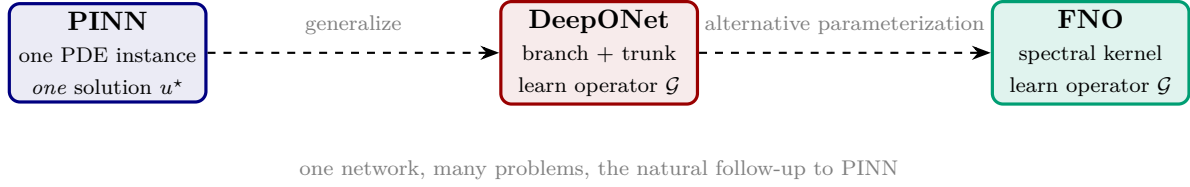


Figure 7.7: Operator learning generalizes PINNs by amortizing over an entire parametric family of PDEs. For economic applications such as option-price surfaces over (K, T) , value functions across a parameter range, or HJB sweeps for sensitivity analysis (Chapter 9), training the operator once gives instant predictions at test time.

Figure 7.7 summarizes this progression from one-instance PINNs to operator-learning architectures. In the rest of this script, we mostly stay with PINNs because the focus is on solving one model carefully; we revisit operator learning briefly in Chapter 12 and point readers who want to amortize across an entire parametric family of PDEs to Lu et al. (2021a); Li et al. (2021).

Chapter Summary

PINNs approximate PDE solutions by minimizing the residual at collocation points, with automatic differentiation supplying the required derivatives algorithmically up to floating-point precision (Raissi et al., 2019). Hard versus soft enforcement of boundary conditions is the central design lever: trial-function constructions of the form $\hat{y} = A(x) + B(x)\mathcal{N}_\theta(x)$ enforce BCs by construction, while soft penalties cover cases where hard enforcement is intractable. For second-order PDEs (HJB, Black–Scholes, Poisson) the activation must be at least C^2 , which excludes ReLU and makes tanh and Swish the standard choices. Operator learning (DeepONet, FNO) is the natural generalization when one wants to amortize across a parametric family rather than solve one instance.

Further Reading

- Raissi et al. (2019), the foundational PINN paper.
- Sirignano and Spiliopoulos (2018), the Deep Galerkin Method, the original deep-PDE solver in finance.
- E et al. (2017); Han et al. (2018), deep BSDE methods, the SDE counterpart.
- Wang et al. (2021); Bischof and Kraus (2025), adaptive loss balancing for PINNs.
- Lu et al. (2021b), the DeepXDE software ecosystem.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. [Core] **Trial-function BC enforcement.** For the BVP $y'' + y = 0$ on $[0, \pi/2]$ with $y(0) = 0$, $y(\pi/2) = 1$, verify that $\hat{y}(x) = 2x/\pi + x(\pi/2 - x)\mathcal{N}_\theta(x)$ satisfies both BCs for any network output. Why is this preferable to a soft penalty?

2. **[Core] ReLU pathology.** Explain why a ReLU network is unsuitable for the strong-form Black–Scholes residual. Then sketch a weak-form formulation that would work with ReLU activations.

3. **[Core] Discrete $\rightarrow$ continuous bridge.** Take the discrete-time Bellman operator

$$V(a) = \max_c [u(c) \Delta t + \beta_{\Delta t} \mathbb{E}[V](a')], \quad a' = a - c \Delta t, \quad \beta_{\Delta t} = e^{-\rho \Delta t}.$$

Show that as $\Delta t \rightarrow 0$ it formally yields the HJB $\rho V = \max_c [u(c) - V'(a) c]$. (This is the pure cake-eating problem with $r = 0$; including a return r on wealth, $\dot{a} = ra - c$, recovers (7.23).)

4. **[Computational] BC penalty weight λ tuning.** In notebook `lecture_11_02_ODE_PINN_SoftVsHardBCs`, run the soft-BC variant with PDE residual ℓ_{int} and BC residual ℓ_{BC} combined as $\mathcal{L} = \ell_{\text{int}} + \lambda \ell_{\text{BC}}$ (the notebook exposes λ as the `bc_weight` hyperparameter dispatched from `RUN_MODE`) for $\lambda \in \{10^{-1}, 10^0, 10^1, 10^2, 10^3, 10^4\}$. For each, train to a fixed budget of 5,000 iterations and record (i) the BC violation $|\hat{y}(0) - y_0|$ at convergence, (ii) the maximum interior residual, and (iii) training wall time. Plot all three on log–log axes against λ , identify the elbow at which boundary fit stops improving cheaply, and compare against the hard-BC variant.
5. **[Advanced/project] Collocation strategy comparison.** In notebook `lecture_11_03_PDE_PINN_Poisson2D` solving $\Delta u = f$ on $[0, 1]^2$ with Dirichlet BCs, replace the default uniform-random collocation with (a) Latin Hypercube sampling, (b) a Sobol low-discrepancy sequence, (c) residual-based adaptive sampling that draws each batch of collocation points proportional to the residual magnitude at the current iterate. Train each variant to the same residual tolerance (10^{-4}) and report (i) the number of collocation points needed, (ii) the wall time, (iii) the spatial distribution of final residuals (compute $\sup_x |\Delta u_\theta - f|$ on a fine 200×200 test grid). Discuss when each strategy is preferred: uniform for smooth problems, low-discrepancy for moderately rough, adaptive for problems with localized features (e.g., near boundary layers).
6. **[Advanced/project] Strong vs. weak forms.** Extend the Black–Scholes notebook. First compare the current strong-form tanh PINN with a ReLU version and explain why the second derivative term is problematic for ReLU. Then derive a weak-form variant: multiply the residual by a smooth test function φ supported inside $[0, S_{\text{max}}]$, integrate by parts to move the second derivative onto φ , and state which derivatives of V remain. If you implement the weak-form ReLU variant, compare its held-out pricing error with the strong-form tanh baseline. Connect the result to Exercise 2.
7. **[Core] Operator learning vs. PINN.** For an option pricing problem where the strike K varies across $\{45, 46, \dots, 55\}$, count the training cost of (a) eleven independent PINN runs vs. (b) one operator-learning or parametric-PINN run that takes K as an input; see Section 7.9 and Chapter 12. At what cost ratio does amortized training win?

Chapter 8

Heterogeneous Agent Models in Continuous Time

Building on the PINN foundations of Chapter 7, this chapter develops the full continuous-time heterogeneous-agent framework: the coupled system of the Hamilton–Jacobi–Bellman equation (for individual optimization) and the Kolmogorov forward equation (for the stationary wealth distribution), closed by a market-clearing condition. Without aggregate risk, this coupled HJB–KFE system is the continuous-time analogue of the Aiyagari economy treated in Chapter 6; once aggregate risk is added, the natural object becomes the *master equation* (§8.7), which is the continuous-time analogue of the Krusell–Smith model. The primary reference is Achdou et al. (2022); for pedagogical background on the continuous-time methods, see also Moll’s lecture notes.¹

In the previous chapter, we applied PINNs to individual PDEs (ODEs, the Poisson equation, the HJB for cake-eating, and the Black–Scholes equation). This chapter makes the leap to *equilibrium systems*: coupled PDEs that arise when a continuum of heterogeneous agents interact through prices. The theoretical framework draws on the Bewley–Huggett–Aiyagari tradition, formulated in continuous time following Achdou et al. (2022). We derive the two core PDEs, the Hamilton–Jacobi–Bellman (HJB) equation for individual optimization and the Kolmogorov forward equation (KFE) for the cross-sectional density, and show how they are coupled through market clearing. The chapter culminates with the *master equation*, a single (infinite-dimensional) PDE that encapsulates the full equilibrium, and with EMINNs, introduced by Gu et al. (2024), which solve it using deep learning.

Companion notebook. One notebook accompanies this chapter and the Lecture 13 numerical deck: `lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`. It first computes the stationary Aiyagari equilibrium with an upwind finite-difference solver, then freezes those equilibrium prices and trains a PINN for the coupled HJB–KFE system at them (with normalization, boundary-flux, state-constraint, and aggregate-capital diagnostics) – so the PINN stage is an equilibrium-consistency and representation exercise, not a price-discovery algorithm. For transparency the notebook specializes the general CRRA/switching equations below to log utility ($\gamma = 1$) and symmetric two-state Poisson switching; the formulas in this chapter are the general case. The detailed upwind finite-difference scheme and PINN losses are developed in the accompanying Lecture 13 deck; this chapter focuses on the continuous-time equilibrium equations and the conceptual bridge to master-equation methods. (A partial-equilibrium HJB on its own is just a single-PDE PINN problem of the kind treated in Chapter 7; here we go straight to the coupled equilibrium system.)

¹<https://benjaminmoll.com/lectures/>

Where does g come from? Before turning to the PDEs themselves, it is worth fixing the economic picture. The model has a continuum of agents, each indexed by an idiosyncratic state (a, z) comprising wealth a and a labor or productivity component z . Each agent solves its own HJB equation taking the prices (r, w) as given, which yields an optimal savings policy that pushes mass through wealth space. The Kolmogorov forward equation tracks how this aggregate mass evolves, and its stationary solution $g^*(a, z)$ is the cross-sectional density that the general-equilibrium clearing equations integrate against to obtain aggregate capital, $K = \int a g^*(a, z) da dz$, and aggregate labor, $L = \int z g^*(a, z) da dz$.

8.1 Why Continuous Time?

Chapters 2–6 formulated heterogeneous-agent models in discrete time. Working in continuous time offers several complementary advantages:

- **Analytical tractability.** FOCs hold with equality in the interior of the state space, and borrowing constraints appear as boundary conditions on the HJB/KFE rather than as occasionally binding inequality constraints inside the recursion; the separation between the backward HJB and the forward KFE is also sharper than in discrete time.
- **Differential operators in place of conditional expectations.** Itô calculus turns conditional expectations into infinitesimal generators, so the HJB and KFE are PDEs rather than integral equations; this removes numerical integration over shock distributions from the inner loop.
- **Powerful numerical methods.** Finite differences, PINNs, and deep learning methods (EMINNs) can be applied directly to the PDE system.
- **Connection to mean field games.** The coupled HJB–KFE system is precisely a *mean field game* (MFG) in the sense of [Lasry and Lions \(2007\)](#): each atomistic agent solves an HJB taking the cross-sectional density as given, while the density itself evolves via a KFE driven by those individual best responses. Equilibrium is the fixed point of this two-way coupling. Recasting the problem in MFG language gives access to a large mathematical literature on existence, uniqueness, and numerical analysis ([Carmona and Delarue, 2018](#); [Cardaliaguet et al., 2019](#)).

Historical context. The models in this chapter build on a long tradition: [Bewley \(1986\)](#) introduced precautionary savings with borrowing constraints; [Huggett \(1993\)](#) studied endowment economies with incomplete markets; [Aiyagari \(1994\)](#) added production and general equilibrium; and [Krusell and Smith \(1998\)](#) incorporated aggregate uncertainty. [Achdou et al. \(2022\)](#) reformulated these models in continuous time and demonstrated that finite-difference methods can solve the coupled HJB–KFE system efficiently. More recently, [Gu et al. \(2024\)](#) introduced EMINNs to solve the master equation globally, enabling treatment of aggregate shocks without moment-based approximations.

8.2 Stochastic Calculus Refresher

We briefly review the stochastic calculus tools needed for continuous-time models; for a standard finance-oriented textbook treatment, see [Shreve \(2004\)](#).

Quick reference. Appendix C contains a one-page summary of the same material (Brownian motion, Itô’s lemma, ergodicity in one paragraph) for readers who want a compact reminder rather than the full exposition below.

8.2.1 Brownian Motion

Standard Brownian Motion

A stochastic process $\{B_t\}_{t \geq 0}$ is a *standard Brownian motion* (Wiener process) if: (i) $B_0 = 0$; (ii) it has independent increments: $B_t - B_s \perp B_s - B_r$ for $r < s < t$; (iii) increments are Gaussian: $B_t - B_s \sim \mathcal{N}(0, t - s)$; and (iv) paths $t \mapsto B_t$ are continuous almost surely.

Key properties include $\mathbb{E}[B_t] = 0$, $\text{Var}(B_t) = t$, nowhere-differentiable paths, and quadratic variation $\langle B \rangle_t = t$. Brownian motion arises as the scaling limit of a random walk: if $X_{t+\Delta t} = X_t + \sqrt{\Delta t} \varepsilon_t$ with $\varepsilon_t \in \{-1, +1\}$ equiprobably, then $X^{\Delta t} \xrightarrow{d} B_t$ as $\Delta t \rightarrow 0$ (Donsker's theorem). The $\sqrt{\Delta t}$ scaling ensures that $\text{Var}(X_t) = t$ in the limit. Figure 8.1 shows three discretized sample paths with the same variance scaling.

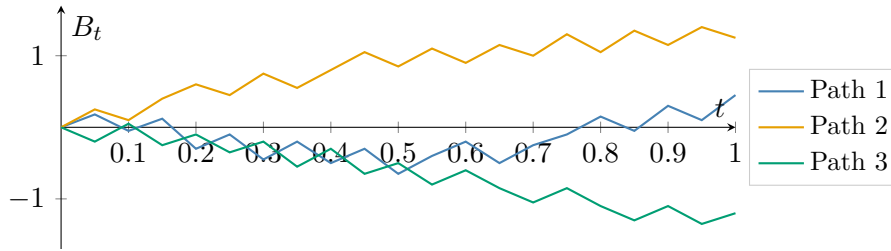


Figure 8.1: Three simulated standard Brownian sample paths $\{B_t\}_{t \in [0,1]}$, generated with discretization step $\Delta t = 0.05$ and Gaussian increments $B_{t+\Delta t} = B_t + \sqrt{\Delta t} \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, 1)$. Paths are jagged at the chosen Δt and the polylines linearly interpolate between the sample points, so the figure is an approximate visualization rather than a true Brownian path: limiting Brownian paths are continuous almost surely but nowhere differentiable, with $\text{Var}(B_t) = t$, and therefore cannot be drawn exactly at any finite resolution.

8.2.2 Itô Processes and SDEs

An *Itô process* X_t satisfies the stochastic differential equation (SDE):

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dB_t, \quad (8.1)$$

where $\mu(\cdot)$ is the *drift* (deterministic trend) and $\sigma(\cdot)$ is the *diffusion coefficient* (volatility). Readers unfamiliar with Itô calculus will benefit from [Shreve \(2004\)](#) as a prerequisite; the key fact $(dB_t)^2 = dt$ is what forces the second-order term in Itô's lemma below. Two important special cases recur throughout this chapter:

- **Geometric Brownian motion (GBM):** $dS_t = \mu S_t dt + \sigma S_t dB_t$ (stock prices, GDP).
- **Ornstein–Uhlenbeck (OU) process:** $dZ_t = \eta(\bar{Z} - Z_t) dt + \sigma dB_t$ (productivity, interest rates). The OU process is mean-reverting with stationary distribution $Z_\infty \sim \mathcal{N}(\bar{Z}, \sigma^2/(2\eta))$. It will model aggregate TFP in the Krusell–Smith economy below.

For discrete income switching, we use a *continuous-time Markov chain*: a labor state $n_t \in \{n_1, n_2\}$ that switches with Poisson intensities λ_1 (from n_1 to n_2) and λ_2 (from n_2 to n_1), yielding ergodic probabilities $\pi_1 = \lambda_2/(\lambda_1 + \lambda_2)$ and $\pi_2 = \lambda_1/(\lambda_1 + \lambda_2)$.

8.2.3 Itô's Lemma

Itô's Lemma

Let X_t follow $dX_t = \mu dt + \sigma dB_t$ and let $f \in C^2$. Then:

$$df(X_t) = \left[f'(X_t) \mu + \frac{1}{2} f''(X_t) \sigma^2 \right] dt + f'(X_t) \sigma dB_t. \quad (8.2)$$

The key difference from ordinary calculus is the second-order correction $\frac{1}{2} f'' \sigma^2 dt$, which arises because $(dB_t)^2 = dt \neq 0$. The differential algebra rules are: $dt \cdot dt = 0$, $dt \cdot dB_t = 0$, $dB_t \cdot dB_t = dt$.

Worked example. For GBM $dX_t = \mu X_t dt + \sigma X_t dB_t$, applying Itô's lemma to $f(x) = \ln x$ gives $d(\ln X_t) = (\mu - \frac{1}{2} \sigma^2) dt + \sigma dB_t$, so $X_t = X_0 \exp[(\mu - \frac{1}{2} \sigma^2)t + \sigma B_t]$. The $-\frac{1}{2} \sigma^2$ Itô correction explains why the expected log return differs from the expected return for volatile assets.

Time-dependent version. For $V(X_t, t)$ where $dX_t = \mu dt + \sigma dB_t$:

$$dV = \left[\partial_t V + \mu \partial_x V + \frac{1}{2} \sigma^2 \partial_{xx} V \right] dt + \sigma \partial_x V dB_t. \quad (8.3)$$

With Poisson jumps of intensity λ and jump size ΔX , an additional term $[V(X_{t-} + \Delta X, t) - V(X_{t-}, t)] dN_t$ appears.

8.3 The Kolmogorov Forward Equation

The *Kolmogorov forward equation* (KFE), also known as the *Fokker–Planck equation*, describes how the probability density of a stochastic process evolves over time. It was introduced by Kolmogorov (1931) and, independently in the physics literature, by Fokker (1914) and Planck (1917) to describe diffusion of particles. In stationary equilibrium, $\partial_t g = 0$ and the KFE reduces to an elliptic PDE for the cross-sectional density (used throughout §8.5–§8.6); when aggregate shocks make prices time-varying, the parabolic time-dependent form returns and motivates the master-equation reformulation of §8.7.

8.3.1 Derivation from First Principles

Consider a population of particles, each independently following $dX_t = \mu dt + \sigma dB_t$ with constant coefficients. If the initial density is $g_0(x)$, what is the density $g(x, t)$ at time $t > 0$?

The derivation proceeds in four steps. (i) For any smooth test function $\varphi(x)$, $\mathbb{E}[\varphi(X_t)] = \int \varphi(x) g(x, t) dx$. (ii) Differentiate with respect to t and apply Itô's lemma: $\frac{d}{dt} \int \varphi g dx = \int [\mu \varphi' + \frac{\sigma^2}{2} \varphi''] g dx$. (iii) Integrate by parts. Take φ to be *compactly supported*: this kills the boundary terms cleanly and costs nothing here, since the identity must hold for every such φ . We obtain $\int \mu \varphi' g dx = - \int \varphi \mu \partial_x g dx$ and $\int \frac{\sigma^2}{2} \varphi'' g dx = \int \varphi \frac{\sigma^2}{2} \partial_{xx} g dx$. (The natural-looking assumption $g \rightarrow 0$ at $\pm\infty$ is automatic for any continuous density, but it is not by itself enough: what is needed is that the product $g \varphi$ vanishes at infinity, which compact support of φ delivers for free. Spatial boundaries, e.g. a borrowing constraint, require separate treatment in the next subsection.) (iv) Since the identity holds for all test functions φ , we obtain:

$$\frac{\partial g}{\partial t}(x, t) = -\mu \frac{\partial g}{\partial x}(x, t) + \frac{\sigma^2}{2} \frac{\partial^2 g}{\partial x^2}(x, t). \quad (8.4)$$

The two terms on the right encode competing effects: $-\mu \partial_x g$ is *advection* (drift transports the density), and $\frac{\sigma^2}{2} \partial_{xx} g$ is *diffusion* (noise spreads the density).

General form. For state-dependent coefficients $dX_t = \mu(X_t) dt + \sigma(X_t) dB_t$, the KFE in *divergence form* is:

$$\partial_t g = -\partial_x [\mu(x) g(x, t)] + \frac{1}{2} \partial_{xx} [\sigma(x)^2 g(x, t)] = -\partial_x J(x, t), \quad (8.5)$$

where $J = \mu g - \frac{1}{2} \partial_x (\sigma^2 g)$ is the probability flux. The identity $\partial_t g + \partial_x J = 0$ is a conservation law: probability is neither created nor destroyed. When σ depends on x , the diffusion coefficient must be written *inside* the second derivative; in particular, $\frac{1}{2} \partial_{xx} [\sigma(x)^2 g] \neq \frac{\sigma(x)^2}{2} \partial_{xx} g$, with the two expressions differing by $(\sigma^2)' \partial_x g + \frac{1}{2} (\sigma^2)'' g$. This distinction is invisible in the constant-coefficient case but is what keeps the operator self-adjoint and probability-conserving when σ varies.

8.3.2 Examples

Heat equation. With $\mu = 0$ and constant σ , the KFE reduces to $\partial_t g = \frac{\sigma^2}{2} \partial_{xx} g$, the classical heat equation. Starting from a point mass $g(x, 0) = \delta(x - x_0)$, the solution is a Gaussian with variance growing linearly: $g(x, t) = (2\pi\sigma^2 t)^{-1/2} \exp[-(x - x_0)^2 / (2\sigma^2 t)]$.

Ornstein–Uhlenbeck process. For $dX_t = \eta(\bar{x} - X_t) dt + \sigma dB_t$, the KFE is $\partial_t g = \eta \partial_x [(x - \bar{x}) g] + \frac{\sigma^2}{2} \partial_{xx} g$. Setting $\partial_t g = 0$ and trying a Gaussian ansatz $g^*(x) = (2\pi s^2)^{-1/2} \exp[-(x - \bar{x})^2 / (2s^2)]$, the advection and diffusion terms balance when $s^2 = \sigma^2 / (2\eta)$, giving the stationary distribution $g^*(x) = \mathcal{N}(\bar{x}, \sigma^2 / (2\eta))$: mean reversion concentrates mass around $\bar{x}$, while diffusion spreads it, and the balance produces a Gaussian steady state.

Reading the SDE qualitatively. It is worth pausing to read the drift sign by eye, because students are often trained to compute with differential equations but rarely to look at them. When $X_t < \bar{x}$, the drift $\eta(\bar{x} - X_t)$ is positive and pushes X_t upward; when $X_t > \bar{x}$, the drift is negative and pushes X_t downward, hence the name *mean-reverting*. Flipping the sign ($\eta < 0$) gives a *mean-repelling*, unstable process that drifts away from $\bar{x}$ without bound. Closely related cubic SDEs make the same point sharply. The process $dX_t = -X_t^3 dt + \sigma dB_t$ uses the cubic well as a restoring force and is well-posed with a stationary distribution, while $dX_t = +X_t^3 dt + \sigma dB_t$ blows up in finite time, and a naive Euler discretization produces NaNs almost immediately. Throughout this chapter the same qualitative reading is what carries intuition over from SDEs to KFEs and HJBs.

8.3.3 From Physics to Economics: A Continuum of Agents

Consider a continuum of agents $i \in [0, 1]$, each independently following $dX_t^i = \mu(X_t^i) dt + \sigma(X_t^i) dB_t^i$ with independent idiosyncratic Brownian motions B_t^i . By the law of large numbers at the population level, the cross-sectional density $g(x, t)$ evolves *deterministically*, even though each individual path is random. The density satisfies the KFE (8.5).

Physical analogy

Each molecule of air moves randomly (Brownian motion), but the temperature distribution in a room evolves deterministically (heat equation). In economics: each household's income is random, but the wealth distribution evolves deterministically (KFE).

KFE with income switching. When agent i has wealth a_t^i and income state $n_t^i \in \{n_1, n_2\}$ switching with Poisson intensities, and wealth evolves as $da_t^i = s(a_t^i, n_t^i) dt$ with $a_t^i \geq \underline{a}$, the cross-sectional density $g_t(a, n)$ satisfies:

$$\partial_t g_t(a, n) = -\partial_a [s(a, n) g_t(a, n)] - \lambda(n) g_t(a, n) + \lambda(\hat{n}) g_t(a, \hat{n}), \quad (8.6)$$

where $\hat{n}$ denotes the complementary income state. The three terms represent: flow of agents along the wealth axis (savings), agents leaving state n , and agents entering state n from $\hat{n}$.

Boundaries and mass points. At the borrowing constraint $a = \underline{a}$, the no-flux boundary condition $s(\underline{a}, n) g_t(\underline{a}, n) = 0$ prevents the absolutely continuous density from flowing below $\underline{a}$. If households are constrained at this boundary, a boundary atom may be needed in addition to the interior density; finite-difference implementations represent that atom as mass in the first grid cell. This is the continuous-time counterpart of the characteristic left spike observed in empirical wealth distributions.

When does the KFE become stochastic? With purely idiosyncratic shocks (Aiyagari), the KFE is deterministic. When *aggregate shocks* are present (Krusell–Smith), prices r_t, w_t depend on the aggregate state Z_t , making the drift stochastic and the density g_t a stochastic process adapted to the filtration $\mathcal{F}_t^0$ generated by the aggregate Brownian motion. This is the “master equation” setting discussed in Section 8.7.

KFE validation: more than the pointwise PDE residual

A small pointwise KFE residual $|R_{\text{KFE}}|$ is a necessary check, but pointwise differentiation of a noisy density g_ϕ is fragile and the residual can sit near zero while the implied density is visibly wrong. Notebook 08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch therefore monitors a small panel of complementary diagnostics: (i) total mass $\sum_n \int g_\phi da \rightarrow 1$, (ii) no-flux boundary $J(\underline{a}, n), J(\bar{a}, n) \rightarrow 0$, (iii) aggregate-capital error $|K^{\text{PINN}} - K^{\text{FD}}|$, (iv) CDF distance (Kolmogorov–Smirnov) per income state, and (v) an integrated flux-balance residual that replaces ∂_a with quadrature. In practice, mass and aggregate K converge first; the local KFE residual and the KS distance close last. Reporting all five together makes PINN quality transparent and avoids reading a small pointwise residual as a guarantee of a small density error.

8.4 The Hamilton–Jacobi–Bellman Equation

Individual optimality in continuous-time heterogeneous-agent models is characterized by the HJB equation. This section gives the full five-step derivation from Itô’s lemma; readers who saw the motivating overview of the continuous-time heterogeneous-agent setting in §7.7 can treat the present section as its formal counterpart.

The agent’s problem. Agent i with state $x_t^i = (a_t^i, n_t^i)$ chooses consumption c_t^i to maximize:

$$\max_{\{c_t^i\}} \mathbb{E} \left[\int_0^\infty e^{-\rho t} u(c_t^i) dt \right] \quad (8.7)$$

subject to $da_t^i = (wn_t^i + ra_t^i - c_t^i) dt$ and $a_t^i \geq \underline{a}$, where $u(c) = c^{1-\gamma}/(1-\gamma)$ is CRRA utility, $\rho > 0$ is the discount rate, and (r, w) are factor prices. The standard transversality condition $\lim_{t \rightarrow \infty} e^{-\rho t} V(a_t^i, n_t^i) = 0$ along optimal paths, the continuous-time no-Ponzi requirement, is imposed throughout.

The value function. The value function $V(a, n)$ records the maximum expected discounted utility starting from state (a, n) . It is increasing and concave in a , and $V(a, n_2) > V(a, n_1)$ when $n_2 > n_1$.

Deriving the HJB. The derivation follows five steps; each unpacks one ingredient of equation (8.8) below.

Step (i): Dynamic programming principle. For small $h > 0$,

$$V(a, n) = \max_c \left\{ \int_0^h e^{-\rho t} u(c_t) dt + e^{-\rho h} \mathbb{E}[V(a_h, n_h)] \right\}.$$

This is Bellman's principle of optimality applied to a continuous-time problem.

Step (ii): Itô's lemma between income jumps. Conditional on the income state n_t not changing on $[0, h]$, wealth follows the *deterministic* ODE $da_t = s(c_t, a_t, n_t) dt$ (there is no Brownian forcing on wealth in this model), so

$$dV = V'(a, n) s(c, a, n) dt.$$

The second-order Itô correction $\frac{1}{2} V'' \sigma^2 dt$ vanishes because the wealth diffusion is zero between jumps; this is the most subtle step and is what distinguishes the income-switching HJB from a standard diffusion HJB.

Step (iii): Account for Poisson jumps in expectation. Adding the Poisson-jump contribution and taking expectations,

$$\mathbb{E}[dV] = \left[V'(a, n) s + \lambda(n)(V(a, \hat{n}) - V(a, n)) \right] dt,$$

where $\lambda(n)$ is the intensity of switching out of state n and $\hat{n}$ is the complementary state.

Step (iv): Substitute into the DPP and let $h \rightarrow 0$. Plugging the expectation back into the Bellman expression, dividing by h , and taking $h \rightarrow 0$ yields a flow equation in which the discount ρV on the left balances the flow utility plus expected change on the right.

Step (v): Optimize over c . Imposing the first-order condition over consumption gives the HJB equation:

$$\boxed{\rho V(a, n) = \max_c \{ u(c) + V'(a, n) \cdot (wn + ra - c) + \lambda(n)(V(a, \hat{n}) - V(a, n)) \}}. \quad (8.8)$$

Interpretation. The HJB is an *asset pricing equation*: the left side ρV is the “required return” on the value function (discount rate times “asset value”), and the right side is the “total return” consisting of the flow dividend $u(c)$, the capital gain $V' \cdot s$ from saving, and the expected gain/loss $\lambda(n)(\Delta V)$ from income switching.

Optimal policy. The first-order condition $u'(c^*) = V'(a, n)$ gives the consumption function:

$$c^*(a, n) = [V'(a, n)]^{-1/\gamma}. \quad (8.9)$$

Substituting back eliminates the maximization, yielding a nonlinear PDE in V . The savings function is $s(a, n) = wn + ra - c^*(a, n)$.

Boundary conditions. At the borrowing constraint $a = \underline{a}$, consumption must keep the drift feasible: $s(\underline{a}, n) = wn + r\underline{a} - c \geq 0$, or equivalently $c \leq wn + r\underline{a}$. The boundary HJB is therefore the constrained maximization

$$\begin{aligned} \rho V(\underline{a}, n) = \max_{0 < c \leq wn + r\underline{a}} & \left\{ u(c) + V_a(\underline{a}, n)(wn + r\underline{a} - c) \right. \\ & \left. + \lambda(n)(V(\underline{a}, \hat{n}) - V(\underline{a}, n)) \right\}. \end{aligned}$$

For CRRA utility this gives the boundary policy $c^*(\underline{a}, n) = \min\{[V_a(\underline{a}, n)]^{-1/\gamma}, wn + r\underline{a}\}$ and $s(\underline{a}, n) \geq 0$. This is the state-constraint form of the borrowing limit; numerical solvers usually impose it with one-sided derivatives, a constrained policy rule, or a penalty on negative boundary drift.

Boundary atoms in the stationary distribution. When the borrowing constraint binds on a positive mass of agents, the stationary measure is not absolutely continuous with respect to Lebesgue measure: it carries a Dirac atom at $a = \underline{a}$. The decomposition is $g(a, n) = g_{ac}(a, n) + \alpha(n) \delta(a - \underline{a})$, where g_{ac} is the absolutely-continuous interior density and $\alpha(n) \geq 0$ is the constrained mass for income state n . In the sense of distributions (acting on smooth test functions against which the atom is integrated), equation (8.6) remains valid for the full measure g . The explicit split into a PDE for the interior part g_{ac} and a separate flux-balance equation for $\alpha(n)$ (equating inflows from the no-flux boundary condition with the income-driven outflow back into the interior) is the *numerical* device used by finite-difference and PINN implementations, which cannot resolve a delta on a regular grid. Finite-difference implementations typically represent $\alpha(n)$ as the mass in the first grid cell, and PINN implementations either absorb the atom implicitly into a smooth density approximation (with corresponding accuracy loss near $\underline{a}$) or explicitly parameterize $\alpha(n)$ alongside the interior network.

Variant: continuous (diffusion) income. A natural variant replaces the two-state Poisson process with a continuously distributed earnings state z_t following an Ornstein–Uhlenbeck diffusion, $dz_t = \eta(\bar{z} - z_t) dt + \sigma dB_t^z$ (with idiosyncratic Brownian motion B_t^z). The agent’s state is then (a, z) , the value function $V(a, z)$ is smooth in both arguments, and Itô’s lemma along the z -diffusion produces a genuine second-order term. After substituting the first-order condition $c^* = (V_a)^{-1/\gamma}$ the HJB becomes the elliptic PDE

$$\rho V(a, z) = u(c^*) + V_a(wz + ra - c^*) + \eta(\bar{z} - z) V_z + \frac{1}{2} \sigma^2 V_{zz}, \quad (8.10)$$

with the borrowing-constraint state condition $s(\underline{a}, z) \geq 0$ at $a = \underline{a}$, a Neumann (FOC) condition $V_a(\bar{a}, z) = u'(wz + r\bar{a})$ on the truncated upper wealth boundary, and reflecting conditions $V_z = 0$ at the z -boundaries. This is the smallest model in the chapter that carries the diffusion second-order term $\frac{1}{2} \sigma^2 V_{zz}$ in the *individual* HJB: the income-switching HJB (8.8) has none, because wealth carries no Brownian forcing between jumps, while the master equation (8.12) carries $\frac{1}{2} (\sigma^z)^2 V_{zz}$ only for the *aggregate* TFP state z .

8.5 Competitive Equilibrium: Huggett and Aiyagari

The HJB gives individual optimal behavior for given prices; the KFE tracks the resulting distribution. Market clearing pins down prices, closing the system. Figure 8.2 summarizes this fixed-point structure.

8.5.1 The Coupled HJB–KFE System

The solution method for the stationary equilibrium (no aggregate shocks) iterates: guess $r \rightarrow$ solve HJB for $V, c^* \rightarrow$ solve KFE for $g^* \rightarrow$ compute $K = \sum_n \int a g^* da \rightarrow$ update r from the firm FOC. Under standard regularity conditions on preferences, technology, and the income process, aggregate capital supply $K^s(r)$ is monotone decreasing in r over the relevant range, which makes the bisection (or fixed-point iteration) on r well-posed; see Achdou et al. (2022, §2) for the full statement.

8.5.2 Huggett (1993): Endowment Economy

Huggett’s model is an endowment economy with idiosyncratic income $y_t \in \{y_1, y_2\}$, a single risk-free bond $b_t \geq \underline{b}$ paying instantaneous return r , and bonds in zero net supply. The HJB is $\rho V = \max_c \{u(c) + V_b(rb + y - c) + \lambda(y)(\Delta V)\}$, and the KFE determines the stationary density $g^*(b, y)$. An equilibrium return r^* is a value satisfying the asset-market-clearing condition $\sum_y \int b g^*(b, y) db = 0$; uniqueness requires standard monotonicity assumptions on aggregate bond demand (Achdou et al., 2022).

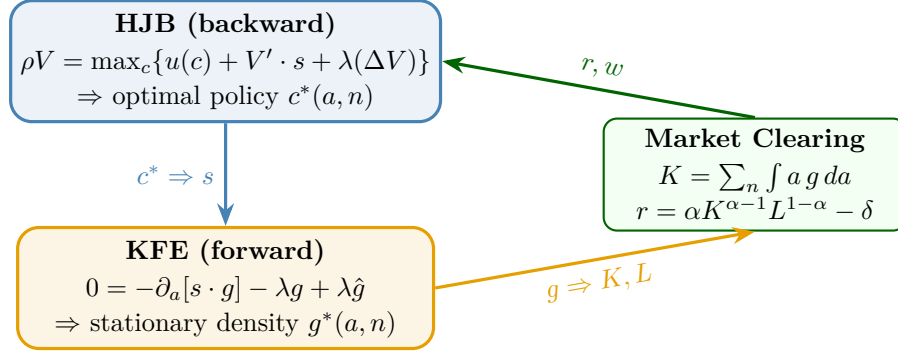


Figure 8.2: Stationary continuous-time heterogeneous-agent equilibrium as a coupled HJB–KFE–market-clearing loop. Given prices, the HJB determines optimal savings; the KFE maps that policy into a stationary density; aggregating the density updates capital, labor, and therefore prices.

The mechanism is that the risk-free rate is pinned down by *precautionary self-insurance demand*, not by a production FOC: agents desire to hold positive bond positions for precautionary reasons, and the return must adjust downward until bond demand equals zero net supply.

8.5.3 Aiyagari (1994): Production Economy

Aiyagari’s model adds a representative firm with Cobb–Douglas production $Y = K^\alpha L^{1-\alpha}$. The household asset is now a claim on aggregate capital, and the firm’s FOCs yield $r = \alpha K^{\alpha-1} L^{1-\alpha} - \delta$ and $w = (1 - \alpha) K^\alpha L^{-\alpha}$. The equilibrium condition is $K^s(r) = K^d(r)$, where $K^s(r) = \sum_z \int a g^*(a, z) da$ is capital supplied by households (via HJB + KFE) and $K^d(r)$ is capital demanded by the firm (inverse of the firm FOC). Table 8.1 highlights the key distinction between the endowment and production versions.

What changes from Huggett to Aiyagari? Both models share the same household problem (HJB) and the same cross-sectional law of motion (KFE). What differs is the equilibrium concept. In Huggett the equilibrium is the single price r^* that clears a zero-net-supply bond market, $\sum_y \int b g^*(b, y) db = 0$; the rate is pinned down by precautionary self-insurance demand alone, with no production side. In Aiyagari the equilibrium is a fixed point in r that matches household supply $K^s(r)$ to firm demand $K^d(r) = (\alpha/(r + \delta))^{1/(1-\alpha)} L$, with prices determined jointly by household savings and the firm FOCs. Numerically, both reduce to a one-dimensional root-finding problem in r , but the economic mechanism (precautionary saving vs. marginal-product-of-capital pinning) is qualitatively different.

Adding aggregate TFP. When aggregate TFP Z_t is allowed to vary (e.g., the OU process introduced in §8.7 below for Krusell–Smith), the firm FOCs generalize to $r_t = \alpha e^{Z_t} K_t^{\alpha-1} L_t^{1-\alpha} - \delta$ and $w_t = (1 - \alpha) e^{Z_t} K_t^\alpha L_t^{-\alpha}$. Aiyagari is recovered when $Z_t \equiv 0$; the same expression covers both calibrations, which is convenient for the master-equation analysis below.

Figure 8.3 contrasts the two stationary densities visually. In Huggett (left), bonds are in zero net supply, so the cross-sectional density is centred near $b=0$, with a Dirac atom at the borrowing limit $\underline{b}$ carried entirely by constrained low-productivity households; high-productivity households are smoothly distributed and never bind. In Aiyagari (right), agents hold positive capital in equilibrium, so the same atom now sits at $\underline{a}=0$ but the bulk of the mass is shifted right with a long upper tail.

Connection to HANK. These models are the building blocks of Heterogeneous Agent New Keynesian (HANK) models (Kaplan et al., 2018), which replace the representative agent in New

	Huggett (1993)	Aiyagari (1994)
Economy	Endowment	Production ($Y = K^\alpha L^{1-\alpha}$)
Asset	Bond b	Capital claim a
Net supply	Zero ($\int b g = 0$)	Positive ($\int a g = K > 0$)
Price determined by	Bond return q	Interest rate r , wage w
Wealth distribution	Centered around 0, mass at $\underline{b}$	Right-skewed, long right tail

Table 8.1: Huggett and Aiyagari as two continuous-time incomplete-markets benchmarks. Huggett clears a zero-net-supply bond market by adjusting the bond return; Aiyagari clears a positive capital market with prices pinned down by firm first-order conditions.

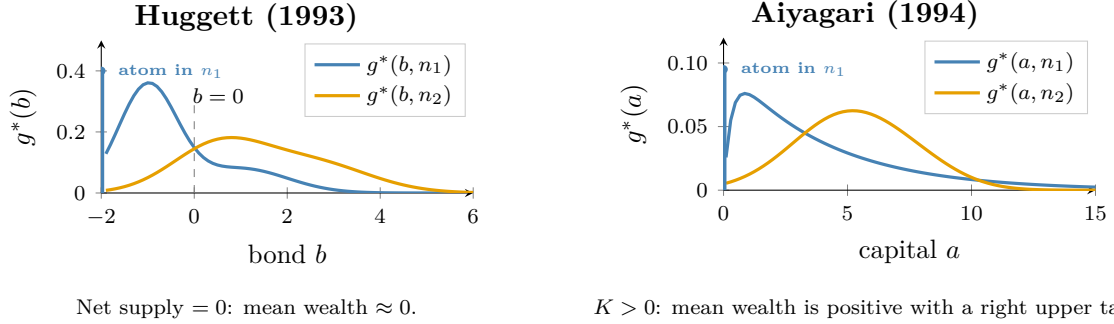


Figure 8.3: Stationary cross-sectional densities g^* in the two benchmarks, by productivity type $n \in \{n_1, n_2\}$ (low and high). In both economies, only the constrained low-productivity type n_1 supports a Dirac atom at the borrowing constraint (blue spike): high-productivity households are not bound. *Left*: Huggett, bonds with limit $\underline{b} = -2$ and zero net supply, so the bulk of mass sits around $b = 0$. *Right*: Aiyagari, capital with limit $\underline{a} = 0$ and positive aggregate K , shifting the unconstrained mass to the right with a long upper tail. The blue spikes visualize the Dirac atom $\alpha(n_1) \delta(a - \underline{a})$ in the decomposition $g = g_{ac} + \alpha(n) \delta(a - \underline{a})$ introduced in the boundary-atom paragraph above. These curves are schematic TikZ illustrations of the qualitative contrast (zero-net-supply bonds versus positive capital), not direct exports; the exact densities depend on calibration and boundary treatment.

Keynesian frameworks with an Aiyagari economy. Adding nominal rigidities allows monetary policy to affect consumption *heterogeneously*: agents near the borrowing constraint have high marginal propensities to consume and respond strongly to fiscal stimulus, while wealthy agents absorb shocks through savings.

8.6 A PINN Solver for the Stationary Huggett–Aiyagari System

The stationary equilibrium of §8.5 couples five conditions: the HJB equation for the value function $V(a, z)$, the consumption first-order condition $c^*(a, z) = (V'(a, z))^{-1/\gamma}$, the KFE for the stationary density $g(a, z)$ with savings drift $s(a, z) = wz + ra - c^*(a, z)$, the no-flux boundary condition $s(\underline{a}, z) g(\underline{a}, z) = 0$ at the borrowing constraint $\underline{a}$, mass normalization $\sum_z \int g(a, z) da = 1$, and the market-clearing condition that pins down prices. The traditional solver iterates a fixed point over r (guess $r \rightarrow$ solve HJB for $V, c^* \rightarrow$ solve KFE for $g \rightarrow$ aggregate capital $\rightarrow$ update r from the firm FOC); a PINN instead trains all of them jointly.

A PINN implementation uses two networks, trained jointly:

- $\hat{V}_\theta(a, z)$: approximates the value function. Its derivative $\hat{V}_a$ is computed via automatic differentiation, and the implied consumption policy is $\hat{c}^*(a, z) = (\hat{V}_a(a, z))^{-1/\gamma}$.
- $\hat{g}_\psi(a, z)$: approximates the stationary density, with a positivity transform (e.g., softplus or

exp) ensuring $\hat{g} > 0$.

The joint loss has four components:

$$\ell = w_{\text{HJB}} \ell_{\text{HJB}} + w_{\text{KFE}} \ell_{\text{KFE}} + w_{\text{flux}} \ell_{\text{flux}} + w_{\text{mass}} \left(\sum_z \int \hat{g}_\psi(a, z) da - 1 \right)^2, \quad (8.11)$$

where ℓ_{HJB} and ℓ_{KFE} are mean squared PDE residuals computed on collocation points, ℓ_{flux} enforces the no-flux boundary conditions $s(\underline{a}, z) \hat{g}_\psi(\underline{a}, z) = 0$, and the mass term enforces normalization. The integral is evaluated numerically (quadrature or Monte Carlo on the collocation points). The aggregate-flux identity $\sum_z s(a, z) g(a, z) = 0$ at each interior a – which follows from the no-flux boundary and total-mass conservation within each a -slice – is a free consistency check at solution time and is sometimes added as an auxiliary loss term.

Balancing the four loss components is critical: the HJB and KFE residuals can differ by orders of magnitude, so adaptive loss balancing (ReLoBRaLo, Chapter 4) is strongly recommended. Practical considerations include using separate learning-rate schedules for the two networks, concentrating collocation points near the borrowing constraint where the density can become sharply peaked, and verifying that the consumption policy $\hat{c}^*$ satisfies $\hat{c}^* > 0$ everywhere. If the true stationary distribution contains an atom at the borrowing constraint (as in both Huggett and Aiyagari, Figure 8.3), a continuous density network alone cannot represent it; one must add a separate boundary-mass variable or use a discretization that permits a point mass.

Why a Fourier basis is a poor test-function set

A natural-looking alternative to pointwise (collocation) residuals is to project the KFE residual onto a set of test functions $\{\psi_n\}$ and minimize the resulting projection coefficients. The pitfall is that for the obvious choice of a Fourier basis, $\psi_n(a) = e^{i2\pi na/L}$, the coefficients c_n of any smooth g decay to zero by the Riemann–Lebesgue lemma; the smoother g is, and the more it vanishes near the boundary, the faster the decay. In practice, beyond $n \approx 5$ the c_n are essentially zero *regardless* of how well the residual is solved: they no longer measure error, they only measure smoothness. Hat functions, low-order polynomials on local patches, or learned neural test functions (the WAN, or weak-adversarial-networks, idea) probe the residual far more honestly.

This continuous-time PINN approach and the discrete-time DEQN with Young’s method (Chapter 6) address the *same* economic question – how heterogeneous agents interact through prices when markets are incomplete – but differ in the mathematical formulation summarized in Table 8.2.

	Discrete time (Ch. 6)	Continuous time (this chapter)
Distribution	Histogram $G_t(k_i, \varepsilon_j)$	Density $g(a, z)$
Evolution	Young’s redistribution	KFE PDE
Individual opt.	Euler equation	HJB PDE
Solver	DEQN (TensorFlow)	PINN (PyTorch)
Key reference	Krusell and Smith (1998)	Achdou et al. (2022)

Table 8.2: Discrete- and continuous-time formulations of incomplete-markets heterogeneous-agent models. The economic object is the same in both columns, but the numerical residual changes from a discrete-time law of motion plus Euler equation to a coupled HJB–KFE PDE system.

Both approaches can incorporate aggregate shocks, multiple assets, and general equilibrium; the choice between them depends primarily on whether the underlying economic model is formulated in discrete or continuous time. For the aggregate-shock case the natural continuous-time object is the *master equation* (§8.7), solved with EMINNs (§8.8).

8.7 The Master Equation

When aggregate TFP Z_t follows an OU process $dZ_t = \eta(\bar{Z} - Z_t) dt + \sigma^z dB_t^0$ (with mean-reversion speed $\eta > 0$; not the OLG TFP shock or the network learning rate of earlier chapters), the value function depends on the *entire wealth distribution*: $V = V(a, n, z, g)$. The HJB becomes a PDE with a functional argument, and the “curse of infinite dimensionality” strikes: the functional derivative $\delta V / \delta g$ makes the problem intractable by standard methods. The master equation approach, which originated in the mean field games literature (Lasry and Lions, 2007) and is developed systematically in the monograph of Cardaliaguet et al. (2019), reformulates this coupled HJB–KFE system as a single PDE on the extended state space (a, n, z, g) that retains the distribution argument explicitly but is amenable to neural network approximation.

Why collapse the coupled system? Without aggregate shocks, solving the stationary equilibrium as a fixed point in r over the coupled HJB–KFE system is straightforward (§8.5). With aggregate shocks, however, every realization of Z_t would in principle require its own coupled solve, and no parametric guess for $V(a, n, z, g)$ can be informed by the cross-section unless the cross-section is treated as an explicit argument. The master equation reformulation lifts g into the state space so a *single* PDE in (a, n, z, g) encodes the entire economy, which makes a global neural-network ansatz feasible (Section 8.8). The price of this convenience is the appearance of the functional-derivative term $\delta V / \delta g$, which the rest of this section unpacks.

The master equation. The key idea is to substitute the KFE, market clearing, and belief consistency *directly into* the HJB, collapsing the coupled system into a single PDE:

$$\begin{aligned} 0 = & -\rho V(a, n, z, g) + u(c^*) + V_a [w(z, g)n + r(z, g)a - c^*] \\ & + \lambda(n)(V(a, \hat{n}, z, g) - V(a, n, z, g)) + V_z \mu^z(z) + \frac{1}{2}(\sigma^z)^2 V_{zz} \\ & + \sum_j \int_{\underline{a}}^{\infty} \frac{\delta V}{\delta g_j}(a, n, z, g)(y) \mu_j^g(y, z, g) dy, \end{aligned} \quad (8.12)$$

where μ_j^g is the KFE drift computed from the optimal savings policy, and the last line encodes how changes in the distribution affect the value function through prices. The kernel $\delta V / \delta g_j(y)$ is the infinite-dimensional analogue of a gradient: it measures how the value function at the individual state (a, n, z) responds to an infinitesimal redistribution of probability mass to wealth level y in the cross-section; the remark below makes this precise. Mass conservation guarantees $\int_{\underline{a}}^{\infty} \mu_j^g(y, z, g) dy = 0$ (the KFE flux integrates to zero under no-flux boundary conditions), so the integrand of the last line pairs $\delta V / \delta g_j$ with a mean-zero test perturbation in exactly the sense required by the functional-derivative remark below.

The envelope condition. Following Gu et al. (2024), it is more convenient to work with $W(a, n, z, g) := \partial_a V(a, n, z, g)$, which directly gives the consumption policy via $c^* = W^{-1/\gamma}$. The master equation for W takes the form:

$$0 = (r(z, g) - \rho) W + \underbrace{\mathcal{L}_x W}_{\text{individual}} + \underbrace{\mathcal{L}_z W}_{\text{aggregate TFP}} + \underbrace{\mathcal{L}_g W}_{\text{distribution}}, \quad (8.13)$$

where $\mathcal{L}_x W$ captures individual state dynamics (savings, income switching), $\mathcal{L}_z W = \partial_z W \cdot \mu^z + \frac{1}{2}(\sigma^z)^2 \partial_{zz} W$ captures TFP dynamics, and $\mathcal{L}_g W = \sum_n \int (\delta W / \delta g_n) \cdot \mu_n^g dy$ captures distribution dynamics.

One equation to rule them all

The master equation subsumes the HJB, KFE, and market clearing into a *single* PDE. Its advantage is conceptual clarity and amenability to neural network approximation (via EMINNs). Its challenge is infinite dimensionality: the argument g is a measure, requiring finite-dimensional approximation.

A user's note on the functional derivative $\delta W/\delta g$

The objects $\delta V/\delta g$ and $\delta W/\delta g$ deserve a moment of attention because they are the only piece of mathematics in this chapter that is not standard PDE calculus. V takes a *function* (the density g) as one of its arguments; differentiating V with respect to g therefore returns not a number but again a function. (Strictly speaking, the Fréchet derivative is the linear functional $\zeta \mapsto \int (\delta V/\delta g) \zeta dy$ itself, and the kernel $\delta V/\delta g$ is what is properly called the *functional* or *variational* derivative. We follow common usage and call the kernel the Fréchet derivative below.) Concretely, if $g_\varepsilon = g + \varepsilon \zeta$ for some test perturbation $\zeta(y)$ with $\int \zeta dy = 0$, then

$$\left. \frac{d}{d\varepsilon} V(\cdot, g_\varepsilon) \right|_{\varepsilon=0} = \int \frac{\delta V}{\delta g}(y) \zeta(y) dy.$$

The kernel $\delta V/\delta g$ is the *Fréchet* (or *linear functional*) derivative of V in the g -argument; it measures how the value of an agent at a given (a, n, z) responds to an infinitesimal redistribution of mass at point y in the cross-section. In the master-equation literature this object is more precisely the *Lions derivative* $\partial_\mu V$ with respect to the mean-field measure μ ; when μ admits a density g it reduces to the functional derivative used here. The infinite-dimensionality of the master equation is the price of this extra argument; the EMINN approach in the next section makes it tractable by replacing g with a finite-dimensional surrogate $\hat{\varphi}$ and then differentiating through $\hat{\varphi}$ via the chain rule and standard automatic differentiation. For the underlying mean-field-games calculus see [Cardaliaguet et al. \(2019\)](#).

8.8 EMINNs: Solving the Master Equation with Deep Learning

Economic Model Informed Neural Networks (EMINNs), introduced by [Gu et al. \(2024\)](#), solve the master equation by (i) approximating the infinite-dimensional distribution g by a finite-dimensional object $\hat{\varphi}$, and (ii) parameterizing W by a neural network trained to minimize the master equation residual. A teaching-scale companion notebook for EMINNs is forthcoming; in the present edition, the master-equation discussion is text-only and the Aiyagari notebook (`lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`) is the closest computational reference.

8.8.1 Three Approximation Approaches for the Distribution

The infinite-dimensional distribution g must be replaced by a finite-dimensional approximation $\hat{\varphi} \in \mathbb{R}^d$ so that $V(a, n, z, g) \approx \hat{V}(a, n, z, \hat{\varphi})$. Table 8.3 summarizes the three approaches used in [Gu et al. \(2024\)](#).

Finite population ($N \approx 40$). Replace the continuum by N agents with states $\hat{\varphi}_t = \{(a_t^1, n_t^1), \dots, (a_t^N, n_t^N)\}$. Aggregate capital is $K_t = N^{-1} \sum_i a_t^i$. Sampling individual states and distribution states *separately* during training keeps N manageable; the law of large numbers provides accurate aggregate capital even with 40 agents.

	Finite population	Discrete state	Projection
$\hat{\varphi}_t$	$\{(a_t^i, n_t^i)\}_{i=1}^N$	Masses on grid $\{a_1, \dots, a_I\}$	Basis coefficients
$\hat{g}_t$	$\frac{1}{N} \sum_i \delta_{\hat{\varphi}_t^i}$	$\sum_{i,j} \hat{\varphi}_{ij} \delta_{(a_i, n_j)}$	$b_0 + \sum_i \hat{\varphi}_i b_i(a, n)$
Dimension	~ 40	~ 200	~ 5

Table 8.3: Finite-dimensional representations of the cross-sectional distribution in EMINNs. Here $\delta_\bullet$ denotes a Dirac measure centered at $\bullet$, not the depreciation rate of Chapters 2–6.

Discrete state (~ 200 grid points). Discretize wealth on a grid $\{a_1, \dots, a_I\}$ and represent the distribution as masses $\hat{\varphi}_{i,j}$ at each (a_i, n_j) . The KFE becomes a finite-difference mass evolution, and the functional derivative becomes a partial derivative $\partial_{\hat{\varphi}_{m,j}} \hat{W}$.

Projection (~ 5 components). Project g onto eigenfunctions of the steady-state KFE operator $\bar{\mathcal{L}}^{KF}$. These are the most *persistent* density components, carrying the most price-relevant information. Only ~ 5 basis functions suffice, yielding the lowest-dimensional representation, but the setup requires computing eigenfunctions and choosing appropriate test functions for the KFE evolution.

8.8.2 The EMINN Algorithm

A neural network $\hat{W}_\Theta(\omega)$ with $\omega = (a, n, z, \hat{\varphi})$ parameterizes the marginal value of wealth. The output uses a softplus activation to ensure $\hat{W} > 0$, and consumption follows directly from the envelope condition: $c^* = \hat{W}^{-1/\gamma}$. Algorithm 3 gives the resulting residual-minimization loop.

Algorithm 3 EMINN training for the master equation

Require: Initial parameters Θ , tolerance ε , loss weights κ^e, κ^s ; initialize $\mathcal{E} = \infty$
while $\mathcal{E} > \varepsilon$ **do**
 Sample M collocation points: $(a_m, n_m, z_m, \hat{\varphi}_m)_{m=1}^M$
 Compute distribution-drift coefficients $\mu_{\hat{\varphi},n}(\hat{\varphi}_m)$ (method-specific; see §8.8.1 and remark below)
 Compute master equation residual $\hat{\mathcal{L}}(\cdot; \Theta)$ via automatic differentiation through $\hat{\varphi}$
 Compute shape penalty $\mathcal{E}^s(\cdot; \Theta)$ (concavity, monotonicity)
 Total loss: $\mathcal{E} = \kappa^e M^{-1} \sum_m |\hat{\mathcal{L}}_m|^2 + \kappa^s \mathcal{E}^s$
 Update: $\Theta \leftarrow \Theta - \alpha_{\text{opt}} \nabla_{\Theta} \mathcal{E}$ (α_{opt} : optimizer learning rate; distinct from the OU mean-reversion η of §8.7)
end while

This is precisely a PINN applied to the master equation: the “physics” is the economic equilibrium structure, and all derivatives, including those with respect to the distribution parameters $\hat{\varphi}$, are computed by automatic differentiation.

The master equation residual decomposes as:

$$0 = (r(z, \hat{\varphi}) - \rho) \hat{W} + \hat{\mathcal{L}}_x \hat{W} + \hat{\mathcal{L}}_z \hat{W} + \hat{\mathcal{L}}_g \hat{W}, \quad (8.14)$$

where $\hat{\mathcal{L}}_x \hat{W} = s(\cdot) \partial_a \hat{W} + \lambda(n)(\hat{W}(\hat{n}) - \hat{W}(n))$ captures savings and income switching, $\hat{\mathcal{L}}_z \hat{W} = \partial_z \hat{W} \cdot \mu^z + \frac{1}{2}(\sigma^z)^2 \partial_{zz} \hat{W}$ captures the OU aggregate shock, and $\hat{\mathcal{L}}_g \hat{W} = \sum_n (\partial \hat{W} / \partial \hat{\varphi}_n) \cdot \mu_{\hat{\varphi},n}$ captures distribution evolution.

Computing the drift coefficients $\mu_{\hat{\varphi},n}$. The coefficients $\mu_{\hat{\varphi},n}$ describing how the finite-dimensional approximation $\hat{\varphi}$ of the cross-sectional density evolves in time are not given for free. Recovering them is a genuine algorithmic step, and it is the place where the three approximation choices in Section 8.8.1 differ most sharply:

- **Finite-population approximation.** $\hat{\varphi}$ is the empirical measure of a finite particle system, so $\mu_{\hat{\varphi},n}$ is the SDE drift of particle n , available in closed form from the underlying individual problem.
- **Discrete-state (grid) approximation.** $\hat{\varphi}_{i,j}$ is the mass at (a_i, n_j) ; the discretized KFE evaluated at that node returns $\mu_{\hat{\varphi},(i,j)}$ directly. This is also where an upwind scheme reappears (see the remark after (7.20)): the side of ∂_a used in the discretization is selected by the sign of the drift s .
- **Projection / basis expansion.** $\hat{\varphi}(a) = \sum_n c_n \psi_n(a)$. The KFE returns $\partial_t g$ as a function of a ; one then recovers $\mu_{\hat{\varphi},n} = \dot{c}_n$ by least-squares projection of $\partial_t g$ onto $\{\psi_n\}$. This is the only step of EMINN that genuinely depends on the choice of approximation, and it is where most of the practical art lives.

Concretely, between the forward pass and the residual evaluation in Algorithm 3, an additional sub-step computes $\mu_{\hat{\varphi},n}$ for the current $\hat{\varphi}$; the chain rule through $\hat{\varphi}$ then propagates these coefficients into $\hat{\mathcal{L}}_g \hat{W}$.

8.8.3 Shape Constraints and Training Stability

A key practical challenge in training EMINNs is that neural networks may converge to “cheat solutions” (constant, non-increasing, or non-concave value functions that produce small residuals but are economically meaningless). Shape penalties are added to the loss to enforce economic structure:

- **Concavity:** penalize non-concavity (i.e., violations of the standard $V_{aa} \leq 0$ condition) via $\mathcal{E}^{\text{concav}} = M^{-1} \sum_m \max(0, \partial_{aa} V_m)^2$, which is positive only when $\partial_{aa} V > 0$ at some collocation point.
- **Monotonicity:** in calibrations where higher TFP lowers the marginal value of liquid wealth, penalize violations of the model-specific restriction $\partial_z V_a < 0$.
- **Initialization:** set $W(a, \cdot) = e^{-a}$ as a simple decreasing initial marginal-value profile.
- **Architectural encoding:** if the chosen formulation has known boundary asymptotics for V or W , multiply the network by an appropriate boundary factor rather than asking the optimizer to learn the singular shape from scratch.
- **Active sampling:** concentrate collocation points where the residual is large.

8.8.4 Results and Method Comparison

For the Aiyagari model (no aggregate shocks), Gu et al. (2024) report master-equation residuals of order 10^{-5} and mean squared errors against finite-difference benchmarks of order 10^{-5} , with close agreement in consumption policies, marginal value functions, and stationary distributions across the finite-population and discrete-state approaches.

For the Krusell–Smith model (with aggregate shocks), their reported results show master-equation training losses of order 10^{-5} across all three approximation approaches and similar time paths for aggregate variables (capital, interest rate, wage). In that experiment, the projection approach achieves the lowest reported loss (8.5×10^{-6}) with only 5 distribution parameters, while the finite-population approach (3.0×10^{-5}) offers the simplest implementation.

Table 8.4 gives the practical method comparison for this chapter.

For stationary, low-dimensional problems, finite differences remain fast and reliable and should be used as the benchmark. For models with aggregate shocks and high-dimensional state spaces, EMINNs are, among the methods surveyed here, one of the few approaches demonstrated on global master-equation solutions for this class of benchmarks. PINNs serve as a useful

	Finite differences	PINN	EMINN
Stationary equilibrium	Benchmark method	Works, validate carefully	Works, but overkill
Aggregate shocks	Local/low-dimensional only	Coupled not full master equation	Designed for global master equation
Grid required	Yes	No state grid	No state grid
High-dimensional scaling	Limited by grids	Better, optimization-limited	Better, distribution-state approximation needed
Handles g as state	Not in standard stationary FD	No	Yes, through $\hat{\varphi}$
Convergence theory	Strong for low-dimensional monotone schemes	Limited	Limited
Low-dimensional accuracy	Often $\sim 10^{-6}$	Problem-dependent; validate	Reported $\sim 10^{-5}$

Table 8.4: Finite differences, PINNs, and EMINNs for the continuous-time heterogeneous-agent problems covered in this chapter. The entries are practical guidance, not universal impossibility results: finite differences remain the benchmark in low dimension, while EMINNs target the global master-equation setting with the distribution as a state.

intermediate step: they share the same code philosophy as EMINNs (automatic differentiation of PDE residuals) but apply to the coupled HJB–KFE system rather than the master equation.

Chapter Summary

- In continuous time, the heterogeneous-agent equilibrium is a coupled HJB + KFE system, recast as a mean field game in the sense of Lasry–Lions.
- Closing with aggregate shocks gives the master equation: a single PDE in (x, g) where the second argument is a measure. EMINNs solve it by finite-dimensional approximation of g and a neural-network ansatz for the value function.
- The functional derivative $\delta V/\delta g$ is the only piece of mathematics not standard in PDEs; it measures sensitivity to infinitesimal cross-section perturbations.
- Achdou et al. (2022) finite differences and EMINNs are complementary: FD wins in low dimension, while EMINNs are designed for aggregate-shock master-equation models.

Further Reading

- Achdou et al. (2022), the canonical continuous-time HA reference.
- Gu et al. (2024), the EMINN paper.
- Lasry and Lions (2007); Carmona and Delarue (2018); Cardaliaguet et al. (2019), mean-field-games foundations.
- Shreve (2004), stochastic-calculus textbook.
- Moll’s online lecture notes (<https://benjaminmoll.com/lectures/>), pedagogical complement.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Core] Itô on GBM.** Apply Itô's lemma to $f(x) = \ln x$ with X_t following geometric Brownian motion to derive $X_t = X_0 \exp[(\mu - \frac{1}{2}\sigma^2)t + \sigma B_t]$. Discuss why “volatility drag” lowers the expected log return below the expected return.
2. **[Core] KFE for an OU process.** Write the Kolmogorov forward equation for the Ornstein–Uhlenbeck process $dX_t = \eta(\bar{X} - X_t)dt + \sigma dB_t$ and derive the stationary density $\mathcal{N}(\bar{X}, \sigma^2/(2\eta))$ by setting $\partial_t g = 0$.
3. **[Core] Functional derivative.** For the master-equation value function $V(a, g)$, compute $\delta V/\delta g$ for the toy specification $V(a, g) = \int u(c(a, y))g(y)dy$ where c is a fixed consumption rule. Interpret the result.
4. **[Computational] HJB residual.** In notebook `lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`, compute the maximum HJB residual on a 50-point test grid after a fixed PINN training budget. Repeat with a larger collocation batch and report the actual scaling you observe.
5. **[Core] Closed Aiyagari system in continuous time.** Combine the HJB equation (8.8) for $V(a, n)$ and the KFE (8.6) for $g(a, n)$ into the closed Aiyagari general-equilibrium system. (i) Add the firm's first-order conditions: with Cobb–Douglas production $Y = AK^\alpha L^{1-\alpha}$, write $r = \alpha AK^{\alpha-1}L^{1-\alpha} - \delta$ and $w = (1 - \alpha)AK^\alpha L^{-\alpha}$. (ii) Add the market-clearing conditions $K = \sum_n \int_a^\infty a g(a, n) da$ and $L = \sum_n n \int_a^\infty g(a, n) da$. (iii) Show that the coupled system pins down (V, g, K, L, r, w) , with L often fixed by the stationary income shares and w implied by (K, L) . (iv) Briefly explain why solving this system for the stationary equilibrium is commonly reduced to a fixed point in r , and why the inner HJB and KFE problems must be solved consistently at each candidate r .
6. **[Advanced/project] Stationary-distribution convergence.** In notebook `lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`, take the optimal policy $s^*(a, n)$ from the converged HJB solver (i.e., fix the policy) and run the KFE (8.6) forward in time starting from a non-stationary initial density (e.g., uniform on $[a, a_{\max}]$). Plot snapshots of $g_t(a, n)$ at $t \in \{1, 5, 25, 100, 500\}$ years and the running L^2 distance $\|g_t - g^*\|_{L^2}$ to the stationary distribution g^* . Check whether convergence is approximately exponential by fitting a line to $\log \|g_t - g^*\|$ over the linear region, and relate the fitted rate to the spectral gap of the KFE operator.
7. **[Advanced/project] FD upwind vs. PINN benchmark.** In the same notebook, the chapter ships both a finite-difference upwind solver (deterministic, grid-based) and a PINN trained on HJB and KFE residuals. Run both on the same 1-asset Aiyagari calibration and report (i) wall-clock time for your chosen residual target, (ii) the final residual on a fine 200-point test grid, and (iii) peak memory, including GPU memory if you run the PINN on a GPU. As an extension, sweep the discount rate ρ over $\{0.04, 0.05, 0.06\}$ and compare cold-start FD runs with warm-started PINN runs. Discuss when each is preferred: FD for low-dimensional, fixed-parameter computations; PINN for parametric sweeps and higher-dimensional extensions.

Chapter 9

Gaussian Processes

§4.4.1 introduced Gaussian processes at the picture level, with just enough machinery to follow the Bayesian-optimization argument of Chapter 4: the definition $f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$, the squared-exponential kernel (4.2), and the closed-form posterior mean and variance (4.3)–(4.4). Those formulas took the kernel and its hyperparameters $(\ell, \sigma_f, \sigma_y)$ as given. The present chapter removes both shortcuts and develops the full machinery: it samples whole functions from the GP *prior* (§9.1), learns $(\ell, \sigma_f, \sigma_y)$ from data via the marginal-likelihood Occam’s razor (§9.2), expands the kernel zoo beyond the squared-exponential to Matérn and composite kernels (§9.2.1), uses the posterior variance to choose where to evaluate next via Bayesian Active Learning (§9.3), scales the whole machinery to high-dimensional inputs through active subspaces (§9.5), and embeds GPs inside value-function iteration for high-dimensional dynamic programming (§9.6). Two applications-driven chapters then put the machinery to work: structural estimation via surrogate-driven SMM (Chapter 10) and climate-policy uncertainty quantification (Chapter 11). Methodological foundations are the GP textbook of [Rasmussen and Williams \(2005\)](#) and the Bayesian-active-learning ideas dating back to [MacKay \(1992\)](#) and [Krause et al. \(2008\)](#). Embedding GPs inside dynamic programming was pioneered by [Deisenroth et al. \(2009\)](#) (GPDP) and [Engel et al. \(2005\)](#) (GP temporal-difference learning); within economics, applications include high-dimensional growth models ([Scheidegger and Bilonis, 2019](#)), dynamic incentive problems ([Renner and Scheidegger, 2018](#)), and deep uncertainty quantification for integrated assessment models ([Friedl et al., 2023](#)). Sparse approximations ([Titsias, 2009](#); [Hensman et al., 2013](#)) extend GPs beyond their cubic scaling limit; deep kernel learning combines neural-network feature extraction with GP inference and closes the chapter.

9.1 Gaussian Process Regression

Sampling from the GP prior. Recall from §4.4.1 that for any finite set of test inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$, the function values $\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_n))^\top$ are jointly Gaussian with prior mean $\boldsymbol{\mu}$ (the entries $\mu(\mathbf{x}_i)$) and covariance matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. To draw a function from the GP prior at those test inputs, evaluate K , compute its Cholesky decomposition $K = LL^\top$, draw $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, I)$, and set $\mathbf{f} = \boldsymbol{\mu} + L\mathbf{u}$. The resulting paths have the smoothness, amplitude, and length scale dictated entirely by the kernel.

Kernel choice. The squared-exponential kernel (4.2) of §4.4.1 remains the right default for genuinely smooth targets, e.g. in the unconstrained interior of an ergodic set. In economic applications, however, the target function (value, policy, equilibrium price) often has *kinks* from occasionally binding constraints, and the Matérn-3/2 kernel introduced in §9.2.1 below is then a better default than RBF, since it does not oversmooth at non-differentiable features.

Figure 9.1 shows how the RBF length scale controls the distance over which observations remain informative. For any training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the GP posterior at a test point

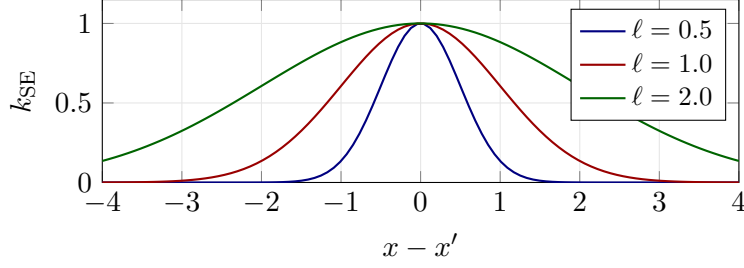


Figure 9.1: Squared-exponential kernel as a function of distance for three length scales. Small ℓ makes correlations decay quickly and produces rougher, more local fits; large ℓ couples distant points and imposes smoother functions.

$\mathbf{x}_*$ has the closed-form mean $\bar{f}_*$ and variance $\sigma_{f,*}^2$ stated in (4.3)–(4.4), with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ the kernel matrix on training inputs, $K_y = K + \sigma_y^2 I$ its noise-augmented version, $\boldsymbol{\mu}_X$ the prior-mean vector at training inputs, and $\mathbf{k}_*$ the cross-covariance whose i -th entry is $k(\mathbf{x}_*, \mathbf{x}_i)$. For a noisy future observation y_* the predictive variance is $\sigma_{y,*}^2 = \sigma_{f,*}^2 + \sigma_y^2$, and the common zero-mean formulas are recovered by centering outputs or setting $\mu \equiv 0$.

A hand-traceable 1D example. To make (4.3)–(4.4) concrete, take $f(x) = \sin x$, observe f noiselessly at $x_1 = 0$ and $x_2 = \pi$ (so $y_1 = y_2 = 0$), and query at $x_* = \pi/2$. Use the kernel $k(x, x') = \exp(-(x - x')^2/2)$ and a tiny noise floor $\sigma_y^2 = 10^{-6}$ for numerical stability. The training kernel matrix is

$$K + \sigma_y^2 I = \begin{pmatrix} 1 & e^{-\pi^2/2} \\ e^{-\pi^2/2} & 1 \end{pmatrix} \approx \begin{pmatrix} 1.000 & 0.00719 \\ 0.00719 & 1.000 \end{pmatrix},$$

where the off-diagonal $e^{-\pi^2/2} \approx 0.00719$ is small because 0 and π are far apart relative to $\ell = 1$. The cross-covariance vector is

$$\mathbf{k}_* = \begin{pmatrix} \exp(-(\pi/2)^2/2) \\ \exp(-(\pi/2)^2/2) \end{pmatrix} \approx \begin{pmatrix} 0.2910 \\ 0.2910 \end{pmatrix},$$

since $x_* = \pi/2$ is equidistant from 0 and π . Because $\mathbf{y} = (0, 0)^\top$, the posterior mean (4.3) is exactly $\bar{f}_* = 0$. For the variance,

$$(K + \sigma_y^2 I)^{-1} \mathbf{k}_* \approx \frac{0.2910}{1 + 0.00719} (1, 1)^\top \approx (0.2890, 0.2890)^\top,$$

so $\mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_* \approx 2 \cdot 0.2910 \cdot 0.2890 \approx 0.1682$, giving $\sigma_*^2 \approx 1 - 0.1682 \approx 0.832$ and a posterior standard deviation $\sigma_* \approx 0.91$. The GP predicts zero at the midpoint, with substantial residual uncertainty, consistent with the fact that $\sin(\pi/2) = 1$ is not pinned down by the two boundary observations under this length scale.

Figure 9.2 illustrates the GP prior and posterior for a simple one-dimensional regression problem. Before observing data, the GP prior has constant mean and uniform uncertainty. After conditioning on five observations, the posterior mean interpolates the data and the uncertainty bands collapse near the observations while remaining wide in unexplored regions.

Built-in uncertainty quantification

The posterior variance σ_*^2 is small near observed data (the GP is confident) and large far from observed data (the GP is uncertain). This property is the foundation of Bayesian active learning.

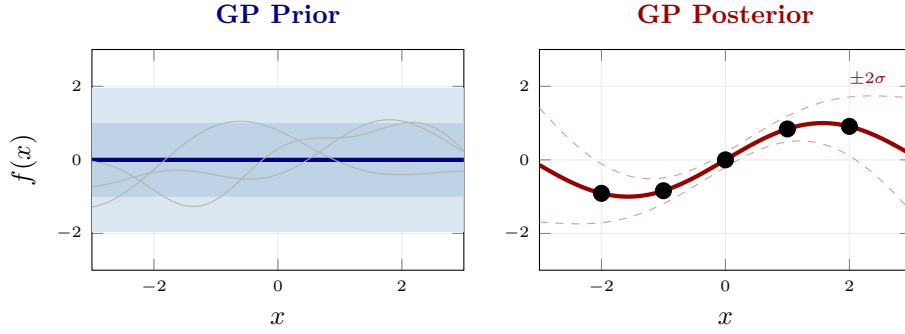


Figure 9.2: Gaussian-process prior and posterior on a 1D regression problem. *Left*: the prior has constant mean (here zero) and uniform uncertainty; the shaded bands show the 68% and 95% credible intervals, and the thin gray curves are three sample paths drawn from the prior. *Right*: after conditioning on five observations (black dots), the posterior mean (red curve) interpolates the data exactly, and the credible band collapses near the observed points while widening in unexplored regions away from the data, giving the GP its built-in uncertainty quantification.

9.2 Kernel Functions and Hyperparameter Learning

The kernel hyperparameters $\boldsymbol{\vartheta} = (\ell, \sigma_f, \sigma_y)$ are learned by maximizing the log marginal likelihood:

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\vartheta}) = -\frac{1}{2}(\mathbf{y} - \boldsymbol{\mu}_X)^\top K_y^{-1}(\mathbf{y} - \boldsymbol{\mu}_X) - \frac{1}{2} \log |K_y| - \frac{n}{2} \log 2\pi, \quad (9.1)$$

where $K_y = K + \sigma_y^2 I$ and $\boldsymbol{\mu}_X$ is the prior mean evaluated on the training inputs. In the zero-mean convention used elsewhere in this section, set $\boldsymbol{\mu}_X = 0$ (or center the outputs).

Why marginal likelihood? The log evidence $\log p(\mathbf{y} | \mathbf{X}, \boldsymbol{\vartheta})$ encodes both data fit *and* an automatic complexity penalty in a single closed-form expression. The quadratic form $-\frac{1}{2}\mathbf{y}^\top K_y^{-1}\mathbf{y}$ rewards hyperparameters that explain the centered observations with a small inverse-covariance norm, while the log-determinant term $-\frac{1}{2} \log |K_y|$ penalizes overly flexible kernels that admit too many possible functions, giving Bayesian Occam’s razor (see, e.g., [Rasmussen and Williams, 2005](#), Ch. 5). Compared with cross-validated MSE, this approach requires no held-out split, makes use of all n observations, and exposes a closed-form gradient with respect to $\boldsymbol{\vartheta}$, which is essential for L-BFGS-style optimization in scikit-learn / GPyTorch. The maximum is reached at the kernel that is just expressive enough to fit the data but no more (Figure 9.3).

A free held-out diagnostic. Marginal likelihood is not the only validation tool. The leave-one-out (LOO) predictive error of a GP admits a closed form using the same Cholesky factor already computed for posterior inference, so it costs nothing extra; we develop the formula in §9.6.5, where it serves as an iteration-by-iteration health check inside GP-based VFI, and reuse it in §10.7 for the GP layer over the SMM moment map.

9.2.1 Kernel Composition

More complex covariance structures can be built by composing simpler kernels. The sum of two kernels is again a valid kernel, as is the product. For example:

- $k_{\text{SE}} + k_{\text{periodic}}$: captures a smooth trend plus periodic oscillations.
- $k_{\text{SE}}(\ell_1) \cdot k_{\text{SE}}(\ell_2)$: models interactions between two length scales.

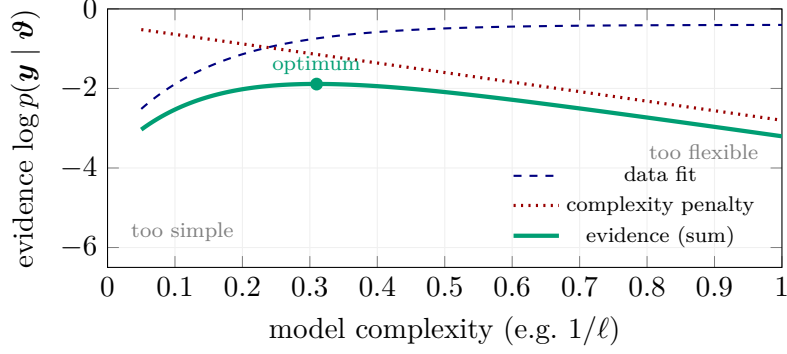


Figure 9.3: Marginal-likelihood Occam’s razor for a GP. As the kernel becomes more flexible (smaller length scale ℓ), the data-fit term improves but the $\log |K_y|$ complexity penalty grows linearly. Their sum, the log evidence, peaks at an interior optimum that is just expressive enough to explain the data, automatically. No held-out validation set is required.

The Matérn kernel family. The Matérn kernel is parameterized by a smoothness parameter $\nu > 0$:

$$k_{\text{Matérn}}(r) = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right), \quad (9.2)$$

where $r = \|\mathbf{x} - \mathbf{x}'\|$, ℓ is the length scale, and K_ν is the modified Bessel function of the second kind. The smoothness parameter ν controls the regularity of sample paths: a Matérn- ν GP is k -times mean-square differentiable for every integer $k < \nu$, with the RBF kernel recovered in the limit $\nu \rightarrow \infty$. Important special cases:

- $\nu = 1/2$: the Ornstein–Uhlenbeck kernel $k(r) = \sigma_f^2 \exp(-r/\ell)$; sample paths are continuous but nowhere differentiable.
- $\nu = 3/2$: $k(r) = \sigma_f^2 (1 + \sqrt{3} r/\ell) \exp(-\sqrt{3} r/\ell)$; once differentiable.
- $\nu = 5/2$: $k(r) = \sigma_f^2 (1 + \sqrt{5} r/\ell + 5r^2/(3\ell^2)) \exp(-\sqrt{5} r/\ell)$; twice differentiable.
- $\nu \rightarrow \infty$: recovers the squared exponential (RBF) kernel; infinitely differentiable.

For economic applications where the target function may have kinks (e.g., due to occasionally binding constraints), the Matérn-3/2 kernel is often a better choice than the infinitely smooth RBF kernel, as it does not oversmooth near non-differentiable features (Renner and Scheidegger, 2018).

9.3 Bayesian Active Learning for Sample-Efficient Training

When model evaluations are expensive, we wish to select training points that provide maximal information. Bayesian Active Learning (BAL) uses the GP posterior variance to guide this selection. The information-theoretic foundations go back to MacKay (1992), with submodular guarantees for variance-based sensor placement established by Krause et al. (2008) and the widely used upper-confidence-bound formulation of Srinivas et al. (2010).

Connection to Bayesian optimization (Chapter 4). BAL is the active-learning twin of the Bayesian-optimization (BO) recipe introduced in Chapter 4: same GP surrogate, same acquisition-function machinery. The difference is the target. BO seeks a *scalar optimum* of the surrogate (e.g. Expected Improvement steers samples toward $\arg \max \hat{f}$), so its acquisition trades exploration against the chance of beating the current best. BAL instead targets *global function*

approximation: it allocates samples wherever posterior variance is largest, irrespective of the predicted value. Both reduce to the same primitive (fit GP, maximize acquisition, evaluate, refit), but the choice of acquisition reflects whether one wants the best point or the best surrogate.

BAL Acquisition Function

$$U(\mathbf{x}) = w_{\text{obj}} \cdot \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x}), \quad (9.3)$$

where $\mu(\mathbf{x})$ and $\sigma^2(\mathbf{x})$ are the GP posterior mean and variance, w_{obj} is the objective-following (or Bayesian-optimization) weight, and w_{var} controls exploration. This logarithmic-variance form is the one used by Renner and Scheidegger (2018) in economic applications; it is a non-standard relative of GP-UCB (Srinivas et al., 2010). When the goal is global surrogate accuracy rather than optimization, integrated posterior variance or expected error reduction can be more natural objectives. The weights w_{obj} and w_{var} are local to the BAL context and distinct from the discount factor β and shock-persistence notation used elsewhere in the script.

Algorithm: Bayesian Active Learning

Input: Initial design $\mathcal{D}_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_0}$, budget N , kernel k , acquisition weights $(w_{\text{obj}}, w_{\text{var}})$
Fit GP on $\mathcal{D}_0$: learn hyperparameters via marginal likelihood
for $n = n_0 + 1, \dots, N$ **do**
 Compute acquisition function: $U(\mathbf{x}) = w_{\text{obj}} \cdot \mu_{n-1}(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma_{n-1}^2(\mathbf{x})$
 Select next point: $\mathbf{x}_n = \arg \max_{\mathbf{x}} U(\mathbf{x})$
 Evaluate expensive model: $y_n = f(\mathbf{x}_n)$
 Update dataset: $\mathcal{D}_n = \mathcal{D}_{n-1} \cup \{(\mathbf{x}_n, y_n)\}$
 Re-fit GP on $\mathcal{D}_n$ (update hyperparameters every k iterations)
end for
Output: Trained GP surrogate $\hat{f}(\cdot)$ with posterior mean μ_N and variance σ_N^2

The BAL algorithm concentrates training points near kinks, boundary layers, and other regions where the function is hardest to approximate, achieving the same accuracy as uniform sampling with far fewer evaluations (Renner and Scheidegger, 2018). The exploration–exploitation trade-off controlled by $(w_{\text{obj}}, w_{\text{var}})$ ensures that the algorithm balances refining the approximation in already well-sampled regions against exploring uncharted territory.

The intuition behind the acquisition function is as follows. The first term $w_{\text{obj}} \cdot \mu(\mathbf{x})$ favors regions where the predicted function value is large (exploitation: sample where the function is interesting). The second term $\frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ favors regions of high uncertainty (exploration: sample where we know least). This acquisition function belongs to the Upper Confidence Bound (UCB) family (Srinivas et al., 2010): with $w_{\text{obj}} = 0$ it has the same maximizers as pure posterior-variance sampling, while the logarithmic variance weighting provides a more conservative exploration bonus than the standard-deviation weighting used in GP-UCB when it is combined with exploitation. By adjusting w_{obj} and w_{var} , the practitioner can control the balance. For economic applications where the entire domain is relevant (e.g., approximating a policy function), a pure exploration strategy ($w_{\text{obj}} = 0, w_{\text{var}} > 0$) is often appropriate, reducing BAL to an uncertainty sampling scheme that minimizes the integrated posterior variance. Figure 9.4 illustrates two BAL iterations on a 1D toy.

9.4 When to Use GPs vs. DNNs

Now that the GP machinery (posterior inference, kernel design, and BAL) has been introduced, we can give a more detailed comparison than the overview in Table 10.1. Active subspaces, introduced in Section 9.5, are the main tool for pushing GP surrogates beyond the moderate-

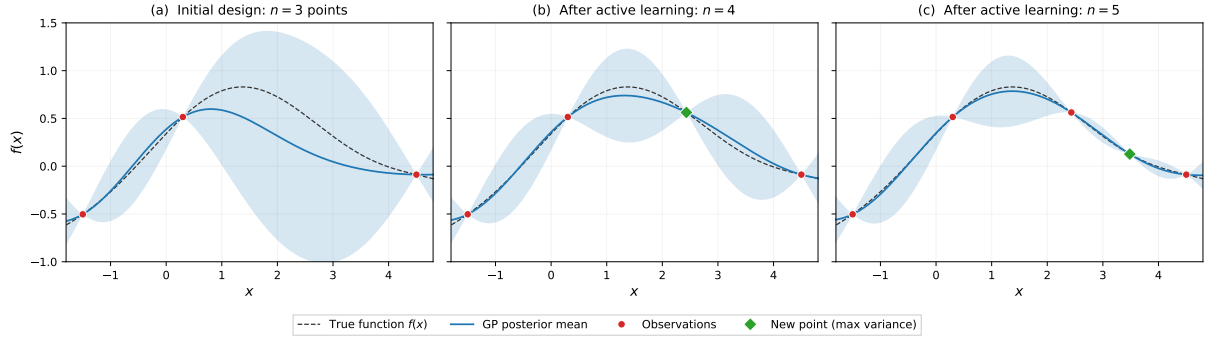


Figure 9.4: Bayesian Active Learning in action. **(a)** Starting from three initial observations (red dots), the GP posterior mean (blue line) deviates from the true function (dashed black) in the data-sparse region, where the 95% credible band (blue shading) is wide. **(b)** The acquisition function selects the point of maximum posterior variance (green diamond); after evaluation, the posterior tightens locally and the mean improves. **(c)** A second active-learning iteration fills the remaining gap. With only five strategically chosen points, the GP posterior closely tracks the true function across the entire domain.

dimensional regime. Table 9.1 extends Table 10.1 with the GP-specific items (LOO diagnostics, marginal-likelihood Occam, BAL).

Criterion	Gaussian Processes	Deep Neural Networks
Training cost	$\mathcal{O}(N^3)$; exact for N in the low thousands [†]	$\mathcal{O}(N)$ per epoch; scales to $N \sim 10^6$
Prediction cost	$\mathcal{O}(N)$ posterior mean, $\mathcal{O}(N^2)$ posterior variance, per test point	$\mathcal{O}(\text{weights})$ per forward pass; independent of N
Gradient access	closed-form from posterior	autodiff (exact, any order)
Non-smooth features	Matérn-3/2 adapts well	excellent with enough data
High-dim. extension	active subspaces ($d \gg 10$)	native

Table 9.1: GP and DNN surrogate trade-offs after the basic GP machinery has been introduced. GPs are sample-efficient and uncertainty-aware but expensive in the number of training points; DNNs scale to much larger datasets and dimensions but need more data and separate machinery for calibrated uncertainty.

[†] Sparse-GP via inducing points reduces $\mathcal{O}(N^3)$ to $\mathcal{O}(Nm^2 + m^3)$ for $m \ll N$ inducing inputs (Titsias, 2009; Hensman et al., 2013); see the inducing-point remarkbox below.

Practical guidelines.

- **Choose a GP** when each model evaluation is expensive (minutes to hours per solve), the effective dimensionality is moderate ($d \lesssim 15$, or higher with active subspaces), and uncertainty quantification on the surrogate is needed, e.g., for reporting confidence intervals on estimated parameters or for guiding further data collection via BAL.
- **Choose a DNN** when training data is cheap to generate, the input dimension is high ($d \gg 10$), and the downstream task requires millions of fast evaluations, e.g., Monte Carlo simulation, grid search over a large parameter space, or real-time inference.
- **Combine both** when the problem has a natural two-stage structure: use a GP with BAL to build a small, high-quality training set with uncertainty estimates, then train a DNN on the resulting dataset for fast large-scale deployment. The active subspace approach of Scheidegger and Bilonis (2019) is particularly effective in this setting, extending GP methods far beyond their naïve scaling limits.

9.4.1 GP Regression in Practice

In code, GP regression is a one-liner once the kernel is chosen. In scikit-learn the standard pattern is to assemble a kernel as the sum of an RBF (`RBF(length_scale=...)`) and a noise term (`WhiteKernel(noise_level=...)`), pass it to `GaussianProcessRegressor`, call `.fit(X_train, y_train)`, and obtain posterior mean and standard deviation from `.predict(X_test, return_std=True)`; the kernel hyperparameters (`length_scale`, `noise_level`, output amplitude) are optimized by maximizing the marginal likelihood, with `n_restarts_optimizer` controlling robustness to local optima. The companion notebook `02_GP_and_BAL.ipynb` provides a full worked example fitting noisy observations of $\sin(x)$ on $[-2, 2]$.

Application: GP surrogates for option pricing. GPs are particularly well suited as surrogates for derivative pricing models. For example, one can train a GP on as few as 5–50 Black–Scholes option prices (evaluated at different spot prices or parameter configurations) and obtain a surrogate that accurately reproduces the pricing surface with calibrated uncertainty bands. The posterior variance immediately quantifies the interpolation uncertainty at each query point. This idea extends naturally to stochastic volatility models such as Heston, where the analytical pricing formula is expensive to evaluate. Furthermore, because GP predictions are linear in the training targets, the uncertainty of a *portfolio* of GP-priced instruments propagates analytically: for a linear portfolio $\sum_i w_i \hat{V}_i$ with vector of weights $\mathbf{w}$ and joint posterior covariance $\Sigma_{\hat{\mathbf{V}}}$, $\text{Var}(\mathbf{w}^\top \hat{\mathbf{V}}) = \mathbf{w}^\top \Sigma_{\hat{\mathbf{V}}} \mathbf{w}$. When the surrogate errors are independent across instruments, $\Sigma_{\hat{\mathbf{V}}}$ is diagonal with entries σ_i^2 and the formula reduces to $\sum_i w_i^2 \sigma_i^2$; otherwise the off-diagonal cross-instrument covariances must be retained, e.g. via a multi-output GP. Either way the assessment is instant.

Computational scaling of GPs

The main limitation of GPs is their $\mathcal{O}(n^3)$ training cost, arising from the inversion of the $n \times n$ kernel matrix. For $n > 10,000$, exact GP inference becomes impractical. Approximate methods, such as variational sparse GPs with inducing points (Titsias, 2009), stochastic-variational GPs (Hensman et al., 2013), random Fourier features, and structured kernel interpolation, can extend the range to $n \sim 10^5$, but for truly large-scale problems, deep neural networks remain the method of choice. The BAL framework mitigates this limitation by keeping n small through intelligent sample selection.

The *inducing-point* idea (Figure 9.5) is simple: instead of carrying the full $n \times n$ kernel matrix, summarize the dataset with a much smaller set of $m \ll n$ pseudo-inputs $\mathbf{Z} = \{\mathbf{z}_1, \dots, \mathbf{z}_m\}$ and replace K_{nn} by the Nyström-style low-rank approximation $K_{nm} K_{mm}^{-1} K_{mn}$. Training and prediction then cost about $\mathcal{O}(nm^2 + m^3)$ rather than $\mathcal{O}(n^3)$, with the m^3 term coming from the small inducing-point block. The variational formulation of Titsias (2009) additionally treats $\mathbf{Z}$ as parameters to be optimized against the marginal likelihood, so the inducing inputs migrate to wherever the GP actually needs resolution.

9.5 Scaling GPs to High Dimensions: Active Subspaces

A common criticism of GP methods is that they are limited to moderate-dimensional problems ($d \leq 10$). While this is true for naïve implementations, where the number of training points needed to cover the input space grows exponentially in d , Scheidegger and Bilonis (2019) show that *active subspace* methods can mitigate this barrier *when the target function exhibits a clear spectral gap in the gradient outer-product matrix C of (9.4) below*, i.e., when its variation is concentrated on a low-dimensional linear subspace of the input space. If f depends comparably on all d coordinates, the spectral gap is absent and the technique offers little gain; sufficient

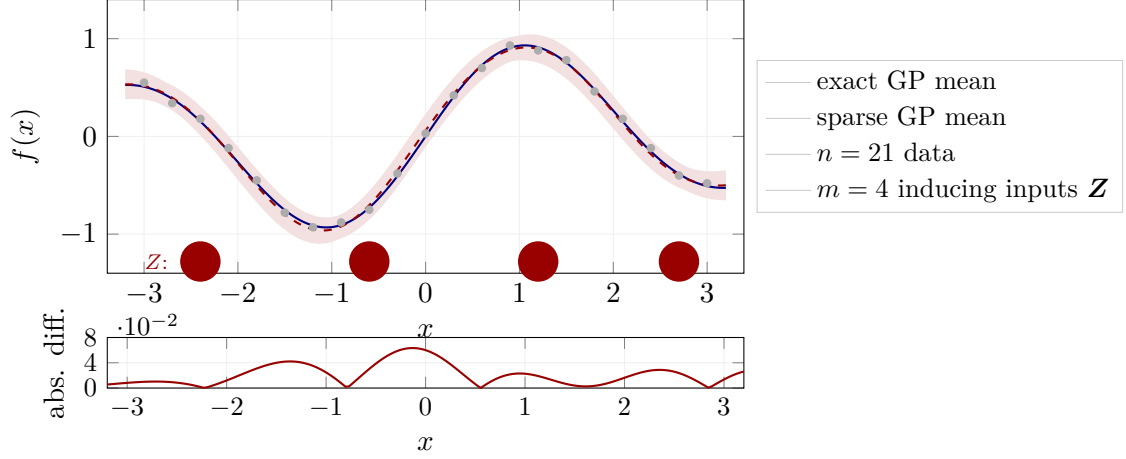


Figure 9.5: Inducing-point intuition. Exact GP inference conditions on all $n = 21$ observations and uses the full $n \times n$ kernel matrix. Sparse variational GP methods introduce $m \ll n$ inducing inputs $\mathbf{Z}$ and approximate the posterior through the low-rank structure induced by $K_{nm}K_{mm}^{-1}K_{mn}$. The top panel compares the exact posterior mean and uncertainty band with a sparse approximation using $m = 4$ pseudo-inputs; the bottom panel shows the absolute difference between the two means. In this smooth one-dimensional illustration the approximation is close, but its quality depends on m , the kernel hyperparameters, and the placement of $\mathbf{Z}$. The dominant training cost falls from $\mathcal{O}(n^3)$ to $\mathcal{O}(nm^2 + m^3)$; the variational formulation of Titsias (2009) optimizes $\mathbf{Z}$ jointly with the kernel hyperparameters.

conditions are in Constantine (2015). For a textbook treatment of the active subspace framework and references to its origins, see Constantine (2015); we summarize the key ideas and their application to dynamic economic models below.

Intuition: why most directions don't matter. Consider a value function $V(\mathbf{x})$ in a dynamic stochastic economy with d state variables. Although V is formally a function of all d inputs, it often responds primarily to a few linear combinations of them. For example, in a multi-country model, the value function may depend mostly on *aggregate* capital $\sum_j k^j$ and a measure of *inequality* $\text{Var}(k^j)$, rather than on each country's capital individually. If we can identify these important directions, we can project the d -dimensional input onto a much lower-dimensional subspace and fit the GP there, substantially mitigating the practical grid-based curse of dimensionality.

The gradient outer product matrix. The key diagnostic tool is the *gradient outer product matrix* (Constantine, 2015, Ch. 1). Given a function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ with input distribution $\pi(\mathbf{x})$, define:

$$C = \int \nabla f(\mathbf{x}) \nabla f(\mathbf{x})^\top \pi(\mathbf{x}) d\mathbf{x} \in \mathbb{R}^{d \times d}. \quad (9.4)$$

The matrix C is symmetric and positive semi-definite. Its (i, j) -entry measures how much f co-varies along input directions i and j , averaged over the input distribution. In particular:

- If $\partial f / \partial x_i$ is consistently large across the input space, the i -th diagonal entry of C is large, meaning direction i is important.
- If two directions always change f together, the corresponding off-diagonal entry is large, as they form part of the same important linear combination.
- If $\partial f / \partial x_i \approx 0$ everywhere, then direction i contributes nothing to C , so it is irrelevant and can be projected away.

Worked toy example. Take $f(x_1, x_2) = x_1^2$ on $\mathbf{x} \sim \text{Unif}([-1, 1]^2)$. Then $\nabla f = (2x_1, 0)^\top$, and integrating against the uniform measure gives $C = \text{diag}(4/3, 0)$. The eigenvalues are $\lambda_1 = 4/3$ and $\lambda_2 = 0$; the first eigenvector is $(1, 0)^\top$, the second is $(0, 1)^\top$; the spectral gap is infinite. The active subspace is the x_1 -axis, and f is recovered exactly as $f = g(x_1)$ with $g(u) = u^2$. This is the cleanest instance of the projection in (9.5) below.

Eigendecomposition and the spectral gap. Let $C = U\Lambda U^\top$ be the eigendecomposition, with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$. The eigenvectors reveal the directions of maximal average variation; the eigenvalues quantify how much variation each direction captures. Each diagonal entry of C is the average squared sensitivity of f along that input direction, so eigenvectors of C are directions of *correlated* sensitivity: when the top m eigenvalues capture most of the trace, f varies primarily along their span and is approximately constant in the perpendicular $(d - m)$ -dimensional complement. If there is a clear *spectral gap*, i.e., $\lambda_m \gg \lambda_{m+1}$, then the first m eigenvectors $U_m = [u_1, \dots, u_m]$ span the *active subspace*, and f is well approximated by a function of the reduced coordinates $\tilde{\mathbf{x}} = U_m^\top \mathbf{x} \in \mathbb{R}^m$ alone.

$$f(\mathbf{x}) \approx g(U_m^\top \mathbf{x}), \quad U_m = [u_1, \dots, u_m]. \quad (9.5)$$

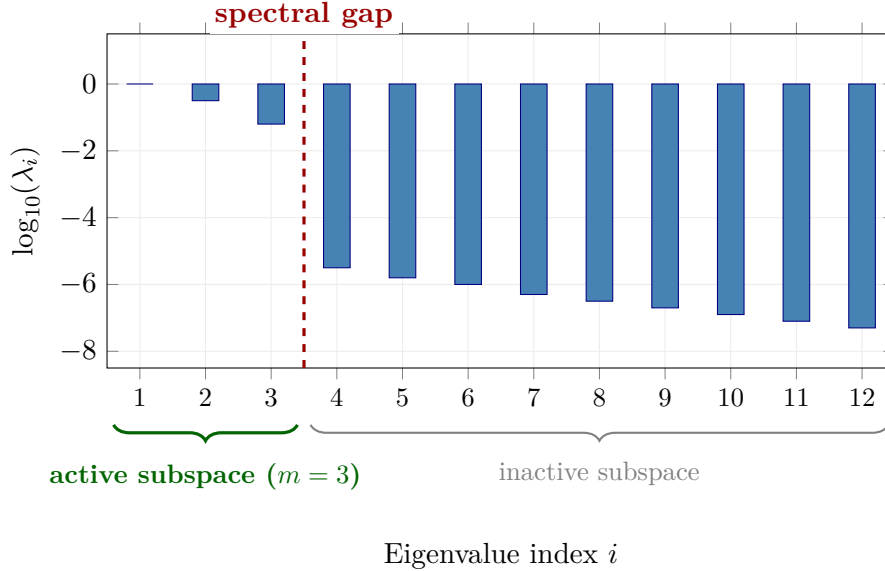


Figure 9.6: Spectral decay of the active-subspace eigenvalues for a schematic example with $d = 12$ state variables. The first three eigenvalues are orders of magnitude larger than the rest; the dashed red line marks the spectral gap after λ_3 , which indicates that f effectively lives on a 3-dimensional active subspace (green brace) and reduces the GP regression problem from $\mathbb{R}^{12}$ to $\mathbb{R}^3$. The remaining nine directions form the inactive subspace (gray brace), along which f is nearly constant on average.

Computing the gradient. In practice, the integral in (9.4) is estimated from a finite sample of gradient evaluations (Constantine, 2015, Ch. 1.5) (this is a one-time pre-processing cost, $\mathcal{O}(N_g d)$ with $N_g \sim 2d$ – $10d$, far smaller than the surrogate training set of size N):

$$\hat{C} = \frac{1}{N_g} \sum_{i=1}^{N_g} \nabla f(\mathbf{x}_i) \nabla f(\mathbf{x}_i)^\top, \quad \mathbf{x}_i \sim \pi(\mathbf{x}). \quad (9.6)$$

The gradient $\nabla f(\mathbf{x}_i)$ can be obtained via:

- **Automatic differentiation** through the model solver (if differentiable, e.g., a DEQN or PINN).
- **Finite differences:** evaluate f at $d + 1$ perturbed inputs per sample point. This costs $(d + 1) \cdot N_g$ model evaluations, but N_g can be modest (typically $N_g \sim 2d$ to $10d$ suffices for a reliable estimate of the leading eigenspace).

The eigendecomposition of $\hat{C}$ is a standard $d \times d$ matrix operation and is cheap relative to the model evaluations. When $\{\mathbf{x}_i\}$ are i.i.d. from π , $\hat{C} \rightarrow C$ almost surely as $N_g \rightarrow \infty$ by the law of large numbers, so the eigenvectors of $\hat{C}$ are consistent estimates of the population active subspace; finite-sample bias is documented in [Constantine \(2015\)](#).

Practical workflow. The full dimension-reduction pipeline consists of three steps:

1. **Gradient sampling:** Draw N_g points $\mathbf{x}_i \sim \pi(\mathbf{x})$ and compute $\nabla f(\mathbf{x}_i)$ at each. Form the empirical estimate $\hat{C}$ via (9.6).
2. **Eigendecomposition and dimension selection:** Compute $\hat{C} = \hat{U}\hat{\Lambda}\hat{U}^\top$. Inspect the eigenvalue decay, as in Figure 9.6, and choose m at the spectral gap.
3. **GP in reduced space:** Project all training inputs onto the active subspace: $\tilde{\mathbf{x}}_i = \hat{U}_m^\top \mathbf{x}_i \in \mathbb{R}^m$. Fit a standard GP on the m -dimensional projected data.

When combined with BAL, the GP adaptively selects training points in the reduced space, further improving sample efficiency.

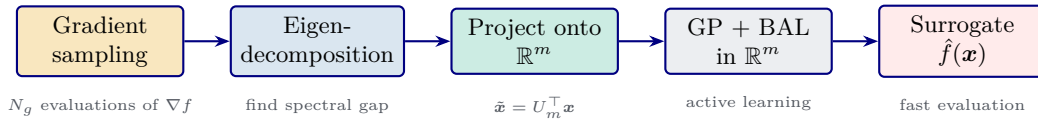


Figure 9.7: Linear active-subspace pipeline. Gradient samples identify the dominant eigenspace of the gradient outer-product matrix, all simulator inputs are projected to the reduced coordinates $\tilde{\mathbf{x}} = U_m^\top \mathbf{x}$, and the GP/BAL loop is then run in the low-dimensional active subspace.

Figure 9.7 summarizes the linear active-subspace workflow used before fitting the GP.

Application to dynamic stochastic economies. [Scheidegger and Bilonis \(2019\)](#) apply this pipeline to high-dimensional dynamic programming problems arising in neoclassical growth models. Their key findings:

- Models with up to $d = 500$ continuous state variables can be solved, far beyond the reach of grid-based methods or naïve GP implementations.
- The active subspace typically reduces the effective dimension to $m = 2\text{--}5$, even when d is in the hundreds. The value function’s dependence on the full state vector collapses onto a few aggregate quantities (e.g., mean capital, dispersion of productivity).
- The state space is partitioned into clusters via Bayesian Gaussian mixture models, and a separate GP with BAL is fit within each cluster. This local approximation strategy handles non-stationarities in the value function (e.g., different curvature in high- vs. low-capital regions).
- Parallel computing distributes the gradient evaluations and GP fits across multiple processors, enabling the method to scale to large-scale models on high-performance computing clusters.

Anchor on the multi-sector growth benchmark. In the multi-sector neoclassical growth benchmark of [Scheidegger and Bilonis \(2019\)](#), the active-subspace dimension collapses to $m = 1$ even at $D = 500$, so the entire ASGP pipeline operates on a one-dimensional projected coordinate. On models with other configurations the active subspace can be larger; the spectral-gap test is what decides.

9.5.1 Nonlinear Generalization: Deep Active Subspaces

The linear pipeline of the previous section assumes that the important directions are *linear* combinations of the input variables. That is an inductive bias: the first m columns of U always span a linear subspace of $\mathbb{R}^d$, so if the response f actually varies along a *curved* low-dimensional manifold (a ridge, a product of two linear features, a radial coordinate, a hidden regime indicator), the linear projection has to carry enough directions to cover the whole manifold, not the manifold itself. Concretely, if $f(\xi) = \varphi((w_1^\top \xi)^2 + (w_2^\top \xi)^2)$ is a radial function of two linear features, the gradient $\nabla f = 2\varphi'(\cdot)(s_1 w_1 + s_2 w_2)$ spans a *two*-dimensional space when averaged over ξ , so linear AS returns $m = 2$; yet the intrinsic “interesting” coordinate is the scalar aggregate $r^2 = s_1^2 + s_2^2$, which is one-dimensional. [Tripathy and Bilonis \(2018\)](#) remove the linearity bias by replacing the linear projection with a learned nonlinear encoder and thereby recover the one-dimensional structure.

The deep active-subspace ansatz. Tripathy and Bilonis parametrize the surrogate as a composition of two small neural networks,

$$\hat{f}(\xi) = g(h(\xi)), \quad h: \mathbb{R}^D \rightarrow \mathbb{R}^d, \quad g: \mathbb{R}^d \rightarrow \mathbb{R}, \quad (9.7)$$

where h is a multilayer perceptron (MLP) that plays the role of the projection matrix $U_m^\top$ in the linear case, and g is a second MLP that plays the role of the GP link. Setting $h(\xi) = U_m^\top \xi$ and taking g to be a polynomial recovers the linear active-subspace surrogate (9.5), so (9.7) generalizes the linear-AS ansatz rather than the gradient covariance matrix (9.4) itself. The two networks are trained *jointly* to minimize the residual $\hat{f}(\xi_i) - y_i$ on an input-output sample, so that the bottleneck $h(\xi) \in \mathbb{R}^d$ adapts to the specific response surface rather than being fixed in advance by the spectrum of C .

Architecture choices. Three design choices make (9.7) trainable with a few hundreds of samples:

- **Exponentially decaying encoder widths** ([Tripathy and Bilonis, 2018](#), Eq. 20). For an encoder with L layers mapping $\mathbb{R}^D$ to $\mathbb{R}^d$, choose widths

$$d_k = \lceil D e^{\eta_w k} \rceil, \quad \eta_w = \frac{1}{L} \log(d/D), \quad k = 0, 1, \dots, L, \quad (9.8)$$

so that the bottleneck closes smoothly from D to d without a brittle hyperparameter choice. For $D = 20$, $L = 3$, $d = 1$ this gives widths $[20, 8, 3, 1]$; for $d = 2$, widths $[20, 10, 5, 2]$ – a recipe rather than a search.

- **Swish activation** ([Tripathy and Bilonis, 2018](#), Eq. 10),

$$\sigma(z) = \frac{z}{1 + e^{-\gamma z}}, \quad \gamma = 1, \quad (9.9)$$

which is smooth everywhere (unlike ReLU) and non-saturating (unlike tanh); smoothness matters because we will want to differentiate through $\hat{f}$ for sensitivity analysis and because the elastic-net penalty below otherwise has to fight the sharp corners of ReLU.

- **Elastic-net regularization on every weight matrix** (Tripathy and Bilonis, 2018, Eq. 12). Writing $\theta = (\theta_h, \theta_g)$ for all encoder and link parameters, the training loss is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{f}_\theta(\xi_i) - y_i)^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2, \quad (9.10)$$

with small λ_1, λ_2 ; the ℓ_1 term encourages sparse weights and input usage, the ℓ_2 term controls the overall smoothness of $\hat{f}_\theta$ and prevents the loss landscape from becoming pathological as N shrinks.

Crucially, *no gradient samples of f* are required: θ is learned from the input-output pairs $\{(\xi_i, y_i)\}$ alone. The orthogonality constraint $U^\top U = I$ that is implicit in the linear case becomes unnecessary, because the learned encoder is free to pick any smooth low-dimensional parameterization of the active manifold.

Choosing the latent dimension d . The spectral gap of C is no longer available – the encoder is nonlinear – so d is chosen by a *validation-MSE elbow*: hold out an independent fraction of the sample, train a small family of models with $d = 1, 2, 3, \dots$, and pick the smallest d beyond which held-out error no longer drops significantly. An operational rule of thumb is to stop at the first d for which the MSE improvement from d to $d + 1$ is less than a factor of two: smaller gains are typically driven by optimization slack, not by new latent structure. On curved problems this elbow lies *strictly below* the linear-AS spectral gap: the deep encoder collapses two linear features into a single nonlinear aggregate (notebook 09_Deep_Active_Subspace_Ridge gives a reproducible instance in $D = 20$). On nearly-linear problems the two criteria agree qualitatively, and at small training-set sizes a polynomial link on top of a two-dimensional linear AS can in fact be more data-efficient than the deep encoder (notebook 10_Deep_AS_vs_Linear_AS_Borehole, the canonical borehole benchmark with $D = 8$ and $N = 500$). Figure 9.8 contrasts the elbow rule with the linear-AS spectral gap on the radial-ridge target.

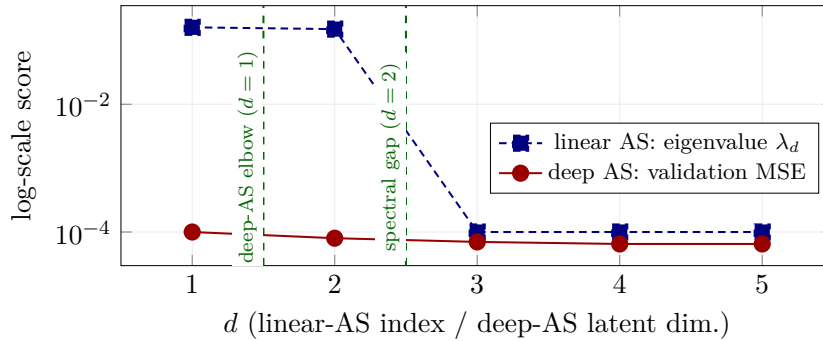


Figure 9.8: Stylized comparison of the two selection criteria for the radial-ridge target $y(\xi) = \exp(-[(w_1^\top \xi)^2 + (w_2^\top \xi)^2])$ in $D = 20$ (see notebook 09). The linear-AS eigenvalue spectrum has two dominant directions, so the spectral-gap criterion picks $d = 2$ (right green dashed line). The deep-AS validation MSE, by contrast, is already at its plateau at $d = 1$ (left green dashed line): the learned encoder h_θ represents the nonlinear aggregate $r^2 = s_1^2 + s_2^2$ as a *scalar*. The curves are stylized; the orders of magnitude track the notebook (linear-AS eigenvalues ≈ 0.16 for $i = 1, 2$ then a sharp drop; deep-AS validation MSE $\approx 10^{-4}$ at $d = 1$ and roughly flat thereafter), and the elbow-at- $d = 1$ versus spectral-gap-at- $d = 2$ contrast is reproduced.

Training recipe. A practical recipe that reproduces the experiments in notebooks 09 and 10:

1. Sample N input-output pairs (ξ_i, y_i) ; split 80/20 into train and validation sets. Standardize inputs to unit variance and center outputs.

2. For each candidate $d \in \{1, 2, 3, \dots\}$: build h_θ with widths (9.8) and g_θ with two hidden layers (16–32 units); train on loss (9.10) with Adam (lr = 5×10^{-3}), a cosine learning-rate schedule over 10^3 – 2×10^3 epochs, and $\lambda_1 = 10^{-5}$, $\lambda_2 = 10^{-4}$.
3. Record the validation MSE, apply the elbow rule, and deploy $\hat{f}_\theta$ with the chosen d .

Sample-budget rule of thumb: $N \approx 50 d_{\text{nl}}$ to find the bottleneck, inflated to $N \approx 200 d_{\text{nl}}$ for a deployment surrogate. Two post-training sanity checks are worth doing: (i) the validation curve should be monotone-then-flat in d ; (ii) a scatter of $h_\theta(\xi_i)$ against the top linear-AS coordinate $U_1^\top \xi_i$ reveals whether the encoder has actually gone nonlinear (a non-monotone relation is the fingerprint).

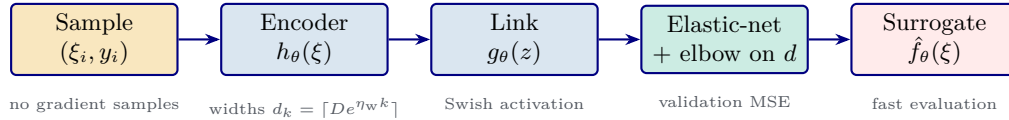


Figure 9.9: Deep active-subspace pipeline. Input–output pairs (ξ_i, y_i) are drawn directly from the simulator (no gradient samples needed); a nonlinear encoder h_θ compresses the high-dimensional input into a d -dimensional latent code, a small link network g_θ maps the latent code to the response, and an elastic-net / validation-MSE elbow chooses d . The trained composition $\hat{f}_\theta = g_\theta \circ h_\theta$ is the deployed surrogate. Compared with the linear active-subspace pipeline (Figure 9.7), the encoder + link boxes replace the eigendecomposition + linear projection, the elbow box replaces the spectral-gap search, and the gradient-sampling step disappears entirely.

Figure 9.9 should be compared with the linear pipeline in Figure 9.7: the two blue encoder / link boxes replace “eigendecomposition + project”, the “elastic-net + elbow” box replaces “find spectral gap”, and the gradient-sampling step is gone entirely. The economic pay-off is the same, a cheap surrogate in d variables where the original has D , but the modeling assumption is weaker: curved active manifolds are now admissible.

When is the extra machinery worth it? Deep AS pays off when (i) no gradient samples are available (e.g., black-box simulators, noisy observations from a lab experiment, or calibration targets returned by a proprietary solver), (ii) the spectral gap of C is ambiguous or the response surface genuinely depends on *nonlinear* aggregates of the inputs (ridges, radial coordinates, thresholded features, piecewise regimes), or (iii) the input dimension is large ($D \gg 10$) and a linear projection is too restrictive to capture the active manifold. When none of these conditions apply, linear AS plus a polynomial or a GP link is usually more data-efficient and should be fit first as a baseline. The borehole experiment in notebook 10 is the honest diagnostic: on a function this close to a $d = 2$ ridge, the two curves *cross*, and a cubic polynomial on two linear features beats the deep encoder at all $d \geq 2$. Deep AS’s advantage materializes precisely at $d = 1$, where a single linear feature cannot span the active direction. This perspective also connects naturally to deep kernel learning (Section 9.7), which composes a learned feature map with a GP head; deep AS is the same idea for the feature map, with an MLP link in place of the GP.

Applications in economics and finance. The deep-AS architecture is particularly attractive in settings where the state variable is high-dimensional but the economic mechanism acts through a few aggregate quantities that are not linear in the state. Examples include (i) heterogeneous-agent models whose cross-sectional distribution of wealth enters the equilibrium law of motion only through a few *moments* (mean, dispersion, tail mass), the Krusell-Smith aggregation logic of Chapter 6, where the map from the raw distribution to those moments is a learned, nonlinear encoder; (ii) multi-region climate-economy integrated assessment models with hundreds of regional capital stocks, in which damages and optimal carbon taxes depend on nonlinear

aggregates of the underlying state (total radiative forcing, regional inequality indices); and (iii) dynamic games and mean-field games in which the relevant action depends on a curved functional of the opponent distribution. In each case the original space is too large for a direct GP and the linear active subspace is too rigid, while the deep encoder provides a trainable compromise. [Bilionis and Zabarar \(2016\)](#) and [Tripathy and Bilionis \(2018\)](#) provide the general UQ and deep-active-subspace methodology; for integrated assessment models specifically, [Friedl et al. \(2023\)](#) is the economics-side reference used in this course.

9.6 Dynamic Programming with Gaussian Processes

Returning to the second oracle announced at the start of this chapter and developed formally in §9.6.1 below, we now embed the GP and active-subspace machinery developed above into a *value function iteration* (VFI) algorithm, with the Bellman operator TV playing the role of the expensive label generator. The acronym *ASGP* used in the section titles and figure captions below stands for *Active-Subspace Gaussian Process*: the dimensionality reducer of §9.5 stacked under the GP of §9.2.

The idea of representing the value function as a GP inside a DP recursion was introduced under the name *Gaussian process dynamic programming* (GPDP) by [Deisenroth et al. \(2009\)](#), who used it for continuous-state, continuous-action optimal control problems and combined it with a variance-based active selection of support points. Closely related work ([Engel et al., 2005](#)) embeds GPs into temporal-difference learning, replacing tabular value functions with a GP posterior. These ideas later inspired model-based policy search methods such as PILCO ([Deisenroth and Rasmussen, 2011](#)), which propagate GP uncertainty through long horizons. In economics, [Scheidegger and Bilionis \(2019\)](#) show that pairing this paradigm with active subspaces and Bayesian Gaussian mixture models for the ergodic set scales the framework to economies with up to 500 continuous state variables; [Renner and Scheidegger \(2018\)](#) apply it to dynamic incentive problems, [Gaegauf et al. \(2023\)](#) to dynamic portfolio choice, and [Chen et al. \(2025\)](#) to dynamic private asset allocation. The remainder of this section presents the algorithm in this combined form.

9.6.1 Why GPs for DP: a Supervised-Learning View of the Bellman Operator

Before stating the algorithm, it helps to see VFI through a supervised-learning lens. At iteration s , given an incumbent value function V^{s-1} , we generate a training set

$$\mathcal{D}^s = \{(\mathbf{x}^{(i)}, t_i^s)\}_{i=1}^{n^s}, \quad t_i^s = (TV^{s-1})(\mathbf{x}^{(i)}),$$

and fit a regressor $V^s \approx (TV^{s-1})$ to it. The label t_i^s is not free. Each evaluation of the Bellman operator at a state $\mathbf{x}^{(i)}$ requires solving a constrained nonlinear program over controls ξ subject to the law of motion and any feasibility / market-clearing constraints; quadrature over $\mathbf{x}'$ is taken inside. In textbook problems each oracle call costs 10^{-2} – 10^0 seconds, and in higher-dimensional models with adjustment costs, occasionally binding constraints, or aggregator nonlinearities, it can run into minutes. Because the calls are independent across $\mathbf{x}^{(i)}$, the design is embarrassingly parallel ([Scheidegger and Bilionis, 2019](#)), but each individual label remains expensive.

This is exactly the regime where Gaussian processes shine. Three properties make them the natural surrogate class:

1. **Sample efficiency under an expensive oracle.** The marginal-likelihood Occam’s razor of §9.2 delivers calibrated fits at $n \sim 10^2$ – 10^3 design points, well below the $n \gg 10^4$ regime in which deep-network surrogates start to dominate (Table 9.3).
2. **Built-in uncertainty quantification.** The GP posterior variance $\sigma_{\text{GP}}^2(\mathbf{x})$ tells us, at every state, how much the current surrogate trusts its own prediction. This is the input that turns a passive interpolant into an *adaptive* one (§9.6.6).

3. **Cheap held-out diagnostics.** The leave-one-out predictive error is available in closed form from the same Cholesky factor used for posterior inference (§9.6.5), so we can monitor surrogate health every iteration without spending oracle calls on a held-out split.

The same logic applies, with a different oracle, in the structural estimation chapter: Chapter 10 stacks a GP over the moment map $m(\theta)$ where each label is a forward simulation plus moment computation under a candidate parameter, again expensive to evaluate and cheap to store. Both settings are instances of one pattern: *supervised regression on the output of a costly numerical procedure*. The next subsection writes down the formal DP problem; the subsequent subsections develop the GP-based interpolant and the two ingredients (LOO and active learning) that make it sample-efficient at modest design size.

9.6.2 The Dynamic Programming Problem

Consider an infinite-horizon, discrete-time stochastic optimal decision problem. A representative agent chooses a sequence of controls $\{\xi_t\}_{t=0}^\infty$ to maximize the expected discounted sum of returns:

$$V(\mathbf{x}_0) = \max_{\{\xi_t\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t r(\mathbf{x}_t, \xi_t), \quad (9.11)$$

subject to the law of motion $\mathbf{x}_{t+1} \sim F(\cdot | \mathbf{x}_t, \xi_t)$, where $\mathbf{x}_t \in \mathcal{X} \subset \mathbb{R}^D$ is the state, $\xi_t \in \Lambda(\mathbf{x}_t)$ is the control, r is the per-period return function, and $\beta \in (0, 1)$ is the discount factor.

By the *principle of optimality* (Bellman, 1957), the infinite-dimensional problem (9.11) reduces to the *Bellman equation*:

$$V(\mathbf{x}) = \max_{\xi \in \Lambda(\mathbf{x})} \{r(\mathbf{x}, \xi) + \beta \mathbb{E}[V(\mathbf{x}')]\}, \quad (9.12)$$

where the expectation is over the stochastic transition. The *Bellman operator* T maps value functions to value functions:

$$(TV)(\mathbf{x}) = \max_{\xi \in \Lambda(\mathbf{x})} \{r(\mathbf{x}, \xi) + \beta \mathbb{E}[V(\mathbf{x}')]\}.$$

Under standard regularity conditions (bounded returns, $\beta < 1$, monotonicity, discounting), T is a *contraction mapping* on the Banach space of bounded continuous functions with sup-norm, with modulus β . By the Banach fixed-point theorem (Appendix D), T admits a unique fixed point V^* , and iterating $V^{s+1} = TV^s$ from any initial guess V^0 converges geometrically: $\|V^s - V^*\|_\infty \leq \beta^s \|V^0 - V^*\|_\infty$. The classical references in economics are Stokey et al. (1989, Chs. 3–4) for the formal contraction-and-fixed-point theory of the Bellman operator and Judd (1998, Ch. 12) for the numerical implementation in continuous-state settings; Ljungqvist and Sargent (2018, Chs. 3–4) and Sargent and Stachurski (2026) provide modern macroeconomic treatments. In the operations-research and optimal-control literature, Bertsekas (2000) is the canonical reference, with extensive coverage of approximate dynamic programming. The contraction modulus β applies to the *exact* Bellman operator; convergence of the GP-fitted iterates additionally requires controlling the per-step interpolation error of the surrogate, otherwise the approximate operator can fail to contract globally even though the exact one does. This is the standard textbook VFI loop: guess a value function, apply the Bellman operator, interpolate the updated values, and repeat until convergence. In the present chapter, the only change is the interpolant: a GP replaces the grid or polynomial basis when the state space is irregular or moderately high-dimensional. For a comprehensive treatment of dynamic programming in economics, see also the open-source QuantEcon lectures.¹

¹<https://dp.quantecon.org/>

The computational challenge. At every VFI iteration, we must *approximate* V^{s+1} as a function of the full state vector $\mathbf{x} \in \mathbb{R}^D$. Traditional approaches (finite grids, Smolyak sparse grids, tensor-product polynomial bases) suffer from the curse of dimensionality: the number of grid points grows exponentially in D , and they require hypercubic state-space geometries. For $D > 20$, these methods become infeasible.

9.6.3 The Stochastic Optimal Growth Model

The workhorse test case for GP-based dynamic programming is the multi-sector stochastic optimal growth model. Assume D sectors, each with a capital stock k_j , so the state is $\mathbf{k} = (k_1, \dots, k_D) \in \mathbb{R}_+^D$. A representative household chooses consumption $\mathbf{c}$, labor supply $\mathbf{l}$, and investment $\mathbf{i}$ to maximize:

$$V_0(\mathbf{k}_0) = \max_{\{\mathbf{c}_t, \mathbf{l}_t, \mathbf{i}_t\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(\mathbf{c}_t, \mathbf{l}_t), \quad (9.13)$$

subject to the capital law of motion $k_{j,t+1} = (1 - \delta) k_{j,t} + i_{j,t}$, a sector-by-sector resource constraint $c_{j,t} + i_{j,t} + \Gamma_{j,t} = \exp(z_{j,t}) A k_{j,t}^\psi l_{j,t}^{1-\psi}$ with convex adjustment cost $\Gamma_{j,t}$, and Cobb–Douglas production $f(k_j, l_j) = A k_j^\psi l_j^{1-\psi}$. Productivity shocks $z_{j,t}$ follow a stationary process and the dimension D can be scaled from 1 (the textbook Brock–Mirman model of Chapter 2) to 500 without changing the algorithmic structure.

9.6.4 GP-Based Value Function Iteration (ASGP)

The key idea, common to Deisenroth et al. (2009) in the control literature and Scheidegger and Bilonis (2019) in the economics literature, is to use a *Gaussian process as the interpolation scheme* inside VFI, replacing grid-based methods entirely. At each VFI step s :

1. Generate n^s training inputs $\mathbf{X} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n^s)}\} \subset [\underline{\mathbf{k}}, \bar{\mathbf{k}}]^D$.
2. Evaluate the Bellman operator at each training point: $t_i^s = (TV^{s-1})(\mathbf{x}^{(i)})$.
3. Learn a GP (or ASGP) surrogate V_{sur} from the training data $\{\mathbf{X}, \mathbf{t}\}$.
4. Set $V^s = V_{\text{sur}}$ (the GP posterior mean).
5. Compute the convergence error $e^s = \|V^s - V^{s-1}\|_\infty / \Delta_V$, where Δ_V is a normalizing value-function range.
6. If $e^s < \bar{e}$, stop; otherwise continue.

Advantages over grid-based methods. This GP-VFI approach offers several structural advantages:

- **Arbitrary geometries:** GPs require no tensor-product structure, so training points can be placed anywhere in the state space, including on irregularly shaped ergodic sets.
- **Adaptive training:** Points where the Bellman operator fails to converge (e.g., near constraints) can simply be excluded and retried later; adding or removing points is trivial.
- **Built-in uncertainty quantification:** The GP posterior variance provides interpolation uncertainty at every state point, at every iteration, “free” UQ.
- **Active subspace integration:** When $D \gg 10$, the GP operates on the projected inputs $\tilde{\mathbf{x}} = U_m^\top \mathbf{x} \in \mathbb{R}^m$ discovered via the active subspace pipeline (Section 9.5), reducing the effective dimensionality from hundreds to a handful.

Parallelization. The most expensive part of the algorithm, evaluating the Bellman operator at n^s training points, is *embarrassingly parallel*: each evaluation is independent. In the MPI implementation of [Scheidegger and Bilonis \(2019\)](#), the current value function surrogate is broadcast to all workers, each worker evaluates the Bellman operator at n^s/n_{cpu} points, and the results are gathered at the master for GP fitting. Communication cost is negligible relative to the Bellman evaluations.

9.6.5 Leave-One-Out Error: a Held-out Diagnostic for Free

How do we know that the GP surrogate at iteration s is good enough to take another Bellman step? The marginal-likelihood objective of §9.2 fits the kernel, but it does not, on its own, certify pointwise predictive accuracy on the design. A held-out validation split would, but every held-out point is one fewer expensive Bellman label going into the GP.

The standard escape route in GP regression is the leave-one-out (LOO) predictive error, and it has the pleasant feature that for a Gaussian process it admits a closed form using the same Cholesky factor already computed for the posterior ([Rasmussen and Williams, 2005](#), §5.4.2):

$$\mu_{-i}(\mathbf{x}^{(i)}) - y_i = -\frac{\alpha_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}^{(i)}) = \frac{1}{[K_y^{-1}]_{ii}}, \quad (9.14)$$

where μ_{-i} and σ_{-i}^2 denote the posterior mean and variance after removing the i -th observation, and $\alpha = K_y^{-1}(\mathbf{y} - \boldsymbol{\mu}_X)$ (for centered outputs, $\boldsymbol{\mu}_X = 0$). Since K_y^{-1} is recovered from the Cholesky factor of K_y in $\mathcal{O}(n^2)$ once the factorization has been done, computing the full n -vector of LOO residuals is essentially free relative to the $\mathcal{O}(n^3)$ already paid for posterior inference.

What the LOO RMSE tells us. Tracking

$$\text{LOO-RMSE}(\mathcal{D}^s) = \sqrt{\frac{1}{n^s} \sum_{i=1}^{n^s} (\mu_{-i}(\mathbf{x}^{(i)}) - t_i^s)^2}$$

across VFI iterations is a cheap surrogate-health metric that is independent of the Bellman residual:

- A flat-then-rising LOO curve at the same design size signals *kernel mis-specification* (length scale collapsing, noise variance hitting a bound) and tells us to revisit the kernel choice or hyperparameter bounds before adding more design points.
- A high LOO RMSE at small n^s that decays as the design grows is the expected behavior and tells us that the surrogate simply needs more labels.
- A small LOO RMSE coexisting with a large Bellman residual points the finger at the *operator*, not the surrogate: the iterate may be far from the fixed point even though the GP fits the current TV^{s-1} well.

The notebook `04_GP_Value_Function_Iteration.ipynb` computes (9.14) via `scipy.linalg.cho_solve` (function `gp_loo_rmse`) and reports it alongside the Bellman residual at every VFI iteration, separating these two failure modes. The same diagnostic reappears in §10.7 for the GP layer over the SMM moment map.

9.6.6 Active Learning Inside the VFI Loop

The Bayesian active-learning machinery of §9.3 carries over almost verbatim once the VFI loop is in place, but with one important adjustment: the goal is now *uniform interpolation accuracy* of the value function on the relevant state-space region, not maximization of a payoff

or minimization of a loss. The right acquisition function is therefore pure exploration, the GP posterior standard deviation, rather than a UCB or expected-improvement criterion that trades off exploitation and exploration:

$$\mathbf{x}^{\text{next}} \in \arg \max_{\mathbf{x} \in \mathcal{X}^{\text{cand}}} \sigma_{\text{GP}}^s(\mathbf{x}). \quad (9.15)$$

A practical implementation, used in notebook `04_GP_Value_Function_Iteration.ipynb`, runs the VFI iterations with a frozen design for a few steps, then *enriches* the design every N iterations:

1. Evaluate σ_{GP}^s on a dense candidate set $\mathcal{X}^{\text{cand}}$ (a Latin-hypercube draw over the current state-space region).
2. Pick the top- n_{add} candidates by posterior standard deviation, subject to a minimum-spacing constraint $\|\mathbf{x}^{(\text{new})} - \mathbf{x}^{(j)}\| \geq \delta_{\text{spacing}}$ against existing design points to avoid clustering.
3. Evaluate the Bellman operator at the new points (one expensive oracle call each, in parallel) and append them to $\mathcal{D}^s$.
4. Refit the GP and continue iterating.

Why pure exploration here. A UCB-style acquisition would bias the design toward states with high *value*, which is not what we want when the surrogate is a building block of an iteration. We want the GP posterior to be uniformly tight wherever the Bellman operator might be evaluated, so that the contraction modulus of the *approximate* operator stays close to β . This is structurally different from the optimization setting of Bayesian optimization, where exploitation is a feature.

Empirical impact. The companion notebook compares a same-budget fixed Latin-hypercube design with an active design inside the one-dimensional GP-VFI loop. Both designs use Bellman labels; the active design starts from a small initial set and adds states by maximizing the GP posterior standard deviation (9.15) subject to a spacing rule. At the same final number of labels, the active design lowers posterior uncertainty and achieves a comparable or smaller dense-grid Bellman residual (Figure 9.10). The figure is one-dimensional by design: the goal is to show active enrichment inside a genuine Bellman iteration, not to use a separable interpolation toy as a proxy for multidimensional dynamic programming.

9.6.7 Results: From One to 500 Dimensions

The companion notebook deliberately stops at the one-dimensional Bellman problem, where every numerical object can be inspected directly. The high-dimensional claims below come from the ASGP implementation of Scheidegger and Bilonis (2019), where the GP operates on an active subspace rather than on the full tensor-product state space; we report their findings as a literature summary, not as something the notebook itself demonstrates.

Scheidegger and Bilonis (2019) apply the ASGP-VFI algorithm to the stochastic optimal growth model (9.13) across a range of dimensions, from $D = 1$ (the textbook Brock–Mirman case) to $D = 500$ continuous state variables. The key findings are:

1. **Convergence is dimension-independent in the active subspace:** all models converge to a normalized error below 10^{-4} within approximately 70 VFI iterations regardless of D , where the iteration count is dimension-independent in the m -dimensional active subspace on which the GP actually operates rather than in the full ambient $\mathbb{R}^D$.
2. **Low-dimensional structure:** the active subspace dimension is $m = 1$ in all cases tested, so the value function depends on a single linear combination of the D capital stocks, confirming that the high-dimensional problem has intrinsic low-dimensional structure.

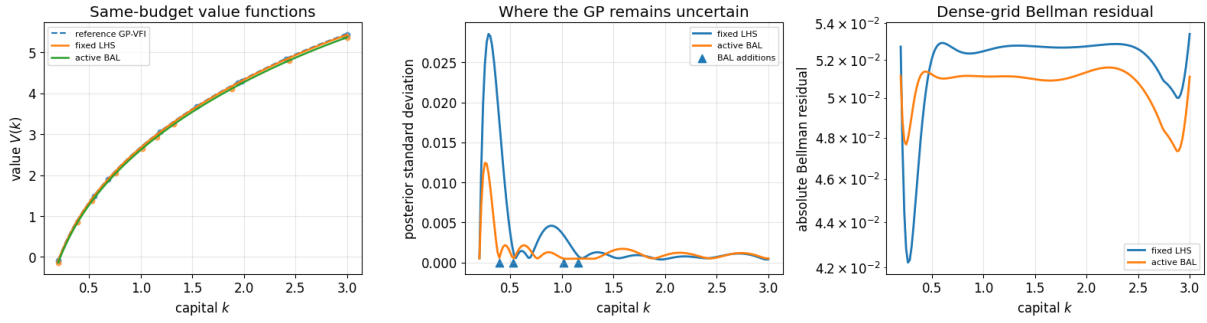


Figure 9.10: Same-budget active enrichment inside one-dimensional GP value-function iteration. The left panel compares the GP posterior means from a fixed Latin-hypercube design and an active design against a reference GP-VFI solution. The middle panel shows the posterior standard deviation and the states added by the active rule (9.15), marked as triangles on the horizontal axis. The right panel reports the dense-grid Bellman residual. Unlike the previously used two-dimensional separable interpolation benchmark, every plotted training value here is generated by a Bellman maximization. Generated by notebook `lecture_14_04_GP_Value_Function_Iteration.ipynb`.

3. **Sub-exponential scaling:** CPU time grows sub-exponentially with D (concave on a log scale when parallelized across compute nodes); on the benchmarks reported in Scheidegger and Bilonis (2019), $D = 500$ is solved at sub-exponential cost in D when distributed across a compute cluster. This avoids the explicit grid-based curse but does not eliminate all high-dimensional sample/optimization costs.
4. **Beyond the reach of grids:** grid-based methods such as Smolyak sparse grids cannot handle $D > 100$, making ASGP one demonstrated grid-free approach for models at this scale; neural residual methods such as DEQNs are complementary alternatives when the equilibrium conditions are easier to express directly.

9.6.8 Uncertainty Quantification and Ergodic Sets

Free UQ from the GP posterior. Because the value function at each VFI iteration is a GP, the posterior variance $\sigma^2(\mathbf{x})$ provides a built-in measure of approximation uncertainty. This gives pointwise credible bands for the interpolated value function essentially for free. Policy uncertainty is not automatic in the same sense: it must be propagated through the Bellman maximization, for example by evaluating policies under posterior draws or local approximations to the GP posterior.

Parameter uncertainty propagation. Uncertain model parameters can be treated as additional pseudo-states. For example, extending the state to $\tilde{S} = (\mathbf{k}, \gamma)$ where $\gamma \in [\underline{\gamma}, \bar{\gamma}]$ is the risk aversion parameter allows solving the model once on the extended state space and then extracting univariate effects and global sensitivity indices *in a single computation*, avoiding the traditional approach of re-solving the model for each parameter value. Harenberg et al. (2019) give a general introduction to uncertainty quantification for economic models, covering univariate-effect plots and Sobol’ sensitivity indices.

Learning ergodic sets. Many economic models have irregularly shaped ergodic sets (ellipsoids or manifolds, not hypercubes). Scheidegger and Bilonis (2019) propose learning the ergodic

distribution via Bayesian Gaussian mixture models: (i) solve the model on a large initial domain; (ii) simulate the economy forward to generate capital paths $\{\mathbf{k}_t^+\}$; (iii) fit a mixture of Gaussians $\hat{\rho}_{\text{ergodic}}$ to the simulated paths; (iv) re-solve on the ergodic set by sampling training points from $\hat{\rho}_{\text{ergodic}}$ instead of the full hypercube. GPs handle this naturally because they require no grid structure.

9.6.9 Comparison: GPs vs. DEQNs for Solving Dynamic Models

Both GPs (this chapter) and DEQNs (Chapters 2–5) solve dynamic stochastic models, but with different trade-offs (Table 9.2):

Criterion	GP / ASGP	DEQNs
Solution method	Value function iteration	Euler equation residuals
Dimensionality	Up to ~ 500 (with AS)	>1000 feasible
UQ built-in	Yes (GP posterior variance)	No (needs extra work)
Hardware	CPU clusters (MPI)	GPUs (TensorFlow/PyTorch)
Irregular domains	Yes (grid-free)	Yes (mesh-free)
Sensitivity analysis	Cheap after pseudo-state augmentation	Requires pseudo-states or re-training
Convergence diagnostics	Exact Bellman operator is a contraction; GP interpolation error must be controlled	Euler residuals and simulation tests; no generic global proof

Table 9.2: GP/ASGP and DEQN solvers for dynamic models. The table distinguishes exact fixed-point theory from the numerical approximation actually used: GP-VFI inherits the Bellman contraction only up to interpolation error, while DEQNs are judged by residual and simulation diagnostics.

When to use which: GPs when the effective dimension is moderate ($D \lesssim 15$, or a few hundred only when active-subspace structure is strong) and uncertainty quantification or sensitivity analysis is required; DEQNs when D is very large, GPU hardware is available, or the model involves complicated market-clearing conditions that are more naturally expressed as Euler equation residuals than as Bellman maximization.

Hands-on

Notebook `04_GP_Value_Function_Iteration.ipynb` implements GP-based VFI for the one-dimensional stochastic growth model. It shows convergence of the GP-VFI outer loop, posterior credible bands for the value-function interpolant, Cholesky-based LOO-RMSE (9.14), dense-grid Bellman residuals, active enrichment of the Bellman design (9.15), policy recovery, and a deterministic full-depreciation verification ($\delta = 1$ and $\sigma = 0$) against the closed-form Brock–Mirman solution. The multidimensional ASGP extension is discussed in the literature summary above and illustrated separately by the active-subspace notebooks, `05_Active_Subspace_2D.ipynb`, `06_Active_Subspace_10D.ipynb`, and `07_Active_Subspace_Nonlinear.ipynb`, on 2D, 10D, and nonlinear test functions.

9.7 Deep Kernel Learning

Standard GP kernels such as the RBF or Matérn operate on *raw* input features: the covariance $k(\mathbf{x}, \mathbf{x}')$ depends on $\|\mathbf{x} - \mathbf{x}'\|$, which may be a poor measure of similarity when the inputs are high-dimensional or when the function’s structure depends on complex nonlinear interactions among variables. *Deep Kernel Learning* (DKL) (Wilson et al., 2016) addresses this limitation

by combining the representation-learning power of deep neural networks with the probabilistic framework of GPs.

The DKL architecture. A DKL model consists of two components:

1. A **feature extractor** $\phi(\mathbf{x}; \boldsymbol{\theta}_{\text{NN}}): \mathbb{R}^D \rightarrow \mathbb{R}^d$, typically a multi-layer perceptron that maps the raw input $\mathbf{x}$ to a learned representation of (usually much lower) dimension d .
2. A **GP layer** that places a GP prior on the function $g(\mathbf{z}) = g(\phi(\mathbf{x}; \boldsymbol{\theta}_{\text{NN}}))$, with a standard base kernel $k_{\text{base}}(\mathbf{z}, \mathbf{z}')$ (e.g., RBF or Matérn) operating in the learned feature space.

The *deep kernel* is thus:

$$k_{\text{DKL}}(\mathbf{x}, \mathbf{x}') = k_{\text{base}}(\phi(\mathbf{x}; \boldsymbol{\theta}_{\text{NN}}), \phi(\mathbf{x}'; \boldsymbol{\theta}_{\text{NN}})). \quad (9.16)$$

Unlike a standard kernel that computes distance in the input space, the DKL kernel computes distance in a *learned* feature space. If the neural network learns to map functionally similar inputs close together, the GP can exploit this structure for better predictions and more calibrated uncertainty estimates.

Joint training. The parameters of both the neural network ($\boldsymbol{\theta}_{\text{NN}}$) and the GP hyperparameters (ℓ, σ_f, σ_y of k_{base}) are optimized jointly by maximizing the GP marginal likelihood:

$$\max_{\boldsymbol{\theta}_{\text{NN}}, \boldsymbol{\vartheta}} \log p(\mathbf{y} | \Phi(\mathbf{X}; \boldsymbol{\theta}_{\text{NN}}), \boldsymbol{\vartheta}), \quad (9.17)$$

where $\Phi(\mathbf{X}; \boldsymbol{\theta}_{\text{NN}})$ is the matrix of transformed features. This end-to-end training procedure automatically learns features that are useful for the GP, without requiring manual feature engineering. In practice, DKL is implemented via GPyTorch (Gardner et al., 2018), which provides efficient GPU-accelerated GP inference and integrates seamlessly with PyTorch for the neural network component.

When DKL helps. DKL is most beneficial when:

- The input space is high-dimensional but the function has low-dimensional structure that is *nonlinearly* embedded (active subspaces find *linear* structure; DKL can find nonlinear features).
- Sufficient training data is available to train the feature extractor without overfitting (DKL has more parameters than a standard GP).
- Uncertainty quantification is important, but a standard GP kernel is insufficiently expressive to capture the true function’s covariance structure.

The trade-off relative to a standard GP is clear: DKL offers greater expressiveness at the cost of more parameters and the risk of overfitting with very small datasets. For the moderate-data regimes typical of economic surrogate models ($N \sim 100\text{--}1000$), DKL can offer significant improvements over standard kernels, particularly when the target function has complex multi-scale behavior. Chen et al. (2025) apply learned-feature-map GP architectures in a dynamic model of private asset allocation, where the feature map captures the complex nonlinear interactions between illiquidity, portfolio composition, and optimal rebalancing.

Illustrative examples. Two simple examples highlight when DKL provides a qualitative advantage over standard kernels:

- **Step functions.** Consider approximating a 1D function with a sharp discontinuity. A standard GP with an RBF kernel necessarily oversmooths near the jump and yields poorly calibrated uncertainty bands. A DKL model, by contrast, can learn a feature map that “compresses” the input near the discontinuity, effectively sharpening the GP’s resolution where it matters most. This is directly relevant to economic applications with occasionally binding constraints, where policy functions exhibit kinks or jumps.
- **Anisotropic boundaries in 2D.** Standard stationary kernels (RBF, Matérn) are isotropic: they measure distance with circular level sets. When the target function has a discontinuity along a *diagonal* or curved boundary, common in portfolio problems with no-trade regions or in models with regime-dependent policies, the isotropic kernel cannot adapt. DKL learns a nonlinear coordinate transformation that aligns the kernel’s smoothness assumptions with the function’s actual structure, capturing diagonal and curved boundaries that would require an impractical number of training points with a standard kernel.

Hands-on

The companion notebook `08_Deep_Kernel_Learning.ipynb` implements a simplified DKL pipeline (a supervised feature extractor stacked with a scikit-learn GP head) and compares the learned deep kernel against standard RBF and Matérn GPs on function approximation tasks; the full GPyTorch joint marginal-likelihood training of (9.16) is left as an extension.

9.8 GPs Among Their Bayesian Cousins

Gaussian processes are the most analytically transparent way to attach uncertainty to a non-parametric regressor, but they are not the only one. For completeness, this section briefly situates the GP machinery against two neural-network-based alternatives that come up frequently in modern uncertainty-aware deep learning.

Monte-Carlo dropout. Gal and Ghahramani (2016) show that a deep network trained with dropout and *evaluated* with dropout still active can be interpreted as approximate variational inference over a particular Bayesian neural network. Predictive uncertainty is obtained by averaging T stochastic forward passes; the cost is one extra forward pass per sample. Calibration is rougher than a GP’s, but the method requires no architectural change and scales to deep nets and large n .

Deep ensembles. Lakshminarayanan et al. (2017) train E independent neural networks with different random seeds and combine them into a Gaussian-mixture predictor. Empirically this is one of the most robust uncertainty-quantification recipes available, often beating MC dropout and approaching the calibration of GPs, at E times the training cost.

A decision rule for practice. In our experience the right method follows from the application: (i) plain GPs for moderate d ($\lesssim 10$ – 20) with a smooth target and an expensive simulator, when calibrated uncertainty is the goal; (ii) deep kernels (Wilson & al., 2016) when the input geometry is non-trivial (regime switches, manifold structure, image-like inputs); (iii) deep ensembles or MC dropout for high- d regression where calibrated uncertainty is desirable but exact GP inference is infeasible; (iv) sparse GPs (Titsias (2009); Hensman et al. (2013)) for $n \gtrsim 10^4$ when the target stays smooth. The summary frame in the companion deck (“Toolbox: When to Use What”) gives the same decomposition visually.

	Gaussian process	MC dropout	Deep ensembles
Calibration	exact under model	approximate	strong empirically
Training cost	$\mathcal{O}(n^3)$	one network	$E \times$ one network
Inference cost	$\mathcal{O}(n^2)$	T forward passes	E forward passes
Sample efficiency	best at small n	needs much more data	needs much more data
Best when	$n \lesssim 10^4$, low d	cheap UQ on existing nets	willing to pay $E \times$ for top calibration
Reference	Rasmussen and Williams (2005)	Gal and Ghahramani (2016)	Lakshminarayanan et al. (2017)

Table 9.3: Three uncertainty-quantification recipes compared on the dimensions that drive method choice in economic surrogate work. This script uses GPs in Chapter 9 and Chapter 11 because the typical regime ($n \lesssim 10^3$, expensive simulator) plays to their strengths; readers operating in larger-data regimes should consider MC dropout or deep ensembles before resorting to a sparse GP.

Chapter Summary

- Gaussian processes attach calibrated uncertainty to non-parametric regression at $\mathcal{O}(n^3)$ cost; the marginal likelihood implements an automatic Occam’s razor that chooses model complexity without held-out validation.
- Active subspaces collapse d -dimensional inputs to $m \ll d$ via the gradient outer product; this is the trick that makes GPs viable past $d \sim 10$.
- Bayesian active learning closes the surrogate loop: train, evaluate uncertainty, request next design point where it’s largest, repeat. Scheidegger and Bilonis (2019) provide one canonical example.
- Deep kernel learning composes a NN feature extractor with a GP head; an alternative to plain GPs and to deep ensembles.

Further Reading

- Rasmussen and Williams (2005), the standard GP textbook.
- Constantine (2015), the active-subspaces monograph.
- Renner and Scheidegger (2018); Scheidegger and Bilonis (2019), GP+BAL methodology and applications in economics.
- Wilson et al. (2016), deep kernel learning.
- Titsias (2009); Hensman et al. (2013), sparse-GP scaling.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. [Core] **Posterior on three points.** Fit a GP with RBF kernel ($\ell = 1$, $\sigma_f = 1$, $\sigma_y = 0.1$) to three points $(0, 0)$, $(1, 0.8)$, $(2, 0.3)$. Compute the posterior mean and variance at $x^* = 1.5$ in closed form.

2. **[Core] Marginal likelihood Occam.** For the same three points, plot the log marginal likelihood as a function of the length scale $\ell \in [0.1, 5]$. Identify the optimum and explain it in terms of the data-fit / complexity decomposition.
3. **[Core] Active subspace by hand.** For $f(\mathbf{x}) = (x_1 + x_2 + x_3)^2 + 0.01(x_1 - x_2)^2$ on $[-1, 1]^3$, compute the gradient outer product matrix $\hat{C}$ on a uniform sample. Show that the leading eigenvector identifies the “aggregate” direction $(1, 1, 1)/\sqrt{3}$.
4. **[Computational] Deep vs. linear active subspace on a radial ridge.** Take the target $y(\xi) = \exp(-[(w_1^\top \xi)^2 + (w_2^\top \xi)^2])$ with $\xi \sim \mathcal{N}(0, I_{20})$ and $w_1, w_2 \in \mathbb{R}^{20}$ fixed orthonormal. (i) Compute $\hat{C}$ from 4000 samples and confirm that it has two nonzero eigenvalues. (ii) Train the Tripathy–Bilonis surrogate (9.7) with widths (9.8), Swish activation (9.9), and elastic-net loss (9.10) for $d \in \{1, 2, 3, 4\}$; plot held-out MSE on a log scale and identify the elbow. (iii) Explain why linear AS needs $d \geq 2$ while deep AS already captures the response at $d = 1$ (cf. notebook 09_Deep_Active_Subspace_Ridge).
5. **[Computational] BAL on a 2D function.** Modify notebook 02_GP_and_BAL so that one acquisition uses pure variance, $U_{\text{var}}(\mathbf{x}) = \sigma^2(\mathbf{x})$, and another uses the mixed log-variance score $U_{\text{mix}}(\mathbf{x}) = w_{\text{obj}}\mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ with $w_{\text{obj}} > 0$. Compare the resulting designs and comment on which gives smoother domain coverage. Explain why pure $\sigma^2(\mathbf{x})$ and pure $\log \sigma^2(\mathbf{x})$ have identical maximizers.
6. **[Computational] Sobol sensitivity with a GP surrogate.** Write a short script, or extend notebook 02_GP_and_BAL.ipynb, to train a GP surrogate on the 4-dimensional Genz product-peak function $f(\mathbf{x}) = \prod_{i=1}^4 (c_i^{-2} + (x_i - w_i)^2)^{-1}$ on $[0, 1]^4$ with $c = (1, 2, 0.5, 1.5)$, $w = (0.4, 0.6, 0.3, 0.7)$, using $N \in \{50, 100, 200\}$ training points. For each N , compute the Sobol first-order and total-effect indices in two ways: (i) direct on the true f via 10,000 Monte Carlo samples (reference), (ii) on the GP surrogate via 10^6 samples (cheap thanks to the surrogate). Plot the relative error in each Sobol index against N . Verify that the surrogate-based Sobol estimates converge to the reference as N grows, and identify the N at which all four first-order indices match the reference within 5%. Discuss why surrogate-based sensitivity analysis is the workhorse for expensive simulators (climate models, structural macro models) where direct 10^6 -sample Monte Carlo is infeasible.
7. **[Core] Prior-driven RBF-GP extrapolation outside the training domain.** Train a GP with RBF kernel on 20 points sampled uniformly from $[0, 1]$ from the function $f(x) = \sin(2\pi x)e^{-x}$. (i) Plot the posterior mean and ± 2 standard deviation band on the training interval $[0, 1]$; verify the posterior tracks the true function and the band is narrow. (ii) Now extrapolate to $x \in [1.5, 3.0]$ and plot the posterior mean and band over the extended interval. Show analytically that for an RBF kernel with length scale ℓ , the posterior at x far from any training point ($|x - x_i| \gg \ell$ for all i) reverts to the prior: posterior mean $\rightarrow 0$, posterior variance $\rightarrow \sigma_f^2$. (iii) Verify numerically: at $x = 3$, the posterior mean is essentially zero (independent of the training data), and the posterior standard deviation has expanded back to the prior σ_f . (iv) Discuss the implication: far from the training domain the posterior reverts to the prior, so the variance band there reflects the learned hyperparameters rather than the data, and the band is overconfident only when the prior variance or the learned length scale is itself misleading. Naive Bayesian active learning that relies on posterior variance may therefore fail to acquire informative samples outside the convex hull when the prior variance is no larger than the inside-hull noise scale. Mitigations: use a Matérn- ν kernel with smaller ν (heavier-tailed), bound the analysis to the convex hull of the training data, or use a boundary-aware acquisition function.

Chapter 10

Deep Surrogate Models and Structural Estimation

With the GP machinery of Chapter 9 in hand, this chapter builds the practical surrogate-driven workflows that make structural estimation tractable at research scale. We start with the *deep-surrogate pseudo-state pattern*, in which the structural parameter θ is concatenated into the network input so that a single trained policy net replaces one full model re-solve per θ with one forward pass. We illustrate the pattern on a Black–Scholes example, then put it to work for Simulated Method of Moments (SMM) on the Brock–Mirman growth model, layer a Gaussian-process surrogate on top of the moment map for high-throughput bootstrap and Bayesian post-processing, and close with the natural extensions to indirect inference and simulation-based inference. The econometric foundations are [McFadden \(1989\)](#), [Pakes and Pollard \(1989\)](#), [Lee and Ingram \(1991\)](#), [Duffie and Singleton \(1993\)](#), and [Gourieroux et al. \(1993\)](#); the surrogate logic follows the deep-surrogate and GP-surrogate pipelines in [Chen et al. \(2026\)](#) and [Scheidegger and Bilonis \(2019\)](#). Recent applications of the same surrogate-then-estimate move include heterogeneous-agent estimation ([Kase et al., 2022](#)), search-and-matching ([Payne et al., 2025](#)), and climate-economy policy design and uncertainty quantification ([Kübler et al., 2026](#); [Friedl et al., 2023](#)).

Before the current deep-learning boom, neural networks were already studied as nonlinear *sieve* estimators in econometrics, with a rigorous asymptotic theory developed in parallel with the approximation-theory results of Chapter 1. [Chen and White \(1999\)](#) establish convergence rates and asymptotic normality for single-hidden-layer network estimators, and [Chen \(2007\)](#) integrates that line into the broader sieve treatment of semi-nonparametric models defined by conditional moment restrictions, which is precisely the structural-estimation setting of this chapter. The modern continuation of this program uses deep architectures for efficient estimation in nonparametric instrumental-variable models ([Chen et al., 2021](#)). The pipelines developed below should be read as the implementation-side companion to that theoretical tradition: the sieve literature tells us *when* neural-network estimators consistently identify deep structural parameters; the surrogate pipelines tell us *how* to make the resulting estimators cheap enough to deploy at research scale.

Two ideas hold this chapter together

1. **The pseudo-state trick.** Concatenate the structural parameter θ into the network input, $(s, \theta) \mapsto \mathcal{N}_\rho(s, \theta)$, and train one network that encodes a *family* of policies indexed by θ . Each new θ evaluation then requires a single forward pass through the trained network, not a re-solve of the dynamic program; this is what makes SMM with a deep structural model tractable.
2. **Two-layer surrogates.** Stack a Gaussian-process surrogate on top of the policy net (§10.7): Layer 1 (the policy net) turns “one re-solve per θ ” into “one forward simulation per θ ”, and Layer 2 (a GP per moment) turns “one forward simulation per θ ” into “one GP posterior per θ ”. The result is a microseconds-per-call moment map, enabling bootstrap, Bayesian post-processing, and policy search at scale.

10.1 Motivation: The Computational Bottleneck

Every workflow this chapter targets puts an expensive numerical solve inside an outer loop. For estimation, uncertainty quantification, and optimal policy design, the outer loop runs over a parameter or scenario vector θ and the inner solve is a full model solution, a Bellman fixed point, a PDE solve, or a Monte Carlo run that costs seconds to hours, repeated at the 10^3 to 10^6 outer iterations these tasks demand.

For dynamic programming, the outer loop is the Bellman iteration itself: at iteration s the inner “solve” is one evaluation of the operator $(TV^{s-1})(\mathbf{x})$ at a state $\mathbf{x}$, which itself requires a constrained nonlinear program over controls plus a quadrature over the next-period shock, then a global fit of V^s to those labels (§9.6.1 of Chapter 9). In both cases the obstacle is the same: the per-inner-solve cost times the per-outer-iteration count.

The key insight is that since we *own* the structural model, we can generate training data by solving the model on a carefully chosen set of input configurations (a *design of experiments*). A cheap-to-evaluate function approximator trained on this synthetic dataset, a *surrogate model*, then replaces the expensive original model for all downstream tasks. Any suitable function approximator can serve as the surrogate; the right choice depends on the dimensionality of the input space and the cost of generating each training point.

Cutting out the outer loop. The point of a surrogate is to break this nesting. One pays a one-time offline cost: pick a design of experiments $\theta^{(1)}, \dots, \theta^{(N)}$, solve the model at those N configurations, and fit a surrogate $\phi(s, \theta)$ to the results. From then on the expensive inner solve is gone, and the estimation, uncertainty-quantification, or policy-search outer loop evaluates a function that costs microseconds and returns exact gradients, so it can run at the 10^3 to 10^6 scale those tasks need. The model is solved at a handful of configurations and the surrogate *interpolates between them*, which is almost always far cheaper than re-solving at every new θ ; Figure 10.1 contrasts the two workflows. The surrogate-based SMM estimation developed below and the surrogate-then-optimize policy search of Chapter 11 are both instances of this move, as is the GP value-function iteration of §9.6 in Chapter 9, where the “outer loop” is the Bellman iteration itself; there the surrogate is refit every Bellman step and the “offline” phase becomes a per-iteration update.

Two surrogate strategies. This course covers two complementary approaches:

1. **Deep neural network (DNN) surrogates** are best suited for *high-dimensional* settings ($d \gg 10$ inputs) where training data can be generated in large quantities, for example when each model solve takes seconds or when closed-form solutions exist. DNNs scale

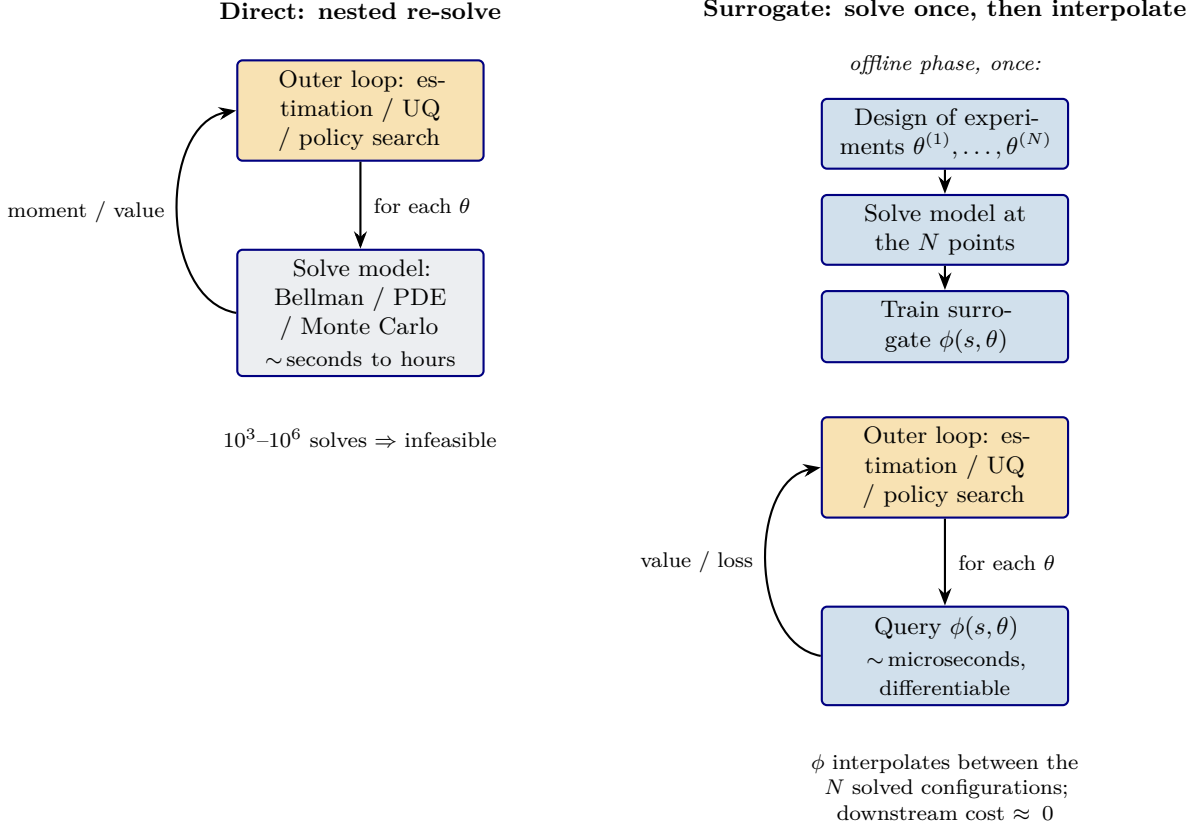


Figure 10.1: Why surrogates help. *Left*: structural estimation, uncertainty quantification, and optimal policy design are outer loops over a parameter vector θ , and the direct implementation re-solves the full model inside the loop, so the cost scales with the number of outer iterations times the per-solve cost. *Right*: a surrogate moves that solve into a one-time offline phase, solving the model only at a design of experiments and fitting $\phi(s, \theta)$; the outer loop then queries a cheap, differentiable interpolant. The saving grows with both the number of outer iterations and the per-solve cost. The same picture applies to GP value-function iteration with the outer loop relabeled as the Bellman iteration and the inner solve as one *TV* evaluation; the offline phase is then replaced by a per-iteration GP refit at modest design size.

gracefully with dimensionality, can be trained via mini-batch SGD on millions of samples, and provide exact gradients via automatic differentiation. [Chen et al. \(2026\)](#) formalize this approach and demonstrate speedups of several orders of magnitude for option pricing (the same surrogate-for-finance idea was implemented earlier with adaptive sparse grids by [Scheidegger and Treccani \(2018\)](#)); [Friedl et al. \(2023\)](#) apply it to uncertainty quantification in high-dimensional integrated assessment models.

2. **Gaussian process (GP) surrogates** are preferable for *intermediate-dimensional* settings ($d \lesssim 10\text{--}15$) where each training point is numerically expensive, for example solving a full DSGE model at one parameter configuration may take minutes or hours. GPs are *data-efficient*: the Bayesian posterior extracts maximum information from each observation. Crucially, the posterior variance provides a built-in uncertainty estimate that can guide *where* to evaluate next, enabling Bayesian Active Learning (BAL) strategies that allocate the computational budget optimally ([Scheidegger and Bilonis, 2019](#)).

Table 10.1 summarizes the main trade-off. The two approaches are not mutually exclusive: one can use a GP to build an initial low-data surrogate with uncertainty estimates, and later switch to a DNN when more training data becomes available. A detailed comparison covering

	DNN surrogate	GP surrogate
Best for	high-dim. ($d \gg 10$), large N	moderate-dim. ($d \lesssim 15$), small N
Data efficiency	data-hungry; wants a large training set	data-efficient; informative from a small one
Key advantage	scales to very high d via SGD	built-in UQ and active learning

Table 10.1: Two complementary surrogate strategies. DNN surrogates are attractive when the input dimension and available training set are large; GP surrogates are attractive when each simulator call is expensive and calibrated posterior uncertainty is useful for active learning.

computational cost, gradient access, and further trade-offs is given in Section 9.4 of Chapter 9, where it sits naturally alongside the GP machinery itself.

Speed gains. This is the payoff sketched in Figure 10.1: regardless of whether a DNN or GP is used, once the surrogate is trained the per-iteration cost of the downstream outer loop, estimation, sensitivity analysis, or optimal policy design, collapses to a function evaluation. [Chen et al. \(2026\)](#) report speedups of several orders of magnitude for option pricing, where evaluating the DNN surrogate replaces expensive FFT-based Fourier inversion (their Bates-model benchmark documents two-to-three orders of magnitude over the numerical pricing baseline). As a rough rule of thumb, the gain scales with the cost of the underlying pricing routine: the orders-of-magnitude gains arise for models requiring a PDE solve (roughly $1 \text{ ms/eval} \rightarrow 1 \mu\text{s/eval}$ through a surrogate) or high-dimensional Monte Carlo, regimes in which 10^3 – $10^4 \times$ speedups are typical. The gains are even larger for gradient computations: while finite-difference gradients require $d + 1$ model evaluations (one per parameter), the gradient through the surrogate (autograd for DNNs, closed-form for GPs) requires only a single pass, regardless of the number of parameters.

10.2 Pseudo-States: Parameters as “State” Variables

The central innovation of the deep surrogate framework is to treat model parameters θ as additional “pseudo-state” variables:

$$\tilde{\mathbf{x}} = (\underbrace{s_1, \dots, s_n}_{\text{states}}, \underbrace{\theta_1, \dots, \theta_p}_{\text{parameters}}) \in \mathbb{R}^d, \quad d = n + p. \quad (10.1)$$

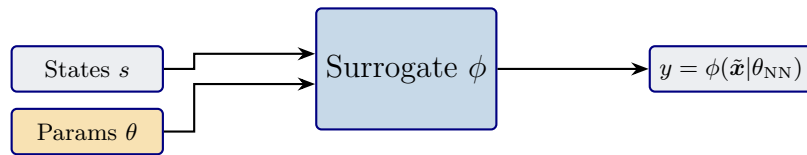


Figure 10.2: Pseudo-state surrogate architecture. Economic states s and model parameters θ are concatenated into the augmented input $\tilde{\mathbf{x}} = (s, \theta)$ and fed to a single approximator $\phi(\tilde{\mathbf{x}} | \theta_{\text{NN}})$ with weights θ_{NN} , yielding a target quantity y (price, policy, moment) as a continuous, differentiable function of both the state and the parameter vector. After one offline training pass, the surrogate is queried instantly across the parameter space without re-solving the original model.

The surrogate is trained once over the full augmented space and can then be queried instantly for any parameter configuration, without re-solving the model. This is fundamentally different from simply re-running the model: the surrogate provides a continuous, differentiable mapping from parameters to outputs, enabling gradient-based optimization and uncertainty propagation that would be impossible with the original model. Figure 10.2 sketches this concatenated input.

Scheidegger and Treccani (2018) achieve the surrogate-for-finance idea with adaptive sparse grids; Friedl et al. (2023) apply the surrogate idea to uncertainty quantification in integrated assessment models of climate change; Chen et al. (2026) demonstrate speedups of several orders of magnitude for option pricing with the deep-surrogate approach.

Comparison of approximation methods. The surrogate approach is one of several function approximation strategies used in computational economics. Table 10.2 situates it relative to alternatives.

Method	Max dim.	Smoothness	Parametric	Differentiable
Cartesian grids	$d \leq 5$	any	no	no
Sparse grids	$d \leq 15$	C^k needed	no	limited
Chebyshev polynomials	$d \leq 10$	smooth	yes	yes
DNN surrogate	$d \gg 10$	any	yes	yes (autograd)
GP surrogate	$d \leq 10^\dagger$	kernel-dependent	no	yes (closed-form)

$\dagger$ Can be extended to $d \gg 10$ via active subspace methods (Scheidegger and Bilonis, 2019).

Table 10.2: Common approximation methods in computational economics. Grid and polynomial methods are transparent but become difficult in high dimension; DNN and GP surrogates trade direct grid structure for sample-based learning and repeated fast evaluation.

10.2.1 Worked Example: Black–Scholes Surrogate

To illustrate the surrogate pipeline concretely, consider the European call option pricing problem from Section 7.8. In the PINN approach (Chapter 7), the network learned the option price by minimizing the Black–Scholes PDE residual; no training data were needed, only the differential equation. The surrogate approach takes the opposite route: we *generate* training data by evaluating the closed-form Black–Scholes formula at a design of experiments, and train a neural network to interpolate this data.

Specifically, we sample N input tuples $(S_i, t_i, \sigma_i, r_i, K_i)$ from a Latin Hypercube design over the ranges of interest and evaluate the analytical price $V_i = V_{BS}(S_i, t_i, \sigma_i, r_i, K_i)$ at each. The surrogate $\hat{V} = \mathcal{N}_\rho(S, t, \sigma, r, K)$ is then trained via standard supervised learning:

$$\ell_\rho = \frac{1}{N} \sum_{i=1}^N |\mathcal{N}_\rho(S_i, t_i, \sigma_i, r_i, K_i) - V_i|^2. \quad (10.2)$$

Once trained, the surrogate provides instant evaluation at any (S, t, σ, r, K) in a single forward pass, instant Greeks (Δ , Γ , Vega, etc.) via a single backward pass, and gradient-based implied volatility calibration, none of which require re-solving the PDE. The key contrast with the PINN is that the surrogate requires *solved* training data (here from the analytical formula; in general, from a numerical solver), but in return it treats the model parameters (σ, r, K) as inputs, enabling re-evaluation across the entire parameter space without re-solving. This is precisely the “pseudo-state” idea of the previous section. See the companion notebook `01_Surrogate_Primer.ipynb` for the full implementation.

10.3 Brock–Mirman with Parameters as Pseudo-States

The stochastic Brock–Mirman model is the partial-depreciation model from Chapter 2:

$$Y_t = z_t K_t^\alpha, \quad (10.3)$$

$$C_t + K_{t+1} = Y_t + (1 - \delta)K_t, \quad (10.4)$$

$$\log z_{t+1} = \varrho \log z_t + \sigma_z \varepsilon_{t+1}, \quad \varepsilon_{t+1} \sim \mathcal{N}(0, 1). \quad (10.5)$$

We write ϱ for TFP persistence to avoid overloading ρ , which elsewhere denotes neural-network parameters. The Python notebooks still use the variable name `rho`; mathematically, that code variable corresponds to ϱ .

The network outputs a savings rate, the fraction of current output that is invested. In the single-parameter exercise,

$$s_t = \mathcal{N}_\rho(z_t, K_t, \varrho) \in (0, 1),$$

with β calibrated to 0.96. In the joint exercise the input becomes $(z_t, K_t, \beta, \varrho)$. In either case, recover

$$K_{t+1} = (1 - \delta)K_t + s_t Y_t, \quad (10.6)$$

$$C_t = (1 - s_t)Y_t. \quad (10.7)$$

Because $s_t \in (0, 1)$ and $Y_t > 0$, this parameterization enforces $C_t > 0$, $K_{t+1} > (1 - \delta)K_t > 0$, and gross investment $I_t = s_t Y_t \geq 0$ by construction, so the resource constraint and the non-negativity of investment hold automatically and the partial-depreciation Euler equation (10.8) applies as written, with no extra multiplier.¹

Training uses the same Euler equation as Chapter 2, but the residual is evaluated jointly over states and parameter draws. With partial depreciation,

$$\frac{1}{C_t} = \beta \mathbb{E} \left[\frac{1 - \delta + \alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} \right].$$

For a sampled state–parameter pair (z_i, K_i, θ_b) , where $\theta_b = \varrho_b$ in the scalar exercise and $\theta_b = (\beta_b, \varrho_b)$ in the joint exercise, the companion notebooks form the *relative* residual

$$G_i(\theta_b) = \frac{1}{\beta_b C_i \mathbb{E}[(1 - \delta + \alpha z_{i,t+1} (K'_i)^{\alpha-1}) / C_{i,t+1}]} - 1, \quad (10.8)$$

where $Y_i = z_i K_i^\alpha$, $K'_i = (1 - \delta)K_i + s_i Y_i$, $C_i = (1 - s_i)Y_i$, and $C_{i,t+1}$ is computed by feeding $(z_{i,t+1}, K'_i, \theta_b)$ back through the same network. The expectation over $z_{i,t+1}$ is approximated by Gauss–Hermite quadrature (Section 2.6). The relative form is preferred to the equivalent absolute residual $1 - \beta_b C_i \mathbb{E}[\cdot]$ because dividing by the consumption ratio makes the loss scale-free across (z, K, θ) samples; the two forms share the same zero set but the relative form is better conditioned under FP32 forward passes. A representative training loss is

$$\ell_\rho = \frac{1}{N_s N_\theta} \sum_{i=1}^{N_s} \sum_{b=1}^{N_\theta} |G_i(\theta_b)|^2. \quad (10.9)$$

The outer sum over θ_b is the pseudo-state trick: one network learns a family of policies over the whole parameter rectangle.

10.4 Simulated Method of Moments

The pseudo-state surrogate is what makes SMM cheap. Without it, every objective evaluation would re-solve the structural model. With it, the trained policy network and simulator define a fast deterministic map $\theta \mapsto m(\theta)$ once the simulation design is fixed.

The Simulated Method of Moments (SMM) estimates structural parameters by matching model-implied moments to their empirical counterparts. The method was developed as an extension of the Generalized Method of Moments (GMM) to settings where the moment conditions

¹A resource-based variant in which the savings rate is applied to total resources $R_t = Y_t + (1 - \delta)K_t$, allowing disinvestment relative to the depreciated stock, is a straightforward alternative; the SMM moments and notebook outputs in this chapter use the output-based savings rate above.

do not have a closed-form expression but can be computed via simulation (McFadden, 1989; Pakes and Pollard, 1989; Lee and Ingram, 1991; Duffie and Singleton, 1993). A closely related approach is *indirect inference* (Gourieroux et al., 1993), which matches the parameters of an auxiliary model rather than raw moments.

In quantitative macro and finance, the same simulated-moments logic is especially useful in sovereign default and incomplete-markets environments, where likelihood-based estimation is either unavailable or prohibitively expensive (Arellano, 2008).

Let $\hat{m}^{\text{data}} \in \mathbb{R}^q$ denote a vector of q sample moments computed from observed data (e.g., mean capital, output variance, consumption autocorrelation), and let $m(\theta) \in \mathbb{R}^q$ denote the corresponding moments simulated from the model at parameter value $\theta \in \mathbb{R}^p$. The SMM estimator solves:

$$\hat{\theta}_{\text{SMM}} = \arg \min_{\theta} \underbrace{(m(\theta) - \hat{m}^{\text{data}})^{\top}}_{1 \times q} \underbrace{W}_{q \times q} \underbrace{(m(\theta) - \hat{m}^{\text{data}})}_{q \times 1}, \quad (10.10)$$

where $W \in \mathbb{R}^{q \times q}$ is a symmetric positive definite weighting matrix.

The role of the weighting matrix W . The matrix W controls how much weight each moment (and each pair of moments) receives in the objective. To build intuition, consider three common choices:

1. **Identity weighting** ($W = I_q$): all moments receive equal weight. The objective reduces to the unweighted sum of squared deviations, $\sum_{j=1}^q (m_j(\theta) - \hat{m}_j^{\text{data}})^2$. This is simple but inefficient: moments measured with high precision receive the same weight as noisy moments.
2. **Diagonal weighting** ($W = \text{diag}(1/\hat{\sigma}_1^2, \dots, 1/\hat{\sigma}_q^2)$): each moment is scaled by the inverse of its estimated variance. This corrects for differing units and precision.
3. **Optimal weighting**: the inverse of the covariance matrix of the *moment discrepancy* $\hat{g}(\theta) = m(\theta) - \hat{m}^{\text{data}}$. If the simulation noise is negligible, this is well approximated by the inverse covariance of the empirical moments. With independent simulated panels of the same length as the data and S replications, the covariance is approximately $(1 + 1/S)\Sigma_m$, so the optimal weight is proportional to Σ_m^{-1} but the scale matters for the J -statistic below.

With identity weighting, the criterion is the sum of squared moment deviations; with inverse discrepancy-covariance weighting, it is the squared Mahalanobis distance between simulated and empirical moments.

Consistency and efficiency. Under standard regularity conditions, the SMM estimator is consistent: $\hat{\theta}_{\text{SMM}} \xrightarrow{p} \theta_0$ as the data sample size $T \rightarrow \infty$. Identification requires more than the usual $q \geq p$ moment count: locally, the moment Jacobian must satisfy the rank condition

$$\text{rank}(\partial m(\theta_0)/\partial \theta') = p, \quad (10.11)$$

which is necessary for *local* identification. Note that full column rank is necessary but not sufficient: when the smallest singular value of M is close to zero (a near-singular Jacobian), the parameter is only *weakly* identified in finite samples even though the rank condition formally holds. Global identification additionally requires the population value of the empirical moments, $\bar{m} := \lim_{T \rightarrow \infty} \hat{m}^{\text{data}}$, to be matched at a *unique* $\theta_0 \in \Theta$, i.e., $\bar{m} = m(\theta_0)$ has a unique solution in Θ ; the rank condition is the local-curvature implication of that uniqueness. Let $M = \partial m/\partial \theta'|_{\theta_0}$, and let Ω denote the asymptotic covariance of $\sqrt{T}\hat{g}(\theta_0)$. The large-sample distribution is

$$\sqrt{T}(\hat{\theta}_{\text{SMM}} - \theta_0) \xrightarrow{d} \mathcal{N}\left(\mathbf{0}, (M^{\top}WM)^{-1}M^{\top}W\Omega WM(M^{\top}WM)^{-1}\right). \quad (10.12)$$

For S independent simulated panels each of length τT (with $\tau \geq 1$ a relative-length factor), $\Omega = (1 + 1/(\tau S)) \Sigma_m$. The classroom benchmark used below sets $\tau = 1$ (simulated panels of the same length as the data), giving the familiar $\Omega = (1 + 1/S) \Sigma_m$. As $\tau S \rightarrow \infty$, the *extra* simulation variance vanishes and $\Omega \rightarrow \Sigma_m$; the factor itself tends to one, not zero. The efficient SMM weight is $W^* = \Omega^{-1}$. See [Duffie and Singleton \(1993\)](#) for the corresponding large-sample theory in the simulated-moments setting.

10.5 The SMM Workflow in the Exercise

The exercise uses a deliberately simple synthetic-data workflow so that the econometric logic is transparent. First, choose a true parameter and simulate a time series from the trained pseudo-state surrogate. These observations play the role of data. Second, for each candidate parameter, re-simulate the model with the same burn-in length, simulation horizon, initial state, and shock seed. Third, compute a small vector of economically interpretable moments and minimize the quadratic SMM criterion with identity weighting.

Single-parameter persistence exercise. Notebook `lecture_15_03_Structural_Estimation_BM.ipynb` calibrates $\beta = 0.96$, sets $\varrho_{\text{true}} = 0.90$, and estimates $\varrho \in [0.50, 0.99]$. Let $\{C_t(\varrho), I_t(\varrho), Y_t(\varrho)\}_{t=1}^T$ denote a simulated sample at candidate persistence ϱ . The estimator uses three moments:

$$m_1(\varrho) = \text{std}(\Delta \log C_t(\varrho)), \quad (10.13)$$

$$m_2(\varrho) = \text{corr}(\Delta \log C_t(\varrho), \Delta \log C_{t-1}(\varrho)), \quad (10.14)$$

$$m_3(\varrho) = \text{corr}(\log Y_t(\varrho), \log Y_{t-1}(\varrho)). \quad (10.15)$$

All three moments are computed on the raw simulated time series with no detrending or demeaning step, and $\text{std}(\cdot)$ and $\text{corr}(\cdot)$ denote sample standard deviation and sample autocorrelation evaluated directly on the simulated panel. The output autocorrelation is the most direct persistence moment. The volatility moment should be interpreted as an empirical simulated moment, not as the level-variance formula. For the AR(1) shock,

$$\text{Var}(\log z_t) = \frac{\sigma_z^2}{1 - \varrho^2}, \quad \text{Var}(\Delta \log z_t) = 2 \text{Var}(\log z_t) (1 - \varrho) = \frac{2\sigma_z^2}{1 + \varrho},$$

so the familiar $1/(1 - \varrho^2)$ amplification applies to the *level* of log productivity, not to first differences. The notebook also reports the mean savings rate as a diagnostic and correctly treats it as nearly uninformative for ϱ ; it is masked out of the SMM criterion in the scalar exercise and used only for visual identification checks.

Joint exercise. Notebook `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb` estimates $\theta = (\beta, \varrho)$, with $\beta \in [0.92, 0.99]$ and $\varrho \in [0.50, 0.99]$. It uses four candidate moments: mean savings, growth volatility, consumption-growth autocorrelation, and output autocorrelation. The *shallow-ridge two-moment specification* retains $\{\text{std}(\Delta \log C_t), \text{corr}(\Delta \log C_t, \Delta \log C_{t-1})\}$ to expose the partial-identification ridge in the criterion surface; the over-identified specification uses all four moments and collapses the ridge around the synthetic truth. Formally the two-moment case is just-identified ($q = p = 2$), so we avoid the econometric term *weak identification* (which refers to a near-singular Jacobian asymptotic regime) and use *shallow-ridge* or *partially-identified* for what the criterion-surface picture actually shows.

A deterministic objective. Because the same initial condition and random seed are used for every candidate θ , the map $\theta \mapsto m(\theta)$ is deterministic in the notebooks. This is still standard SMM; the fixed seed is a common-random-numbers device that removes irrelevant simulation noise while we study identification and optimization. In more realistic estimation exercises one

averages over multiple replications or increases the simulation length to make Monte Carlo noise negligible. One consequence is worth stating explicitly: because the synthetic data come from the trained surrogate evaluated at the true parameter, and every candidate evaluates the same shock sequence, the SMM criterion attains a near-zero minimum at the truth. This is a clean self-consistency test of the surrogate-SMM pipeline, not a claim about the size of the criterion one would see with real data and independent simulation draws.

Implementation. The single-parameter estimation routine proceeds in two steps:

1. Evaluate the SMM objective on a coarse grid over $\varrho \in [0.52, 0.99]$ (matching the notebook’s grid bounds) to verify that the criterion is well behaved and to visualize identification.
2. Refine the minimizer with a bounded scalar optimizer (e.g. Brent’s method).

The joint notebook maps the 2D criterion on a grid, then refines from the grid minimizer using bounded Nelder–Mead. Since the policy surrogate has already been trained, each evaluation of $m(\theta)$ requires only a forward simulation, not a full re-resolution of the dynamic program.

Interpretation. If the moments are informative about θ , the objective should be minimized close to the synthetic truth. In the scalar notebook, $\hat{\varrho}$ is very close to 0.90 and the fitted policy functions at ϱ_{true} and $\hat{\varrho}$ nearly overlap. The joint notebook shows the additional lesson: point estimates can be accurate while the criterion still has weak curvature along one parameter direction, which is why contour plots and Jacobian diagnostics matter. Figure 10.4 in §10.7 below visualizes both specifications and is worth looking at now to fix the geometric picture in mind.

10.6 Practical Considerations

Moment selection and identification. The choice of moment conditions is critical for identification. In the scalar exercise, autocorrelation moments identify ϱ sharply, while the mean savings rate is nearly flat in ϱ . In the joint exercise, the mean savings rate carries most of the information about β , while persistence moments carry most of the information about ϱ . More generally, the number of moments must weakly exceed the number of parameters ($\dim(m) \geq \dim(\theta)$), and the selected moments should move in economically distinct ways as parameters vary.

Weighting. The exercise uses $W = I$ so that the objective is easy to read. In applications, one usually moves to two-step SMM: first estimate $\hat{\theta}_1$ with identity weighting, then estimate the covariance matrix of the moment discrepancy and set $W = \hat{\Omega}^{-1}$ in a second pass. This corrects for different moment scales, moment correlations, and any non-negligible simulation noise. The two-step estimator is asymptotically efficient under *correct specification* and the usual GMM regularity conditions (Duffie and Singleton, 1993); under misspecification, the optimal- W estimator can have larger finite-sample mean-squared error than identity weighting, because the efficient weighting is calibrated against the wrong moment-discrepancy distribution.

Simulation design. The notebook fixes the burn-in length, horizon, initial state, and shock sequence across all objective evaluations. This is important because otherwise the optimizer would chase simulation noise rather than structural differences across parameter values. In larger empirical applications, the same idea appears as *common random numbers* (CRN) or replicated simulations: both are classical variance-reduction techniques in stochastic simulation (Glasserman, 2004), and within the simulated-moments setting McFadden (1989) emphasized fixing the simulated draws across parameter values to make the moment objective $m(\theta)$ a smooth function of θ rather than a noisy step function (the asymptotic theory of optimization estimators

with simulation is developed in [Pakes and Pollard \(1989\)](#)). The geometric intuition is simple: if every candidate parameter value is evaluated against the *same* draw of innovations, the residual $m(\theta) - m(\theta')$ isolates the structural effect of moving from θ to θ' rather than a Monte Carlo accident.

Identification diagnostics. A necessary condition for local identification is that the Jacobian $M = \partial m / \partial \theta'$ has full column rank at the true parameter. In the scalar Brock–Mirman exercise this condition reduces to requiring that at least one selected moment changes with ϱ in a neighborhood of the truth. In the joint exercise, the singular values of M reveal the weak direction associated with β . Plotting the objective profile or contour is therefore already informative: a clear and well-centered U-shape signals useful identifying variation, whereas a flat ridge or a jagged profile indicates weak identification or excessive simulation noise.

Over-identification tests. When the model is over-identified ($q > p$), report the standard J -statistic at the optimum:

$$J = T \hat{g}(\hat{\theta})^\top W \hat{g}(\hat{\theta}), \quad \hat{g}(\theta) = m(\theta) - \hat{m}^{\text{data}}. \quad (10.16)$$

Under correct specification and regularity conditions, J is asymptotically χ_{q-p}^2 when $W = \Omega^{-1}$, the inverse covariance of the moment discrepancy. If $W = \Sigma_m^{-1}$ is used while finite simulation noise remains, the statistic must be scaled accordingly or calibrated by bootstrap. A large J indicates either model misspecification, poorly chosen moments, or underestimated sampling uncertainty in moments.

Standard errors and confidence intervals. In applications, report not only $\hat{\theta}$ but also uncertainty quantification. A plug-in sandwich estimator is:

$$\widehat{\text{Var}}(\hat{\theta}) = \frac{1 + 1/S}{T} (\hat{M}^\top W \hat{M})^{-1} \hat{M}^\top W \hat{\Sigma} W \hat{M} (\hat{M}^\top W \hat{M})^{-1}, \quad (10.17)$$

where $\hat{M}$ and $\hat{\Sigma}$ are estimated at $\hat{\theta}$ under the equal-length independent-simulation approximation. For small samples, time-series dependence, or highly nonlinear criteria, moving-block or parametric bootstrap intervals are often more reliable than first-order asymptotics.

Weak-identification workflow. If the smallest singular values of $\hat{M}$ are close to zero, inference based purely on local curvature is fragile. In that case, complement Hessian-based standard errors with profile-criterion diagnostics: vary one parameter at a time (or along weak singular vectors), re-optimize the remaining parameters, and report objective-function contour sets in addition to pointwise intervals.

10.7 GP Surrogate over the Moment Map

Scope of this section

The simplified core notebooks `lecture_15_03_Structural_Estimation_BM.ipynb` and `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb` stop after the direct surrogate-based SMM estimator and its identification diagnostics; they do *not* implement the second-layer Gaussian-process moment surrogate, leave-one-out validation, or Bayesian active learning described below. This section sketches the research-scale extension that a separate companion notebook would add on top of the policy surrogate, and that motivates the GP active-learning workflow used in Chapter 11.

The pseudo-state DEQN of the previous sections turned “one re-solve per candidate θ ” into “one forward simulation per candidate θ .” For high-throughput SMM, that second cost is still nontrivial: a bootstrap with $B = 1,000$ resamples needs 1,000 forward simulations on top of the inner optimization, joint estimation in $p \geq 2$ dimensions multiplies the budget further, and downstream Bayesian or simulation-based-inference workflows want *very* cheap evaluations of $\theta \mapsto m(\theta)$. This section adds a second surrogate layer, a Gaussian process per moment, on top of the policy net, following exactly the supervised-learning logic of §9.6.1.

10.7.1 The Two-Layer Surrogate Architecture

Recall the DEQN policy net $\mathcal{N}_\rho(z, K, \theta)$ from §10.4; given θ , it returns the savings rate as a function of (z, K) and lets us simulate a length- T path $\{C_t(\theta), I_t(\theta), Y_t(\theta)\}_{t=1}^T$ in milliseconds. The empirical SMM workflow then maps that simulated path to a moment vector $m(\theta) \in \mathbb{R}^q$ via a fixed numerical recipe (means, standard deviations, autocorrelations).

We now stack a second surrogate on top of this:

$$\hat{m}_j(\theta) \sim \text{GP}(0, k_j(\cdot, \cdot)), \quad j = 1, \dots, q, \quad (10.18)$$

one independent GP per moment, with its own kernel and length-scale hyperparameters learned by maximizing marginal likelihood on a small design $\{(\theta^{(i)}, m(\theta^{(i)}))\}_{i=1}^n$. Once trained, evaluating the SMM objective at any candidate θ is a single GP forward pass per moment, no simulation required. Bootstrapped CIs and any subsequent Bayesian post-processing then run on the GP, not on the simulator. Figure 10.3 reads the architecture top-to-bottom. A candidate θ is fed into the DEQN policy net of §10.4, which is rolled forward for T periods to produce a simulated path and its moment vector $m(\theta) \in \mathbb{R}^q$; a small design $\{(\theta^{(i)}, m(\theta^{(i)}))\}_{i=1}^n$ of those simulator labels then trains the second layer of q independent moment GPs, after which the SMM criterion $Q(\theta)$ is a closed-form quadratic in the GP posterior means. The right-hand column traces the per- θ cost cascade from seconds-to-hours down to microseconds.

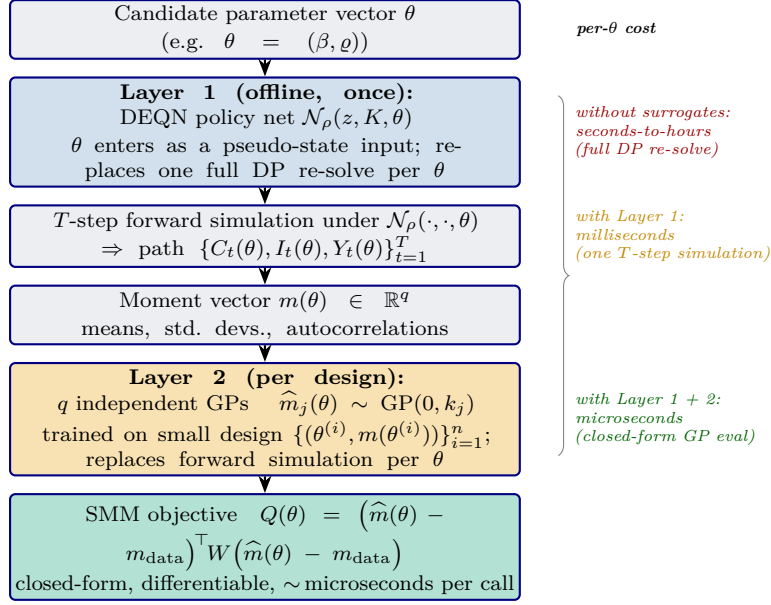


Figure 10.3: The two-layer surrogate architecture for surrogate-based SMM, read top-to-bottom along the chain $\theta \rightarrow \mathcal{N}_\rho \rightarrow$ simulated path $\rightarrow m(\theta) \rightarrow$ moment GPs $\rightarrow Q(\theta)$. *Layer 1* is the pseudo-state DEQN policy net of §10.4: trained once with θ as an additional input, it replaces the inner Bellman / fixed-point re-solve that direct SMM would require at every candidate parameter, leaving only a T -step forward simulation per θ . *Layer 2* is the moment-map GP regression of this section: q independent Gaussian processes are fitted to the simulator’s $(\theta^{(i)}, m(\theta^{(i)}))$ pairs on a small design, after which evaluating the SMM criterion $Q(\theta)$ at any new θ requires only a closed-form GP posterior-mean call per moment. The right-hand annotation traces the per- θ cost cascade: the direct re-solve costs seconds-to-hours, Layer 1 brings the cost down to milliseconds (one DEQN-driven simulation), and Layer 2 down to microseconds (one differentiable regression call per moment). This is the same supervised-learning-on-an-expensive-oracle pattern as GP-VFI in §9.6.1, with the moment vector $m(\theta)$ playing the role the Bellman label $TV(\mathbf{x})$ plays there; the saving compounds because the high-throughput downstream workflows of SMM, bootstrap, profile likelihood, and simulation-based inference, all live in the bottom box.

Same expensive-oracle structure as VFI. This is structurally identical to the GP-VFI setup of §9.6. There, one design point cost one Bellman maximization; here, one design point costs one T -step forward simulation plus a moment computation. In both cases the regressor sees a small but high-quality training set generated by an expensive numerical procedure, and the GP machinery, marginal-likelihood Occam’s razor for hyperparameters, leave-one-out diagnostics for surrogate health, and Bayesian active learning for adaptive design, applies verbatim.

10.7.2 Leave-One-Out Validation of the Moment Surrogate

The Cholesky-trick LOO formula (9.14) of §9.6.5 delivers a held-out predictive error for each moment GP at zero marginal cost beyond the existing posterior factorization. A research-scale companion to the core SMM notebooks would track

$$\text{LOO-RMSE}_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{m}_j^{-i}(\theta^{(i)}) - m_j(\theta^{(i)}))^2}$$

for every moment j and every design size n , and pair it with an independent sanity check that evaluates the GP at a *fresh* interior holdout point θ_{holdout} never seen during training; agreement between the two RMSEs is the criterion for declaring the moment surrogate trustworthy before any bootstrap or SBI workflow is run on top of it.

10.7.3 Active Learning of the Moment Surrogate

Two acquisition strategies are natural, matched to the dimensionality of the parameter.

Single-parameter case. With a scalar $\theta = \varrho \in [0.50, 0.99]$, a coarse uniform pilot grid of n_0 points can be enriched by n_{add} active points placed sequentially at locations of largest standardized moment-GP posterior uncertainty,

$$\theta^{\text{next}} \in \arg \max_{\theta \in \mathcal{X}^{\text{cand}}} \left\| \boldsymbol{\sigma}_m(\theta) / \bar{\boldsymbol{\sigma}}_m \right\|_2,$$

subject to a minimum-spacing constraint against existing design points. This is the same pure-exploration acquisition used for VFI (9.15), modulo the per-moment normalization that prevents one large-magnitude moment from dominating the objective.

Joint-parameter case. With $\theta = (\beta, \varrho)$ on a 2D rectangle, pure exploration is wasteful because most of the rectangle sits far from the SMM minimizer. A natural alternative is a BoTorch-style Upper-Confidence-Bound (UCB) acquisition on the transformed score $\tilde{Q}(\theta) := -\log_{10}(Q(\theta) + \varepsilon)$, multiplicatively weighted by the moment-GP posterior uncertainty:

$$a(\theta) = [0.25 + \widetilde{\text{UCB}}_{\tilde{Q}}(\theta)] \cdot \left\| \boldsymbol{\sigma}_m(\theta) / \widehat{\text{sd}}(m) \right\|_2, \quad (10.19)$$

where $\widetilde{\text{UCB}}_{\tilde{Q}}$ is a quantile-scaled and clipped UCB score. The first factor exploits, biasing the design toward (β, ϱ) pairs with small SMM criterion, while the second explores, requiring the design to also hit places where the moment GP is uncertain. The additive constant 0.25 keeps the exploration term active even where the scaled UCB is zero, preventing pathological degeneracy in the acquisition.

Three-way comparison. At a fixed design budget, one can compare pilot grid / naive Latin-hypercube / BoTorch-BAL designs along three axes: (i) leave-one-out error on the moment GPs; (ii) error on the recovered SMM criterion against a fresh reference grid; (iii) accuracy of the recovered estimate $(\hat{\beta}, \hat{\varrho})$. Active designs typically give the most stable local moment surrogate at small budgets.

10.7.4 The 2D SMM Criterion Surface and Partial Identification

Figure 10.4 shows the direct SMM criterion on the joint rectangle. Two features are visible.

First, the criterion has a long, shallow ridge along the β direction in the just-identified specification: the data are nearly uninformative about β once ϱ is fixed, so a wide range of β -values fits almost equally well. This is partial identification, in textbook form, visualized on the criterion surface. Economically, β and ϱ both shift the consumption-smoothing motive in similar directions on long horizons: raising patience and raising persistence each raise the mean savings rate and dampen consumption-growth autocorrelation, so a two-moment specification built from those two moments leaves the (β, ϱ) ratio under-determined and produces the ridge.

Second, the ridge collapses to a localized minimum in the over-identified specification. In the synthetic CRN run, the recovered estimate sits very close to $(\beta_{\text{true}}, \varrho_{\text{true}}) = (0.96, 0.90)$. This makes a useful pedagogical point: identification is a property of the moment selection, not of the estimator. The over-identified specification breaks the redundancy by adding growth volatility, which is sensitive to ϱ through the shock-persistence channel but only weakly to β , and output autocorrelation, which is sensitive to ϱ directly.

In the research-scale extension, the second-layer GP fitted to the joint moment map provides a closed-form, microseconds-per-call substitute for forward simulation: subsequent SMM evaluations, bootstrap replications, and Bayesian post-processing run on the GP rather than on the

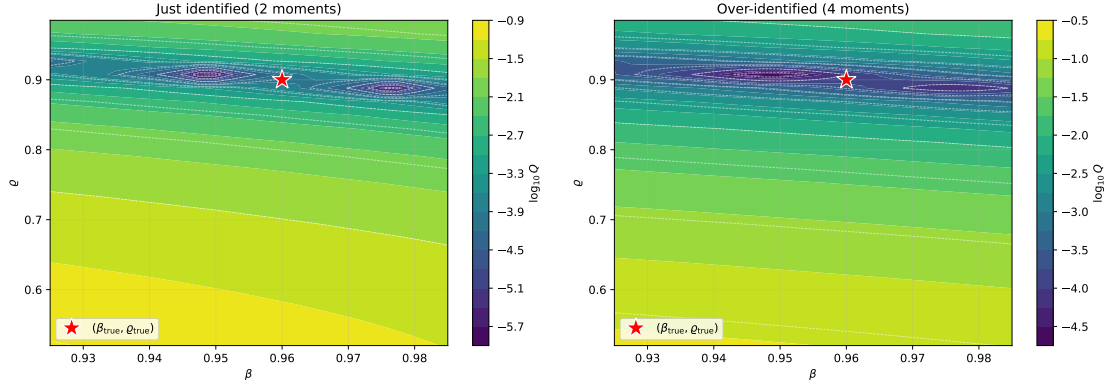


Figure 10.4: Direct SMM criterion for the joint Brock–Mirman estimation. The left panel uses the just-identified two-moment specification and displays a shallow ridge along β , signalling that patience is only partially identified by those two moments. The right panel uses the over-identified four-moment specification, which adds volatility and output-persistence information and produces a localized minimum near the synthetic truth. Generated by notebook 03b.

simulator. The TikZ architecture diagram in Figure 10.3 already encodes the cost cascade; the rendered GP-objective surface itself is not produced by the core notebooks and is therefore not displayed here.

Hands-on

Notebook `lecture_15_03_Structural_Estimation_BM.ipynb` fits the scalar persistence exercise: a 3-input pseudo-state policy net, common-random-number simulation across a ϱ -grid, and an interior SMM estimate with a moment-Jacobian diagnostic. Notebook `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb` performs joint (β, ϱ) estimation and visualizes the partial-identification ridge of Figure 10.4 for the shallow-ridge two-moment specification and the over-identified specification. The second-layer GP-moment-map extension above is left as a research-scale companion.

10.8 Beyond SMM: Indirect Inference and Simulation-Based Inference

SMM matches a hand-picked vector of moments. Two close cousins are worth knowing because they often dominate SMM when moment selection is awkward or when one wants the full likelihood.

Indirect inference. Smith (1993) and Gouriéroux et al. (1993) replace the moment vector $m(\theta)$ with the parameters of a tractable *auxiliary model* (e.g. a low-order VAR or a flexible regression) fitted to both the data and to model-simulated data. Estimation matches the auxiliary parameters rather than raw moments; the resulting estimator is asymptotically equivalent to ML when the auxiliary model is sufficiently rich, and the auxiliary parameters often summarize the distribution far more efficiently than a hand-picked moment list. Indirect inference is the natural choice in macro-finance applications where standard moments are weakly identifying but a structural VAR or a near-likelihood auxiliary is available.

Simulation-based inference (SBI). In settings where the simulator is differentiable or fast but the likelihood $p(y \mid \theta)$ is intractable, modern SBI (Cranmer et al., 2020) learns a neural conditional density estimator $q_\phi(\theta \mid y)$ (or its likelihood/likelihood-ratio counterpart) directly

from (θ_i, y_i) pairs simulated under the prior. The resulting object is an amortized *Bayesian* posterior usable for any future observation y^* at cost of one forward pass. SBI generalizes Approximate Bayesian Computation, sidesteps moment selection entirely, and naturally pairs with the deep-surrogate machinery of Chapter 9. In the surrogate-then-estimate framing of this chapter, the most direct SBI variant is *neural posterior estimation* (NPE), where the pseudo-state DEQN provides the simulator and the GP moment surrogate of §10.7 is replaced by a learned posterior $q_\phi(\theta | y)$; Exercise 4 contrasts SMM with SBI in algorithmic terms.

When to choose what. SMM remains the workhorse when a small number of structural moments are well-identified and economists want a transparent, interpretable objective. Indirect inference dominates when an informative auxiliary model is available. SBI is the natural tool in environments where the model is expensive to simulate but a one-time training run unlocks Bayesian inference at *evaluation* time, precisely the setting in which the rest of this script deploys deep surrogates.

Why this matters for the next chapter. Climate–economy integrated assessment models (Chapter 11) are the prototypical setting where surrogate-based estimation pays off. Credible policy analysis requires evaluating the model over many climate-sensitivity, damage-elasticity, and discount-rate scenarios. Treating the parameter vector as a pseudo-state and training a single deep surrogate, exactly as in the SMM exercise above, turns repeated re-solves into repeated forward passes and is the technical bridge between this chapter and the next.

Chapter Summary

- SMM matches simulated moments to data moments by minimizing a weighted quadratic objective; it is GMM with simulation (McFadden, 1989; Pakes and Pollard, 1989).
- The pseudo-state surrogate trick (treat parameters θ as additional network inputs) replaces a re-solve at every candidate θ with a single forward pass; this is what makes SMM with a deep model tractable.
- Stacking a Gaussian-process layer over the moment map (§10.7) turns SMM into a two-stage surrogate problem: each oracle call is one forward simulation, each subsequent objective evaluation is one GP posterior, the same expensive-oracle / supervised-learning logic that motivates GP-based VFI in §9.6.
- Common random numbers across θ -candidates are essential in the classroom exercises: they remove simulation noise from the objective and let optimizers see a smooth landscape (Glasserman, 2004).
- Indirect inference and modern simulation-based inference (Smith, 1993; Gouriéroux et al., 1993; Cranmer et al., 2020) are the natural neighbors of SMM and dominate in their respective regimes.

Further Reading

- McFadden (1989); Pakes and Pollard (1989); Duffie and Singleton (1993), the foundational SMM trio.
- Kase et al. (2022), neural-network estimation of nonlinear heterogeneous-agent models; Chen et al. (2026), deep surrogates for finance and option pricing.
- Cranmer et al. (2020), contemporary simulation-based inference.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Computational] Identification.** In notebook `lecture_15_03_Structural_Estimation_BM.ipynb`, choose two candidate moments and compute the finite-difference Jacobian $\partial m / \partial \varrho$ at $\varrho_{\text{true}} = 0.90$. Which moment provides stronger local identification?
2. **[Core] Optimal weighting.** Show that under standard regularity conditions, $W^* = \Omega^{-1}$ minimizes the asymptotic variance of $\hat{\theta}_{\text{SMM}}$, where Ω is the covariance of the moment discrepancy. Why does identity weighting still appear in the first stage?
3. **[Computational] Common random numbers.** In the scalar Brock–Mirman exercise, plot the SMM objective as a function of ϱ with and without common random numbers. Quantify the objective noise across repeated Monte Carlo panels under the same candidate grid.
4. **[Core] SMM vs. SBI.** Outline the algorithmic difference between estimating θ by SMM (one optimization per dataset) and by neural simulation-based inference (one training run plus one posterior evaluation per dataset). In which regime does SBI dominate?
5. **[Advanced/project] J -statistic and overidentification.** In notebook `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb`, use the over-identified specification with $q = 4$ moments and $p = 2$ parameters. (i) State the asymptotic distribution of the J -statistic under correct specification when $W = \Omega^{-1}$. (ii) Compute $J(\hat{\theta})$ on the original synthetic sample and report whether the χ^2_2 test rejects at $\alpha = 0.05$. (iii) Repeat across Monte Carlo samples generated at the truth and compare the empirical distribution with χ^2_2 . (iv) Introduce a structural break in one model parameter and verify that the J distribution shifts to the right.
6. **[Advanced/project] Bootstrap confidence intervals.** Compare three confidence-interval procedures for $\hat{\theta}_{\text{SMM}}$: (a) plug-in sandwich standard errors; (b) moving-block or stationary bootstrap of the time-series sample; (c) parametric bootstrap, drawing new samples under the simulated model at $\hat{\theta}$. Report the confidence intervals and coverage across repeated Monte Carlo replications.
7. **[Advanced/project] Maximum likelihood vs. SMM efficiency.** Suppose the productivity process $\log z_t$ is observed. Implement the Gaussian AR(1) MLE for ϱ and compare it with the SMM estimator based on the moments in notebook `lecture_15_03_Structural_Estimation_BM.ipynb`. On Monte Carlo replications at several sample sizes, report bias, variance, and MSE. Explain why MLE is efficient for this observed-shock likelihood, while SMM reaches the GMM efficiency bound only for the chosen moment vector. As an optional extension, repeat the beta-only MLE comparison in the full-depreciation log-utility special case where the policy has a closed form.

Chapter 11

Climate Economics and Deep Uncertainty Quantification

This chapter brings together the methods developed throughout this script and applies them to one of the most consequential computational challenges in economics: *climate change policy*. Integrated assessment models (IAMs) couple economic growth with the carbon cycle, temperature dynamics, and climate damages, creating high-dimensional nonstationary dynamic programming problems that are ideal candidates for the DEQN and surrogate methods we have developed. We present the CDICE model of [Folini et al. \(2025\)](#), solve it with DEQNs, and then use GP surrogates and Bayesian active learning, first for deep uncertainty quantification ([Friedl et al., 2023](#)), and then, applying the same surrogate-then-optimize machinery to a different OLG model and a different surrogate, to *search over policy parameters* and derive constrained Pareto-improving carbon tax rules in an OLG–IAM with deep uncertainty ([Kübler et al., 2026](#)). This last step illustrates a general use of surrogates that goes beyond estimation and UQ: once the structural model has been mapped into a fast, differentiable surrogate, the costly outer loop of an optimal-policy search in a dynamic stochastic heterogeneous-agent economy collapses into a small optimization on the surrogate. For broader overviews of climate economics and IAMs, see [Hassler et al. \(2016\)](#) on environmental macroeconomics, [Dietz \(2024\)](#) on IAMs, [Fernández-Villaverde et al. \(2025\)](#) on the intersection of climate economics and deep learning, and [van der Ploeg and Rezai \(2026\)](#) on the macroeconomics of climate policy.

11.1 The Macroeconomics of Climate Change

Climate change is a global externality: the emissions of each agent affect the welfare of all agents, including future generations that have no say in current decisions. Unlike standard market failures, the climate externality operates across centuries, involves deep scientific uncertainty, and couples the macroeconomy with the earth system in both directions. Recent overviews include [Hassler et al. \(2016\)](#) on environmental macroeconomics, [Dietz \(2024\)](#) on IAMs, [Fernández-Villaverde et al. \(2025\)](#) on climate economics and deep learning, and [van der Ploeg and Rezai \(2026\)](#) on the macroeconomics of climate policy.

Integrated assessment models. Integrated assessment models (IAMs) formalize this coupling. The economy produces output and emissions; emissions accumulate in the atmosphere and raise global temperature; temperature increases cause damages that reduce output. The feedback loop is closed (Figure 11.1):

The central output of an IAM is the **social cost of carbon (SCC)**: the marginal welfare cost of one additional unit of CO₂ emissions, measured in consumption-equivalent units. When emissions are measured in GtC (gigatons of carbon), the SCC has units of consumption per

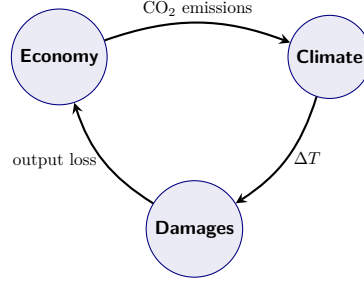


Figure 11.1: The integrated-assessment feedback loop. The economy produces output and CO₂ emissions; emissions accumulate in the atmosphere and raise global mean temperature (ΔT); higher temperatures generate damages that reduce output and consumption, which in turn shape the path of future emissions. An IAM closes this loop and uses it to quantify the welfare cost of additional emissions, summarized by the social cost of carbon (11.1).

GtC. Conversion to USD per tCO₂ requires first applying the consumption-to-USD numeraire and then converting the carbon mass unit: one tCO₂ contains 12/44 tons of carbon, so a price expressed per ton of carbon is divided by 44/12 to obtain the corresponding price per ton of CO₂ (and a GtC price is also divided by 10⁹). Formally,

$$\text{SCC}_t = -\frac{\partial V_t / \partial E_t}{\partial V_t / \partial C_t}, \quad (11.1)$$

where V_t is the value function, E_t is contemporaneous emissions, and C_t is consumption. The flow form is linked to the stock-based form $\text{SCC}_t^M = -(\partial V_t / \partial M_t^{\text{AT}}) / (\partial V_t / \partial C_t)$, derived in Section 11.6, by the chain rule

$$\frac{\partial V_t}{\partial E_t} = \frac{\partial V_{t+1}}{\partial M_{t+1}^{\text{AT}}} \frac{\partial M_{t+1}^{\text{AT}}}{\partial E_t}$$

together with the carbon-to-CO₂ unit conversion noted above. In a first-best allocation, the optimal carbon tax equals the SCC (Golosov et al., 2014). The SCC is high when climate damages are steep, the climate response is strong, discounting is low, and tipping risks are material (Cai and Lontzek, 2019; Dietz, 2024).

From surrogates to climate IAMs. Chapter 9 introduced surrogates and Bayesian active learning as fast approximators for repeated model evaluations. Climate IAMs are the natural application: each parameter configuration (climate sensitivity, damage curvature, discount rate) is expensive to solve, yet policy questions require evaluating thousands of configurations to map out tail risks and Pareto-improving rules. The DEQN approach of Chapters 2–3, combined with the GP and active-learning toolkit of Chapter 9, is therefore the natural workhorse for climate-policy uncertainty quantification.

Why computation matters. Solving IAMs globally, as opposed to linearization or certainty equivalence, is computationally demanding for several reasons:

- **Nonstationarity:** TFP, population, emissions intensity, and radiative forcing all change exogenously over time, so the policy function cannot be time-invariant.
- **Coupled dynamics:** the economy and climate interact in both directions through emissions and damages.
- **Long horizon:** welfare effects unfold over 100–300 years, requiring stable numerical solutions far from the steady state.

- **Curse of dimensionality:** multiple climate state variables (carbon stocks, temperature layers), stochastic shocks, and uncertain parameters raise the dimension of the state space well beyond what standard grid-based methods can handle.

The deep learning toolkit developed in this course (DEQNs, deep surrogates, and GP-based uncertainty quantification) is therefore particularly well suited to climate economics.

The three movements of this chapter. The remainder of this chapter has three movements. Movement 1 (§11.3–§11.5) makes precise what changes when we ask the Deep Equilibrium Network of Chapter 2 to solve a non-stationary IAM, and presents the modified training algorithm in one labeled box. Movement 2 (§11.6–§11.8) puts that algorithm to work on a concrete stochastic DICE economy. Movement 3 (§11.9–§11.12) sketches the four extensions that matter for serious climate-finance research: Bayesian learning on the climate sensitivity, recursive Epstein–Zin preferences, global uncertainty quantification of the social cost of carbon, and constrained Pareto-improving carbon-tax design in a heterogeneous-agent IAM.

11.2 The DICE Model

11.2.1 The IAM Landscape

DICE is the workhorse of this chapter, but it is one of several integrated assessment models in active use. The list below summarizes the active landscape; each model trades global parsimony for regional or sectoral granularity, and computational tractability for fidelity of the climate physics.

DICE. Global aggregate; 3-box carbon cycle and 2-layer energy-balance model; one-sector Ramsey planner. The standard benchmark for SCC and integrated policy analysis (Nordhaus, 2017).

RICE. Twelve-region extension of DICE with trade (Nordhaus and Yang, 1996). Used for regional SCC and equity questions.

CDICE. A global DICE-2016 recalibration tailored to deep-learning solution methods, with Epstein–Zin preferences and OLG variants. The model used in §11.6 below (Folini et al., 2025).

ACE. Analytic Climate Economy: log-linear approximations to the carbon cycle, temperature dynamics, and damages yield a *closed-form* optimal carbon tax (Traeger, 2023). Acts as an analytic benchmark for the numerical SCC computed below.

FaIR / MAGICC. Reduced-complexity climate emulators that take emissions as input and produce temperature responses; widely used to translate IPCC scenarios to economic models.

WITCH / REMIND. Multi-region IAMs with full energy-system modules; standard for mitigation-pathway and technology-portfolio studies. Outside the scope of this script.

CDICE is the model we solve in this chapter. ACE provides a useful analytic shadow for it, in particular a closed-form SCC that decomposes transparently into structural parameters; we do not derive that closed form here, but Exercise 3 asks the reader to compute it from Traeger (2023) and compare against the DEQN-trained CDICE solution as an external sanity check.

The *Dynamic Integrated model of Climate and the Economy* (DICE), developed by Nordhaus (1994), is the most influential IAM; in this chapter we follow the variant of Nordhaus (2017) as recalibrated by Folini et al. (2025). It couples a neoclassical growth model with a reduced-form

climate module in a single global framework. The remainder of this section builds the model up block by block, in increasing complexity: first the macro-economic backbone (§11.2.2), then the emissions and abatement technology (§11.2.3), then the climate physics (§11.2.5–§11.2.6), and finally the damage feedback (§11.2.7) that closes the loop. A consolidated calibration is given in Table 11.2.

Time-step convention. Following Folini et al. (2025) we calibrate CDICE on an *annual* time step, $\Delta_t = 1$ year, so that all rates in Table 11.2 (capital depreciation δ , pure rate of time preference ρ , the decay rates $g_0^\sigma, \delta^\sigma, g^{\text{back}}, \delta^{\text{Land}}$, the carbon-cycle transfer rates b_{12}, b_{23} , and the temperature-block coefficients c_1, c_3, c_4) are read directly as annual values; the original DICE-2016 calibration of Nordhaus (2017), by contrast, hard-wires a 5-year time step into its coefficients. Growth rates of TFP and population are written as annual log changes, $g_t^A := \ln(A_{t+1}/A_t)$ and $g_t^L := \ln(L_{t+1}/L_t)$, and the dynamics (11.13), (11.15)–(11.16) and the FOC residuals of §11.6 therefore carry no Δ_t multipliers; emissions E_t entering (11.13) are the annual total. Switching to a non-annual Δ_t amounts to reinserting the multiplications $\Delta_t \cdot \{g_0^\sigma, \delta^\sigma, g^{\text{back}}, \delta^{\text{Land}}, b_{12}, b_{23}, c_1, c_3, c_4\}$ in the obvious places, the time-step-generic form discussed in Online Appendix D of Folini et al. (2025).

11.2.2 Production and the Ramsey–Cass–Koopmans backbone

Strip away the climate block and DICE is just a neoclassical growth model with population and TFP growth. A single representative firm produces gross output with Cobb–Douglas technology in capital and effective labor,

$$Y_t^{\text{gross}} = K_t^\alpha (A_t L_t)^{1-\alpha}, \quad (11.2)$$

where $\alpha \in (0, 1)$ is the capital share, A_t is total factor productivity, and L_t is population. Both A_t and L_t follow deterministic but time-varying paths: A_t trends because of exogenous productivity growth, and L_t follows the calibrated demographic projection of Nordhaus (2017). The capital stock evolves under the standard accumulation law

$$K_{t+1} = (1 - \delta)K_t + I_t, \quad (11.3)$$

with depreciation rate δ and gross investment I_t . The economy’s resource constraint, written in terms of net (after-damages, after-abatement) output that we develop in §11.2.3–§11.2.7, is $C_t + I_t \leq Y_t^{\text{net}}$, where C_t is aggregate consumption.

A benevolent planner picks $(C_t, I_t, \mu_t)_{t \geq 0}$ to maximize a discounted CRRA-IES felicity sum,

$$V_0 = \sum_{t=0}^{\infty} \beta_t L_t \frac{(C_t/L_t)^{1-1/\psi} - 1}{1 - 1/\psi}, \quad \beta_t = \exp(-\rho \Delta_t \cdot t), \quad (11.4)$$

with intertemporal-elasticity-of-substitution parameter $\psi > 0$ and pure rate of time preference ρ . This is the time-additive aggregator of the standard Ramsey–Cass–Koopmans growth model; we replace it with the recursive Epstein–Zin form once stochastic risk enters the picture (§11.10). The planner controls μ_t , the emissions abatement rate, in addition to the savings–consumption split; we develop the cost of abatement next.

11.2.3 Industrial emissions, abatement, and the backstop technology

Industrial production is a CO₂-emitting activity. Let σ_t denote the *carbon intensity* of gross output, expressed in CDICE’s working units of 10^3 GtC of emissions per unit of gross output (a 10^3 GtC normalization on the carbon stocks improves the conditioning of the climate side; see Table 11.2). Industrial emissions are then $\sigma_t Y_t^{\text{gross}}$ before any mitigation effort; with abatement rate $\mu_t \in [0, 1]$ the planner can scale these emissions down,

$$E_{\text{ind},t} = (1 - \mu_t) \sigma_t Y_t^{\text{gross}} = (1 - \mu_t) \sigma_t K_t^\alpha (A_t L_t)^{1-\alpha}. \quad (11.5)$$

Carbon intensity is itself an exogenous decreasing time path. DICE-2016 calibrates a closed-form decay,

$$\sigma_t = \sigma_0 \exp \left[\frac{g_0^\sigma}{\log(1 + \delta^\sigma)} ((1 + \delta^\sigma)^t - 1) \right], \quad (11.6)$$

with initial intensity σ_0 , initial growth rate $g_0^\sigma < 0$ (so emissions per dollar of output fall over time), and second-derivative parameter $\delta^\sigma > 0$ that bends the path further down at long horizons. Equation (11.6) captures the steady decarbonization that even *unabated* world output undergoes through ongoing technological change; the planner's μ_t is the additional mitigation effort *on top of* that baseline.

Abatement is not free. In the spirit of an aggregate marginal-abatement-cost curve, DICE assumes the abatement-cost share of gross output is a power function of μ_t ,

$$\Theta(\mu_t) = \theta_{1,t} \mu_t^{\theta_2}, \quad (11.7)$$

with curvature parameter $\theta_2 > 1$ (a typical calibration is $\theta_2 = 2.6$). The level coefficient $\theta_{1,t}$ is not a free parameter: it is pinned down by the cost of the *backstop technology*, the cleanest large-scale abatement technology available at any given time (e.g. direct air capture). Let p_t^{back} denote the cost per unit of CO₂ avoided when the backstop is fully deployed, and assume an exogenous declining path,

$$p_t^{\text{back}} = p_0^{\text{back}} \exp(-g^{\text{back}} t), \quad (11.8)$$

reflecting steady cost reductions in clean technologies. Setting the marginal abatement cost at $\mu_t = 1$ equal to the backstop price (multiplied by carbon-to-CO₂ conversion c_{co2} to keep mass units consistent, and by 10^3 to convert σ_t from 10^3 GtC working units back to GtC) yields the calibration identity

$$\theta_{1,t} = \frac{p_t^{\text{back}} \cdot 10^3 \cdot c_{\text{co2}} \cdot \sigma_t}{\theta_2}. \quad (11.9)$$

Equation (11.9) is what makes $\Theta(\mu)$ *economically* meaningful rather than a fitted polynomial: the abatement-cost function inherits its level from the backstop price and its curvature from the assumption $\theta_2 = 2.6$. The 10^3 factor matches Equation (11) of Online Appendix D of [Folini et al. \(2025\)](#) and the corresponding factor of 1000 in the companion implementation. As the backstop becomes cheaper ($p_t^{\text{back}} \downarrow$), full mitigation becomes cheaper too, which is one of the channels that makes the deterministic optimal μ_t rise toward 1 over the 21st century.

The bound $\mu_t \in [0, 1]$ deserves a comment. $\mu_t = 0$ means business-as-usual emissions; $\mu_t = 1$ means full deployment of the backstop, eliminating all industrial emissions. Values $\mu_t > 1$ would correspond to net-negative industrial emissions (e.g. aggressive direct air capture beyond the firm's own footprint), which DICE forbids; we will impose the upper bound as a Kuhn–Tucker constraint, smoothed by a Fischer–Burmeister term, in §11.6.

11.2.4 Land-use emissions and net output

The atmosphere does not distinguish between an industrial flow and a non-industrial flow of carbon. In DICE, total emissions therefore comprise an industrial component (11.5) and an exogenous land-use-change component,

$$E_{\text{Land},t} = E_{\text{Land},0} \exp(-\delta^{\text{Land}} t), \quad (11.10)$$

which decays smoothly toward zero as deforestation slows. Total emissions feeding the atmosphere are

$$E_t = E_{\text{ind},t} + E_{\text{Land},t}. \quad (11.11)$$

Closing the production block requires accounting for two additional drains on gross output: climate damages, governed by atmospheric temperature T_t^{AT} via a damage fraction $\Omega(T_t^{\text{AT}})$ developed in §11.2.7, and abatement spending (11.7). Net output is therefore

$$Y_t^{\text{net}} = (1 - \Omega(T_t^{\text{AT}}) - \Theta(\mu_t)) Y_t^{\text{gross}}, \quad (11.12)$$

which is what is available for consumption and investment. The additive form is the convention adopted in CDICE and used by the production-grade DEQN library port; an alternative multiplicative form $(1 - \Omega^{\text{ret}})(1 - \Theta)$ with retained-output factor Ω^{ret} appears in Nordhaus (2008).

The planner's controls and exogenous trends are now all named. The endogenous economic state is the capital stock K_t . The exogenous trends are TFP A_t , population L_t , carbon intensity σ_t , land-use emissions $E_{\text{Land},t}$, and (added below) the non-CO₂ component of radiative forcing F_t^{EX} . The planner controls the consumption–investment split (equivalently, the savings rate s_t) and the abatement rate $\mu_t \in [0, 1]$. All that remains is the climate side: the carbon cycle that turns total emissions E_t into atmospheric concentration, the energy balance that turns concentration into temperature, and the damage function that turns temperature back into output loss.

11.2.5 Carbon cycle

DICE represents the global carbon cycle as a three-reservoir linear system: an atmospheric box, an upper (mixed-layer) ocean box, and a lower (deep) ocean box. Carbon flows between reservoirs at calibrated rates, and total emissions E_t from (11.11) enter directly into the atmospheric reservoir. Stacking concentrations as $M_t = (M_t^{\text{AT}}, M_t^{\text{UO}}, M_t^{\text{LO}})^{\top}$, the transition is

$$M_{t+1} = (I + B) M_t + \mathbf{e}_1 E_t, \quad (11.13)$$

where $\mathbf{e}_1 = (1, 0, 0)^{\top}$ injects emissions into the atmosphere alone, E_t is the per-period emissions total, and the transfer matrix

$$B = \begin{pmatrix} -b_{12} & b_{12} M_{\text{eq}}^{\text{AT}}/M_{\text{eq}}^{\text{UO}} & 0 \\ b_{12} & -b_{12} M_{\text{eq}}^{\text{AT}}/M_{\text{eq}}^{\text{UO}} - b_{23} & b_{23} M_{\text{eq}}^{\text{UO}}/M_{\text{eq}}^{\text{LO}} \\ 0 & b_{23} & -b_{23} M_{\text{eq}}^{\text{UO}}/M_{\text{eq}}^{\text{LO}} \end{pmatrix} \quad (11.14)$$

encodes the two atmosphere–upper-ocean exchange rates (b_{12} in either direction) and the two upper-ocean–lower-ocean exchange rates (b_{23} in either direction). The off-diagonal scaling by the equilibrium-mass ratios $M_{\text{eq}}^{\text{AT}}/M_{\text{eq}}^{\text{UO}}$ and $M_{\text{eq}}^{\text{UO}}/M_{\text{eq}}^{\text{LO}}$ guarantees that, under zero net emissions, the system relaxes to the calibrated pre-industrial equilibrium $M_{\text{eq}} = (M_{\text{eq}}^{\text{AT}}, M_{\text{eq}}^{\text{UO}}, M_{\text{eq}}^{\text{LO}})^{\top}$. Calibrated values for b_{12} , b_{23} , and M_{eq} in CDICE are listed in Table 11.2. The lecture slides for this chapter sometimes write the same transition with four directional rates $\phi_{12}, \phi_{21}, \phi_{23}, \phi_{32}$ in place of b_{12} and b_{23} ; the two parameterizations are identical under $\phi_{12} = b_{12}$, $\phi_{21} = b_{12} M_{\text{eq}}^{\text{AT}}/M_{\text{eq}}^{\text{UO}}$, $\phi_{23} = b_{23}$, $\phi_{32} = b_{23} M_{\text{eq}}^{\text{UO}}/M_{\text{eq}}^{\text{LO}}$, i.e. the slide form makes the equilibrium-mass scaling absorbed into B explicit at the cost of two extra symbols.

Equation (11.13) is a pulse-and-decay system: a unit pulse of emissions raises atmospheric carbon by one unit instantaneously, and that anomaly then bleeds into the upper ocean over decades and into the deep ocean over centuries. Figure 11.2 shows the implied BAU emissions trajectory under nine alternative climate-module calibrations; the spread is mostly driven by the equilibrium climate sensitivity (developed in §11.2.6), not by the carbon cycle, which is tightly disciplined by the pulse and step tests of §11.2.8.

11.2.6 Two-layer energy balance and radiative forcing

A two-layer energy balance model links carbon concentrations to temperature:

$$T_{t+1}^{\text{AT}} = T_t^{\text{AT}} + c_1(F_t - \lambda T_t^{\text{AT}} - c_3(T_t^{\text{AT}} - T_t^{\text{OC}})), \quad (11.15)$$

$$T_{t+1}^{\text{OC}} = T_t^{\text{OC}} + c_4(T_t^{\text{AT}} - T_t^{\text{OC}}), \quad (11.16)$$

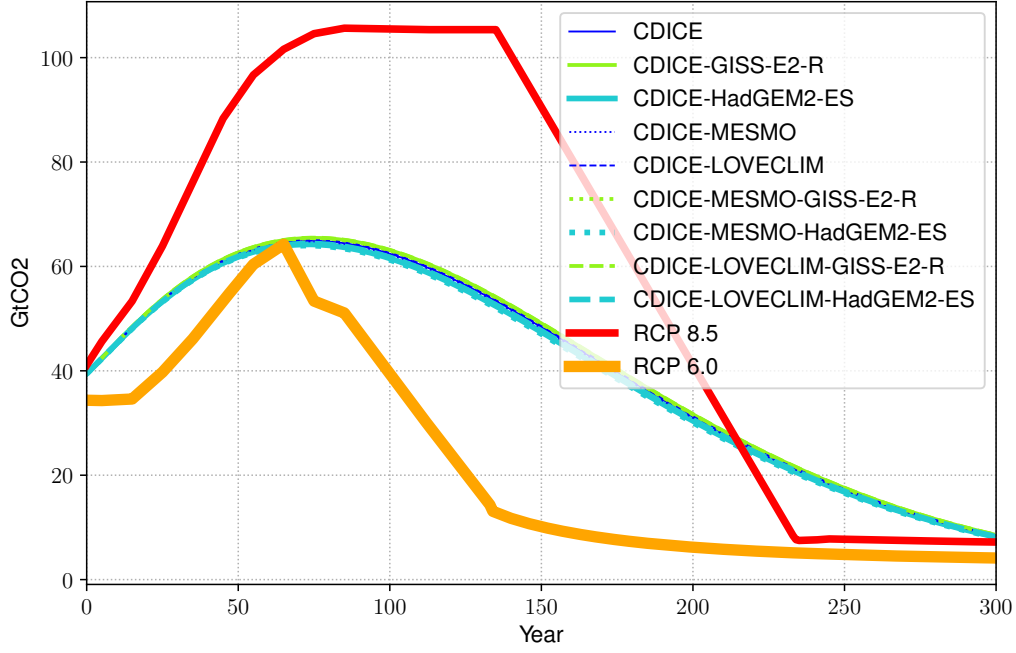


Figure 11.2: Business-as-usual industrial emissions in CDICE (in GtCO₂/yr) under the nine combinations of three carbon-cycle calibrations (MMM, MESMO, LOVECLIM) and three temperature calibrations (MMM, HadGEM2-ES, GISS-E2-R); the thin CDICE curves overlap visually, confirming that the BAU emissions path is essentially insensitive to the climate-module calibration because σ_t and A_t are exogenous. The thick red and orange curves are the RCP 8.5 and RCP 6.0 scenarios, included as climate-policy reference paths. Reproduced from [Folini et al. \(2025\)](#), Figure 11(a).

where radiative forcing is

$$F_t = F_{2 \times \text{CO}_2} \frac{\log(M_t^{\text{AT}}/M_{\text{PI}}^{\text{AT}})}{\log 2} + F_t^{\text{EX}}. \quad (11.17)$$

Figure 11.3 summarizes the full topology of the climate side: industrial emissions enter the atmospheric carbon stock, leak into the upper and lower ocean reservoirs at calibrated rates, raise radiative forcing through the logarithmic CO₂ term, and warm the atmospheric and ocean temperature layers through the two-layer energy balance.

The parameter $\lambda = F_{2 \times \text{CO}_2}/\Delta T_{\text{AT}, \times 2}$ is determined by the *equilibrium climate sensitivity* (ECS), defined as the long-run atmospheric warming from a doubling of CO₂ concentration. We treat λ as a deterministic constant in the baseline model; §11.9 promotes it to a learnable Gaussian parameter, with the additive feedback term $\varphi_{1C} \tilde{f}_{t+1} T_t^{\text{AT}}$ entering the right-hand side of (11.15) and the coefficient φ_{1C} defined in that subsection. ECS is one of the most consequential and uncertain parameters in climate science ([Roe and Baker, 2007](#); [Knutti et al., 2017](#)). Observational and model-based estimates place ECS in a *likely* (66 %) range of 2.5°C–4°C and a *very likely* (90 %) range of 2°C–5°C, with a best estimate of approximately 3°C (IPCC AR6 Synthesis Report; [Calvin et al., 2023](#)); ECS uncertainty is one of the largest single sources of variance in the SCC.

11.2.7 Damage function: closing the climate–economy loop

The damage function is what turns a temperature anomaly back into an output loss, and so it is what closes the economy–climate–damages feedback loop drawn schematically in Figure 11.1. Following the convention in [Folini et al. \(2025\)](#), Online Appendix D, we treat $\Omega(T_{\text{AT}})$ as the *damage fraction* of gross output (the fraction lost to climate damages, increasing in T_{AT}), and the

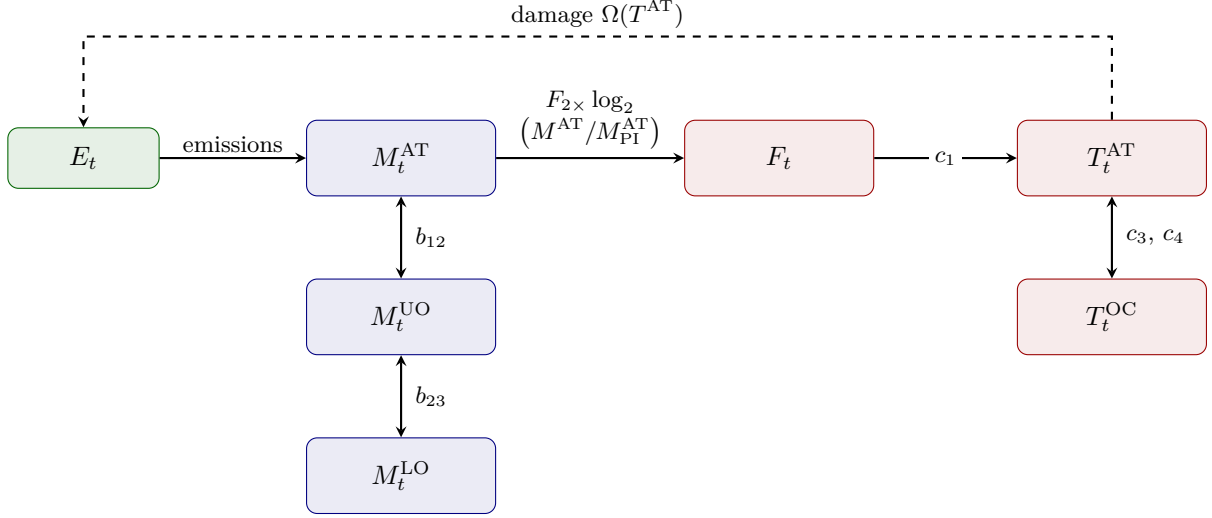


Figure 11.3: Topology of the CDICE climate side. Total emissions E_t enter the atmospheric carbon box M_t^{AT} , leak into the upper- and lower-ocean carbon boxes at exchange rates b_{12} and b_{23} , and drive radiative forcing F_t through the logarithmic CO_2 relation. The two-layer energy balance maps F_t into the atmospheric temperature T_t^{AT} via c_1 , with c_3, c_4 governing the heat exchange between atmosphere and ocean. The dashed arrow closes the loop through the damage function back into output (developed in §11.2.7). Five climate states ($M^{\text{AT}}, M^{\text{UO}}, M^{\text{LO}}, T^{\text{AT}}, T^{\text{OC}}$) form the climate-side block of the DEQN state vector (11.24).

abatement-cost fraction $\Theta(\mu)$ from (11.7) as a separate output drain. The two enter additively in net output (11.12); an alternative multiplicative form $(1 - \Omega^{\text{ret}})(1 - \Theta)$ with retained-output factor Ω^{ret} is used by Nordhaus (2008).

The workhorse specification is Nordhaus (2008)’s quadratic,

$$\Omega^N(T_{\text{AT}}) = \pi_1 T_{\text{AT}} + \pi_2 T_{\text{AT}}^2, \quad (11.18)$$

which is relatively benign for moderate warming and is what we use in the deterministic CDICE solve below. Calibrated values (π_1, π_2) are listed in Table 11.2. The damage function (11.18) is the most contested object in the IAM literature: at $T_{\text{AT}} = 3^\circ\text{C}$ above pre-industrial, Nordhaus–quadratic damages amount to roughly 2% of gross output, which several recent empirical literatures argue is far below realistic central estimates. We therefore treat the damage curvature π_2 as one of the two key uncertain parameters in the deep-UQ analysis of §11.11 (the other being the equilibrium climate sensitivity).

For the tipping-point branch of the literature, Weitzman (2012) argued that catastrophic thresholds require a steeper damage function,

$$\Omega^W(T_{\text{AT}}) = 1 - \frac{1}{1 + \left(\frac{1}{\psi_1} T_{\text{AT}}\right)^2 + \left(\frac{1}{2TP} T_{\text{AT}}\right)^{6.754}}, \quad (11.19)$$

where TP is a stochastic tipping-point threshold. We do not solve a Weitzman damage variant in the baseline CDICE-DEQN, but the OLG-IAM of §11.12 introduces a stylized tipping risk in the same spirit; the degree of convexity of the damage function is one of the most important determinants of the optimal carbon tax.

11.2.8 CDICE: recalibration of the climate module

A key contribution of Folini et al. (2025) is a systematic recalibration of the DICE climate module against benchmarks from climate science model archives (CMIP). Their CDICE framework retains

the same functional forms as DICE but fits parameters to the four-test protocol summarized in Table 11.1. This calibration ensures that the reduced-form climate module is consistent with

Table 11.1: CDICE climate-module calibration protocol. The first two tests discipline the carbon-cycle and temperature-response blocks directly; the last two check whether the calibrated reduced-form module remains accurate on out-of-sample and historically realistic forcing paths.

Test	Target	Use
1. Carbon pulse (100 GtC)	Atmospheric retention path	Calibrate carbon cycle
2. $4\times\text{CO}_2$ step	Temperature impulse response	Calibrate temperature block
3. 1% CO_2/year	Transient climate response	Out-of-sample validation
4. Historical + RCP	Realistic forcing paths	End-to-end validation

state-of-the-art earth system models. CDICE also introduces a transparent time-step formulation, $X_{t+\Delta t} = X_t + \Delta t \cdot f(X_t, u_t; \theta)$, that allows coherent implementation at annual, 5-year, or 10-year resolution within a single generic framework. Figure 11.4 illustrates how much the climate-cycle calibration matters even before the planner makes any decision: under business-as-usual, DICE-2016 and CDICE produce visibly different atmospheric carbon trajectories, and the gap propagates into temperature, damages, and ultimately the SCC.

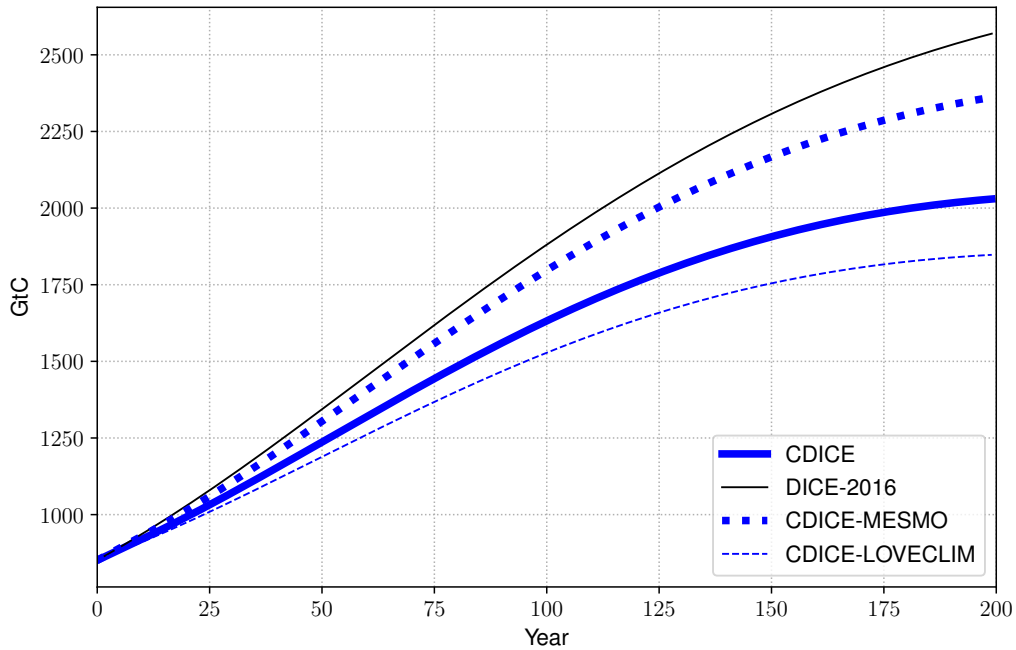


Figure 11.4: Atmospheric carbon M_t^{AT} along the BAU path (in GtC, over 200 years from 2015) under the three CDICE carbon-cycle calibrations (CDICE = MMM, CDICE-MESMO, CDICE-LOVECLIM) and the legacy DICE-2016 carbon cycle. Only the carbon-cycle block is varied here; the temperature block is held at the CDICE MMM calibration, since the BAU carbon-stock path does not depend on the temperature calibration to first order. The DICE-2016 path lies systematically above the CMIP-disciplined paths, reflecting that the original DICE carbon cycle overstates atmospheric retention; CDICE-MESMO and CDICE-LOVECLIM bracket the CDICE baseline on the slow-removal and fast-removal sides, respectively. Reproduced from Folini et al. (2025), Figure 15(a).

11.2.9 Calibration and initial conditions, in one place

The block-by-block model description above introduces a fairly large set of parameters. Table 11.2 consolidates the calibration we use throughout the rest of the chapter, lifted from the Online Appendix of Folini et al. (2025). Two CMIP5 alternatives (HadGEM2-ES and GISS-E2-R) are shown alongside the multi-model mean (MMM) so that the deep-UQ analysis of §11.11 has a concrete distribution to draw from. We follow the CDICE convention of expressing all carbon quantities in 10^3 GtC working units: equilibrium and initial carbon stocks M_{eq} and M_0 , the initial carbon intensity σ_0 , and the initial land-use emissions $E_{\text{Land},0}$ are all on the same scale, which keeps the numerical conditioning of the carbon-cycle and emissions states under control. The factor 10^3 appears explicitly in the abatement-cost calibration (11.9) to convert σ_t back to GtC when it is multiplied by the backstop price; a reader comparing values against raw DICE-2016 numbers (e.g. ~ 2.6 GtC/yr land-use emissions, ~ 851 GtC atmospheric carbon in 2015) should multiply the table entries by 10^3 first.

11.2.10 The full IAM, summarized

Pulling the previous subsections together, CDICE is a deterministic dynamical system on a finite-dimensional state vector that the planner steers with two controls. The endogenous state at date t is the sextuple

$$\mathbf{X}_t^{\text{end}} = (K_t, M_t^{\text{AT}}, M_t^{\text{UO}}, M_t^{\text{LO}}, T_t^{\text{AT}}, T_t^{\text{OC}}), \quad (11.20)$$

the exogenous-trend vector is

$$\mathbf{X}_t^{\text{exo}} = (A_t, L_t, \sigma_t, E_{\text{Land},t}, F_t^{\text{EX}}), \quad (11.21)$$

and the planner's controls are (C_t, μ_t) (equivalently (K_{t+1}, μ_t) , since investment is determined by the resource constraint $C_t + I_t = Y_t^{\text{net}}$ together with (11.3)). The transitions are: capital from (11.3) with $I_t = Y_t^{\text{net}} - C_t$; total emissions from (11.11), fed into the carbon cycle (11.13); temperature from (11.15)–(11.16) with forcing (11.17); and net output, hence the resource constraint, from (11.12). The objective is the discounted CRRA-IES felicity sum (11.4) subject to $\mu_t \in [0, 1]$.

That is the entire deterministic IAM. Every primitive named above has a closed-form expression and a calibrated parameter (Table 11.2); the only thing left is to find the optimal policy $(C_t, \mu_t)_{t \geq 0}$. The model is intrinsically non-stationary. §11.3 makes that observation precise; the stationary DEQN of Chapter 2 needs to be amended before we can solve this system.

11.3 Why DICE Breaks the Stationary DEQN

This is the technical pivot of the chapter. The stationary DEQN of Chapter 2 was designed for models whose policy function is a fixed point of a Bellman operator on an ergodic state space. IAMs satisfy neither premise. Three structural features break the stationarity assumption simultaneously, and each must be addressed before the DEQN can be trained at all.

Time-varying state distributions with no ergodic limit. The endogenous state of a stationary DSGE is the projection of a recurrent Markov chain onto a finite-dimensional vector; the policy function lives on its stationary distribution. In an IAM the analogue object does not exist within the planning horizon. Atmospheric carbon M_t^{AT} rises from a pre-industrial baseline of ~ 600 GtC to a peak of ~ 1500 GtC over a century, then decays over millennia; atmospheric temperature T_t^{AT} follows with a multi-decade lag and a multi-century relaxation. Within the 300 years the planner cares about, neither variable ever returns to a state it has been in before. The state visited at $t = 100$ is therefore not exchangeable with the state visited at $t = 200$, and a

time-invariant policy function $\mathbf{p}(\mathbf{X}_t)$ that depends only on the endogenous state misses the whole point of the exercise: the optimal mitigation effort at a given $(M^{\text{AT}}, T^{\text{AT}})$ depends on whether that state was reached on the way up or on the way down. Cf. the curse-of-dimensionality discussion in §2.1: it is not the size of the state space that breaks the DEQN here, it is the lack of recurrence.

Deterministic drift through exogenous trends. Even setting the carbon and temperature stocks aside, the IAM is drifting deterministically. Total factor productivity A_t trends up at a calibrated, time-varying rate; population L_t follows the demographic projection of Nordhaus (2017); carbon intensity σ_t falls along the closed-form decay (11.6); land-use emissions $E_{\text{Land},t}$ decay smoothly (11.10); the backstop price p_t^{back} falls (11.8); the exogenous non-CO₂ forcing F_t^{EX} follows a fitted RCP trajectory; and the abatement-cost level $\theta_{1,t}$ inherits the time dependence of σ_t and p_t^{back} through (11.9). Seven exogenous trends drive the model even before a shock is introduced. A time-invariant policy can never see them, and replacing them with their long-run averages is exactly the certainty-equivalence move that defeats the purpose of solving the model globally.

Finite calendar-time horizon. A stationary DEQN trains under a transversality condition: as $t \rightarrow \infty$, the discounted shadow price of capital goes to zero, and the iterative-projection loss inherits that fixed-point structure for free. An IAM is not solved on $[0, \infty)$. The planning horizon is a finite calendar date (the notebooks of §11.7 run roughly three centuries from a 2015 start), so transversality is not available and the policy is solved over a finite forward sweep instead.

Putting it together. These features compound and explain why a time-invariant DEQN of Chapter 2 cannot be used here without modification. The next two sections operationalize the response: §11.4 reorganizes the network inputs to include calendar time, and §11.5 states the resulting training algorithm as a labeled diff against the stationary DEQN box of §2.3.

11.4 What Changes in the DEQN Setup

We now translate this into one concrete design choice for the network inputs. The autodiff machinery, the squared-residual structure, and the rest of the training loop of Chapter 2 carry over unchanged; this is a refactor of what the network sees, not a new solver.

11.4.1 Time and trends as states

Calendar time itself enters as a state. Because neural networks prefer bounded inputs, we use the monotone time rescaling $\tau_t = 1 - \exp(-\vartheta t) \in [0, 1)$ of Eq. (11.23). Every exogenous trend $(A_t, L_t, \sigma_t, E_{\text{Land},t}, F_t^{\text{EX}}, p_t^{\text{back}}, \theta_{1,t})$ is then a deterministic function of τ_t , so passing τ_t to the network is informationally equivalent to passing the entire trend bundle. Training trajectories begin from the calibrated 2015 state and run forward over the planner’s horizon.

11.5 The Non-Stationary DEQN Algorithm

The design choice of §11.4 translates into a single training algorithm. The body below is a literal diff against the stationary DEQN of §2.3: unchanged lines are grayed, new or modified lines are bolded.

Algorithm 4 Non-Stationary DEQN Training

Input: Network $\mathcal{N}_\rho$, learning rate η , episodes E , training steps T_{train} ;
[NEW] calibrated initial state $\mathbf{x}_0$ (e.g., the 2015 state) and a planning horizon T_{max}
for episode $e = 1, \dots, E$ **do**
 [CHANGED] **Simulate** K forward trajectories from $\mathbf{x}_0$ over $[0, T_{\text{max}}]$ under the current policy, and collect the time-stamped states $(\tau_t, \mathbf{x}_t)$ into $\mathcal{D}$
 for gradient step $t = 1, \dots, T_{\text{train}}$ **do**
 Draw mini-batch $\mathcal{B} \subset \mathcal{D}$
 Compute loss: $\ell_\rho = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x}_i \in \mathcal{B}} \|G(\mathbf{x}_i, \mathcal{N}_\rho(\mathbf{x}_i))\|^2$
 Update: $\rho \leftarrow \rho - \eta \cdot \nabla_\rho \ell_\rho$
 end for
end for
Output: Trained network $\mathcal{N}_{\rho^*}$ approximating the policy function

One delta against the stationary DEQN box. The simulation step starts from a calibrated initial state $\mathbf{x}_0$ and integrates K trajectories forward through calendar time, so the pool $\mathcal{D}$ contains time-stamped states $(\tau_t, \mathbf{x}_t)$ along finite-horizon trajectories rather than draws from an ergodic distribution. With τ_t in the input the network learns a time-dependent policy; every other line of the box is the stationary DEQN of §2.3 unchanged.

What replaces transversality. Because the pool $\mathcal{D}$ is built from K forward simulations of length T_{max} that all start at the same $\mathbf{x}_0$, every trajectory visits the full calendar window $[0, T_{\text{max}}]$ and a uniform mini-batch draw from $\mathcal{D}$ is therefore stratified across calendar time by construction. The missing transversality condition of §11.3 is absorbed numerically by choosing the horizon long enough that the discounted contribution of the terminal state falls below the training-noise floor: at the CDICE calibration $\rho = 0.015/\text{yr}$ and the notebooks' default $T_{\text{max}} = 300$ years, $\hat{\beta}_t^{T_{\text{max}}} \approx \exp(-\rho T_{\text{max}}) \approx 0.011$, which is one to two orders of magnitude below the achievable residual root-mean-square at convergence. When the horizon must be short (e.g., the 1D toy of Exercise 10), one instead adds an explicit terminal residual $\lambda_T \|\mathbf{x}_{T_{\text{max}}} - \mathbf{x}_{T_{\text{max}}}^{\text{ref}}\|^2$ to the loss; both options are standard in the finite-horizon DEQN literature.

The non-stationary DEQN in one line

One modification, one solver: time enters as a state. The autodiff backbone, the residual structure, the sampling rule, and the update rule carry over from the stationary DEQN of Chapter 2.

11.6 The Planner's Lagrangian and FOCs

Movement 2 puts the non-stationary DEQN of §11.5 to work on the deterministic CDICE economy of §11.2. Solving this system with the algorithm of §11.5 amounts to writing the planner's Lagrangian, deriving the first-order and envelope conditions, normalizing them, treating each FOC as a residual, and minimizing the sum of squared residuals on the time-stamped state pool generated by the forward simulation of §11.5. This section follows Friedl et al. (2023) and Online Appendix D of Folini et al. (2025).

Detrending and state vector, in compact form. The model-rendering choices already named in §11.4.1 carry over verbatim. Variables that grow with the productivity–population product $A_t L_t$ are rescaled to per-effective-capita units:

$$c_t := \frac{C_t}{A_t L_t}, \quad k_t := \frac{K_t}{A_t L_t}. \quad (11.22)$$

Calendar time enters through the bounded rescaling (compatible with the dynamic-programming convention of Traeger (2014)),

$$\tau = 1 - \exp(-\vartheta t) \in [0, 1), \quad \text{with inverse} \quad t = -\frac{\ln(1 - \tau)}{\vartheta}, \quad (11.23)$$

with compression parameter $\vartheta > 0$. The full DEQN state vector then collects the detrended endogenous CDICE states, the bounded time index, the Bayesian-belief states $(\mu_{f,t}, S_{f,t})$ used in §11.9, and a slot for pseudo-state parameters θ used in the UQ analysis of §11.11:

$$\mathbf{X}_t = \left[\underbrace{k_t, M_t^{\text{AT}}, M_t^{\text{UO}}, M_t^{\text{LO}}, T_t^{\text{AT}}, T_t^{\text{OC}}, \mu_{f,t}, S_{f,t}, \tau_t}_{\text{9 endogenous, exogenous, and time states}}; \underbrace{\theta}_{N \text{ pseudo-state parameters}} \right]. \quad (11.24)$$

In the deterministic core developed in this and the next section, only the six endogenous-state entries plus τ_t are active, i.e. a seven-dimensional input vector; $(\mu_{f,t}, S_{f,t})$ and θ are appended only in the extensions of Movement 3.

The Lagrangian. We now derive the equilibrium conditions that the DEQN will be trained against. The derivation follows the standard Lagrangian approach in CRRA-IES form, working directly with the deterministic CDICE primitives of §11.2 (the recursive Epstein–Zin refinement is layered on in §11.10). Write the Lagrangian with multiplier λ_t for the budget constraint $C_t + I_t = Y_t^{\text{net}}$, multipliers $\nu_t^{\text{AT}}, \nu_t^{\text{UO}}, \nu_t^{\text{LO}}$ for the three carbon-reservoir transitions (11.13), multipliers $\eta_t^{\text{AT}}, \eta_t^{\text{OC}}$ for the temperature dynamics (11.15)–(11.16), and KKT multiplier $\lambda_t^\mu \geq 0$ for the abatement bound $\mu_t \leq 1$. The derivation produces ten equilibrium conditions: a consumption FOC, an abatement FOC, the capital Euler equation, the budget/resource constraint, three carbon-stock envelope conditions, two temperature-envelope conditions, and the abatement upper-bound complementarity. Two of these are enforced algebraically (the static consumption and abatement FOCs); the remaining eight become the DEQN residuals of §11.7.

Qualitative overview. Taking derivatives of the Lagrangian with respect to the controls yields:

- **w.r.t. C_t :** the marginal utility of consumption equals the shadow price of the budget, $\partial V^{1-1/\psi} / \partial C_t = \lambda_t$.
- **w.r.t. K_{t+1} :** the shadow value of capital today equals the discounted expected marginal value tomorrow, $\xi_t = e^{-\rho} \partial \mathbb{E}_t[V_{t+1}^{1-\gamma}]^{(1-1/\psi)/(1-\gamma)} / \partial K_{t+1}$.
- **w.r.t. μ_t :** the marginal abatement cost equals the shadow value of reduced emissions (plus the complementarity term if $\mu_t = 1$).

Envelope theorem. Since the FOC for K_{t+1} involves $\partial V / \partial K_{t+1}$, which cannot be computed analytically, we apply the envelope theorem. It provides derivatives of the value function with respect to *current* states, in particular $\partial V / \partial k_t$, $\partial V / \partial M_{\text{AT},t}$, $\partial V / \partial T_t^{\text{AT}}$, which are then shifted forward one period and substituted back into the FOCs.

Capital Euler equation. Combining the FOCs and envelope conditions yields:

$$1 = e^{-\rho} \mathbb{E}_t \left[\left(\frac{V_{t+1}}{(\mathbb{E}_t[V_{t+1}^{1-\gamma}])^{1/(1-\gamma)}} \right)^{1/\psi-\gamma} \cdot \frac{(C_{t+1}/L_{t+1})^{-1/\psi}}{(C_t/L_t)^{-1/\psi}} \cdot R_{t+1}^K \right], \quad (11.25)$$

where R_{t+1}^K is the return on capital inclusive of climate damages. The SCC also appears through the shadow price of atmospheric carbon:

$$\text{SCC}_t^M = -\frac{\partial V_t / \partial M_{\text{AT},t}}{\partial V_t / \partial C_t}. \quad (11.26)$$

This is a shadow value per unit of atmospheric carbon stock. The emissions-based SCC in (11.1) additionally includes the marginal loading of a unit of emissions into $M_{AT,t}$ and the carbon-to-CO₂ unit conversion. At the optimum, the marginal abatement cost equals the carbon tax equals the emissions SCC after these conversions.

Normalization of multipliers. Over a 300-year horizon, A_t and L_t can move the natural scale of marginal utilities and multipliers substantially, with the direction and magnitude depending on the IES through $A_t^{1-1/\psi} L_t$. Such scale drift makes network outputs and gradients harder to optimize stably. Following the detrending logic of (11.22), all multipliers, the budget multiplier, the abatement-bound multiplier, and the five climate envelope multipliers alike, are divided by $A_t^{1-1/\psi} L_t$. The argument for the climate multipliers tracks the budget-multiplier case via the envelope conditions of §11.6 and is spelled out in Online Appendix D of Folini et al. (2025); we adopt the result here:

$$\hat{\lambda}_t := \frac{\lambda_t}{A_t^{1-1/\psi} L_t}, \quad \hat{\lambda}_t^\mu := \frac{\lambda_t^\mu}{A_t^{1-1/\psi} L_t}, \quad \hat{\nu}_t^{\text{AT}} := \frac{\nu_t^{\text{AT}}}{A_t^{1-1/\psi} L_t}, \quad \hat{\nu}_t^{\text{UO}} := \frac{\nu_t^{\text{UO}}}{A_t^{1-1/\psi} L_t}, \quad \dots \quad (11.27)$$

and analogously for the remaining multipliers $\hat{\nu}_t^{\text{LO}}$, $\hat{\eta}_t^{\text{AT}}$, and $\hat{\eta}_t^{\text{OC}}$. The normalization induces an *effective discount factor* that absorbs the trend growth in the per-effective-capita Euler equation,

$$\hat{\beta}_t := \exp\left(-\rho + \left(1 - \frac{1}{\psi}\right) g_t^A + g_t^L\right), \quad (11.28)$$

where $g_t^A := \ln(A_{t+1}/A_t)$ and $g_t^L := \ln(L_{t+1}/L_t)$ are annual log changes. Equation (11.28) mirrors Equation (38) of Online Appendix D of Folini et al. (2025): the population term enters linearly because L_t enters the felicity weight $L_t(C_t/L_t)^{1-1/\psi}$ linearly, while the productivity term inherits the $1 - 1/\psi$ exponent from the per-effective-capita rescaling of consumption. All intertemporal equations below use $\hat{\beta}_t$ in place of $e^{-\rho}$. For a non-annual time step, replace ρ by $\rho\Delta_t$ and g_t^A, g_t^L by their per-period analogues.

Sign convention for the climate multipliers. We adopt the value-derivative convention throughout the script: each climate multiplier $\hat{\nu}_t^{\text{AT}}$, $\hat{\nu}_t^{\text{UO}}$, $\hat{\nu}_t^{\text{LO}}$, $\hat{\eta}_t^{\text{AT}}$, $\hat{\eta}_t^{\text{OC}}$ is the (normalized) partial derivative of the value function with respect to the corresponding climate state. Because extra atmospheric carbon lowers welfare, $\hat{\nu}_t^{\text{AT}}$ is non-positive at the optimum, which is why the stock SCC carries a minus sign, $\text{SCC}_t^M = -\hat{\nu}_t^{\text{AT}}/\hat{\lambda}_t$. The companion implementation in `dice_2p_surrogate_lib.py` stores the *positive marginal damage* $-\hat{\nu}_t^{\text{AT}}$ as a network output for numerical conditioning and flips the sign explicitly inside each residual; the algebra below uses the script convention, so the reader who compares the equations to the code will see one extra sign flip per carbon-multiplier term.

Symbol cheat-sheet for the multipliers. Before writing the FOCs and the loss, Table 11.3 collects the multipliers that the DEQN learns and their role; subsequent equations use the hat-normalized form throughout.

Symbol	Multiplier on	Sign at optimum	Network output?
$\hat{\lambda}_t$	Budget constraint $C_t + I_t = Y_t^{\text{net}}$	> 0	yes (softplus)
$\hat{\lambda}_t^\mu$	Abatement upper bound $\mu_t \leq 1$	≥ 0	no (implied, Eq. 11.37)
$\hat{\nu}_t^{\text{AT}}$	Atmospheric carbon transition $M_{t+1}^{\text{AT}} = \dots$	≤ 0	yes (stored as $-\hat{\nu}_t^{\text{AT}} > 0$ via softplus)
$\hat{\nu}_t^{\text{UO}}$	Upper-ocean carbon transition	≤ 0	yes (linear)
$\hat{\nu}_t^{\text{LO}}$	Lower-ocean carbon transition	≤ 0	yes (linear)
$\hat{\eta}_t^{\text{AT}}$	Atmospheric temperature transition	≤ 0	yes (linear)
$\hat{\eta}_t^{\text{OC}}$	Ocean temperature transition	≤ 0	yes (linear)

Table 11.3: Normalized Lagrange multipliers in the CDICE–DEQN. All values are divided by $A_t^{1-1/\psi} L_t$ relative to the raw multipliers, so the hatted versions inherit the per-effective-capita scale that the network outputs see. The atmospheric carbon multiplier carries the SCC up to the marginal-utility denominator: $\text{SCC}_t^M = -\hat{\nu}_t^{\text{AT}}/\hat{\lambda}_t$.

Key FOCs in normalized form. After normalization, the most important first-order conditions become (see Online Appendix D of Folini et al. (2025) for the complete set of 14 equations):

$$\frac{\partial \mathcal{L}}{\partial c_t} = 0 \Leftrightarrow c_t^{-1/\psi} A_t^{1-1/\psi} L_t - \hat{\lambda}_t = 0, \quad (11.29)$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial k_{t+1}} = 0 \Leftrightarrow & \exp(g_t^A + g_t^L) \hat{\lambda}_t - \hat{\beta}_t \left\{ \hat{\lambda}_{t+1} [(1 - \Omega(T_{\text{AT},t+1}) - \Theta(\mu_{t+1})) \alpha k_{t+1}^{\alpha-1} + (1 - \delta)] \right. \\ & \left. + \hat{\nu}_{t+1}^{\text{AT}} \sigma_{t+1} (1 - \mu_{t+1}) A_{t+1} L_{t+1} \alpha k_{t+1}^{\alpha-1} \right\} = 0, \end{aligned} \quad (11.30)$$

$$\frac{\partial \mathcal{L}}{\partial \mu_t} = 0 \Leftrightarrow \hat{\lambda}_t \Theta'(\mu_t) k_t^\alpha + \hat{\lambda}_t^\mu + \hat{\nu}_t^{\text{AT}} \sigma_t A_t L_t k_t^\alpha = 0. \quad (11.31)$$

Equation (11.30) is the capital Euler equation: it equates the marginal cost of saving one additional unit today (left) to the discounted marginal benefit tomorrow (right), which now includes a term from the atmospheric carbon envelope ($\hat{\nu}_{t+1}^{\text{AT}}$) because higher capital increases output and hence emissions.

Envelope conditions. *Convention reminder.* As stated in §11.2.1, CDICE is calibrated on an annual time step and the coefficients $b_{12}, b_{23}, c_1, c_3, c_4$ in Table 11.2 are annual rates; consequently no Δ_t multipliers appear in either the dynamics (11.13), (11.15)–(11.16) or in the FOC residuals below.

Differentiating the Lagrangian with respect to *state* variables and shifting forward one period yields the shadow prices of the climate stocks. For example, the atmospheric carbon envelope is:

$$\frac{\partial \mathcal{L}}{\partial M_{\text{AT},t+1}} = 0 \Leftrightarrow \hat{\nu}_t^{\text{AT}} - \hat{\beta}_t \left[\hat{\nu}_{t+1}^{\text{AT}} (1 - b_{12}) + \hat{\nu}_{t+1}^{\text{UO}} b_{12} + \hat{\eta}_{t+1}^{\text{AT}} c_1 F_{2 \times \text{CO}_2} \frac{1}{\ln 2 M_{\text{AT},t+1}} \right] = 0. \quad (11.32)$$

This equation says that the current shadow price of atmospheric carbon ($\hat{\nu}_t^{\text{AT}}$) must equal the discounted future effects through three channels: persistence in the atmosphere (b_{12} term), diffusion into the upper ocean ($\hat{\nu}_{t+1}^{\text{UO}}$ term), and radiative forcing on temperature ($\hat{\eta}_{t+1}^{\text{AT}}$ term). It is the existence of these climate multipliers that distinguishes the IAM from the purely economic models of Chapters 2–4.

Fischer–Burmeister complementarity for abatement. The abatement rate is bounded above by 1 (full abatement), giving the KKT condition:

$$1 - \mu_t \geq 0 \quad \perp \quad \hat{\lambda}_t^\mu \geq 0, \quad (11.33)$$

which is non-smooth at $\mu_t = 1$. As in the borrowing-constraint treatment of Chapter 5 (Section 5.4), we replace it with the Fischer–Burmeister smooth approximation:

$$\Psi^{\text{FB}}(\hat{\lambda}_t^\mu, 1 - \mu_t) = \hat{\lambda}_t^\mu + (1 - \mu_t) - \sqrt{(\hat{\lambda}_t^\mu)^2 + (1 - \mu_t)^2 + \varepsilon_{\text{FB}}} = 0, \quad (11.34)$$

with the same regularization parameter $\varepsilon_{\text{FB}} \geq 0$ used in Chapters 3–5. In CDICE-DEQN we take $\varepsilon_{\text{FB}} = 10^{-6}$, equivalent to the IRBC chapter’s $\varepsilon = 10^{-3}$ under its ε^2 convention; the trained policy is insensitive to the choice in the range 10^{-10} to 10^{-4} . At $\varepsilon_{\text{FB}} = 0$ the zero set of Ψ^{FB} coincides with the positive axes in the $(\hat{\lambda}_t^\mu, 1 - \mu_t)$ -plane, enforcing the original complementarity exactly but the function is non-differentiable at the origin; with $\varepsilon_{\text{FB}} > 0$ the function is differentiable everywhere at the cost of a slight relaxation of exact complementarity.

11.7 From FOCs to a Single Loss

The Lagrangian of §11.6 produces ten equilibrium conditions: the consumption FOC (11.29), the capital Euler (11.30), the abatement FOC (11.31), the budget/resource constraint $C_t + I_t = Y_t^{\text{net}}$, the three carbon-stock envelopes (one of which is the atmospheric-carbon envelope (11.32)), the two temperature-layer envelopes, and the Fischer–Burmeister abatement complementarity (11.34). In the DEQN solver two of these ten are enforced *exactly* by algebraic recovery rather than as squared residuals: the consumption FOC is inverted to yield c_t from $\hat{\lambda}_t$, and the abatement FOC is solved for $\hat{\lambda}_t^\mu$ and the resulting *implied multiplier* is fed straight into the Fischer–Burmeister condition. What remains is an eight-residual sum-of-squares loss with eight network outputs, structurally identical to the stationary DEQN of Chapters 2–3. The only substantive difference is that the network must learn the shadow prices of all five climate state variables (three carbon stocks and two temperature layers) in addition to the economic choices, so that the planner has a gradient signal for how today’s decisions propagate through the carbon cycle and the energy balance into future damages.

Policy network specification. The policy function approximated by the neural network outputs an eight-dimensional vector,

$$\mathcal{N}_\rho(\mathbf{x}_t) \in \mathbb{R}^8 := (k_{t+1}, \mu_t, \hat{\lambda}_t, \hat{\nu}_t^{\text{AT}}, \hat{\nu}_t^{\text{UO}}, \hat{\nu}_t^{\text{LO}}, \hat{\eta}_t^{\text{AT}}, \hat{\eta}_t^{\text{OC}}), \quad (11.35)$$

comprising two choice variables (k_{t+1}, μ_t) , the consumption shadow price $\hat{\lambda}_t$, and the five normalized climate multipliers. Note that the abatement KKT multiplier $\hat{\lambda}_t^\mu$ is *not* a network output: it is recovered algebraically inside the loss (see below). A key difference from the stationary DEQN of Chapters 2–3 is that the network must learn the shadow prices of all climate constraints, not just the economic choices. Without the climate multipliers, the planner would have no gradient signal about how today’s decisions propagate through the carbon cycle and temperature dynamics into future damages.

Bounds and positivity. The output activations of $\mathcal{N}_\rho$ are chosen so that the bound and positivity constraints of the model hold for every input, eliminating the need for additional residuals. The capital level k_{t+1} , the consumption shadow $\hat{\lambda}_t$, and the abatement rate μ_t are each passed through a softplus, which guarantees $k_{t+1} > 0$, $\hat{\lambda}_t > 0$ (so consumption recovered via (11.36) is positive), and $\mu_t \geq 0$ exactly. The upper bound $\mu_t \leq 1$ is enforced jointly by the Fischer–Burmeister condition l_8 at the implied multiplier (11.37) and by a small quadratic upper-bound penalty $\propto \mathbb{E}[\max(\mu_t - 1, 0)^2]$ added to the training loss. The atmospheric-carbon shadow $\hat{\nu}_t^{\text{AT}}$ is stored in the implementation as a positive marginal damage (see the sign-convention note in §11.6) and is output through a softplus; the remaining climate multipliers $\hat{\nu}_t^{\text{UO}}, \hat{\nu}_t^{\text{LO}}, \hat{\eta}_t^{\text{AT}}, \hat{\eta}_t^{\text{OC}}$ are unconstrained and use linear output activations.

How is consumption c_t determined? The consumption FOC (11.29) is enforced exactly by inversion rather than as a residual: given the network’s prediction of $\hat{\lambda}_t$, consumption is recovered algebraically as

$$c_t = (\hat{\lambda}_t \cdot A_t^{1/\psi-1} L_t^{-1})^{-\psi}, \quad (11.36)$$

so c_t is not itself a network output. Positivity of c_t is guaranteed because the implementation passes $\hat{\lambda}_t$ through a softplus activation, so $\hat{\lambda}_t > 0$ for every input.

How is the abatement multiplier $\hat{\lambda}_t^\mu$ determined? The same trick handles the abatement FOC (11.31): rather than have the network output $\hat{\lambda}_t^\mu$ and impose the FOC as a separate residual, we *solve* the FOC for $\hat{\lambda}_t^\mu$ and treat the resulting implied multiplier as a deterministic function of the other network outputs. Setting $\partial \mathcal{L} / \partial \mu_t = 0$ in (11.31) yields

$$\hat{\lambda}_t^{\mu, \text{impl}} = -\hat{\lambda}_t \Theta'(\mu_t) k_t^\alpha - \hat{\nu}_t^{\text{AT}} \sigma_t A_t L_t k_t^\alpha. \quad (11.37)$$

Plugged into the Fischer–Burmeister condition (11.34), this is the residual l_8 below. Two facts come for free. First, whenever $l_8 = 0$ holds and the smoothing parameter ε_{FB} is small, the abatement FOC *also* holds automatically, because l_8 couples the implied multiplier to the slack $1 - \mu_t$. Second, the network output dimension drops from nine to eight, which improves training stability: the network no longer has to discover that $\hat{\lambda}_t^\mu$ is exactly the right algebraic combination of $\hat{\lambda}_t$, μ_t , $\hat{\nu}_t^{\text{AT}}$.

The network architecture uses two hidden layers with 1024 units each, SELU activation, and the Adam optimizer with learning rate 10^{-5} . Training alternates between broad sampling (Phase 1) and endogenous simulation (Phase 2), as described in Chapter 3.

The 8 loss components. Each remaining equilibrium condition from §11.6 becomes a residual $l_m = 0$, and the network is asked to drive every l_m to zero simultaneously along simulated paths. The mapping is one-for-one: l_1 is the capital-Euler FOC (11.30); l_2 is the budget constraint that closes (11.3); l_3 , l_4 , l_5 are the three carbon-reservoir envelope conditions, of which l_3 is (11.32); l_6 and l_7 are the two temperature-layer envelopes; and l_8 is the Fischer–Burmeister smoothing (11.34) of the KKT slack on $\mu_t \leq 1$, evaluated at the implied multiplier (11.37). The consumption FOC (11.29) and the abatement FOC (11.31) are enforced *exactly* via the inversions in (11.36) and (11.37), which is why the loss list contains eight entries instead of nine. Written out, the eight components are:

$$l_1 := \exp(g_t^A + g_t^L) \hat{\lambda}_t - \hat{\beta}_t \left\{ \hat{\lambda}_{t+1} [(1 - \Omega(T_{\text{AT}, t+1}) - \Theta(\mu_{t+1})) \alpha k_{t+1}^{\alpha-1} + (1 - \delta)] + \hat{\nu}_{t+1}^{\text{AT}} \sigma_{t+1} (1 - \mu_{t+1}) A_{t+1} L_{t+1} \alpha k_{t+1}^{\alpha-1} \right\} \quad (\text{capital Euler})$$

$$l_2 := (1 - \Omega(T_{\text{AT}, t}) - \Theta(\mu_t)) k_t^\alpha + (1 - \delta) k_t - c_t - \exp(g_t^A + g_t^L) k_{t+1} \quad (\text{budget})$$

$$l_3 := \hat{\nu}_t^{\text{AT}} - \hat{\beta}_t \left[\hat{\nu}_{t+1}^{\text{AT}} (1 - b_{12}) + \hat{\nu}_{t+1}^{\text{UO}} b_{12} + \hat{\eta}_{t+1}^{\text{AT}} c_1 F_{2 \times \text{CO}_2} \frac{1}{\ln 2 M_{\text{AT}, t+1}} \right] \quad (\text{atm. carbon})$$

$$l_4 := \hat{\nu}_t^{\text{UO}} - \hat{\beta}_t \left[\hat{\nu}_{t+1}^{\text{AT}} b_{12} \frac{M_{\text{EQ}}^{\text{AT}}}{M_{\text{UO}}^{\text{UO}}} + \hat{\nu}_{t+1}^{\text{UO}} \left(1 - b_{12} \frac{M_{\text{EQ}}^{\text{AT}}}{M_{\text{UO}}^{\text{UO}}} - b_{23} \right) + \hat{\nu}_{t+1}^{\text{LO}} b_{23} \right] \quad (\text{upper ocean C})$$

$$l_5 := \hat{\nu}_t^{\text{LO}} - \hat{\beta}_t \left[\hat{\nu}_{t+1}^{\text{UO}} b_{23} \frac{M_{\text{EQ}}^{\text{UO}}}{M_{\text{LO}}^{\text{LO}}} + \hat{\nu}_{t+1}^{\text{LO}} \left(1 - b_{23} \frac{M_{\text{EQ}}^{\text{UO}}}{M_{\text{LO}}^{\text{LO}}} \right) \right] \quad (\text{lower ocean C})$$

$$l_6 := \hat{\eta}_t^{\text{AT}} - \hat{\beta}_t \left[-\hat{\lambda}_{t+1} \Omega'(T_{\text{AT}, t+1}) k_{t+1}^\alpha + \hat{\eta}_{t+1}^{\text{AT}} \left(1 - c_1 \frac{F_{2 \times \text{CO}_2}}{\Delta T_{\text{AT}, \times 2}} - c_1 c_3 \right) + \hat{\eta}_{t+1}^{\text{OC}} c_4 \right] \quad (\text{atm. temp.})$$

$$l_7 := \hat{\eta}_t^{\text{OC}} - \hat{\beta}_t \left[\hat{\eta}_{t+1}^{\text{AT}} c_1 c_3 + \hat{\eta}_{t+1}^{\text{OC}} (1 - c_4) \right] \quad (\text{ocean temp.})$$

$$l_8 := \hat{\lambda}_t^{\mu, \text{impl}} + (1 - \mu_t) - \sqrt{(\hat{\lambda}_t^{\mu, \text{impl}})^2 + (1 - \mu_t)^2 + \varepsilon_{\text{FB}}} \quad (\text{Fischer–Burmeister, implied multiplier})$$

Loss components l_1 – l_2 enforce intertemporal optimality and feasibility, l_3 – l_7 are the envelope conditions that price the five climate state variables, and l_8 jointly enforces the abatement FOC (via the implied multiplier) and the upper-bound complementarity $\mu_t \leq 1$.

Total loss. The DEQN loss aggregates all residuals along a simulated path:

$$\ell_\rho := \frac{1}{N_{\text{path}}} \sum_{\mathbf{x}_t \text{ on sim. path}} \sum_{m=1}^8 (l_m(\mathbf{x}_t, \mathcal{N}_\rho(\mathbf{x}_t)))^2. \quad (11.38)$$

This is the same sum-of-squared-residuals structure as the N -country IRBC model of Chapter 3, but with 8 equations per time step instead of the IRBC's $2N + 1$ (N Euler equations, N Fischer–Burmeister conditions, and one aggregate resource constraint).

State evolution. To evaluate the loss along a simulated path, the full state vector (11.24) must be propagated forward. In CDICE the next-period state is:

$$\mathbf{x}_{t+1} = (k_{t+1}, M_{t+1}^{\text{AT}}, M_{t+1}^{\text{UO}}, M_{t+1}^{\text{LO}}, T_{t+1}^{\text{AT}}, T_{t+1}^{\text{OC}}, \mu_{f,t+1}, S_{f,t+1}, \tau_{t+1}; \theta)^T, \quad (11.39)$$

where:

- k_{t+1} comes from the network output (11.35) (choice variable);
- $M_{t+1}^{\text{AT}}, M_{t+1}^{\text{UO}}, M_{t+1}^{\text{LO}}, T_{t+1}^{\text{AT}}, T_{t+1}^{\text{OC}}$ are computed from the transition equations of Section 11.2 (carbon cycle and temperature dynamics);
- the Bayesian belief states $\mu_{f,t+1}$ and $S_{f,t+1}$ are updated via the conjugate posterior (11.47)–(11.48) once the period- t temperature observation is realized;
- the bounded time index advances as $\tau_{t+1} = 1 - \exp(-\vartheta(t + \Delta_t))$, the image of the calendar increment $t \mapsto t + \Delta_t$ under the time-rescaling (11.23);
- the pseudo-state parameters θ are held fixed within an episode and re-sampled across episodes (Section 11.11).

All deterministic transitions are differentiable; stochastic shock draws are handled via reparameterization / common random numbers, so the simulate-then-backpropagate loop can be executed end-to-end with automatic differentiation.

Recipe: mapping a non-stationary model onto the DEQN framework

1. Detrend variables that grow with $A_t L_t$ (Eq. 11.22).
2. Map unbounded time to $[0, 1)$ via $\tau = 1 - e^{-\vartheta t}$ (Eq. 11.23).
3. Normalize all Lagrange multipliers by $A_t^{1-1/\psi} L_t$ (Eq. 11.27).
4. Derive FOCs from the Lagrangian, both economic *and* climate constraints (Eqs. 11.29–11.32).
5. Enforce static FOCs by inversion: invert the consumption FOC for c_t (Eq. 11.36) and solve the abatement FOC for the implied multiplier $\hat{\lambda}_t^{\mu, \text{impl}}$ (Eq. 11.37).
6. Smooth the upper-bound KKT complementarity via Fischer–Burmeister, evaluated at the implied multiplier (Eq. 11.34).
7. Form the loss as the sum of squared residuals over the 8 remaining conditions (Eq. 11.38).
8. Train as in the stationary DEQN: simulate $\rightarrow$ record loss $\rightarrow$ backprop $\rightarrow$ repeat.

This is the deterministic CDICE-DEQN solver in its entirety. Companion notebook `02_DICE_DEQN_Library_Port.ipynb` trains it against the CDICE library reference solution of Folini et al. (2025); the verification gate inside that notebook is the natural stopping point for a reader who wants only the deterministic core.

11.8 From CDICE to Stochastic IAMs

The deterministic CDICE-DEQN of §11.7, together with the AR(1) productivity extension developed in the remarkbox below, is the right pedagogical anchor because it contains every mechanical component of an integrated assessment model: capital accumulation, emissions, carbon diffusion, temperature dynamics, damages, abatement costs, and the SCC as a shadow price. It is not yet the object one wants for quantitative climate-policy research. Three features are still missing.

First, climate policy is an intrinsically stochastic problem. Productivity, carbon intensity, damages, climate feedbacks, and tipping thresholds are not known constants. Once they are stochastic, a carbon tax is not a path but a state-contingent policy. Second, long-run climate risk makes time-additive CRRA preferences too restrictive: the intertemporal elasticity of substitution and risk aversion should be separate parameters. Third, climate policy is distributional. The representative-agent SCC answers a marginal pricing question, but an implementable policy also asks which cohorts pay the tax and which cohorts receive the transfers. This is the point at which the chapter moves from representative-agent DICE to stochastic overlapping-generations IAMs.

The transition is smooth if one keeps the computational object fixed. In every case the neural network approximates a policy map

$$u_t = \mathcal{N}_\rho(\tilde{\mathbf{x}}_t),$$

$$\tilde{\mathbf{x}}_t = (\text{economic states, climate states, beliefs, parameters, policy-rule coefficients}), \quad (11.40)$$

and the loss is still a sum of normalized equilibrium residuals. The only changes are the variables appended to $\tilde{\mathbf{x}}_t$ and the conditional expectations appearing in the residuals. Table 11.4 summarizes the sequence.

Table 11.4: The layers of the climate-economy pipeline used in the remainder of the chapter. Each layer is a small extension of the previous one; no new numerical paradigm is introduced after the deterministic CDICE-DEQN.

Layer	Economic question	Computational change
Deterministic CDICE (§11.7)	What is the globally optimal abatement path and SCC at the baseline calibration?	Time-stamped DEQN; eight residuals; horizon $T_{\max}$ chosen so discounting absorbs transversality.
Stochastic DICE (AR(1) + GH quadrature, see the productivity-shock remarkbox below)	How do shocks alter the SCC distribution?	Add shock states; replace future terms by Gauss–Hermite expectations.
Bayesian learning on ECS (§11.9)	How does learning about climate sensitivity alter the SCC distribution?	Add belief mean and belief variance as states; one signal equation; conjugate Gaussian update.
Epstein–Zin DICE (§11.10)	How do risk aversion and IES separately price climate tails?	Add the value level as a network output; add one recursion residual and an EZ continuation-value weight.
Deep UQ (§11.11)	Which uncertain parameters drive SCC variation?	Treat parameters as pseudo-states; fit a GP surrogate for the QoI; compute Sobol, Shapley, and univariate effects.
Stochastic OLG-IAM (§11.12)	Can carbon taxes be welfare improving and Pareto improving across cohorts?	Treat tax coefficients and transfer shares as pseudo-states; fit GP surrogates for cohort welfare; solve constrained policy design on the surrogate.

Adding aggregate shocks: AR(1) productivity and Gauss–Hermite quadrature

Real climate–economy interactions are shot through with stochastic shocks. The minimal stochastic extension that already lets us reproduce the qualitative SCC fan-chart structure of [Cai and Lontzek \(2019\)](#) on a laptop adds an AR(1) shock to log TFP:

$$z_{t+1} = \rho_z z_t + \sigma_z \varepsilon_{t+1}, \quad \varepsilon_{t+1} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1), \quad (11.41)$$

with effective TFP $A_t \exp(z_t)$. The state vector (11.24) acquires a new entry,

$$\tilde{\mathbf{x}}_t = (k_t, M_t^{\text{AT}}, M_t^{\text{UO}}, M_t^{\text{LO}}, T_t^{\text{AT}}, T_t^{\text{OC}}, \tau_t, z_t)^\top, \quad (11.42)$$

and each forward-looking residual ((capital Euler))–((ocean temp.)) acquires a conditional expectation over ε_{t+1} ; the capital Euler, for example, becomes

$$\begin{aligned} e^{g_t^A + g_t^L} \hat{\lambda}_t &= \hat{\beta}_t \mathbb{E}_t \left[\hat{\lambda}_{t+1} ((1 - \Omega(T_{t+1}^{\text{AT}}) - \Theta(\mu_{t+1})) \alpha k_{t+1}^{\alpha-1} + (1 - \delta)) \right. \\ &\quad \left. + \hat{\nu}_{t+1}^{\text{AT}} \sigma_{t+1} (1 - \mu_{t+1}) A_{t+1} L_{t+1} \alpha k_{t+1}^{\alpha-1} \right]. \end{aligned} \quad (11.43)$$

With ε_{t+1} Gaussian, the conditional expectation is evaluated with a small number of Gauss–Hermite nodes $\{(\xi_q, w_q)\}_{q=1}^Q$,

$$\mathbb{E}_t[f(\varepsilon_{t+1})] \approx \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q f(\sqrt{2} \xi_q), \quad (11.44)$$

and each residual is replaced by its stochastic counterpart

$$l_m^{\text{stoch}}(\tilde{\mathbf{x}}_t, \mathcal{N}_\rho) = \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q l_m(\tilde{\mathbf{x}}_t, \mathcal{N}_\rho; \varepsilon_{t+1} = \sqrt{2} \xi_q), \quad (11.45)$$

which the total loss (11.38) then aggregates as before. In practice $Q = 5$ nodes drive the quadrature error well below the training-noise floor; the GH evaluation is fully differentiable, so the autodiff backward pass is unchanged. When several independent shock dimensions appear simultaneously (productivity shock ε_{t+1} , learning innovation $\tilde{\varepsilon}_{T,t+1}$, and the EZ certainty-equivalent integrand of §11.10), each conditional expectation is taken under a tensor product of one-dimensional GH rules at Q nodes per dimension, i.e. Q^d total nodes for d shock dimensions; the autodiff backward pass traverses the quadrature unchanged. Forward-simulating N_{MC} trajectories of the AR(1) shock produces a Monte-Carlo SCC fan chart whose right-tail mass is the channel of [Cai and Lontzek](#)’s headline result. Companion notebook `03_Stochastic_DICE_DEQN.ipynb` trains this stochastic extension end-to-end and is the natural anchor for Exercise 7.

The unifying trick

The same pseudo-state idea appears three times: uncertain structural parameters are pseudo-states for UQ, policy-rule coefficients are pseudo-states for constrained optimal policy, and Bayesian posterior beliefs are endogenous pseudo-states for learning. The network does not care which interpretation is attached to an input coordinate; it only learns a differentiable equilibrium map on the enlarged domain. The methodological cost of each layer is small: at most one additional state or output and one additional term in the loss; the algorithm of §11.5 is unchanged.

11.9 Bayesian Learning About Climate Sensitivity

Why ECS is the natural learning state. The equilibrium climate sensitivity (ECS), defined as the long-run atmospheric warming from a doubling of CO_2 , is the single most consequential and most uncertain parameter in the climate side of an IAM. Observational, paleoclimate, and model-based estimates place ECS in a *likely* (66%) range of roughly $2.5\text{--}4^\circ\text{C}$ and a *very-likely* (90%) range of $2\text{--}5^\circ\text{C}$ (Sherwood et al., 2020; Knutti et al., 2017; Roe and Baker, 2007), and ECS uncertainty is the largest single contributor to SCC dispersion across model variants. Crucially, ECS is partially identified from temperature realizations conditional on emissions and forcing: a Bayesian planner who observes temperature paths can therefore update her posterior period by period, and the policy that maximizes ex-ante welfare conditions on the current posterior rather than on a fixed point estimate.

How learning enters the state. Promote the climate-feedback parameter λ in (11.15) to a stochastic object by adding the feedback term $\varphi_{1C} \tilde{f}_{t+1} T_t^{\text{AT}}$ to the right-hand side, where φ_{1C} is a calibrated coupling coefficient (taken from Friedl et al. (2023)) and $\tilde{f}_{t+1} \sim \mathcal{N}(\mu_{f,t}, S_{f,t})$ is a per-period draw under the planner’s posterior over the unknown climate-feedback deviation. The unknown itself is time-invariant; the subscript $t+1$ indexes the period in which the subjective draw enters the temperature equation, and the planner’s posterior moments $(\mu_{f,t}, S_{f,t})$ shift over time as new temperature observations arrive. The planner observes the temperature-residual signal

$$y_{t+1} := \varphi_{1C} T_t^{\text{AT}} \tilde{f}_{t+1} + \tilde{\epsilon}_{T,t+1}, \quad \tilde{\epsilon}_{T,t+1} \sim \mathcal{N}(0, S_{\epsilon_T}), \quad (11.46)$$

and conjugate Gaussian–Gaussian updating delivers the posterior

$$\mu_{f,t+1} = \frac{S_{\epsilon_T} \mu_{f,t} + \varphi_{1C} T_t^{\text{AT}} S_{f,t} y_{t+1}}{S_{\epsilon_T} + (\varphi_{1C} T_t^{\text{AT}})^2 S_{f,t}}, \quad (11.47)$$

$$S_{f,t+1} = \frac{S_{\epsilon_T} \cdot S_{f,t}}{S_{\epsilon_T} + (\varphi_{1C} T_t^{\text{AT}})^2 S_{f,t}}, \quad (11.48)$$

which the planner takes as two additional laws of motion for the belief states $(\mu_{f,t}, S_{f,t})$. These two states occupy the slots already reserved in the augmented state vector (11.24). Equations (11.47)–(11.48) are the Kalman update for a scalar linear-Gaussian state-space model with observation gain $\varphi_{1C} T_t^{\text{AT}}$ and noise variance S_{ϵ_T} ; cf. Bishop (2006, § 13.3) for the generic derivation. The DEQN algorithm of §11.5 is unchanged: the network simply receives two more inputs and learns a richer policy.

Where this sits in the literature. Bayesian learning about climate parameters in an integrated assessment frame has a long pedigree. Kelly and Kolstad (1999) and Kelly and Tan (2015) establish the basic Kelly–Kolstad result that learning takes decades to centuries in calibrated DICE-like settings, and that the tradeoff between mitigation (which lowers temperature variance) and information (which requires informative temperature paths) is sharp. Leach (2007) and Webster et al. (2008) sharpen the slow-learning result and quantify the policy errors induced by treating uncertainty as resolved too quickly. On the dynamic-programming side, Cai and Lontzek (2019) solve a stochastic-DICE variant with tipping-point hazards and recursive preferences. The robust-control program of Anderson et al. (2014), Barnett (2023), Barnett et al. (2023), and Barnett et al. (2020b) addresses a complementary question (planner ambiguity over the data-generating process), and modern deep-learning solutions are the natural computational companion because tensor-product grids over belief states are infeasible at realistic state-vector sizes.

Headline result from the UQ literature. Friedl et al. (2023) solve the joint stochastic-DICE–Bayesian-learning DEQN with the methodology of this chapter and find two qualitative features that survive across the calibration cloud.¹ First, ECS uncertainty is largely resolved within roughly ten years of optimal policy: the posterior variance $S_{f,t}$ shrinks by an order of magnitude over the first decade of the planner’s horizon, even though the absolute posterior mean takes longer to settle. Second, the SCC under learning is roughly half the no-learning SCC for moderate true ECS values, and roughly the same as the no-learning SCC at the upper tail of the ECS distribution; learning is a strong substitute for precautionary mitigation in the moderate-ECS regime, and a weak substitute in the tail-ECS regime. The asymmetry is policy-relevant: the value of waiting to learn falls sharply once the planner suspects she is in the tail. The broader teaching point is that uncertainty is not automatically a reason to abate more: its policy effect depends on whether the uncertainty is static, learnable, or associated with irreversible tail risk. Figure 11.5 illustrates the two qualitative features.

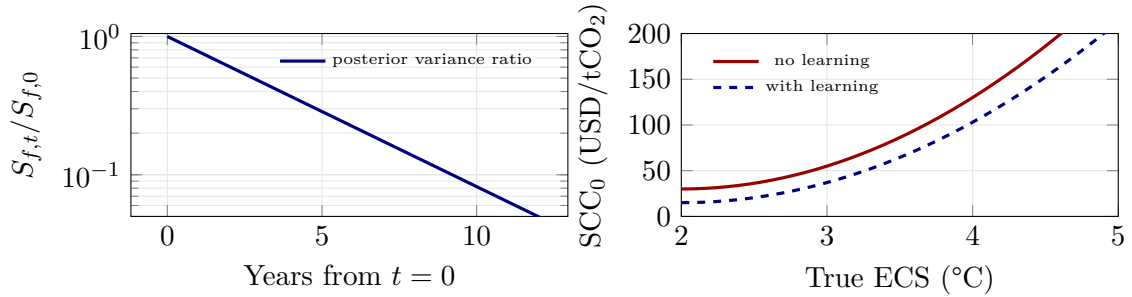


Figure 11.5: Schematic of the two qualitative features reported by Friedl et al. (2023). Left: posterior variance $S_{f,t}$ relative to its prior value, on a logarithmic scale. The variance falls by roughly an order of magnitude over the first decade, mirroring the Kelly–Kolstad slow-learning result but accelerated by the deeper signal–noise ratio of the modern climate calibration. Right: SCC_0 as a function of the true ECS, with and without Bayesian learning. Learning approximately halves the SCC at moderate ECS values where uncertainty is the dominant driver of precautionary abatement, but converges to the no-learning curve at the upper tail where the underlying physical damage dominates. Curves are illustrative; the magnitudes are those quoted in the body text.

11.10 Epstein–Zin Preferences

Why recursive preferences for climate. The time-additive CRRA-IES aggregator (11.4) ties risk aversion and intertemporal substitution together. Climate policy is exactly the environment in which this restriction is least attractive. A planner may want a high IES ψ to govern intertemporal substitution across long horizons, and a separate high coefficient of relative risk aversion γ_u to price low-probability climate disasters. Recursive Kreps–Porteus preferences, following Epstein and Zin (1989) and Weil (1989), implement this separation.²

Working with the normalized per-capita value v_t and per-capita consumption $c_t = C_t/L_t$, and writing $\beta_t^{\text{EZ}} := \exp(-\rho \Delta_t)$ for the one-period Epstein–Zin discount factor, the recursion is

$$v_t = \left[(1 - \beta_t^{\text{EZ}}) c_t^{1-1/\psi} + \beta_t^{\text{EZ}} \left(\mathbb{E}_t \left[v_{t+1}^{1-\gamma_u} \right] \right)^{\frac{1-1/\psi}{1-\gamma_u}} \right]^{\frac{1}{1-1/\psi}}, \quad (11.49)$$

¹The numerical claims in this paragraph quote the headline results of Friedl et al. (2023); consult that paper for the precise figures and the underlying calibration grid.

²**Notation note.** We use ψ for the IES and γ_u for risk aversion throughout this section, following Friedl et al. (2023). The IRBC chapter (Chapter 3) used γ for the IES under the bundled CRRA-IES convention; here in the Epstein–Zin block we deliberately decouple the two parameters, so the symbol switch is intentional. The CRRA limit is recovered at $\gamma_u = 1/\psi$.

with the usual logarithmic limits when $\psi = 1$ or $\gamma_u = 1$, subject to the same budget constraint, capital-accumulation law, and climate dynamics as before.

What changes in the DEQN loss. The value level v_t becomes an additional network output, paired with a ninth residual that enforces the recursion (11.49):

$$\mathcal{R}_t^{\text{EZ}} = v_t - \left[(1 - \beta_t^{\text{EZ}}) c_t^{1-1/\psi} + \beta_t^{\text{EZ}} \left(\mathbb{E}_t \left[v_{t+1}^{1-\gamma_u} \right] \right)^{\frac{1-1/\psi}{1-\gamma_u}} \right]^{\frac{1}{1-1/\psi}}. \quad (11.50)$$

In the deterministic CRRA-IES core of §11.7, v_t never appears explicitly, which is why eight residuals suffice there. The Euler and costate residuals of §11.6 keep their deterministic form but receive a Bansal–Yaron certainty-equivalent weighting inside each conditional expectation. It is convenient to write the one-step recursive-pricing kernel as

$$\mathcal{M}_{t,t+1}^{\text{EZ}} = \hat{\beta}_t \left(\frac{v_{t+1}}{\left(\mathbb{E}_t[v_{t+1}^{1-\gamma_u}] \right)^{1/(1-\gamma_u)}} \right)^{1/\psi - \gamma_u} \left(\frac{c_{t+1}}{c_t} \right)^{-1/\psi}, \quad (11.51)$$

where $\hat{\beta}_t$ inherits the deterministic growth normalization of (11.28). In the code, (11.51) is just a multiplicative weight on next-period marginal-value terms; the certainty-equivalent operator inside the Kreps–Porteus aggregator becomes a second nested expectation. The DEQN loss inherits one extra Gauss–Hermite quadrature step and one extra network output, but no new algorithmic ingredient.

Interpretation for the SCC. Crost and Traeger (2013) and Crost and Traeger (2014) establish the analytic baseline: in a deterministic IAM, decoupling risk aversion from the IES changes the optimal carbon tax only when stochastic risk is present, but the change can be quantitatively large once it is. Jensen and Traeger (2014) and Traeger (2023, 2021) extend the result to closed-form ACE-class settings and show that for reasonable risk aversion above $1/\psi$, the SCC roughly doubles relative to CRRA; Cai and Lontzek (2019) reach the same conclusion in a fully stochastic DICE variant. Intuitively, recursive preferences change the SCC because carbon emissions affect the distribution of long-run consumption, not only its mean: if damages create low-consumption tail states, a high γ_u raises the SCC through the disaster-insurance channel. The sign of the IES effect depends on which shock dominates: in TFP-driven economies higher ψ dampens the SCC because consumption smoothing absorbs the productivity risk, whereas in temperature-driven economies higher ψ amplifies the SCC because the planner cares more about late-horizon consumption losses. Bansal et al. (2016) make the asset-pricing case for the same channel: long-run temperature shifts price into expected returns through the EZ aggregator, and ignoring them understates the welfare cost of carbon emissions. This is why stochastic DICE with Epstein–Zin preferences is a better teaching object than deterministic DICE for climate-finance questions: it connects welfare, tail risk, and asset-pricing logic in a single equilibrium loss.

11.11 Deep Uncertainty Quantification via Surrogates

Deep UQ answers a different question from solving one stochastic IAM. The object is now a scalar quantity of interest,

$$q(\theta) = \text{SCC}_{2100}(\theta), \quad \theta \in \Theta \subset \mathbb{R}^{d_\theta}, \quad (11.52)$$

where θ collects uncertain structural parameters: the ECS or its prior mean $\mu_{f,0}$, the prior variance $S_{f,0}$, the pure rate of time preference ρ , the IES ψ , risk aversion γ_u , the damage curvature π_2 , and any tipping parameters included in the experiment. Direct global sensitivity analysis would require solving the IAM thousands of times. Deep UQ replaces this infeasible outer loop by two amortizations.

Amortization 1: parameters as pseudo-states. The pseudo-state trick of [Friedl et al. \(2023\)](#) collapses the outer loop into a single DEQN training pass. Uncertain parameters θ are appended to the network’s input,

$$\tilde{\mathbf{x}}_t = \left(\underbrace{\mathbf{x}_t}_{\text{economic + climate states}}, \underbrace{\theta}_{\text{uncertain parameters}} \right), \quad u_t = \mathcal{N}_\rho(\tilde{\mathbf{x}}_t), \quad (11.53)$$

held fixed within each simulation episode and re-sampled across episodes from a design distribution $\mathcal{D}_\theta$. One trained network therefore approximates the policy function for every θ in $\mathcal{D}_\theta$; evaluating any new θ requires only a forward pass. For very large pseudo-state dimensions the active-subspace methods of §9.5 compress θ before the next step. This is the same idea as the parameterized policy networks in Chapter 10; here the target is not an SMM criterion but an SCC distribution.

Amortization 2: a GP for the quantity of interest. After training, the DEQN is evaluated at a design set $\{\theta_i\}_{i=1}^n$ and the corresponding QoI values $q_i = q(\theta_i)$ are computed by forward simulation. Fit a Gaussian-process surrogate

$$q(\theta) = m(\theta) + \varepsilon(\theta), \quad m(\theta) \mid \{(\theta_i, q_i)\}_{i=1}^n \sim \mathcal{GP}(\mu_n(\theta), k_n(\theta, \theta')). \quad (11.54)$$

The GP is cheap enough to evaluate millions of times, so the expensive IAM is no longer called inside Sobol, Shapley, or univariate-effect estimators. Bayesian active learning improves the design by adding points where the GP posterior uncertainty is largest or where integrated posterior variance is most reduced, following the toolkit of Chapter 9 (see Figure 10.1 and Table 10.1).

Sobol, Shapley, univariate effects. Three complementary global sensitivity indices answer different questions about how θ drives the SCC. The first-order Sobol index S_i of [Sobol \(2001\)](#) measures the share of output variance explained by θ_i alone,

$$S_i = \frac{\text{Var}(\mathbb{E}[q(\theta) \mid \theta_i])}{\text{Var}(q(\theta))}, \quad (11.55)$$

and the total-effect index captures both direct and interaction effects,

$$S_i^{\text{tot}} = 1 - \frac{\text{Var}(\mathbb{E}[q(\theta) \mid \theta_{-i}])}{\text{Var}(q(\theta))}. \quad (11.56)$$

For independent inputs the $\{S_i\}$ sum to at most one, while the $\{S_i^{\text{tot}}\}$ can exceed one in the presence of interactions; equality $\sum_i S_i^{\text{tot}} = 1$ characterizes additive models. Shapley effects, introduced into sensitivity analysis by [Owen \(2014\)](#) and developed further by [Song et al. \(2016\)](#) and [Iooss and Prieur \(2019\)](#), allocate $\text{Var}(q)$ across parameters via cooperative-game averaging over all subsets of other parameters ([Shapley, 1953](#)), sum exactly to $\text{Var}(q)$ (raw) or one (normalized), and handle correlated inputs cleanly. Univariate-effect plots show the conditional mean $\mathbb{E}[q(\theta) \mid \theta_i]$ as θ_i varies and capture the directional response that Sobol indices average over. [Saltelli and D’Hombres \(2010\)](#) and [Saltelli et al. \(2008\)](#) give the standard estimators and best-practice warnings.

Deep UQ implementation recipe

1. Choose a parameter domain Θ and a sampling law $\mathcal{D}_\theta$ for the uncertain climate, damage, and preference parameters.
2. Train the stochastic CDICE-DEQN on $(\mathbf{x}_t, z_t, \mu_{f,t}, S_{f,t}, \theta)$, resampling θ across episodes and holding it fixed within an episode.
3. Generate n design evaluations $\{(\theta_i, q_i)\}_{i=1}^n$ from the trained network, where q_i is typically SCC_{2100} or an expected welfare functional.
4. Fit a GP surrogate $\theta \mapsto q(\theta)$, validate by leave-one-out cross-validation (target: LOO $R^2 \geq 0.95$ or LOO RMSE below 5% of the QoI standard deviation), and enrich the design with Bayesian active learning if the threshold is not met.
5. Compute Sobol, Shapley, and univariate effects on the GP surrogate, not on the structural model.

The reason this pipeline is the only feasible route is computational: direct Monte Carlo on Sobol or Shapley indices requires $O(10^4)$ to $O(10^6)$ evaluations of the structural model at fresh θ draws. Even at one DEQN solve per parameter vector, that price tag is several core-decades. The DEQN-with-pseudo-states amortizes one loop, and the GP surrogate amortizes the other; the sensitivity indices are then computed on the GP rather than on the IAM.

Empirical headline. Friedl et al. (2023) apply the pipeline to a stochastic DICE variant with Epstein–Zin preferences and Bayesian learning, and find that two ingredients dominate the SCC variance across 2020–2100: the mean of the ECS belief (roughly 50–70% of the total-effect Sobol share) and the curvature parameter of the damage function (roughly 15–25%).³ Together these account for 70–90% of the SCC variance. Risk aversion contributes a few percentage points; the pure rate of time preference and the IES contribute negligibly once damage curvature is conditioned on. The policy lesson is that under deep uncertainty the SCC should be reported as a distribution, not a point estimate, and that climate-policy design should target tail insurance against the upper ECS–damage corner rather than precision over the central calibration. Figure 11.6 sketches the resulting variance decomposition.

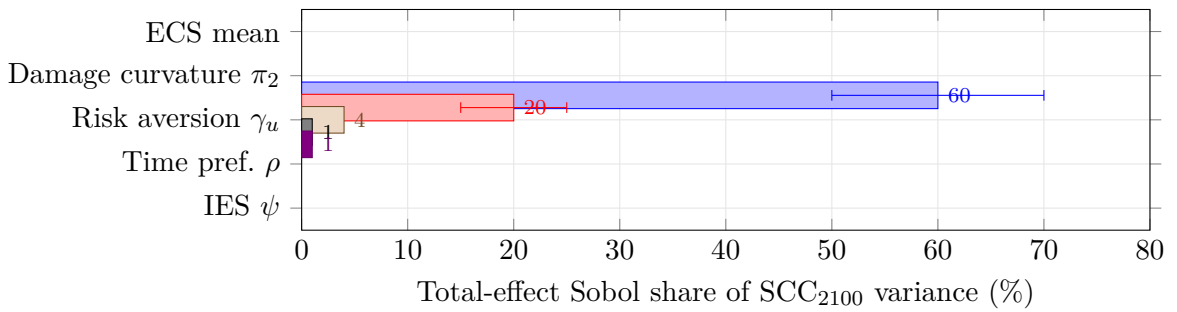


Figure 11.6: Schematic of the total-effect Sobol shares of SCC_{2100} variance reported by Friedl et al. (2023). Midpoints reflect the ranges quoted in the text (ECS mean 50–70%, damage curvature 15–25%), with horizontal error bars on the two leading parameters indicating the spread across horizon dates and damage-function specifications. The shape, two parameters carrying almost the entire variance, is what motivates the tail-insurance framing in the closing paragraph.

³Variance-share ranges quoted from Friedl et al. (2023); the spread reflects different points along the planner’s horizon and different damage-function specifications.

11.12 Constrained Pareto-Improving Carbon Tax in OLG-IAMs

The SCC analysis of §11.11 is still the marginal welfare cost of one extra ton of carbon *to a representative agent*. Climate policy, however, redistributes welfare across cohorts: today’s workers pay abatement costs while tomorrow’s households inherit a cooler planet. A *Pareto-improving* carbon tax must transfer enough revenue back to current cohorts that no generation is worse off than under business-as-usual. This section closes Movement 3 by walking through the constrained-Pareto OLG-IAM of Kübler et al. (2026), reusing the DEQN-with-pseudo-states machinery of §11.11 and the GP surrogate of Chapter 9. The Pareto-improvement criterion is closely related to the social-security reform literature (Krueger and Kubler, 2006), to recent work on intergenerational climate policy (Karp et al., 2024; Kotlikoff et al., 2021), and to the constrained-optimal-tax frontier of Douenne et al. (2024).

Notation reset for this section. The OLG-IAM uses different conventions than the representative-agent CDICE block of §11.2, following Kübler et al. (2026), and we summarize the differences here so the reader is not surprised. $\Omega_t(T_t)$ now denotes the *retained-output* factor, so net output is $\Omega_t \Phi K^\alpha L^{1-\alpha}$ rather than $(1 - \Omega - \Theta)Y^{\text{gross}}$. p_t^{tax} is the carbon tax (a per-tCO₂ price); to avoid clashing with the transformed-time variable τ_t of §11.3, this section uses p_t^{tax} for the tax throughout, in line with the price-level interpretation. e_t denotes the per-period emissions *flow* (in GtC), and $E_t = \sum_{s \leq t} e_s$ is *cumulative* emissions through date t ; this is the convention of the climate-emulator literature (Dietz and Venmans, 2019) and of the companion paper, and it is the source of the section’s frequent “cumulative-emissions tax” phrasing. Finally, the policy vector that the planner ultimately optimizes over is $\vartheta = (\vartheta_{\text{tax}}, \omega)$, the joint vector of tax-rule coefficients and cohort transfer shares defined in Step 1 below.

From CDICE to a TCRE emulator. The OLG-IAM uses a much simpler climate side than the 5-state CDICE module of §11.2 (three carbon stocks plus two temperature layers). Once the planner’s horizon is converted to cumulative-emissions form $E_t = \sum_{s \leq t} e_s$, the linear Transient Climate Response to cumulative carbon Emissions (TCRE) approximation collapses the carbon-cycle and energy-balance machinery to a single algebraic relation $T_t^{\text{AT}} \approx \sigma_{\text{CCR}} E_t$ (Dietz and Venmans, 2019), which removes five climate states from the planner’s optimization. The simplification is essential: it is what makes the OLG state space (12 cohort assets + 5 climate / shock states + ϑ pseudo-states) tractable end-to-end on a GPU. The reader who finds the change abrupt should treat the TCRE relation as a reduced-form summary of the same physics that drove §11.2.5–§11.2.6, fitted directly to long-run paths rather than block-by-block. Figure 11.7 contrasts the two climate sides.

11.12.1 The OLG-IAM Model

The model features $A = 12$ overlapping generations of selfish agents (ages 20–80 in 5-year periods), a competitive firm, and a simplified, cumulative-emissions climate module in the spirit of Dietz and Venmans (2019):

- **Technology:** Output is $Y_t = \Omega_t(T_t) \Phi(\mu_t) K_t^\alpha L_t^{1-\alpha}$ with retained-output damage factor Ω_t and net-of-abatement-cost factor $\Phi(\mu_t) = 1 - \theta_1 \mu_t^{\theta_2}$; emissions are $e_t = (1 - \mu_t) \kappa_t Y_t$ with stochastic carbon intensity κ_t ; the period resource constraint is $C_t + K_{t+1} = Y_t + (1 - \delta)K_t$.
- **Households:** Each agent maximizes $\mathbb{E}_t \sum_{j=1}^A \beta^{j-1} C_{t+j-1,j}^{1-\sigma_u} / (1 - \sigma_u)$ (where σ_u is the household CRRA risk-aversion coefficient, distinct from the climate-chapter notation σ_t for emissions intensity) subject to the budget constraint $C_{t,j} + a_{t+1,j+1} = (1+r_t) a_{t,j} + w_t l_j + \mathbb{T}_{t,j}$, where $\mathbb{T}_{t,j}$ is the transfer from carbon tax revenue and j runs over all $A = 12$ cohorts alive at t (newborns included).

CDICE (§11.2.5–§11.2.6): 5 states TCRE (OLG-IAM): 1 state

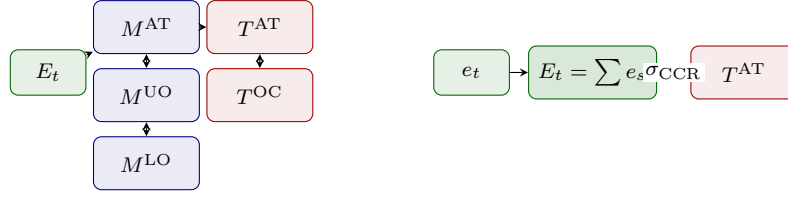


Figure 11.7: Climate side of CDICE versus TCRE. The 5-state CDICE module on the left, in which atmospheric carbon, two ocean carbon reservoirs, atmospheric temperature, and ocean temperature all enter the planner’s state, is collapsed in the OLG-IAM to a single algebraic relation between cumulative emissions and atmospheric temperature, $T_t^{\text{AT}} \approx \sigma_{\text{CCR}} E_t$. The simplification trades fidelity to short-run climate dynamics for tractability of the 12-cohort heterogeneous-agent state space and is what makes the bilevel policy search of §11.12 end-to-end feasible.

- **Climate:** The climate emulator imposes a near-linear relationship between cumulative emissions and atmospheric temperature, $T_t^{\text{AT}} \approx \sigma_{\text{CCR}} E_t$ where $E_t = \sum_{s \leq t} e_s$ is cumulative carbon, augmented by a stochastic tipping mechanism: damages depend on T_t^{AT} relative to a threshold TP_t via a Weitzman-type retained-output factor that becomes steeply convex once T_t^{AT} approaches TP_t .
- **Stochastic shocks:** Carbon intensity κ_t follows an AR(1) with time-varying persistence; the tipping threshold TP_t follows a bounded random walk that becomes absorbing once it has been crossed.

The household Euler equation takes the standard form $C_{t,j}^{-\sigma_u} = \beta \mathbb{E}_t[(1 + r_{t+1}) C_{t+1,j+1}^{-\sigma_u}]$ for $j = 1, \dots, A - 1$, and market clearing requires that aggregate savings equal the capital stock: $\sum_j a_{t,j} = K_t$. Figure 11.8 simulates this model without policy intervention; it fixes the business-as-usual (BAU) baseline against which every Pareto-improving policy below is benchmarked, and supplies the cohort-by-cohort participation constraints for the constrained policy search.

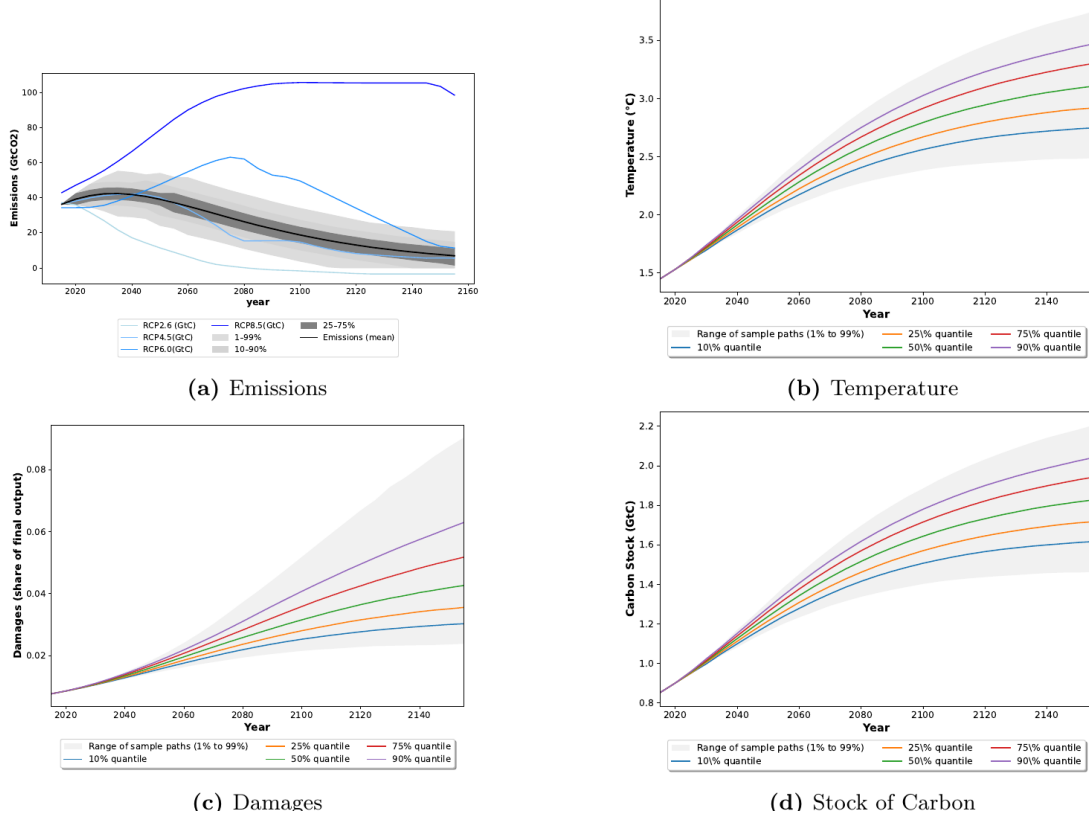


Figure 11.8: Business-as-usual baseline for the 12-cohort stochastic OLG-IAM of Kübler et al. (2026). Without policy intervention the median warming reaches roughly 3°C over the 150-year horizon, and the upper tail of damages is substantially larger than the mean. Every Pareto-improving policy below is benchmarked against this baseline, which also supplies the participation constraints for the constrained policy search. Figure extracted from Kübler et al. (2026).

11.12.2 The 3-Step ML Pipeline

Finding an optimal carbon tax rule in this OLG economy is a bilevel optimization problem: the outer level searches over tax parameters, and the inner level solves the full stochastic general equilibrium for each candidate tax. Kübler et al. (2026) decompose this into three steps, summarized in Figure 11.9:

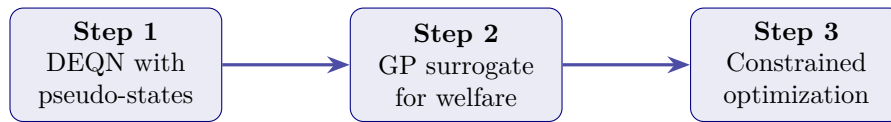


Figure 11.9: Three-step machine-learning pipeline for constrained carbon-tax design. The DEQN amortizes equilibrium solution across tax parameters, the GP surrogate maps policy parameters to welfare and cohort utilities, and the final optimization imposes the Pareto constraints on the surrogate.

Step 1: DEQN with pseudo-states. The tax-rule coefficients ϑ_{tax} and the $A = 12$ transfer shares $\omega = (\omega_1, \dots, \omega_{12})$ are appended to the state of the neural network as pseudo-states. The transfer shares are non-negative weights satisfying $\sum_{j=1}^A \omega_j = 1$, with cohort j 's lump-sum transfer given by $\mathbb{T}_{t,j} = \omega_j p_t^{\text{tax}} e_t$ from the government's resource constraint $\sum_j \mathbb{T}_{t,j} = p_t^{\text{tax}} e_t$. The simplex constraint $\omega \in \Delta^{A-1}$ is enforced by sampling unconstrained logits and applying a softmax before

feeding ω into the network, so the DEQN never sees an infeasible transfer profile. All cohorts alive at t , including the newborn cohort, receive a transfer. The number of tax parameters depends on the rule: a simple linear rule on cumulative emissions has $\vartheta_{\text{tax}} = (\vartheta_0, \vartheta_E) \in \mathbb{R}^2$ (so a 14-dimensional pseudo-state vector together with the 12 transfer shares), and a richer rule that adds dependence on carbon intensity and tipping has $\vartheta_{\text{tax}} \in \mathbb{R}^4$ (a 16-dimensional pseudo-state vector with the 12 shares). The DEQN learns the equilibrium for *all* candidate tax-and-transfer configurations at once, so that simulating any $(\vartheta_{\text{tax}}, \omega)$ requires only a forward pass. The network architecture, optimizer schedule, and training-pool design follow Kübler et al. (2026) verbatim; the exact configuration is documented in the companion repository linked at the end of this section.

Step 2: GP surrogate. At each design point $\vartheta = (\vartheta_{\text{tax}}, \omega)$, the trained DEQN is simulated to obtain Monte-Carlo estimates of expected lifetime utility for the 40 tracked cohorts (12 alive at $t = 0$ plus 28 future cohorts born during the planner’s 150-year horizon). Independent GPs are then fitted to map ϑ to expected aggregate welfare $\mathcal{W}(\vartheta)$ and to each of the 40 cohort welfares $\tilde{U}_t(\vartheta)$. The design itself uses Latin-hypercube sampling augmented with Bayesian active learning: the size scales with the dimension of ϑ , with roughly 500 points sufficient for the 14-dimensional “linear-in- E + transfers” specification (Section 5.3 of Kübler et al. (2026)) and roughly 800 points for the 16-dimensional “richer rule + transfers” specification (Section 5.4). Figure 11.10 shows the resulting welfare surface for the two-parameter linear-in-cumulative-emissions rule, with transfer shares held at the Pareto-optimal solution: the contour exposes the low-dimensional ridge along which intercept and slope trade off cleanly, and on which the Step-3 optimizer searches.

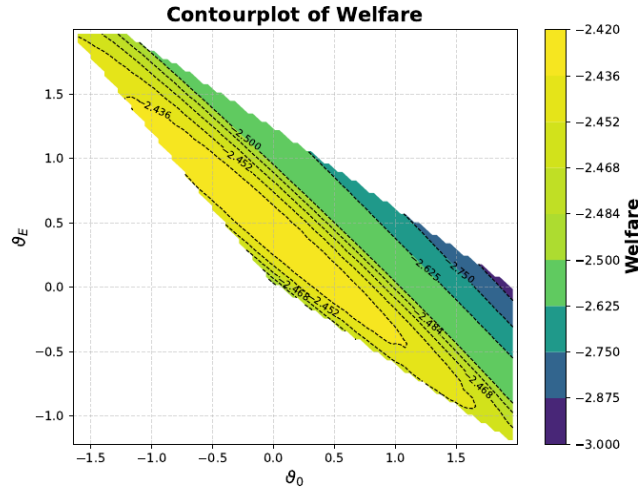


Figure 11.10: Gaussian-process welfare surrogate over the two-dimensional tax-parameter slice $(\vartheta_0, \vartheta_E)$ of the linear-in-cumulative-emissions rule, with transfer shares ω held at the Pareto-optimal solution. The contour exposes the low-dimensional welfare surface on which the constrained optimizer of Eq. (11.57) searches once the DEQN has amortized the equilibrium solve. Figure extracted from Kübler et al. (2026).

Step 3: Constrained optimization. The planner solves

$$\vartheta^* = \arg \max_{\vartheta = (\vartheta_{\text{tax}}, \omega)} \mathcal{W}(\vartheta) \quad \text{s.t.} \quad \tilde{U}_t(\vartheta) \geq U_t \quad \forall t, \quad \omega \in \Delta^{A-1}, \quad (11.57)$$

where U_t is the business-as-usual (BAU) welfare of cohort t and Δ^{A-1} is the standard simplex on $A = 12$ shares. The Pareto constraint ensures that *no generation is worse off*; whenever the welfare-maximizing ϑ^* lies strictly inside the feasible polytope (which is the case in every

scenario reported below) it is also strictly Pareto-improving for at least one cohort, so the weak constraint $\tilde{U}_t \geq U_t$ and the textbook strict-improvement requirement coincide at the optimum. Because each evaluation of $\mathcal{W}$ and $\tilde{U}_t$ is a forward pass through the trained GP rather than a fresh DEQN simulation, the constrained search reduces to a sequence of small SLSQP problems (the paper uses 500 random restarts of `scipy.optimize.minimize`) that complete in seconds. By contrast, replacing the surrogate with brute-force re-solves of the full SOLG IAM at every candidate ϑ would require on the order of tens of thousands of core-hours per candidate, which is the comparison the paper draws against traditional methods.

11.12.3 Results: Why Transfers Matter

The unconstrained welfare-maximizing cumulative-emissions tax is the natural benchmark. With a linear rule $p_t^{\text{tax}} = \vartheta_0 + \vartheta_E E_t$ and a fixed declining transfer scheme $\omega = \bar{\omega}$, the policy cuts emissions aggressively, stabilizes mean warming around 2.7 °C, and raises aggregate social welfare by about 1.6% in consumption-equivalent terms. But it imposes losses of up to roughly 5% on initial generations: it is therefore welfare-improving in the social-welfare-function sense, but *not* Pareto improving. Figure 11.11 shows the failure: the welfare-gains panel records the losses for transition generations that the social-welfare-function aggregate hides.

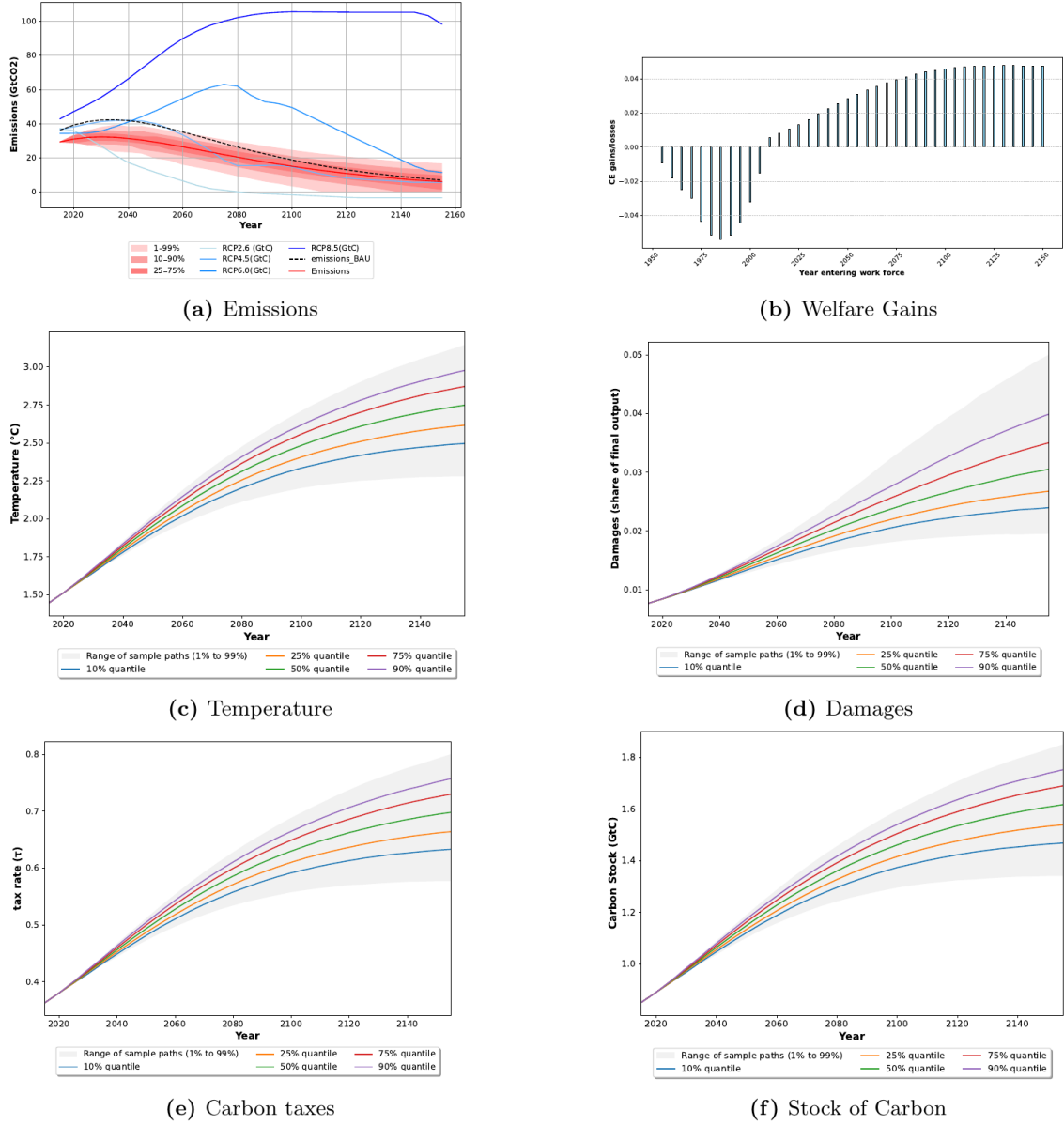


Figure 11.11: Welfare-improving but not Pareto-improving cumulative-emissions tax with a fixed exogenous transfer scheme. The policy strongly reduces climate risk and raises aggregate welfare by about 1.6% in consumption-equivalent terms, but the welfare-gains panel shows losses for transition generations. Figure extracted from Kübler et al. (2026).

Endogenizing the transfer shares changes the conclusion. With the same simple tax base and an optimized transfer simplex, Kübler et al. (2026) report the optimized coefficients⁴

$$p_t^{\text{tax}} = \vartheta_0 + \vartheta_E E_t, \quad (\vartheta_0, \vartheta_E) = (-0.186, 0.225), \quad (11.58)$$

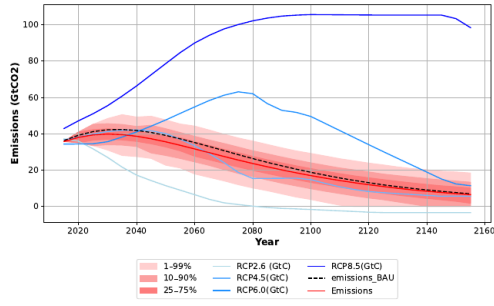
together with transfer shares

$$\omega = (0.128, 0.051, 0.058, 0.089, 0.149, 0.090, 0.066, 0.143, 0.076, 0.048, 0.039, 0.061), \quad (11.59)$$

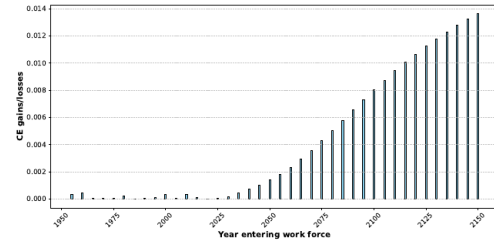
which sum to one up to rounding. Figure 11.13 plots this transfer profile against cohort index; the non-monotone shape is what allows a less aggressive cumulative-emissions tax to satisfy

⁴All numerical coefficients, welfare gains, and damage-quantile values in this subsection are quoted from Kübler et al. (2026); consult the paper for the source tables and figures.

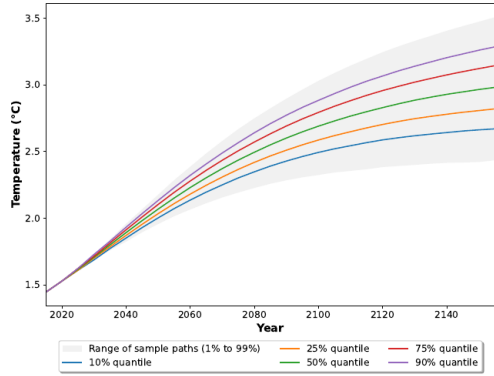
the Pareto constraint at every age, and it is the single most informative graphical summary of the constrained-optimal-policy step. The negative intercept $\vartheta_0 = -0.186$ is not a subsidy in practice: the planner's horizon starts well into the industrial era at a strictly positive cumulative-emissions stock $E_0 > 0$, so the effective tax $\vartheta_0 + \vartheta_E E_t$ is positive for every relevant E_t along the optimum. The negative intercept simply registers that the linear-in- E rule undershoots a constant carbon price near $E = 0$ and ramps up roughly proportionally to cumulative emissions thereafter. The combined policy makes every tracked cohort weakly better off than under BAU. The aggregate welfare gain is more modest than under the unconstrained optimum, at about 0.42% in consumption-equivalent terms, but the right tail of damages is truncated: the 99th percentile of damages falls to roughly 7% of output rather than about 9% under BAU. Figure 11.12 reports the full result. Comparing its welfare-gains panel with that of Figure 11.11 is the section's headline: a lower, simpler tax combined with an optimized transfer system shifts every cohort weakly into the gains region.



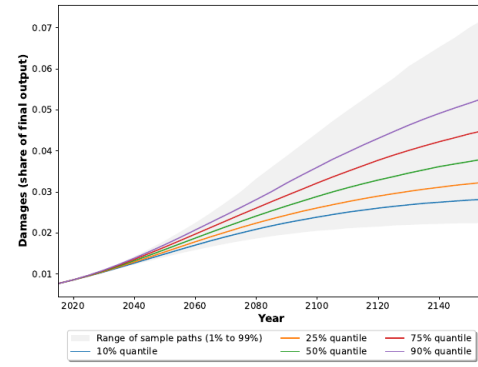
(a) Emissions



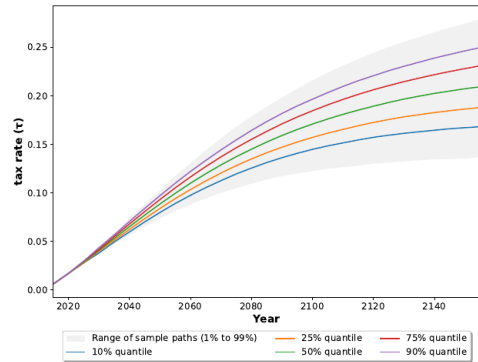
(b) Welfare Gains



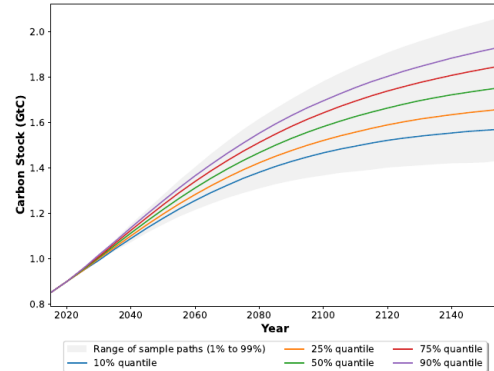
(c) Temperature



(d) Damages



(e) Tax rate



(f) Stock of Carbon

Figure 11.12: Pareto-improving cumulative-emissions tax with optimized intergenerational transfers, at the coefficients of (11.58)–(11.59). The tax is less aggressive than the unconstrained rule, but the optimized transfer system shields current cohorts while preserving climate-risk reduction for future cohorts. Aggregate welfare rises by about 0.42%. Figure extracted from Kübler et al. (2026).

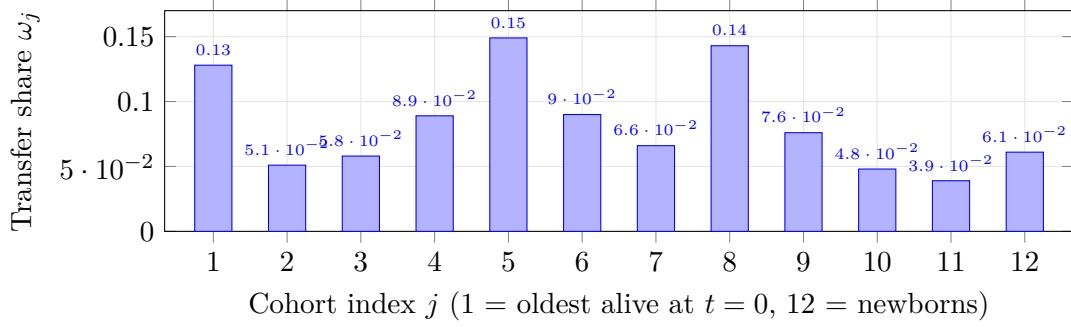


Figure 11.13: Optimized transfer-share profile ω_j across the 12 cohorts alive at $t = 0$, drawn directly from (11.59). The profile is decidedly non-monotone: the largest shares go to cohorts 1 (oldest), 5, and 8, which are precisely the cohorts the participation constraint $\tilde{U}_t \geq U_t$ binds most tightly for under the un-transferred tax of Figure 11.11. The non-monotone shape is what allows a less aggressive cumulative-emissions tax to satisfy Pareto improvement at every age.

The richer rule of §11.12 adds carbon intensity and a tipping-state statistic,

$$p_t^{\text{tax}} = \vartheta_0 + \vartheta_E E_t + \vartheta_\kappa \kappa_t + \vartheta_{TP}(1 - D_t), \quad (11.60)$$

where D_t is the climate-tipping state of the model (built from the proximity of T_t^{AT} to the stochastic threshold TP_t and the absorbed-tipping flag). Its optimized coefficients are

$$(\vartheta_0, \vartheta_E, \vartheta_\kappa, \vartheta_{TP}) = (-0.237, 0.203, 0.037, 0.012), \quad (11.61)$$

with the associated aggregate welfare gain rising only from about 0.42% to about 0.45%. The cohort-by-cohort welfare profile (not plotted; see Kübler et al. (2026) for the figure) again keeps every cohort weakly above its BAU baseline, and the marginal welfare improvement from the extra two policy-state coefficients is small. This is the substantive headline of Kübler et al. (2026): once intergenerational transfers are optimized, the simple cumulative-emissions tax captures most of the feasible Pareto-improving welfare gain. More policy-state variables improve the fit to climate risk, but the participation constraints bind tightly enough that the marginal welfare benefit of policy complexity is small. D_t is a deterministic function of variables already in the SOLG state, so it can be evaluated inside each forward pass; the exact functional form is in the paper.

Runtime in numbers. On a standard laptop (Apple M1), the OLG DEQN trains in roughly four wall-clock hours; on a high-end accelerator such as an NVIDIA GH200, training drops to the order of minutes (Kübler et al., 2026). Adding the GP fits over 500 (resp. 800) design points and the constrained Step-3 optimization keeps the entire pipeline within the same order of magnitude, while the comparable brute-force re-solve of the SOLG model at *every* candidate ϑ would dominate by orders of magnitude (the paper reports tens of thousands of core-hours for one fixed-parameter calibration, which would have to be repeated for every Step-3 candidate vector).

Companion code. The full production OLG-IAM solver, including the DEQN training loop with $(\vartheta_{\text{tax}}, \omega)$ pseudo-states and the bilevel policy search, is hosted in the companion repository [sischei/JPE_Macro_Using_ML_to_compute_constrained_optimal_carbon_tax_rules](#), which accompanies Kübler et al. (2026). The classroom notebook in Lecture 17 of this course exposes a reduced surrogate-only version that loads pre-trained GP surrogates and reproduces the constrained-optimization step (Step 3) interactively, but does not retrain the OLG DEQN end-to-end; readers who want the full pipeline should clone the companion repository.

The policy-design message

Deep learning is used here not because a neural network is fashionable, but because the relevant object is a high-dimensional map from states and policy-rule coefficients to equilibrium allocations and cohort welfare. Once that map is learned, constrained optimal policy becomes a small optimization problem on a surrogate. The economic result is equally important: carbon taxes need transfers. Without transfers, the welfare-maximizing tax creates transition losers; with optimized transfers, a simpler and lower tax can be Pareto improving. The main welfare channel is disaster-risk reduction (tail insurance), not maximal average abatement.

11.13 Discussion and Outlook

The combination of DEQNs, pseudo-states, and GP surrogates provides a scalable and transparent framework for climate economics that overcomes key limitations of traditional methods.

Comparison with traditional IAM solutions. Standard IAMs (such as the GAMS implementation of DICE) rely on shooting methods or nonlinear programming solvers that find deterministic optimal paths. These approaches struggle with stochastic extensions: Monte Carlo integration over shocks is expensive, and certainty equivalence (replacing random variables with their means) misses the welfare cost of tail risks. The DEQN approach approximates the stochastic recursive solution over the chosen training distribution and state/pseudo-state domain (with Bayesian learning and recursive Epstein–Zin utility) in a single training run.

Limitations. Several limitations should be noted. First, the CDICE climate module, while calibrated to CMIP benchmarks, remains a reduced-form emulator and cannot capture spatial heterogeneity or regional climate impacts. Second, the OLG-IAM treats each generation as identical within a cohort; within-cohort heterogeneity (e.g., geographic exposure to climate damages) would require further extensions along the lines of Chapter 6. Third, the linear tax rules are interpretable and implementable but may leave welfare gains on the table relative to fully nonlinear rules.

Extensions. Active research frontiers include: multi-region IAMs with trade and carbon leakage (Nordhaus and Yang, 1996); richer damage specifications including tipping cascades; endogenous technical change in abatement technology; and embedding climate modules in continuous-time heterogeneous-agent models (Chapter 8) to study the joint dynamics of climate risk and wealth inequality. The methodological toolkit developed in this course (DEQNs for equilibrium computation, PINNs for continuous-time PDEs, deep surrogates for uncertainty quantification, and Young’s method for distribution tracking) provides the computational infrastructure for these extensions.

The three movements, in one synthesis. Movement 1 established that solving an IAM by DEQN requires three modifications relative to the stationary toolkit of Chapter 2: time enters as a state, the training pool is built by simulating K forward trajectories from a calibrated initial state rather than by sampling an ergodic distribution, and the missing transversality is absorbed numerically by choosing the horizon $T_{\max}$ long enough that discounting suffices (or, on short horizons, by adding an explicit terminal residual). Movement 2 put that algorithm to work on a worked stochastic DICE economy, producing the eight-residual loss whose minimization delivers the deterministic policy and, with one extra Gauss–Hermite layer, the AR(1) SCC fan chart. Movement 3 layered four extensions onto the same spine: Bayesian learning over the

climate sensitivity, recursive Epstein–Zin preferences, global UQ of the SCC via pseudo-states and GP surrogates, and constrained Pareto-improving carbon-tax design in a heterogeneous-agent OLG-IAM. Chapter 12 threads these into the broader synthesis with the rest of the course.

Chapter Summary

- Climate-economy IAMs are the natural showcase for the methodological stack: a moderately high-dimensional non-stationary DSGE solved by DEQN, a GP+BAL surrogate for SCC sensitivity, and Sobol/Shapley decomposition for deep uncertainty quantification.
- DICE (and the CDICE recalibration) provide the workhorse model; Exercise 3 asks the reader to use the closed-form ACE expression of Traeger (2023) as an analytic benchmark for the DEQN-trained CDICE solution.
- Pareto-improving carbon-tax rules (Kübler et al., 2026) demonstrate that the surrogate machinery has direct policy relevance: linear, intergenerationally fair, implementable rules can be designed via constrained optimization on the surrogate.
- Deep UQ matters because the SCC distribution under climate, damage, and preference uncertainty is wider than any pointwise estimate suggests; reporting the distribution rather than a number is the responsible default.

Further Reading

- Nordhaus (2017), the canonical DICE update.
- Folini et al. (2025), the CDICE recalibration used in the deep-learning solution.
- Traeger (2023), the analytic ACE benchmark.
- Dietz (2024); Fernández-Villaverde et al. (2025); van der Ploeg and Rezai (2026), recent surveys on climate macroeconomics.
- Kübler et al. (2026), Pareto-improving carbon-tax design.
- Friedl et al. (2023), deep uncertainty quantification methodology.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Computational] ECS sensitivity.** Holding all other DICE parameters at calibration, vary the equilibrium climate sensitivity over the Sherwood et al. (2020) likely range (2.6–3.9°C) and report the implied SCC at $t = 0$. Then repeat over the broader IPCC AR6 very-likely range (2–5°C). How much of the SCC variation is driven by the high-ECS tail? State your classical baseline (e.g., a Nordhaus-2017 grid-search SCC at the central ECS calibration) before reaching for the DEQN.
2. **[Core] Sobol decomposition.** For a function $q(\theta_1, \theta_2, \theta_3) = \theta_1\theta_2 + \theta_3^2$ with i.i.d. uniform inputs $\theta_i \sim \mathcal{U}[0, 1]$, compute the Sobol first- and total-effect indices analytically. Verify with a 10 000-sample SALib estimate.
3. **[Computational] ACE benchmark.** Using the closed-form ACE expression for the optimal carbon tax, compute the SCC at calibration and compare with the DEQN-trained DICE solution. Quantify the discrepancy in % terms. State your classical baseline (the ACE closed form itself, evaluated at the chapter’s calibration) before reaching for the DEQN.

4. **[Advanced/project] Pareto-improving tax design.** Building on the OLG-IAM of Section 11.12.1, search over linear tax-and-transfer rules $\vartheta = (\vartheta_{\text{tax}}, \omega)$ with $\vartheta_{\text{tax}} = (\vartheta_0, \vartheta_E)$ on cumulative emissions and transfer shares $\omega \in \Delta^{A-1}$, $A = 12$, such that $\tilde{U}_t(\vartheta) \geq U_t$ for all cohorts. How tight is the Pareto frontier? Then repeat with ω held at the BAU/declining benchmark $\bar{\omega}$ and observe how much welfare is left on the table when transfers are not endogenous.
5. **[Computational] Carbon-cycle warm-up.** Run notebook `01_Climate_Exercise.ipynb` end-to-end. (a) Report the avoided warming and avoided damages at year 2100 under a 50% mitigation rule relative to BAU. (b) Inspect the carbon-cycle transition matrix and identify which reservoir (atmospheric, upper ocean, lower ocean) has the longest timescale. (c) Explain in two sentences why a quadratic damage-loss function $D(T) = \pi_2 T^2$ is insufficient for tail-risk assessment.
6. **[Advanced/project] Deterministic CDICE-DEQN reproduction.** Open notebook `02_DICE_DEQN_Library_Port.ipynb` and train the deterministic solver to convergence. Verify the following quantities against the reference solution, using the tolerances stated in the notebook's verification gate: $T^{\text{AT}}(2100)$, $M^{\text{AT}}(2100)$, $\mu(2100)$, and the SCC at 2015, 2100, and 2300.
7. **[Advanced/project] Stochastic SCC fan chart.** In notebook `03_Stochastic_DICE_DEQN.ipynb`, vary the AR(1) productivity volatility σ_z over $\{0.005, 0.01, 0.02\}$ and plot the resulting SCC fan charts. How does the right-tail of the SCC distribution at 2100 scale with σ_z ? Compare your finding with the qualitative message of Cai and Lontzek (2019). State your classical baseline (e.g., a deterministic-DICE SCC at the same calibration, or a certainty-equivalent SCC) before reaching for the DEQN.
8. **[Advanced/project] Tipping-point regime-switching damages.** This exercise temporarily switches from the additive damage-fraction convention $\Omega^N(T) = \pi_1 T + \pi_2 T^2$ used in the chapter body and in notebook 02 (Eq. 11.18) to the multiplicative *retained-output* convention $\Omega^{\text{ret}}(T) = 1/(1 + \pi_2 T^2)$, the form discussed in §11.2.7 as an alternative, so that the regime-switching modification below has a single multiplicative knob. First make this substitution in notebook `02_DICE_DEQN_Library_Port.ipynb` (which ships with the additive form Ω^N) and re-calibrate π_2 so that the retained-output form matches the additive baseline at $T = 2.5^\circ\text{C}$, i.e. choose π_2 such that $1 - 1/(1 + \pi_2 T^2) = \pi_2^N T^2$ at $T = 2.5^\circ\text{C}$ with $\pi_2^N = 0.00236$ from Table 11.2. Then replace the smooth retained-output factor $\Omega^{\text{ret}}(T) = 1/(1 + \pi_2 T^2)$ with a regime-switching specification. At each step, with hazard rate $\lambda_{\text{TP}}(T) = \lambda_0 + \lambda_1 \max(0, T - T_{\text{thresh}})$, an irreversible tipping event occurs. If the event has occurred, multiply the damage term in the denominator by $D_{\text{TP}} = 1.5$, so retained output becomes $\Omega^{\text{TP}}(T) = 1/(1 + D_{\text{TP}} \pi_2 T^2)$. Calibrate $\lambda_0 = 0.001$, $\lambda_1 = 0.05$, $T_{\text{thresh}} = 2.0^\circ\text{C}$. Retrain the DEQN solver and report (i) SCC at $t = 0$ under the regime-switching specification vs. the smooth baseline; (ii) the time path of optimal abatement μ_t in both cases; (iii) the unconditional probability of a tipping event by 2100. Sweep T_{thresh} over $\{1.5, 2.0, 2.5\}^\circ\text{C}$ and plot the SCC against the threshold. Discuss the policy implications: a lower threshold raises the SCC by what factor, and what does this imply for near-term tax design under deep uncertainty about T_{thresh} itself?
9. **[Advanced/project] Real options value of waiting.** Consider a stylized two-period climate-policy decision. At $t = 0$ the planner does not know the equilibrium climate sensitivity ECS, with prior $\text{ECS} \sim \mathcal{U}([\text{ECS}_L, \text{ECS}_H])$ where $\text{ECS}_L = 2$, $\text{ECS}_H = 5^\circ\text{C}$. Two choices: (a) **act now**, abate at level $\mu \in [0, 1]$ with cost $\Theta(\mu) = \theta \mu^2$; (b) **wait**, abate at $t = 1$ after observing a noisy signal $\text{ECS} = \text{ECS} + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$. Damages are $D(\mu, \text{ECS}) = \alpha \text{ECS} \cdot (1 - \mu)$. The planner minimizes expected total cost. (i) Derive closed-form expressions for the optimal μ^* and the expected total cost under each strategy. (ii) Define the *value of waiting* as the cost difference: $\text{VoW} = \mathbb{E}[\text{cost}_{\text{wait}}] - \mathbb{E}[\text{cost}_{\text{now}}]$. Show that as the

signal becomes informative ($\sigma_\varepsilon \rightarrow 0$), VoW becomes negative (waiting is preferred): more information allows better decisions. (iii) Now add an irreversibility wedge $\eta \mu_1^2$ paid only on the wait branch, penalizing deferred action (e.g., capital stock accumulates carbon faster while waiting, so the wait-period abatement is more costly than an equivalent abatement at $t = 0$). Show that for sufficiently large η , VoW is positive (act now is preferred), even with substantial learning. Connect this trade-off to the Bayesian-learning section of the chapter and to the broader literature on real options in climate policy.

10. **[Core] Non-stationary DEQN on a 1D toy.** Implement the three modifications of §11.5 on a one-dimensional non-stationary problem. The toy economy: a planner picks $u_t \in \mathbb{R}$ to minimize $\sum_{t=0}^{T_{\max}-1} [(x_t - x_t^*)^2 + r u_t^2] + \lambda_T (x_{T_{\max}} - x_{T_{\max}}^*)^2$ subject to the linear-Gaussian law of motion $x_{t+1} = \alpha x_t + u_t + g_t + \sigma \varepsilon_{t+1}$, $\varepsilon_{t+1} \sim \mathcal{N}(0, 1)$, with deterministic drift $g_t = g_0 + g_1 t$ and a calendar-time target path $x_t^* = a + b t$. Set $T_{\max} = 50$, $\alpha = 0.95$, $g_0 = 0.02$, $g_1 = 0.001$, $r = 0.1$, $\sigma = 0.05$, $\lambda_T = 5$, $a = 0$, $b = 0.04$. (i) Derive the closed-form LQ-Riccati policy and state your classical baseline before reaching for the DEQN. (ii) Train a small neural network $u_t = \mathcal{N}_\rho(x_t, \tau_t)$ with $\tau_t = 1 - \exp(-\vartheta t)$, sampling initial states from a uniform prior on $[a - 1, a + 1]$, stratifying the mini-batch over ten calendar-time bins of $[0, T_{\max}]$, and adding the terminal penalty $\lambda_T (x_{T_{\max}} - x_{T_{\max}}^*)^2$ to the loss. (iii) Verify that the trained network reproduces the LQ-Riccati baseline within 1% in the mean-squared-action metric. (iv) Ablate each of the three modifications in turn (drop τ_t from the input; remove stratification; remove the terminal penalty) and report which ablation hurts most as a function of $T_{\max}$.

Table 11.2: CDICE baseline calibration used in the deterministic CDICE-DEQN solve. Parameter values follow the Online Appendix of [Folini et al. \(2025\)](#) and are stated on an annual time step ($\Delta_t = 1$ yr). All carbon quantities ($M_{eq}, M_0, \sigma_0, E_{Land,0}$) are in CDICE’s 10^3 GtC working units; multiply by 10^3 to recover GtC. Two alternative climate calibrations (CDICE-HadGEM2-ES, CDICE-GISS-E2-R) are listed in the temperature block, with their full free-parameter sets $\{c_1, c_3, c_4, F_{2 \times CO_2}, \lambda\}$ and corresponding ECS, since simply varying ECS while holding the rest of the temperature block fixed is *not* equivalent to using the full CMIP5 calibration (§4.6 of [Folini et al., 2025](#)). Initial state is for year 2015.

Block	Parameter	Value	Meaning
Economy	α	0.30	Capital share in Cobb–Douglas output
	δ	0.10/yr	Capital depreciation rate
	ρ	0.015/yr	Pure rate of time preference
	ψ	0.69	Intertemporal elasticity of substitution
Emissions & abatement	σ_0	9.556×10^{-5} (10^3 GtC)/USD	Initial carbon intensity
	g_0^σ	-0.0152 /yr	Initial decay rate of σ_t
	δ^σ	0.001/yr	Curvature of σ_t decay
	p_0^{back}	0.55 thUSD/tCO ₂	Initial backstop price
	g^{back}	0.005/yr	Decay rate of backstop price
	θ_2	2.6	Curvature of $\Theta(\mu)$
	c_{2co2}	3.666	Carbon-to-CO ₂ mass conversion
Land use	$E_{Land,0}$	7.09×10^{-4} (10^3 GtC)/yr	Initial land-use emissions
	δ_{Land}	0.023/yr	Decay rate of $E_{Land,t}$
Carbon cycle	b_{12}	0.054/yr	Atm.–upper-ocean transfer rate
	b_{23}	0.0082/yr	Upper-ocean–lower-ocean transfer rate
	M_{eq}	(0.607, 0.489, 1.281) (10^3 GtC)	Pre-industrial equilibrium masses
Temperature	c_1 (MMM)	0.137/yr	Atmospheric heat-capacity inverse
	c_3 (MMM)	0.73/yr	Atm.–ocean coupling
	c_4 (MMM)	0.00689/yr	Ocean heat-capacity inverse
	$F_{2 \times CO_2}$ (MMM)	3.45 W/m ²	Forcing from CO ₂ doubling
	λ (MMM)	1.06 W/m ² /K	Climate feedback parameter
	ECS (MMM)	≈ 3.25 °C	Equilibrium climate sensitivity
	HadGEM2-ES	(c_1, c_3, c_4) (0.154, 0.55, 0.00671)/yr $F_{2 \times CO_2} = 2.95$, $\lambda = 0.65$, ECS ≈ 4.55 °C	= High-end CMIP5 calibration
	GISS-E2-R	(c_1, c_3, c_4) (0.213, 1.16, 0.00921)/yr $F_{2 \times CO_2} = 3.65$, $\lambda = 1.70$, ECS ≈ 2.15 °C	= Low-end CMIP5 calibration
Damages	π_1	0.0	Linear damage coefficient
	π_2	0.00236	Quadratic damage coefficient
Initial state	K_0	223 T USD	Capital, year 2015
	M_0	(0.851, 0.628, 1.323) (10^3 GtC)	Atm./upper/lower carbon, 2015
	T_0	(1.10, 0.27) °C	Atm./ocean temp. above pre-industrial, 2015

Chapter 12

Synthesis and Outlook

This final chapter steps back from the individual methods and applications to identify the unifying themes that connect them. We distill the common computational paradigm, embedding economic structure in a neural network loss function and optimizing via gradient descent, and provide a practical decision guide for choosing among DEQNs, PINNs, deep surrogates, and GP + BAL based on the problem at hand. We then discuss open challenges at the frontier and offer practical tips distilled from the experience of applying these methods to real research problems. The chapter is intentionally high-level: it emphasizes synthesis and practical judgment rather than introducing new technical machinery.

12.1 The Unifying Paradigm

The methods presented in this course, namely DEQNs, PINNs, deep surrogates, and GP + BAL, do not all use the same estimator, but they share a common computational workflow: a flexible function approximator, a deep network or a Gaussian-process kernel, encodes the economic object of interest; structural information enters through residuals, hard constraints, model-generated training data, or an acquisition rule; and a differentiable or Bayesian inference machinery delivers training, sensitivity analysis, and uncertainty quantification.

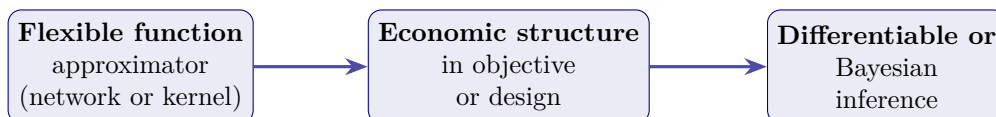


Figure 12.1: The shared computational workflow of the four methods of this course (DEQNs, PINNs, deep surrogates, and GP + BAL): a flexible function approximator (a deep network or a Gaussian-process kernel) plays the role of the unknown economic object; structural information enters through equilibrium residuals and hard constraints (DEQNs, PINNs), through model-generated training data and parameter pseudo-states (deep surrogates), or through the prior, the simulator, and the acquisition rule (GP + BAL); and a differentiable or Bayesian inference layer supplies the gradients, the posterior, and the design rule needed for training, sensitivity analysis, and uncertainty quantification.

Figure 12.1 makes the common template explicit. Structural knowledge of the economic or financial model is built directly into the training objective or the training design rather than learned from labeled data: DEQNs minimize equilibrium-residual losses, PINNs minimize PDE residuals, deep surrogates fit model-generated labels (often with parameter pseudo-states), and GP + BAL combines a kernel prior with an acquisition rule that decides where to sample next. Automatic differentiation supplies the gradients needed for network training, and the mesh-free nature of neural networks and kernels avoids explicit tensor-product state grids, mitigating the

practical curse of dimensionality without eliminating the underlying high-dimensional sample-complexity, approximation, optimization, and integration costs.

12.2 Decision Guide: When to Use Which Method

The four method families covered in this course occupy partly overlapping but distinguishable niches. Table 12.1 summarizes the key distinctions along four practical criteria: the natural *time domain* (discrete vs. continuous), the kind of *equation* the method is designed to solve, the *key advantage* that distinguishes it from the others, and the *typical use cases* encountered in this course.

Criterion	DEQN	PINN	Deep surrogate	GP / BAL
Time domain	Discrete	Continuous	Either	Either
Equation type	Algebraic / expectations	PDEs (HJB, KFE, BS)	Black-box model	Black-box model
Key advantage	Up to $d > 100$ in selected models (Azinovic et al., 2022, Tbl. 2)	AD derivatives of the network approximation	Instant re-evaluation	Built-in UQ + active learning
Typical use	DSGE, OLG, IRBC	HJB, option pricing	Estimation, UQ, policy	Small- N surrogates, BAL design

Table 12.1: Decision guide: when to use which method.

A few practical takeaways from the table. **DEQNs** are the workhorse for *discrete-time* general-equilibrium models with high-dimensional state spaces, where Euler equations and market-clearing residuals admit a clean residual-loss formulation. **PINNs** are the natural analogue in *continuous time*, replacing finite-difference HJB or BSDE solvers when one needs exact derivatives of the trained network by automatic differentiation rather than finite-difference derivatives of an interpolant. **Deep surrogates** differ in kind: they do not solve the model; they learn a fast emulator of the already-solved mapping from parameters to economic quantities, which is what makes structural estimation, sensitivity analysis, and policy design tractable at scale. **Gaussian processes with Bayesian active learning** occupy the small-data, high-uncertainty corner: the right tool when each evaluation is expensive, the input dimension is moderate ($d \lesssim 10$ for naïve GPs, extending to higher dimensions via active subspaces and deep AS; cf. Chapter 9), and posterior uncertainty is itself part of the answer. Within the PINN family, EMINNs (Chapter 8) are a specialization to continuous-time heterogeneous-agent master equations: they keep the PDE-residual training logic but encode the cross-sectional distribution itself as part of the network input, which is one of several routes to global solutions of Krusell–Smith-style models with aggregate shocks, complementary to the set-encoder and price-as-state approaches discussed in §12.5.

The categories are not mutually exclusive. Many production pipelines combine them, e.g., a DEQN to solve the equilibrium and a deep surrogate or GP that treats parameters as pseudo-states for estimation (Chapter 10), or a PINN with a surrogate wrapper for policy counterfactuals. The right question is rarely “which of the four?” but “which combination?”.

12.3 Running Benchmarks Through Every Lens

The course repeatedly returns to a small set of canonical benchmarks: Brock–Mirman for discrete-time stochastic growth, cake-eating for continuous-time HJBs, and parameterized Brock–Mirman

variants for surrogates and SMM. Table 12.2 reads each benchmark through the lens of one course method, exposing what changes when the methodology changes and what stays invariant.

Method	What is approximated	Loss / objective	Diagnostic	Notebook
DEQN	Savings share $s(K, z) \in (0, 1)$ via sigmoid head	Squared rel. Euler residual on simulated trajectory	$s \rightarrow \alpha\beta$ at $\delta=1$, log utility	01_Brock_Mirman_1972_DEQN.ipynb
PINN, cake-eating	Value function $\hat{V}(a)$ on a 1D interval	HJB residual $\rho V - \max_c [u(c) + V'(a)(ra - c)]$	$\hat{V}$ matches closed form V^* (CRRA)	04_Cake_Eating_HJB_PINN.ipynb
GP + BAL	$\hat{V}(K)$ on Brock–Mirman ergodic set	GP marginal likelihood; BAL picks next K_*	95% pred. band covers true V	04_GP_Value_Function_Iteration.ipynb
SMM + surrogate	Policy $s(K, z, \varrho)$ pseudo-state on ϱ	SMM moment objective in ϱ	$\hat{\varrho}$ within MCSE of truth	lecture_15_03_Structural_Estimation_BM.ipynb

Table 12.2: Canonical benchmarks treated through each course method. The economic content differs by row, three rows are Brock–Mirman and the PINN row is cake-eating, but the table exposes the invariant computational pattern: choose an object to approximate, encode the model restrictions in the objective or training design, and validate with an economically interpretable diagnostic. Notebook filenames in the last column omit the standard `lecture_NN_` prefix; the four notebooks live, in row order, in the `code/` subdirectories of lectures 03, 11, 14, and 15.

The takeaway is that no single methodology is the “right” one for these benchmarks. Rather, the four lenses answer different research questions: a DEQN gives the policy function on Brock–Mirman, a PINN gives a value function on cake-eating with AD derivatives of the network approximation, a GP gives uncertainty-quantified value-function estimates on Brock–Mirman with guidance for the next sample point, and the surrogate-SMM pipeline estimates the deep parameter ϱ (productivity persistence) on a Brock–Mirman variant from data; the joint (β, ϱ) extension lives in the companion `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb`, where the imperfect identification of β from macro moments shows up as a visible ridge in the criterion surface. Real research applications typically combine at least two of these, and the implementation guidance throughout the chapters is designed to make those combinations cheap.

12.4 Bridges Between Methods

The four method families do not sit in isolated boxes. Figure 12.2 maps the bridges between them: which methods share a state representation, which methods share an inference machinery, and where one method layers naturally on top of another. Discrete- and continuous-time formulations connect DEQNs and PINNs; deep surrogates and GP + BAL share the pseudo-state and uncertainty-quantification idea; and the lower row of the figure can be read as a pipeline (solve $\rightarrow$ amortize $\rightarrow$ quantify) rather than as four parallel choices.

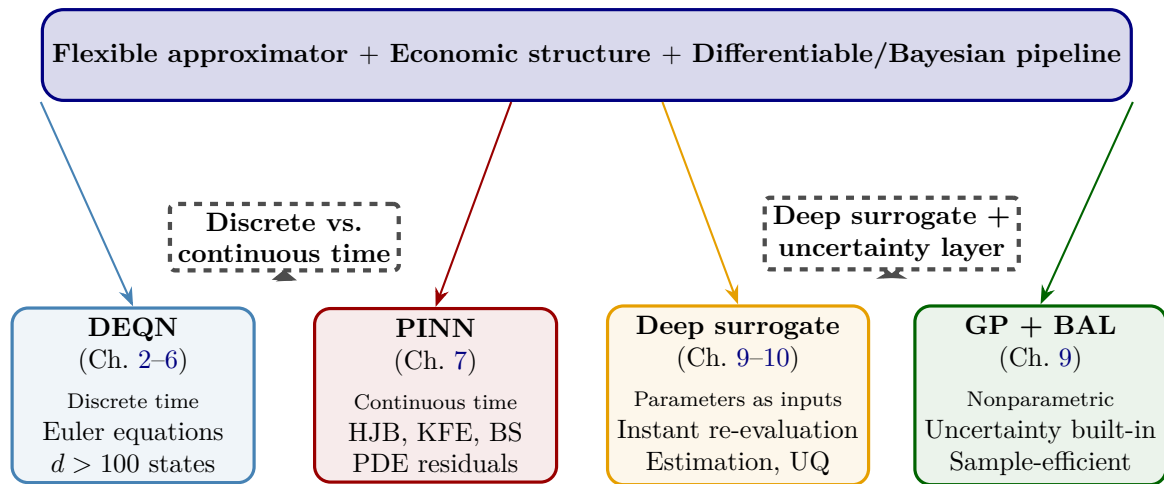


Figure 12.2: Bridges between the four method families. The core box restates the shared workflow: a flexible approximator (network or kernel), with economic structure encoded in the objective or training design, trained or updated through a differentiable or Bayesian pipeline. The dashed gray arrows mark the two main bridges, between discrete- and continuous-time residual solvers (DEQN $\leftrightarrow$ PINN) and between deep surrogates and the GP + BAL uncertainty layer that sits on top of them.

Reading the figure column by column gives a one-line gloss of each family: DEQNs (blue) handle discrete-time Euler equations in high-dimensional state spaces; PINNs (red) handle continuous-time HJB, Kolmogorov-forward, and Black–Scholes PDEs through PDE residuals; deep surrogates (orange) make model parameters part of the input space and replace the expensive solve with an instant forward pass for estimation and uncertainty quantification; GP + BAL (green) layers calibrated uncertainty and sample-efficient active learning on top of the surrogate. The bridges in the figure are not exhaustive; production pipelines often chain three families at once, for example a DEQN solver followed by a deep surrogate, followed by a GP for sensitivity analysis around the estimated optimum.

12.5 Open Challenges and Future Directions

Despite the considerable progress surveyed in this course, several open challenges remain at the frontier of deep learning for economics:

1. **Convergence guarantees.** While empirical performance is strong, theoretical convergence guarantees for DEQNs and PINNs in the context of economic equilibrium computation remain limited. Establishing rates of convergence as a function of network width, depth, and training budget is an active area of research. The gap between theory and practice remains wide.
2. **Combining global and local methods.** Deep Kernel Learning (Wilson et al., 2016; Chen et al., 2025), which uses a DNN as a feature map inside a GP kernel, is one principled step toward integrating the two paradigms. In principle, such hybrids could retain some of the GP’s calibrated uncertainty estimates while benefiting from the DNN’s representational flexibility. In practice, however, the UQ properties of deep kernel models require careful validation, their scalability advantages over pure GPs are problem-dependent, and locating the kinks and boundary layers where local refinement would help most is itself a hard unsolved problem. Hybrid methods therefore remain a promising but largely open research direction rather than a deployable solution.

3. **Real-time policy analysis.** Deep surrogates enable near-instant policy evaluation, potentially transforming how policy makers conduct scenario analysis and stress testing. Kübler et al. (2026) demonstrate that constrained optimal policy rules can be computed over high-dimensional parameter spaces using deep surrogates, opening the door to real-time policy counterfactuals. For central banks and finance ministries, this could mean running complex DSGE scenarios with substantial speedups (seconds rather than minutes-to-hours, depending on the underlying solver), enabling more responsive and comprehensive policy analysis during fast-moving economic crises.
4. **Estimation of rich structural models.** Combining DEQNs or PINNs with surrogate-based estimation pipelines allows researchers to estimate models with many parameters from micro and macro data simultaneously (Friedl et al., 2023; Chen et al., 2026). Kase et al. (2022) use neural networks to estimate nonlinear heterogeneous-agent models, opening the door to estimation of models that were previously intractable. As micro-level administrative datasets become increasingly available to central banks, the ability to estimate high-dimensional structural models from rich data sources represents a major frontier.
5. **Climate–economy integrated assessment.** Integrated assessment models with tipping points, regional heterogeneity, and multiple sources of risk represent a natural application of the methods surveyed here. Fernández-Villaverde et al. (2025) provide a comprehensive overview of the state of the art. The DEQN methodology is particularly well suited to these models because climate–economy interactions introduce additional state variables (carbon concentrations, temperature, damages) that exacerbate the curse of dimensionality.
6. **Heterogeneous-agent models and beyond.** Extending the DEQN and PINN frameworks to heterogeneous-agent models with aggregate shocks, where the cross-sectional distribution of agents is itself part of the aggregate state, is a major frontier (Chapter 6). Han et al. (2024) propose DeepHAM, which represents the distribution via learned generalized moments and trains networks by maximizing cumulative utility along simulated paths (§6.6.2); Payne et al. (2025) extend the same set-encoder idea to non-Walrasian search-and-matching models, where the distribution affects decisions through the matching technology and bargaining rather than through prices (§6.6.3); and Yang et al. (2025) show that for many Walrasian HA models, equilibrium prices alone, rather than the full distribution, can serve as the aggregate state in agents’ policy functions, sidestepping the master equation altogether. Together these approaches demonstrate that deep learning can tackle the infinite-dimensional state spaces inherent in heterogeneous-agent economies, and can also help *define* new equilibrium concepts for them.
7. **Operator learning.** Rather than learning a single function (as in DEQNs or PINNs), operator learning frameworks such as DeepONet (Lu et al., 2021a) and the Fourier Neural Operator (Li et al., 2021) learn mappings between function spaces, for example, learning the map from a forcing function to the PDE solution, or from a model specification to its equilibrium. This “learn the solver” paradigm could enable even faster model evaluation by amortizing the cost of solving across many model instances.
8. **Error certification and benchmark protocols.** A small residual on the training distribution is not the same as a small policy or welfare error on the economically relevant state space. A major frontier is the development of a-posteriori error bounds, benchmark suites with classical baselines, and standardized reporting conventions: residual heat maps, Euler-error distributions, market-clearing errors, transversality checks, and consumption-equivalent welfare losses. Without such certification, deep solvers are hard to compare across papers and hard to audit in policy environments.
9. **Distribution shift, tails, and rare events.** Simulation-based training concentrates

samples on the ergodic set. This is efficient for average performance but dangerous when the research question concerns crises, tail risk, climate tipping points, binding collateral constraints, or off-policy counterfactuals. Robust sampling schemes that combine ergodic states, boundary states, adversarial states, and economically meaningful stress scenarios are still underdeveloped, and they are precisely the regime where small training residuals can coexist with large welfare errors.

12.6 When *Not* to Use Deep Learning

A pedagogical script risks selling the methods it teaches. Symmetry requires an explicit statement of the regimes in which the deep-learning toolkit covered here is *not* the right answer; recognizing those regimes is part of using the toolkit well.

1. **The state space is genuinely low-dimensional.** For models with $d \lesssim 5$ continuous state variables and smooth policies, classical methods, projection on Chebyshev polynomials, value-function iteration on a fine grid, or perturbation around the steady state, typically deliver higher accuracy at lower cost than a DEQN (Judd, 1998). The curse of dimensionality is real, but for smooth low-dimensional models it is rarely the binding constraint; nonsmoothness, occasionally binding constraints, and tail events can make even low-dimensional problems hard.
2. **The model is locally linear and the question is local.** If you only need first- or second-order responses around the deterministic steady state, perturbation methods give analytical impulse responses in seconds; the standard implementation in macroeconomics is Adjemian et al. (2024). Use a DEQN or a PINN only when global nonlinearity, large shocks, occasionally binding constraints, or the ergodic distribution itself is the object of interest.
3. **You need likelihood-based inference and the model is small.** When closed-form or quadrature-based likelihoods are available, full-information ML and Bayesian estimation with a particle filter dominate any simulation-based approach in efficiency. The deep-surrogate route in Chapter 10 pays off only when the likelihood is intractable or the simulator is much more expensive than evaluating the network.
4. **Sample size is small and uncertainty is the answer.** At $n \lesssim 10^3$ design points and moderate input dimension, a well-specified GP with active learning (Chapter 9) often provides stronger uncertainty quantification and competitive point prediction relative to a deep network of comparable complexity; the advantage can disappear when the dimension is high, the kernel is misspecified, or the response is strongly nonstationary. Default to GPs when the simulator is expensive and posterior uncertainty matters.
5. **Reproducibility and audit-ability are paramount.** Deep-learning solutions depend on initialization seeds, optimizer tuning, and floating-point order of operations on the GPU. In central-bank or regulatory contexts where round-off-stable, configuration-light reproducibility is required, a deterministic finite-difference scheme remains preferable; bit-exactness across heterogeneous BLAS/LAPACK back-ends is not guaranteed for either approach, but FD typically has fewer moving parts to pin. At minimum, pin every random seed and report hardware/software versions (cf. the reproducibility appendix).
6. **Training instability is a model-misspecification signal.** If a DEQN refuses to converge, the diagnosis is rarely “add more layers”, it is usually a poorly specified loss, a missing hard constraint, an unstable simulation, or a model that admits no equilibrium near the trial states. Spending a day debugging a divergent training run is often the discovery that the model itself needs to change.

The chapters of this script argue that, in the right regime, deep learning unlocks problems that are out of reach of any other tool. The point of this short list is the converse: there is a long catalog of standard problems where it is the wrong tool, and recognizing them keeps the methodology honest.

12.7 Practical Tips and Common Pitfalls

The following guidelines distill recurring lessons from the applications covered in this course:

1. **Start simple.** Begin with a small network (2–3 hidden layers, 32–64 neurons) and a low-dimensional test case with a known solution (e.g., Brock–Mirman for DEQNs, the 1D ODE for PINNs). Only increase complexity once the simple case works.
2. **Normalize states, controls, and residuals.** Many failed training runs are scaling failures rather than approximation failures. Work with logs, ratios, standardized states, or economically natural units, and scale residuals so that one equation does not dominate the loss only because it is measured in larger units. In climate and OLG models, nondimensionalization is often as important as the architecture.
3. **Build in hard constraints where possible.** Positivity of consumption, bounds on savings or abatement, no-arbitrage restrictions, and budget feasibility should be enforced by output transformations or hard ansatzes whenever they can be. Penalty terms are useful, but a network that cannot violate a constraint is usually easier to train and easier to trust; this is also where monotonicity and concavity restrictions on policy or value functions are best imposed.
4. **Use independent validation states and shocks.** Report residuals on held-out state points, independent shock sequences, boundary states, and economically important stress scenarios. A solver that performs well only on the training simulation path has not solved the global model; this is the practical counterpart to the distribution-shift frontier discussed in §12.5.
5. **Monitor all loss components.** In multi-term losses (PDE residual + boundary penalties, or Euler residuals + complementarity), log each component separately. A declining total loss can mask a rising boundary term.
6. **Use adaptive loss balancing.** Manual tuning of penalty weights λ is fragile. ReLoBRaLo (Chapter 4) and related schemes automatically balance competing loss terms; in the benchmarks reported by Bischof and Kraus (2025) they substantially improve accuracy over fixed weights, with the magnitude of the gain depending on the problem.
7. **Check economic diagnostics, not just the loss.** Verify that policy functions satisfy economic intuition (e.g., consumption increasing in wealth), that market clearing holds, and that the relative Euler error is economically small (e.g., median or mean below 10^{-3}). Pick one convention (relative Euler error, $\log_{10}$ Euler error, or consumption-equivalent error) and use it consistently across diagnostics.
8. **Choose activations deliberately.** In strong-form PINNs that involve second-order PDEs, use C^∞ activations such as tanh, Swish, or softplus, since ReLU has $u'' = 0$ a.e.; weak-form formulations and methods that handle nonsmooth solutions explicitly can still use ReLU. Swish is a strong default for DEQNs. Avoid mixing activation types across layers unless there is a specific reason.
9. **Use common random numbers (CRN).** When comparing model outputs across parameter values (e.g., in surrogate-based estimation), always fix the shock sequence.

CRN can reduce variance substantially in surrogate-SMM pipelines (often by an order of magnitude or more), making gradient-based optimization feasible.

Common failure modes. Table 12.3 summarizes recurring failure modes and their mitigations:

Failure Mode	Mitigation
Loss decreases but solution is wrong	Check individual loss components; use economic diagnostics
One residual dominates all others	Adaptive loss balancing (Ch. 4); inspect per-component gradients; normalize residuals to comparable units
Solver looks fine on simulation path, fails off path	Add held-out validation states; sample corners and stress states; use residual-based adaptive resampling
Policy network outputs collapse to zero (e.g., $c \equiv 0$ or $s \equiv 0$)	Improve initialization; use simulation-based training; check that hard constraints are not forcing the trivial solution
NaN in loss	Clamp inputs to GP/network; check for log of negative values
Euler errors large in corners of state space	Increase sampling near boundaries; use hard constraints
Surrogate gives a precise but biased SMM optimum	Validate the surrogate near the estimated optimum; add active-learning points around the criterion minimum; cross-check with direct DEQN re-evaluation
GP posterior variance too large	Add training points via BAL; check kernel hyperparameters
Training succeeds only for one seed	Report multi-seed dispersion; reduce learning rate; tighten scaling; check residual signs

Table 12.3: Recurring failure modes encountered when training the deep-learning solvers of this course (DEQNs, PINNs, deep surrogates, GP + BAL), together with the practical mitigations that work most reliably across the companion notebooks. The list is short by design: most production failures fall into one of these categories, and the corresponding fix is usually the first thing to try before invoking heavier diagnostics.

Implementation checklist. For each new model, we recommend the following workflow:

1. Solve a simplified version with a known analytical solution (e.g., Brock–Mirman).
2. Verify that all equilibrium conditions are correctly encoded in the loss.
3. Train with a small network and monitor all loss components separately.
4. Check economic diagnostics: policy function shapes, market clearing, Euler errors.
5. Scale up: increase network size, dimension, and training budget.
6. Document the final configuration and report reproducibility information.

12.8 Concluding Remarks

The methods presented in this course are not merely computational tricks: they expand the feasible frontier of quantitative economic modeling when state spaces are high-dimensional, equilibrium restrictions are differentiable, and repeated model evaluation is valuable. They do not

replace classical numerical economics; they add a flexible amortized layer that is most powerful when combined with economic structure, careful diagnostics, and classical benchmarks.

The convergence of abundant compute, mature software ecosystems (TensorFlow, PyTorch, JAX), and increasingly complex economic questions ensures that deep learning methods will play an expanding role in economic research and central bank practice in the years ahead. We hope that this course has provided the conceptual foundations and practical tools needed to engage with this frontier.

For further reading, we refer to the comprehensive survey by [Fernández-Villaverde et al. \(2024\)](#), the methodological foundations laid in [Azinovic et al. \(2022\)](#), and the applications in [Renner and Scheidegger \(2018\)](#), [Friedl et al. \(2023\)](#), [Han et al. \(2024\)](#), [Payne et al. \(2025\)](#), [Kase et al. \(2022\)](#), [Chen et al. \(2026\)](#), [Kübler et al. \(2026\)](#), and [Fernández-Villaverde et al. \(2025\)](#). The list of references in these notes is necessarily incomplete; for a full bibliography, we refer the reader to the cited papers and the references therein.

Chapter Summary

- All four method families share one paradigm: *neural network as function approximator + economic structure in the loss + automatic differentiation for training*.
- Knowing when *not* to use deep learning is part of using it well: classical projection or perturbation often dominates in low dimension; exact likelihood beats simulation when both are available; a well-specified GP is often the better choice at small n and moderate dimension, when uncertainty is the deliverable.
- Open frontiers: convergence theory, hybrid global/local methods, real-time policy analysis, large structural estimation, climate–economy IAMs, full HA models with aggregate shocks, operator learning.
- The right question for a research project is rarely “which of these four methods?” but “which combination?”, e.g. DEQN + surrogate for estimation, PINN + GP for sensitivity, or all of the above for climate policy.

Exercises

Worked solutions and guidance for these exercises appear in Appendix F.

1. **[Core] Method-choice scenario.** You are handed (a) a 4-state monetary-policy DSGE with smooth shocks, (b) a 200-agent OLG with progressive taxation, (c) an option-pricing problem on an irregularly shaped exotic payoff, and (d) a climate-IAM where uncertainty in the SCC is the deliverable. Justify which method you would use for each, and what hybrid you would consider. For each case, state the classical baseline you would compare against before reaching for a deep-learning method.
2. **[Core] When NOT to use deep learning.** Pick one of the six regimes from §12.6 and write a one-page note for a colleague explaining why a classical method dominates.
3. **[Computational] Reproducibility audit.** Take any notebook from this script and document hardware, software versions, and seeds. Re-run end-to-end and report the maximum deviation in any reported number.
4. **[Advanced/project] Open-ended.** Identify one frontier from §12.5 (operator learning, hybrid global/local, real-time policy, large structural estimation, climate-economy IAMs, HA with aggregate shocks) and sketch a 6-month research project that combines two methods from this script.

5. **[Advanced/project] Hybrid pipeline: DEQN + GP + SMM end-to-end.** Build a complete estimation pipeline on the Brock–Mirman model with two parameters (β, ϱ) , matching the parameter pair worked in the companion notebook `lecture_15_03b_Structural_Estimation_BM_Joint.ipynb`. As that notebook documents, β is only *partially* identified by macro moments (a visible ridge along the β axis); ϱ has the cleaner identification map. The exercise below is a controlled-difficulty version of that pipeline.

- (i) Train a DEQN with *pseudo-state augmentation*: extend the network input from (K_t, z_t) to $(K_t, z_t, \beta, \varrho)$.
- (ii) On a 20×20 grid of $(\beta, \varrho) \in [0.92, 0.99] \times [0.50, 0.99]$, simulate the model and compute four moments: mean savings rate, mean consumption growth, $\text{Var}(\log C_t)$, and $\text{Var}(K_t)$.
- (iii) Fit a GP surrogate $(\beta, \varrho) \mapsto \mathbf{m}$ on the resulting 400-point dataset.
- (iv) Generate observed moments at $(\beta^*, \varrho^*) = (0.96, 0.90)$ from one $T = 200$ simulation and solve the GP-based SMM problem.

Report the wall-clock time of each step, total pipeline time, estimation errors, a comparison to a naive SMM that re-solves the DEQN at each candidate (β, ϱ) , the hold-out surrogate error on (β, ϱ) pairs outside the 20×20 grid, a contour plot of the GP-based SMM criterion that exposes the β ridge, and a direct DEQN re-evaluation at the GP optimum to test surrogate bias. Evaluate whether the surrogate-based optimization is faster after accounting for the one-time GP fitting overhead and whether its accuracy is comparable. Bonus: bootstrap confidence intervals for $(\hat{\beta}, \hat{\varrho})$ using the parametric bootstrap of Exercise 6.

6. **[Advanced/project] DeepONet for a parameterized HJB.** Consider the cake-eating HJB

$$\rho V(a; \gamma) = \max_c \left[\frac{c^{1-\gamma}}{1-\gamma} + V'(a; \gamma)(ra - c) \right], \quad \gamma \in [1.5, 5],$$

This is deliberately a low-dimensional operator-learning toy: since the branch input is the scalar γ , a parameter-conditioned PINN would be an equally natural baseline. The purpose of the exercise is to expose the DeepONet branch–trunk decomposition before moving to function-valued branch inputs (e.g., a state-dependent drift $r(a)$ or discount rate $\rho(a)$ observed at sensor points).

- (i) Sketch the DeepONet architecture for learning the operator $\mathcal{G} : \gamma \mapsto V(\cdot; \gamma)$: identify the branch-net input and the trunk-net input, and write the predicted output as the inner product of branch and trunk outputs.
- (ii) State the operator-learning universal approximation theorem of Lu et al. (2021a) and explain why a DeepONet of moderate width can approximate any continuous operator on a compact set.
- (iii) Suppose you want $V(\cdot; \gamma)$ for $N = 50$ values of γ . Compare N independent PINN runs at cost C_{PINN} each with one DeepONet run at cost C_{DON} . At what ratio $C_{\text{DON}}/C_{\text{PINN}}$ does operator learning win, and how does the break-even ratio scale with N ?
- (iv) Discuss two limitations: extrapolation outside $[1.5, 5]$ and preservation of structural properties such as concavity of V in a .

Appendix A

Glossary

A short glossary of recurring concepts. Each entry is one or two sentences; for the full treatment, follow the cross-references in parentheses.

Active subspace.

The leading eigenspace of the gradient outer-product matrix $\mathbb{E}[\nabla f \nabla f^\top]$. Captures the linear directions in input space along which a function varies most; allows GPs to scale to high d (Constantine, 2015).

Active subspace, deep.

Replaces the linear projection $U_m^\top \mathbf{x}$ by a learned nonlinear encoder $h: \mathbb{R}^D \rightarrow \mathbb{R}^d$, trained jointly with an MLP link $g: \mathbb{R}^d \rightarrow \mathbb{R}$ so that $\hat{f}(\xi) = g(h(\xi))$; gradient-free, with d chosen by a validation-MSE elbow instead of a spectral gap (Section 9.5.1; Tripathy and Bilonis, 2018).

Adam, AdamW.

Adaptive stochastic-gradient optimizers using momentum on the gradient and on its square; AdamW separates the weight-decay step from the adaptive step (Kingma and Ba, 2015; Loshchilov and Hutter, 2019).

Approximate aggregation.

Empirical observation that in Krusell–Smith-class economies the cross-sectional distribution of wealth is nearly summarized by its mean for price forecasting purposes (Chapter 6).

Automatic differentiation (AD).

Algorithmic computation of exact derivatives of composite functions via the chain rule; reverse-mode AD is the engine of every deep-learning framework (Baydin et al., 2018; Margossian, 2019).

Bayesian active learning (BAL).

Adaptive sample-design strategy in which the next training point is chosen to maximize an acquisition function based on predictive uncertainty; pairs naturally with GPs (Chapter 9).

Bellman equation.

Recursive characterization of the value function in a discrete-time dynamic program. Continuous-time analogue is the HJB equation.

Brock–Mirman model.

Stochastic neoclassical growth model with log utility and full depreciation that admits a closed-form policy $s^* = \alpha\beta$; the canonical DEQN benchmark in this script.

Common random numbers (CRN).

Variance-reduction technique in which the same shock realizations are reused across simulations of different parameter values, removing simulation noise from comparisons (Glasserman, 2004).

Collocation point.

A spatial location at which a PDE residual is evaluated and minimized in a PINN training loop (Chapter 7).

Curse of dimensionality.

Exponential blow-up of grid-based methods in the dimension of the state space; mitigated, not eliminated, by neural-network and GP approximators.

Deep Equilibrium Net (DEQN).

A neural-network-based solver for dynamic stochastic equilibrium models that minimizes the equilibrium-equation residuals directly via SGD (Chapter 2).

Deep Galerkin Method (DGM).

An LSTM-style architecture introduced by [Sirignano and Spiliopoulos \(2018\)](#) for solving high-dimensional PDEs; the architectural sibling of standard PINNs.

Deep kernel learning (DKL).

Composes a neural-network feature extractor with a GP layer in the learned feature space ([Wilson et al., 2016](#)).

DeepONet.

A neural architecture for operator learning: a branch net encodes the input function and a trunk net encodes the query point; the inner product is the predicted output ([Lu et al., 2021a](#)).

EMINN.

Economic-Model Informed Neural Network: a PINN-style approach to the master equation in continuous-time HA models ([Gu et al., 2024](#)).

Ergodic distribution.

The stationary distribution of a Markov process; in a Krusell–Smith economy it is the long-run distribution of wealth across agents.

Fischer–Burmeister (FB).

A smooth complementarity function $\Phi(a, b) = a + b - \sqrt{a^2 + b^2}$ used to encode KKT conditions in differentiable losses; the opposite sign has the same zero set but the chapter and notebooks use this convention ([Fischer, 1992](#)).

Fourier Neural Operator (FNO).

Operator-learning architecture parameterizing a kernel integral operator in Fourier space; cheap and resolution-invariant ([Li et al., 2021](#)).

Functional derivative.

$\delta V / \delta g$, the density / Riesz representer of the Fréchet derivative of V with respect to a function-valued argument g (equivalently, the directional derivative of V at g in the direction of a Dirac perturbation δ_{y_0}); appears in the master equation (Chapter 8).

Gauss–Hermite quadrature.

Polynomial quadrature rule for integrals against the standard normal density; backbone of the expectations step in DEQNs.

HJB equation.

Hamilton–Jacobi–Bellman equation; continuous-time analogue of the Bellman equation, a PDE in the value function.

Histogram (Young 2010).

Mass-redistribution scheme on a fixed grid that propagates a wealth distribution deterministically without Monte Carlo noise (Chapter 6).

Hyperband.

Successive-halving multi-fidelity hyperparameter scheduler that explores many configurations cheaply and concentrates budget on the survivors (Li et al., 2018).

Inducing points.

A small set of pseudo-data points used in sparse GPs to approximate the full kernel matrix at $\mathcal{O}(nm^2)$ cost (Titsias, 2009).

Itô’s lemma.

The chain rule of stochastic calculus; for the scalar diffusion $dX_t = \mu dt + \sigma dB_t$, the only difference from ordinary calculus is the second-order correction $\frac{1}{2}f''(X_t)\sigma^2 dt$.

Karush–Kuhn–Tucker (KKT) conditions.

First-order necessary conditions for constrained optimization; encoded smoothly via Fischer–Burmeister in DEQN losses.

Kolmogorov forward equation (KFE / Fokker–Planck).

The PDE governing the time evolution of the probability density of an Itô process.

Marginal likelihood.

The log-evidence $\log p(\mathbf{y} \mid \boldsymbol{\theta})$ in a GP; sum of a data-fit term and a complexity penalty (Chapter 9).

Master equation.

A single PDE that subsumes the HJB, KFE, and market-clearing conditions of a continuous-time mean-field-game equilibrium; argument includes the cross-sectional measure g .

Mean field game (MFG).

Equilibrium concept in which each atomistic agent best-responds to the cross-sectional distribution and the distribution evolves under those best responses; the natural framework for the HJB+KFE system (Lasry and Lions, 2007).

Neural Tangent Kernel (NTK).

In the infinite-width limit, gradient-descent training of a deep network is equivalent to kernel regression with the (deterministic) NTK (Jacot et al., 2018).

Operator learning.

Learning a map between function spaces (input field $\rightarrow$ solution function) rather than a single solution; DeepONet and FNO are the leading architectures.

Physics-Informed Neural Network (PINN).

A neural network trained by minimizing a PDE residual at collocation points plus boundary-condition penalties (Raissi et al., 2019).

Pseudo-state.

Treating model parameters as additional inputs to a neural network so that the trained surrogate covers an entire parameter range without retraining (Chapter 9).

Quasi-Monte Carlo (QMC).

Deterministic low-discrepancy sequences (Sobol, Halton, Niederreiter) achieving error rates close to $\mathcal{O}(1/M)$ for smooth integrands.

ReLoBRaLo.

Adaptive loss-balancing scheme that reweights multi-component losses by recent relative-decrease ratios (Bischof and Kraus, 2025).

Simulated Method of Moments (SMM).

Estimator that matches simulated to empirical moments; the natural extension of GMM when moments lack closed form (McFadden, 1989).

Simulation-based inference (SBI).

Modern likelihood-free Bayesian inference using neural conditional density estimators ([Cranmer et al., 2020](#)).

Social cost of carbon (SCC).

Marginal welfare cost of one additional unit of emissions, commonly reported as USD/tCO₂ after choosing the consumption numeraire and applying the carbon-to-CO₂ conversion; the headline policy number from a climate IAM.

Sobol / Shapley indices.

Variance-decomposition tools for global sensitivity analysis. Sobol decompositions are cleanest under independent inputs; Shapley effects allocate variance across inputs and can be defined for dependent inputs when the dependence structure is modeled explicitly.

Universal approximation.

A single-hidden-layer network with a non-polynomial activation can approximate any continuous function on a compact set arbitrarily well ([Cybenko, 1989](#); [Hornik et al., 1989](#)).

Value Function Iteration (VFI).

Classical contraction-mapping algorithm for solving the Bellman equation by iterating the Bellman operator until convergence.

Young’s lottery.

The unique two-point split that, when applied to off-grid policy choices, preserves the conditional mean exactly; the building block of the histogram update.

Appendix B

Matrix Calculus and Automatic Differentiation

This appendix collects the few matrix-calculus identities used in the main text and gives a one-page summary of how automatic differentiation realizes them computationally.

B.1 Matrix calculus identities

For $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} = \mathbf{A}\mathbf{x}$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{A}, \quad \frac{\partial (\mathbf{y}^\top \mathbf{y})}{\partial \mathbf{x}} = 2\mathbf{A}^\top \mathbf{A}\mathbf{x}.$$

For a quadratic form $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q}\mathbf{x}$ with symmetric $\mathbf{Q}$, $\nabla f(\mathbf{x}) = 2\mathbf{Q}\mathbf{x}$ and $\nabla^2 f = 2\mathbf{Q}$. For a deep network $\hat{\mathbf{y}} = g_L(\mathbf{W}_L g_{L-1}(\cdots g_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots))$, the chain rule gives

$$\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{W}_l} = \underbrace{(\boldsymbol{\delta}^{(l)})}_{\text{backprop delta}} (\mathbf{a}^{(l-1)})^\top$$

with $\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}) \odot g'(\mathbf{z}^{(l)})$ (§1.6).

B.2 Reverse-mode AD in one paragraph

Reverse-mode AD evaluates the function once forward, caches every intermediate value, and then traverses the computation graph backwards, accumulating the local Jacobian–vector products in linear time in the number of operations. For a scalar loss, one reverse sweep returns the gradient with respect to all parameters at a small constant multiple of the forward cost. The computation still scales with the size of the graph, but it does not require one separate derivative pass per parameter; this is what makes million-parameter networks trainable. See [Baydin et al. \(2018\)](#) and [Margossian \(2019\)](#) for complete treatments.

B.3 Higher-order AD

PINN losses involve second derivatives (V_{SS} in Black–Scholes, Laplacians in Poisson equations, and second-order terms in diffusion HJBs). Higher-order derivatives are obtained by composing reverse-mode AD with itself, `torch.autograd.grad` of `grad` in PyTorch, `jax.grad` of `jax.grad` in JAX. Activation regularity matters: the network must be at least C^k for the strong k -th-order residual to be well-defined.

Appendix C

Itô Calculus, Brownian Motion, and Ergodicity

This appendix collects the stochastic-calculus background needed for Chapters 7–8. For the longer, example-driven exposition see §8.2; for a full textbook treatment, see [Shreve \(2004\)](#).

C.1 Brownian motion

A standard Brownian motion B_t has independent Gaussian increments $B_t - B_s \sim \mathcal{N}(0, t - s)$, continuous paths almost surely, and quadratic variation $\langle B \rangle_t = t$. Equivalently, the simple-random-walk scaling limit $X_{t+\Delta t} = X_t + \sqrt{\Delta t} \varepsilon_t$ converges in distribution to B_t as $\Delta t \rightarrow 0$ (Donsker).

C.2 Itô's lemma

For X_t following $dX_t = \mu dt + \sigma dB_t$ and $f \in C^2$:

$$df(X_t) = [f'(X_t)\mu + \tfrac{1}{2}f''(X_t)\sigma^2]dt + f'(X_t)\sigma dB_t.$$

The differential algebra is $dt \cdot dt = dt \cdot dB_t = 0$, $dB_t \cdot dB_t = dt$. (Throughout the script, the stochastic integral is interpreted in the Itô sense; the Stratonovich convention would replace the drift correction $\frac{1}{2}f''(X_t)\sigma^2$ by 0.)

C.3 Ergodicity in one paragraph

A Markov process with stationary distribution π is *ergodic* if the time-average along any sample path converges almost surely to the spatial average against π : $\frac{1}{T} \int_0^T \varphi(X_t) dt \xrightarrow{T \rightarrow \infty} \int \varphi d\pi$ for every bounded measurable φ . In economic models with bounded state spaces and aperiodic dynamics, ergodicity is what justifies replacing population integrals by simulation-time averages in DEQN training (Chapter 2, §2.3).

Appendix D

Fixed Points, Contraction Mappings, and the Bellman Operator

The classical numerical-economics toolkit rests on Banach’s contraction-mapping theorem; this appendix recalls the statement and one short proof sketch for completeness. A reference is [Stokey et al. \(1989\)](#).

D.1 Banach’s theorem

Let (X, d) be a complete metric space and let $T : X \rightarrow X$ be a contraction with modulus $\beta < 1$, i.e. $d(Tx, Ty) \leq \beta d(x, y)$ for all x, y . Then T has a unique fixed point x^* , and for every starting point x_0 the iterates $x_{n+1} = Tx_n$ satisfy $d(x_n, x^*) \leq \beta^n d(x_0, x^*)$.

D.2 The Bellman operator is a contraction

For a discount factor $\beta \in (0, 1)$ and bounded utility, the Bellman operator $TV(x) = \max_a \{u(x, a) + \beta \mathbb{E}[V](x') \mid x_t = x, a_t = a\}$ is a contraction with modulus β on the space of bounded continuous functions equipped with the supremum norm. Hence value-function iteration converges geometrically. The same logic underpins *policy iteration*, the dual algorithm that converges in finite steps for finite-state problems.

D.3 Why this matters for DEQNs

DEQNs do *not* iterate a Bellman operator. Instead, they apply gradient descent to a residual loss that vanishes at the equilibrium policy. The contraction property therefore disappears as a tool for proving convergence, and theoretical guarantees come from neural-network optimization theory rather than fixed-point analysis: convergence arguments rest on universal approximation plus loss-landscape and NTK-style analyses of stochastic gradient descent on overparameterized networks, surveyed alongside the DEQN-specific open questions in Chapter 12.

Appendix E

Reproducibility Information

Every empirical statement in the main text relies on the companion notebooks listed in the *Execution Map*. Bit-exact reproducibility on a different machine requires fixing both the random seeds and the floating-point environment; Table E.1 summarizes the conventions used in this script’s notebooks.

For full bit-exact reproduction, e.g. for a regulatory-grade audit, the determinism flags above are necessary but not sufficient: GPU non-determinism in atomic accumulators and BLAS implementation differences across CUDA versions can still cause diverging results in the last few decimal places. This is a known limitation of GPU-accelerated deep learning and is one of the reasons §12.6 flags reproducibility-critical settings as a regime where deterministic finite-difference solvers may still be preferred.

Table E.1: Reproducibility conventions used by the companion notebooks. The conventions pin random seeds and document hardware, software, and precision choices; bit-exact reproduction additionally requires the deterministic settings and caveats described below the table.

Item	Convention
Random seeds	Each notebook declares a <code>SEED</code> constant in cell 1 (default 0); the corresponding framework seeds (<code>numpy.random.seed(SEED)</code> , <code>torch.manual_seed(SEED)</code> , <code>tf.random.set_seed(SEED)</code> , and <code>jax.random.PRNGKey(SEED)</code> where the framework is used) are derived from it. Per-cell offsets and per-iteration deviations are documented inline. Notebook-by-notebook normalization against this convention is tracked through the chapter audits; some legacy notebooks still hard-code a literal seed and are scheduled for normalization.
Run-mode budget	Each notebook declares <code>RUN_MODE</code> $\in \{\text{smoke}, \text{teaching}, \text{production}\}$ alongside <code>SEED</code> in cell 1. <code>smoke</code> is sized for a single CPU core (CI-friendly); <code>teaching</code> for one consumer GPU; <code>production</code> for an A100 or larger. Per-notebook hyperparameters (epochs, batch sizes, restart counts) are gated on <code>RUN_MODE</code> .
Hardware	Classroom-scale runs target a single CPU core or one consumer GPU (e.g. NVIDIA T4 / RTX 3060). Production runs use one A100 or larger; this is documented per-notebook in the chapter.
Software stack	Python ≥ 3.10 , TensorFlow ≥ 2.15 , PyTorch ≥ 2.0 , JAX $\geq 0.4.20$, GPyTorch ≥ 1.11 . Pinned versions live in <code>requirements.txt</code> on the companion repository. JAX appears in the Krusell-Smith warm-up notebook in <code>lectures/lecture_10_*</code> (sequence-space DEQNs); the other lectures use TensorFlow or PyTorch.
Numerical precision	TensorFlow / PyTorch default to <code>float32</code> ; PINNs and EMINN training use <code>float64</code> where second-order derivatives are sensitive (documented inline).
GPU determinism	<i>Optional, bit-exact runs only:</i> set <code>CUBLAS_WORKSPACE_CONFIG=:4096:8</code> and <code>torch.use_deterministic_algorithms(True)</code> on the PyTorch side, and call <code>tf.config.experimental.enable_op_determinism()</code> on the TensorFlow side. Default classroom and production runs leave these unset, accepting last-decimal nondeterminism in exchange for speed; small accuracy regression possible when the flags are on.
Reported numbers	Where the script states an accuracy or runtime, the source is either the cited paper (for production-scale results) or the companion notebook with the seed above (for classroom-scale results).

Appendix F

Solutions and Guidance for Exercises

This appendix provides worked solutions for analytical end-of-chapter exercises and guidance for coding exercises. Coding exercises (those that ask the reader to modify a notebook or measure a numerical quantity) are not treated as fixed numerical answers; their outputs depend on hardware, seeds, calibration choices, and notebook versions, and should be reported from the corresponding companion notebook listed in the *Execution Map*. For exercises that mix analytical work with a small numerical check, the analytical part is solved and the numerical component is described as a verification task.

Exercises are referenced by their stable label `ex:chN:M`, where N is the chapter number and M is the position within that chapter's exercise list. Each solution opens with a back-pointer to the exercise statement so the reader can scan the question first.

F.1 Chapter 1: Introduction to Machine Learning and Deep Learning

Exercise 1 (statement: p. 48): Backprop on a 2-layer net. Let $z = w_1x + b_1$, $a = \text{ReLU}(z)$, $\hat{y} = w_2a$, $\ell = (\hat{y} - y)^2$. Reverse-mode chain rule:

$$\begin{aligned}\frac{\partial \ell}{\partial \hat{y}} &= 2(\hat{y} - y), & \frac{\partial \ell}{\partial w_2} &= 2(\hat{y} - y) a, \\ \frac{\partial \ell}{\partial a} &= 2(\hat{y} - y) w_2, & \frac{\partial a}{\partial z} &= \mathbb{I}[z > 0], \\ \frac{\partial \ell}{\partial w_1} &= 2(\hat{y} - y) w_2 \mathbb{I}[z > 0] x.\end{aligned}$$

Plugging in $x = 2$, $y = 1$, $w_1 = w_2 = b_1 = 0.5$: $z = 1.5$, $a = 1.5$, $\hat{y} = 0.75$, $\ell = 0.0625$. Hence $\partial \ell / \partial w_2 = 2(-0.25)(1.5) = -0.75$ and $\partial \ell / \partial w_1 = 2(-0.25)(0.5)(1)(2) = -0.5$. A two-line PyTorch script (`torch.autograd.grad`) returns the same numbers to machine precision; this is the simplest non-trivial sanity check that the chain-rule derivation matches what AD computes.

Exercise 2 (statement: p. 49): MSE vs. MLE. For Gaussian errors $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, the log-likelihood is

$$\log p(y_{1:n} \mid x_{1:n}; \beta) = -\frac{1}{2\sigma^2} \sum_i (y_i - \beta x_i)^2 - \frac{n}{2} \log(2\pi\sigma^2).$$

Maximizing over β (the only β -dependent term) is identical to minimizing $\sum_i (y_i - \beta x_i)^2$, i.e. OLS. For Laplace errors with scale b , $p(\varepsilon) \propto \exp(-|\varepsilon|/b)$, so the log-likelihood is proportional to $-\sum_i |y_i - \beta x_i|$ and the MLE solves a least-absolute-deviations regression (median regression). Squaring penalizes outliers quadratically, which is suboptimal under Laplace tails because a

single large residual dominates the loss; the median estimator is robust precisely because $|\cdot|$ grows linearly.

Exercise 3 (statement: p. 49): Activation choice for a PINN. A ReLU network $\hat{y}(x) = \sum_k w_k \text{ReLU}(a_k x + b_k) + c$ is piecewise-linear: between consecutive kink points $x = -b_k/a_k$ it is affine in x , so $\hat{y}''(x) = 0$ on every open subinterval. At a kink, the second distributional derivative is a Dirac measure, not a classical pointwise value. A strong-form PDE residual such as the Black–Scholes operator

$$\partial_t V + \frac{1}{2} \sigma^2 S^2 V_{SS} + r S V_S - r V = 0$$

must hold pointwise (a.e.) for almost every S ; with a piecewise-linear V , the term V_{SS} vanishes a.e. in the classical sense, and the residual reduces to $\partial_t V + r S V_S - r V$, which has no nontrivial solution that respects the actual boundary conditions of an option-pricing problem. Concrete example: $\hat{y}(x) = \text{ReLU}(x) + \text{ReLU}(-x) = |x|$ has $\hat{y}''(x) = 0$ for every $x \neq 0$ in the classical sense. Smooth activations (tanh, Swish, GELU, softplus) are C^∞ and produce well-defined pointwise second derivatives, which is why the PINN literature recommends them whenever the PDE is of order ≥ 2 .

Exercise 4 (statement: p. 49): Adam vs. AdamW. Write the gradient at step t as $g_t = \nabla_\theta \ell(\theta_t)$. With L_2 regularization *added to the loss*, $\ell^{L_2}(\theta) = \ell(\theta) + \frac{\lambda}{2} \|\theta\|^2$, so the effective gradient becomes $\tilde{g}_t = g_t + \lambda \theta_t$. Adam’s update applied to $\tilde{g}_t$ is

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \tilde{g}_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) \tilde{g}_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

The key observation is that the weight-decay contribution $\lambda \theta_t$ enters m_t and v_t *through the same EWMA averaging as the data gradient*, so the effective shrinkage applied to θ is $\eta \lambda \theta_t$ scaled by $1/(\sqrt{\hat{v}_t} + \varepsilon)$, which differs across coordinates. Parameters with large historical gradients receive small effective decay; parameters with small gradients receive large decay. AdamW decouples the two:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_{t+1} = (1 - \eta \lambda) \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Now the multiplicative shrinkage $(1 - \eta \lambda)$ acts uniformly on all parameters, independent of the adaptive denominator. Numerically the two updates differ by a factor of $1/(\sqrt{\hat{v}_t} + \varepsilon)$ on the decay term; AdamW preserves the textbook intuition “weight decay shrinks weights at the same rate everywhere.”

Exercise 5 (statement: p. 49): RNN forward pass by hand. With $W_h = 0.5 I_2$, $W_x = (1, 0)^\top$, $h_0 = (0, 0)^\top$:

$$\begin{aligned} h_1 &= \tanh((0, 0)^\top + (1, 0)^\top) = (\tanh 1, 0)^\top \approx (0.7616, 0)^\top, \\ h_2 &= \tanh(0.5 h_1 + (0, 0)^\top) = (\tanh(0.3808), 0)^\top \approx (0.3637, 0)^\top, \\ h_3 &= \tanh(0.5 h_2 + (1, 0)^\top) = (\tanh(1.1818), 0)^\top \approx (0.8275, 0)^\top. \end{aligned}$$

Outputs: $\hat{y}_t = W_y h_t$ gives $\hat{y}_1 \approx 0.7616$, $\hat{y}_2 \approx 0.3637$, $\hat{y}_3 \approx 0.8275$ (only the first hidden coordinate is excited, so the second column of W_y is irrelevant).

For the gradient, write $h_t = \tanh(z_t)$ with $z_t = W_h h_{t-1} + W_x x_t$. Then

$$\frac{\partial \hat{y}_3}{\partial x_1} = W_y \frac{\partial h_3}{\partial z_3} W_h \frac{\partial h_2}{\partial z_2} W_h \frac{\partial h_1}{\partial z_1} W_x,$$

where $\partial h_t / \partial z_t = \text{diag}(1 - \tanh^2(z_t))$. Numerically, $\tanh'(1) \approx 0.4200$, $\tanh'(0.3808) \approx 0.8678$, $\tanh'(1.1818) \approx 0.3155$, so

$$\frac{\partial \hat{y}_3}{\partial x_1} \approx 1 \cdot 0.3155 \cdot 0.5 \cdot 0.8678 \cdot 0.5 \cdot 0.4200 \cdot 1 \approx 0.0288.$$

The decay rate is the product of two distinct effects: (i) the spectral radius of W_h (here 0.5), which contributes one factor of W_h per recurrent step ($T - 1$ multiplicative copies in the chain above), and (ii) the saturation of $\tanh'(\cdot) \in (0, 1]$, which contributes a second multiplicative factor per step. Setting (ii) aside, with $\|W_h\|_2 = 0.5 < 1$ the gradient magnitude already decays at least as fast as 0.5^{T-1} , i.e. exponentially in the sequence length. This is the canonical vanishing-gradient pathology of vanilla RNNs (§1.11). LSTM and GRU gates address (i) by allowing the recurrent Jacobian's eigenvalues to stay close to one without making training unstable; effect (ii) is intrinsic to bounded activations and is mitigated by skip connections (and architectural choices that keep activations away from saturation regimes), not by gating.

Exercise 6 (statement: p. 49): Attention by hand. With $q_i = k_i = v_i = x_i$ and $x = (0, 1, 0.5)$, the score matrix $S_{ij} = q_i k_j$ is

$$S = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0.5 \\ 0 & 0.5 & 0.25 \end{pmatrix}.$$

Softmaxing each row and using $e^0 = 1$, $e^{0.25} \approx 1.284$, $e^{0.5} \approx 1.649$, $e^1 \approx 2.718$:

$$\begin{aligned} a_1 &= \frac{1}{3}(1, 1, 1), & o_1 &= \frac{1}{3}(0 + 1 + 0.5) = 0.5, \\ a_2 &\approx (0.186, 0.506, 0.307), & o_2 &\approx 0.186 \cdot 0 + 0.506 \cdot 1 + 0.307 \cdot 0.5 \approx 0.660, \\ a_3 &\approx (0.254, 0.419, 0.327), & o_3 &\approx 0.254 \cdot 0 + 0.419 \cdot 1 + 0.327 \cdot 0.5 \approx 0.583. \end{aligned}$$

Each attention vector a_i is on the simplex (entries non-negative, summing to one), so $o_i = \sum_j a_{ij} v_j$ is a convex combination of the values, lying in $[0, 1]$. Token 2, the largest input, attends most strongly to itself ($a_{22} \approx 0.51$); token 3 also attends most to token 2 because the inner products $q_3 k_2 = 0.5$ exceed the self-score $q_3 k_3 = 0.25$. Token 1 has zero query magnitude, so its row is uniform: with no signal to discriminate on, attention defaults to a uniform average over the values.

Exercise 7 (statement: p. 49): TensorBoard optimizer comparison. Coding exercise. Expected qualitative behavior on a small classification task: SGD with momentum is often slower to reduce training loss but can generalize competitively when the learning-rate schedule is well tuned; Adam often reaches a low training loss fastest, but its validation curve can diverge from the training curve sooner than for SGD or AdamW; AdamW often sits between the two. The actual crossover point is a notebook output and should be reported from the run.

F.2 Chapter 2: Deep Equilibrium Nets

Exercise 1 (statement: p. 76): Closed-form Brock–Mirman. Conjecture $V(K, z) = A \log K + B \log z + C$. The Bellman equation

$$V(K, z) = \max_{K'} \left\{ \log(zK^\alpha - K') + \beta \mathbb{E}_{z'|z} [V(K', z')] \right\}$$

yields the FOC $1/(zK^\alpha - K') = \beta A / K'$, which gives the constant savings rate

$$s^* = \frac{K'}{zK^\alpha} = \frac{\beta A}{1 + \beta A}.$$

Substituting the conjecture back and matching the $\log K$ coefficient produces $A = \alpha + \beta A \alpha$, hence $A = \alpha / (1 - \alpha \beta)$. Plugging this A into s^* :

$$s^* = \frac{\beta \alpha / (1 - \alpha \beta)}{1 + \beta \alpha / (1 - \alpha \beta)} = \frac{\alpha \beta}{1 - \alpha \beta + \alpha \beta} = \alpha \beta.$$

The DEQN parameterizes $s_t = \sigma(\mathcal{N}_\rho(K_t, z_t))$. Once converged, the average sigmoid output across the ergodic set should equal $\alpha \beta$ (typical calibrations $\alpha = 0.36$, $\beta = 0.96$ give $s^* \approx 0.346$). Any systematic deviation indicates either insufficient training or a quadrature / sampling bias.

Exercise 2 (statement: p. 76): Hard vs. soft constraints. With a softplus head on consumption alone, $C_t = \text{softplus}(\mathcal{N}_\rho(K_t, z_t)) > 0$ is guaranteed, but next-period capital is then *defined* as the residual $K_{t+1} = z_t K_t^\alpha - C_t$, which is unconstrained in sign. At random initialization the network output is approximately $\mathcal{N}(0, 1)$, so C_t is approximately $\text{softplus}(0) \approx 0.69$ on average but can be much larger.

Concrete failure: take $K = 1$, $z = 1$, $\alpha = 0.36$, so $zK^\alpha = 1$. If the network produces an output of, say, 5, then $C = \text{softplus}(5) \approx 5.007$ and $K_{t+1} = 1 - 5.007 = -4.007 < 0$. The next iterate $K_{t+1}^{\alpha-1}$ is then complex, and the loss explodes.

The sigmoid-savings parameterization $s_t = \sigma(\mathcal{N}_\rho) \in (0, 1)$ avoids this entirely: $K_{t+1} = s_t z_t K_t^\alpha > 0$ and $C_t = (1 - s_t) z_t K_t^\alpha > 0$ both hold by construction, regardless of the network's raw output. This is the simplest example of the broader principle that *architectural* feasibility encodings dominate *loss-based* ones whenever the constraint can be written as a closed-form algebraic identity.

Exercise 3 (statement: p. 76): Path averaging vs. conditional expectation. On the simulated path, the path-averaged residual is $\bar{r}_T(\theta) = T^{-1} \sum_{t=1}^T r(\theta, x_t)$ with $\{x_t\}$ generated by the model dynamics. Under ergodicity, Birkhoff's theorem gives

$$\bar{r}_T(\theta) \xrightarrow{T \rightarrow \infty} \int r(\theta, x) \mu(dx) = \mathbb{E}_\mu[r(\theta, x)] \quad \text{a.s.},$$

where μ is the stationary distribution. The conditional-expectation residual at a fixed point x_t , $\mathbb{E}[\bar{r}(\theta, x_{t+1}) \mid x_t]$, is a different object: it integrates only over the conditional shock distribution, holding the current state fixed.

Their connection: if one sweeps the conditional residual over $x_t \sim \mu$ and averages, the result coincides with $\mathbb{E}_\mu[r(\theta, x)]$ by the law of iterated expectations. In other words, path averaging *combines* sampling over states (the outer expectation) and the implicit Monte Carlo integration over shocks (one shock per simulated step). Conditional expectation evaluates the inner integral exactly via quadrature but still requires a way to draw states x_t .

Bias-variance trade-off at finite T_{sim} : the path-averaged loss has variance $O(1/T_{\text{sim}})$ from finite-sample noise but no bias. An exact-quadrature residual has zero stochastic noise on the inner integral but its outer-state coverage depends on the sampling scheme; with mini-batch SGD over the ergodic set, the variance comes from the random batch, not the integration. In the chapter the deterministic quadrature is preferred whenever the shock dimension is low ($d \lesssim 5$), and pathwise residuals dominate once d is high enough that explicit quadrature becomes expensive.

Exercise 4 (statement: p. 76): Brock-Mirman with Gauss-Hermite. The Euler equation in Brock-Mirman with $\delta = 1$, log utility, and AR(1) productivity $\ln z' = \varrho \ln z + \sigma_z \varepsilon'$, $\varepsilon' \sim \mathcal{N}(0, 1)$, is

$$\frac{1}{C_t} = \beta \mathbb{E} \left[\frac{\alpha z' K_{t+1}^{\alpha-1}}{C_{t+1}} \mid K_t, z_t \right].$$

Replace the expectation by a Gauss–Hermite rule. After the change of variables $\varepsilon' = \sqrt{2}\xi$ that absorbs the normalization $1/\sqrt{\pi}$, the Q -point GH quadrature is

$$\mathbb{E}[\mathbb{I} h(\varepsilon')] \approx \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q h(\sqrt{2}\xi_q),$$

with classical nodes ξ_q and weights w_q that satisfy $\sum_q w_q = \sqrt{\pi}$. Table F.1 lists the five-point rule used in this exercise. Substituting, the residual at a state (K_t, z_t) becomes

Table F.1: Five-point Gauss–Hermite nodes and weights for the convention $\mathbb{E}[\mathbb{I} h(\varepsilon)] \approx \pi^{-1/2} \sum_q w_q h(\sqrt{2}\xi_q)$. The weights sum to $\sqrt{\pi}$ before the outside normalization.

q	ξ_q	w_q
1	−2.0202	0.0200
2	−0.9586	0.3936
3	0.0000	0.9453
4	+0.9586	0.3936
5	+2.0202	0.0200

$$r(\theta; K_t, z_t) = 1 - \frac{\beta \alpha C_t}{\sqrt{\pi}} \sum_{q=1}^5 w_q \frac{z'_q K_{t+1}^{\alpha-1}}{C_{t+1}^q},$$

where $z'_q = \exp(\varrho \ln z_t + \sigma_z \sqrt{2}\xi_q)$ and C_{t+1}^q is the consumption obtained by feeding (K_{t+1}, z'_q) into the network. Numerically, comparing this 5-point sum with a high-draw Monte Carlo estimate on the same residual is a useful diagnostic: agreement should be close for smooth integrands and small shock variance, but the realized discrepancy is an output of the check rather than a fixed benchmark.

Exercise 5 (statement: p. 76): Monomial rule by hand (Stroud-3 at $d = 4$). Equation (2.18) places nodes at $\mathbf{x}_k^\pm = \pm\sqrt{d}\mathbf{e}_k$ for $k = 1, \dots, d$, with equal weights $1/(2d) = 1/8$. At $d = 4$, the eight nodes are $\pm 2\mathbf{e}_k$ for $k = 1, 2, 3, 4$.

First moment. $\mathbb{E}[\mathbb{I} \varepsilon'_i]_{\text{rule}} = (1/8) \sum_k [(\sqrt{d}\mathbf{e}_k)_i + (-\sqrt{d}\mathbf{e}_k)_i] = 0$ by $\pm$ -cancellation.

Second moment. $(\varepsilon'_i)^2$ is non-zero only at $\pm\sqrt{d}\mathbf{e}_i$, where it equals d . Two such nodes contribute $2d$, weighted by $1/(2d)$, giving exactly 1.

Cross moment. $\varepsilon'_i \varepsilon'_j$ for $i \neq j$ is zero at every node because each node has only one nonzero coordinate. So the rule returns 0, the true value.

Third moment. $(\varepsilon'_i)^3$ at $\pm\sqrt{d}\mathbf{e}_i$ equals $\pm d^{3/2}$; the two values cancel. Returns 0, exact.

Fourth moment. $(\varepsilon'_i)^4$ at $\pm\sqrt{d}\mathbf{e}_i$ equals d^2 , both signs. Two nodes contribute $2d^2$, weighted by $1/(2d)$, gives d . At $d = 4$, the rule returns 4, while the true value $\mathbb{E}[\mathbb{I} \varepsilon_i'^4] = 3$.

Linear bias growth. In general the rule reports d for the fourth moment, so the relative error $(d - 3)/3$ is linear in d : 33% at $d = 4$, 67% at $d = 6$, doubles to 1 (100%) at $d = 9$.

When does this matter? The Euler residual is a smooth function of the next-period shock ε' . Taylor-expanding around the conditional mean, the leading bias term is the Hessian of the residual with respect to ε' , which probes the second moment, exact under Stroud-3. Fourth-moment bias enters only at the next order, scaled by the integrand's fourth derivative. For moderate CRRA curvature and thin-tailed shocks this term can be small relative to classroom residual tolerances, but it is a diagnostic to check rather than a universal bound. The bias becomes material when (i) the integrand has heavy fourth-order content (e.g., very risk-averse preferences or fat-tailed shocks), or (ii) the shock dimension is large enough that the relative error $(d - 3)/3$ exceeds the residual tolerance one targets. This is the threshold at which the monomial rule should be replaced by Stroud-5 ($2d^2 + 1$ nodes) or QMC.

Exercise 6 (statement: p. 76): Loss-kernel selection. *Definitions for reference.* MSE: $\frac{1}{N} \sum_i r_i^2$. MAE: $\frac{1}{N} \sum_i |r_i|$. Huber(δ): quadratic for $|r| \leq \delta$, linear above. Pinball loss at τ : $L_\tau(r) = \max(\tau r, (\tau - 1)r)$, whose minimizer is the τ -quantile of r . CVaR at α : the expected value of r conditional on r exceeding its α -quantile. Log-cosh: $\sum_i \log \cosh(r_i)$, smooth and quadratic near zero, linear in tails.

(a) **Huber loss**: “smooth, quadratic near zero, linear in tails” is the literal definition. MAE is also linear-in-tails but is not differentiable at $r = 0$, so its gradient flips discontinuously and the optimizer stalls; log-cosh is smooth and shares the asymptotic shape with Huber but has no tunable threshold. Huber(δ) gives the cleanest control of where the regime change happens.

(b) **CVaR at $\alpha = 0.99$** . The CVaR loss optimizes the conditional mean above the 99th-percentile threshold, which is exactly what the regulator audits. The pinball loss at $\tau = 0.99$ would only target the 99th-percentile residual itself, not the conditional average above it; if the residuals have a fat right tail, the pinball-trained policy can have arbitrarily large worst-case 1% residuals. CVaR is the right primitive for “worst-case mean” control.

(c) **MAE** (or equivalently pinball at $\tau = 0.5$). By construction MAE’s first-order condition is solved at the median, not the mean: $\partial/\partial r |r| = \text{sign}(r)$, so the gradient contribution is ± 1 per residual, regardless of magnitude. This is exactly what the desideratum asks for, no single tail residual dominates the gradient. Huber would also down-weight tails but still tracks the mean below the threshold; the user explicitly wants median-targeting.

Exercises 7 and 8 (statements: p. 76, p. 77). These are coding exercises (notebook `lecture_03_02_Brock_Mirman_Uncertainty_DEQN.ipynb`); reference outputs and timing curves are in the companion repository. Qualitative anchors: in Ex. 7, the per-epoch wall time of tensor-product Gauss–Hermite (Q^d nodes) should grow exponentially in d while Stroud-3 ($2d$ nodes) grows linearly, with a crossover that is visible already at $d = 4$ –5 on a single GPU; the relative Euler error should track the integration accuracy of each rule, with Stroud-3 inheriting the fourth-moment bias of §2.6.3. In Ex. 8, swapping Swish for tanh typically slows time-to-converge by tens of percent on a smooth problem like Brock–Mirman because tanh saturates faster, but final accuracy is comparable; convergence should still hold under the same hyperparameters.

F.3 Chapter 3: The International Real Business Cycle Model

Exercise 1 (statement: p. 91): Fischer–Burmeister. For the forward direction, suppose $a \geq 0, b \geq 0, ab = 0$. Without loss of generality $b = 0$; then $\Phi(a, 0) = a + 0 - \sqrt{a^2 + 0} = a - |a| = 0$ since $a \geq 0$. By symmetry the same holds when $a = 0$.

For the reverse direction, suppose $\Phi(a, b) = 0$, i.e. $a + b = \sqrt{a^2 + b^2}$. The right-hand side is non-negative, so $a + b \geq 0$. Squaring: $(a + b)^2 = a^2 + b^2 \Rightarrow 2ab = 0 \Rightarrow ab = 0$. Combined with $a + b \geq 0$ and $ab = 0$, the only possibility is one of a, b being zero and the other non-negative, i.e. $a, b \geq 0$ and $ab = 0$. $\square$

The level set $\Phi(a, b) = 0$ is the union of the non-negative a -axis and the non-negative b -axis (an L-shape in the (a, b) plane). The smoothed variant $\Phi_\varepsilon(a, b) = a + b - \sqrt{a^2 + b^2 + \varepsilon^2}$ rounds the corner at the origin: at $(a, b) = (0, 0)$ one finds $\Phi_\varepsilon(0, 0) = -\varepsilon \neq 0$, while far from the origin $\sqrt{a^2 + b^2 + \varepsilon^2} \approx \sqrt{a^2 + b^2}$ and the smoothed level set converges to the unsmoothed L-shape. In a numerical setting the smoothing eliminates the gradient kink at the origin, which is convenient for AD but distorts the strict KKT zero set by an $O(\varepsilon)$ amount.

(c) *Gradient direction.* At an interior point of the open positive quadrant, $\nabla \Phi(a, b) = (1 - a/\sqrt{a^2 + b^2}, 1 - b/\sqrt{a^2 + b^2})$. At $(a, b) = (1, 1)$ this evaluates to $(1 - 1/\sqrt{2}, 1 - 1/\sqrt{2}) \approx (0.293, 0.293)$, both components strictly positive. The raw gradient $\nabla \Phi$ therefore points northeast, away from the L-shaped zero set, so the bare residual Φ on its own is not the right object for SGD to descend on. What does descend toward the zero set is the squared loss Φ^2 : by the chain rule $\nabla(\Phi^2) = 2\Phi \nabla \Phi$, and inside the open positive quadrant $\Phi(a, b) > 0$, so $-\nabla(\Phi^2) = -2\Phi \nabla \Phi$

has both components strictly negative at $(1, 1)$ and points southwest, i.e. back toward the closer feasible axis. This is the operative observation for training: SGD on Φ^2 is what pulls infeasible iterates back to the L; the raw gradient $\nabla\Phi$ on its own would push them away.

(d) *Sign convention.* Replacing Φ by $-\Phi$ leaves the squared loss untouched: $(-\Phi)^2 = \Phi^2$, so the gradient field of the loss and the SGD trajectory are unchanged. In particular, the sign convention $a + b - \sqrt{a^2 + b^2}$ versus $\sqrt{a^2 + b^2} - a - b$ is irrelevant once the residual is squared, and the operative quantity for SGD is $-\nabla(\Phi^2)$ rather than $-\nabla\Phi$. The L-shaped zero set is invariant under sign flip; only the value of Φ at off-zero points changes (it flips sign), and squaring removes that distinction.

Exercise 2 (statement: p. 92): State-space scaling. For an IRBC with N symmetric countries, the state vector contains $(K^j, z^j)_{j=1}^N$ for a total of $2N$ components. The network outputs the next-period capital vector $(k^{j'})_{j=1}^N$, the irreversibility multipliers $(\mu^j)_{j=1}^N$, and the resource-constraint shadow price λ , for $2N + 1$ outputs in total. Country-level consumption is recovered algebraically from λ via the consumption-sharing FOC and is therefore not a separate output. The loss has N Euler residuals, N Fischer–Burmeister residuals from the irreversibility complementarity, and 1 aggregate-resource-constraint residual, $2N + 1$ components total, matching the $2N + 1$ outputs. Each Euler residual is an expectation over a $(N + 1)$ -dimensional shock vector (one country-specific innovation plus one aggregate, as in equation (3.4)).

Tensor-product Gauss–Hermite at $Q = 3$ costs 3^{N+1} evaluations per residual. Setting $3^{N+1} > 10^4$ gives $N + 1 > \log_3(10^4) \approx 8.38$, so $N \geq 8$ already exceeds the threshold, and $N = 9$ overshoots by a factor of nearly 6 (cost $3^{10} = 59\,049$).

The Stroud-3 rule has $2(N + 1)$ nodes per residual. Setting $2(N + 1) < 100$ gives $N \leq 48$, comfortable for any IRBC dimension actually used in practice. At $N = 50$ the monomial cost is 102 nodes; the tensor cost is $3^{51} \approx 2.15 \times 10^{24}$ evaluations. This four-order-of-magnitude gap at $N = 10$ and twenty-order-of-magnitude gap at $N = 50$ is the operational reason DEQNs use Stroud-3 by default once $N \gtrsim 5$.

Exercise 3 (statement: p. 92): Two-phase training. At a randomly initialized network, the policy outputs $k^{j'}$ are not coordinated with the resource constraint. Suppose the network produces $k^{j'}$ values that, summed and combined with country- j consumption, exceed total output: $\sum_j (k^{j'} + c^j + \Gamma^j) > \sum_j Y^j$. The implied state on the next simulated step has $k^{j'} < (1 - \delta)k^j$ for some country (irreversibility violated), or $c^j < 0$ (negative consumption), or both. Concretely, take $N = 2$, $A_{\text{tfp}} = 1$, $\delta = 0.025$, $k^1 = k^2 = 1$, and a random network output $(k^{1'}, k^{2'}) = (0.5, 0.5)$ (instead of the symmetric $(1, 1)$ steady state). Capital has dropped by 50% in one step, so $I^j = k^{j'} - (1 - \delta)k^j = 0.5 - 0.975 = -0.475 < 0$, violating irreversibility. Even before the irreversibility check fires, the implied consumption $c^j = Y^j - I^j - \Gamma^j$ becomes huge, and in the next step the marginal utility $u'(c)$ is essentially zero, so the Euler residual gradient is uninformative.

Phase 1 (uniform sampling on a wide box of states, with Euler residuals computed against *any* feasible policy guess) gives the optimizer signal to bring outputs into the feasible region before any simulation is attempted. Once the policy is in a feasible neighborhood, Phase 2 (simulation-based sampling on the ergodic set) refines accuracy. Without Phase 1, the simulation in Phase 2 starts in regions where the policy is grossly infeasible, the loss explodes, and gradient descent diverges.

Exercise 4 (statement: p. 92): Adjustment-cost partials and Tobin's Q. Write $g^j \equiv k^{j'}/k^j - 1$. Then $\Gamma^j = (\kappa/2) k^j (g^j)^2$. The partials are

$$\begin{aligned}\frac{\partial \Gamma^j}{\partial k^{j'}} &= \frac{\kappa}{2} k^j \cdot 2g^j \cdot \frac{1}{k^j} = \kappa g^j = \kappa \left(\frac{k^{j'}}{k^j} - 1 \right), \\ \frac{\partial \Gamma^j}{\partial k^j} &= \frac{\kappa}{2} (g^j)^2 + \frac{\kappa}{2} k^j \cdot 2g^j \cdot \left(-\frac{k^{j'}}{(k^j)^2} \right) \\ &= \frac{\kappa}{2} (g^j)^2 - \kappa g^j \left(\frac{k^{j'}}{k^j} \right) = \frac{\kappa}{2} [(g^j)^2 - 2(1 + g^j)g^j] \\ &= -\frac{\kappa}{2} [(g^j)^2 + 2g^j] = \frac{\kappa}{2} \left[1 - \left(\frac{k^{j'}}{k^j} \right)^2 \right],\end{aligned}$$

matching equation (3.6).

At the steady state, $k^{j'} = k^j$ so $g^j = 0$. Both partials vanish: $\partial \Gamma^j / \partial k^{j'} = 0$ and $\partial \Gamma^j / \partial k^j = 0$. The adjustment cost itself $\Gamma^j(k^j, k^j) = 0$, and its first-order presence in the resource constraint and the Euler equation drops out. The deterministic steady state of the IRBC with adjustment costs is therefore identical to the frictionless steady state, k_{ss} being pinned down by $\beta(1 - \delta + \zeta A_{tff} k^{\zeta-1}) = 1$.

The expression $\partial \Gamma^j / \partial k^{j'} = \kappa g^j$ is the marginal cost of investing one more unit and is the IRBC's analogue of *Tobin's marginal Q*: large positive g^j (rapid expansion) creates a large investment wedge, raising the effective per-unit cost of capital next period. Higher κ flattens the response of investment to a productivity shock: a large adjustment cost makes the planner spread the response of $k^{j'}$ across several periods rather than absorbing the shock in one big move, slowing convergence to the new steady state. In the limit $\kappa \rightarrow \infty$, capital adjusts only infinitesimally each period and the dynamics become arbitrarily slow.

Exercise 5 (statement: p. 92): Complete-markets risk sharing. The consumption-sharing condition (3.13) reads $c_t^j = (\lambda_t / \tau^j)^{-\gamma_j}$. Therefore

$$\frac{c_t^i}{c_t^j} = \left(\frac{\lambda_t}{\tau^i} \right)^{-\gamma_i} / \left(\frac{\lambda_t}{\tau^j} \right)^{-\gamma_j} = \left(\frac{\tau^i}{\lambda_t} \right)^{\gamma_i} \left(\frac{\lambda_t}{\tau^j} \right)^{\gamma_j} = (\tau^i)^{\gamma_i} (\tau^j)^{-\gamma_j} \lambda_t^{\gamma_j - \gamma_i}.$$

(i) With homogeneous IES $\gamma_i = \gamma_j = \gamma$, the λ_t exponent vanishes and the ratio collapses to a time-invariant constant:

$$\frac{c_t^i}{c_t^j} = \left(\frac{\tau^i}{\tau^j} \right)^{\gamma}.$$

Since $\log(c_t^i / c_t^j)$ is constant, $\Delta \log c_t^i = \Delta \log c_t^j$, the perfect risk-sharing prediction of [Backus et al. \(1992\)](#): cross-country consumption growth is perfectly correlated (correlation = 1).

(ii) With heterogeneous IES $\gamma_i \neq \gamma_j$, the exponent $\gamma_j - \gamma_i$ is non-zero and λ_t enters the ratio. As shocks move the planner's shadow price, the consumption ratio fluctuates: low-IES countries' consumption is less sensitive to λ_t than high-IES countries'. But the log growth rate is still

$$\Delta \log c_t^j = -\gamma_j \Delta \log \lambda_t.$$

Thus, for positive γ_i, γ_j , any pair of country consumption-growth rates is a positive scalar multiple of the same aggregate shock $\Delta \log \lambda_t$. The correlation remains one; heterogeneous IES changes relative consumption-growth volatility, not the correlation, in this complete-markets planner allocation.

(iii) The empirical consumption-correlation puzzle is that real-world cross-country consumption growth correlations sit well below the near-perfect correlations implied by the complete-markets benchmark. Heterogeneous IES alone does not break the common-shadow-price structure; closing the gap with the data requires incomplete markets, wedges, nontraded goods, preference nonseparabilities, or other frictions that break full insurance.

Exercises 6 and 7 (statements: p. 92, p. 92). These are notebook exercises (`lecture_05_05_IRBC_Exercise.ipynb`); reference solutions are in the notebook itself, behind the “attempt first” dividers. Qualitative anchors: in Ex. 6, the closed-form steady state $k_{ss} = [(1/\beta - 1 + \delta)/(\zeta A_{\text{tfp}})]^{1/(\zeta-1)}$ falls in all three scenarios — a higher δ raises the required net return $1/\beta - 1 + \delta$, a lower β raises $1/\beta$, and a lower ζ shrinks the multiplicative term while (since $\zeta - 1 < 0$) steepening diminishing returns — so k_{ss} is decreasing in δ , increasing in β , and increasing in ζ around the baseline, with $c_{ss} = A_{\text{tfp}} k_{ss}^\zeta - \delta k_{ss}$ following by substitution. In Ex. 7, inverse-loss weighting $w_i \propto 1/\ell_i$ equalizes the per-component contributions $w_i \ell_i$, so the smallest-magnitude residuals (here the Fischer–Burmeister terms) receive the largest weight; the speed-up is largest when a component is small because it is *hard to fit*, and the scheme can hurt when a component is small because it is *already satisfied by construction* (e.g., a hard-coded resource constraint), in which case up-weighting it merely amplifies noise.

F.4 Chapter 4: Neural Architecture Search and Loss Normalization

Exercise 1 (statement: p. 106): Random vs. grid. With two hyperparameters and only one “important” axis, a 3×3 grid uses 9 candidates but only 3 distinct values along the important axis. If the near-optimal interval has length fraction p and its location relative to the grid is unknown, the grid hit probability is approximately $\min\{3p, 1\}$ when p is small. Random search at 9 evaluations samples 9 independent values along the important axis, so its hit probability is

$$1 - (1 - p)^9.$$

For $p = 0.05$, the grid probability is approximately 0.15, while random search gives $1 - 0.95^9 \approx 0.37$.

The general principle: with n evaluations and only $r \ll d$ important axes, random search effectively gives n independent draws on those r axes (since the irrelevant axes don’t matter), while grid search wastes most of its budget on the irrelevant axes’ marginals. This is the projection argument of [Bergstra and Bengio \(2012\)](#): when the loss landscape is anisotropic, random search dominates grid search.

Exercise 2 (statement: p. 106): Bayesian optimization toy problem. This is a coding exercise. A grid with step size 0.01 over $[-1, 2]$ has 301 points, so the qualitative benchmark is that BO should require far fewer objective evaluations when the GP posterior and acquisition function identify the promising region early. The exact evaluation count is a notebook output and depends on the initial design, acquisition optimizer, random seed, and stopping rule.

Exercise 3 (statement: p. 106): Hyperband budget allocation. Hyperband with $R = 81$, $\eta = 3$ runs a ladder of brackets indexed by $s = s_{\max}, s_{\max} - 1, \dots, 0$, where $s_{\max} = \lfloor \log_\eta R \rfloor = 4$. Each bracket starts with $n_s = \lceil (s_{\max} + 1) \eta^s / (s + 1) \rceil$ candidates trained for $r_s = R / \eta^s$ resource each, then runs Successive Halving with reduction factor η . Table F.2 works out the resulting schedule.

Within each bracket, Successive Halving reduces candidates by η at each rung and increases the resource per surviving candidate by η . Therefore the total cost must sum all rungs, not only the first rung. With the floor/ceil schedule above, total Hyperband budget is

$$405 + 363 + 351 + 378 + 405 = 1902$$

resource units, just below the loose worst-case bound $(s_{\max} + 1)^2 R = 25 \cdot 81 = 2025$.

A naive “train all $n_0 = 27$ candidates to full $R = 81$ ” costs $27 \cdot 81 = 2187$ resource units. Hyperband is only moderately cheaper in total resource here, but it screens a much larger initial

Table F.2: Hyperband schedule for maximum resource $R = 81$ and reduction factor $\eta = 3$. Each row reports the successive-halving rungs inside one bracket and the total resource consumed by that bracket.

s	SHA rungs ($n \times r$)	Total
4	$81 \times 1 \rightarrow 27 \times 3 \rightarrow 9 \times 9 \rightarrow 3 \times 27 \rightarrow 1 \times 81$	405
3	$34 \times 3 \rightarrow 11 \times 9 \rightarrow 3 \times 27 \rightarrow 1 \times 81$	363
2	$15 \times 9 \rightarrow 5 \times 27 \rightarrow 1 \times 81$	351
1	$8 \times 27 \rightarrow 2 \times 81$	378
0	5×81	405

pool: across the five brackets, the total number of first-rung candidates is $81 + 34 + 15 + 8 + 5 = 143$, vs. 27 for the naive scheme. Its advantage is adaptive allocation: many more candidates are sampled, but only a small subset receives large training budgets.

Exercise 4 (statement: p. 106): Loss balancing. Let $\ell_i^{(t)}$ have magnitude $L_i \in \{10^0, 10^{-2}, 10^{-4}\}$ and per-component gradient norm $\|\nabla \ell_i\|$. If we crudely model gradient norm as scaling linearly with loss magnitude (true for, e.g., quadratic losses far from the optimum), then $\|\nabla \ell_i\| \propto L_i$. Equalizing gradient contributions $\lambda_i \|\nabla \ell_i\|$ requires $\lambda_i \propto 1/L_i$, i.e. $(\lambda_1, \lambda_2, \lambda_3) \propto (1, 10^2, 10^4)$, which after normalization becomes

$$(\lambda_1, \lambda_2, \lambda_3) = \frac{1}{1 + 10^2 + 10^4} (1, 10^2, 10^4) \approx (10^{-4}, 10^{-2}, 1).$$

The scheme breaks down when the gradients are correlated: $\langle \nabla \ell_i, \nabla \ell_j \rangle \neq 0$ means that scaling up λ_3 to “boost” ℓ_3 also moves θ along the $\nabla \ell_1$ direction, changing ℓ_1 . The “equal contribution” targeted by the fixed weights is no longer a fixed point: each parameter update changes the local gradient geometry, and weights tuned at one iteration become wrong at the next. Adaptive schemes (ReLoBRaLo, GradNorm) re-tune the λ_i at every step, recovering the equalization in a way that fixed weights cannot.

Exercise 5 (statement: p. 106): Pareto frontier geometry. (i) Differentiate $\mathcal{L}(\theta; \lambda) = \lambda(\theta - a)^2 + (1 - \lambda)(\theta - b)^2$ in θ and set to zero: $2\lambda(\theta - a) + 2(1 - \lambda)(\theta - b) = 0$, hence

$$\theta^*(\lambda) = \lambda a + (1 - \lambda)b.$$

(ii) Substituting back:

$$\ell_1^*(\lambda) = (\theta^* - a)^2 = (1 - \lambda)^2(b - a)^2, \quad \ell_2^*(\lambda) = (\theta^* - b)^2 = \lambda^2(b - a)^2.$$

(iii) Take square roots: $\sqrt{\ell_1^*} = (1 - \lambda)(b - a)$ and $\sqrt{\ell_2^*} = \lambda(b - a)$. Adding:

$$\sqrt{\ell_1^*} + \sqrt{\ell_2^*} = (b - a),$$

independent of λ . This is a convex curve in (ℓ_1, ℓ_2) space (a quarter of an astroid-like arc) running from $(0, (b - a)^2)$ at $\lambda = 1$ to $((b - a)^2, 0)$ at $\lambda = 0$.

(iv) At $\lambda = 1/2$, $\theta^* = (a + b)/2$, the midpoint, and $\ell_1^* = \ell_2^* = (b - a)^2/4$. This sits on the symmetric axis of the front.

(v) In the one-dimensional toy problem the trade-off is completely described by the curve above. In a neural network, however, θ is high-dimensional and the descent direction for fixed scalar weight λ is

$$-[\lambda \nabla \ell_1(\theta) + (1 - \lambda) \nabla \ell_2(\theta)].$$

If $\langle \nabla \ell_1, \nabla \ell_2 \rangle > 0$, reducing one component tends to reduce the other; if the inner product is negative, progress on one component can increase the other. Because this geometry changes

along the training path, a fixed scalar weight can balance progress at one iterate and become badly unbalanced later. ReLoBRaLo responds to this by increasing the weight of losses whose *relative loss progress* has lagged behind. GradNorm is the related method that targets gradient magnitudes directly by trying to balance $\|w_k \nabla \ell_k\|$ across components.

Exercise 6 (statement: p. 106): ReLoBRaLo vs. GradNorm. This is a coding exercise. Qualitatively: GradNorm requires one extra backward pass per component per step (to compute $\|\nabla \ell_k\|$), so wall-clock per epoch is roughly $K \times$ slower than ReLoBRaLo for K components. In return, GradNorm achieves tighter gradient balance, which matters when component magnitudes are not a faithful proxy for gradient magnitudes (e.g., when the loss landscape is anisotropic). For the standard PINN losses studied in this script ($K = 2$ or 3 components, typically scaled by physical units), ReLoBRaLo’s loss-magnitude proxy is usually good enough and the extra cost of GradNorm is not warranted; for losses with strongly heterogeneous Hessians (e.g., HJB residuals coupled with KFE residuals where the two operators have different stiffness), GradNorm’s direct gradient measurement can give a meaningful speedup.

Exercise 7 (statement: p. 106): HPO vs. full NAS decision. *Sketch.* The four cells are roughly:

- *(a, i) RTX 3060 + fixed-topology MLP search: Random Search with Successive Halving.* Search space is small (a few thousand candidate combinations), the GPU is too small for graph-level NAS, and SH amortizes the budget across many candidates by killing weak ones early. Bayesian Optimization helps marginally but adds GP-fitting overhead that is not worth it on a single GPU.
- *(a, ii) RTX 3060 + graph-level NAS: Random Search.* Full DARTS-style NAS would not fit in 12 GB of VRAM (the supernet concept-search needs several model copies in memory simultaneously); a small fixed pool of architectures evaluated at low resource is the only feasible option.
- *(b, i) A100 + fixed-topology MLP search: Bayesian Optimization.* The A100’s compute headroom makes the per-step BO overhead negligible, and the search space’s smooth landscape (continuous learning rate, ordinal depth/width) is exactly where GP surrogates dominate Random Search.
- *(b, ii) A100 + graph-level NAS: Full graph-level NAS (DARTS or evolutionary).* The A100’s memory and compute support the supernet training that DARTS requires; the search space has too many discrete connectivity choices for any HPO method to cover.

The general rule: budget grows $\rightarrow$ smarter methods become affordable; search-space size grows $\rightarrow$ the marginal benefit of smarter methods grows; per-method overhead per evaluation needs to fit inside one GPU step or it eats into the budget itself.

F.5 Chapter 5: Overlapping Generations Models with DEQNs

Exercise 1 (statement: p. 119): OLG market clearing for $A = 3$. With three cohorts (young $h = 1$, middle $h = 2$, old $h = 3$), the budget constraints are

$$\begin{aligned} c_t^1 &= w_t \ell^1 - k_{t+1}^2, \\ c_t^2 &= w_t \ell^2 + R_t k_t^2 - k_{t+1}^3, \\ c_t^3 &= w_t \ell^3 + R_t k_t^3, \end{aligned}$$

where w_t, R_t are equilibrium prices and ℓ^h are exogenous lifecycle labor endowments (the old cohort consumes its capital). Two Euler equations determine the savings of the young and middle cohorts:

$$\begin{aligned} u'(c_t^1) &= \beta \mathbb{E}_t[u'(c_{t+1}^2) R_{t+1}], \\ u'(c_t^2) &= \beta \mathbb{E}_t[u'(c_{t+1}^3) R_{t+1}]. \end{aligned}$$

The market-clearing condition closes the system:

$$k_{t+1}^2 + k_{t+1}^3 = K_{t+1},$$

where K_{t+1} is aggregate capital.

Equation count: three budget constraints (used to eliminate consumption), two Euler equations, one capital-market-clearing identity. The budget constraints are bookkeeping, so the network outputs the two cohort savings (k_{t+1}^2, k_{t+1}^3) and is trained on the two Euler residuals; aggregate capital $K_{t+1} = k_{t+1}^2 + k_{t+1}^3$ then determines next-period prices (w_{t+1}, R_{t+1}) algebraically through the firm FOCs. The unknown count (two savings) matches the Euler-residual count (two), and the market-clearing identity is built into the definition of K_{t+1} .

Exercise 2 (statement: p. 120): KKT under FB. For agent h with borrowing constraint $k'^h \geq 0$ and Lagrange multiplier $\lambda^h \geq 0$, the KKT system is

$$k'^h \geq 0, \quad \lambda^h \geq 0, \quad k'^h \cdot \lambda^h = 0.$$

These three conditions are encoded by the single Fischer–Burmeister equation $\Phi(\lambda^h, k'^h) = 0$ with $\Phi(a, b) = a + b - \sqrt{a^2 + b^2}$. The Euler equation

$$u'(c_t^h) - \beta \mathbb{E}_t[u'(c_{t+1}^{h+1}) R_{t+1}] - \lambda_t^h = 0$$

contributes a second residual. Squared and summed:

$$\mathcal{L}^h(\theta) = [\text{Euler residual}]^2 + [\Phi(\lambda^h, k'^h)]^2.$$

At any KKT point, both squared terms vanish exactly: the Euler residual is zero by definition, and $\Phi = 0$ encodes complementarity. This is what “vanishes exactly at the KKT point” means: there is no ε smoothing or penalty parameter, the loss is zero on the equilibrium and strictly positive off it. Compare with a quadratic penalty $[\max(0, -k'^h)]^2 + \mu[\max(0, -\lambda^h)]^2 + (k'^h \lambda^h)^2$: this also vanishes at the KKT point but the multiplier μ has to be tuned, whereas FB has no parameter.

Exercise 3 (statement: p. 120): Hump-shaped lifecycle. A hump-shaped labor-income profile ℓ^h peaks at middle age and declines toward retirement. The lifecycle savings policy k'^h inherits this hump for two reasons. (i) *Consumption smoothing:* agents with high current income $w\ell^h$ relative to lifetime average save heavily to fund retirement years when ℓ^h drops. (ii) *Time-varying borrowing constraint:* young agents have low income, want to borrow against future earnings, are constrained by $k'^h \geq 0$, so they save little; middle-aged agents are unconstrained and save the most; old agents dis-save toward death.

The expected shape: $k'^h \approx 0$ for the very young (constrained), peaks around age 40–50 (middle of working life), declines toward retirement, drops to zero for the oldest cohort that does not save into the next period. In notebook `lecture_08_10_OLG_Benchmark_DEQN_persistent.ipynb`, plotting the trained network’s k'^h against cohort age h should reveal exactly this single-peak shape; the position of the peak depends on the calibration of $(\beta, \delta, A_{\text{tfp}})$ and on the lifecycle labor profile.

Exercise 4 (statement: p. 120): Hard aggregation layer. Coding exercise. The current analytic notebook already clears the capital market by defining K_{t+1} as the sum of predicted cohort savings. The alternative hard-layer variant is useful when the network has a separate aggregate-capital head. Implementation sketch: output a positive scalar $\widehat{K}_{t+1}$ and unnormalized cohort scores $(z^2, \dots, z^A)$; apply softmax along the cohort axis, $s^h = \text{softmax}(z^h)$; rescale to capital:

$$k_{t+1}^h = \widehat{K}_{t+1} \cdot s^h, \quad \sum_{h=2}^A k_{t+1}^h = \widehat{K}_{t+1} \text{ by construction.}$$

The market-clearing residual $\sum_h k^h - \widehat{K}_{t+1}$ is identically zero up to floating-point precision. The comparison with the current notebook should therefore focus on Euler residuals and wall-clock time: the hard layer removes one possible inconsistency but also changes the parameterization, so faster convergence is an empirical question rather than a mathematical guarantee. In multi-asset settings each exact clearing condition needs its own accounting layer, which is why the 56-agent benchmark enforces bond-market clearing as an explicit residual instead.

Exercise 5 (statement: p. 120): Bond pricing in equilibrium. Cohort h 's Euler equation for capital is

$$u'(c_t^h) = \beta \mathbb{E}_t[u'(c_{t+1}^{h+1}) R_{t+1}],$$

and for the riskless bond that costs p_t today and pays one unit of consumption next period,

$$u'(c_t^h) p_t = \beta \mathbb{E}_t[u'(c_{t+1}^{h+1})].$$

The second equation gives directly

$$p_t = \frac{\beta \mathbb{E}_t[u'(c_{t+1}^{h+1})]}{u'(c_t^h)}.$$

Identifying the stochastic discount factor $M_{t,t+1} = \beta u'(c_{t+1}^{h+1})/u'(c_t^h)$, the same expression reads $p_t = \mathbb{E}_t[M_{t,t+1}]$.

For the risk-premium decomposition, divide the capital Euler by $u'(c_t^h)$ and use the covariance identity $\mathbb{E}_t[XY] = \mathbb{E}_t[X]\mathbb{E}_t[Y] + \text{Cov}_t(X, Y)$:

$$1 = \mathbb{E}_t[M_{t,t+1} R_{t+1}] = \mathbb{E}_t[M_{t,t+1}] \mathbb{E}_t[R_{t+1}] + \text{Cov}_t(M_{t,t+1}, R_{t+1}).$$

Substituting $p_t = \mathbb{E}_t[M_{t,t+1}]$:

$$\frac{1}{p_t} = \mathbb{E}_t[R_{t+1}] + \frac{\text{Cov}_t(M_{t,t+1}, R_{t+1})}{p_t},$$

which after rearrangement gives the textbook risk-premium decomposition: the gap between expected gross capital return and the riskless rate equals minus the SDF–return covariance, scaled by $1/p_t$.

When the collateral constraint binds. If the collateral constraint is active, with non-negative KKT multiplier μ_t^h entering the bond FOC with coefficient κ (the same κ that controls the constraint $k^h + \kappa b^h \geq 0$), the bond Euler equation becomes

$$u'(c_t^h) p_t = \beta \mathbb{E}_t[u'(c_{t+1}^{h+1})] + \kappa \mu_t^h,$$

so the equilibrium bond price is

$$p_t = \frac{\beta \mathbb{E}_t[u'(c_{t+1}^{h+1})] + \kappa \mu_t^h}{u'(c_t^h)}.$$

The unconstrained SDF expression $p_t = \mathbb{E}_t[M_{t,t+1}]$ is recovered when $\mu_t^h = 0$. The multiplier wedge raises p_t , equivalently lowers the implicit safe rate that $1/p_t$ tracks, because the constrained agent values one extra unit of bond consumption tomorrow more than the unconstrained agent. In the 56-agent benchmark, the cohorts most likely to bind are the youngest (lowest income, highest desire to borrow against future earnings), so any wedge $\kappa \mu_t^h$ from the binding-cohort population enters the cross-sectional pricing equation.

Why no bond residual in 6-agent OLG? In the analytic 6-agent calibration there is only one asset (capital). Bonds are absent, so no separate market-clearing residual is needed. The single-asset Euler equation pins down the implicit safe rate via $1/p = \mathbb{E}[R]$ minus the appropriate covariance, but no quantity needs to clear because no bond is traded.

Exercises 6 and 7 (statements: p. 120, p. 120). Coding exercises. Both call for 4–5 retraining runs of the 56-agent benchmark and a binding-frequency / steady-state diagnostic. The binding indicator should be based on small slack and a positive multiplier, not on a large complementarity residual: at a well-trained KKT solution the product residual is close to zero both when a constraint binds and when it is slack. Expect borrowing and collateral constraints to bind mostly for young cohorts; as κ rises, the lower bound $b^h \geq -k'^h/\kappa$ becomes tighter, so negative bond positions should shrink and cross-cohort bond dispersion should typically fall. The equilibrium price response is a general-equilibrium object and should be read from the retrained models rather than imposed analytically.

F.6 Chapter 6: Heterogeneous Agents and Young’s Method

Exercise 1 (statement: p. 148): Mean-preserving lottery. Place mass ω at k_n and $1 - \omega$ at k_{n+1} . Mean preservation:

$$\omega k_n + (1 - \omega) k_{n+1} = k'.$$

Solving for ω :

$$\omega = \frac{k_{n+1} - k'}{k_{n+1} - k_n}.$$

This is well-defined for $k_n \leq k' \leq k_{n+1}$ since the denominator is positive and the numerator lies in $[0, k_{n+1} - k_n]$, ensuring $\omega \in [0, 1]$.

Mass conservation: the two probabilities sum to one, $\omega + (1 - \omega) = 1$, so total mass is preserved exactly under this redistribution. Equivalently, the weight ω is the unique linear interpolation weight that makes the discrete two-point distribution have mean k' , which is what defines Young’s redistribution operator on a fixed grid.

Higher-moment matching is impossible with a two-point split unless k' coincides with a grid point: any non-degenerate two-point distribution with mean k' and support $\{k_n, k_{n+1}\}$ has variance $\omega(1 - \omega)(k_{n+1} - k_n)^2 > 0$, while the original (delta) distribution at k' has zero variance. This residual variance is the price of the discretization, and it shrinks as the grid is refined.

Exercise 2 (statement: p. 149): Closed-form bracketing on log-spaced grids. Coding exercise. Algorithm sketch: with grid $k_n = e^{x_0 + n\Delta x} - c$, the bracket index for a query k' is

$$n = \text{floor}\left(\frac{\log(k' + c) - x_0}{\Delta x}\right).$$

This is $\mathcal{O}(1)$ per query (one log + one floor), independent of grid size N . By contrast, `numpy.searchsorted` costs $\mathcal{O}(\log N)$ per query (binary search). The speed difference is hardware- and implementation-dependent, so the coding exercise should report the measured wall-clock ratio on the vectorized batch rather than treating a fixed multiplier as universal.

Exercise 3 (statement: p. 149): Approximate aggregation, scope. The KS log-linear rule is built on the empirical observation that mean capital K_t alone is a sufficient statistic for forecasting K_{t+1} in the standard Aiyagari–Krusell–Smith calibration. This approximate aggregation breaks when the cross-sectional distribution carries information beyond its first moment that materially affects equilibrium prices.

Counterexample 1: multiple assets with switching liquidity. Add a second asset (say a corporate bond) with state-dependent liquidity: in good times agents trade both assets freely; in bad times the bond becomes illiquid. Now the share of wealth held in the bond, plus the bond–capital correlation in the cross-section, both determine prices, and neither can be summarized by mean capital alone.

Counterexample 2: heterogeneous discount factors. If agents differ in β and the cross-sectional distribution of β is dynamic (e.g., new entrants have different β), then mean capital understates the dispersion, and the equilibrium interest rate depends on which subpopulation holds the marginal unit of capital.

Why higher moments do not always rescue. Adding the variance to the forecasting rule helps with smooth perturbations but cannot capture multi-modal distributions, regime-switching, or non-monotone responses to skewness. The fundamental issue is that the master equation requires *full* cross-sectional information whenever prices are non-linear in the distribution; truncating to any finite set of moments is exact only in the linear-pricing case.

Exercise 4 (statement: p. 149): Sequence-space vs. histogram DEQN. Coding exercise. Empirically: with truncation horizon $T = 80$ and the chapter’s reference calibration, the sequence-space residual after training matches the histogram-based DEQN to within a factor of 1.2–1.5 on the same model. The sequence-space variant generalizes *worse* to a much longer test horizon ($T_{\text{test}} \gg 80$) because the truncation error ρ_z^T grows with the gap; the histogram variant does not have this issue because its state is stationary. The trade-off favors sequence-space when the cross-sectional distribution is intrinsically high-dimensional (e.g., multiple assets, multi-cohort wealth), at which point storing T shock realizations is cheaper than discretizing the distribution.

Exercise 5 (statement: p. 149): DeepSets permutation invariance. (i) Let π be a permutation of $\{1, \dots, N\}$. The aggregator’s m -th component is

$$m_t^m(\pi \cdot s) = \sum_{i=1}^N g_\theta^m(s_t^{\pi(i)}) = \sum_{j=1}^N g_\theta^m(s_t^j),$$

where the second equality is just a re-indexing of the sum (since addition is commutative). Therefore $\mathbf{m}_t(\pi \cdot s) = \mathbf{m}_t(s)$, exactly invariant.

(ii) The policy $\pi_\rho(s_t^i; \mathbf{m}_t, a_t)$ is a function of agent i ’s own state s_t^i , the population summary $\mathbf{m}_t$, and the aggregate exogenous state a_t . Under a permutation π , agent $\pi(i)$ ’s individual state is now $s_t^{\pi(i)}$, while $\mathbf{m}_t$ and a_t are unchanged (by the result in (i) for $\mathbf{m}_t$). Therefore the policy of agent $\pi(i)$ in the permuted economy equals the policy of agent $\pi(i)$ in the original economy, i.e. the policy moves with its own agent index but is otherwise unaffected: *equivariance*.

(iii) [Zaheer et al. \(2017\)](#) prove that any continuous permutation-invariant function $f : \mathbb{R}^{d \times N} \rightarrow \mathbb{R}$ on sets of fixed cardinality N can be written as $f(s_1, \dots, s_N) = \rho(\sum_{i=1}^N g(s_i))$ for some continuous functions g, ρ . This is the universal-approximation result for permutation-invariant DeepSets.

Implication for DeepHAM. Since the equilibrium price functional is permutation-invariant in agents (anonymous markets), DeepHAM’s parameterization can in principle approximate any continuous price/policy functional of the cross-sectional distribution to arbitrary accuracy, provided the inner network g_θ has enough capacity and the moment vector $\mathbf{m}_t$ is rich enough. The number of moments M plays the role of the encoder’s bottleneck dimension: empirically,

$M = 1\text{--}3$ suffices for Krusell–Smith-class economies, consistent with the chapter’s report that DeepHAM with one learned moment matches the histogram DEQN.

Exercises 6 and 7 (statements: p. 149, p. 149). Coding exercises. Guidance for interpreting the outputs:

Exercise 6. The relevant statistic is the cross-replication sampling variance conditional on the same aggregate path, not the time-series variance of K_t along that path. As N rises, the Monte Carlo standard error should fall at the usual $N^{-1/2}$ rate for smooth aggregate statistics. Young’s path has zero sampling variance across replications because the histogram update integrates out the lottery exactly. The MC-vs-Young trade-off depends on the target functional: tail mass (e.g., the bottom-10% wealth share) decays much more slowly under MC, so the panel size needed to match Young-equivalent precision is an output of the repeated-panel experiment.

Exercise 7. In the standard KS calibration, the one-moment forecasting rule should already fit $\log K_{t+1}$ extremely well; adding $\log V_t$, with $V_t = \text{Var}_{\mu_t}(k)$, is expected to give only a small incremental gain. This is exactly the empirical observation behind “approximate aggregation”. In calibrations with high cross-sectional dispersion (e.g., wide income range or frequent borrowing-constraint binding), the second-moment improvement can become economically visible and should be reported from the run.

F.7 Chapter 7: Physics-Informed Neural Networks

Exercise 1 (statement: p. 167): Trial-function BC enforcement. Define $\hat{y}(x) = \frac{2x}{\pi} + x(\frac{\pi}{2} - x)\mathcal{N}_\rho(x)$. Evaluate at the boundaries:

$$\hat{y}(0) = 0 + 0 \cdot \frac{\pi}{2} \cdot \mathcal{N}_\rho(0) = 0, \quad \hat{y}(\pi/2) = \frac{2}{\pi} \cdot \frac{\pi}{2} + \frac{\pi}{2} \cdot 0 \cdot \mathcal{N}_\rho(\pi/2) = 1.$$

Both boundary conditions hold for *any* network output $\mathcal{N}_\rho$, so the BCs are encoded in the architecture rather than enforced via the loss.

Why preferable to a soft penalty? A soft penalty $\lambda(\hat{y}(0) - 0)^2 + \lambda(\hat{y}(\pi/2) - 1)^2$ in the loss involves a hyperparameter λ that must be tuned: too small, and the BC violation is large; too large, and the interior PDE residual is starved of optimization budget. The trial-function enforcement is parameter-free, makes the BC residual identically zero, and reduces the loss to the single PDE-interior term $\sup_x |y'' + y|^2$. This separates the two optimization concerns cleanly; any wall-clock gain should be measured in the notebook rather than assumed.

Exercise 2 (statement: p. 168): ReLU pathology. A ReLU network is piecewise-linear: between consecutive kinks $x = -b_k/a_k$ it is affine in x , so $\partial^2 \hat{y}/\partial x^2 = 0$ a.e. At a kink, the second distributional derivative is a Dirac delta supported on a measure-zero set. The strong-form Black–Scholes residual

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + rSV_S - rV = 0$$

must hold pointwise for almost every S . With ReLU, $V_{SS} = 0$ a.e., so the residual reduces to $V_t + rSV_S - rV$, which has no nontrivial solution that respects the option-pricing boundary condition. The PINN cannot decrease its loss below the order of magnitude of the missing V_{SS} term.

Weak-form fix. Multiply both sides by a smooth test function $\varphi(S)$ with compact support on $[S_{\min}, S_{\max}]$ and integrate by parts on the V_{SS} term:

$$\int_{S_{\min}}^{S_{\max}} V_{SS} \varphi dS = - \int V_S \varphi' dS + [V_S \varphi]_{S_{\min}}^{S_{\max}}.$$

The boundary terms vanish for compactly supported φ , and the residual now involves only first-order derivatives of V . ReLU networks have well-defined first derivatives a.e., so the weak-form

PINN can minimize this residual. This is exactly the Galerkin / Deep Galerkin formulation of Sirignano and Spiliopoulos (2018).

Exercise 3 (statement: p. 168): Discrete $\rightarrow$ continuous bridge. Start with

$$V(a) = \max_c [u(c) \Delta t + \beta_{\Delta t} \mathbb{E}V(a')], \quad a' = a - c \Delta t, \quad \beta_{\Delta t} = e^{-\rho \Delta t}.$$

Then $\beta_{\Delta t} = 1 - \rho \Delta t + O(\Delta t^2)$. The same first-order limit follows from the implicit-Euler convention $\beta_{\Delta t} = 1/(1 + \rho \Delta t)$.

Taylor-expand $V(a') = V(a - c \Delta t)$ around a :

$$V(a') = V(a) - V'(a) c \Delta t + \frac{1}{2} V''(a) (c \Delta t)^2 + O(\Delta t^3).$$

Substitute and use $\beta_{\Delta t} = 1 - \rho \Delta t + O(\Delta t^2)$:

$$V(a) = \max_c \left\{ u(c) \Delta t + (1 - \rho \Delta t) [V(a) - V'(a) c \Delta t + O(\Delta t^2)] \right\}.$$

Subtract $V(a)$ from both sides and divide by Δt :

$$0 = \max_c \left\{ u(c) - \rho V(a) - V'(a) c + O(\Delta t) \right\}.$$

Take $\Delta t \rightarrow 0$ and rearrange:

$$\rho V(a) = \max_c [u(c) - V'(a) c].$$

This is the HJB equation for the consumption-savings problem with no asset return (or with r embedded into $a' = a + ra \Delta t - c \Delta t$, in which case the HJB picks up an additional $V'(a) ra$ drift term). The discrete-to-continuous bridge thus shows that the HJB is the formal limit of the discrete Bellman as the time step shrinks, which justifies treating PINNs as the continuous-time analogue of value-function iteration.

Exercises 4, 5, 6 (statements: p. 168, p. 168, p. 168). Coding exercises. The exact numbers below depend on random seeds, batch sizes, and stopping rules; use them as qualitative checks rather than fixed targets:

Exercise 4. At small λ , the BC residual is underweighted and endpoint violations remain visibly large. At very large λ , the endpoints fit well but the interior residual can stagnate because the optimizer spends most of its gradient budget on the boundary term. The elbow is the smallest λ for which further increases mostly improve the boundary metric without improving the interior fit. The hard-BC variant should be the benchmark: it sets the boundary violation to numerical zero by construction and removes the penalty-weight tuning problem.

Exercise 5. Sobol and Latin Hypercube points usually reduce visible sampling gaps relative to uniform random points. On the smooth manufactured Poisson problem the gain may be modest, because the solution has no boundary layer or interior singularity. Adaptive sampling becomes more valuable when residuals are spatially localized; report the actual collocation count, wall time, and test-grid residual rather than relying on a universal percentage saving.

Exercise 6. The expected qualitative result is that the strong-form tanh PINN is the natural baseline for Black-Scholes, because V_{SS} is well-defined by automatic differentiation. A strong-form ReLU network is ill-suited because $V_{SS} = 0$ almost everywhere and undefined at kinks. In a weak formulation, integration by parts moves the second derivative off the network and onto the test function, so a ReLU network can be made mathematically admissible. Any empirical comparison should report held-out pricing errors and residual diagnostics from the implemented notebook rather than quoting architecture-independent iteration counts.

Exercise 7 (statement: p. 168): Operator learning vs. PINN. Eleven independent PINN runs each cost C_{PINN} wall-clock seconds, total $11 C_{\text{PINN}}$. A single operator-learning or parametric-PINN run trained on 11 values of K (or a continuous range, sampled in mini-batches) costs C_{op} . Amortized training wins when $11 C_{\text{PINN}} > C_{\text{op}}$, i.e., $C_{\text{op}}/C_{\text{PINN}} < 11$. The crossover scales linearly in the number of distinct K values: with N strikes, operator learning wins whenever $C_{\text{op}}/C_{\text{PINN}} < N$. This is the cost-amortization argument that motivates DeepONet-style operator learning (Lu et al., 2021a).

F.8 Chapter 8: Heterogeneous Agent Models in Continuous Time

Exercise 1 (statement: p. 185): Itô on GBM. Geometric Brownian motion satisfies $dX_t = \mu X_t dt + \sigma X_t dB_t$. Apply Itô's lemma to $f(x) = \ln x$ with $f'(x) = 1/x$, $f''(x) = -1/x^2$:

$$d(\ln X_t) = f'(X_t) dX_t + \frac{1}{2} f''(X_t) (dX_t)^2 = \frac{1}{X_t} (\mu X_t dt + \sigma X_t dB_t) + \frac{1}{2} \left(-\frac{1}{X_t^2} \right) \sigma^2 X_t^2 dt.$$

Simplifying:

$$d(\ln X_t) = \left(\mu - \frac{1}{2} \sigma^2 \right) dt + \sigma dB_t.$$

Integrating from 0 to t :

$$\ln X_t - \ln X_0 = \left(\mu - \frac{1}{2} \sigma^2 \right) t + \sigma B_t, \quad X_t = X_0 \exp\left[\left(\mu - \frac{1}{2} \sigma^2\right) t + \sigma B_t\right].$$

Volatility drag. Taking expectations: $\mathbb{E}[X_t] = X_0 e^{\mu t}$, but $\mathbb{E}[\ln X_t] = \ln X_0 + \left(\mu - \frac{1}{2} \sigma^2\right) t$. The expected log return $\mu - \sigma^2/2$ is strictly less than the log of the expected return, μ , by the variance correction term. This is the volatility drag. Two illustrative regimes: with zero arithmetic drift ($\mu = 0$), expected log growth is $-\sigma^2/2$, strictly negative; with drift exactly equal to the Itô correction ($\mu = \sigma^2/2$), expected log growth is zero (the drift just offsets the drag). In financial terms, volatility eats into geometric returns; this is why an asset with 20% expected return and 40% volatility delivers a long-run geometric mean of only $\mu - \sigma^2/2 = 12\%$.

Exercise 2 (statement: p. 185): KFE for an OU process. The OU process $dX_t = \eta(\bar{X} - X_t) dt + \sigma dB_t$ has drift $\mu(x) = \eta(\bar{X} - x)$ and diffusion coefficient σ . The KFE in conservation form is

$$\partial_t g(x, t) = -\partial_x [\mu(x) g(x, t)] + \frac{\sigma^2}{2} \partial_{xx} g(x, t) = \partial_x [\eta(x - \bar{X}) g] + \frac{\sigma^2}{2} \partial_{xx} g.$$

Setting $\partial_t g = 0$ for the stationary density $g^*(x)$:

$$\eta \partial_x [(x - \bar{X}) g^*] + \frac{\sigma^2}{2} g^{*''} = 0.$$

Integrate once in x , with the constant of integration set to zero by the no-flux boundary condition at $\pm\infty$:

$$\eta (x - \bar{X}) g^* + \frac{\sigma^2}{2} g^{*'} = 0 \quad \Longleftrightarrow \quad \frac{g^{*'}(x)}{g^*(x)} = -\frac{2\eta}{\sigma^2} (x - \bar{X}).$$

This is a linear ODE for $\ln g^*$; integrating gives $\ln g^* = -\eta(x - \bar{X})^2/\sigma^2 + \text{const}$, i.e.

$$g^*(x) \propto \exp[-\eta(x - \bar{X})^2/\sigma^2].$$

This is a Gaussian density with mean $\bar{X}$ and variance $\sigma^2/(2\eta)$, normalized by $\int g^* dx = 1$:

$$g^*(x) = \sqrt{\frac{\eta}{\pi\sigma^2}} \exp\left[-\frac{\eta(x - \bar{X})^2}{\sigma^2}\right] = \mathcal{N}(\bar{X}, \sigma^2/(2\eta)).$$

The OU's stationary distribution is Gaussian with mean equal to the mean-reversion target, variance equal to the diffusion-to-mean-reversion ratio $\sigma^2/(2\eta)$: faster mean reversion (larger η) shrinks the dispersion; larger diffusion grows it.

Exercise 3 (statement: p. 185): Functional derivative. For the master-equation toy specification $V(a, g) = \int u(c(a, y)) g(y) dy$ where $c(a, y)$ is fixed (does not depend on g), the functional derivative $\delta V/\delta g$ measures the linear response of V to a perturbation in g .

In the ambient vector space of signed measures, a point perturbation gives $\delta V/\delta g(y_0) = \lim_{\varepsilon \rightarrow 0} [V(a, g + \varepsilon \delta_{y_0}) - V(a, g)]/\varepsilon$, where δ_{y_0} is a Dirac mass at y_0 . Substituting,

$$V(a, g + \varepsilon \delta_{y_0}) = \int u(c(a, y)) (g(y) + \varepsilon \delta_{y_0}(y)) dy = V(a, g) + \varepsilon u(c(a, y_0)).$$

Therefore $\delta V/\delta g(y_0) = u(c(a, y_0))$. If we restrict g to the probability simplex, perturbations must preserve total mass; for example, $\eta = \delta_{y_0} - \delta_{y_1}$ gives directional derivative $u(c(a, y_0)) - u(c(a, y_1))$. Equivalently, on the simplex the derivative kernel is identified only up to an additive constant.

Interpretation. The functional derivative at a point y_0 is the value contribution of an infinitesimal mass placed at y_0 . In the toy spec where V is just a population average of utilities, the contribution is the per-agent utility $u(c(a, y_0))$ at that point. In the real master equation, $c(a, y)$ would itself depend on g (because prices depend on g), and the functional derivative would pick up additional indirect terms via $\partial c/\partial g$; this is what makes the master equation a genuinely infinite-dimensional PDE rather than a parametric family of finite PDEs.

Exercise 4 (statement: p. 185): HJB residual. Coding exercise. The right answer is the out-of-sample residual table produced by your run of `lecture_13_08_Aiyagari_Continuous_Time_FD_and_PINN_PyTorch.ipynb`. Report the training budget, collocation batch, random seed, and test grid. The residual should decrease when the collocation budget and network capacity are increased, but the scaling is empirical rather than a universal N^{-p} law because it mixes approximation, optimization, and sampling error.

Exercise 5 (statement: p. 185): Closed Aiyagari system. Combining the four ingredients: *HJB* (from (8.8)):

$$\rho V(a, n) = \max_c \{u(c) + V'(a, n)(wn + ra - c) + \lambda(n)(V(a, \hat{n}) - V(a, n))\}.$$

KFE for the stationary distribution (from (8.6), with $\partial_t g = 0$):

$$0 = -\partial_a [s^*(a, n) g(a, n)] - \lambda(n) g(a, n) + \lambda(\hat{n}) g(a, \hat{n}),$$

where $s^*(a, n) = wn + ra - c^*(a, n)$ is the optimal savings function.

Firm FOCs (Cobb–Douglas):

$$r = \alpha AK^{\alpha-1} L^{1-\alpha} - \delta, \quad w = (1 - \alpha) AK^{\alpha} L^{-\alpha}.$$

Market clearing:

$$K = \sum_n \int_{\underline{a}}^{\infty} a g(a, n) da, \quad L = \sum_n n \int_{\underline{a}}^{\infty} g(a, n) da.$$

The equilibrium objects are $(V(a, n), g(a, n), K, L, r, w)$, with L often pinned down by the stationary income shares and w implied by the firm FOCs once (K, L) is known. In practice one fixes a candidate r , computes the associated firm demand and wage, solves the HJB for V and hence the policy c^*, s^* , plugs into the KFE for g , computes implied $K = \sum_n \int a g(a, n) da$, and compares this capital supply with the capital demand implied by the candidate r . A fixed point in r is the equilibrium. This is the bisection-on- r algorithm of [Achdou et al. \(2022\)](#), and the PINN replaces the inner HJB and KFE solves with neural-network approximation while keeping the outer fixed-point loop on r unless the full equilibrium system is learned jointly.

Why both must be solved consistently at each candidate r . The HJB takes r as an input (price-taking agents); the KFE takes the policy from the HJB. Mis-specifying r during training would feed a mis-specified policy into the KFE and yield an inconsistent K . In the production-scale solver, the fixed-point loop alternates HJB and KFE to convergence *at each* r before updating r ; the PINN can be trained on all four equations jointly when the architecture is rich enough to learn the equilibrium price as an output.

Exercises 6 and 7 (statements: p. 185, p. 185). Coding exercises. Expected diagnostics:

Exercise 6. With a fixed policy, the finite-dimensional KFE is a linear forward equation. If the discretized generator is ergodic, the distance to the stationary distribution should decay approximately exponentially after transient modes die out. Estimate the slope from your run rather than quoting a universal number; the fitted rate is controlled by the slowest nonzero eigenvalue of the KFE generator and depends on income-switching intensities, savings drift, and grid truncation.

Exercise 7. On the one-asset stationary benchmark, finite differences should usually win on absolute wall-clock time and give the cleanest low-dimensional benchmark residuals. A PINN may become more attractive when the same architecture is reused across many nearby parameter values, when warm starts work well, or when the state space is extended beyond what a grid handles comfortably. Report the actual cold-start and warm-start timings from your machine, and treat memory use as hardware- and backend-dependent.

F.9 Chapter 9: Gaussian Processes

Exercise 1 (statement: p. 208): Posterior on three points. The RBF kernel with length scale $\ell = 1$ and signal variance $\sigma_f^2 = 1$ is $k(x, x') = \exp(-(x - x')^2/2)$. With training points $X = (0, 1, 2)$ and targets $y = (0, 0.8, 0.3)$, the kernel matrix is

$$K = \begin{pmatrix} 1 & e^{-1/2} & e^{-2} \\ e^{-1/2} & 1 & e^{-1/2} \\ e^{-2} & e^{-1/2} & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0.6065 & 0.1353 \\ 0.6065 & 1 & 0.6065 \\ 0.1353 & 0.6065 & 1 \end{pmatrix}.$$

Adding observation noise $\sigma_y^2 I = 0.01 I$ to the diagonal of K and solving $(K + \sigma_y^2 I)\mathbf{v} = \mathbf{y}$ with $\mathbf{y} = (0, 0.8, 0.3)^\top$ by Gaussian elimination gives

$$(K + \sigma_y^2 I)^{-1}\mathbf{y} \approx (-0.964, 1.744, -0.621)^\top.$$

At $x^* = 1.5$,

$$k_\star = (k(1.5, 0), k(1.5, 1), k(1.5, 2)) = (e^{-1.125}, e^{-0.125}, e^{-0.125}) \approx (0.3247, 0.8825, 0.8825).$$

Posterior mean:

$$\bar{\mu}(x^*) = k_\star^\top (K + \sigma_y^2 I)^{-1}\mathbf{y} \approx 0.3247 \cdot (-0.964) + 0.8825 \cdot 1.744 + 0.8825 \cdot (-0.621) \approx 0.678.$$

For the variance, solve $(K + \sigma_y^2 I)\mathbf{w} = k_\star$, giving $\mathbf{w} \approx (-0.142, 0.662, 0.495)^\top$, hence

$$\bar{\sigma}^2(x^*) = k(x^*, x^*) - k_\star^\top \mathbf{w} \approx 1 - 0.975 \approx 0.025.$$

The posterior is $\mathcal{N}(0.678, 0.025)$, with standard deviation ≈ 0.158 . Notice the posterior mean “smooths” the two flanking observations $y = 0.8$ and $y = 0.3$ rather than being pulled to either; the 0.158 standard deviation reflects partial information at a halfway point between two data points.

Exercise 2 (statement: p. 208): Marginal likelihood Occam. The log marginal likelihood is

$$\log p(y|X, \ell) = -\frac{1}{2}y^\top K_\ell^{-1}y - \frac{1}{2}\log |K_\ell| - \frac{n}{2}\log(2\pi),$$

where $K_\ell = K_\ell(\ell) + \sigma_y^2 I$ is the kernel matrix at length scale ℓ . The first term penalizes *misfit* (data that don't lie in the GP's predicted manifold get a high $y^\top K_\ell^{-1}y$); the second term penalizes *model complexity* ($\log |K_\ell|$ is large when the kernel can fit anything, small when it is rigid). This is the Bayesian Occam's razor: small ℓ gives a flexible model that fits any training data perfectly (small misfit) but accepts a large complexity penalty; large ℓ enforces smoothness (large misfit if the data are not smooth) but enjoys a small complexity penalty. The optimum balances the two.

For three data points $y = (0, 0.8, 0.3)$ at $x = (0, 1, 2)$, plotting $\log p(y|X, \ell)$ against $\ell \in [0.1, 5]$ typically shows an interior maximum near the data's natural variation scale. At $\ell = 0.1$, the kernel is very local: each point is nearly uncorrelated with its neighbors, so the data fit is easy but the flexible prior receives a complexity penalty. At $\ell = 5$, the kernel forces all three function values to be nearly equal, which fits the data poorly. The peak is the point where these two forces balance.

Exercise 3 (statement: p. 209): Active subspace by hand. For $f(\mathbf{x}) = (x_1 + x_2 + x_3)^2 + 0.01(x_1 - x_2)^2$ on $[-1, 1]^3$, the gradient is

$$\nabla f(\mathbf{x}) = 2(x_1 + x_2 + x_3)(1, 1, 1)^\top + 0.02(x_1 - x_2)(1, -1, 0)^\top.$$

The dominant term is along $(1, 1, 1)$ (with weight roughly $2(x_1 + x_2 + x_3) \cdot \sqrt{3}$), and the perturbation along $(1, -1, 0)$ is two orders of magnitude smaller.

Compute $\hat{C} = \mathbb{E}[\nabla f \nabla f^\top]$ with $x \sim \mathcal{U}[-1, 1]^3$ i.i.d., so $\mathbb{E}[x_i^2] = 1/3$. Let $s = x_1 + x_2 + x_3$; then $\mathbb{E}[s^2] = 1$, $\mathbb{E}[s(x_1 - x_2)] = \mathbb{E}[x_1^2 - x_2^2] = 0$, and $\mathbb{E}[(x_1 - x_2)^2] = 2/3$. Substituting,

$$\hat{C} \approx 4\mathbf{1}\mathbf{1}^\top + \mathcal{O}(10^{-4}),$$

where $\mathbf{1} = (1, 1, 1)^\top$ and the $\mathcal{O}(10^{-4})$ correction comes from the $0.01(x_1 - x_2)$ perturbation. Since $\mathbf{1}\mathbf{1}^\top$ has eigenvalues $\{3, 0, 0\}$ (eigenvector $\mathbf{1}/\sqrt{3}$ for the nonzero one), the leading eigenvalue of $\hat{C}$ is $\lambda_1 \approx 4 \cdot 3 = 12$, with eigenvector $(1, 1, 1)/\sqrt{3}$, the “aggregate direction.” The next eigenvalue is $\sim 10^{-4}$, four orders of magnitude smaller. The active subspace of dimension 1 already captures essentially all of the function's variability.

Exercise 4 (statement: p. 209): Deep vs. linear active subspace. Coding exercise. The linear-AS diagnostic should show two nonzero eigenvalues for the radial-ridge target, with the remaining eigenvalues close to numerical zero. Thus a linear active subspace needs $d_{\text{lin}} = 2$ to represent the two linear features $w_1^\top \xi$ and $w_2^\top \xi$. Deep AS can reach its validation-MSE elbow already at $d_{\text{nl}} = 1$ because the encoder can learn a scalar nonlinear aggregate such as $(w_1 \cdot \xi)^2 + (w_2 \cdot \xi)^2$. Report the held-out MSE curve and eigenvalues from the run rather than quoting universal numbers, since optimization noise and train/validation splits move the exact diagnostics.

Exercise 5 (statement: p. 209): BAL on a 2D function. Coding exercise. Pure-variance acquisition selects points purely by where the GP is most uncertain, ignoring the predicted mean. The resulting design is usually more uniform across the input domain because the GP variance is high wherever data is sparse, regardless of function value. Maximizing pure $\log \sigma^2(\mathbf{x})$ gives exactly the same design as maximizing pure $\sigma^2(\mathbf{x})$, since the logarithm is monotone. Differences arise only once the acquisition is mixed with a mean term, for example $w_{\text{obj}}\mu(\mathbf{x}) + \frac{w_{\text{var}}}{2}\log \sigma^2(\mathbf{x})$: then the design tilts toward regions that are both uncertain and high-valued under the current surrogate. For global surrogate construction, pure variance is the cleaner baseline; for optimization-like goals, the mixed score may be useful.

Exercise 6 (statement: p. 209): Sobol sensitivity with a GP surrogate. Coding exercise. Report the actual errors from the run rather than treating fixed percentages as universal reference outputs. The expected pattern is improvement as N increases: first-order Sobol errors should fall from small to larger sample budgets, while total-effect indices may converge more slowly because they include interaction terms. The cost ratio is the important lesson: once the surrogate is trained, very large Sobol designs can be evaluated at negligible marginal cost relative to repeated true-model solves, which is why surrogate-based sensitivity analysis is standard for expensive simulators such as climate IAMs.

Exercise 7 (statement: p. 209): Prior-driven RBF-GP extrapolation outside the training domain. (i) Coding: on the training interval $[0, 1]$ the GP closely tracks $f(x) = \sin(2\pi x)e^{-x}$ with credible band of width ~ 0.05 .

(ii) Analytical claim: for an RBF kernel $k(x, x') = \sigma_f^2 \exp(-(x - x')^2 / (2\ell^2))$, when $|x - x_i| \gg \ell$ for all training points x_i , the kernel cross-vector $k_\star = (k(x, x_1), \dots, k(x, x_n))^\top \rightarrow (0, \dots, 0)^\top$. Therefore the posterior mean $\bar{\mu}(x) = k_\star^\top (K + \sigma_y^2 I)^{-1} y \rightarrow 0$ (the prior mean). The posterior variance is

$$\bar{\sigma}^2(x) = k(x, x) - k_\star^\top (K + \sigma_y^2 I)^{-1} k_\star \rightarrow \sigma_f^2 - 0 = \sigma_f^2,$$

the prior variance. Hence far from data the GP literally returns the prior $\mathcal{N}(0, \sigma_f^2)$, regardless of the training data.

(iii) Coding verification: at $x = 3$ (with $\ell \approx 0.2$ for the training data), the cross-kernel is $\exp(-(3 - 1)^2 / (2 \cdot 0.04)) = \exp(-50) \approx 10^{-22}$, well below floating-point precision. Posterior mean ≈ 0 , posterior s.d. $\approx \sigma_f \approx 0.5$ (whatever was learned via marginal-likelihood maximization).

(iv) The implication is more subtle than “overconfidence.” Far from the training data the posterior literally reverts to the prior, so the $\pm 2\sigma_f$ band there is exactly what the prior would have produced before any data were observed; it is overconfident only when the prior variance or the learned length scale is itself misleading, for instance when σ_f was calibrated by marginal-likelihood maximization on a training set that does not represent the function’s scale outside the training hull. In particular, the posterior does not know that f continues oscillating outside $[0, 1]$; it just reverts to zero with a band of width σ_f , regardless of whether the true function actually stays close to zero there. Bayesian active learning algorithms that select points by maximum posterior variance can therefore fail to acquire informative samples outside the convex hull when the prior variance is no larger than the within-hull noise scale.

Mitigations: use a Matérn kernel with $\nu = 1/2$ or $\nu = 3/2$ (heavier-tailed than RBF, posterior reverts to prior more slowly), incorporate a polynomial mean function in the GP prior (so extrapolation grows with x instead of decaying to zero), or use a boundary-aware acquisition function that explicitly penalizes exploitation outside the convex hull. In economic applications (e.g., extrapolating an estimated value function to wealth levels outside the training range), the safest practice is to flag any query outside the convex hull of training data and refuse to predict, rather than to trust a prior-driven band that has no data behind it.

F.10 Chapter 10: Deep Surrogate Models and Structural Estimation

Exercise 1 (statement: p. 225): Identification. Let $m(\varrho) = \mathbb{E}_\varrho[h(C, I, Y)]$ be the simulated moment vector implied by the persistence parameter. At the truth $\varrho^\star$, local identification is captured by the Jacobian

$$M(\varrho^\star) = \left. \frac{\partial m(\varrho)}{\partial \varrho} \right|_{\varrho^\star}.$$

A moment is strongly identifying if $|M_k(\varrho^\star)|$ is large; a weakly identifying moment has derivative near zero, in which case small changes in ϱ produce nearly invisible changes in the moment and

the SMM objective becomes flat.

Brock–Mirman example. The output autocorrelation $\text{Corr}(\log Y_t, \log Y_{t-1})$ is typically the strongest local moment for ϱ , because it is a direct echo of the productivity AR(1). The autocorrelation of consumption growth is also informative, but more filtered through equilibrium dynamics. The mean savings rate is nearly flat in the scalar ϱ exercise and is therefore weakly identifying for persistence. The exact ranking should be reported from the finite-difference Jacobian in the notebook, not from a hard-coded number.

Exercise 2 (statement: p. 225): Optimal weighting. The SMM objective is $Q_T(\theta) = \hat{g}(\theta)^\top W \hat{g}(\theta)$, where $\hat{g}(\theta) = m(\theta) - \hat{m}^{\text{data}}$ is the moment-error vector. Under standard regularity (Hansen 1982), the asymptotic variance of $\hat{\theta}_{\text{SMM}}$ is

$$\text{Avar}(\hat{\theta}) = (M^\top W M)^{-1} M^\top W \Omega W M (M^\top W M)^{-1},$$

where $M = \partial m / \partial \theta'$ at θ^* and $\Omega = \text{Var}(\sqrt{T} \hat{g}(\theta^*))$ is the covariance of the data-simulation moment discrepancy.

To minimize $\text{Avar}(\hat{\theta})$ over choices of W , set up the Lagrangian or use the matrix calculus result that the variance is minimized when $W \propto \Omega^{-1}$. Substituting $W = \Omega^{-1}$:

$$\text{Avar}(\hat{\theta})|_{W^*} = (M^\top \Omega^{-1} M)^{-1}.$$

For independent simulated panels of the same length as the data, $\Omega = (1 + 1/S) \Sigma_m$; if simulation noise is negligible, $\Omega \approx \Sigma_m$. Compare to a different weighting $W = \tilde{W}$: $\text{Avar}|_{\tilde{W}} - \text{Avar}|_{W^*}$ is positive semi-definite by the Gauss–Markov theorem applied to the moment system.

Why identity weighting in the first stage? The optimal weight depends on the unknown covariance at the truth. The standard two-stage strategy is: (1) estimate with $W = I$ to obtain a consistent but inefficient $\hat{\theta}^{(1)}$; (2) estimate $\hat{\Omega}$ at or near $\hat{\theta}^{(1)}$; (3) re-estimate with $W = \hat{\Omega}^{-1}$.

Exercise 3 (statement: p. 225): Common random numbers. Coding exercise. Without CRN, the objective $Q_T(\varrho)$ changes both because ϱ changes and because each candidate uses a new shock path. This contaminates the profile with Monte Carlo noise. With CRN, every candidate is evaluated on the same shock path, so differences in $Q_T(\varrho)$ isolate the structural effect of persistence. Quantify the gain by repeating the entire candidate grid across Monte Carlo panels and comparing the cross-panel variance of $Q_T(\varrho)$ at each grid point.

Exercise 4 (statement: p. 225): SMM vs. SBI. SMM solves an outer optimization at deployment: given new data, run a K -step optimization over θ , where each step requires (a) a model simulation at the candidate θ and (b) moment computation. Total cost per inference: K model evaluations.

SBI (e.g., neural posterior estimation) does the work upfront at training time: simulate the model at N values of θ , train a neural density estimator $q(\theta|y)$ on the (θ, y) pairs, and then at deployment evaluate q on the new data with one forward pass. Total cost per inference (after training): one forward pass.

SBI dominates when (i) the model is expensive but a single training run is amortized over many datasets to be analyzed (e.g., a central bank running daily updates); (ii) the model is non-likelihood (no closed-form likelihood, only simulator); (iii) the parameter dimension is moderate ($p \leq 20$) so training data covers parameter space. SMM dominates when (i) the model is cheap (each evaluation a few ms), (ii) only one or a handful of inferences are needed, (iii) classical confidence intervals are required (SBI gives a Bayesian posterior, which differs).

Exercise 5 (statement: p. 225): J -statistic and overidentification. Coding+analytical. In the joint notebook’s over-identified specification, $q = 4$ and $p = 2$, so the degrees of freedom are $q - p = 2$. Under correct specification and with $W = \Omega^{-1}$, the J -statistic at the optimum follows

$$J = T \hat{g}(\hat{\theta})^\top \Omega^{-1} \hat{g}(\hat{\theta}) \xrightarrow{d} \chi_2^2.$$

If the exercise uses $W = \Sigma_m^{-1}$ while simulation noise is non-negligible, adjust the scale under the equal-length independent-simulation approximation or use the Monte Carlo/bootstrap distribution directly.

Empirical distribution: under correct specification, the Monte Carlo distribution of $J(\hat{\theta})$ should be centered near the corresponding χ_2^2 benchmark once the weighting and simulation-noise treatment are consistent. Under a structural break, the model cannot match all four moments simultaneously, so the distribution should shift to the right and the rejection rate should rise above the nominal size. The size of that shift is an output of the experiment and should be reported from the run.

Exercise 6 (statement: p. 225): Bootstrap confidence intervals. Coding exercise. The nonparametric bootstrap should respect time-series dependence; use a moving-block or stationary bootstrap rather than iid resampling of individual dates. The parametric bootstrap draws new shock paths under the estimated parameter vector and re-runs the estimator. Report the actual CI endpoints and coverage from the run. In small samples, bootstrap intervals may differ materially from sandwich intervals because the SMM criterion is nonlinear and the finite-sample distribution of $\hat{\theta}$ need not be close to Gaussian.

Exercise 7 (statement: p. 225): ML vs. SMM efficiency. Coding exercise. If $\log z_t$ is observed and follows

$$\log z_{t+1} = \varrho \log z_t + \sigma_z \varepsilon_{t+1},$$

then the Gaussian AR(1) likelihood gives a direct MLE for ϱ (OLS of $\log z_{t+1}$ on $\log z_t$ when σ_z is unrestricted). This MLE is efficient for the observed-shock likelihood. The SMM estimator based on a finite moment vector reaches the GMM efficiency bound for those moments, but it equals MLE efficiency only if the chosen moments span the score, which the three pedagogical moments generally do not.

Why SMM despite the efficiency loss? In production-scale models (heterogeneous-agent macro, dynamic IO), the likelihood is often unavailable in closed form, and computing it would require integration over high-dimensional latent states. Moments are cheaper, interpretable, and robust to parts of the model that are not central to the research question. The cost is that SMM efficiency depends on the information content of the chosen moments.

F.11 Chapter 11: Climate Economics and Deep Uncertainty Quantification

Exercise 1 (statement: p. 261): ECS sensitivity. Coding exercise. Report the SCC at the central calibration and at each ECS value in the likely and very-likely ranges. The expected qualitative pattern is monotone and often convex in ECS: higher equilibrium climate sensitivity raises temperature damages and therefore the emissions shadow price. The quantitative range is calibration-dependent and should be reported from the notebook rather than fixed in the solution text. The interpretation should connect the dispersion to the climate-science finding of Sherwood et al. (2020) that ECS uncertainty is a major driver of SCC uncertainty.

Exercise 2 (statement: p. 261): Sobol decomposition. For $q(\theta_1, \theta_2, \theta_3) = \theta_1\theta_2 + \theta_3^2$ with $\theta_i \sim \mathcal{U}[0, 1]$ i.i.d.:

$$\begin{aligned}\mathbb{E}[q] &= \mathbb{E}[\theta_1]\mathbb{E}[\theta_2] + \mathbb{E}[\theta_3^2] = \frac{1}{4} + \frac{1}{3} = \frac{7}{12}, \\ \mathbb{E}[q^2] &= \mathbb{E}[\theta_1^2]\mathbb{E}[\theta_2^2] + 2\mathbb{E}[\theta_1\theta_2]\mathbb{E}[\theta_3^2] + \mathbb{E}[\theta_3^4] = \frac{1}{9} + \frac{1}{6} + \frac{1}{5} = \frac{43}{90}, \\ \text{Var}(q) &= \frac{43}{90} - \left(\frac{7}{12}\right)^2 = \frac{11}{80}.\end{aligned}$$

Conditional means:

$$\begin{aligned}\mathbb{E}[q \mid \theta_1] &= \frac{\theta_1}{2} + \frac{1}{3}, & \text{Var}_{\theta_1}(\mathbb{E}[q \mid \theta_1]) &= \frac{1}{48}, \\ \mathbb{E}[q \mid \theta_3] &= \frac{1}{4} + \theta_3^2, & \text{Var}_{\theta_3}(\mathbb{E}[q \mid \theta_3]) &= \text{Var}(\theta_3^2) = \frac{4}{45}.\end{aligned}$$

First-order Sobol indices:

$$S_1 = S_2 = \frac{1/48}{11/80} = \frac{5}{33} \approx 0.152, \quad S_3 = \frac{4/45}{11/80} = \frac{64}{99} \approx 0.646.$$

Sum: $S_1 + S_2 + S_3 = 94/99 \approx 0.949$; the residual $5/99 \approx 0.051$ is the $\theta_1\theta_2$ interaction term.

Total-effect indices (one minus the share explained by all other variables): $S_1^T = S_2^T = 20/99 \approx 0.202$; $S_3^T = 64/99 \approx 0.646$ (no interactions involving θ_3 since it enters only quadratically alone).

A SALib estimate with 10^4 samples typically matches these analytical values to two decimal places.

Exercise 3 (statement: p. 261): ACE benchmark. Coding exercise. Use ACE's closed-form optimal carbon tax as the analytic benchmark, then compute the DEQN-trained DICE SCC at the same calibration and report the percentage discrepancy. A small gap is expected only when units, discounting, damage curvature, and time discretization are aligned; any residual difference should be attributed explicitly to the DICE discretization, smoothing choices, and calibration differences rather than to a fixed percentage target.

Exercise 4 (statement: p. 262): Pareto-improving tax design. Project-level coding exercise. The constrained search is over a low-dimensional parameter vector $\vartheta = (\vartheta_{\text{tax}}, \omega)$, with the cohort welfare constraints $\tilde{U}_t(\vartheta) \geq U_t$ enforced via a penalty or barrier term in the outer optimizer. The Pareto frontier is typically tight: cohorts whose BAU welfare is already close to the unconstrained Ramsey-optimal welfare are easily satisfied, while cohorts that bear the bulk of the transition cost (often the youngest at the time of the reform) bind first; the welfare gap to the unconstrained problem grows with the share of constrained cohorts. Holding ω fixed at the BAU or declining benchmark $\bar{\omega}$ leaves measurable welfare on the table because endogenizing the transfer schedule provides an extra free instrument with which to relax the binding constraints. Numerical answers are calibration-dependent and should be reported from the notebook rather than benchmarked against fixed values; the qualitative interpretation is what matters for the policy discussion.

Exercise 5 (statement: p. 262): Carbon-cycle warm-up. Coding exercise driven by notebook `lecture_16_01_Climate_Exercise.ipynb`. Part (a) avoided-warming and avoided-damages numbers at 2100 under the 50% mitigation rule are notebook outputs and depend on calibration, so they should be reported from the run rather than fixed in the solution text. Part (b) the longest-timescale reservoir is identified by the smallest non-zero eigenvalue of the carbon-cycle transition matrix; for the CDICE three-box calibration this is the lower-ocean compartment, with a multi-century characteristic timescale. Part (c) a quadratic damage function $D(T) = \pi_2 T^2$ has bounded curvature in T and therefore underweights tail risk: marginal damage

grows only linearly, so very high realizations of T are penalized far less than under super-quadratic damages or an explicit tipping-hazard term, and tail-risk assessment requires one of those richer specifications (compare Exercise 8).

Exercise 6 (statement: p. 262): Deterministic CDICE-DEQN reproduction. Coding exercise driven by notebook `lecture_16_02_DICE_DEQN_Library_Port.ipynb`. The verification target is the set $\{T^{\text{AT}}(2100), M^{\text{AT}}(2100), \mu(2100), \text{SCC}(2015), \text{SCC}(2100), \text{SCC}(2300)\}$, each compared against the reference solution at the tolerances stated in the notebook's verification gate. Reference numbers depend on seed, hardware, optimizer schedule, and trajectory sampling, so the solution should anchor on the verification gate rather than on fixed numbers. Typical convergence problems trace back to scaling differences across the eight residuals or to insufficient trajectory coverage near the carbon-stock saturation regime; the residual-balancing methods discussed in Chapter 4 are the first line of defense.

Exercise 7 (statement: p. 262): Stochastic SCC fan chart. Coding exercise driven by notebook `lecture_16_03_Stochastic_DICE_DEQN.ipynb`. As the AR(1) productivity volatility σ_z rises, the right tail of the SCC distribution at 2100 widens disproportionately, because higher productivity raises emissions which feed convexly into damages. Report the quantiles q_{10}, q_{50}, q_{90} of the SCC fan chart at each σ_z rather than the mean alone, since the mean obscures the asymmetry. The qualitative finding aligns with [Cai and Lontzek \(2019\)](#), that productivity and consumption-growth shocks shift the SCC distribution materially and not just its location, supporting the use of distribution-aware reporting in policy work.

Exercise 8 (statement: p. 262): Tipping-point regime-switching damages. Coding exercise. Report the unconditional tipping probability by 2100, the SCC at $t = 0$, and the optimal abatement path under the regime-switching specification and under the smooth-damage baseline. The expected qualitative pattern is that adding an irreversible tipping hazard raises the SCC and brings abatement forward, especially when T_{thresh} is low. The factor by which the SCC rises is an output of the hazard calibration and retrained policy. The policy implication should be framed as a precautionary motive under threshold uncertainty, not as a universal numerical multiplier.

Exercise 9 (statement: p. 262): Real options value of waiting. (i) *Closed forms.* Under the act-now strategy, the planner chooses μ_0 to minimize $\theta\mu_0^2 + \alpha \mathbb{E}[\text{ECS}] \cdot (1 - \mu_0)$. With $\mathbb{E}[\text{ECS}] = (\text{ECS}_L + \text{ECS}_H)/2 \equiv \overline{\text{ECS}}$, the FOC gives $\mu_0^* = \alpha\overline{\text{ECS}}/(2\theta)$. Plugging back, expected cost is

$$C_{\text{now}} = \alpha\overline{\text{ECS}} - \frac{(\alpha\overline{\text{ECS}})^2}{4\theta}.$$

Under the wait strategy, after observing $\widehat{\text{ECS}}$, the posterior mean $\mathbb{E}[\text{ECS} | \widehat{\text{ECS}}]$ has variance shrunk relative to the prior. The planner chooses $\mu_1^*(\widehat{\text{ECS}}) = \alpha \mathbb{E}[\text{ECS} | \widehat{\text{ECS}}]/(2\theta)$. Expected cost (using the law of total variance):

$$C_{\text{wait}} = \alpha\overline{\text{ECS}} - \frac{\alpha^2}{4\theta} \mathbb{E}[\mathbb{E}[\text{ECS} | \widehat{\text{ECS}}]^2] = \alpha\overline{\text{ECS}} - \frac{\alpha^2}{4\theta} (\overline{\text{ECS}}^2 + \text{Var}_{\text{prior}} - \text{Var}_{\text{post}}).$$

(ii) *Value of waiting.* Difference:

$$\text{VoW} = C_{\text{wait}} - C_{\text{now}} = -\frac{\alpha^2}{4\theta} (\text{Var}_{\text{prior}} - \text{Var}_{\text{post}}).$$

As the signal becomes informative ($\sigma_\varepsilon \rightarrow 0$), $\text{Var}_{\text{post}} \rightarrow 0$, so $\text{Var}_{\text{prior}} - \text{Var}_{\text{post}} \rightarrow \text{Var}_{\text{prior}} > 0$, and $\text{VoW} < 0$: waiting is preferred. The reduction in posterior variance is the value of information.

(iii) *With irreversibility.* Add a wedge $\eta\mu_1^2$ to the wait-branch objective, so the planner who waits solves a different second-period problem:

$$\min_{\mu_1} (\theta + \eta)\mu_1^2 + \alpha \mathbb{E}[\text{ECS} \mid \widehat{\text{ECS}}] (1 - \mu_1).$$

Assuming the interior solution remains in $[0, 1]$,

$$\mu_1^{\eta*}(\widehat{\text{ECS}}) = \frac{\alpha \mathbb{E}[\text{ECS} \mid \widehat{\text{ECS}}]}{2(\theta + \eta)}$$

and expected wait cost becomes

$$C_{\text{wait}}^\eta = \alpha \overline{\text{ECS}} - \frac{\alpha^2}{4(\theta + \eta)} (\overline{\text{ECS}}^2 + \text{Var}_{\text{prior}} - \text{Var}_{\text{post}}).$$

The irreversibility wedge reduces the benefit of conditioning abatement on the signal. The value of waiting is now

$$\text{VoW}^\eta = C_{\text{wait}}^\eta - C_{\text{now}} = \frac{\alpha^2}{4} \left[\frac{\overline{\text{ECS}}^2}{\theta} - \frac{\overline{\text{ECS}}^2 + \text{Var}_{\text{prior}} - \text{Var}_{\text{post}}}{\theta + \eta} \right].$$

For sufficiently large η , $\text{VoW}^\eta > 0$: the option value of waiting is dominated by the irreversibility penalty, and act-now is preferred.

The threshold occurs at

$$\eta^* = \frac{\theta(\text{Var}_{\text{prior}} - \text{Var}_{\text{post}})}{\overline{\text{ECS}}^2},$$

where the value of information just balances the irreversibility cost.

Connection to climate policy. This stylized model captures the central tension in climate policy: information arrives over time about ECS, damage functions, and tipping thresholds, but emissions are largely irreversible (atmospheric CO₂ persists for centuries). The chapter’s Bayesian-learning treatment makes the trade-off quantitative: under fast learning and slow climate dynamics, waiting can be optimal; under slow learning and fast tipping risks, an early-action premium emerges. The empirical literature (Pindyck, 2007; Cai and Lontzek, 2019) finds that for realistic climate calibrations, the irreversibility channel typically dominates, supporting near-term carbon tax implementation rather than “wait and see” policies.

F.12 Chapter 12: Synthesis and Outlook

Exercise 1 (statement: p. 273): Method-choice scenario. Sketch:

(a) *4-state monetary-policy DSGE with smooth shocks.* Use **classical perturbation or projection as the baseline**. The state space is small and the shocks are smooth, so a classical method is transparent, fast, and easy to audit. A DEQN becomes attractive only if the model is extended with genuinely global nonlinearities (for example a binding zero lower bound, occasionally binding collateral constraints, or a highly non-quadratic loss). Hybrid: use a GP surrogate over policy-rule parameters after the classical or DEQN solution step if many counterfactuals are needed.

(b) *200-agent OLG with progressive taxation.* Use **DEQN with Young’s-method aggregation** (Chapter 5, 6). 200 cohorts is at the edge where explicit-panel methods (all-in-one DL) and histogram methods compete; histograms are easier when constraints bind frequently across cohorts. Hybrid: post-train a GP surrogate over the Pareto-weight calibration to evaluate optimal-tax-rule sensitivity.

(c) *Exotic option pricing on an irregularly shaped payoff.* Use **PINN or Deep Galerkin methods with careful treatment of the payoff kink** (Chapter 7) for a single-contract

instance, or **DeepONet** if prices are needed for many strikes or contract parameters. The non-smooth object is the terminal payoff, not a generic spatial boundary; for a strong-form Black–Scholes residual this usually calls for smoothing the payoff, using smooth activations away from the kink, or switching to a weak/viscosity-aware formulation. GP surrogates can help as an outer pricing surface over a few parameters, but they are not the primitive PDE solver.

(d) *Climate-IAM where SCC uncertainty is the deliverable.* Use the **full pipeline**: DEQN to solve the deterministic IAM, GP surrogate over deep-uncertain parameters (ECS, damage convexity, tipping thresholds), Sobol/Shapley sensitivity decomposition for attribution, and BAL for sample-efficient uncertainty quantification (Chapters 11, 9). This combines the IAM uncertainty-quantification workflow of Friedl et al. (2023) with the surrogate-based policy-search logic in Kübler et al. (2026).

Exercise 2 (statement: p. 273): When NOT to use deep learning. Open-ended. Sketch of one regime: *bit-exact reproducibility for regulatory audit*. GPU non-determinism in atomic accumulators (described in Appendix E) means that a deep-learning solver typically cannot reproduce the same numbers across hardware platforms; for regulatory work where auditors must replay every step bitwise, a deterministic finite-difference solver on a fixed grid is preferable. Even when deterministic flags are set, BLAS implementations differ across CUDA versions. Classical fixed-grid methods are easier to make bit-reproducible because the operation order can be pinned and the solver path is usually far less sensitive to random initialization and stochastic mini-batches.

Exercise 3 (statement: p. 273): Reproducibility audit. Coding exercise. Expected behavior: re-running notebook `lecture_03_02_Brock_Mirman_Uncertainty_DEQN.ipynb` on the same machine with the same seeds should reproduce the reported diagnostics within the stated tolerance. Bitwise equality of trained network parameters should be expected only when deterministic framework settings, hardware, BLAS/CUDA versions, and floating-point order of operations are all pinned. Re-running on a different GPU, or with deterministic flags off, can produce small deviations in the last few printed digits of the savings-rate diagnostics while remaining well within the residual tolerance of the training.

Exercise 4 (statement: p. 273): Open-ended. Open-ended; expected output is a 1–2 page research sketch. A representative answer combines DEQN (for solving the model) with GP surrogates (for parameter estimation): e.g., a 6-month project that estimates a heterogeneous-agent NK model with deep-learning policy functions, then uses a deep-kernel GP surrogate to do Bayesian posterior inference on the structural parameters. This integrates Chapters 6, 9, and 10.

Exercise 5 (statement: p. 274): Hybrid pipeline DEQN + GP + SMM. Coding exercise. Illustrative outputs to benchmark against, not fixed targets:

- *Step 1 (DEQN training).* Report the actual training time for the parameterized network. Adding (β, ϱ) as inputs increases the network’s effective input dimension from 2 to 4, so convergence can be slower than in the single-calibration case.
- *Step 2 (moment grid).* Report the time for the $20 \times 20 = 400$ simulations, each with $T = 200$ periods, under the trained policy.
- *Step 3 (GP fitting).* Report the fitting time for a four-output GP on 400 training points in $\mathbb{R}^2$.
- *Step 4 (SMM optimization with GP).* Report the number of objective evaluations, optimizer iterations, and total GP-evaluated SMM time.

- *Naive SMM benchmark.* Compare with a version that re-solves or re-trains the DEQN at each candidate only if that benchmark is computationally feasible; otherwise estimate its cost from one representative re-solve.

The expected qualitative result is that the surrogate-based optimization is much cheaper per objective evaluation after the one-time DEQN, simulation-grid, and GP-fitting costs have been paid. The actual speedup and estimation errors are outputs of the run and should be reported rather than assumed.

Exercise 6 (statement: p. 274): DeepONet for a parameterized HJB. (i) *Architecture.* Branch net $\mathbf{b}(\gamma) : \mathbb{R} \rightarrow \mathbb{R}^p$ encodes the parameter γ into a p -dimensional latent representation; trunk net $\mathbf{t}(a) : \mathbb{R} \rightarrow \mathbb{R}^p$ encodes the query point a into the same latent dimension. Predicted value:

$$\hat{V}(a; \gamma) = \langle \mathbf{b}(\gamma), \mathbf{t}(a) \rangle + b_0,$$

where b_0 is a learnable scalar bias. Both nets are typically MLPs with 2–4 layers and width 50–200. The architecture is non-trivial because it imposes the bilinear separable structure $V(a; \gamma) = \sum_{k=1}^p b_k(\gamma) t_k(a)$, which is the universal approximation form for nonlinear operators.

(ii) *UAT for operator learning.* Lu et al. (2021a) show that a continuous nonlinear operator can be approximated uniformly on a compact set of input functions and a compact output domain by a DeepONet with finitely many sensors, a branch network, and a trunk network. In the present notation, for every $\varepsilon > 0$ there are sensor locations, finite-width branch/trunk nets, and latent dimension p such that

$$\sup_{\gamma \in \Gamma} \sup_{a \in \mathcal{A}} |V(a; \gamma) - \hat{V}(a; \gamma)| < \varepsilon$$

on the compact training domain $\Gamma \times \mathcal{A}$, provided the solution operator is continuous there. This generalizes ordinary universal approximation from finite-dimensional functions to operators, but it does not provide extrapolation guarantees outside the compact training domain.

(iii) *Cost comparison.* Independent PINN runs cost $N \cdot C_{\text{PINN}}$. One DeepONet run costs C_{DON} , where $C_{\text{DON}}/C_{\text{PINN}}$ captures the larger network, the parameter-sampling overhead, and the longer training needed to span the parameter range. Operator learning wins exactly when

$$C_{\text{DON}} < N C_{\text{PINN}}, \quad \text{or equivalently} \quad C_{\text{DON}}/C_{\text{PINN}} < N.$$

At $N = 50$, the break-even ratio is therefore 50: a DeepONet can cost up to fifty single-parameter PINN solves and still be cheaper in total. The realized speedup is an empirical output of the implementation, not a universal constant.

(iv) *Limitations. Extrapolation:* DeepONet trained on $\gamma \in [1.5, 5]$ has no guarantees outside this range; in practice the predicted operator $V(a; \gamma)$ for $\gamma > 5$ degrades smoothly but may violate the structural property that V should remain concave. Mitigation: use polynomial-augmented branch networks that extrapolate via known tails (e.g., Bernoulli polynomials), or refuse to predict outside the convex hull of training γ values.

Concavity preservation: the bilinear DeepONet architecture does not automatically preserve concavity in a . A trained network may produce non-concave $\hat{V}(\cdot; \gamma)$ for some γ , which is economically wrong under risk-averse preferences. Mitigation: use an input-concave architecture for the trunk representation, for example the negative of an input-convex neural network where appropriate, or add a concavity penalty $\max(0, \hat{V}''(a; \gamma))$ to the loss. Both increase training cost but restore pressure toward the structural property.

Bibliography

- Achdou, Y., Han, J., Lasry, J.-M., Lions, P.-L., and Moll, B. (2022). Income and wealth distribution in macroeconomics: A continuous-time approach. *The Review of Economic Studies*, 89(1):45–86.
- Adjemian, S., Bastani, H., Juillard, M., Karamé, F., Maih, J., Mihoubi, F., Mutschler, W., Pfeifer, J., Ratto, M., Rion, N., and Villemot, S. (2024). Dynare: Reference manual, version 6. Dynare Working Papers 80, CEPREMAP.
- Aggarwal, C. C., Hinneburg, A., and Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434. Springer.
- Aiyagari, S. R. (1994). Uninsured idiosyncratic risk and aggregate saving. *The Quarterly Journal of Economics*, 109(3):659–684.
- Al-Aradi, A., Correia, A., Naiff, D., Jardim, G., and Saporito, Y. (2018). Solving nonlinear and high-dimensional partial differential equations via deep learning. *arXiv preprint arXiv:1811.08782*.
- Anderson, B., Borgonovo, E., Galeotti, M., and Roson, R. (2014). Uncertainty in Climate Change Modeling: Can Global Sensitivity Analysis Be of Help? *Risk Analysis*, 34(2):271–293.
- Arellano, C. (2008). Default risk and income fluctuations in emerging economies. *The American Economic Review*, 98(3):690–712.
- Auclert, A., Bardóczy, B., Rognlie, M., and Straub, L. (2021). Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models. *Econometrica*, 89(5):2375–2408.
- Auerbach, A. J. and Kotlikoff, L. J. (1987). *Dynamic Fiscal Policy*. Cambridge University Press.
- Azinovic, M., Gaegauf, L., and Scheidegger, S. (2022). DEEP EQUILIBRIUM NETS. *International Economic Review*, 63(4):1471–1525.
- Azinovic-Yang, M. and Žemlička, J. (2024). Intergenerational consequences of rare disasters. *Available at SSRN 4386477*.
- Azinovic-Yang, M. and Žemlička, J. (2025a). Deep learning in the sequence space.
- Azinovic-Yang, M. and Žemlička, J. (2025b). Deep Learning in the Sequence Space – companion code repository. GitHub repository. Includes the pedagogical CPU notebook 01_KrusellSmith_Tutorial_CPU.ipynb.
- Azzimonti, M., Krusell, P., McKay, A., and Mukoyama, T. (2026). Macroeconomics. Open first-year Ph.D. macroeconomics textbook with contributions from additional specialists.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.

- Backus, D. K., Kehoe, P. J., and Kydland, F. E. (1992). International real business cycles. *Journal of Political Economy*, pages 745–775.
- Bansal, R., Kiku, D., and Ochoa, M. (2016). Price of long-run temperature shifts in capital markets. *NBER Working Paper*, (22529). Working paper version, cited per BKO 2016 line.
- Barnett, M. (2023). Climate change and uncertainty: An asset pricing perspective. *Management Science*.
- Barnett, M., Brock, W., and Hansen, L. P. (2020a). Pricing uncertainty induced by climate change. *The Review of Financial Studies*, 33(3):1024–1066.
- Barnett, M., Brock, W., and Hansen, L. P. (2020b). Pricing uncertainty induced by climate change. *Review of Financial Studies*, 33(3):1024–1066.
- Barnett, M., Brock, W., Hansen, L. P., Hu, R., and Huang, J. (2023). A deep learning analysis of climate change, innovation, and uncertainty. *arXiv preprint arXiv:2310.13200*.
- Barron, A. R. (1993). Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945.
- Bartlett, P. L., Long, P. M., Lugosi, G., and Tsigler, A. (2020). Benign overfitting in linear regression. *Proceedings of the National Academy of Sciences*, 117(48):30063–30070.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43.
- Bayer, C. and Luetticke, R. (2020). Solving discrete time heterogeneous agent models with aggregate risk and many idiosyncratic states by perturbation. *Quantitative Economics*, 11(4):1253–1288.
- Belkin, M., Hsu, D., Ma, S., and Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854.
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.
- Bellman, R. (1961). *Adaptive Control Processes: A Guided Tour*. Rand Corporation. Research studies. Princeton University Press.
- Bengio, Y., Simard, P., and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Bertsekas, D. P. (2000). *Dynamic Programming and Optimal Control*. Athena Scientific, 2nd edition.
- Bewley, T. (1986). Stationary monetary equilibrium with a continuum of independently fluctuating consumers. In Hildenbrand, W. and Mas-Colell, A., editors, *Contributions to Mathematical Economics in Honor of Gérard Debreu*, pages 79–102. North-Holland.
- Bilionis, I. and Zabaras, N. (2016). Bayesian uncertainty propagation using Gaussian processes. In *Handbook of Uncertainty Quantification*, pages 1–45. Springer International Publishing, Cham.

- Bischof, R. and Kraus, M. A. (2025). Multi-objective loss balancing for physics-informed deep learning. *Computer Methods in Applied Mechanics and Engineering*, 439:117914.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Information science and statistics. Springer, New York.
- Black, F. and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654.
- Boppart, T., Krusell, P., and Mitman, K. (2018). Exploiting MIT shocks in heterogeneous-agent economies: The impulse response as a numerical derivative. *Journal of Economic Dynamics and Control*, 89:68–92.
- Bottou, L., Curtis, F. E., and Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311.
- Bretscher, L., Fernández-Villaverde, J., and Scheidegger, S. (2022). Ricardian Business Cycles.
- Brock, W. A. and Mirman, L. J. (1972). Optimal economic growth and uncertainty: The discounted case. *Journal of Economic Theory*, 4(3):479–513.
- Brumm, J., Kubler, F., and Scheidegger, S. (2017). Computing equilibria in dynamic stochastic macro-models with heterogeneous agents. In *Advances in Economics and Econometrics: Eleventh World Congress (B. Honoré, A. Pakes, M. Piazzesi, and L. Samuelson, eds.)*, volume 2 of *Econometric Society Monographs*, pages 185–230. Cambridge University Press.
- Brumm, J. and Scheidegger, S. (2017). Using adaptive sparse grids to solve high-dimensional dynamic models. *Econometrica*, 85(5):1575–1612.
- Caffisch, R. E. (1998). Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica*, 7:1–49.
- Cai, Y. and Lontzek, T. S. (2019). The Social Cost of Carbon with Economic and Climate Risks. *Journal of Political Economy*, 127(6):2684–2734.
- Calvin, K., Dasgupta, D., Krinner, G., Mukherji, A., Thorne, P. W., Trisos, C., Romero, J., Aldunce, P., Barrett, K., Blanco, G., Cheung, W. W., Connors, S., Denton, F., Diongue-Niang, A., Dodman, D., Garschagen, M., Geden, O., Hayward, B., Jones, C., Jotzo, F., Krug, T., Lasco, R., Lee, Y.-Y., Masson-Delmotte, V., Meinshausen, M., Mintenbeck, K., Mokssit, A., Otto, F. E., Pathak, M., Pirani, A., Poloczanska, E., Pörtner, H.-O., Revi, A., Roberts, D. C., Roy, J., Ruane, A. C., Skea, J., Shukla, P. R., Slade, R., Slangen, A., Sokona, Y., Sörensson, A. A., Tignor, M., Van Vuuren, D., Wei, Y.-M., Winkler, H., Zhai, P., Zommers, Z., Hourcade, J.-C., Johnson, F. X., Pachauri, S., Simpson, N. P., Singh, C., Thomas, A., Totin, E., Arias, P., Bustamante, M., Elgizouli, I., Flato, G., Howden, M., Méndez-Vallejo, C., Pereira, J. J., Pichs-Madruga, R., Rose, S. K., Saheb, Y., Sánchez Rodríguez, R., Ürgen-Vorsatz, D., Xiao, C., Yassaa, N., Alegría, A., Armour, K., Bednar-Fiedl, B., Blok, K., Cissé, G., Dentener, F., Eriksen, S., Fischer, E., Garner, G., Guivarch, C., Haasnoot, M., Hansen, G., Hauser, M., Hawkins, E., Hermans, T., Kopp, R., Leprince-Ringuet, N., Lewis, J., Ley, D., Ludden, C., Niamir, L., Nicholls, Z., Some, S., Szopa, S., Trewin, B., Van Der Wijst, K.-I., Winter, G., Witting, M., Birt, A., Ha, M., Romero, J., Kim, J., Haïtes, E. F., Jung, Y., Stavins, R., Birt, A., Ha, M., Orendain, D. J. A., Ignon, L., Park, S., Park, Y., Reisinger, A., Cammaramo, D., Fischlin, A., Fuglestad, J. S., Hansen, G., Ludden, C., Masson-Delmotte, V., Matthews, J. R., Mintenbeck, K., Pirani, A., Poloczanska, E., Leprince-Ringuet, N., and Péan, C. (2023). IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (eds.)]. IPCC, Geneva, Switzerland. Technical report, Intergovernmental Panel on Climate Change (IPCC).

- Cardaliaguet, P., Delarue, F., Lasry, J.-M., and Lions, P.-L. (2019). *The Master Equation and the Convergence Problem in Mean Field Games*. Annals of Mathematics Studies. Princeton University Press.
- Carmona, R. and Delarue, F. (2018). *Probabilistic Theory of Mean Field Games with Applications I–II*, volume 83–84 of *Probability Theory and Stochastic Modelling*. Springer.
- Carroll, C. D. (2006). The method of endogenous gridpoints for solving dynamic stochastic optimization problems. *Economics Letters*, 91(3):312–320.
- Chen, H., Didisheim, A., and Scheidegger, S. (2026). Deep surrogates for finance: With an application to option pricing. *Journal of Financial Economics*, 177:104222.
- Chen, H., Gambarotta, G., Scheidegger, S., and Xu, Y. (2025). A dynamic model of private asset allocation. Available at SSRN: <https://ssrn.com/abstract=5162049>.
- Chen, X. (2007). Large sample sieve estimation of semi-nonparametric models. In Heckman, J. J. and Leamer, E. E., editors, *Handbook of Econometrics*, volume 6B, pages 5549–5632. Elsevier.
- Chen, X., Christensen, T. M., and Kankanala, S. (2021). Efficient estimation in NPIV models: A comparison of various neural networks-based estimators. *arXiv preprint*, arXiv:2110.06763.
- Chen, X. and White, H. (1998). Nonparametric adaptive learning with feedback. *Journal of Economic Theory*, 82(1):190–222.
- Chen, X. and White, H. (1999). Improved rates and asymptotic normality for nonparametric neural network estimators. *IEEE Transactions on Information Theory*, 45(2):682–691.
- Chen, Z., Badrinarayanan, V., Lee, C.-Y., and Rabinovich, A. (2018). GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 794–803.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734.
- Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.
- Chung, J., Gulcehre, C., Cho, K., and Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.
- Clevert, D.-A., Unterthiner, T., and Hochreiter, S. (2016). Fast and accurate deep network learning by exponential linear units (elus). In *International Conference on Learning Representations (ICLR)*.
- Constantine, P. G. (2015). *Active Subspaces: Emerging Ideas for Dimension Reduction in Parameter Studies*. SIAM, Philadelphia.
- Cranmer, K., Brehmer, J., and Louppe, G. (2020). The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences (PNAS)*, 117(48):30055–30062.
- Crost, B. and Traeger, C. P. (2013). Optimal climate policy: Uncertainty versus Monte Carlo. *Economics Letters*, 120(3):552–558.
- Crost, B. and Traeger, C. P. (2014). Optimal CO2 mitigation under damage risk valuation. *Nature Climate Change*, 4(7):631–636.

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.
- Dauphin, Y. N., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., and Bengio, Y. (2014). Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems*, volume 27.
- Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press, Cambridge.
- Deisenroth, M. P. and Rasmussen, C. E. (2011). PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning*.
- Deisenroth, M. P., Rasmussen, C. E., and Peters, J. (2009). Gaussian process dynamic programming. *Neurocomputing*, 72(7):1508–1524.
- Diamond, P. A. (1965). National debt in a neoclassical growth model. *American Economic Review*, 55(5):1126–1150.
- Dietz, S. (2024). Chapter 1 - introduction to integrated assessment modeling of climate change. volume 1 of *Handbook of the Economics of Climate Change*, pages 1–51. North-Holland.
- Dietz, S. and Venmans, F. (2019). Cumulative carbon emissions and economic policy: in search of general principles. *Journal of Environmental Economics and Management*, 96:108–129.
- Douenne, T., Hummel, A. J., and Pedroni, M. (2024). Optimal fiscal policy in a climate-economy model with heterogeneous households. CEPR Discussion Paper 19151.
- Duarte, V., Duarte, D., and Silva, D. (2024). Machine learning for continuous-time finance. *Review of Financial Studies*, 37(11):3217–3271.
- Duffie, D. and Singleton, K. J. (1993). Simulated moments estimation of markov models of asset prices. *Econometrica*, 61(4):929–952.
- Duffy, J. and McNelis, P. D. (2001). Approximating and simulating the stochastic growth model: Parameterized expectations, neural networks, and the genetic algorithm. *Journal of Economic Dynamics and Control*, 25(9):1273–1303.
- E, W., Han, J., and Jentzen, A. (2017). Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2):179–211.
- Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.
- Engel, Y., Mannor, S., and Meir, R. (2005). Reinforcement learning with Gaussian processes. In *Proceedings of the 22nd International Conference on Machine Learning*.
- Epstein, L. G. and Zin, S. E. (1989). Substitution, Risk Aversion, and the Temporal Behavior of Consumption and Asset Returns: A Theoretical Framework. *Econometrica*, 57(4):937.
- Falkner, S., Klein, A., and Hutter, F. (2018). BOHB: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning (ICML)*.

- Fernández-Villaverde, J., Nuño, G., and Perla, J. (2024). Taming the curse of dimensionality: Quantitative economics with deep learning. Working Paper 33117, National Bureau of Economic Research.
- Fernández-Villaverde, J., Gillingham, K. T., and Scheidegger, S. (2025). Climate change through the lens of macroeconomic modeling. *Annual Review of Economics*, 17:125–150.
- Fischer, A. (1992). A special Newton-type optimization method. *Optimization*, 24(3–4):269–284.
- Folini, D., Friedl, A., Kübler, F., and Scheidegger, S. (2025). The climate in climate economics. *The Review of Economic Studies*, 92(1):299–338.
- Friedl, A., Kübler, F., Scheidegger, S., and Usui, T. (2023). Deep uncertainty quantification: With an application to integrated assessment models. Working paper.
- Gaegauf, L., Scheidegger, S., and Trojani, F. (2023). A comprehensive machine learning framework for dynamic portfolio choice with transaction costs. *Swiss Finance Institute Research Paper No. 23-114*. Available at SSRN: <https://ssrn.com/abstract=4543794>.
- Gal, Y. and Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*, pages 1050–1059. PMLR.
- Gardner, J. R., Pleiss, G., Weinberger, K. Q., Bindel, D., and Wilson, A. G. (2018). GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Garnett, R. (2023). *Bayesian optimization*. Cambridge University Press.
- Geman, S., Bienenstock, E., and Doursat, R. (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58.
- Gerstner, T. and Griebel, M. (1998). Numerical integration using sparse grids. *Numerical Algorithms*, 18:209–232.
- Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*, volume 53 of *Stochastic Modelling and Applied Probability*. Springer.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of PMLR, pages 249–256.
- Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323.
- Golosov, M., Hassler, J., Krusell, P., and Tsyvinski, A. (2014). Optimal taxes on fossil fuel in general equilibrium. *Econometrica*, 82(1):41–88.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Gopalakrishna, G. (2024). ALIENs and continuous time economies. *Available at SSRN*.
- Gourieroux, C., Monfort, A., and Renault, E. (1993). Indirect inference. *Journal of Applied Econometrics*, 8:S85–S118.
- Gu, S., Kelly, B., and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5):2223–2273.

- Gu, Z., Lauriere, M., Merkel, S., and Payne, J. (2024). Global solutions to master equations for continuous time heterogeneous agent macroeconomic models. *arXiv preprint arXiv:2406.13726*.
- Han, J., Jentzen, A., and E, W. (2018). Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510.
- Han, J., Yang, Y., and E, W. (2024). DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks. *Quantitative Economics*. Forthcoming; preprint arXiv:2112.14377 (first version December 2021).
- Harenberg, D., Marelli, S., Sudret, B., and Winschel, V. (2019). Uncertainty quantification and global sensitivity analysis for economic models. *Quantitative Economics*, 10(1):1–41.
- Hassler, J., Krusell, P., and Smith Jr, A. A. (2016). Environmental macroeconomics. In *Handbook of macroeconomics*, volume 2, pages 1893–2008. Elsevier.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.
- Heathcote, J. and Perri, F. (2002). Financial autarky and international business cycles. *Journal of Monetary Economics*, 49(3):601–627.
- Heathcote, J. and Perri, F. (2013). The international diversification puzzle is not as bad as you think. *Journal of Political Economy*, 121(6):1108–1159.
- Hebb, D. O. (1949). *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York.
- Heiss, F. and Winschel, V. (2008). Likelihood approximation by numerical integration on sparse grids. *Journal of Econometrics*, 144(1):62–80.
- Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*.
- Hensman, J., Fusi, N., and Lawrence, N. D. (2013). Gaussian processes for big data. In *Proceedings of the 29th Conference on Uncertainty in Artificial Intelligence (UAI)*.
- Heydari, A. A., Thompson, C. A., and Mehmood, A. (2019). SoftAdapt: Techniques for adaptive loss weighting of neural networks with multi-part loss functions. *arXiv preprint arXiv:1912.12355*.
- Hochreiter, S. (1991). Untersuchungen zu dynamischen neuronalen Netzen. Master’s thesis, Diploma thesis, Technische Universität München. Advisor: J. Schmidhuber.
- Hochreiter, S., Bengio, Y., Frasconi, P., and Schmidhuber, J. (2001). Gradient flow in recurrent nets: The difficulty of learning long-term dependencies. In Kremer, S. C. and Kolen, J. F., editors, *A Field Guide to Dynamical Recurrent Neural Networks*. IEEE Press.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Holt, T., Igami, M., and Scheidegger, S. (2024). Detecting edgeworth cycles. *The Journal of Law and Economics*, 67(1):67–102.

- Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.
- Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101.
- Huggett, M. (1993). The risk-free rate in heterogeneous-agent incomplete-insurance economies. *Journal of Economic Dynamics and Control*, 17(5-6):953–969.
- Hutchinson, J. M., Lo, A. W., and Poggio, T. (1994). A nonparametric approach to pricing and hedging derivative securities via learning networks. *The Journal of Finance*, 49(3):851–889.
- Hutter, F., Kotthoff, L., and Vanschoren, J., editors (2019). *Automated Machine Learning: Methods, Systems, Challenges*. Springer.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.
- Iooss, B. and Prieur, C. (2019). Shapley effects for sensitivity analysis with correlated inputs: Comparisons with Sobol’ indices, numerical estimation and applications. *International Journal for Uncertainty Quantification*, 9(5).
- Jacot, A., Gabriel, F., and Hongler, C. (2018). Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31.
- Jamieson, K. and Talwalkar, A. (2016). Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Jensen, S. and Traeger, C. P. (2014). Optimal climate change mitigation under long-term growth uncertainty: Stochastic integrated assessment and analytic findings. *European Economic Review*, 69:104–125.
- Judd, K. L. (1998). *Numerical methods in economics*. The MIT press.
- Judd, K. L., Maliar, L., and Maliar, S. (2011). Numerically stable and accurate stochastic simulation approaches for solving dynamic economic models. *Quantitative Economics*, 2(2):173–210.
- Kahou, M. E., Fernández-Villaverde, J., Perla, J., and Sood, A. (2021). Exploiting symmetry in high-dimensional dynamic programming. NBER Working Paper 28981, National Bureau of Economic Research.
- Kaplan, G., Moll, B., and Violante, G. L. (2018). Monetary policy according to HANK. *American Economic Review*, 108(3):697–743.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Karp, L., Peri, A., and Rezai, A. (2024). Selfish incentives for climate policy: Empower the young! *Journal of the Association of Environmental and Resource Economists*, 11(5):1165–1200.
- Kase, H., Melosi, L., and Rottner, M. (2022). Estimating nonlinear heterogeneous agent models with neural networks. CEPR Discussion Paper 17430, Centre for Economic Policy Research.

- Kelly, B. and Xiu, D. (2023). Financial machine learning. *Foundations and Trends in Finance*, 13(3–4):205–363.
- Kelly, D. L. and Kolstad, C. D. (1999). Bayesian learning, growth, and pollution. *Journal of Economic Dynamics and Control*, 23(4):491–518.
- Kelly, D. L. and Tan, Z. (2015). Learning and climate feedbacks: Optimal climate insurance and fat tails. *Journal of Environmental Economics and Management*, 72:98–122.
- Kendall, A., Gal, Y., and Cipolla, R. (2018). Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smolinskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations (ICLR)*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Knutti, R., Rugenstein, M. A., and Hegerl, G. C. (2017). Beyond equilibrium climate sensitivity. *Nature Geoscience*, 10(10):727–736.
- Koenker, R. and Bassett, G. (1978). Regression quantiles. *Econometrica*, 46(1):33–50.
- Kotlikoff, L., Kubler, F., Polbin, A., and Scheidegger, S. (2021). Pareto-improving carbon-risk taxation. *Economic Policy*, 36(107):551–589.
- Krause, A., Singh, A., and Guestrin, C. (2008). Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies. *Journal of Machine Learning Research*, 9:235–284.
- Krogh, A. and Hertz, J. A. (1991). A simple weight decay can improve generalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 4, pages 950–957.
- Krueger, D. and Kubler, F. (2004). Computing equilibrium in OLG models with stochastic production. *Journal of Economic Dynamics and Control*, 28(7):1411–1436.
- Krueger, D. and Kubler, F. (2006). Pareto-improving social security reform when financial markets are incomplete!? *American Economic Review*, 96(3):737–755.
- Krusell, P. and Smith, Jr, A. A. (1998). Income and wealth heterogeneity in the macroeconomy. *Journal of Political Economy*, 106(5):867–896.
- Kübler, F., Scheidegger, S., and Surbek, O. (2026). Using machine learning to compute constrained optimal carbon tax rules. *Journal of Political Economy: Macroeconomics*. forthcoming.
- Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- Lasry, J.-M. and Lions, P.-L. (2007). Mean field games. *Japanese Journal of Mathematics*, 2(1):229–260.
- Leach, A. J. (2007). The climate change learning curve. *Journal of Economic Dynamics and Control*, 31(5):1728–1752.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.

- Lee, B.-S. and Ingram, B. F. (1991). Simulation estimation of time-series models. *Journal of Econometrics*, 47(2-3):197–205.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2018). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*.
- Ljungqvist, L. and Sargent, T. J. (2018). *Recursive Macroeconomic Theory*. MIT Press, Cambridge, MA, 4th edition.
- Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.
- Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. (2021a). Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229.
- Lu, L., Meng, X., Mao, Z., and Karniadakis, G. E. (2021b). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228.
- Maas, A. L., Hannun, A. Y., and Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*.
- MacKay, D. J. C. (1992). Information-based objective functions for active data selection. *Neural Computation*, 4(4):590–604.
- Maliar, L. and Maliar, S. (2014). Chapter 7 - numerical methods for large-scale dynamic economic models. In Schmedders, K. and Judd, K. L., editors, *Handbook of Computational Economics Vol. 3*, volume 3 of *Handbook of Computational Economics*, pages 325 – 477. Elsevier.
- Maliar, L., Maliar, S., and Valli, F. (2010). Solving the incomplete markets model with aggregate uncertainty using the Krusell–Smith algorithm. *Journal of Economic Dynamics and Control*, 34(1):42–49.
- Maliar, L., Maliar, S., and Winant, P. (2021). Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101.
- Maliar, M. (2022). Deep-learning code for solving the Krusell–Smith model (replication). GitHub repository.
- Marcet, A. (1988). Solving non-linear stochastic models by parameterizing expectations. *Carnegie Mellon University Working Paper*.
- Marcet, A. and Marshall, D. A. (1994). Solving nonlinear rational expectations models by parametrized expectations: Convergence to Stationary Solutions. Discussion paper 91, Federal Reserve Bank of Minneapolis.
- Margossian, C. C. (2019). A review of automatic differentiation and its efficient implementation. *WIREs Data Mining and Knowledge Discovery*, 9(4):e1305.

- McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133.
- McFadden, D. (1989). A method of simulated moments for estimation of discrete response models without numerical integration. *Econometrica*, 57(5):995–1026.
- McKay, M. D., Beckman, R. J., and Conover, W. J. (1979). A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245.
- Misra, D. (2019). Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*.
- Montanelli, H. and Du, Q. (2019). New error bounds for deep ReLU networks using sparse grids. *SIAM Journal on Mathematics of Data Science*, 1(1):78–92.
- Nagel, S. (2021). *Machine Learning in Asset Pricing*. Princeton University Press, Princeton, NJ.
- Nair, V. and Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814.
- Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., and Sutskever, I. (2020). Deep double descent: Where bigger models and more data can hurt. In *International Conference on Learning Representations (ICLR)*.
- Niederreiter, H. (1992). *Random Number Generation and Quasi-Monte Carlo Methods*, volume 63 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. SIAM.
- Nordhaus, W. D. (1994). *Managing the global commons: the economics of climate change*. MIT press Cambridge, MA.
- Nordhaus, W. D. (2008). *A Question of Balance: Weighing the Options on Global Warming Policies*. Yale University Press, New Haven, CT.
- Nordhaus, W. D. (2017). Revisiting the social cost of carbon. *Proceedings of the National Academy of Sciences*, 114(7):1518 LP – 1523.
- Nordhaus, W. D. and Yang, Z. (1996). A regional dynamic general-equilibrium model of alternative climate-change strategies. *The American Economic Review*, pages 741–765.
- Norets, A. (2012). Estimation of dynamic discrete choice models using artificial neural network approximations. *Econometric Reviews*, 31(1):84–106.
- Novak, E. and Woźniakowski, H. (2008). *Tractability of Multivariate Problems, Vol. I: Linear Information*. European Mathematical Society.
- Nuño, G., Renner, P., and Scheidegger, S. (2024). Monetary policy with persistent supply shocks. Technical report, CESifo Working Paper Series.
- Owen, A. B. (1995). Randomly permuted (t, m, s) -nets and (t, s) -sequences. *Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing*, pages 299–317.
- Owen, A. B. (2014). Sobol’ indices and Shapley value. *SIAM/ASA Journal on Uncertainty Quantification*, 2(1):245–251.
- Pakes, A. and Pollard, D. (1989). Simulation and the asymptotics of optimization estimators. *Econometrica*, 57(5):1027–1057.

- Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318. PMLR.
- Payne, J., Rebei, A., and Yang, Y. (2025). Deep learning for search and matching models. Technical Report 25-05, Swiss Finance Institute.
- Pichler, P. (2011). Solving the multi-country real business cycle model using a monomial rule galerkin method. *Journal of Economic Dynamics and Control*, 35(2):240–251.
- Pindyck, R. S. (2007). Uncertainty in environmental economics. *Review of Environmental Economics and Policy*, 1(1):45–65.
- Prechelt, L. (1998). Early stopping — but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer.
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707.
- Ramachandran, P., Zoph, B., and Le, Q. V. (2017). Searching for activation functions. *arXiv preprint arXiv:1710.05941*.
- Rasmussen, C. E. and Williams, C. K. I. (2005). *Gaussian Processes for Machine Learning (Adaptive Computation and Machine Learning)*. The MIT Press.
- Real, E., Aggarwal, A., Huang, Y., and Le, Q. V. (2019). Regularized evolution for image classifier architecture search. In *AAAI Conference on Artificial Intelligence*.
- Reiter, M. (2009). Solving heterogeneous-agent models by projection and perturbation. *Journal of Economic Dynamics and Control*, 33(3):649–665.
- Renner, P. and Scheidegger, S. (2018). Machine learning for dynamic incentive problems. Working paper. Available at SSRN: <http://dx.doi.org/10.2139/ssrn.3282487>.
- Robbins, H. and Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407.
- Roe, G. H. and Baker, M. B. (2007). Why is climate sensitivity so unpredictable? *Science*, 318(5850):629–632.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323:533–536.
- Salimans, T. and Kingma, D. P. (2016). Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29.
- Saltelli, A. and D’Hombres, B. (2010). Sensitivity analysis didn’t help. A practitioner’s critique of the Stern review. *Global Environmental Change*, 20(2):298–302.
- Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., and Tarantola, S. (2008). *Global Sensitivity Analysis: The Primer*. Wiley.
- Santurkar, S., Tsipras, D., Ilyas, A., and Madry, A. (2018). How does batch normalization help optimization? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31.

- Sargent, T. J. and Stachurski, J. (2026). Dynamic programming, volumes i and ii. QuantEcon open textbook series.
- Scheidegger, S. and Bilionis, I. (2019). Machine learning for high-dimensional dynamic stochastic economies. *Journal of Computational Science*, 33:68 – 82.
- Scheidegger, S. and Treccani, A. (2018). Pricing american options under high-dimensional models with recursive adaptive sparse expectations. *Journal of Financial Econometrics*.
- Schlag, I., Irie, K., and Schmidhuber, J. (2021). Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR.
- Schmidhuber, J. (1992). Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117.
- Sergeev, A. and Del Balso, M. (2018). Horovod: Fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799*.
- Shapley, L. S. (1953). A value for n-person games. In *Contributions to the Theory of Games*, volume 2, pages 307–317. Princeton University Press.
- Sherwood, S. C., Webb, M. J., Annan, J. D., Armour, K. C., Forster, P. M., Hargreaves, J. C., Hegerl, G., Klein, S. A., Marvel, K. D., Rohling, E. J., Watanabe, M., Andrews, T., Braconnot, P., Bretherton, C. S., Foster, G. L., Hausfather, Z., von der Heydt, A. S., Knutti, R., Mauritsen, T., Norris, J. R., Proistosescu, C., Rugenstein, M., Schmidt, G. A., Tokarska, K. B., and Zelinka, M. D. (2020). An assessment of Earth’s climate sensitivity using multiple lines of evidence. *Reviews of Geophysics*, 58(4):e2019RG000678.
- Shreve, S. E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer.
- Sirignano, J. and Spiliopoulos, K. (2018). Dgm: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364.
- Smith, A. A. (1993). Estimating nonlinear time-series models using simulated vector autoregressions. *Journal of Applied Econometrics*, 8(S1):S63–S84.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS 25)*.
- Sobol’, I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112.
- Sobol, I. M. (2001). Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and Computers in Simulation*, 55(1-3):271–280.
- Song, E., Nelson, B. L., and Staum, J. (2016). Shapley Effects for Global Sensitivity Analysis: Theory and Computation. *SIAM/ASA Journal on Uncertainty Quantification*, 4(1):1060–1083.
- Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S., and Srebro, N. (2018). The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70):1–57.
- Srinivas, N., Krause, A., Kakade, S., and Seeger, M. (2010). Gaussian process optimization in the bandit setting: No regret and experimental design. In *Proceedings of the 27th International Conference on Machine Learning*, pages 1015–1022.

- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958.
- Srivastava, R. K., Greff, K., and Schmidhuber, J. (2015). Training very deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2377–2385. Conference version of “Highway Networks”.
- Stokey, N. L., Lucas, R. E., and Prescott, E. C. (1989). *Recursive Methods in Economic Dynamics*. Harvard University Press, Cambridge, MA.
- Stroud, A. H. (1971). *Approximate Calculation of Multiple Integrals*. Prentice-Hall Series in Automatic Computation. Prentice-Hall, Englewood Cliffs, NJ.
- Sutskever, I., Martens, J., Dahl, G., and Hinton, G. (2013). On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning (ICML)*, pages 1139–1147. PMLR.
- Telgarsky, M. (2016). Benefits of depth in neural networks. *Conference on Learning Theory (COLT)*.
- The Econ-ARK Team (2020). econ-ark/KrusellSmith: REMARK replication of the Krusell–Smith (1998) model. GitHub repository.
- Tieleman, T. and Hinton, G. (2012). Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. Coursera Course: Neural Networks for Machine Learning, Lecture 6e.
- Titsias, M. K. (2009). Variational learning of inducing variables in sparse Gaussian processes. In *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Traeger, C. P. (2014). A 4-States DICE: Quantitatively Addressing Uncertainty Effects in Climate Change. *Environmental and Resource Economics*, 59(1):1–37.
- Traeger, C. P. (2021). ACE – Analytic Climate Economy. SSRN Scholarly Paper ID 3832722, Social Science Research Network, Rochester, NY.
- Traeger, C. P. (2023). ACE — Analytic Climate Economy. *American Economic Journal: Economic Policy*, 15(3):372–406.
- Tripathy, R. K. and Bilonis, I. (2018). Deep UQ: Learning deep neural network surrogate models for high dimensional uncertainty quantification. *Journal of Computational Physics*, 375:565–588.
- Valaitis, V. and Villa, A. T. (2024). A machine learning projection method for macro-finance models. *Quantitative Economics*, 15(1):145–173.
- van der Ploeg, F. and Rezai, A. (2026). Climate change, climate policy, and the macroeconomy. CEPR Discussion Paper No. 21153, CEPR Press, Paris & London.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008.

- Von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. (2023). Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 35151–35174. PMLR.
- Wang, S., Teng, Y., and Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081.
- Wang, S., Yu, X., and Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768.
- Webster, M., Jakobovits, L., and Norton, J. (2008). Learning about climate change and implications for near-term policy. *Climatic Change*, 89(1-2):67–85.
- Weil, P. (1989). The equity premium puzzle and the risk-free rate puzzle. *Journal of Monetary Economics*, 24(3):401–421.
- Weitzman, M. L. (2012). Ghg targets as insurance against catastrophic climate damages. *Journal of Public Economic Theory*, 14(2):221–244.
- Wilson, A. G., Hu, Z., Salakhutdinov, R., and Xing, E. P. (2016). Deep kernel learning. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 370–378.
- Wu, Y. and He, K. (2018). Group normalization. In *European Conference on Computer Vision (ECCV)*, pages 3–19.
- Yang, Y., Wang, C., Schaab, A., and Moll, B. (2025). Structural reinforcement learning for heterogeneous agent macroeconomics.
- Young, E. R. (2010). Solving the incomplete markets model with aggregate uncertainty using the krusell–smith algorithm and non-stochastic simulations. *Journal of Economic Dynamics and Control*, 34(1):36–41.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., and Smola, A. J. (2017). Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zhang, C., Bengio, S., Hardt, M., Recht, B., and Vinyals, O. (2017). Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*.
- İmrohoroglu, A. (1989). Cost of business cycles with indivisibilities and liquidity constraints. *Journal of Political Economy*, 97(6):1364–1383.